

NSI

Première

Édition de travail 2026-2027

01001110 01010011 01001001
01001110 01000101 01011000
01010101 01010011 00100001

Sommaire

1	Représentation des données : types et valeurs de base	1
1.1	Écriture d'un entier dans une base	1
1.2	Représentation binaire des entiers relatifs	2
1.3	Représentation approximative des nombres flottants	3
1.4	Valeurs et opérateurs booléens	4
1.5	Représentation d'un texte en machine	5
1.6	Écriture d'un entier dans une base	6
1.7	Représentation binaire des entiers relatifs	7
1.8	Représentation approximative des nombres flottants	9
1.9	Valeurs et opérateurs booléens	10
1.10	Représentation d'un texte en machine	11
2	Types construits : p-uplets, tableaux, dictionnaires	31
2.1	Les p-uplets (tuples)	33
2.2	Les tableaux (listes Python)	34
2.3	Les grilles : tableaux de tableaux	36
2.4	Les p-uplets (tuples)	37
2.5	Les tableaux (listes Python)	39
2.6	Les grilles : tableaux de tableaux	41
2.7	Les dictionnaires	42
2.8	Mutabilité et références	43
3	Traitement de données en tables	103
3.1	Importer une table	103
3.2	Recherche dans une table	104
3.3	Trier une table	106
3.4	Fusionner deux tables	107
3.5	Importer une table	108
3.6	Recherche dans une table	109
3.7	Trier une table	111
3.8	Fusionner deux tables	112
4	Langages et programmation	135
4.1	Constructions élémentaires	135
4.2	Diversité et unité des langages de programmation	136

4.3	Spécifier une fonction	137
4.4	Mise au point de programmes	138
4.5	Utiliser une bibliothèque	139
4.6	Constructions élémentaires	140
4.7	Diversité et unité des langages de programmation	141
4.8	Constructions élémentaires	142
4.9	Diversité et unité des langages de programmation	143
4.10	Spécifier une fonction	144
4.11	Mise au point de programmes	145
4.12	Utiliser une bibliothèque	147

5 Algorithmique 1 : parcours et tris 179

5.1	Recherche dichotomique	179
5.2	Algorithmes gloutons	180
5.3	L'algorithme des k plus proches voisins	181
5.4	Recherche dichotomique	182
5.5	Algorithmes gloutons	184
5.6	L'algorithme des k plus proches voisins	185
5.7	Parcours séquentiel d'un tableau	186
5.8	Le tri par insertion	187
5.9	Le tri par sélection	189

6 Algorithmique 2 : dichotomie, algorithmes gloutons, k plus proches voisins 207

6.1	Recherche dichotomique	207
6.2	Algorithmes gloutons	208
6.3	L'algorithme des k plus proches voisins	209

7 Interactions entre l'homme et la machine sur le Web 225

7.1	Composants graphiques et événements	225
7.2	Interaction client-serveur	226
7.3	Formulaires, requêtes GET et POST	228
7.4	Composants graphiques et événements	229
7.5	Interaction client-serveur	231
7.6	Formulaires, requêtes GET et POST	232
7.7	Composants graphiques et événements	233
7.8	Interaction client-serveur	235
7.9	Formulaires, requêtes GET et POST	236
7.10	Composants graphiques et événements	238
7.11	Interaction client-serveur	239
7.12	Formulaires, requêtes GET et POST	241

8	Architecture de von Neumann et systèmes d'exploitation	277
8.1	L'architecture de von Neumann	277
8.2	Dérouler l'exécution d'instructions machine	278
8.3	Systèmes d'exploitation	279
8.4	La ligne de commande	280
8.5	Droits et permissions d'accès	280
8.6	L'architecture de von Neumann	281
8.7	Dérouler l'exécution d'instructions machine	282
8.8	L'architecture de von Neumann	283
8.9	Dérouler l'exécution d'instructions machine	285
8.10	Systèmes d'exploitation	286
8.11	La ligne de commande	286
8.12	Droits et permissions d'accès	287
9	Réseaux : transmission de données	309
9.1	Découpage en paquets et encapsulation	309
9.2	Le protocole du bit alterné	310
9.3	Simuler un réseau	311
9.4	Capteurs, actionneurs et IHM programmée	312
9.5	Découpage en paquets et encapsulation	313
9.6	Le protocole du bit alterné	315
9.7	Simuler un réseau	316
9.8	Découpage en paquets et encapsulation	317
9.9	Le protocole du bit alterné	318
9.10	Simuler un réseau	319
9.11	Capteurs, actionneurs et IHM programmée	320
10	Projet et méthodes	341
10.1	Repères d'histoire de l'informatique	341
10.2	La démarche de projet	342
10.3	Déboguer méthodiquement	343
10.4	Documenter son code et préparer une présentation orale	345
10.5	Repères d'histoire de l'informatique	346
10.6	La démarche de projet	347
10.7	Déboguer méthodiquement	348
10.8	Documenter son code et préparer une présentation orale	350

1 Représentation des données : types et valeurs de base

1.1 Écriture d'un entier dans une base

DÉFINITION D1

La **base** b (avec $b \geq 2$) d'un système de numération est le nombre de chiffres distincts utilisés pour écrire un nombre. En base b , un nombre s'écrit $\overline{d_k d_{k-1} \dots d_1 d_0}_b$ et sa valeur en base 10 est $\sum_{i=0}^k d_i \times b^i$.

EXEMPLE En base 2 (*binaire*, chiffres 0 et 1), $101101_2 = 1 \times 32 + 0 \times 16 + 1 \times 8 + 1 \times 4 + 0 \times 2 + 1 \times 1 = 45$. En base 16 (*hexadécimal*, chiffres 0-9 puis A-F), $2D_{16} = 2 \times 16 + 13 = 45$.

◆ **PROPRIÉTÉ Conversion base b vers base 10** Pour convertir un nombre écrit en base b vers la base 10, on calcule la somme des chiffres multipliés par les puissances de b correspondantes (méthode dite « d'Horner » pour un calcul efficace : on part du chiffre de poids fort et on accumule $r \leftarrow r \times b + d_i$).

CODE DE RÉFÉRENCE — Conversion base b vers décimal

```
1 def vers_decimal(chiffres, base):
2     """Convertit une liste de chiffres (poids fort en premier) en un entier
3     decimal.
4
5     Preconditions : base ≥ 2, chaque chiffre de 'chiffres' est dans [0, base[.
6     """
7     valeur = 0
8     for d in chiffres:
9         valeur = valeur * base + d
10    return valeur
11 print(vers_decimal([1, 0, 1, 1, 0, 1], 2)) # 45
12 print(vers_decimal([2, 13], 16))         # 45
```

◆ **PROPRIÉTÉ Conversion base 10 vers base b** Pour convertir un entier positif de la base 10 vers la base b , on effectue des divisions euclidiennes successives par b : les restes obtenus, lus de bas en haut (dernier reste calculé en premier), forment l'écriture en base b .

EXEMPLE Pour convertir 45 en base 2 : $45 = 2 \times 22 + 1$, $22 = 2 \times 11 + 0$, $11 = 2 \times 5 + 1$, $5 = 2 \times 2 + 1$, $2 = 2 \times 1 + 0$, $1 = 2 \times 0 + 1$. En lisant les restes de bas en haut : 101101_2 .

ERREUR FRÉQUENTE

Confondre la **valeur** d'un nombre et son **écriture**. Le nombre quarante-cinq est le même objet mathématique, qu'on l'écrive 101101 en base 2, 2D en base 16 ou 45 en base 10 : seule son écriture change selon la base.

» **Python À la machine.** Écris une fonction `vers_base(n, base)` qui renvoie la liste des chiffres de l'entier positif n écrit en base `base` (poids fort en premier), puis vérifie que `vers_base(255, 16)` donne `[15, 15]`. Compare avec les fonctions natives `bin`, `oct` et `hex` de Python.

1.2 Représentation binaire des entiers relatifs

DÉFINITION D2

Un entier écrit sur n **bits** peut représenter 2^n valeurs distinctes. Pour un entier **positif**, ces valeurs vont de 0 à $2^n - 1$: sur 8 bits, de 0 à 255.

♦ **PROPRIÉTÉ Nombre de bits nécessaires** Le nombre de bits nécessaires pour écrire un entier positif n (avec $n \geq 1$) est $\lfloor \log_2(n) \rfloor + 1$. En pratique, on utilise des tailles standard : 8, 16, 32 ou 64 bits (Python, lui, ajuste automatiquement la taille de ses entiers, sans limite fixe).

EXEMPLE 200 s'écrit sur 8 bits ($200 < 256 = 2^8$), mais $200 + 150 = 350$ nécessite 9 bits ($350 \geq 256$ mais $350 < 512 = 2^9$) : une addition peut faire « déborder » le nombre de bits disponibles.

DÉFINITION D3

Le **complément à 2** est la méthode standard pour représenter les entiers **relatifs** (positifs et négatifs) en binaire, sur un nombre fixe n de bits. Pour représenter $-x$ (avec $x > 0$) : on écrit x en binaire sur n bits, on inverse chaque bit (0 devient 1, 1 devient 0), puis on ajoute 1 au résultat.

EXEMPLE Sur 8 bits, pour représenter -5 : 5 s'écrit 00000101 ; en inversant chaque bit on obtient 11111010 ; en ajoutant 1 on obtient 11111011, qui est la représentation de -5 en complément à 2 sur 8 bits.

CODE DE RÉFÉRENCE — Complément à 2 sur n bits

```
1 def complement_a_2(x, n):
2     """Renvoie l'écriture sur n bits de l'entier relatif x (complement a 2).
3
4     Preconditions : -2**(n-1) ≤ x < 2**(n-1).
5     """
6     if x < 0:
7         x = (1 << n) + x # equivaut a l'inversion des bits puis +1
8     return format(x, f'0{n}b')
9
10 print(complement_a_2(5, 8))      # 00000101
11 print(complement_a_2(-5, 8))     # 11111011
12 print(complement_a_2(-1, 8))     # 11111111
```

◆ **PROPRIÉTÉ Intérêt du complément à 2** Le complément à 2 permet d'additionner deux entiers relatifs **avec le même circuit binaire** que pour des entiers positifs, sans traitement particulier du signe : le bit de poids fort indique le signe (0 = positif, 1 = négatif), et l'addition « déborde » naturellement de façon cohérente.

⚠ ERREUR FRÉQUENTE

Croire qu'un nombre négatif se code en changeant simplement le bit de poids fort (« signe + valeur absolue »). Ce n'est **pas** le complément à 2 : cette méthode naïve poserait des problèmes pour l'addition (deux écritures de 0) et n'est pas celle utilisée par les machines réelles.

» **Python À la machine.** Vérifie avec la fonction `complement_a_2(x, 8)` ci-dessus l'écriture de -1 , -128 et 127 sur 8 bits. Que se passe-t-il si tu essaies d'appeler la fonction avec $x = -129$ ou $x = 128$? Pourquoi (relie ta réponse à la précondition indiquée dans la docstring) ?

1.3 Représentation approximative des nombres flottants

DÉFINITION D4

Un **nombre flottant** est une valeur numérique approchée d'un nombre réel, stockée sur un nombre fixe de bits. Beaucoup de nombres qui s'écrivent simplement en base 10 (comme 0,1) n'ont **pas** d'écriture binaire finie, exactement comme $1/3 = 0,333 \dots$ n'a pas d'écriture décimale finie.

EXEMPLE En Python, 0.1 n'est pas stocké exactement : c'est une valeur très proche mais légèrement différente. La conséquence la plus visible :

```
>>> 0.1 + 0.2
0.30000000000000004
>>> 0.1 + 0.2 == 0.3
False
```

◆ **PROPRIÉTÉ Ne jamais tester l'égalité de deux flottants** Comparer deux flottants avec `==` est une erreur classique : à cause des erreurs d'arrondi, deux calculs mathématiquement égaux peuvent donner des valeurs flottantes légèrement différentes. Il faut comparer leur **écart** à une petite tolérance près.

CODE DE RÉFÉRENCE — Comparaison de flottants avec tolérance

```
1 def presque_egaux(a, b, tolerance=1e-9):
2     """Teste si deux flottants sont égaux à une tolerance pres."""
3     return abs(a - b) < tolerance
4
5 print(0.1 + 0.2 == 0.3)           # False (piege classique)
6 print(presque_egaux(0.1 + 0.2, 0.3)) # True
```

ERREUR FRÉQUENTE

Écrire `if resultat == valeur_attendue` pour tester un calcul flottant (EF). Cette comparaison peut échouer même quand le calcul est correct, à cause des erreurs d'arrondi : il faut toujours comparer avec une tolérance, via une fonction comme `presque_egaux` ci-dessus.

POUR ALLER PLUS LOIN

La norme utilisée par la quasi-totalité des langages (dont Python) est IEEE-754, qui code un flottant en trois parties : un bit de signe, un exposant, et une mantisse (partie fractionnaire). Aucune connaissance précise de cette norme n'est exigible au programme, mais elle explique pourquoi $1/3$, $0,1$ ou $0,2$ ne sont pas représentables exactement en base 2 (seules les fractions dont le dénominateur est une puissance de 2, comme $0,25 = 1/4$, le sont).

EXEMPLE $0,25 = 1/4 = 2^{-2}$ est représentable **exactement** en binaire, contrairement à $0,1$:

» **Python À la machine.** Dans un éditeur Python, calcule $0.1 + 0.1 + 0.1$ et compare-le à 0.3 avec `==`, puis avec la fonction `presque_egaux` ci-dessus. Cherche un autre triplet de flottants dont la somme affiche un résultat « bizarre » comme `0.30000000000000004`.

1.4 Valeurs et opérateurs booléens

DÉFINITION D5

Une **valeur booléenne** ne prend que deux valeurs : `True` (vrai, souvent codé 1) ou `False` (faux, souvent codé 0). Les trois opérateurs booléens de base sont `and` (et), `or` (ou) et `not` (non).

PROPRIÉTÉ Table de vérité de `and` et `or`

<i>a</i>	<i>b</i>	<i>a and b</i>	<i>a or b</i>
Vrai	Vrai	Vrai	Vrai
Vrai	Faux	Faux	Vrai
Faux	Vrai	Faux	Vrai
Faux	Faux	Faux	Faux

`and` est vrai seulement si **les deux** opérandes sont vrais ; `or` est vrai dès qu'**au moins un** des deux opérandes est vrai.

DÉFINITION D6

Le **ou exclusif** (xor, noté $\oplus$) est vrai lorsque **exactement un** des deux opérandes est vrai (mais pas les deux). En Python, on l'obtient avec `a != b` pour des booléens, ou l'opérateur `^` bit à bit sur des entiers.

EXEMPLE Le xor sert par exemple à additionner deux bits sans retenue : $0 \oplus 0 = 0$, $0 \oplus 1 = 1$, $1 \oplus 0 = 1$, $1 \oplus 1 = 0$ (la retenue, elle, correspond à `a and b`).

CODE DE RÉFÉRENCE — Addition d'un bit (demi-additionneur)

```

1 def demi_additionneur(a, b):
2     """Additionne deux bits (0 ou 1) : renvoie (somme, retenue)."""
3     somme = a ^ b
4     retenue = a & b
5     return somme, retenue
6
7 print(demi_additionneur(1, 1)) # (0, 1) : 1+1 = 10 en binaire
8 print(demi_additionneur(1, 0)) # (1, 0)

```

◆ **PROPRIÉTÉ Table de vérité d'une expression composée** Pour dresser la table de vérité d'une expression booléenne à n variables, on énumère les 2^n combinaisons possibles de valeurs des variables, et on calcule la valeur de l'expression pour chacune.

EXEMPLE Table de vérité de $(a \text{ and } b) \text{ or } (\text{not } a \text{ and } c)$:

⚠ ERREUR FRÉQUENTE

Croire que `and` et `or` évaluent toujours leurs deux opérandes. En Python, l'évaluation est **séquentielle** et **court-circuitée** : dans `a and b`, si `a` vaut `False`, `b` n'est même pas évalué (le résultat est déjà déterminé). Cela peut avoir des effets de bord si `b` contient un appel de fonction.

» **Python À la machine.** Écris une fonction `table_verite(expr, n)` qui... en fait, dresse « à la main » (avec des boucles `for`) la table de vérité de l'expression $(a \text{ or } b) \text{ and } (\text{not } c)$ pour les 8 combinaisons de `a`, `b`, `c`. Vérifie ton résultat avec Python.

1.5 Représentation d'un texte en machine

DÉFINITION D7

Un **encodage de caractères** est une correspondance entre des caractères (lettres, chiffres, symboles) et des suites de bits. Coder un texte, c'est appliquer cette correspondance à chacun de ses caractères.

◆ **PROPRIÉTÉ Trois encodages historiques**

- ▶ **ASCII** (1963) : code chaque caractère sur 7 bits (128 caractères) : lettres non accentuées, chiffres, ponctuation de base. Ne code pas les caractères accentués (é, à, ç...).
- ▶ **ISO-8859-1** (« Latin-1 ») : étend ASCII à 8 bits (256 caractères), en ajoutant les caractères accentués usuels des langues d'Europe de l'Ouest.
- ▶ **Unicode** (via l'encodage **UTF-8**) : vise à coder **tous** les caractères de toutes les langues (et les emojis). Chaque caractère occupe de 1 à 4 octets selon sa position dans la table Unicode.

EXEMPLE Le caractère 'A' a pour code 65 dans ASCII, ISO-8859-1 et UTF-8 (les 128 premiers caractères Unicode coïncident avec ASCII). Le caractère 'é' n'existe pas en ASCII, vaut 233 en ISO-8859-1 (un seul octet), et occupe **deux** octets en UTF-8.

⚠ CONTRE-EXEMPLE Tenter d'encoder un caractère accentué en ASCII provoque une erreur, car ce caractère n'existe pas dans cet encodage :

```
>>> 'café'.encode('ascii')
UnicodeEncodeError: 'ascii' codec can't encode character '\xe9' in position 3:
ordinal not in range(128)
```

CODE DE RÉFÉRENCE — Convertir un texte d'un encodage à un autre

```
1 def reencoder(texte, encodage_source, encodage_cible):
2     """Reencode un texte : decode avec la source puis encode avec la cible.
3
4     Preconditions : 'texte' est déjà une chaîne Python (str), pas des octets.
5     """
6     octets = texte.encode(encodage_source)
7     return octets.decode(encodage_source).encode(encodage_cible)
8
9 octets_latin1 = reencoder('café', 'utf-8', 'iso-8859-1')
10 print(octets_latin1)          # b'caf\xe9'
11 print(len(octets_latin1))     # 4 octets (contre 5 en UTF-8)
```

◆ **PROPRIÉTÉ Intérêt d'un encodage universel** Unicode/UTF-8 s'est imposé car il permet d'échanger des textes entre systèmes utilisant des langues différentes (et donc des jeux de caractères différents) sans ambiguïté ni perte d'information, contrairement à ASCII (trop limité) ou aux nombreux encodages régionaux incompatibles entre eux (ISO-8859-1 pour l'Europe de l'Ouest, d'autres pour le cyrillique, l'arabe, etc.).

⚠ ERREUR FRÉQUENTE

Confondre le **nombre de caractères** d'une chaîne et son **nombre d'octets** une fois encodée. `len('café')` vaut 4 (quatre caractères), mais `len('café'.encode('utf-8'))` vaut 5 (le « é » occupe deux octets en UTF-8).

» **Python À la machine.** Dans un éditeur Python, encode la chaîne "NSI 2026 – à bientôt !" en UTF-8 puis en ISO-8859-1, et compare le nombre d'octets obtenus avec `len()`. Essaie ensuite de l'encoder en ASCII : observe et explique l'erreur.

1.6 Écriture d'un entier dans une base

DÉFINITION D1

La **base** b (avec $b \geq 2$) d'un système de numération est le nombre de chiffres distincts utilisés pour écrire un nombre. En base b , un nombre s'écrit $\overline{d_k d_{k-1} \dots d_1 d_0}_b$ et sa valeur en base 10 est

$$\sum_{i=0}^k d_i \times b^i.$$

EXEMPLE En base 2 (*binaire*, chiffres 0 et 1), $101101_2 = 1 \times 32 + 0 \times 16 + 1 \times 8 + 1 \times 4 + 0 \times 2 + 1 \times 1 = 45$. En base 16 (*hexadécimal*, chiffres 0-9 puis A-F), $2D_{16} = 2 \times 16 + 13 = 45$.

◆ **PROPRIÉTÉ Conversion base b vers base 10** Pour convertir un nombre écrit en base b vers la base 10, on calcule la somme des chiffres multipliés par les puissances de b correspondantes (méthode dite « d'Horner » pour un calcul efficace : on part du chiffre de poids fort et on accumule $r \leftarrow r \times b + d_i$).

CODE DE RÉFÉRENCE — Conversion base b vers décimal

```
1 def vers_decimal(chiffres, base):
2     """Convertit une liste de chiffres (poids fort en premier) en un entier
3     decimal.
4
5     Preconditions : base ≥ 2, chaque chiffre de 'chiffres' est dans [0, base[.
6     """
7     valeur = 0
8     for d in chiffres:
9         valeur = valeur * base + d
10    return valeur
11
12 print(vers_decimal([1, 0, 1, 1, 0, 1], 2)) # 45
13 print(vers_decimal([2, 13], 16))         # 45
```

◆ **PROPRIÉTÉ Conversion base 10 vers base b** Pour convertir un entier positif de la base 10 vers la base b , on effectue des divisions euclidiennes successives par b : les restes obtenus, lus de bas en haut (dernier reste calculé en premier), forment l'écriture en base b .

EXEMPLE Pour convertir 45 en base 2 : $45 = 2 \times 22 + 1$, $22 = 2 \times 11 + 0$, $11 = 2 \times 5 + 1$, $5 = 2 \times 2 + 1$, $2 = 2 \times 1 + 0$, $1 = 2 \times 0 + 1$. En lisant les restes de bas en haut : 101101_2 .

⚠ ERREUR FRÉQUENTE

Confondre la **valeur** d'un nombre et son **écriture**. Le nombre quarante-cinq est le même objet mathématique, qu'on l'écrive 101101 en base 2, 2D en base 16 ou 45 en base 10 : seule son écriture change selon la base.

» **Python À la machine.** Écris une fonction `vers_base(n, base)` qui renvoie la liste des chiffres de l'entier positif n écrit en base $base$ (poids fort en premier), puis vérifie que `vers_base(255, 16)` donne `[15, 15]`. Compare avec les fonctions natives `bin`, `oct` et `hex` de Python.

1.7 Représentation binaire des entiers relatifs

DÉFINITION D2

Un entier écrit sur n **bits** peut représenter 2^n valeurs distinctes. Pour un entier **positif**, ces valeurs vont de 0 à $2^n - 1$: sur 8 bits, de 0 à 255.

◆ **PROPRIÉTÉ Nombre de bits nécessaires** Le nombre de bits nécessaires pour écrire un entier positif n (avec $n \geq 1$) est $\lfloor \log_2(n) \rfloor + 1$. En pratique, on utilise des tailles standard : 8, 16, 32 ou 64 bits (Python, lui, ajuste automatiquement la taille de ses entiers, sans limite fixe).

EXEMPLE 200 s'écrit sur 8 bits ($200 < 256 = 2^8$), mais $200 + 150 = 350$ nécessite 9 bits ($350 \geq 256$ mais $350 < 512 = 2^9$) : une addition peut faire « déborder » le nombre de bits disponibles.

DÉFINITION D3

Le **complément à 2** est la méthode standard pour représenter les entiers **relatifs** (positifs et négatifs) en binaire, sur un nombre fixe n de bits. Pour représenter $-x$ (avec $x > 0$) : on écrit x en binaire sur n bits, on inverse chaque bit (0 devient 1, 1 devient 0), puis on ajoute 1 au résultat.

EXEMPLE Sur 8 bits, pour représenter -5 : 5 s'écrit 00000101 ; en inversant chaque bit on obtient 11111010 ; en ajoutant 1 on obtient 11111011, qui est la représentation de -5 en complément à 2 sur 8 bits.

CODE DE RÉFÉRENCE — Complément à 2 sur n bits

```
1 def complement_a_2(x, n):
2     """Renvoie l'écriture sur n bits de l'entier relatif x (complément à 2).
3
4     Preconditions : -2**(n-1) ≤ x < 2**(n-1).
5     """
6     if x < 0:
7         x = (1 << n) + x # equivaut a l'inversion des bits puis +1
8     return format(x, f'0{n}b')
9
10 print(complement_a_2(5, 8))      # 00000101
11 print(complement_a_2(-5, 8))     # 11111011
12 print(complement_a_2(-1, 8))     # 11111111
```

◆ **PROPRIÉTÉ Intérêt du complément à 2** Le complément à 2 permet d'additionner deux entiers relatifs **avec le même circuit binaire** que pour des entiers positifs, sans traitement particulier du signe : le bit de poids fort indique le signe (0 = positif, 1 = négatif), et l'addition « déborde » naturellement de façon cohérente.

⚠ ERREUR FRÉQUENTE

Croire qu'un nombre négatif se code en changeant simplement le bit de poids fort (« signe + valeur absolue »). Ce n'est **pas** le complément à 2 : cette méthode naïve poserait des problèmes pour l'addition (deux écritures de 0) et n'est pas celle utilisée par les machines réelles.

» **Python À la machine.** Vérifie avec la fonction `complement_a_2(x, 8)` ci-dessus l'écriture de -1 , -128 et 127 sur 8 bits. Que se passe-t-il si tu essaies d'appeler la fonction avec $x = -129$ ou $x = 128$? Pourquoi (relie ta réponse à la précondition indiquée dans la docstring) ?

1.8 Représentation approximative des nombres flottants

DÉFINITION D4

Un **nombre flottant** est une valeur numérique approchée d'un nombre réel, stockée sur un nombre fixe de bits. Beaucoup de nombres qui s'écrivent simplement en base 10 (comme 0,1) n'ont **pas** d'écriture binaire finie, exactement comme $1/3 = 0,333\dots$ n'a pas d'écriture décimale finie.

EXEMPLE En Python, 0.1 n'est pas stocké exactement : c'est une valeur très proche mais légèrement différente. La conséquence la plus visible :

```
>>> 0.1 + 0.2
0.30000000000000004
>>> 0.1 + 0.2 == 0.3
False
```

◆ **PROPRIÉTÉ Ne jamais tester l'égalité de deux flottants** Comparer deux flottants avec `==` est une erreur classique : à cause des erreurs d'arrondi, deux calculs mathématiquement égaux peuvent donner des valeurs flottantes légèrement différentes. Il faut comparer leur **écart** à une petite tolérance près.

CODE DE RÉFÉRENCE — Comparaison de flottants avec tolérance

```
1 def presque_egaux(a, b, tolerance=1e-9):
2     """Teste si deux flottants sont égaux à une tolerance pres."""
3     return abs(a - b) < tolerance
4
5 print(0.1 + 0.2 == 0.3)           # False (piege classique)
6 print(presque_egaux(0.1 + 0.2, 0.3)) # True
```

⚠ ERREUR FRÉQUENTE

Écrire `if resultat == valeur_attendue:` pour tester un calcul flottant (EF). Cette comparaison peut échouer même quand le calcul est correct, à cause des erreurs d'arrondi : il faut toujours comparer avec une tolérance, via une fonction comme `presque_egaux` ci-dessus.

+ POUR ALLER PLUS LOIN

La norme utilisée par la quasi-totalité des langages (dont Python) est IEEE-754, qui code un flottant en trois parties : un bit de signe, un exposant, et une mantisse (partie fractionnaire). Aucune connaissance précise de cette norme n'est exigible au programme, mais elle explique pourquoi $1/3$, 0,1 ou 0,2 ne sont pas représentables exactement en base 2 (seules les fractions dont le dénominateur est une puissance de 2, comme $0,25 = 1/4$, le sont).

EXEMPLE $0,25 = 1/4 = 2^{-2}$ est représentable **exactement** en binaire, contrairement à 0,1 :

» **Python À la machine.** Dans un éditeur Python, calcule $0.1 + 0.1 + 0.1$ et compare-le à 0.3 avec `==`, puis avec la fonction `presque_egaux` ci-dessus. Cherche un autre triplet de flottants dont la somme affiche un résultat « bizarre » comme 0.30000000000000004.

1.9 Valeurs et opérateurs booléens

DÉFINITION D5

Une **valeur booléenne** ne prend que deux valeurs : `True` (vrai, souvent codé 1) ou `False` (faux, souvent codé 0). Les trois opérateurs booléens de base sont `and` (et), `or` (ou) et `not` (non).

◆ PROPRIÉTÉ Table de vérité de `and` et `or`

a	b	$a \text{ and } b$	$a \text{ or } b$
Vrai	Vrai	Vrai	Vrai
Vrai	Faux	Faux	Vrai
Faux	Vrai	Faux	Vrai
Faux	Faux	Faux	Faux

`and` est vrai seulement si **les deux** opérandes sont vrais ; `or` est vrai dès qu'**au moins un** des deux opérandes est vrai.

DÉFINITION D6

Le **ou exclusif** (`xor`, noté $\oplus$) est vrai lorsque **exactement un** des deux opérandes est vrai (mais pas les deux). En Python, on l'obtient avec `a != b` pour des booléens, ou l'opérateur `^` bit à bit sur des entiers.

EXEMPLE Le `xor` sert par exemple à additionner deux bits sans retenue : $0 \oplus 0 = 0$, $0 \oplus 1 = 1$, $1 \oplus 0 = 1$, $1 \oplus 1 = 0$ (la retenue, elle, correspond à `a and b`).

CODE DE RÉFÉRENCE — Addition d'un bit (demi-additionneur)

```
1 def demi_additionneur(a, b):
2     """Additionne deux bits (0 ou 1) : renvoie (somme, retenue)."""
3     somme = a ^ b
4     retenue = a & b
5     return somme, retenue
6
7 print(demi_additionneur(1, 1)) # (0, 1) : 1+1 = 10 en binaire
8 print(demi_additionneur(1, 0)) # (1, 0)
```

◆ **PROPRIÉTÉ Table de vérité d'une expression composée** Pour dresser la table de vérité d'une expression booléenne à n variables, on énumère les 2^n combinaisons possibles de valeurs des variables, et on calcule la valeur de l'expression pour chacune.

EXEMPLE Table de vérité de $(a \text{ and } b) \text{ or } (\text{not } a \text{ and } c)$:

ERREUR FRÉQUENTE

Croire que `and` et `or` évaluent toujours leurs deux opérandes. En Python, l'évaluation est **séquentielle** et **court-circuitée** : dans `a and b`, si `a` vaut `False`, `b` n'est même pas évalué (le résultat est déjà déterminé). Cela peut avoir des effets de bord si `b` contient un appel de fonction.

» **Python À la machine.** Écris une fonction `table_verite(expr, n)` qui... en fait, dresse « à la main » (avec des boucles `for`) la table de vérité de l'expression `(a or b) and (not c)` pour les 8 combinaisons de `a`, `b`, `c`. Vérifie ton résultat avec Python.

1.10 Représentation d'un texte en machine

DÉFINITION D7

Un **encodage de caractères** est une correspondance entre des caractères (lettres, chiffres, symboles) et des suites de bits. Coder un texte, c'est appliquer cette correspondance à chacun de ses caractères.

◆ **PROPRIÉTÉ** Trois encodages historiques

- ▶ **ASCII** (1963) : code chaque caractère sur 7 bits (128 caractères) : lettres non accentuées, chiffres, ponctuation de base. Ne code pas les caractères accentués (é, à, ç...).
- ▶ **ISO-8859-1** (« Latin-1 ») : étend ASCII à 8 bits (256 caractères), en ajoutant les caractères accentués usuels des langues d'Europe de l'Ouest.
- ▶ **Unicode** (via l'encodage **UTF-8**) : vise à coder **tous** les caractères de toutes les langues (et les emojis). Chaque caractère occupe de 1 à 4 octets selon sa position dans la table Unicode.

EXEMPLE Le caractère 'A' a pour code 65 dans ASCII, ISO-8859-1 et UTF-8 (les 128 premiers caractères Unicode coïncident avec ASCII). Le caractère 'é' n'existe pas en ASCII, vaut 233 en ISO-8859-1 (un seul octet), et occupe **deux** octets en UTF-8.

⚠ **CONTRE-EXEMPLE** Tenter d'encoder un caractère accentué en ASCII provoque une erreur, car ce caractère n'existe pas dans cet encodage :

```
»» 'café'.encode('ascii')
UnicodeEncodeError: 'ascii' codec can't encode character '\xe9' in position 3:
ordinal not in range(128)
```

CODE DE RÉFÉRENCE — Convertir un texte d'un encodage à un autre

```
1 def reencoder(texte, encodage_source, encodage_cible):
2     """Reencode un texte : decode avec la source puis encode avec la cible.
3
4     Preconditions : 'texte' est déjà une chaîne Python (str), pas des octets.
5     """
6     octets = texte.encode(encodage_source)
7     return octets.decode(encodage_source).encode(encodage_cible)
8
9 octets_latin1 = reencoder('café', 'utf-8', 'iso-8859-1')
```

```

10 print(octets_latin1)      # b'caf\xe9'
11 print(len(octets_latin1)) # 4 octets (contre 5 en UTF-8)

```

◆ **PROPRIÉTÉ Intérêt d'un encodage universel** Unicode/UTF-8 s'est imposé car il permet d'échanger des textes entre systèmes utilisant des langues différentes (et donc des jeux de caractères différents) sans ambiguïté ni perte d'information, contrairement à ASCII (trop limité) ou aux nombreux encodages régionaux incompatibles entre eux (ISO-8859-1 pour l'Europe de l'Ouest, d'autres pour le cyrillique, l'arabe, etc.).

⚠ ERREUR FRÉQUENTE

Confondre le **nombre de caractères** d'une chaîne et son **nombre d'octets** une fois encodée. `len('café')` vaut 4 (quatre caractères), mais `len('café'.encode('utf-8'))` vaut 5 (le « é » occupe deux octets en UTF-8).

» **Python À la machine.** Dans un éditeur Python, encode la chaîne "NSI 2026 – à bientôt !" en UTF-8 puis en ISO-8859-1, et compare le nombre d'octets obtenus avec `len()`. Essaie ensuite de l'encoder en ASCII : observe et explique l'erreur.

Méthodes

► Méthode directe de travail sur le sujet.

Exercice 1 ◆

1. Convertir 156 (écriture décimale) en base 2 puis en base 16, en détaillant les divisions euclidiennes successives.
2. Convertir 11001_2 en base 10.
3. Convertir $3F_{16}$ en base 10.

Exercice 2 ◆

1. Combien de bits sont nécessaires pour écrire 300 en binaire ?
2. Écrire 20 en complément à 2 sur 8 bits.
3. Écrire -12 en complément à 2 sur 8 bits (détailler : écriture de 12, inversion des bits, ajout de 1).

Exercice 3 ◆◆

Un élève écrit le programme suivant pour vérifier que trois pièces de 0,10 € font bien 0,30 € :

```

1 prix = [0.10] * 3
2 if sum(prix) == 0.30:
3     print("Ca fait bien 0,30 euro")
4 else:
5     print("Erreur de calcul !")

```

1. Que va afficher ce programme ? Pourquoi (relier à la représentation des flottants) ?

- Écrire une fonction `presque_egaux(a, b, tolerance=1e-9)` qui teste si deux flottants sont égaux à une tolérance près.
- Réécrire le test `if sum(prix) == 0.30` : en utilisant cette fonction, pour que le programme affiche "Ca fait bien 0,30 euro".

Exercice 4 ♦♦

Un système d'alarme se déclenche selon l'expression booléenne $(a \text{ or } b) \text{ and } (\text{not } c)$, où a = « capteur porte activé », b = « capteur fenêtre activé », c = « mode maintenance activé ».

- Dresser la table de vérité complète de cette expression ($2^3 = 8$ lignes).
- Pour combien de combinaisons de (a, b, c) l'alarme se déclenche-t-elle ?
- En mode maintenance ($c = \text{Vrai}$), l'alarme peut-elle se déclencher, quels que soient a et b ? Justifier.

Exercice 5 ♦♦♦

On considère la chaîne "Bonjour à tous !".

- Combien cette chaîne compte-t-elle de **caractères** ?
- Combien d'**octets** occupe-t-elle une fois encodée en UTF-8 ? En ISO-8859-1 ? Pourquoi ces deux nombres diffèrent-ils entre eux, et diffèrent-ils du nombre de caractères ?
- Que se passe-t-il si on tente de l'encoder en ASCII ? Justifier précisément (quel caractère pose problème et pourquoi).
- Un serveur web français reçoit des messages d'utilisateurs du monde entier (arabe, chinois, russe...). Pourquoi ISO-8859-1 serait-il un mauvais choix d'encodage pour stocker ces messages, contrairement à UTF-8 ?

Exercice 6 ♦

- Convertir 156 (écriture décimale) en base 2 puis en base 16, en détaillant les divisions euclidiennes successives.
- Convertir 11001_2 en base 10.
- Convertir $3F_{16}$ en base 10.

Exercice 7 ♦

- Combien de bits sont nécessaires pour écrire 300 en binaire ?
- Écrire 20 en complément à 2 sur 8 bits.
- Écrire -12 en complément à 2 sur 8 bits (détailler : écriture de 12, inversion des bits, ajout de 1).

Exercice 8 ♦♦

Un élève écrit le programme suivant pour vérifier que trois pièces de 0,10 € font bien 0,30 € :

```
1 prix = [0.10] * 3
2 if sum(prix) == 0.30:
3     print("Ca fait bien 0,30 euro")
4 else:
5     print("Erreur de calcul !")
```

- Que va afficher ce programme ? Pourquoi (relier à la représentation des flottants) ?
- Écrire une fonction `presque_egaux(a, b, tolerance=1e-9)` qui teste si deux flottants sont égaux à une tolérance près.

3. Réécrire le test `if sum(prix) == 0.30` : en utilisant cette fonction, pour que le programme affiche "Ca fait bien 0,30 euro".

Exercice 9

Un système d'alarme se déclenche selon l'expression booléenne $(a \text{ or } b) \text{ and } (\text{not } c)$, où a = « capteur porte activé », b = « capteur fenêtre activé », c = « mode maintenance activé ».

1. Dresser la table de vérité complète de cette expression ($2^3 = 8$ lignes).
2. Pour combien de combinaisons de (a, b, c) l'alarme se déclenche-t-elle ?
3. En mode maintenance ($c = \text{Vrai}$), l'alarme peut-elle se déclencher, quels que soient a et b ? Justifier.

Exercice 10

On considère la chaîne "Bonjour à tous !".

1. Combien cette chaîne compte-t-elle de **caractères** ?
2. Combien d'**octets** occupe-t-elle une fois encodée en UTF-8 ? En ISO-8859-1 ? Pourquoi ces deux nombres diffèrent-ils entre eux, et diffèrent-ils du nombre de caractères ?
3. Que se passe-t-il si on tente de l'encoder en ASCII ? Justifier précisément (quel caractère pose problème et pourquoi).
4. Un serveur web français reçoit des messages d'utilisateurs du monde entier (arabe, chinois, russe...). Pourquoi ISO-8859-1 serait-il un mauvais choix d'encodage pour stocker ces messages, contrairement à UTF-8 ?

Exercice 11

1. Convertir 156 (écriture décimale) en base 2 puis en base 16, en détaillant les divisions euclidiennes successives.
2. Convertir 11001_2 en base 10.
3. Convertir $3F_{16}$ en base 10.

Exercice 12

1. Combien de bits sont nécessaires pour écrire 300 en binaire ?
2. Écrire 20 en complément à 2 sur 8 bits.
3. Écrire -12 en complément à 2 sur 8 bits (détailler : écriture de 12, inversion des bits, ajout de 1).

Exercice 13

Un élève écrit le programme suivant pour vérifier que trois pièces de 0,10€ font bien 0,30€ :

```
1 prix = [0.10] * 3
2 if sum(prix) == 0.30:
3     print("Ca fait bien 0,30 euro")
4 else:
5     print("Erreur de calcul !")
```

1. Que va afficher ce programme ? Pourquoi (relier à la représentation des flottants) ?
2. Écrire une fonction `presque_egaux(a, b, tolerance=1e-9)` qui teste si deux flottants sont égaux à une tolérance près.
3. Réécrire le test `if sum(prix) == 0.30` : en utilisant cette fonction, pour que le programme affiche "Ca fait bien 0,30 euro".

Exercice 14 ♦♦

Un système d'alarme se déclenche selon l'expression booléenne $(a \text{ or } b) \text{ and } (\text{not } c)$, où a = « capteur porte activé », b = « capteur fenêtre activé », c = « mode maintenance activé ».

1. Dresser la table de vérité complète de cette expression ($2^3 = 8$ lignes).
2. Pour combien de combinaisons de (a, b, c) l'alarme se déclenche-t-elle ?
3. En mode maintenance ($c = \text{Vrai}$), l'alarme peut-elle se déclencher, quels que soient a et b ? Justifier.

Exercice 15 ♦♦♦

On considère la chaîne "Bonjour à tous !".

1. Combien cette chaîne compte-t-elle de **caractères** ?
2. Combien d'**octets** occupe-t-elle une fois encodée en UTF-8 ? En ISO-8859-1 ? Pourquoi ces deux nombres diffèrent-ils entre eux, et diffèrent-ils du nombre de caractères ?
3. Que se passe-t-il si on tente de l'encoder en ASCII ? Justifier précisément (quel caractère pose problème et pourquoi).
4. Un serveur web français reçoit des messages d'utilisateurs du monde entier (arabe, chinois, russe...). Pourquoi ISO-8859-1 serait-il un mauvais choix d'encodage pour stocker ces messages, contrairement à UTF-8 ?

Exercice 16 ♦

1. Convertir 156 (écriture décimale) en base 2 puis en base 16, en détaillant les divisions euclidiennes successives.
2. Convertir 11001_2 en base 10.
3. Convertir $3F_{16}$ en base 10.

Exercice 17 ♦

1. Combien de bits sont nécessaires pour écrire 300 en binaire ?
2. Écrire 20 en complément à 2 sur 8 bits.
3. Écrire -12 en complément à 2 sur 8 bits (détailler : écriture de 12, inversion des bits, ajout de 1).

Exercice 18 ♦♦

Un élève écrit le programme suivant pour vérifier que trois pièces de 0,10 € font bien 0,30 € :

```
1 prix = [0.10] * 3
2 if sum(prix) == 0.30:
3     print("Ca fait bien 0,30 euro")
4 else:
5     print("Erreur de calcul !")
```

1. Que va afficher ce programme ? Pourquoi (relier à la représentation des flottants) ?
2. Écrire une fonction presque_egaux($a, b, \text{tolerance}=1e-9$) qui teste si deux flottants sont égaux à une tolérance près.
3. Réécrire le test `if sum(prix) == 0.30:` en utilisant cette fonction, pour que le programme affiche "Ca fait bien 0,30 euro".

Exercice 19 ♦♦

Un système d'alarme se déclenche selon l'expression booléenne $(a \text{ or } b) \text{ and } (\text{not } c)$, où a = « capteur porte activé », b = « capteur fenêtre activé », c = « mode maintenance activé ».

1. Dresser la table de vérité complète de cette expression ($2^3 = 8$ lignes).
2. Pour combien de combinaisons de (a, b, c) l'alarme se déclenche-t-elle ?
3. En mode maintenance ($c = \text{Vrai}$), l'alarme peut-elle se déclencher, quels que soient a et b ? Justifier.

Exercice 20 ◆◆◆

On considère la chaîne "Bonjour à tous !".

1. Combien cette chaîne compte-t-elle de **caractères** ?
2. Combien d'**octets** occupe-t-elle une fois encodée en UTF-8 ? En ISO-8859-1 ? Pourquoi ces deux nombres diffèrent-ils entre eux, et diffèrent-ils du nombre de caractères ?
3. Que se passe-t-il si on tente de l'encoder en ASCII ? Justifier précisément (quel caractère pose problème et pourquoi).
4. Un serveur web français reçoit des messages d'utilisateurs du monde entier (arabe, chinois, russe...). Pourquoi ISO-8859-1 serait-il un mauvais choix d'encodage pour stocker ces messages, contrairement à UTF-8 ?

Exercice 21 ◆

1. Convertir 156 (écriture décimale) en base 2 puis en base 16, en détaillant les divisions euclidiennes successives.
2. Convertir 11001_2 en base 10.
3. Convertir $3F_{16}$ en base 10.

Exercice 22 ◆

1. Combien de bits sont nécessaires pour écrire 300 en binaire ?
2. Écrire 20 en complément à 2 sur 8 bits.
3. Écrire -12 en complément à 2 sur 8 bits (détailler : écriture de 12, inversion des bits, ajout de 1).

Exercice 23 ◆◆

Un élève écrit le programme suivant pour vérifier que trois pièces de 0,10 € font bien 0,30 € :

```
1 prix = [0.10] * 3
2 if sum(prix) == 0.30:
3     print("Ca fait bien 0,30 euro")
4 else:
5     print("Erreur de calcul !")
```

1. Que va afficher ce programme ? Pourquoi (relier à la représentation des flottants) ?
2. Écrire une fonction presque_egaux(a, b , tolerance= $1e-9$) qui teste si deux flottants sont égaux à une tolérance près.
3. Réécrire le test `if sum(prix) == 0.30:` en utilisant cette fonction, pour que le programme affiche "Ca fait bien 0,30 euro".

Exercice 24 ◆◆

Un système d'alarme se déclenche selon l'expression booléenne $(a \text{ or } b) \text{ and } (\text{not } c)$, où a = « capteur porte activé », b = « capteur fenêtre activé », c = « mode maintenance activé ».

1. Dresser la table de vérité complète de cette expression ($2^3 = 8$ lignes).
2. Pour combien de combinaisons de (a, b, c) l'alarme se déclenche-t-elle ?
3. En mode maintenance ($c = \text{Vrai}$), l'alarme peut-elle se déclencher, quels que soient a et b ? Justifier.

Exercice 25 ◆◆◆

On considère la chaîne "Bonjour à tous !".

1. Combien cette chaîne compte-t-elle de **caractères** ?
2. Combien d'**octets** occupe-t-elle une fois encodée en UTF-8 ? En ISO-8859-1 ? Pourquoi ces deux nombres diffèrent-ils entre eux, et diffèrent-ils du nombre de caractères ?
3. Que se passe-t-il si on tente de l'encoder en ASCII ? Justifier précisément (quel caractère pose problème et pourquoi).
4. Un serveur web français reçoit des messages d'utilisateurs du monde entier (arabe, chinois, russe...). Pourquoi ISO-8859-1 serait-il un mauvais choix d'encodage pour stocker ces messages, contrairement à UTF-8 ?

Exercice 26 ◆

1. Convertir 156 (écriture décimale) en base 2 puis en base 16, en détaillant les divisions euclidiennes successives.
2. Convertir 11001_2 en base 10.
3. Convertir $3F_{16}$ en base 10.

Exercice 27 ◆

1. Combien de bits sont nécessaires pour écrire 300 en binaire ?
2. Écrire 20 en complément à 2 sur 8 bits.
3. Écrire -12 en complément à 2 sur 8 bits (détailler : écriture de 12, inversion des bits, ajout de 1).

Exercice 28 ◆◆

Un élève écrit le programme suivant pour vérifier que trois pièces de 0,10 € font bien 0,30 € :

```
1 prix = [0.10] * 3
2 if sum(prix) == 0.30:
3     print("Ca fait bien 0,30 euro")
4 else:
5     print("Erreur de calcul !")
```

1. Que va afficher ce programme ? Pourquoi (relier à la représentation des flottants) ?
2. Écrire une fonction `presque_egaux(a, b, tolerance=1e-9)` qui teste si deux flottants sont égaux à une tolérance près.
3. Réécrire le test `if sum(prix) == 0.30:` en utilisant cette fonction, pour que le programme affiche "Ca fait bien 0,30 euro".

Exercice 29 ◆◆

Un système d'alarme se déclenche selon l'expression booléenne $(a \text{ or } b) \text{ and } (\text{not } c)$, où a = « capteur porte activé », b = « capteur fenêtre activé », c = « mode maintenance activé ».

1. Dresser la table de vérité complète de cette expression ($2^3 = 8$ lignes).
2. Pour combien de combinaisons de (a, b, c) l'alarme se déclenche-t-elle ?
3. En mode maintenance ($c = \text{Vrai}$), l'alarme peut-elle se déclencher, quels que soient a et b ? Justifier.

Exercice 30 ◆◆◆

On considère la chaîne "Bonjour à tous !".

1. Combien cette chaîne compte-t-elle de **caractères** ?
2. Combien d'**octets** occupe-t-elle une fois encodée en UTF-8 ? En ISO-8859-1 ? Pourquoi ces deux nombres diffèrent-ils entre eux, et diffèrent-ils du nombre de caractères ?
3. Que se passe-t-il si on tente de l'encoder en ASCII ? Justifier précisément (quel caractère pose problème et pourquoi).
4. Un serveur web français reçoit des messages d'utilisateurs du monde entier (arabe, chinois, russe...). Pourquoi ISO-8859-1 serait-il un mauvais choix d'encodage pour stocker ces messages, contrairement à UTF-8 ?

QCM d'auto-évaluation — Types et valeurs de base

Pour chaque question, une seule réponse est exacte. Entourez la lettre correspondante.

C1 — Écriture dans une base

1. [Q1] L'écriture 1010_2 correspond, en base 10, à :
 - A. 8
 - B. 10
 - C. 12
 - D. 1010

C2 — Complément à 2

2. [Q2] Sur 8 bits, combien de valeurs entières distinctes peut-on représenter ?
 - A. 8
 - B. 128
 - C. 255
 - D. 256
3. [Q3] En complément à 2, le bit de poids fort indique :
 - A. la parité du nombre.
 - B. le signe du nombre.
 - C. le nombre de bits utilisés.
 - D. rien de particulier.

C3 — Flottants

4. [Q4] L'expression $0.1 + 0.2 == 0.3$ en Python renvoie :
 - A. True, car $0,1 + 0,2 = 0,3$.
 - B. False, à cause des erreurs d'arrondi de la représentation flottante.
 - C. Une erreur de syntaxe.
 - D. Cela dépend de la version de Python.

C4 — Booléens

5. [Q5] L'expression `True and False` vaut :
 - A. True
 - B. False
 - C. None

D. Une erreur

C5 — Encodage de texte

6. [Q6] Un texte contenant uniquement des lettres non accentuées et des chiffres peut être encodé sans erreur :
- uniquement en UTF-8.
 - uniquement en ISO-8859-1.
 - en ASCII, en ISO-8859-1 et en UTF-8.
 - dans aucun de ces trois encodages.

Corrigé

1. Divisions par 2 : $90 = 2 \times 45 + 0$, $45 = 2 \times 22 + 1$, $22 = 2 \times 11 + 0$, $11 = 2 \times 5 + 1$, $5 = 2 \times 2 + 1$, $2 = 2 \times 1 + 0$, $1 = 2 \times 0 + 1$. Restes de bas en haut : $90 = 1011010_2$. En base 16 : $90 = 16 \times 5 + 10$ (10 = A), soit $90 = 5A_{16}$.

2. $2^6 = 64 \leq 90 < 128 = 2^7$, donc 90 nécessite 7 bits.

3. $7 = 0000111_2$, soit sur 8 bits : 0000111. Inversion des bits : 1111000. Ajout de 1 : 1111001. Donc -7 s'écrit 1111001.

Corrigé

1. Le test renvoie False. Chaque 0.1 est stocké de façon approchée en mémoire ; après trois additions, la valeur accumulée (0.30000000000000004) n'est pas identique bit à bit à la représentation flottante de 0.3.

2.

a	b	résultat
V	V	F
V	F	V
F	V	V
F	F	F

Cette expression reproduit le **ou exclusif** (xor) : elle est vraie lorsque a et b ont des valeurs différentes.

3. Oui, les trois encodages donnent 4 octets. Le mot "info" ne contient que des lettres non accentuées, présentes à l'identique dans ASCII (7 bits), ISO-8859-1 (8 bits, compatible ASCII sur ses 128 premiers caractères) et UTF-8 (qui code les 128 premiers caractères Unicode sur un seul octet, identique à ASCII).

Évaluation A — Types et valeurs de base

Durée : 45 minutes. Calculatrice et brouillon autorisés, pas d'ordinateur.

Barème indicatif : Ex. 1 : 10 pts (bases, complément à 2); Ex. 2 : 10 pts (flottants, booléens, encodage)

Exercice 1 — Bases et complément à 2

(10 points)

- (4 pts) Convertir 90 (écriture décimale) en base 2 puis en base 16, en détaillant les calculs.
- (3 pts) Combien de bits sont nécessaires pour écrire 90 en binaire ?
- (3 pts) Écrire -7 en complément à 2 sur 8 bits, en détaillant la méthode (écriture de 7, inversion des bits, ajout de 1).

Exercice 2 — Flottants, booléens, encodage

(10 points)

- (3 pts) Que renvoie le test $0.1 + 0.1 + 0.1 == 0.3$ en Python ? Expliquer pourquoi.
- (4 pts) Dresser la table de vérité de l'expression $(a \text{ and not } b) \text{ or } (\text{not } a \text{ and } b)$. Quel opérateur booléen usuel cette expression reproduit-elle ?
- (3 pts) Le mot "info" (lettres non accentuées) occupe-t-il le même nombre d'octets une fois encodé en ASCII, en ISO-8859-1 et en UTF-8 ? Justifier.

Corrigé

1. Divisions par 2 : $150 = 2 \times 75 + 0$, $75 = 2 \times 37 + 1$, $37 = 2 \times 18 + 1$, $18 = 2 \times 9 + 0$, $9 = 2 \times 4 + 1$, $4 = 2 \times 2 + 0$, $2 = 2 \times 1 + 0$, $1 = 2 \times 0 + 1$. Restes de bas en haut : $150 = 10010110_2$. En base 16 : $150 = 16 \times 9 + 6$, soit $150 = 96_{16}$.

2. $2^7 = 128 \leq 150 < 256 = 2^8$, donc 150 nécessite 8 bits.

3. $20 = 00010100_2$ sur 8 bits. Inversion des bits : 11101011. Ajout de 1 : 11101100. Donc -20 s'écrit 11101100.

Corrigé

1. Le test renvoie False : 0.2 n'a pas d'écriture binaire finie, et l'accumulation de trois additions produit une valeur (0.6000000000000001) légèrement différente de la représentation flottante de 0.6. Une réécriture correcte : $\text{abs}((0.2+0.2+0.2) - 0.6) < 1e-9$.

2.

a	b	c	résultat
V	V	V	V
V	V	F	F
V	F	V	V
V	F	F	F
F	V	V	V
F	V	F	F
F	F	V	F
F	F	F	F

L'expression est vraie pour 3 combinaisons sur 8.

3. Non. En UTF-8, "vélo" occupe 5 octets (le « é » occupe deux octets en UTF-8, les trois autres lettres un octet chacune). En ISO-8859-1, il occupe 4 octets (chaque caractère, y compris « é », est codé sur un seul octet dans cet encodage).

Évaluation B — Types et valeurs de base

Durée : 45 minutes. Calculatrice et brouillon autorisés, pas d'ordinateur.

Barème indicatif : Ex. 1 : 10 pts (bases, complément à 2) ; Ex. 2 : 10 pts (flottants, booléens, encodage)

Exercice 1 — Bases et complément à 2

(10 points)

- (4 pts) Convertir 150 (écriture décimale) en base 2 puis en base 16, en détaillant les calculs.
- (3 pts) Combien de bits sont nécessaires pour écrire 150 en binaire ?
- (3 pts) Écrire -20 en complément à 2 sur 8 bits, en détaillant la méthode.

Exercice 2 — Flottants, booléens, encodage

(10 points)

- (3 pts) Que renvoie le test $0.2 + 0.2 + 0.2 == 0.6$ en Python ? Expliquer pourquoi, et proposer une réécriture correcte de ce test.

2. (4 pts) Dresser la table de vérité de l'expression $(a \text{ or } b) \text{ and } c$. Pour combien de combinaisons de (a, b, c) cette expression est-elle vraie ?
3. (3 pts) Le mot "vélo" occupe-t-il le même nombre d'octets une fois encodé en ISO-8859-1 et en UTF-8 ? Justifier précisément la valeur trouvée pour chaque encodage.

Exercice 31

Un élève écrit une boucle qui ajoute 0,1 trois fois de suite à une variable x initialisée à 0,0, puis teste `if x == 0.3`. Il s'attend à ce que ce test soit vérifié, mais ce n'est pas le cas.

1. Quelle est la valeur exacte affichée par `print(x)` après la boucle ?
2. Pourquoi le test `x == 0.3` échoue-t-il malgré tout ?
3. Proposer une correction du test qui fonctionne.

Exercice 32

Un élève écrit une boucle qui ajoute 0,1 trois fois de suite à une variable x initialisée à 0,0, puis teste `if x == 0.3`. Il s'attend à ce que ce test soit vérifié, mais ce n'est pas le cas.

1. Quelle est la valeur exacte affichée par `print(x)` après la boucle ?
2. Pourquoi le test `x == 0.3` échoue-t-il malgré tout ?
3. Proposer une correction du test qui fonctionne.

Exercice 33

Un élève écrit une boucle qui ajoute 0,1 trois fois de suite à une variable x initialisée à 0,0, puis teste `if x == 0.3`. Il s'attend à ce que ce test soit vérifié, mais ce n'est pas le cas.

1. Quelle est la valeur exacte affichée par `print(x)` après la boucle ?
2. Pourquoi le test `x == 0.3` échoue-t-il malgré tout ?
3. Proposer une correction du test qui fonctionne.

Exercice 34

Un élève écrit une boucle qui ajoute 0,1 trois fois de suite à une variable x initialisée à 0,0, puis teste `if x == 0.3`. Il s'attend à ce que ce test soit vérifié, mais ce n'est pas le cas.

1. Quelle est la valeur exacte affichée par `print(x)` après la boucle ?
2. Pourquoi le test `x == 0.3` échoue-t-il malgré tout ?
3. Proposer une correction du test qui fonctionne.

Exercice 35

Un élève écrit une boucle qui ajoute 0,1 trois fois de suite à une variable x initialisée à 0,0, puis teste `if x == 0.3`. Il s'attend à ce que ce test soit vérifié, mais ce n'est pas le cas.

1. Quelle est la valeur exacte affichée par `print(x)` après la boucle ?
2. Pourquoi le test `x == 0.3` échoue-t-il malgré tout ?
3. Proposer une correction du test qui fonctionne.

Exercice 36

Un élève écrit une boucle qui ajoute 0,1 trois fois de suite à une variable x initialisée à 0,0, puis teste `if x == 0.3`. Il s'attend à ce que ce test soit vérifié, mais ce n'est pas le cas.

1. Quelle est la valeur exacte affichée par `print(x)` après la boucle ?
2. Pourquoi le test `x == 0.3` échoue-t-il malgré tout ?
3. Proposer une correction du test qui fonctionne.

Corrigé

1. Divisions par 2 : $156 = 2 \times 78 + 0$, $78 = 2 \times 39 + 0$, $39 = 2 \times 19 + 1$, $19 = 2 \times 9 + 1$, $9 = 2 \times 4 + 1$, $4 = 2 \times 2 + 0$, $2 = 2 \times 1 + 0$, $1 = 2 \times 0 + 1$. En lisant les restes de bas en haut : $156 = 10011100_2$.

Divisions par 16 : $156 = 16 \times 9 + 12$ ($12 = C$ en hexadécimal), $9 = 16 \times 0 + 9$. Donc $156 = 9C_{16}$.

2. $11001_2 = 1 \times 16 + 1 \times 8 + 0 \times 4 + 0 \times 2 + 1 \times 1 = 25$.

3. $3F_{16} = 3 \times 16 + 15 = 48 + 15 = 63$.

Corrigé

1. $2^8 = 256 \leq 300 < 512 = 2^9$, donc 300 nécessite 9 bits.

2. $20 = 16 + 4 = 10100_2$, soit sur 8 bits : 00010100.

3. $12 = 1100_2$, soit sur 8 bits : 00001100. En inversant chaque bit : 11110011. En ajoutant 1 : 11110100. Donc -12 s'écrit 11110100 en complément à 2 sur 8 bits.

Corrigé

1. Le programme affiche "Erreur de calcul !". En effet, 0.10 n'a pas d'écriture binaire finie : $\text{sum}([0.10, 0.10, 0.10])$ vaut en réalité 0.30000000000000004, une valeur très proche de mais **différente** de 0.30 (qui, lui aussi, est stocké de façon approchée). Le test `==` compare des valeurs qui ne sont pas bit à bit identiques.

2.

```
1 def presque_egaux(a, b, tolerance=1e-9):
2     return abs(a - b) < tolerance
```

3.

```
1 prix = [0.10] * 3
2 if presque_egaux(sum(prix), 0.30):
3     print("Ca fait bien 0,30 euro")
4 else:
5     print("Erreur de calcul !")
```

Avec ce test tolérant, le programme affiche bien "Ca fait bien 0,30 euro".

Corrigé

1.

a	b	c	$(a \text{ or } b) \text{ and } (\text{not } c)$
V	V	V	F
V	V	F	V
V	F	V	F
V	F	F	V
F	V	V	F
F	V	F	V
F	F	V	F
F	F	F	F

2. L'alarme se déclenche pour 3 combinaisons sur 8 (celles où $c = \text{Faux}$ et $(a \text{ or } b) = \text{Vrai}$).

3. Non : dès que $c = \text{Vrai}$ (mode maintenance), $\text{not } c = \text{Faux}$, donc $(a \text{ or } b) \text{ and } \text{Faux} = \text{Faux}$ quelles que soient les valeurs de a et b : l'alarme ne peut jamais se déclencher en mode maintenance.

Corrigé

1. La chaîne compte 16 caractères.

2. En UTF-8 : 17 octets. En ISO-8859-1 : 16 octets. Le « à » occupe 2 octets en UTF-8 (car son code Unicode dépasse la plage codée sur un seul octet) mais 1 seul octet en ISO-8859-1 (qui code directement les caractères accentués usuels d'Europe de l'Ouest sur un octet) : d'où l'écart d'un octet entre les deux encodages. En ISO-8859-1, tous les caractères de cette chaîne tiennent sur un octet, donc le nombre d'octets égale le nombre de caractères ; ce n'est pas le cas en UTF-8 à cause du « à ».

3. L'encodage échoue avec une `UnicodeEncodeError` : le caractère 'à' n'existe pas dans le jeu de caractères ASCII (qui ne code que les 128 premiers caractères Unicode, c'est-à-dire les lettres non accentuées, chiffres et ponctuation de base).

4. ISO-8859-1 ne code que les caractères d'Europe de l'Ouest (256 caractères au total) : il ne peut pas représenter les caractères arabes, chinois ou cyrilliques. UTF-8, en revanche, peut coder **tous** les caractères de toutes les langues du monde (table Unicode), ce qui en fait le choix adapté pour un serveur recevant des messages internationaux.

Corrigé

1. `print(x)` affiche 0.30000000000000004.

2. Le littéral 0.1 est déjà stocké sous une valeur approchée. Dans cette boucle, les deux premières additions sont exactes relativement à leurs opérandes flottants, mais la troisième doit être arrondie. Après trois additions, la valeur accumulée n'est donc plus exactement égale à la représentation flottante de 0,3 : les deux calculs produisent deux nombres flottants représentables distincts. Le test `==` compare les valeurs selon les règles d'égalité de Python, pas leurs représentations binaires bit à bit. Par exemple, `-0.0 == 0.0` vaut `True`, tandis que `float("nan") == float("nan")` vaut `False`.

3. Il faut remplacer le test par une comparaison à tolérance près :

```
1 def presque_egaux(a, b, tolerance=1e-9):
2     return abs(a - b) < tolerance
3
4 if presque_egaux(x, 0.3):
5     ...
```

Corrigé

1. Divisions par 2 : $156 = 2 \times 78 + 0$, $78 = 2 \times 39 + 0$, $39 = 2 \times 19 + 1$, $19 = 2 \times 9 + 1$, $9 = 2 \times 4 + 1$, $4 = 2 \times 2 + 0$, $2 = 2 \times 1 + 0$, $1 = 2 \times 0 + 1$. En lisant les restes de bas en haut : $156 = 10011100_2$.

Divisions par 16 : $156 = 16 \times 9 + 12$ ($12 = C$ en hexadécimal), $9 = 16 \times 0 + 9$. Donc $156 = 9C_{16}$.

2. $11001_2 = 1 \times 16 + 1 \times 8 + 0 \times 4 + 0 \times 2 + 1 \times 1 = 25$.

3. $3F_{16} = 3 \times 16 + 15 = 48 + 15 = 63$.

Corrigé

1. $2^8 = 256 \leq 300 < 512 = 2^9$, donc 300 nécessite 9 bits.

2. $20 = 16 + 4 = 10100_2$, soit sur 8 bits : 00010100.

3. $12 = 1100_2$, soit sur 8 bits : 00001100. En inversant chaque bit : 11110011. En ajoutant 1 : 11110100. Donc -12 s'écrit 11110100 en complément à 2 sur 8 bits.

Corrigé

1. Le programme affiche "Erreur de calcul !". En effet, 0.10 n'a pas d'écriture binaire finie : `sum([0.10, 0.10, 0.10])` vaut en réalité 0.30000000000000004, une valeur très proche de mais **différente** de 0.30 (qui, lui aussi, est stocké de façon approchée). Le test `==` compare des valeurs qui ne sont pas bit à bit identiques.

2.

```
1 def presque_egaux(a, b, tolerance=1e-9):
2     return abs(a - b) < tolerance
```

3.

```

1 prix = [0.10] * 3
2 if presque_egaux(sum(prix), 0.30):
3     print("Ca fait bien 0,30 euro")
4 else:
5     print("Erreur de calcul !")

```

Avec ce test tolérant, le programme affiche bien "Ca fait bien 0,30 euro".

Corrigé

1.

<i>a</i>	<i>b</i>	<i>c</i>	<i>(a or b) and (not c)</i>
V	V	V	F
V	V	F	V
V	F	V	F
V	F	F	V
F	V	V	F
F	V	F	V
F	F	V	F
F	F	F	F

2. L'alarme se déclenche pour 3 combinaisons sur 8 (celles où c = Faux et $(a \text{ or } b)$ = Vrai).

3. Non : dès que c = Vrai (mode maintenance), $\text{not } c$ = Faux, donc $(a \text{ or } b) \text{ and Faux}$ = Faux quelles que soient les valeurs de a et b : l'alarme ne peut jamais se déclencher en mode maintenance.

Corrigé

1. La chaîne compte 16 caractères.

2. En UTF-8 : 17 octets. En ISO-8859-1 : 16 octets. Le « à » occupe 2 octets en UTF-8 (car son code Unicode dépasse la plage codée sur un seul octet) mais 1 seul octet en ISO-8859-1 (qui code directement les caractères accentués usuels d'Europe de l'Ouest sur un octet) : d'où l'écart d'un octet entre les deux encodages. En ISO-8859-1, tous les caractères de cette chaîne tiennent sur un octet, donc le nombre d'octets égale le nombre de caractères ; ce n'est pas le cas en UTF-8 à cause du « à ».

3. L'encodage échoue avec une UnicodeEncodeError : le caractère 'à' n'existe pas dans le jeu de caractères ASCII (qui ne code que les 128 premiers caractères Unicode, c'est-à-dire les lettres non accentuées, chiffres et ponctuation de base).

4. ISO-8859-1 ne code que les caractères d'Europe de l'Ouest (256 caractères au total) : il ne peut pas représenter les caractères arabes, chinois ou cyrilliques. UTF-8, en revanche, peut coder **tous** les caractères de toutes les langues du monde (table Unicode), ce qui en fait le choix adapté pour un serveur recevant des messages internationaux.

Corrigé

1. `print(x)` affiche 0.30000000000000004.

2. Le littéral 0.1 est déjà stocké sous une valeur approchée. Dans cette boucle, les deux premières additions sont exactes relativement à leurs opérandes flottants, mais la troisième doit être arrondie. Après trois additions, la valeur accumulée n'est donc plus exactement égale à la représentation flottante de 0,3 : les deux calculs produisent deux nombres flottants représentables distincts. Le test `==` compare les valeurs selon les règles d'égalité de Python, pas leurs représentations binaires bit à bit. Par exemple, `-0.0 == 0.0` vaut True, tandis que `float("nan") == float("nan")` vaut False.

3. Il faut remplacer le test par une comparaison à tolérance près :

```

1 def presque_egaux(a, b, tolerance=1e-9):
2     return abs(a - b) < tolerance

```

```

3
4 if presque_egaux(x, 0.3):
5     ...

```

Corrigé

1. Divisions par 2 : $156 = 2 \times 78 + 0$, $78 = 2 \times 39 + 0$, $39 = 2 \times 19 + 1$, $19 = 2 \times 9 + 1$, $9 = 2 \times 4 + 1$, $4 = 2 \times 2 + 0$, $2 = 2 \times 1 + 0$, $1 = 2 \times 0 + 1$. En lisant les restes de bas en haut : $156 = 10011100_2$.

Divisions par 16 : $156 = 16 \times 9 + 12$ ($12 = C$ en hexadécimal), $9 = 16 \times 0 + 9$. Donc $156 = 9C_{16}$.

2. $11001_2 = 1 \times 16 + 1 \times 8 + 0 \times 4 + 0 \times 2 + 1 \times 1 = 25$.

3. $3F_{16} = 3 \times 16 + 15 = 48 + 15 = 63$.

Corrigé

1. $2^8 = 256 \leq 300 < 512 = 2^9$, donc 300 nécessite 9 bits.

2. $20 = 16 + 4 = 10100_2$, soit sur 8 bits : 00010100 .

3. $12 = 1100_2$, soit sur 8 bits : 00001100 . En inversant chaque bit : 11110011 . En ajoutant 1 : 11110100 . Donc -12 s'écrit 11110100 en complément à 2 sur 8 bits.

Corrigé

1. Le programme affiche "Erreur de calcul !". En effet, 0.10 n'a pas d'écriture binaire finie : $\text{sum}([0.10, 0.10, 0.10])$ vaut en réalité 0.30000000000000004 , une valeur très proche de mais **différente** de 0.30 (qui, lui aussi, est stocké de façon approchée). Le test $==$ compare des valeurs qui ne sont pas bit à bit identiques.

2.

```

1 def presque_egaux(a, b, tolerance=1e-9):
2     return abs(a - b) < tolerance

```

3.

```

1 prix = [0.10] * 3
2 if presque_egaux(sum(prix), 0.30):
3     print("Ca fait bien 0,30 euro")
4 else:
5     print("Erreur de calcul !")

```

Avec ce test tolérant, le programme affiche bien "Ca fait bien 0,30 euro".

Corrigé

1.

a	b	c	$(a \text{ or } b) \text{ and } (\text{not } c)$
V	V	V	F
V	V	F	V
V	F	V	F
V	F	F	V
F	V	V	F
F	V	F	V
F	F	V	F
F	F	F	F

2. L'alarme se déclenche pour 3 combinaisons sur 8 (celles où $c = \text{Faux}$ et $(a \text{ or } b) = \text{Vrai}$).

3. Non : dès que $c = \text{Vrai}$ (mode maintenance), $\text{not } c = \text{Faux}$, donc $(a \text{ or } b) \text{ and Faux} = \text{Faux}$ quelles que soient les valeurs de a et b : l'alarme ne peut jamais se déclencher en mode maintenance.

Corrigé

1. La chaîne compte 16 caractères.
2. En UTF-8 : 17 octets. En ISO-8859-1 : 16 octets. Le « à » occupe 2 octets en UTF-8 (car son code Unicode dépasse la plage codée sur un seul octet) mais 1 seul octet en ISO-8859-1 (qui code directement les caractères accentués usuels d'Europe de l'Ouest sur un octet) : d'où l'écart d'un octet entre les deux encodages. En ISO-8859-1, tous les caractères de cette chaîne tiennent sur un octet, donc le nombre d'octets égale le nombre de caractères ; ce n'est pas le cas en UTF-8 à cause du « à ».
3. L'encodage échoue avec une `UnicodeEncodeError` : le caractère 'à' n'existe pas dans le jeu de caractères ASCII (qui ne code que les 128 premiers caractères Unicode, c'est-à-dire les lettres non accentuées, chiffres et ponctuation de base).
4. ISO-8859-1 ne code que les caractères d'Europe de l'Ouest (256 caractères au total) : il ne peut pas représenter les caractères arabes, chinois ou cyrilliques. UTF-8, en revanche, peut coder **tous** les caractères de toutes les langues du monde (table Unicode), ce qui en fait le choix adapté pour un serveur recevant des messages internationaux.

Corrigé

1. `print(x)` affiche 0.30000000000000004.
2. Le littéral 0.1 est déjà stocké sous une valeur approchée. Dans cette boucle, les deux premières additions sont exactes relativement à leurs opérandes flottants, mais la troisième doit être arrondie. Après trois additions, la valeur accumulée n'est donc plus exactement égale à la représentation flottante de 0,3 : les deux calculs produisent deux nombres flottants représentables distincts. Le test `==` compare les valeurs selon les règles d'égalité de Python, pas leurs représentations binaires bit à bit. Par exemple, `-0.0 == 0.0` vaut `True`, tandis que `float("nan") == float("nan")` vaut `False`.
3. Il faut remplacer le test par une comparaison à tolérance près :

```
1 def presque_egaux(a, b, tolerance=1e-9):
2     return abs(a - b) < tolerance
3
4 if presque_egaux(x, 0.3):
5     ...
```

Corrigé

1. Divisions par 2 : $156 = 2 \times 78 + 0$, $78 = 2 \times 39 + 0$, $39 = 2 \times 19 + 1$, $19 = 2 \times 9 + 1$, $9 = 2 \times 4 + 1$, $4 = 2 \times 2 + 0$, $2 = 2 \times 1 + 0$, $1 = 2 \times 0 + 1$. En lisant les restes de bas en haut : $156 = 10011100_2$.
Divisions par 16 : $156 = 16 \times 9 + 12$ ($12 = C$ en hexadécimal), $9 = 16 \times 0 + 9$. Donc $156 = 9C_{16}$.
2. $11001_2 = 1 \times 16 + 1 \times 8 + 0 \times 4 + 0 \times 2 + 1 \times 1 = 25$.
3. $3F_{16} = 3 \times 16 + 15 = 48 + 15 = 63$.

Corrigé

1. $2^8 = 256 \leq 300 < 512 = 2^9$, donc 300 nécessite 9 bits.
2. $20 = 16 + 4 = 10100_2$, soit sur 8 bits : 00010100.
3. $12 = 1100_2$, soit sur 8 bits : 00001100. En inversant chaque bit : 11110011. En ajoutant 1 : 11110100. Donc -12 s'écrit 11110100 en complément à 2 sur 8 bits.

Corrigé

1. Le programme affiche "Erreur de calcul !". En effet, 0.10 n'a pas d'écriture binaire finie : `sum([0.10, 0.10, 0.10])` vaut en réalité 0.30000000000000004, une valeur très proche de mais **différente** de

0.30 (qui, lui aussi, est stocké de façon approchée). Le test `==` compare des valeurs qui ne sont pas bit à bit identiques.

2.

```
1 def presque_egaux(a, b, tolerance=1e-9):
2     return abs(a - b) < tolerance
```

3.

```
1 prix = [0.10] * 3
2 if presque_egaux(sum(prix), 0.30):
3     print("Ca fait bien 0,30 euro")
4 else:
5     print("Erreur de calcul !")
```

Avec ce test tolérant, le programme affiche bien "Ca fait bien 0,30 euro".

Corrigé

1.

<i>a</i>	<i>b</i>	<i>c</i>	<i>(a or b) and (not c)</i>
V	V	V	F
V	V	F	V
V	F	V	F
V	F	F	V
F	V	V	F
F	V	F	V
F	F	V	F
F	F	F	F

2. L'alarme se déclenche pour 3 combinaisons sur 8 (celles où *c* = Faux et *(a or b)* = Vrai).

3. Non : dès que *c* = Vrai (mode maintenance), *not c* = Faux, donc *(a or b) and Faux* = Faux quelles que soient les valeurs de *a* et *b* : l'alarme ne peut jamais se déclencher en mode maintenance.

Corrigé

1. La chaîne compte 16 caractères.

2. En UTF-8 : 17 octets. En ISO-8859-1 : 16 octets. Le « à » occupe 2 octets en UTF-8 (car son code Unicode dépasse la plage codée sur un seul octet) mais 1 seul octet en ISO-8859-1 (qui code directement les caractères accentués usuels d'Europe de l'Ouest sur un octet) : d'où l'écart d'un octet entre les deux encodages. En ISO-8859-1, tous les caractères de cette chaîne tiennent sur un octet, donc le nombre d'octets égale le nombre de caractères ; ce n'est pas le cas en UTF-8 à cause du « à ».

3. L'encodage échoue avec une `UnicodeEncodeError` : le caractère 'à' n'existe pas dans le jeu de caractères ASCII (qui ne code que les 128 premiers caractères Unicode, c'est-à-dire les lettres non accentuées, chiffres et ponctuation de base).

4. ISO-8859-1 ne code que les caractères d'Europe de l'Ouest (256 caractères au total) : il ne peut pas représenter les caractères arabes, chinois ou cyrilliques. UTF-8, en revanche, peut coder **tous** les caractères de toutes les langues du monde (table Unicode), ce qui en fait le choix adapté pour un serveur recevant des messages internationaux.

Corrigé

1. `print(x)` affiche 0.30000000000000004.

2. Le littéral 0.1 est déjà stocké sous une valeur approchée. Dans cette boucle, les deux premières additions sont exactes relativement à leurs opérandes flottants, mais la troisième doit être arrondie. Après trois additions, la valeur accumulée n'est donc plus exactement égale à la représentation

flottante de 0,3 : les deux calculs produisent deux nombres flottants représentables distincts. Le test `==` compare les valeurs selon les règles d'égalité de Python, pas leurs représentations binaires bit à bit. Par exemple, `-0.0 == 0.0` vaut `True`, tandis que `float("nan") == float("nan")` vaut `False`.

3. Il faut remplacer le test par une comparaison à tolérance près :

```
1 def presque_egaux(a, b, tolerance=1e-9):
2     return abs(a - b) < tolerance
3
4 if presque_egaux(x, 0.3):
5     ...
```

Corrigé

1. Divisions par 2 : $156 = 2 \times 78 + 0$, $78 = 2 \times 39 + 0$, $39 = 2 \times 19 + 1$, $19 = 2 \times 9 + 1$, $9 = 2 \times 4 + 1$, $4 = 2 \times 2 + 0$, $2 = 2 \times 1 + 0$, $1 = 2 \times 0 + 1$. En lisant les restes de bas en haut : $156 = 10011100_2$.

Divisions par 16 : $156 = 16 \times 9 + 12$ ($12 = C$ en hexadécimal), $9 = 16 \times 0 + 9$. Donc $156 = 9C_{16}$.

2. $11001_2 = 1 \times 16 + 1 \times 8 + 0 \times 4 + 0 \times 2 + 1 \times 1 = 25$.

3. $3F_{16} = 3 \times 16 + 15 = 48 + 15 = 63$.

Corrigé

1. $2^8 = 256 \leq 300 < 512 = 2^9$, donc 300 nécessite 9 bits.

2. $20 = 16 + 4 = 10100_2$, soit sur 8 bits : 00010100 .

3. $12 = 1100_2$, soit sur 8 bits : 00001100 . En inversant chaque bit : 11110011 . En ajoutant 1 : 11110100 . Donc -12 s'écrit 11110100 en complément à 2 sur 8 bits.

Corrigé

1. Le programme affiche "Erreur de calcul !". En effet, 0.10 n'a pas d'écriture binaire finie : `sum([0.10, 0.10, 0.10])` vaut en réalité 0.30000000000000004 , une valeur très proche de mais **différente** de 0.30 (qui, lui aussi, est stocké de façon approchée). Le test `==` compare des valeurs qui ne sont pas bit à bit identiques.

2.

```
1 def presque_egaux(a, b, tolerance=1e-9):
2     return abs(a - b) < tolerance
```

3.

```
1 prix = [0.10] * 3
2 if presque_egaux(sum(prix), 0.30):
3     print("Ca fait bien 0,30 euro")
4 else:
5     print("Erreur de calcul !")
```

Avec ce test tolérant, le programme affiche bien "Ca fait bien 0,30 euro".

Corrigé

1.

<i>a</i>	<i>b</i>	<i>c</i>	(<i>a</i> or <i>b</i>) and (not <i>c</i>)
V	V	V	F
V	V	F	V
V	F	V	F
V	F	F	V
F	V	V	F
F	V	F	V
F	F	V	F
F	F	F	F

2. L'alarme se déclenche pour 3 combinaisons sur 8 (celles où $c = \text{Faux}$ et $(a \text{ or } b) = \text{Vrai}$).

3. Non : dès que $c = \text{Vrai}$ (mode maintenance), not $c = \text{Faux}$, donc $(a \text{ or } b) \text{ and Faux} = \text{Faux}$ quelles que soient les valeurs de a et b : l'alarme ne peut jamais se déclencher en mode maintenance.

Corrigé

1. La chaîne compte 16 caractères.

2. En UTF-8 : 17 octets. En ISO-8859-1 : 16 octets. Le « à » occupe 2 octets en UTF-8 (car son code Unicode dépasse la plage codée sur un seul octet) mais 1 seul octet en ISO-8859-1 (qui code directement les caractères accentués usuels d'Europe de l'Ouest sur un octet) : d'où l'écart d'un octet entre les deux encodages. En ISO-8859-1, tous les caractères de cette chaîne tiennent sur un octet, donc le nombre d'octets égale le nombre de caractères ; ce n'est pas le cas en UTF-8 à cause du « à ».

3. L'encodage échoue avec une `UnicodeEncodeError` : le caractère 'à' n'existe pas dans le jeu de caractères ASCII (qui ne code que les 128 premiers caractères Unicode, c'est-à-dire les lettres non accentuées, chiffres et ponctuation de base).

4. ISO-8859-1 ne code que les caractères d'Europe de l'Ouest (256 caractères au total) : il ne peut pas représenter les caractères arabes, chinois ou cyrilliques. UTF-8, en revanche, peut coder **tous** les caractères de toutes les langues du monde (table Unicode), ce qui en fait le choix adapté pour un serveur recevant des messages internationaux.

Corrigé

1. `print(x)` affiche 0.30000000000000004.

2. Le littéral 0.1 est déjà stocké sous une valeur approchée. Dans cette boucle, les deux premières additions sont exactes relativement à leurs opérandes flottants, mais la troisième doit être arrondie. Après trois additions, la valeur accumulée n'est donc plus exactement égale à la représentation flottante de 0,3 : les deux calculs produisent deux nombres flottants représentables distincts. Le test `==` compare les valeurs selon les règles d'égalité de Python, pas leurs représentations binaires bit à bit. Par exemple, `-0.0 == 0.0` vaut `True`, tandis que `float("nan") == float("nan")` vaut `False`.

3. Il faut remplacer le test par une comparaison à tolérance près :

```
1 def presque_egaux(a, b, tolerance=1e-9):
2     return abs(a - b) < tolerance
3
4 if presque_egaux(x, 0.3):
5     ...
```

Corrigé

1. `print(x)` affiche 0.30000000000000004.

2. Le littéral 0.1 est déjà stocké sous une valeur approchée. Dans cette boucle, les deux premières additions sont exactes relativement à leurs opérandes flottants, mais la troisième doit être arrondie. Après trois additions, la valeur accumulée n'est donc plus exactement égale à la représentation flottante de 0,3 : les deux calculs produisent deux nombres flottants représentables distincts. Le test `==` compare les valeurs selon les règles d'égalité de Python, pas leurs représentations binaires bit à bit. Par exemple, `-0.0 == 0.0` vaut `True`, tandis que `float("nan") == float("nan")` vaut `False`.

3. Il faut remplacer le test par une comparaison à tolérance près :

```
1 def presque_egaux(a, b, tolerance=1e-9):  
2     return abs(a - b) < tolerance  
3  
4 if presque_egaux(x, 0.3):  
5     ...
```

2 Types construits : p-uplets, tableaux, dictionnaires

TD 1 — Station météo connectée

Durée : 50 minutes. Objectif : manipuler tuples, tableaux et dictionnaires dans un contexte réaliste.

Une station météo enregistre des mesures toutes les heures. Chaque mesure est un tuple (heure, température, humidité). L'ensemble des mesures d'une journée est stocké dans un tableau.

```
1 mesures = [  
2     (0, 15.2, 82), (1, 14.8, 85), (2, 14.1, 88),  
3     (6, 13.5, 90), (7, 14.0, 87), (8, 15.5, 80),  
4     (12, 22.3, 55), (13, 23.1, 52), (14, 23.8, 50),  
5     (18, 20.1, 60), (19, 18.5, 65), (20, 17.2, 70),  
6 ]
```

Partie A — Exploration (C1, C2)

Exercice 1 ♦

1. Donner le type de `mesures[0]` et la valeur de `mesures[0][1]`.
2. Écrire une fonction `temperature_max(mesures)` qui renvoie un tuple (heure, temp) correspondant à la température la plus élevée.

Exercice 2 ♦

Écrire une fonction `temperatures(mesures)` qui renvoie le tableau des températures extraites de chaque mesure, en compréhension de liste.

Partie B — Analyse (C4)

Exercice 3 ♦♦

On veut regrouper les mesures par tranche horaire dans un dictionnaire : "nuit" (0h–6h), "matin" (7h–12h), "après-midi" (13h–18h), "soir" (19h–23h).

Écrire une fonction `par_tranche(mesures)` qui renvoie un dictionnaire dont les clés sont les tranches et les valeurs sont les tableaux de mesures correspondantes.

Exercice 4 ♦♦

Écrire une fonction `resume(mesures)` qui renvoie un dictionnaire de la forme :

```
1 { "temp_min": 13.5, "temp_max": 23.8,  
2   "hum_moy": 72.0, "nb_mesures": 12 }
```

Partie C — Synthèse (C1, C2, C4)

Exercice 5 ♦♦

Écrire un programme complet qui :

1. Calcule le résumé de la journée.
2. Affiche les mesures de la tranche "après-midi".
3. Identifie l'heure la plus humide.

TD 2 — Classement d'un tournoi

Durée : 50 minutes. Objectif : croiser tableaux, dictionnaires et mutabilité dans un projet de classement.

Un tournoi oppose 6 joueurs sur 4 manches. Les résultats sont stockés dans une grille (tableau de tableaux) : chaque ligne est un joueur, chaque colonne une manche.

```
1 pseudos = ["Ace", "Bob", "Cat", "Dan", "Eve", "Fox"]
2 scores = [
3     [120, 150, 130, 140],
4     [180, 160, 170, 190],
5     [110, 140, 120, 100],
6     [200, 180, 210, 190],
7     [90, 110, 100, 120],
8     [160, 170, 150, 180],
9 ]
```

Partie A — Lecture de la grille (C3)

Exercice 6 ◆

1. Combien de lignes et de colonnes a la grille scores ?
2. Donner l'expression Python pour obtenir le score de Cat à la manche 3.
3. Écrire une fonction `total_joueur(scores, i)` qui renvoie la somme des scores du joueur d'indice `i`.

Partie B — Construction du classement (C2, C4)

Exercice 7 ◆◆

Écrire une fonction `classement(pseudos, scores)` qui renvoie un tableau de dictionnaires triés par total décroissant. Chaque dictionnaire contient les clés "pseudo", "total" et "rang".

Exemple de sortie (premier élément) :

```
1 {"pseudo": "Dan", "total": 780, "rang": 1}
```

Exercice 8 ◆◆

Écrire une fonction `meilleur_manche(scores, j)` qui renvoie l'indice du joueur ayant le meilleur score à la manche `j`.

Partie C — Mutabilité et copies (C5)

Exercice 9 ◆◆

On veut simuler un bonus : ajouter 10 points à chaque score de la manche 1, **sans modifier la grille originale**.

1. Un élève écrit :

```
1 copie = list(scores)
2 for i in range(len(copie)):
3     copie[i][0] += 10
```

- Expliquer pourquoi la grille originale est aussi modifiée.
- Corriger le code pour que la grille originale reste intacte.
 - Vérifier en affichant `scores[0][0]` avant et après.

Partie D — Synthèse (C2, C3, C4)

Exercice 10

Écrire un programme complet qui :

- Affiche le classement complet.
- Identifie le MVP (joueur avec la meilleure moyenne par manche).
- Crée un dictionnaire équipes regroupant les 6 joueurs en 2 équipes de 3 (indices 0–2 et 3–5), avec le total par équipe.

2.1 Les p-uplets (tuples)

DÉFINITION D1

Un **p-uplet** (ou *tuple*) est une séquence ordonnée de p éléments, éventuellement de types différents, séparés par des virgules et entourés de parenthèses. Un p-uplet est **non mutable** : une fois créé, on ne peut ni ajouter, ni retirer, ni modifier ses éléments.

EXEMPLE Le tuple `station = ("Tunis–Carthage", 36.8, 10.2)` contient trois éléments : une chaîne de caractères et deux flottants. On accède à chaque élément par son indice, en commençant à 0 :

```
1 station = ("Tunis-Carthage", 36.8, 10.2)
2 nom = station[0]      # "Tunis-Carthage"
3 latitude = station[1] # 36.8
4 longitude = station[2] # 10.2
```

⚠ CONTRE-EXEMPLE Toute tentative de modification provoque une erreur :

```
>> station[1] = 37.0
TypeError: 'tuple' object does not support item assignment
```

⚠ ERREUR FRÉQUENTE

Modifier un tuple comme une liste (EF1). Un tuple est immuable par conception : si l'on a besoin de modifier des données, il faut utiliser une liste.

DÉFINITION D2

La fonction `len(t)` renvoie le nombre d'éléments d'un tuple `t`. L'opérateur `in` teste l'appartenance d'une valeur à un tuple.

EXEMPLE Vérifier qu'une station est dans une zone :

```

1 coordonnees = (36.8, 10.2)
2 print(len(coordonnees))      # 2
3 print(36.8 in coordonnees)   # True

```

DÉFINITION D3

Une fonction peut renvoyer un tuple pour communiquer *plusieurs résultats* à la fois. On parle de **déballage** (*unpacking*) lorsqu'on affecte chaque élément du tuple à une variable distincte.

CODE DE RÉFÉRENCE — Fonction renvoyant un tuple

```

1 def milieu(a, b):
2     """Renvoie le milieu de deux points (tuples de coordonnées).
3
4     Préconditions : a et b sont des tuples de 2 flottants.
5     """
6     mx = (a[0] + b[0]) / 2
7     my = (a[1] + b[1]) / 2
8     return (mx, my)
9
10 # Déballage du résultat
11 x, y = milieu((1, 2), (5, 8))
12 print(x, y) # 3.0 5.0

```

+ POUR ALLER PLUS LOIN

Un tuple de longueur 1 s'écrit (42,) avec une virgule obligatoire. Sans virgule, (42) est simplement l'entier 42 entre parenthèses. Le tuple vide s'écrit ().

» **Python À la machine.** Ouvre un éditeur Python et définis un tuple *elevé* contenant un nom, un âge et une moyenne. Accède à chaque élément par son indice. Essaie de modifier l'âge : observe l'erreur. Écris une fonction `presenter(eleve)` qui renvoie une chaîne du type "Ali, 16 ans, moyenne 14.5".

2.2 Les tableaux (listes Python)

DÉFINITION D4

Un **tableau** est une séquence ordonnée d'éléments accessibles par leur indice, en commençant à 0. En Python, il est représenté par le type `list`, délimité par des crochets. Un tableau est **mutable** : on peut modifier, ajouter ou supprimer des éléments.

EXEMPLE Créer et manipuler un tableau de relevés de températures :

```

1 releves = [18, 20, 19, 22, 17]
2 print(releves[0])      # 18 (premier élément)
3 print(releves[-1])     # 17 (dernier élément)
4 print(len(releves))    # 5
5 releves[2] = 21        # modification autorisée

```

```
6 print(relevés) # [18, 20, 21, 22, 17]
```

⚠ **CONTRE-EXEMPLE** Accéder à un indice inexistant provoque une erreur :

```
>>> relevés = [18, 20, 19]
>>> relevés[3]
IndexError: list index out of range
```

⚠ ERREUR FRÉQUENTE

Indice hors limites (off-by-one). Un tableau de n éléments a des indices de 0 à $n - 1$. L'indice n n'existe pas.

DÉFINITION D5

Les méthodes principales d'une liste :

- ▶ `t.append(v)` : ajoute v en fin de liste.
- ▶ `t.pop()` : supprime et renvoie le dernier élément.
- ▶ `t.insert(i, v)` : insère v à l'indice i .
- ▶ `v in t` : teste si v est dans t .

CODE DE RÉFÉRENCE — Parcours d'un tableau

```
1 relevés = [18, 20, 19, 22, 17]
2
3 # Parcours par valeur (préféré quand l'indice est inutile)
4 for température in relevés:
5     print(température)
6
7 # Parcours par indice (quand on a besoin de la position)
8 for i in range(len(relevés)):
9     print(f"Relevé {i} : {relevés[i]}")
```

⚠ ERREUR FRÉQUENTE

Parcourir les indices quand les valeurs suffisent (EF2). Écrire `for i in range(len(t)): print(t[i])` quand `for v in t: print(v)` suffit. Le parcours par valeur est plus lisible et moins sujet aux erreurs d'indice.

DÉFINITION D6

Un **tableau en compréhension** se construit en une seule expression : `[expression for variable in itérable]` avec un filtre optionnel `if condition`.

EXEMPLE Construire un tableau des carrés des entiers de 0 à 9 :

```
1 carrés = [x ** 2 for x in range(10)]
2 print(carrés) # [0, 1, 4, 9, 16, 25, 36, 49, 64, 81]
```

```

3
4 # Avec filtre : garder uniquement les carrés pairs
5 pairs = [x ** 2 for x in range(10) if x % 2 == 0]
6 print(pairs) # [0, 4, 16, 36, 64]

```

CODE DE RÉFÉRENCE — Moyenne d'un tableau

```

1 def moyenne(notes):
2     """Renvoie la moyenne d'un tableau de nombres.
3
4     Précondition : notes est un tableau non vide de nombres.
5     Lève ValueError si le tableau est vide.
6     """
7     if len(notes) == 0:
8         raise ValueError("le tableau est vide")
9     return sum(notes) / len(notes)
10
11 print(moyenne([12, 14, 16])) # 14.0

```

» **Python À la machine.** Crée un tableau prenomms contenant cinq prénoms. Ajoute un prénom avec `append`, retire le dernier avec `pop`. Construis en compréhension le tableau des longueurs de chaque prénom. Affiche la longueur moyenne.

2.3 Les grilles : tableaux de tableaux

DÉFINITION D7

Une **grille** est un tableau de tableaux où chaque sous-tableau représente une ligne. On accède à l'élément de la ligne *i* et de la colonne *j* par `grille[i][j]`.

EXEMPLE Une grille de pixels en niveaux de gris (0 = noir, 255 = blanc) :

```

1 image = [
2     [0, 0, 0],
3     [0, 255, 0],
4     [0, 0, 0],
5 ]
6 print(image[1][1]) # 255 (pixel central)

```

	0	1	2
ligne 0	0	0	0
ligne 1	0	255	0
ligne 2	0	0	0

CODE DE RÉFÉRENCE — Parcours d'une grille

```

1 def afficher_grille(grille):
2     """Affiche une grille ligne par ligne."""
3     for ligne in grille:

```



```

4     for valeur in ligne:
5         print(f"{valeur:4d}", end=" ")
6     print()
7
8 def nb_lignes(grille):
9     """Renvoie le nombre de lignes."""
10    return len(grille)
11
12 def nb_colonnes(grille):
13     """Renvoie le nombre de colonnes (suppose grille non vide)."""
14    return len(grille[0])

```

⚠ ERREUR FRÉQUENTE

Confondre lignes et colonnes. Dans `grille[i][j]`, `i` est le numéro de *ligne* et `j` le numéro de *colonne*.

DÉFINITION D8

Pour construire une grille $n \times m$ remplie de zéros, on utilise une **compréhension imbriquée** : `[[0 for j in range(m)] for i in range(n)]`.

⚠ **CONTRE-EXEMPLE** Ne jamais écrire `[[0] * m] * n` : toutes les lignes seraient des *alias* du même objet (voir section 2.8).

```

1 # CORRECT : chaque ligne est un objet distinct
2 grille = [[0 for j in range(3)] for i in range(3)]
3 grille[0][0] = 1
4 print(grille) # [[1, 0, 0], [0, 0, 0], [0, 0, 0]]
5
6 # INCORRECT : alias dangereux
7 mauvaise = [[0] * 3] * 3
8 mauvaise[0][0] = 1
9 print(mauvaise) # [[1, 0, 0], [1, 0, 0], [1, 0, 0]]

```

» **Python À la machine.** Construis une grille 4x4 de zéros en compréhension. Place un 1 sur la diagonale (`grille[i][i] = 1` pour chaque `i`). Affiche le résultat et vérifie que tu obtiens la matrice identité.

2.4 Les p-uplets (tuples)

DÉFINITION D1

Un **p-uplet** (ou *tuple*) est une séquence ordonnée de p éléments, éventuellement de types différents, séparés par des virgules et entourés de parenthèses. Un p-uplet est **non mutable** : une fois créé, on ne peut ni ajouter, ni retirer, ni modifier ses éléments.

EXEMPLE Le tuple `station = ("Tunis–Carthage", 36.8, 10.2)` contient trois éléments : une chaîne de caractères et deux flottants. On accède à chaque élément par son indice, en commençant à 0 :

```

1 station = ("Tunis-Carthage", 36.8, 10.2)
2 nom = station[0]      # "Tunis-Carthage"
3 latitude = station[1] # 36.8
4 longitude = station[2] # 10.2

```

⚠ CONTRE-EXEMPLE Toute tentative de modification provoque une erreur :

```

>>> station[1] = 37.0
TypeError: 'tuple' object does not support item assignment

```

⚠ ERREUR FRÉQUENTE

Modifier un tuple comme une liste (EF1). Un tuple est immuable par conception : si l'on a besoin de modifier des données, il faut utiliser une liste.

DÉFINITION D2

La fonction `len(t)` renvoie le nombre d'éléments d'un tuple `t`. L'opérateur `in` teste l'appartenance d'une valeur à un tuple.

EXEMPLE Vérifier qu'une station est dans une zone :

```

1 coordonnees = (36.8, 10.2)
2 print(len(coordonnees))      # 2
3 print(36.8 in coordonnees)   # True

```

DÉFINITION D3

Une fonction peut renvoyer un tuple pour communiquer *plusieurs résultats* à la fois. On parle de **déballage** (*unpacking*) lorsqu'on affecte chaque élément du tuple à une variable distincte.

CODE DE RÉFÉRENCE — Fonction renvoyant un tuple

```

1 def milieu(a, b):
2     """Renvoie le milieu de deux points (tuples de coordonnées).
3
4     Préconditions : a et b sont des tuples de 2 flottants.
5     """
6     mx = (a[0] + b[0]) / 2
7     my = (a[1] + b[1]) / 2
8     return (mx, my)
9
10 # Déballage du résultat
11 x, y = milieu((1, 2), (5, 8))
12 print(x, y) # 3.0 5.0

```

✚ POUR ALLER PLUS LOIN

Un tuple de longueur 1 s'écrit (42,) avec une virgule obligatoire. Sans virgule, (42) est simplement l'entier 42 entre parenthèses. Le tuple vide s'écrit ().

» **Python À la machine.** Ouvre un éditeur Python et définis un tuple `eleve` contenant un nom, un âge et une moyenne. Accède à chaque élément par son indice. Essaie de modifier l'âge : observe l'erreur. Écris une fonction `presenter(eleve)` qui renvoie une chaîne du type "Ali, 16 ans, moyenne 14.5".

2.5 Les tableaux (listes Python)

DÉFINITION D4

Un **tableau** est une séquence ordonnée d'éléments accessibles par leur indice, en commençant à 0. En Python, il est représenté par le type `list`, délimité par des crochets. Un tableau est **mutable** : on peut modifier, ajouter ou supprimer des éléments.

EXEMPLE Créer et manipuler un tableau de relevés de températures :

```
1 relevés = [18, 20, 19, 22, 17]
2 print(relevés[0])      # 18 (premier élément)
3 print(relevés[-1])     # 17 (dernier élément)
4 print(len(relevés))    # 5
5 relevés[2] = 21        # modification autorisée
6 print(relevés)         # [18, 20, 21, 22, 17]
```

⚠ **CONTRE-EXEMPLE** Accéder à un indice inexistant provoque une erreur :

```
>>> relevés = [18, 20, 19]
>>> relevés[3]
IndexError: list index out of range
```

⚠ ERREUR FRÉQUENTE

Indice hors limites (off-by-one). Un tableau de n éléments a des indices de 0 à $n - 1$. L'indice n n'existe pas.

DÉFINITION D5

Les méthodes principales d'une liste :

- ▶ `t.append(v)` : ajoute v en fin de liste.
- ▶ `t.pop()` : supprime et renvoie le dernier élément.
- ▶ `t.insert(i, v)` : insère v à l'indice i .
- ▶ `v in t` : teste si v est dans t .

CODE DE RÉFÉRENCE — Parcours d'un tableau

```

1 releves = [18, 20, 19, 22, 17]
2
3 # Parcours par valeur (préfééré quand l'indice est inutile)
4 for temperature in releves:
5     print(temperature)
6
7 # Parcours par indice (quand on a besoin de la position)
8 for i in range(len(reLeves)):
9     print(f"Relevé {i} : {reLeves[i]}")

```

⚠ ERREUR FRÉQUENTE

Parcourir les indices quand les valeurs suffisent (EF2). Écrire `for i in range(len(t)): print(t[i])` quand `for v in t: print(v)` suffit. Le parcours par valeur est plus lisible et moins sujet aux erreurs d'indice.

DÉFINITION D6

Un **tableau en compréhension** se construit en une seule expression : `[expression for variable in itérable]` avec un filtre optionnel `if condition`.

EXEMPLE Construire un tableau des carrés des entiers de 0 à 9 :

```

1 carres = [x ** 2 for x in range(10)]
2 print(carres) # [0, 1, 4, 9, 16, 25, 36, 49, 64, 81]
3
4 # Avec filtre : garder uniquement les carrés pairs
5 pairs = [x ** 2 for x in range(10) if x % 2 == 0]
6 print(pairs) # [0, 4, 16, 36, 64]

```

CODE DE RÉFÉRENCE — Moyenne d'un tableau

```

1 def moyenne(notes):
2     """Renvoie la moyenne d'un tableau de nombres.
3
4     Précondition : notes est un tableau non vide de nombres.
5     Lève ValueError si le tableau est vide.
6     """
7     if len(notes) == 0:
8         raise ValueError("le tableau est vide")
9     return sum(notes) / len(notes)
10
11 print(moyenne([12, 14, 16])) # 14.0

```

» **Python À la machine.** Crée un tableau `preNoms` contenant cinq prénoms. Ajoute un prénom avec `append`, retire le dernier avec `pop`. Construis en compréhension le tableau des longueurs de chaque prénom. Affiche la longueur moyenne.

2.6 Les grilles : tableaux de tableaux

DÉFINITION D7

Une **grille** est un tableau de tableaux où chaque sous-tableau représente une ligne. On accède à l'élément de la ligne i et de la colonne j par `grille[i][j]`.

EXEMPLE Une grille de pixels en niveaux de gris (0 = noir, 255 = blanc) :

```
1 image = [
2     [0, 0, 0],
3     [0, 255, 0],
4     [0, 0, 0],
5 ]
6 print(image[1][1]) # 255 (pixel central)
```

	0	1	2
ligne 0	0	0	0
ligne 1	0	255	0
ligne 2	0	0	0

CODE DE RÉFÉRENCE — Parcours d'une grille

```
1 def afficher_grille(grille):
2     """Affiche une grille ligne par ligne."""
3     for ligne in grille:
4         for valeur in ligne:
5             print(f"{valeur:4d}", end=" ")
6         print()
7
8 def nb_lignes(grille):
9     """Renvoie le nombre de lignes."""
10    return len(grille)
11
12 def nb_colonnes(grille):
13     """Renvoie le nombre de colonnes (suppose grille non vide)."""
14    return len(grille[0])
```

⚠ ERREUR FRÉQUENTE

Confondre lignes et colonnes. Dans `grille[i][j]`, i est le numéro de *ligne* et j le numéro de *colonne*.

DÉFINITION D8

Pour construire une grille $n \times m$ remplie de zéros, on utilise une **compréhension imbriquée** : `[[0 for j in range(m)] for i in range(n)]`.

⚠ CONTRE-EXEMPLE Ne jamais écrire `[[0] * m] * n` : toutes les lignes seraient des *alias* du même objet (voir section 2.8).

```

1 # CORRECT : chaque ligne est un objet distinct
2 grille = [[0 for j in range(3)] for i in range(3)]
3 grille[0][0] = 1
4 print(grille) # [[1, 0, 0], [0, 0, 0], [0, 0, 0]]
5
6 # INCORRECT : alias dangereux
7 mauvaise = [[0] * 3] * 3
8 mauvaise[0][0] = 1
9 print(mauvaise) # [[1, 0, 0], [1, 0, 0], [1, 0, 0]]

```

» **Python À la machine.** Construis une grille 4x4 de zéros en compréhension. Place un 1 sur la diagonale (`grille[i][i] = 1` pour chaque `i`). Affiche le résultat et vérifie que tu obtiens la matrice identité.

2.7 Les dictionnaires

DÉFINITION D9

Un **dictionnaire** est une collection non ordonnée de paires *clé : valeur*, délimitée par des accolades. Les clés sont uniques et non mutables (chaîne, entier, tuple). Les valeurs peuvent être de n'importe quel type. Un dictionnaire est **mutable**.

EXEMPLE Un dictionnaire de mesures météorologiques :

```

1 mesures = {"temperature": 21, "vent": 12, "humidite": 65}
2 print(mesures["temperature"]) # 21

```

⚠ CONTRE-EXEMPLE Accéder à une clé inexistante provoque une erreur :

```

>>> mesures["pression"]
KeyError: 'pression'

```

⚠ ERREUR FRÉQUENTE

Accéder à une clé sans vérifier sa présence (EF3). Toujours tester avec `if cle in dico` ou utiliser `dico.get(cle, default)` qui renvoie la valeur par défaut si la clé est absente.

DÉFINITION D10

Opérations principales sur un dictionnaire `d` :

- ▶ `d[cle]` = valeur : ajoute ou modifie une entrée.
- ▶ `del d[cle]` : supprime une entrée.
- ▶ `cle in d` : teste la présence d'une clé.
- ▶ `len(d)` : nombre de paires.
- ▶ `d.get(cle, default)` : accès sécurisé.

CODE DE RÉFÉRENCE — Parcours d'un dictionnaire

```

1 mesures = {"temperature": 21, "vent": 12, "humidite": 65}
2
3 # Parcours des clés
4 for cle in mesures:
5     print(cle, ":", mesures[cle])
6
7 # Parcours des paires clé-valeur (préfééré)
8 for cle, valeur in mesures.items():
9     print(f"{cle} = {valeur}")

```

CODE DE RÉFÉRENCE — Construire un dictionnaire depuis des données

```

1 def stations_chaudes(stations, seuil):
2     """Renvoie les noms des stations dont la température
3     dépasse le seuil.
4
5     Précondition : stations est une liste de dictionnaires
6     avec les clés 'nom' et 'temperature'.
7     """
8     resultat = []
9     for s in stations:
10         if s["temperature"] > seuil:
11             resultat.append(s["nom"])
12     return resultat
13
14 stations = [
15     {"nom": "Nord", "temperature": 18},
16     {"nom": "Sud", "temperature": 24},
17     {"nom": "Est", "temperature": 20},
18 ]
19 print(stations_chaudes(stations, 17)) # ["Nord", "Sud", "Est"]

```

✚ POUR ALLER PLUS LOIN

Un dictionnaire en compréhension s'écrit {cle: valeur for cle, valeur in iterable}. Par exemple, {s["nom"]: s["temperature"] for s in stations} produit {"Nord": 18, "Sud": 24, "Est": 20}.

» **Python À la machine.** Crée un dictionnaire `eleve` contenant les clés "nom", "classe" et "notes" (cette dernière étant un tableau de nombres). Ajoute une clé "moyenne" calculée à partir des notes. Affiche toutes les informations avec une boucle sur `.items()`.

2.8 Mutabilité et références

DÉFINITION D11

Lorsqu'on écrit `b = a` pour un objet mutable, `b` ne contient pas une copie de `a` : les deux variables désignent **le même objet** en mémoire. On dit que `b` est un **alias** de `a`. Toute modification via l'une des variables est visible via l'autre.

EXEMPLE L'affectation crée un alias, pas une copie :

```
1 notes = [12, 15, 9]
2 copie = notes      # alias : même objet
3 copie.append(18)
4 print(notes)       # [12, 15, 9, 18]
5 print(len(copie))  # 4
```



⚠ ERREUR FRÉQUENTE

Croire que `copie = notes` crée une copie indépendante. C'est l'erreur la plus fréquente en NSI sur les types mutables.

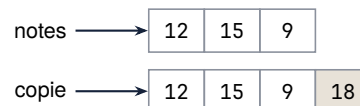
DÉFINITION D12

Pour obtenir une **copie indépendante** d'une liste, on utilise :

- ▶ `copie = list(original)` ou `copie = original[:]` (copie *superficielle*).
- ▶ `copie = [list(ligne) for ligne in original]` pour une grille (copie de chaque sous-liste).

EXEMPLE Copie indépendante :

```
1 notes = [12, 15, 9]
2 copie = list(notes) # copie indépendante
3 copie.append(18)
4 print(notes)       # [12, 15, 9] (inchangé)
5 print(copie)       # [12, 15, 9, 18]
```



⚠ CONTRE-EXEMPLE Copie superficielle insuffisante pour les listes imbriquées :

```
1 grille = [[1, 2], [3, 4]]
2 copie = list(grille) # copie superficielle
3 copie[0][0] = 99
4 print(grille)       # [[99, 2], [3, 4]]; modifié !
```

⚠ ERREUR FRÉQUENTE

Présenter `[list(ligne) for ligne in grille]` comme une copie profonde générale. Cette compréhension réalise seulement une **copie des deux premiers niveaux** : la liste externe et chaque ligne. Son contrat est le suivant : les **cellules sont des valeurs scalaires atomiques non mutables**, par exemple de type `int`, `float`, `bool` ou `str` ; les conteneurs imbriqués sont exclus. Elle ne constitue pas une copie profonde générale.

CODE DE RÉFÉRENCE — Copier une grille sous un contrat à deux niveaux

```

1 def copier_grille_deux_niveaux(grille):
2     """Copie la liste externe et chaque ligne.
3
4     Precondition : les cellules sont des valeurs scalaires atomiques non mutables,
5     sans conteneur imbriqué.
6     """
7     return [list(ligne) for ligne in grille]

```

◆ **PROPRIÉTÉ Deux niveaux distincts** Pour la grille numérique `grille = [[1, 2], [3, 4]]`, la fonction crée une nouvelle liste externe et une nouvelle liste pour chaque ligne. La copie a les mêmes valeurs, mais ses lignes sont des objets distincts : réaffecter `copie[0][0] = 99` ne modifie pas `grille`, qui reste égale à `[[1, 2], [3, 4]]`.

⚠ **CONTRE-EXEMPLE** Hors contrat, avec `[[[1]]]`, la cellule est elle-même une liste mutable. La fonction copie la liste externe et sa ligne, mais cette cellule-liste reste partagée : lui ajouter 2 par la copie modifie aussi la grille d'origine, qui devient `[[[1, 2]]]`.

CODE DE RÉFÉRENCE — Effet de bord dans une fonction

```

1 def ajouter_note(bulletin, note):
2     """Ajoute une note au bulletin.
3
4     ATTENTION : modifie le bulletin passé en argument
5     (effet de bord).
6     """
7     bulletin.append(note)
8
9 notes_ali = [12, 15]
10 ajouter_note(notes_ali, 18)
11 print(notes_ali) # [12, 15, 18]

```

✦ POUR ALLER PLUS LOIN

L'opérateur `is` teste si deux variables désignent le *même objet* (même adresse mémoire), tandis que `==` teste l'égalité des *valeurs*. Après `b = a`, on a `b is a` qui vaut `True`. Après `b = list(a)`, on a `b is a` qui vaut `False` mais `b == a` qui vaut `True`.

» **Python À la machine.** Crée une liste `original = [1, 2, 3]`. Fais un alias `alias = original` et une copie `copie = list(original)`. Modifie `alias[0] = 99`. Affiche les trois variables et vérifie : `original` est modifié, `copie` ne l'est pas. Teste `alias is original` et `copie is original`.

🔗 MÉTHODE M1 Choisir entre tuple et liste

Quand l'utiliser ? Quand tu dois stocker plusieurs valeurs ensemble et que tu hésites entre un tuple et une liste.

Pas à pas :

1. Demande-toi si les données seront modifiées après création (ajout, suppression, remplacement).
2. Si **non** (données fixes) : utilise un **tuple**. Exemples : coordonnées d'un point, date de naissance, résultat renvoyé par une fonction.
3. Si **oui** (données évolutives) : utilise une **liste**. Exemples : liste de scores en cours de partie, panier d'achats.
4. En cas de doute, pose-toi la question : « cette collection va-t-elle grandir ou changer ? »

Exemple :

```

1 # Donnees fixes : on choisit un tuple
2 position = (3, 7)
3 couleur_rgb = (255, 128, 0)
4
5 # Donnees evolutives : on choisit une liste
6 scores = [12, 8, 15]
7 scores.append(20) # on ajoute un score

```

Pièges :

- Utiliser une liste pour des données qui ne changent jamais : cela fonctionne, mais un tuple exprime mieux l'intention et protège contre les modifications accidentelles.
- Tenter de modifier un tuple avec `t[0] = 5` provoque une `TypeError`.

Vérifier : ton choix est cohérent si tu n'as jamais besoin de `append`, `remove` ou d'affectation par indice sur un tuple.

S'entraîner : exercices C1.

① MÉTHODE M2 Construire un tableau en compréhension

Quand l'utiliser ? Quand tu veux créer une liste à partir d'une règle de calcul ou d'un filtre, en une seule ligne.

Pas à pas :

1. Identifie la **source** : sur quoi itères-tu ? (un range, une liste existante, etc.)
2. Identifie l'**expression** : que calcules-tu pour chaque élément ?
3. (Optionnel) Identifie la **condition** : quels éléments gardes-tu ?
4. Écris la compréhension selon le patron : `[expr for var in source if cond]`.

Exemple :

```

1 # Carres des entiers de 0 à 9
2 carres = [x**2 for x in range(10)]
3 # [0, 1, 4, 9, 16, 25, 36, 49, 64, 81]
4
5 # Garder uniquement les nombres pairs d'une liste
6 nombres = [3, 8, 12, 5, 20, 7]
7 pairs = [n for n in nombres if n % 2 == 0]
8 # [8, 12, 20]

```

Pièges :

- Oublier les crochets `[...]` : sans eux, tu obtiens un générateur, pas une liste.
- Placer le `if` avant le `for` : la condition de filtrage se place *après* le `for`.
- Écrire une compréhension trop complexe : si tu as besoin de plus de deux lignes de logique, préfère une boucle classique.

Vérifier : affiche la liste obtenue avec `print` et contrôle sa longueur avec `len`.

S'entraîner : exercices C2.

🔗 MÉTHODE M3 Parcourir une grille ligne par ligne

Quand l'utiliser ? Quand tu travailles avec un tableau de tableaux (grille, image, plateau de jeu) et que tu dois accéder à chaque case.

Pas à pas :

1. Repère le nombre de lignes : `nb_lignes = len(grille)`.
2. Repère le nombre de colonnes : `nb_colonnes = len(grille[0])`.
3. Écris une première boucle `for i in range(nb_lignes)` pour parcourir les lignes.
4. Écris une seconde boucle imbriquée `for j in range(nb_colonnes)` pour parcourir les colonnes.
5. Accède à la case courante avec `grille[i][j]`.

Exemple :

```
1 grille = [
2     [1, 2, 3],
3     [4, 5, 6],
4 ]
5
6 for i in range(len(grille)):
7     for j in range(len(grille[0])):
8         print(grille[i][j], end=" ")
9     print() # retour a la ligne apres chaque ligne
10 # Affiche :
11 # 1 2 3
12 # 4 5 6
```

Pièges :

- Confondre lignes et colonnes : `grille[i]` est la ligne `i` ; `grille[i][j]` est la case à la ligne `i`, colonne `j`.
- Inverser les indices : `grille[j][i]` au lieu de `grille[i][j]` donne un résultat faux (et parfois une erreur d'indice).
- Supposer que toutes les lignes ont la même longueur sans le vérifier.

Vérifier : affiche `i, j, grille[i][j]` dans la boucle pour contrôler que chaque case est bien visitée.

S'entraîner : exercices C3.

🔗 MÉTHODE M4 Chercher dans un dictionnaire sans erreur

Quand l'utiliser ? Quand tu veux accéder à une valeur dans un dictionnaire sans risquer une `KeyError` si la clé est absente.

Pas à pas :

1. **Méthode 1 — tester avec `in`** : vérifie si la clé existe avant d'y accéder.
2. **Méthode 2 — utiliser `.get()`** : appelle `dico.get(cle, valeur_par_defaut)` qui renvoie la valeur par défaut si la clé est absente (au lieu de provoquer une erreur).
3. Choisis la méthode adaptée : `in` quand tu dois exécuter un bloc particulier selon la présence de la clé ; `.get()` quand tu veux simplement récupérer une valeur avec un `repli`.

Exemple :

```
1 inventaire = {"pommes": 5, "bananes": 3}
2
3 # Methode 1 : tester avec in
4 if "cerises" in inventaire:
5     print(inventaire["cerises"])
6 else:
7     print("Pas de cerises en stock")
```

```

8
9 # Methode 2 : utiliser .get()
10 qte = inventaire.get("cerises", 0)
11 print(qte) # 0 (valeur par défaut)

```

Pièges :

- ▶ Accéder directement avec dico[cle] sans vérification : si la clé est absente, Python lève une KeyError.
- ▶ Oublier le second argument de .get() : sans valeur par défaut, .get() renvoie None (pas une erreur, mais potentiellement inattendu).
- ▶ Confondre in sur un dictionnaire (cherche parmi les *clés*) et in sur une liste (cherche parmi les *éléments*).

Vérifier : teste ton code avec une clé présente *et* une clé absente pour confirmer qu'aucune erreur ne se produit.

S'entraîner : exercices C4.

④ MÉTHODE M5 Copier une liste sans alias

Quand l'utiliser ? Quand tu veux créer une copie indépendante d'une liste, de sorte que modifier la copie ne modifie pas l'original.

Pas à pas :

1. Identifie le problème : copie = originale ne crée **pas** une copie, mais un **alias** (les deux noms désignent le même objet en mémoire).
2. Pour obtenir une vraie copie, utilise l'une de ces méthodes :
 - ▶ copie = list(originale)
 - ▶ copie = originale[:] (tranche complète)
3. Vérifie que les deux listes sont bien distinctes en modifiant la copie et en affichant l'original.

Exemple :

```

1 originale = [1, 2, 3]
2
3 # Alias : les deux variables pointent vers le meme objet
4 alias = originale
5 alias.append(4)
6 print(originale) # [1, 2, 3, 4] -- modifiée aussi !
7
8 # Copie indépendante
9 originale = [1, 2, 3]
10 copie = list(originale)
11 copie.append(4)
12 print(originale) # [1, 2, 3] -- inchangée
13 print(copie)    # [1, 2, 3, 4]

```

Pièges :

- ▶ Croire que = copie une liste : c'est l'erreur la plus fréquente. L'affectation crée un alias, pas une copie.
- ▶ Attention aux listes imbriquées : list() et [:] ne font qu'une copie *superficielle*. Si la liste contient d'autres listes, les sous-listes restent partagées.

Vérifier : après la copie, teste copie is originale. Si le résultat est False, tu as bien deux objets distincts.

S'entraîner : exercices C5.

Exercice 11 ◆

On considère le programme suivant :

```
1 notes = [12, 15, 9]
2 copie = notes
3 copie.append(18)
4 print(notes)
5 print(len(copie))
```

1. Donner, sans exécuter le programme, ce qu'affichent les deux instructions print.
2. Expliquer pourquoi la liste notes est modifiée alors que l'on n'a appelé append que sur copie.
3. Modifier la ligne 2 pour que notes ne soit pas modifiée.

Exercice 12 ◆

On donne le tuple suivant :

```
1 station = ("Tunis", 36.8, 10.2)
```

1. Donner le type de station et le nombre de ses éléments.
2. Donner la valeur de station[0] et de station[2].
3. Peut-on écrire station[1] = 37.0 ? Justifier.

Exercice 13 ◆

Écrire une fonction distance(a, b) qui prend deux tuples de coordonnées (x, y) et renvoie la distance euclidienne entre les deux points.

Exemples :

```
»> distance((0, 0), (3, 4))
5.0
»> distance((1, 1), (1, 1))
0.0
```

Exercice 14 ◆

On donne le tuple suivant, qui représente un joueur d'e-sport :

```
1 joueur = ("Fatou", 2350, "diamant")
2 nom, elo, rang = joueur
3 print(nom)
4 print(elo > 2000)
5 print(joueur[2])
```

1. Donner, sans exécuter le programme, ce qu'affichent les trois instructions print.
2. Quel mécanisme Python permet d'écrire nom, elo, rang = joueur ?
3. Peut-on écrire joueur[1] = 2400 ? Justifier.

Exercice 15 ◆

On donne la liste de tuples suivante, contenant des relevés de température :

```
1 releves = [("Paris", 32.5), ("Lyon", 35.1), ("Lille", 28.7)]
```

```
2 for ville, temp in releves:
3     if temp > 30:
4         print(ville)
```

1. Donner, sans exécuter le programme, ce qu'affiche ce code.
2. Quel est le type de chaque élément de la liste releves ?
3. Donner la valeur de releves[1][0].

Exercice 16

Écrire une fonction `coordonnees_milieu(a, b)` qui prend deux tuples représentant des coordonnées (x, y) et renvoie le tuple des coordonnées du milieu du segment [AB].

Exemples :

```
>> coordonnees_milieu((0, 0), (4, 6))
(2.0, 3.0)
>> coordonnees_milieu((-2, 3), (2, -3))
(0.0, 0.0)
```

Exercice 17

On modélise un match d'e-sport par un tuple (equipe1, equipe2, score1, score2).

```
1 match = ("TeamA", "TeamB", 2, 1)
2 equipe1, equipe2, score1, score2 = match
3 resultat = equipe1 if score1 > score2 else equipe2
4 print(resultat)
5 print(score1 + score2)
```

1. Donner, sans exécuter le programme, ce qu'affichent les deux print.
2. Que contient la variable resultat si le match est ("X", "Y", 0, 3) ?

Exercice 18

On représente une station météo par un tuple (nom, temperature).

Écrire une fonction `station_la_plus_chaude(stations)` qui prend une liste non vide de tuples et renvoie le nom de la station ayant la température la plus élevée.

Exemple :

```
>> station_la_plus_chaude([("Paris", 32), ("Lyon", 35), ("Lille", 28)])
'Lyon'
```

Exercice 19

On représente chaque joueur d'e-sport par un tuple (nom, elo, rang).

1. Écrire une fonction `filtrer_par_rang(joueurs, rang_min)` qui prend une liste de tuples et un entier rang_min, et renvoie la liste des tuples dont le score Elo est supérieur ou égal à rang_min.
2. Tester ta fonction avec la liste suivante :

```
1 joueurs = [("Alice", 2400, "diamant"),
2            ("Bob", 1800, "or"),
3            ("Clara", 2100, "platine")]
```

Exercice 20 ♦♦

On considère le programme suivant :

```

1 def swap(t):
2     return (t[1], t[0])
3
4 a = (3, 7)
5 b = swap(a)
6 print(a)
7 print(b)
8 c = (1, 2, 3)
9 print(swap(c))

```

1. Donner, sans exécuter, les trois affichages produits.
2. Le tuple a est-il modifié par l'appel swap(a) ? Justifier.
3. Que se passe-t-il si on appelle swap(c) avec c = (1, 2, 3) ? Le troisième élément est-il perdu ?

Exercice 21 ♦♦

On modélise les résultats d'un tournoi par une liste de tuples (equipe, score_pour, score_contre).

Écrire une fonction classer_matches(matches) qui renvoie un tuple de trois listes : les équipes victorieuses, les équipes perdantes et les matchs nuls.

Exemple :

```

»> classer_matches([("A", 3, 1), ("B", 0, 2), ("C", 1, 1)])
(['A'], ['B'], ['C'])

```

Exercice 22 ♦♦♦

On dispose d'une liste de tuples (x, y) représentant les positions de stations météo sur une carte.

Écrire une fonction enveloppe_convexe_1d(points) qui renvoie un tuple de deux tuples : le coin inférieur gauche (x_min, y_min) et le coin supérieur droit (x_max, y_max) du rectangle englobant toutes les stations.

Exemple :

```

»> enveloppe_convexe_1d([(1, 2), (3, 0), (-1, 5)])
((-1, 0), (3, 5))

```

Indication : parcourir tous les points pour trouver les extremums.

Exercice 23 ♦♦♦

On représente un classement e-sport par une liste de tuples (nom, elo).

Écrire une fonction top_joueurs(joueurs, n) qui renvoie un tuple contenant les noms des n meilleurs joueurs, classés par score Elo décroissant.

Tu peux utiliser un tri par sélection ou tout autre algorithme de tri vu en cours.

Exemple :

```

»> joueurs = [("Alice", 2400), ("Bob", 1800),
...           ("Clara", 2600), ("Dan", 2100)]
»> top_joueurs(joueurs, 2)
('Clara', 'Alice')

```

Exercice 24

On donne le tableau suivant :

```
1 temperatures = [15, 22, 18, 30, 25]
2 print(temperatures[0])
3 print(temperatures[-1])
4 print(len(temperatures))
5 temperatures[2] = 20
6 print(temperatures)
```

1. Donner, sans exécuter le programme, ce qu'affichent les quatre print.
2. Pourquoi peut-on écrire `temperatures[2] = 20` alors qu'on ne pourrait pas le faire avec un tuple ?

Exercice 25

On considère le programme suivant :

```
1 scores = [12, 8, 15, 3, 20]
2 resultat = []
3 for s in scores:
4     if s ≥ 10:
5         resultat.append(s)
6 print(resultat)
7 print(len(resultat))
```

1. Donner, sans exécuter le programme, les deux affichages produits.
2. Quel est le rôle de ce programme ?

Exercice 26

Écrire une fonction `moyenne(tab)` qui prend un tableau non vide d'entiers et renvoie la moyenne de ses éléments.

Exemples :

```
>>> moyenne([10, 20, 30])
20.0
>>> moyenne([12, 8, 15, 3, 20])
11.6
```

Exercice 27

On considère le programme suivant :

```
1 notes = [12, 8, 15, 6, 14, 9]
2 tab = [n for n in notes if n ≥ 10]
3 print(tab)
4 doubles = [n * 2 for n in [1, 2, 3]]
5 print(doubles)
```

1. Donner, sans exécuter, les deux affichages.
2. Réécrire la construction de `tab` en utilisant une boucle `for` et `append` (sans compréhension de liste).

Exercice 28 ♦♦

Écrire une fonction `indice_maximum(tab)` qui prend un tableau non vide d'entiers et renvoie l'indice de son plus grand élément. En cas d'égalité, on renvoie le plus petit indice.

Exemples :

```

>>> indice_maximum([3, 7, 2, 9, 1])
3
>>> indice_maximum([5, 5, 5])
0

```

Exercice 29 ♦♦

On considère la fonction suivante :

```

1 def mystere(tab):
2     resultat = []
3     for i in range(len(tab)):
4         if tab[i] not in resultat:
5             resultat.append(tab[i])
6     return resultat
7 print(mystere([3, 1, 3, 2, 1, 3]))

```

1. Donner, sans exécuter, l'affichage produit.
2. Décrire en une phrase le rôle de cette fonction.
3. Réécrire le corps de la fonction en parcourant directement les éléments (sans utiliser les indices).

Exercice 30 ♦♦

Une station météo enregistre des températures horaires dans un tableau. On veut détecter les valeurs anormales.

Écrire une fonction `temperatures_anormales(relevés, seuil_bas, seuil_haut)` qui renvoie la liste des tuples (indice, valeur) pour chaque relevé en dehors de l'intervalle `[seuil_bas, seuil_haut]`.

Exemple :

```

>>> temperatures_anormales([15, 42, 18, -3, 22], 0, 40)
[(1, 42), (3, -3)]

```

Exercice 31 ♦♦

On considère la fonction suivante :

```

1 def decaler(tab):
2     dernier = tab[-1]
3     for i in range(len(tab) - 1, 0, -1):
4         tab[i] = tab[i - 1]
5     tab[0] = dernier
6
7 scores = [10, 20, 30, 40]
8 decaler(scores)
9 print(scores)

```

1. Donner, sans exécuter, l'affichage produit. Détailler l'état du tableau après chaque itération de la boucle.
2. Décrire en une phrase le rôle de cette fonction.

3. La fonction renvoie-t-elle une valeur ? Comment le tableau est-il modifié ?

Exercice 32

Compléter la trace du programme suivant, qui affiche les élèves ayant la moyenne :

```
1 eleves = ["Alice", "Bob", "Clara"]
2 notes = [14, 8, 17]
3 for i in range(len(eleves)):
4     if notes[i] ≥ 10:
5         print(eleves[i], ":", notes[i])
```

1. Donner les affichages produits.
2. Quel est le lien entre les deux tableaux `eleves` et `notes` ?
3. Pourquoi utilise-t-on `range(len(eleves))` plutôt que `for e in eleves` ?

Exercice 33

On dispose de deux tableaux d'entiers, chacun trié par ordre croissant.

Écrire une fonction `fusionner(tab1, tab2)` qui renvoie un nouveau tableau contenant tous les éléments des deux tableaux, trié par ordre croissant, sans utiliser de fonction de tri.

Exemple :

```
>> fusionner([1, 3, 5], [2, 4, 6])
[1, 2, 3, 4, 5, 6]
```

Indication : utiliser deux indices parcourant chacun des tableaux.

Exercice 34

En météorologie, on lisse les relevés de température en calculant des **moyennes glissantes** : la valeur en position i est remplacée par la moyenne des k valeurs consécutives commençant en i .

Écrire une fonction `moyennes_glissantes(relevés, k)` qui renvoie le tableau des moyennes glissantes de taille k . Ce tableau contient $n - k + 1$ éléments (où n est la longueur du tableau).

Exemple :

```
>> moyennes_glissantes([10, 20, 30, 40, 50], 3)
[20.0, 30.0, 40.0]
```

Exercice 35

On donne la grille suivante :

```
1 grille = [[1, 2, 3],
2           [4, 5, 6]]
3 print(grille[0][1])
4 print(grille[1][2])
5 print(len(grille))
6 print(len(grille[0]))
```

1. Donner, sans exécuter, les quatre affichages.
2. Combien de lignes et de colonnes possède cette grille ?
3. Donner l'expression permettant d'accéder à la valeur 4.

Exercice 36

Une grille représente les températures relevées en différents points d'une carte :

```
1 carte = [[20, 22, 19],
2           [25, 28, 23],
3           [18, 17, 21]]
4 total = 0
5 for ligne in carte:
6     for val in ligne:
7         total = total + val
8 print(total)
```

1. Donner, sans exécuter, l'affichage produit.
2. Modifier le code pour qu'il affiche la moyenne des températures de la grille.

Exercice 37

Écrire une fonction `creer_grille(n, p, valeur)` qui renvoie une grille de n lignes et p colonnes, où chaque case contient `valeur`.

Exemple :

```
>>> creer_grille(2, 3, 0)
[[0, 0, 0], [0, 0, 0]]
```

Attention : ne pas écrire `[[valeur] * p] * n`. Pourquoi ?

Exercice 38

On considère le programme suivant :

```
1 grille = [[0, 0, 0],
2           [0, 0, 0],
3           [0, 0, 0]]
4 for i in range(3):
5     grille[i][i] = 1
6 for ligne in grille:
7     print(ligne)
```

1. Donner, sans exécuter, les trois affichages produits.
2. Quel objet mathématique cette grille représente-t-elle ?

Exercice 39

On représente une carte de températures par une grille (liste de listes).

Écrire une fonction `maximum_grille(grille)` qui renvoie un tuple (ligne, colonne, valeur) correspondant à la position et la valeur du maximum dans la grille.

Exemple :

```
>>> maximum_grille([[1, 5], [9, 3]])
(1, 0, 9)
```

Exercice 40

Écrire une fonction `transposer(grille)` qui renvoie la transposée d'une grille : les lignes deviennent les colonnes et réciproquement.

Exemple :

```
>> transposer([[1, 2, 3], [4, 5, 6]])
[[1, 4], [2, 5], [3, 6]]
```

La grille d'entrée ne doit pas être modifiée.

Exercice 41

Dans une grille binaire (contenant des 0 et des 1), on veut compter les voisins d'une case.

Écrire une fonction `compter_voisins(grille, lig, col)` qui renvoie le nombre de cases voisines (horizontales, verticales et diagonales) contenant la valeur 1, sans compter la case elle-même.

Exemple :

```
>> g = [[0, 1, 0], [1, 1, 0], [0, 0, 1]]
>> compter_voisins(g, 1, 1)
3
>> compter_voisins(g, 0, 0)
3
```

Attention : gérer les bords de la grille.

Exercice 42

Trois équipes d'e-sport ont chacune disputé trois matchs. Leurs scores sont enregistrés dans la grille suivante :

```
1 scores = [[10, 8, 12],
2           [15, 11, 9],
3           [7, 14, 13]]
4 moyennes = []
5 for ligne in scores:
6     s = 0
7     for v in ligne:
8         s = s + v
9     moyennes.append(s / len(ligne))
10 print(moyennes)
```

1. Donner, sans exécuter, l'affichage produit.
2. Modifier le code pour n'afficher que la moyenne la plus élevée.

Exercice 43

On dispose d'une carte de températures sous forme de grille. On veut créer une **carte binaire** identifiant les zones de chaleur.

1. Écrire une fonction `zone_chaude(carte, seuil)` qui renvoie une nouvelle grille de même taille contenant 1 si la température est supérieure ou égale au seuil, et 0 sinon.
2. Tester avec la carte :

```
1 carte = [[30, 25, 32],
2           [28, 35, 31],
3           [22, 19, 27]]
```

et un seuil de 30.

3. Proposer une fonction qui compte le nombre de cases à 1 dans la grille résultat.

Exercice 44 ◆◆◆

On souhaite effectuer une rotation de 90 degrés dans le sens horaire d'une grille.

1. Sur papier, dessiner la grille `[[1, 2], [3, 4]]` puis sa version après rotation de 90 degrés.
2. Écrire une fonction `rotation_90(grille)` qui renvoie une nouvelle grille correspondant à la rotation. Si la grille d'entrée a `n` lignes et `p` colonnes, la grille résultat a `p` lignes et `n` colonnes.
3. Vérifier qu'appliquer quatre fois la rotation redonne la grille initiale.

Exemple :

```
>>> rotation_90([[1, 2], [3, 4]])
[[3, 1], [4, 2]]
```

Exercice 45 ◆

On donne le dictionnaire suivant :

```
1 eleve = {"nom": "Alice", "classe": "1NSI", "moyenne": 14.5}
2 print(eleve["nom"])
3 print(eleve["moyenne"])
4 print(len(eleve))
```

1. Donner, sans exécuter, les trois affichages.
2. Quelles sont les clés de ce dictionnaire ? Quelles sont les valeurs ?
3. Écrire l'instruction qui ajoute la clé "email" avec la valeur "alice@lycee.fr".

Exercice 46 ◆

On considère le programme suivant :

```
1 meteo = {"ville": "Lyon", "temp": 28, "vent": 15}
2 meteo["temp"] = 30
3 meteo["humidite"] = 65
4 print(meteo["temp"])
5 print("humidite" in meteo)
6 print("pluie" in meteo)
```

1. Donner, sans exécuter, les trois affichages.
2. Quelle est la différence entre modifier une clé existante et ajouter une nouvelle clé ?

Exercice 47 ◆

Écrire une fonction `creer_joueur(nom, elo, rang)` qui renvoie un dictionnaire avec les clés "nom", "elo" et "rang".

Exemple :

```
>>> creer_joueur("Alice", 2400, "diamant")
{'nom': 'Alice', 'elo': 2400, 'rang': 'diamant'}
```

Exercice 48 ◆

On considère le programme suivant :

```
1 station = {"nom": "Paris", "temp": 25, "vent": 10}
2 for cle in station:
3     print(cle, ":", station[cle])
```

1. Donner, sans exécuter, les affichages produits.
2. Que parcourt la boucle `for cle in station` : les clés, les valeurs, ou les deux ?
3. Réécrire la boucle en utilisant `station.items()`.

Exercice 49

Écrire une fonction `compter_occurrences(tab)` qui prend un tableau et renvoie un dictionnaire associant à chaque élément son nombre d'apparitions.

Exemple :

```
>> compter_occurrences(["a", "b", "a", "c", "b", "a"])
{'a': 3, 'b': 2, 'c': 1}
```

Exercice 50

On considère la fonction suivante :

```
1 def fusionner_dicos(d1, d2):
2     resultat = {}
3     for cle in d1:
4         resultat[cle] = d1[cle]
5     for cle in d2:
6         resultat[cle] = d2[cle]
7     return resultat
8
9 a = {"x": 1, "y": 2}
10 b = {"y": 10, "z": 3}
11 print(fusionner_dicos(a, b))
```

1. Donner, sans exécuter, l'affichage produit.
2. Que se passe-t-il lorsqu'une clé est présente dans les deux dictionnaires ?
3. Les dictionnaires `a` et `b` sont-ils modifiés ?

Exercice 51

On dispose d'une liste de dictionnaires représentant des élèves, chacun ayant les clés "nom", "classe" et "note".

Écrire une fonction `moyenne_par_classe(elevs)` qui renvoie un dictionnaire associant à chaque classe la moyenne des notes de ses élèves.

Exemple :

```
>> elevs = [{"nom": "A", "classe": "1A", "note": 12},
...         {"nom": "B", "classe": "1B", "note": 15},
...         {"nom": "C", "classe": "1A", "note": 14}]
>> moyenne_par_classe(elevs)
{'1A': 13.0, '1B': 15.0}
```

Exercice 52

On considère le code suivant :

```
1 joueur = {"nom": "Bob", "elo": 1800}
2 print(joueur["rang"])
```

1. Quelle erreur ce programme produit-il ? Pourquoi ?
2. Proposer deux façons différentes d'éviter cette erreur :
 - ▶ en testant l'existence de la clé avant l'accès ;
 - ▶ en utilisant la méthode get.
3. Réécrire le code pour qu'il affiche "inconnu" si la clé "rang" n'existe pas.

Exercice 53 ♦♦

Écrire une fonction `inverser_dico(d)` qui prend un dictionnaire et renvoie un nouveau dictionnaire où les clés et les valeurs sont échangées.

On suppose que toutes les valeurs du dictionnaire d'entrée sont distinctes et peuvent servir de clés.

Exemple :

```
>>> inverser_dico({"a": 1, "b": 2, "c": 3})
{1: 'a', 2: 'b', 3: 'c'}
```

Exercice 54 ♦♦♦

On modélise un tournoi d'e-sport par une liste de tuples (`equipe1, score1, equipe2, score2`).

Écrire une fonction `classement_tournoi(matches)` qui renvoie un dictionnaire associant à chaque équipe son total de points : 3 points pour une victoire, 1 point pour un match nul, 0 pour une défaite.

Exemple :

```
>>> matches = [("A", 2, "B", 1), ("A", 0, "C", 0), ("B", 3, "C", 1)]
>>> classement_tournoi(matches)
{'A': 4, 'B': 3, 'C': 1}
```

Exercice 55 ♦♦♦

On dispose d'une liste de dictionnaires représentant des stations météo, chacun ayant les clés "nom", "region" et "temp".

1. Écrire une fonction `stations_par_region(stations)` qui renvoie un dictionnaire dont les clés sont les régions et les valeurs sont les listes des noms de stations dans chaque région.
2. Compléter par une fonction `region_la_plus_chaude(stations)` qui renvoie le nom de la région ayant la température moyenne la plus élevée.

Exemple :

```
>>> stations = [{"nom": "Lyon", "region": "ARA", "temp": 30},
...             {"nom": "Paris", "region": "IDF", "temp": 28},
...             {"nom": "Grenoble", "region": "ARA", "temp": 27}]
>>> stations_par_region(stations)
{'ARA': ['Lyon', 'Grenoble'], 'IDF': ['Paris']}
```

Exercice 56 ♦

On considère le programme suivant :

```
1 a = [1, 2, 3]
```

```
2 b = a
3 b.append(4)
4 print(a)
5 print(a is b)
```

1. Donner, sans exécuter, les deux affichages.
2. Pourquoi la liste a est-elle modifiée alors qu'on n'a appelé append que sur b ?
3. Que signifie a is b ?

Exercice 57

On considère le programme suivant :

```
1 a = [1, 2, 3]
2 b = list(a)
3 b.append(4)
4 print(a)
5 print(b)
6 print(a is b)
```

1. Donner, sans exécuter, les trois affichages.
2. Quelle est la différence avec b = a ?
3. Citer deux autres façons de créer une copie superficielle d'une liste.

Exercice 58

On considère le programme suivant :

```
1 def ajouter(tab, val):
2     tab.append(val)
3
4 scores = [10, 20]
5 ajouter(scores, 30)
6 print(scores)
```

1. Donner, sans exécuter, l'affichage produit.
2. La fonction ajouter ne contient pas de return. Comment se fait-il que la liste scores soit modifiée ?
3. On parle d'**effet de bord**. Expliquer ce terme.

Exercice 59

On considère le programme suivant :

```
1 t = (1, 2, 3)
2 t[0] = 10
3 print(t)
```

1. Que se passe-t-il lorsqu'on exécute ce programme ? Quelle erreur obtient-on ?
2. Expliquer pourquoi on dit qu'un tuple est **immuable** (non mutable).
3. Donner un avantage d'utiliser un tuple plutôt qu'une liste quand on ne souhaite pas modifier les données.

Exercice 60 ♦♦

On considère le programme suivant :

```
1 grille = [[0, 0]] * 3
2 grille[0][0] = 1
3 for ligne in grille:
4     print(ligne)
```

1. Donner, sans exécuter, les trois affichages. Le résultat est-il celui attendu ?
2. Expliquer pourquoi toutes les lignes sont modifiées alors qu'on n'a changé que grille[0][0].
3. Proposer une construction correcte de la grille qui évite ce piège.

Exercice 61 ♦♦

Un élève écrit le code suivant pour supprimer les valeurs négatives d'un tableau :

```
1 def supprimer_negatifs_v1(tab):
2     for val in tab:
3         if val < 0:
4             tab.remove(val)
5     return tab
```

1. Tester mentalement supprimer_negatifs_v1([5, -2, -3, 8]). Le résultat est-il correct ? Expliquer le problème.
2. Voici une version corrigée :

```
1 def supprimer_negatifs(tab):
2     resultat = []
3     for val in tab:
4         if val ≥ 0:
5             resultat.append(val)
6     return resultat
```

Pourquoi cette version est-elle plus fiable ? Le tableau d'origine est-il modifié ?

Exercice 62 ♦♦

On considère le programme suivant :

```
1 equipe = {"nom": "Phoenix", "score": 100}
2 copie = equipe
3 copie["score"] = 200
4 print(equipe["score"])
5 print(equipe is copie)
```

1. Donner, sans exécuter, les deux affichages.
2. Les dictionnaires sont-ils mutables ? Justifier.
3. Réécrire le code pour que la modification de copie ne modifie pas equipe. On pourra utiliser dict(equipe).

Exercice 63 ♦♦

On considère le programme suivant :

```
1 grille = [[1, 2], [3, 4]]
2 copie = list(grille)
3 copie[0][0] = 99
4 print(grille[0][0])
5 copie[0] = [10, 20]
6 print(grille[0])
```

1. Donner, sans exécuter, les deux affichages.
2. Expliquer pourquoi grille[0][0] vaut 99 alors qu'on a modifié copie.
3. On parle de **copie superficielle**. Qu'est-ce que cela signifie ?
4. On suppose désormais que les cellules sont des valeurs scalaires atomiques non mutables, sans conteneur imbriqué. Écrire une fonction copier_grille_deux_niveaux(grille) qui réalise une **copie des deux premiers niveaux**. Sa garantie est limitée aux modifications de la structure externe, aux remplacements de lignes et aux réaffectations de cellules dans la copie, qui ne doivent pas modifier la grille d'origine.
5. Hors de cette précondition, analyser le contre-exemple [[[1]]]. Expliquer pourquoi la cellule-liste reste partagée entre l'original et la copie.

Exercice 64

On suppose que les cellules sont des valeurs scalaires atomiques non mutables, sans conteneur imbriqué. On veut réaliser une **copie des deux premiers niveaux** d'une grille.

1. Écrire une fonction copier_grille_deux_niveaux(grille) qui crée une nouvelle liste externe et une nouvelle liste pour chaque ligne. Sa garantie est limitée aux modifications de la structure externe, aux remplacements de lignes et aux réaffectations de cellules dans la copie : elles ne doivent pas modifier la grille d'origine.
2. Étudier le scénario numérique suivant :

```
1 g = [[1, 2], [3, 4]]
2 c = copier_grille_deux_niveaux(g)
3 c[0][0] = 99
4 print(g[0][0])
```

1

Justifier l'affichage en comparant l'identité de la liste externe et de chacune des lignes.

3. Le cas [[[1]]] est hors contrat : la cellule-liste reste partagée. Expliquer pourquoi une mutation de cette liste interne par la copie affecte aussi l'original, et pourquoi la fonction ne garantit rien au-delà des deux premiers niveaux.

Exercice 65

On représente des équipes d'e-sport par une liste de dictionnaires ayant les clés "nom" et "scores" (une liste d'entiers).

1. Écrire une fonction historique_scores(equipes, resultats) qui :
 - prend la liste des équipes et une liste de tuples (nom, score) ;
 - renvoie une **nouvelle** liste d'équipes où les scores ont été ajoutés ;
 - **ne modifie pas** la liste d'origine (ni les dictionnaires, ni les listes de scores qu'ils contiennent).
2. Tester avec :

```
1 equipes = [{"nom": "A", "scores": [10, 20]},
2           {"nom": "B", "scores": [15]}]
3 nouvelles = historique_scores(equipes, [("A", 30), ("B", 25)])
```

```

4 print(equipes[0]["scores"]) # [10, 20]
5 print(nouvelles[0]["scores"]) # [10, 20, 30]

```

3. Expliquer pourquoi il faut copier à la fois les dictionnaires et les listes de scores.

Coup de pouce 1. Demande-toi combien de listes existent réellement en mémoire après la ligne 2 : une ou deux ?

Coup de pouce 2. Dessine la mémoire : deux étiquettes (notes, copie) et des flèches vers les objets. Où pointent-elles ?

Coup de pouce 3. Compare `copie = notes` (partage de référence) et `copie = list(notes)` (nouvel objet). Rejoue le programme avec chacune.

Coup de pouce 1. Rappelle-toi qu'un tuple est entouré de parenthèses et qu'on accède à ses éléments par un indice commençant à 0.

Coup de pouce 2. Pour la question 3, essaie la commande dans la console Python et observe le message d'erreur.

Coup de pouce 1. Utilise la formule de distance : $d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$.

Coup de pouce 2. En Python, la racine carrée se calcule avec `** 0.5`. Accède aux coordonnées du tuple avec `a[0]` et `a[1]`.

Coup de pouce 1. On te demande de prévoir les affichages d'un programme qui utilise un tuple et le déballage (*unpacking*), puis d'expliquer pourquoi un tuple ne peut pas être modifié.

Coup de pouce 2. Commence par identifier ce que contient chaque variable après `nom, elo, rang = joueur` : chaque variable reçoit l'élément du tuple à la position correspondante.

Coup de pouce 1. On te demande de prévoir l'affichage d'un programme qui parcourt une liste de tuples et filtre selon une condition sur la température.

Coup de pouce 2. À chaque tour de boucle, `ville` et `temp` reçoivent les deux éléments du tuple courant (déballage). Vérifie pour chaque tuple si `temp > 30`.

Coup de pouce 1. On te demande d'écrire une fonction qui calcule les coordonnées du milieu d'un segment à partir de deux points représentés par des tuples (x, y) .

Coup de pouce 2. La formule du milieu est $(\frac{x_A + x_B}{2}, \frac{y_A + y_B}{2})$. Accède aux coordonnées avec `a[0]`, `a[1]`, `b[0]`, `b[1]` et renvoie un tuple.

Coup de pouce 1. On te demande de prévoir les affichages d'un programme qui utilise le déballage d'un tuple et une expression conditionnelle (`... if ... else ...`).

Coup de pouce 2. L'expression `equipe1 if score1 > score2 else equipe2` renvoie `equipe1` si la condition est vraie, sinon `equipe2`. Compare les scores pour déterminer `resultat`.

Coup de pouce 1. On te demande de prévoir les affichages d'un programme qui accède aux éléments d'un tableau (liste) par leur indice, y compris un indice négatif, et qui modifie un élément.

Coup de pouce 2. Rappelle-toi que `temperatures[-1]` désigne le dernier élément du tableau. Pour la question 2, compare les listes (mutables) aux tuples (immuables).

Coup de pouce 1. On te demande de prévoir les affichages d'un programme qui construit un nouveau tableau en filtrant les éléments d'un tableau existant selon une condition.

Coup de pouce 2. Suis la boucle élément par élément : pour chaque valeur `s` de scores, vérifie si `s >= 10` ; si oui, elle est ajoutée à `resultat` avec `append`.

Coup de pouce 1. On te demande d'écrire une fonction qui calcule la moyenne des éléments d'un tableau d'entiers.

Coup de pouce 2. Parcourt le tableau avec une boucle `for` pour accumuler la somme dans une variable, puis divise cette somme par `len(tab)`.

Coup de pouce 1. On te demande de prévoir les affichages de deux constructions par compréhension de liste, puis de réécrire l'une d'elles avec une boucle `for` classique.

Coup de pouce 2. La syntaxe `[n for n in notes if n >= 10]` signifie : « pour chaque `n` dans `notes`, garde `n` seulement si `n >= 10` ». Pour réécrire, utilise un tableau vide, une boucle et `append`.

Coup de pouce 1. On te demande de tracer l'exécution d'un programme qui utilise deux tableaux parallèles (élèves et notes) et affiche ceux qui ont la moyenne.

Coup de pouce 2. L'indice `i` sert à accéder à la même position dans les deux tableaux : `elevés[i]` et `notes[i]` vont ensemble. C'est pour cela qu'on utilise `range(len(elevés))` et non `for e in élevés`.

Coup de pouce 1. On te demande de lire les valeurs d'une grille (tableau de tableaux) en utilisant la double indexation `grille[ligne][colonne]`.

Coup de pouce 2. `grille[0]` est la première ligne `[1, 2, 3]`, donc `grille[0][1]` est le deuxième élément de cette ligne. `len(grille)` donne le nombre de lignes.

Coup de pouce 1. On te demande de calculer la somme de toutes les valeurs d'une grille avec une double boucle, puis de modifier le code pour obtenir la moyenne.

Coup de pouce 2. La boucle externe parcourt chaque ligne, la boucle interne chaque valeur de la ligne. Pour la moyenne, divise le total par le nombre total de cases (lignes × colonnes).

Coup de pouce 1. On te demande d'écrire une fonction qui construit une grille de `n` lignes et `p` colonnes remplie d'une même valeur.

Coup de pouce 2. Utilise deux boucles imbriquées : la boucle externe crée chaque ligne, la boucle interne remplit la ligne avec `append`. Ajoute chaque ligne terminée à la grille.

Coup de pouce 3. `[[valeur] * p] * n` crée `n` références vers la *même* ligne : modifier une case les modifie toutes. C'est pourquoi il faut créer une nouvelle ligne à chaque itération.

Coup de pouce 1. On te demande de tracer l'exécution d'un programme qui modifie la diagonale d'une grille 3×3, puis d'identifier l'objet mathématique obtenu.

Coup de pouce 2. La boucle `for i in range(3)` place un 1 aux positions `grille[0][0]`, `grille[1][1]` et `grille[2][2]`, c'est-à-dire sur la diagonale.

Coup de pouce 1. On te demande de lire les valeurs d'un dictionnaire en utilisant ses clés, puis d'ajouter une nouvelle entrée.

Coup de pouce 2. Accède avec `elevé["nom"]`. Ajoute avec `elevé["email"] = "..."`. Compte avec `len(elevé)`.

Coup de pouce 1. On te demande de prévoir les affichages d'un programme qui modifie une clé existante, ajoute une nouvelle clé et teste l'appartenance d'une clé à un dictionnaire.

Coup de pouce 2. La syntaxe est la même pour modifier et ajouter : `dico[cle] = valeur`. L'opérateur `in` teste si une *clé* (pas une valeur) est présente dans le dictionnaire.

Coup de pouce 1. On te demande d'écrire une fonction qui crée et renvoie un dictionnaire à partir de trois paramètres.

Coup de pouce 2. Construis le dictionnaire directement avec la syntaxe `{"nom": nom, "elo": elo, "rang": rang}` et renvoie-le avec `return`.

Coup de pouce 1. On te demande de prévoir les affichages d'un parcours de dictionnaire avec une boucle `for`, puis de réécrire ce parcours avec `.items()`.

Coup de pouce 2. `for cle in station` parcourt uniquement les *clés*. Pour obtenir la valeur associée, on écrit `station[cle]`. Avec `.items()`, la boucle devient `for cle, val in station.items()`.

Coup de pouce 1. On te demande de comprendre pourquoi modifier `b` modifie aussi `a` quand on écrit `b = a` avec une liste.

Coup de pouce 2. L'affectation `b = a` ne copie pas la liste : `a` et `b` sont deux noms pour le *même* objet en mémoire. `a is b` renvoie `True` car c'est la même référence.

Coup de pouce 1. On te demande de comprendre la différence entre `b = a` (alias) et `b = list(a)` (copie) d'une liste, puis de citer d'autres façons de copier.

Coup de pouce 2. `list(a)` crée un *nouvel* objet contenant les mêmes éléments : modifier `b` ne modifie plus `a`. Pense aussi à `a[:]` et `a.copy()`.

Coup de pouce 1. On te demande de comprendre pourquoi une fonction sans `return` peut quand même modifier une liste passée en argument. C'est la notion d'effet de bord.

Coup de pouce 2. Quand tu passes une liste à une fonction, la fonction reçoit une *référence* vers le même objet. Tout `append` ou modification d'élément à l'intérieur de la fonction agit sur l'original.

Coup de pouce 1. On te demande de comprendre pourquoi l'affectation `t[0] = 10` provoque une erreur quand `t` est un tuple, et d'expliquer l'intérêt de l'immuabilité.

Coup de pouce 2. Python lève une `TypeError` car un tuple est immuable : une fois créé, on ne peut ni ajouter, ni supprimer, ni modifier ses éléments. C'est une garantie que les données ne changeront pas.

TD 1 — Station météo connectée

Durée : 50 minutes. Objectif : manipuler tuples, tableaux et dictionnaires dans un contexte réaliste.

Une station météo enregistre des mesures toutes les heures. Chaque mesure est un tuple (heure, température, humidité). L'ensemble des mesures d'une journée est stocké dans un tableau.

```
1 mesures = [
2     (0, 15.2, 82), (1, 14.8, 85), (2, 14.1, 88),
3     (6, 13.5, 90), (7, 14.0, 87), (8, 15.5, 80),
4     (12, 22.3, 55), (13, 23.1, 52), (14, 23.8, 50),
5     (18, 20.1, 60), (19, 18.5, 65), (20, 17.2, 70),
6 ]
```

Partie A — Exploration (C1, C2)

Exercice 66 ♦

1. Donner le type de `mesures[0]` et la valeur de `mesures[0][1]`.
2. Écrire une fonction `temperature_max(mesures)` qui renvoie un tuple (heure, temp) correspondant à la température la plus élevée.

Exercice 67 ♦

Écrire une fonction `temperatures(mesures)` qui renvoie le tableau des températures extraites de chaque mesure, en compréhension de liste.

Partie B — Analyse (C4)

Exercice 68 ♦♦

On veut regrouper les mesures par tranche horaire dans un dictionnaire : "nuit" (0h–6h), "matin" (7h–12h), "après-midi" (13h–18h), "soir" (19h–23h).

Écrire une fonction `par_tranche(mesures)` qui renvoie un dictionnaire dont les clés sont les tranches et les valeurs sont les tableaux de mesures correspondantes.

Exercice 69 ♦♦

Écrire une fonction `resume(mesures)` qui renvoie un dictionnaire de la forme :

```
1 {"temp_min": 13.5, "temp_max": 23.8,
2  "hum_moy": 72.0, "nb_mesures": 12}
```

Partie C — Synthèse (C1, C2, C4)

Exercice 70 ♦♦

Écrire un programme complet qui :

1. Calcule le résumé de la journée.
2. Affiche les mesures de la tranche "après-midi".
3. Identifie l'heure la plus humide.

TD 2 — Classement d'un tournoi

Durée : 50 minutes. Objectif : croiser tableaux, dictionnaires et mutabilité dans un projet de classement.

Un tournoi oppose 6 joueurs sur 4 manches. Les résultats sont stockés dans une grille (tableau de tableaux) : chaque ligne est un joueur, chaque colonne une manche.

```
1 pseudos = ["Ace", "Bob", "Cat", "Dan", "Eve", "Fox"]
2 scores = [
3     [120, 150, 130, 140],
4     [180, 160, 170, 190],
5     [110, 140, 120, 100],
6     [200, 180, 210, 190],
7     [90, 110, 100, 120],
8     [160, 170, 150, 180],
9 ]
```

Partie A — Lecture de la grille (C3)

Exercice 71 ♦

- Combien de lignes et de colonnes a la grille scores ?
- Donner l'expression Python pour obtenir le score de Cat à la manche 3.
- Écrire une fonction `total_joueur(scores, i)` qui renvoie la somme des scores du joueur d'indice `i`.

Partie B — Construction du classement (C2, C4)

Exercice 72 ♦♦

Écrire une fonction `classement(pseudos, scores)` qui renvoie un tableau de dictionnaires triés par total décroissant. Chaque dictionnaire contient les clés "pseudo", "total" et "rang".

Exemple de sortie (premier élément) :

```
1 {"pseudo": "Dan", "total": 780, "rang": 1}
```

Exercice 73 ♦♦

Écrire une fonction `meilleur_manche(scores, j)` qui renvoie l'indice du joueur ayant le meilleur score à la manche `j`.

Partie C — Mutabilité et copies (C5)

Exercice 74 ♦♦

On veut simuler un bonus : ajouter 10 points à chaque score de la manche 1, **sans modifier la grille originale**.

- Un élève écrit :

```
1 copie = list(scores)
2 for i in range(len(copie)):
3     copie[i][0] += 10
```

Expliquer pourquoi la grille originale est aussi modifiée.

- Corriger le code pour que la grille originale reste intacte.
- Vérifier en affichant `scores[0][0]` avant et après.

Partie D — Synthèse (C2, C3, C4)

Exercice 75

Écrire un programme complet qui :

1. Affiche le classement complet.
2. Identifie le MVP (joueur avec la meilleure moyenne par manche).
3. Crée un dictionnaire équipes regroupant les 6 joueurs en 2 équipes de 3 (indices 0–2 et 3–5), avec le total par équipe.

MINI-PROJET — CLASSEMENT D'UN TOURNOI DE E-SPORT

Un tournoi de e-sport oppose 8 joueurs sur 5 manches. Chaque joueur est représenté par un dictionnaire contenant son pseudo, son équipe et ses scores (un tableau d'entiers). L'objectif est de construire un programme complet qui gère le classement.

Jalon 1. Représenter les données. Définis un tableau joueurs contenant 8 dictionnaires avec les clés "pseudo", "equipe" et "scores" (tableau de 5 entiers). Vérifie que tu peux accéder au 3e score du 2e joueur.

Jalon 2. Calculer les totaux. Écris une fonction total(joueur) qui renvoie la somme des scores d'un joueur. Ajoute une clé "total" à chaque joueur.

Jalon 3. Trier le classement. Écris une fonction classement(joueurs) qui renvoie un nouveau tableau (pas un alias !) des joueurs triés par total décroissant. Indice : utilise sorted avec le paramètre key.

Jalon 4. Afficher les résultats. Écris une fonction afficher_classement(joueurs) qui affiche le classement sous la forme : 1. Ace (TeamA) – 850 pts.

Critères de validation (testables) : Jalon 1 : accès joueurs[1]["scores"][2] renvoie un entier ; Jalon 2 : total(joueurs[0]) == somme des 5 scores ; Jalon 3 : le premier du classement a le total le plus élevé ET joueurs n'est pas modifié (copie, pas alias) ; Jalon 4 : l'affichage montre rang, pseudo, équipe et total.

Extensions

- ▶ Ajouter le classement par équipe (somme des totaux des joueurs de chaque équipe).
- ▶ Gérer les ex aequo (même total → départage par le meilleur score individuel).
- ▶ Exporter le classement dans un fichier texte.

Diagnostics du QCM — Types construits

Compare tes réponses et lis le diagnostic correspondant.

Q1. Réponse : B.

- ▶ Si A : tu confonds tuple (parenthèses) et liste (crochets) → M1.
- ▶ Si C : la virgule crée un tuple, pas un entier → D1.

Q2. Réponse : B.

- ▶ Si A : un tuple est non mutable, on ne peut pas modifier ses éléments → EF1.
- ▶ Si C : t[0] = 5 n'ajoute pas, il tente une modification interdite → D1.

Q3. Réponse : C.

- ▶ Si A : le déballage associe le premier élément à a → D3.
- ▶ Si B : le déballage sépare les éléments, il ne copie pas le tuple → D3.

Q4. Réponse : B.

- ▶ Si A : append ajoute un élément, la longueur augmente de 1 → D5.
- ▶ Si C : un seul élément est ajouté par append → D5.

Q5. Réponse : A.

- ▶ Si B : cette expression produit 10 éléments (0 à 18), pas 5 → D6.

- ▶ Si C : aucun filtre pair, et 8 est exclu de range(8) → D6.
Q6. Réponse : B.
- ▶ Si A : l'indice 3 n'existe pas dans un tableau de 3 éléments (indices 0-2).
- ▶ Si C : Python ne renvoie pas None pour un indice invalide.
Q7. Réponse : B.
- ▶ Si A : tu confonds ligne et colonne. g[1] est [3, 4] → D7.
- ▶ Si C : g[1] n'est pas [1, 2] → D7.
Q8. Réponse : B.
- ▶ Si A : * crée des alias de la même ligne → D8.
- ▶ Si C : cette expression crée une grille 2×3, pas 3×2.
Q9. Réponse : B.
- ▶ Si A : chaque sous-liste est une ligne, il y en a 2 → D7.
Q10. Réponse : C.
- ▶ Si A : Python ne renvoie pas None mais lève une exception → EF3, D10.
Q11. Réponse : B.
- ▶ Si A : append est une méthode de liste, pas de dictionnaire → D10.
Q12. Réponse : B.
- ▶ Si A : cette boucle ne donne que les clés.
- ▶ Si C : cette boucle ne donne que les valeurs.
Q13. Réponse : B.
- ▶ Si A : b = a crée un alias, pas une copie → D11.
Q14. Réponse : B.
- ▶ Si A : ceci crée un alias, pas une copie → D11, D12.
- ▶ Si C : t * 1 crée une copie superficielle, mais list(t) est plus explicite.
Q15. Réponse : B.
- ▶ Si A : la copie superficielle ne copie pas les sous-listes → EF4.
Q16. Réponse : B.
- ▶ Si A : tuple et str sont non mutables → D11.
- ▶ Si C : int est non mutable → D11.

QCM — Types construits

Pour chaque question, une seule réponse est correcte.

Q1. (C1) Que renvoie type((3, 5)) ?

- A. <class 'list'>
- B. <class 'tuple'>
- C. <class 'int'>

Q2. (C1) Que se passe-t-il si on exécute ce code ?

```
1 t = (1, 2)
2 t[0] = 5
```

- A. t vaut (5, 2)
- B. Une erreur TypeError
- C. t vaut (1, 2, 5)

Q3. (C1) $a, b = (10, 20)$. Quelle est la valeur de b ?

- A. 10
- B. (10, 20)
- C. 20

Q4. (C2) `t = [3, 1, 4]; t.append(1); print(len(t))`. Qu'affiche ce programme ?

- A. 3
- B. 4
- C. 5

Q5. (C2) Quelle expression construit la liste `[0, 2, 4, 6, 8]` ?

- A. `[x for x in range(10) if x % 2 == 0]`
- B. `[x * 2 for x in range(10)]`
- C. `[x for x in range(8)]`

Q6. (C2) `t = [10, 20, 30]; print(t[3])`. Qu'obtient-on ?

- A. 30
- B. `IndexError`
- C. `None`

Q7. (C3) `g = [[1, 2], [3, 4]]; print(g[1][0])`. Qu'affiche ce programme ?

- A. 2
- B. 3
- C. 1

Q8. (C3) Quelle expression crée une grille 3x2 de zéros sans alias ?

- A. `[[0, 0]] * 3`
- B. `[[0] * 2 for _ in range(3)]`
- C. `[[0 for _ in range(3)] for _ in range(2)]`

Q9. (C3) Combien de lignes et de colonnes a `g = [[5, 6, 7], [8, 9, 10]]` ?

- A. 3 lignes, 2 colonnes
- B. 2 lignes, 3 colonnes
- C. 6 lignes, 1 colonne

Q10. (C4) `d = {"a": 1, "b": 2}; print(d["c"])`. Qu'obtient-on ?

- A. `None`
- B. 0
- C. `KeyError`

Q11. (C4) Comment ajouter la clé "c" avec la valeur 3 à `d` ?

- A. `d.append("c", 3)`
- B. `d["c"] = 3`
- C. `d.add("c": 3)`

Q12. (C4) Quelle boucle parcourt clés ET valeurs d'un dictionnaire `d` ?

- A. `for k in d:`
- B. `for k, v in d.items():`
- C. `for v in d.values():`

Q13. (C5) `a = [1, 2]; b = a; b.append(3); print(a)`. Qu'affiche-t-il ?

- A. `[1, 2]`
- B. `[1, 2, 3]`

C. Une erreur

Q14. (C5) Comment obtenir une copie indépendante d'une liste `t` ?

- A. `copie = t`
- B. `copie = list(t)`
- C. `copie = t * 1`

Q15. (C5) `g = [[1], [2]]; h = list(g); h[0].append(9); print(g[0]).`

- A. `[1]`
- B. `[1, 9]`
- C. Une erreur

Q16. (C5) Parmi ces types, lesquels sont mutables ?

- A. tuple et str
- B. list et dict
- C. int et list

Évaluation A — Types construits

Durée : 55 minutes. Calculatrice interdite. Documents interdits.

Barème indicatif : Ex. 1 : 4 pts (analyser); Ex. 2 : 6 pts (programmer); Ex. 3 : 5 pts (analyser, programmer); Ex. 4 : 5 pts (programmer)

Exercice 1 — Lecture de programme

(4 points)

On considère le programme suivant :

```
1 joueurs = [
2     {"pseudo": "Ace", "score": 150},
3     {"pseudo": "Bob", "score": 200},
4     {"pseudo": "Cat", "score": 180},
5 ]
6 meilleur = joueurs[0]
7 for j in joueurs:
8     if j["score"] > meilleur["score"]:
9         meilleur = j
10 print(meilleur["pseudo"])
11 print(meilleur["score"])
```

1. Donner, sans exécuter le programme, les deux valeurs affichées.
2. Expliquer en une phrase le rôle de ce programme.
3. Si on ajoute `meilleur["score"] = 0` après la boucle, quelle est la valeur de `joueurs[1]["score"]` ? Justifier.

Exercice 2 — Programmation avec des tableaux

(6 points)

Un tournoi de e-sport enregistre les scores de chaque manche dans un tableau :

```
1 scores = [150, 200, 180, 210, 170]
```

1. Écrire une fonction `au_dessus(scores, seuil)` qui renvoie le tableau des scores strictement supérieurs au seuil.
2. Écrire une fonction `moyenne(scores)` qui renvoie la moyenne des scores. La fonction doit lever `ValueError` si le tableau est vide.

3. Écrire une fonction `classement(scores)` qui renvoie le tableau des tuples (rang, score) triés par score décroissant. Exemple : `classement([150, 200, 180])` renvoie `[(1, 200), (2, 180), (3, 150)]`.

Exercice 3 — Grille et pixels

(5 points)

On représente une image en niveaux de gris par une grille (tableau de tableaux) :

```
1 image = [
2     [100, 200, 100],
3     [200, 50, 200],
4     [100, 200, 100],
5 ]
```

- Combien de lignes et de colonnes contient cette image ?
- Quelle est la valeur du pixel central ? Donner l'expression Python.
- Écrire une fonction `nb_sombres(image, seuil)` qui compte le nombre de pixels dont la valeur est inférieure ou égale au seuil.

Exercice 4 — Dictionnaire et mutabilité

(5 points)

On gère un carnet de contacts :

```
1 contacts = {"Ali": "55123456", "Sara": "55234567"}
```

- Écrire une fonction `chercher(contacts, nom)` qui renvoie le numéro du contact ou "Inconnu" si le nom n'est pas dans le carnet.
- Écrire une fonction `ajouter(contacts, nom, numero)` qui ajoute un contact au carnet. Préciser : cette fonction modifie-t-elle le dictionnaire passé en argument ?
- On exécute :

```
1 carnet1 = {"Ali": "55123456"}
2 carnet2 = carnet1
3 carnet2["Sara"] = "55234567"
```

Quelle est la valeur de `carnet1` après ces trois lignes ? Justifier.

Évaluation B — Types construits

Durée : 55 minutes. Calculatrice interdite. Documents interdits.

Barème indicatif : Ex. 1 : 5 pts (analyser) ; Ex. 2 : 5 pts (programmer) ; Ex. 3 : 5 pts (programmer, analyser) ; Ex. 4 : 5 pts (programmer)

Exercice 1 — Analyse de programme

(5 points)

On donne le programme suivant :

```
1 inventaire = {"pommes": 12, "bananes": 5, "oranges": 8}
2 panier = []
3 for fruit, quantite in inventaire.items():
4     if quantite > 6:
5         panier.append((fruit, quantite))
6 inventaire["pommes"] = 0
7 print(panier)
8 print(inventaire["pommes"])
```

- Donner la valeur de `panier` après la boucle.

2. Quel est le type de chaque élément de panier ?
3. Après `inventaire["pommes"] = 0`, la valeur 12 dans panier est-elle modifiée ? Justifier.
4. Écrire en une ligne une compréhension de liste produisant le même résultat que la boucle.

Exercice 2 — Fonctions sur les tableaux

(5 points)

1. Écrire une fonction `separer(tab, seuil)` qui prend un tableau de nombres et renvoie un tuple de deux tableaux : les éléments $\leq$ au seuil et ceux $>$ au seuil.

```
>> separer([3, 8, 1, 5, 9, 2], 4)
([3, 1, 2], [8, 5, 9])
```

2. Écrire une fonction `sans_doublons(tab)` qui renvoie un nouveau tableau contenant les éléments de tab sans doublons, dans l'ordre de première apparition.

```
>> sans_doublons([3, 1, 3, 2, 1, 4])
[3, 1, 2, 4]
```

Exercice 3 — Grille de jeu

(5 points)

On représente une grille de morpion par un tableau de tableaux :

```
1 grille = [
2     ["X", "O", " "],
3     [" ", "X", " "],
4     ["O", " ", "X"],
5 ]
```

1. Écrire une fonction `compter(grille, symbole)` qui renvoie le nombre d'occurrences de symbole dans la grille.
2. Écrire une fonction `diagonale(grille)` qui renvoie le tableau des éléments de la diagonale principale.
3. La diagonale de l'exemple contient-elle trois symboles identiques ? Écrire une expression Python pour le vérifier.

Exercice 4 — Dictionnaires et mutabilité

(5 points)

On gère un carnet de notes par matière :

```
1 carnet = {"maths": [15, 12], "nsi": [18, 16, 14]}
```

1. Écrire une fonction `ajouter_note(carnet, matiere, note)` qui ajoute une note à la matière donnée. Si la matière n'existe pas, elle est créée avec cette seule note.
2. Écrire une fonction `moyennes(carnet)` qui renvoie un nouveau dictionnaire avec la moyenne par matière.
3. On exécute :

```
1 c1 = {"nsi": [18]}
2 c2 = c1
3 c2["nsi"].append(15)
```

Quelle est la valeur de `c1["nsi"]` ? Expliquer.

Remédiation — Types construits

R1 — Je confonds tuple et liste (C1)

Rappel : un tuple est entouré de parenthèses () et est non mutable ; une liste est entourée de crochets [] et est mutable.

Exercice 76

Pour chaque ligne, indique si la variable est un tuple ou une liste, et si on peut modifier son contenu :

```
1 a = (1, 2, 3)
2 b = [1, 2, 3]
3 c = ("hello",)
4 d = []
```

R2 — Je ne sais pas parcourir un tableau (C2)

Rappel : pour parcourir les éléments, écris `for element in tableau`. Pour parcourir les indices, écris `for i in range(len(tableau))`.

Exercice 77

Écris une fonction `somme(tableau)` qui calcule la somme des éléments d'un tableau de nombres, **sans utiliser la fonction `sum`**.

```
>>> somme([3, 7, 2])
12
```

R3 — Je confonds lignes et colonnes dans une grille (C3)

Rappel : dans `grille[i][j]`, `i` est le numéro de ligne et `j` le numéro de colonne.

Exercice 78

On donne `g = [[10, 20, 30], [40, 50, 60]]`. Donne la valeur de :

1. `g[0][2]`
2. `g[1][0]`
3. `len(g)` et `len(g[0])`

R4 — J'obtiens `KeyError` avec un dictionnaire (C4)

Rappel : avant d'accéder à une clé, vérifie sa présence avec `if cle in dico` ou utilise `dico.get(cle, valeur_par_defaut)`.

Exercice 79

Corrige le programme suivant pour qu'il ne provoque pas d'erreur si la clé "age" est absente :

```
1 personne = {"nom": "Ali"}
2 print(personne["age"])
```

R5 — Je ne comprends pas pourquoi ma liste originale est modifiée (C5)

Rappel : `b = a` crée un alias (même objet en mémoire). Pour une copie indépendante, utilise `b = list(a)`.

Exercice 80

Le programme ci-dessous doit trier une copie sans modifier l'original. Corrige-le :

```
1 original = [5, 2, 8, 1]
2 copie = original
3 copie.sort()
4 print("Original :", original)
5 print("Trie :", copie)
```

Résultat attendu : l'original reste [5, 2, 8, 1].

Remédiation — Types construits

R1 — Je confonds tuple et liste (C1)

Rappel : un tuple est entouré de parenthèses `()` et est non mutable ; une liste est entourée de crochets `[]` et est mutable.

Exercice 81

Pour chaque ligne, indique si la variable est un tuple ou une liste, et si on peut modifier son contenu :

```
1 a = (1, 2, 3)
2 b = [1, 2, 3]
3 c = ("hello",)
4 d = []
```

R2 — Je ne sais pas parcourir un tableau (C2)

Rappel : pour parcourir les éléments, écris `for element in tableau`. Pour parcourir les indices, écris `for i in range(len(tableau))`.

Exercice 82

Écris une fonction `somme(tableau)` qui calcule la somme des éléments d'un tableau de nombres, sans utiliser la fonction `sum`.

```
>> somme([3, 7, 2])
12
```

R3 — Je confonds lignes et colonnes dans une grille (C3)

Rappel : dans `grille[i][j]`, `i` est le numéro de ligne et `j` le numéro de colonne.

Exercice 83

On donne `g = [[10, 20, 30], [40, 50, 60]]`. Donne la valeur de :

1. `g[0][2]`
2. `g[1][0]`

3. len(g) et len(g[0])

R4 — J'obtiens KeyError avec un dictionnaire (C4)

Rappel : avant d'accéder à une clé, vérifie sa présence avec `if cle in dico` ou utilise `dico.get(cle, valeur_par_defaut)`.

Exercice 84

Corrige le programme suivant pour qu'il ne provoque pas d'erreur si la clé "age" est absente :

```
1 personne = {"nom": "Ali"}
2 print(personne["age"])
```

R5 — Je ne comprends pas pourquoi ma liste originale est modifiée (C5)

Rappel : `b = a` crée un alias (même objet en mémoire). Pour une copie indépendante, utilise `b = list(a)`.

Exercice 85

Le programme ci-dessous doit trier une copie sans modifier l'original. Corrige-le :

```
1 original = [5, 2, 8, 1]
2 copie = original
3 copie.sort()
4 print("Original :", original)
5 print("Trie :", copie)
```

Résultat attendu : l'original reste [5, 2, 8, 1].

Remédiation — Types construits

R1 — Je confonds tuple et liste (C1)

Rappel : un tuple est entouré de parenthèses `()` et est non mutable ; une liste est entourée de crochets `[]` et est mutable.

Exercice 86

Pour chaque ligne, indique si la variable est un tuple ou une liste, et si on peut modifier son contenu :

```
1 a = (1, 2, 3)
2 b = [1, 2, 3]
3 c = ("hello",)
4 d = []
```

R2 — Je ne sais pas parcourir un tableau (C2)

Rappel : pour parcourir les éléments, écris `for element in tableau`. Pour parcourir les indices, écris `for i in range(len(tableau))`.

Exercice 87

Écris une fonction `somme(tableau)` qui calcule la somme des éléments d'un tableau de nombres, sans utiliser la fonction `sum`.

```
>>> somme([3, 7, 2])
12
```

R3 — Je confonds lignes et colonnes dans une grille (C3)

Rappel : dans `grille[i][j]`, `i` est le numéro de ligne et `j` le numéro de colonne.

Exercice 88

On donne `g = [[10, 20, 30], [40, 50, 60]]`. Donne la valeur de :

1. `g[0][2]`
2. `g[1][0]`
3. `len(g)` et `len(g[0])`

R4 — J'obtiens `KeyError` avec un dictionnaire (C4)

Rappel : avant d'accéder à une clé, vérifie sa présence avec `if cle in dico` ou utilise `dico.get(cle, valeur_par_defaut)`.

Exercice 89

Corrige le programme suivant pour qu'il ne provoque pas d'erreur si la clé "age" est absente :

```
1 personne = {"nom": "Ali"}
2 print(personne["age"])
```

R5 — Je ne comprends pas pourquoi ma liste originale est modifiée (C5)

Rappel : `b = a` crée un alias (même objet en mémoire). Pour une copie indépendante, utilise `b = list(a)`.

Exercice 90

Le programme ci-dessous doit trier une copie sans modifier l'original. Corrige-le :

```
1 original = [5, 2, 8, 1]
2 copie = original
3 copie.sort()
4 print("Original :", original)
5 print("Trie :", copie)
```

Résultat attendu : l'original reste `[5, 2, 8, 1]`.

Remédiation — Types construits**R1 — Je confonds tuple et liste (C1)**

Rappel : un tuple est entouré de parenthèses `()` et est non mutable ; une liste est entourée de crochets `[]` et est mutable.

Exercice 91

Pour chaque ligne, indique si la variable est un tuple ou une liste, et si on peut modifier son contenu :

```
1 a = (1, 2, 3)
2 b = [1, 2, 3]
3 c = ("hello",)
4 d = []
```

R2 — Je ne sais pas parcourir un tableau (C2)

Rappel : pour parcourir les éléments, écris `for element in tableau`. Pour parcourir les indices, écris `for i in range(len(tableau))`.

Exercice 92

Écris une fonction `somme(tableau)` qui calcule la somme des éléments d'un tableau de nombres, sans utiliser la fonction `sum`.

```
>>> somme([3, 7, 2])
12
```

R3 — Je confonds lignes et colonnes dans une grille (C3)

Rappel : dans `grille[i][j]`, `i` est le numéro de ligne et `j` le numéro de colonne.

Exercice 93

On donne `g = [[10, 20, 30], [40, 50, 60]]`. Donne la valeur de :

1. `g[0][2]`
2. `g[1][0]`
3. `len(g)` et `len(g[0])`

R4 — J'obtiens `KeyError` avec un dictionnaire (C4)

Rappel : avant d'accéder à une clé, vérifie sa présence avec `if cle in dico` ou utilise `dico.get(cle, valeur_par_defaut)`.

Exercice 94

Corrige le programme suivant pour qu'il ne provoque pas d'erreur si la clé "age" est absente :

```
1 personne = {"nom": "Ali"}
2 print(personne["age"])
```

R5 — Je ne comprends pas pourquoi ma liste originale est modifiée (C5)

Rappel : `b = a` crée un alias (même objet en mémoire). Pour une copie indépendante, utilise `b = list(a)`.

Exercice 95

Le programme ci-dessous doit trier une copie sans modifier l'original. Corrige-le :

```
1 original = [5, 2, 8, 1]
2 copie = original
3 copie.sort()
4 print("Original :", original)
5 print("Trie :", copie)
```

Résultat attendu : l'original reste [5, 2, 8, 1].

Remédiation — Types construits

R1 — Je confonds tuple et liste (C1)

Rappel : un tuple est entouré de parenthèses () et est non mutable ; une liste est entourée de crochets [] et est mutable.

Exercice 96 ◆

Pour chaque ligne, indique si la variable est un tuple ou une liste, et si on peut modifier son contenu :

```
1 a = (1, 2, 3)
2 b = [1, 2, 3]
3 c = ("hello",)
4 d = []
```

R2 — Je ne sais pas parcourir un tableau (C2)

Rappel : pour parcourir les éléments, écris `for element in tableau`. Pour parcourir les indices, écris `for i in range(len(tableau))`.

Exercice 97 ◆

Écris une fonction `somme(tableau)` qui calcule la somme des éléments d'un tableau de nombres, sans utiliser la fonction `sum`.

```
»> somme([3, 7, 2])
12
```

R3 — Je confonds lignes et colonnes dans une grille (C3)

Rappel : dans `grille[i][j]`, `i` est le numéro de ligne et `j` le numéro de colonne.

Exercice 98 ◆

On donne `g = [[10, 20, 30], [40, 50, 60]]`. Donne la valeur de :

1. `g[0][2]`
2. `g[1][0]`
3. `len(g)` et `len(g[0])`

R4 — J'obtiens `KeyError` avec un dictionnaire (C4)

Rappel : avant d'accéder à une clé, vérifie sa présence avec `if cle in dico` ou utilise `dico.get(cle, valeur_par_defaut)`.

Exercice 99

Corrige le programme suivant pour qu'il ne provoque pas d'erreur si la clé "age" est absente :

```
1 personne = {"nom": "Ali"}
2 print(personne["age"])
```

R5 — Je ne comprends pas pourquoi ma liste originale est modifiée (C5)

Rappel : `b = a` crée un alias (même objet en mémoire). Pour une copie indépendante, utilise `b = list(a)`.

Exercice 100

Le programme ci-dessous doit trier une copie sans modifier l'original. Corrige-le :

```
1 original = [5, 2, 8, 1]
2 copie = original
3 copie.sort()
4 print("Original :", original)
5 print("Trie :", copie)
```

Résultat attendu : l'original reste [5, 2, 8, 1].

Remédiation — Types construits

R1 — Je confonds tuple et liste (C1)

Rappel : un tuple est entouré de parenthèses `()` et est non mutable ; une liste est entourée de crochets `[]` et est mutable.

Exercice 101

Pour chaque ligne, indique si la variable est un tuple ou une liste, et si on peut modifier son contenu :

```
1 a = (1, 2, 3)
2 b = [1, 2, 3]
3 c = ("hello",)
4 d = []
```

R2 — Je ne sais pas parcourir un tableau (C2)

Rappel : pour parcourir les éléments, écris `for element in tableau`. Pour parcourir les indices, écris `for i in range(len(tableau))`.

Exercice 102

Écris une fonction `somme(tableau)` qui calcule la somme des éléments d'un tableau de nombres, sans utiliser la fonction `sum`.

```
>>> somme([3, 7, 2])
12
```

R3 — Je confonds lignes et colonnes dans une grille (C3)

Rappel : dans `grille[i][j]`, `i` est le numéro de ligne et `j` le numéro de colonne.

Exercice 103

On donne `g = [[10, 20, 30], [40, 50, 60]]`. Donne la valeur de :

1. `g[0][2]`
2. `g[1][0]`
3. `len(g)` et `len(g[0])`

R4 — J'obtiens `KeyError` avec un dictionnaire (C4)

Rappel : avant d'accéder à une clé, vérifie sa présence avec `if cle in dico` ou utilise `dico.get(cle, valeur_par_defaut)`.

Exercice 104

Corrige le programme suivant pour qu'il ne provoque pas d'erreur si la clé "age" est absente :

```
1 personne = {"nom": "Ali"}
2 print(personne["age"])
```

R5 — Je ne comprends pas pourquoi ma liste originale est modifiée (C5)

Rappel : `b = a` crée un alias (même objet en mémoire). Pour une copie indépendante, utilise `b = list(a)`.

Exercice 105

Le programme ci-dessous doit trier une copie sans modifier l'original. Corrige-le :

```
1 original = [5, 2, 8, 1]
2 copie = original
3 copie.sort()
4 print("Original :", original)
5 print("Trie :", copie)
```

Résultat attendu : l'original reste `[5, 2, 8, 1]`.

Version aménagée — Extrait Types construits

Consignes séquencées, mise en page aérée, énoncés allégés.

Exercice 1 — Reconnaître un tuple (C1)

Observe le programme ci-dessous :

```
1 station = ("Tunis", 36.8, 10.2)
```

Étape 1. Complète le tableau :

Expression	Type	Valeur
station[0]	..	...
station[1]	..	...
len(station)	..	...

Étape 2. Essaie dans la console : station[1] = 37.0

Que se passe-t-il ?

Pourquoi ? Un tuple est (mutable / non mutable).

Exercice 2 — Créer une liste (C2)

Étape 1. Crée un tableau de 4 prénoms :

```
1 | prenom = [_____, _____, _____, _____]
```

Étape 2. Ajoute un cinquième prénom :

```
1 | prenom._____("Sami")
```

Étape 3. Combien d'éléments contient prenom maintenant ?

len(prenom) =

Exercice 3 — Copier sans erreur (C5)

Lis le programme ci-dessous, puis **entoure** la bonne réponse.

```
1 | notes = [12, 15, 9]
2 | copie = notes
3 | copie.append(18)
```

Que vaut notes après ces trois lignes ?

[12, 15, 9] ou [12, 15, 9, 18]

Explique en une phrase :

Corrige la ligne 2 pour que notes ne soit pas modifiée :

```
1 | copie = _____(notes)
```

Corrigé

1. Le programme affiche [12, 15, 9, 18] puis 4.

2. L'affectation copie = notes ne crée pas une nouvelle liste : elle fait pointer le nom copie vers le même objet que notes (deux références vers une seule liste en mémoire). L'appel copie.append(18)

modifie cet objet partagé ; `notes` et `copie` désignant la même liste, les deux « voient » la modification. C'est un effet de bord caractéristique des objets *mutables*.

3. Il faut créer une vraie copie de la liste, par exemple `copie = list(notes)` (ou `copie = notes[:]`). Après `copie.append(18)`, on a alors `notes = [12, 15, 9]` et `copie = [12, 15, 9, 18]` : les deux noms désignent des objets distincts.

Corrigé

1. `type(station)` renvoie `<class 'tuple'>`. Le tuple contient 3 éléments, donc `len(station)` vaut 3.
2. `station[0]` vaut "Tunis" (premier élément, indice 0). `station[2]` vaut 10.2 (troisième élément, indice 2).
3. Non. Un tuple est non mutable : toute tentative de modification provoque `TypeError: 'tuple' object does not support item assignment`. Si l'on a besoin de modifier une valeur, il faut construire un nouveau tuple ou utiliser une liste.

Corrigé

On applique la formule de distance euclidienne $d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$. Chaque point est un tuple de deux coordonnées.

```
1 def distance(a, b):
2     return ((a[0] - b[0]) ** 2 + (a[1] - b[1]) ** 2) ** 0.5
```

Vérification :

```
>> distance((0, 0), (3, 4))
5.0
>> distance((1, 1), (1, 1))
0.0
```

Le résultat est cohérent : la distance de (0,0) à (3,4) est $\sqrt{9 + 16} = 5$, et la distance d'un point à lui-même est 0.

Corrigé

1. Les trois affichages sont : Fatou, puis True (car $2350 > 2000$), puis diamant (élément d'indice 2).
2. C'est le **déballage de tuple** (*tuple unpacking*) : Python affecte chaque élément du tuple à la variable correspondante, dans l'ordre.
3. Non. Un tuple est **non mutable** : l'instruction `joueur[1] = 2400` provoque une erreur `TypeError: 'tuple' object does not support item assignment`.

Vérification : les résultats ont été confirmés par exécution.

Corrigé

1. Le code affiche Paris puis Lyon. En effet, seules ces deux villes ont une température strictement supérieure à 30 (32.5 et 35.1). Lille (28.7) ne satisfait pas la condition.
2. Chaque élément de `releves` est un **tuple** de la forme (str, float).
3. `releves[1][0]` vaut "Lyon" : c'est le premier élément (indice 0) du deuxième tuple (indice 1) de la liste.

Vérification : les résultats ont été confirmés par exécution.

Corrigé

```
1 def coordonnees_milieu(a, b):
2     # a et b sont des tuples (x, y)
3     return ((a[0] + b[0]) / 2, (a[1] + b[1]) / 2)
```

La fonction accède aux coordonnées par indexation ($a[0]$ pour x_A , $a[1]$ pour y_A) et renvoie un nouveau tuple contenant les moyennes.

Vérification : `coordonnees_milieu((0, 0), (4, 6))` renvoie bien `(2.0, 3.0)`.

Corrigé

1. Le premier print affiche TeamA car $2 > 1$, donc l'expression conditionnelle choisit `equipe1`. Le second print affiche 3 (somme $2 + 1$).
2. Avec le match `("X", "Y", 0, 3)`, on a $0 > 3$ qui est faux, donc `resultat` contient "Y" (l'équipe gagnante).

Vérification : les résultats ont été confirmés par exécution.

Corrigé

On parcourt la liste en retenant le tuple ayant la plus grande température :

```
1 def station_la_plus_chaude(stations):
2     # On initialise avec la premiere station
3     meilleure = stations[0]
4     for station in stations:
5         # On compare les temperatures (indice 1)
6         if station[1] > meilleure[1]:
7             meilleure = station
8     # On renvoie uniquement le nom (indice 0)
9     return meilleure[0]
```

La variable `meilleure` stocke le tuple complet de la station candidate. À chaque itération, si la température courante dépasse le maximum connu, on met à jour `meilleure`. En fin de boucle, `meilleure[0]` donne le nom recherché.

Vérification : `station_la_plus_chaude([("Paris", 32), ("Lyon", 35), ("Lille", 28)])` renvoie bien "Lyon".

Corrigé

1. On parcourt la liste en déballant chaque tuple et on conserve ceux dont le score Elo atteint le seuil :

```
1 def filtrer_par_rang(joueurs, rang_min):
2     resultat = []
3     for nom, elo, rang in joueurs:
4         # On compare le score Elo au seuil
5         if elo >= rang_min:
6             resultat.append((nom, elo, rang))
7     return resultat
```

Le déballage `for nom, elo, rang in joueurs` affecte directement les trois composantes de chaque tuple aux variables `nom`, `elo` et `rang`, ce qui rend le code lisible.

2. Test avec la liste donnée :

```
>>> filtrer_par_rang(joueurs, 2000)
[('Alice', 2400, 'diamant'), ('Clara', 2100, 'platine')]
```

Bob (Elo 1800) est exclu car $1800 < 2000$.

Vérification : les résultats ont été confirmés par exécution.

Corrigé

1. Les trois affichages sont :

- ▶ (3, 7) — le tuple a n'est pas modifié (les tuples sont non mutables, et swap renvoie un *nouveau* tuple).
- ▶ (7, 3) — b contient le tuple renvoyé par swap(a), dont les deux premiers éléments sont inversés.
- ▶ (2, 1) — swap(c) échange les éléments d'indices 0 et 1 de c.

2. Non, a n'est pas modifié. Un tuple est non mutable : la fonction swap construit et renvoie un nouveau tuple sans toucher à l'original. Après l'appel, a vaut toujours (3, 7).

3. La fonction swap(c) renvoie (2, 1) : elle ne considère que les indices 0 et 1. Le troisième élément (la valeur 3) n'apparaît pas dans le résultat, mais il n'est pas « perdu » : le tuple c vaut toujours (1, 2, 3) puisqu'il est non mutable.

Vérification : les résultats ont été confirmés par exécution.

Corrigé

On crée trois listes accumultrices et on classe chaque équipe selon la comparaison des scores :

```
1 def classer_matches(matches):
2     victoires = []
3     defaites = []
4     nuls = []
5     for equipe, score_pour, score_contre in matches:
6         if score_pour > score_contre:
7             victoires.append(equipe)
8         elif score_pour < score_contre:
9             defaites.append(equipe)
10        else:
11            nuls.append(equipe)
12    return (victoires, defaites, nuls)
```

Le déballage `for equipe, score_pour, score_contre in matches` extrait directement les trois composantes de chaque tuple. La structure conditionnelle `if / elif / else` couvre les trois cas possibles (victoire, défaite, match nul). La fonction renvoie un tuple de trois listes.

Vérification : `classer_matches([("A", 3, 1), ("B", 0, 2), ("C", 1, 1)])` renvoie bien `(["A"], ["B"], ["C"])`.

Corrigé

On cherche indépendamment les minimum et maximum de chaque coordonnée en parcourant tous les points une seule fois :

```
1 def enveloppe_convexe_1d(points):
2     if len(points) == 0:
3         return None
4     # Initialisation avec le premier point
5     x_min = points[0][0]
6     x_max = points[0][0]
7     y_min = points[0][1]
8     y_max = points[0][1]
9     # Parcours de tous les points
10    for x, y in points:
11        if x < x_min:
```



```

12     x_min = x
13     if x > x_max:
14         x_max = x
15     if y < y_min:
16         y_min = y
17     if y > y_max:
18         y_max = y
19     return ((x_min, y_min), (x_max, y_max))

```

Raisonnement : le rectangle englobant est défini par quatre valeurs extrêmes. On initialise chaque extremum avec le premier point, puis on met à jour au fil du parcours. Les comparaisons sur x et y sont indépendantes : le coin inférieur gauche ($x_{\min}$, $y_{\min}$) et le coin supérieur droit ($x_{\max}$, $y_{\max}$) peuvent provenir de points différents.

Complexité : un seul parcours de la liste, donc $O(n)$ où n est le nombre de points.

Vérification : `enveloppe_convexe_1d([(1, 2), (3, 0), (-1, 5)])` renvoie bien `((-1, 0), (3, 5))`.

Corrigé

On utilise un tri par sélection en ordre décroissant sur le score Elo, puis on extrait les n premiers noms :

```

1 def top_joueurs(joueurs, n):
2     # Copie pour ne pas modifier la liste originale
3     classement = list(joueurs)
4     # Tri par selection (Elo décroissant)
5     for i in range(len(classement)):
6         idx_max = i
7         for j in range(i + 1, len(classement)):
8             if classement[j][1] > classement[idx_max][1]:
9                 idx_max = j
10            classement[i], classement[idx_max] = (
11                classement[idx_max], classement[i]
12            )
13    # Extraction des n premiers noms
14    resultat = []
15    for k in range(min(n, len(classement))):
16        resultat.append(classement[k][0])
17    return tuple(resultat)

```

Raisonnement : on copie la liste d'entrée avec `list(joueurs)` pour ne pas modifier l'original. Le tri par sélection cherche à chaque étape le tuple ayant le plus grand Elo parmi les éléments restants (`classement[j][1]` accède au score Elo du j -ème tuple) et le place en position i . Enfin, on construit le tuple résultat en ne conservant que les noms (`classement[k][0]`).

Complexité : le tri par sélection est en $O(n^2)$ où n est le nombre de joueurs. L'extraction finale est en $O(n)$.

Vérification : `top_joueurs(joueurs, 2)` renvoie bien `("Clara", "Alice")`.

Corrigé

1. Les quatre affichages sont :

- ▶ `temperatures[0]` vaut 15 (premier élément, indice 0).
- ▶ `temperatures[-1]` vaut 25 (dernier élément, indice -1).
- ▶ `len(temperatures)` vaut 5 (le tableau contient 5 éléments).
- ▶ Après `temperatures[2] = 20`, le tableau vaut `[15, 22, 20, 30, 25]`.

- On peut écrire `temperatures[2] = 20` car une liste est **mutable** : on peut modifier ses éléments après sa création. Un tuple est **non mutable** (immuable) : toute tentative de modification déclenche une erreur `TypeError`.

Vérification : les résultats ont été confirmés par exécution.

Corrigé

- Les deux affichages sont :
 - `[12, 15, 20]` : seuls les scores supérieurs ou égaux à 10 sont conservés.
 - `3` : le tableau `resultat` contient 3 éléments.
- Ce programme **filtre** le tableau `scores` en ne conservant que les valeurs supérieures ou égales à 10. Il construit un nouveau tableau `resultat` contenant uniquement ces valeurs.

Vérification : les résultats ont été confirmés par exécution.

Corrigé

```
1 def moyenne(tab):
2     somme = 0
3     for val in tab:
4         somme = somme + val
5     return somme / len(tab)
```

On parcourt le tableau pour accumuler la somme des éléments, puis on divise par le nombre d'éléments (`len(tab)`). La division / renvoie un flottant.

Vérification : `moyenne([10, 20, 30])` renvoie 20.0 et `moyenne([12, 8, 15, 3, 20])` renvoie 11.6. Résultats confirmés par exécution.

Corrigé

- Les deux affichages sont :
 - `[12, 15, 14]` : la compréhension de liste ne garde que les notes ≥ 10 .
 - `[2, 4, 6]` : chaque élément de `[1, 2, 3]` est multiplié par 2.
- Construction de `tab` avec une boucle `for` et `append` :

```
1 tab = []
2 for n in notes:
3     if n >= 10:
4         tab.append(n)
```

Vérification : les résultats ont été confirmés par exécution.

Corrigé

```
1 def indice_maximum(tab):
2     # On suppose que l'indice du max est 0
3     idx = 0
4     # On parcourt les indices à partir de 1
5     for i in range(1, len(tab)):
6         if tab[i] > tab[idx]:
7             idx = i
8     return idx
```

Principe : on initialise `idx` à 0, puis on parcourt le tableau à partir de l'indice 1. Si l'élément courant est *strictement* supérieur à `tab[idx]`, on met à jour `idx`. Le test strict `>` (et non `>=`) garantit qu'en cas d'égalité, on conserve le plus petit indice.

Vérification : `indice_maximum([3, 7, 2, 9, 1])` renvoie 3 et `indice_maximum([5, 5, 5])` renvoie 0. Résultats confirmés par exécution.

Corrigé

1. Le programme affiche `[3, 1, 2]`.

Détail du parcours :

- ▶ `i = 0` : 3 n'est pas dans `resultat`, on l'ajoute → `[3]`.
- ▶ `i = 1` : 1 n'est pas dans `resultat`, on l'ajoute → `[3, 1]`.
- ▶ `i = 2` : 3 est déjà dans `resultat`, on ne l'ajoute pas.
- ▶ `i = 3` : 2 n'est pas dans `resultat`, on l'ajoute → `[3, 1, 2]`.
- ▶ `i = 4` : 1 est déjà dans `resultat`, on ne l'ajoute pas.
- ▶ `i = 5` : 3 est déjà dans `resultat`, on ne l'ajoute pas.

2. La fonction supprime les doublons d'un tableau en conservant l'ordre de première apparition de chaque élément.

3. Version parcourant directement les éléments :

```
1 def mystere(tab):
2     resultat = []
3     for element in tab:
4         if element not in resultat:
5             resultat.append(element)
6     return resultat
```

Vérification : les résultats ont été confirmés par exécution.

Corrigé

```
1 def temperatures_anormales(relevés, seuil_bas, seuil_haut):
2     # Tableau des anomalies (indice, valeur)
3     anormales = []
4     for i in range(len(relevés)):
5         # Hors intervalle [seuil_bas, seuil_haut]
6         if relevés[i] < seuil_bas or relevés[i] > seuil_haut:
7             anormales.append((i, relevés[i]))
8     return anormales
```

Principe : on parcourt le tableau par indice avec `range(len(relevés))` car on a besoin à la fois de l'indice `i` et de la valeur `relevés[i]`. Pour chaque relevé hors de l'intervalle autorisé, on ajoute le tuple `(i, relevés[i])` au résultat.

Vérification : `temperatures_anormales([15, 42, 18, -3, 22], 0, 40)` renvoie `[(1, 42), (3, -3)]`. Résultat confirmé par exécution.

Corrigé

1. Le programme affiche `[40, 10, 20, 30]`.

Détail des itérations (on sauvegarde d'abord dernier = 40) :

- ▶ `i = 3` : `tab[3] = tab[2]` → `[10, 20, 30, 30]`
- ▶ `i = 2` : `tab[2] = tab[1]` → `[10, 20, 20, 30]`
- ▶ `i = 1` : `tab[1] = tab[0]` → `[10, 10, 20, 30]`

Puis `tab[0] = dernier` → `[40, 10, 20, 30]`.

- La fonction effectue une **rotation droite** du tableau : le dernier élément passe en première position et tous les autres sont décalés d'une position vers la droite.
- La fonction ne renvoie rien (pas de `return`) : elle renvoie implicitement `None`. Le tableau est modifié **en place** (mutation) car les listes sont des objets mutables. C'est pourquoi `scores` est modifié après l'appel, sans avoir besoin de récupérer une valeur de retour.

Vérification : les résultats ont été confirmés par exécution.

Corrigé

- Les affichages produits sont :

▶ Alice : 14 (car $14 \geq 10$).

▶ Clara : 17 (car $17 \geq 10$).

Bob n'apparaît pas car sa note (8) est inférieure à 10.

- Les deux tableaux sont des **tableaux parallèles** : l'élément d'indice `i` dans `eleves` correspond à l'élément de même indice dans `notes`. Par exemple, `eleves[0]` ("Alice") a pour note `notes[0]` (14).
- On utilise `range(len(eleves))` car on a besoin de l'indice `i` pour accéder simultanément à `eleves[i]` et `notes[i]`. Avec `for e in eleves`, on obtiendrait les noms mais on ne pourrait pas accéder à la note correspondante dans `notes`.

Vérification : les résultats ont été confirmés par exécution.

Corrigé

```

1 def fusionner(tab1, tab2):
2     resultat = []
3     i = 0 # indice de parcours de tab1
4     j = 0 # indice de parcours de tab2
5     # Tant que les deux tableaux ont des elements a traiter
6     while i < len(tab1) and j < len(tab2):
7         if tab1[i] <= tab2[j]:
8             resultat.append(tab1[i])
9             i = i + 1
10        else:
11            resultat.append(tab2[j])
12            j = j + 1
13    # On ajoute les elements restants de tab1
14    while i < len(tab1):
15        resultat.append(tab1[i])
16        i = i + 1
17    # On ajoute les elements restants de tab2
18    while j < len(tab2):
19        resultat.append(tab2[j])
20        j = j + 1
21    return resultat

```

Principe : on utilise deux indices `i` et `j` parcourant respectivement `tab1` et `tab2`. À chaque étape, on compare les éléments courants et on ajoute le plus petit au résultat. Quand l'un des tableaux est entièrement parcouru, on ajoute les éléments restants de l'autre.

Le test `<=` (plutôt que `<`) assure la **stabilité** : en cas d'égalité, l'élément de `tab1` est ajouté en premier.

Vérification : `fusionner([1, 3, 5], [2, 4, 6])` renvoie `[1, 2, 3, 4, 5, 6]`. Résultat confirmé par exécution.

Corrigé

```

1 def moyennes_glissantes(relevés, k):
2     if len(relevés) < k:
3         return []
4     résultat = []
5     for i in range(len(relevés) - k + 1):
6         # Calcul de la moyenne des k éléments à partir de i
7         somme = 0
8         for j in range(k):
9             somme = somme + relevés[i + j]
10            résultat.append(somme / k)
11    return résultat

```

Principe : pour chaque position i de 0 à $\text{len}(\text{relevés}) - k$, on calcule la moyenne des k éléments consécutifs $\text{relevés}[i]$, $\text{relevés}[i+1]$, ..., $\text{relevés}[i+k-1]$. Le tableau résultat contient $\text{len}(\text{relevés}) - k + 1$ valeurs.

Exemple détaillé avec $\text{relevés} = [10, 20, 30, 40, 50]$ et $k = 3$:

- ▶ $i = 0$: moyenne de 10, 20, 30 = 20.0
- ▶ $i = 1$: moyenne de 20, 30, 40 = 30.0
- ▶ $i = 2$: moyenne de 30, 40, 50 = 40.0

Vérification : `moyennes_glissantes([10, 20, 30, 40, 50], 3)` renvoie `[20.0, 30.0, 40.0]`. Résultat confirmé par exécution.

Corrigé

1. Les quatre affichages sont :
 - ▶ `grille[0][1]` vaut 2 (ligne 0, colonne 1).
 - ▶ `grille[1][2]` vaut 6 (ligne 1, colonne 2).
 - ▶ `len(grille)` vaut 2 (nombre de lignes).
 - ▶ `len(grille[0])` vaut 3 (nombre de colonnes).
2. La grille possède 2 lignes et 3 colonnes.
3. L'expression `grille[1][0]` permet d'accéder à la valeur 4 (ligne d'indice 1, colonne d'indice 0).

Corrigé

1. Le programme affiche 193.
En effet, la double boucle parcourt toutes les valeurs de la grille et les accumule dans `total` : $20 + 22 + 19 + 25 + 28 + 23 + 18 + 17 + 21 = 193$.
2. Pour afficher la moyenne, on divise le total par le nombre de cases. La grille contient $3 \times 3 = 9$ valeurs :

```

1 carte = [[20, 22, 19],
2           [25, 28, 23],
3           [18, 17, 21]]
4 total = 0
5 nb = 0
6 for ligne in carte:
7     for val in ligne:
8         total = total + val
9         nb = nb + 1
10 print(total / nb)

```

L'affichage est 21.44444444444443.

Corrigé

```

1 def creer_grille(n, p, valeur):
2     grille = []
3     for i in range(n):
4         ligne = []
5         for j in range(p):
6             ligne.append(valeur)
7         grille.append(ligne)
8     return grille

```

L'écriture `[[valeur] * p] * n` est incorrecte car l'opérateur `*` sur une liste crée `n` références vers la même sous-liste. Modifier une case d'une ligne modifierait toutes les lignes (effet de bord dû à l'aliasing).

Corrigé

1. Les trois affichages sont :

```

[1, 0, 0]
[0, 1, 0]
[0, 0, 1]

```

La boucle `for i in range(3)` place un 1 à la position `grille[i][i]`, c'est-à-dire sur la diagonale principale.

2. Cette grille représente la **matrice identité** d'ordre 3 (notée I_3), avec des 1 sur la diagonale principale et des 0 partout ailleurs.

Corrigé

On initialise le maximum avec la première case, puis on parcourt toute la grille en mettant à jour la position et la valeur du maximum :

```

1 def maximum_grille(grille):
2     maxi = grille[0][0]
3     ligne_max = 0
4     col_max = 0
5     for i in range(len(grille)):
6         for j in range(len(grille[i])):
7             if grille[i][j] > maxi:
8                 maxi = grille[i][j]
9                 ligne_max = i
10                col_max = j
11    return (ligne_max, col_max, maxi)

```

```

>>> maximum_grille([[1, 5], [9, 3]])
(1, 0, 9)

```

On utilise `range(len(...))` pour disposer des indices `i` et `j`, nécessaires au renvoi de la position.

Corrigé

La colonne `j` de la grille d'entrée devient la ligne `j` de la grille résultat. On construit une nouvelle grille sans modifier l'originale :

```

1 def transposer(grille):
2     n = len(grille)

```

```

3  p = len(grille[0])
4  resultat = []
5  for j in range(p):
6      ligne = []
7      for i in range(n):
8          ligne.append(grille[i][j])
9          resultat.append(ligne)
10 return resultat

```

```

>>> transposer([[1, 2, 3], [4, 5, 6]])
[[1, 4], [2, 5], [3, 6]]

```

Si la grille d'entrée a n lignes et p colonnes, la transposée a p lignes et n colonnes.

Corrigé

On explore les 8 cases adjacentes en combinant les décalages $-1, 0$ et 1 sur les deux axes, en excluant le décalage $(0,0)$ qui correspond à la case elle-même. On vérifie que les indices restent dans les bornes de la grille :

```

1 def compter_voisins(grille, lig, col):
2     compteur = 0
3     for di in [-1, 0, 1]:
4         for dj in [-1, 0, 1]:
5             if di == 0 and dj == 0:
6                 continue
7             ni = lig + di
8             nj = col + dj
9             if 0 ≤ ni < len(grille) and 0 ≤ nj < len(grille[0]):
10                if grille[ni][nj] == 1:
11                    compteur = compteur + 1
12 return compteur

```

```

>>> g = [[0, 1, 0], [1, 1, 0], [0, 0, 1]]
>>> compter_voisins(g, 1, 1)
3
>>> compter_voisins(g, 0, 0)
3

```

La case $(1,1)$ a 8 voisins potentiels ; trois d'entre eux valent 1 : $(0,1)$, $(1,0)$ et $(2,2)$, d'où le résultat 3. La case $(0,0)$ est un coin : elle n'a que 3 voisins $((0,1)$, $(1,0)$ et $(1,1))$, qui valent tous 1, d'où le résultat 3.

Corrigé

1. Le programme calcule la moyenne de chaque ligne :

- ▶ Ligne 0 : $(10 + 8 + 12)/3 = 10,0$
- ▶ Ligne 1 : $(15 + 11 + 9)/3 \approx 11,67$
- ▶ Ligne 2 : $(7 + 14 + 13)/3 \approx 11,33$

L'affichage est : `[10.0, 11.666666666666666, 11.333333333333334]`.

2. Pour n'afficher que la moyenne la plus élevée, on remplace la dernière ligne par :

```

1 print(max(moyennes))

```

L'affichage est alors 11.666666666666666 (l'équipe 2, d'indice 1).

Corrigé

1. On construit une nouvelle grille de même taille en comparant chaque valeur au seuil :

```
1 def zone_chaude(carte, seuil):
2     n = len(carte)
3     p = len(carte[0])
4     zone = []
5     for i in range(n):
6         ligne = []
7         for j in range(p):
8             if carte[i][j] ≥ seuil:
9                 ligne.append(1)
10            else:
11                ligne.append(0)
12            zone.append(ligne)
13     return zone
```

2. Avec la carte donnée et un seuil de 30 :

```
>>> carte = [[30, 25, 32], [28, 35, 31], [22, 19, 27]]
>>> zone_chaude(carte, 30)
[[1, 0, 1], [0, 1, 1], [0, 0, 0]]
```

Les cases à 30, 32, 35 et 31 sont marquées 1, les autres 0.

3. Fonction comptant les cases à 1 :

```
1 def compter_cases(grille):
2     compteur = 0
3     for ligne in grille:
4         for val in ligne:
5             if val == 1:
6                 compteur = compteur + 1
7     return compteur
```

```
>>> compter_cases([[1, 0, 1], [0, 1, 1], [0, 0, 0]])
4
```

Corrigé

1. Grille initiale et après rotation de 90 degrés dans le sens horaire :

1	2		3	1
3	4	rotation 90° →	4	2

La colonne j de la grille d'origine, lue de bas en haut, devient la ligne j de la grille résultat.

2. Fonction de rotation :

```
1 def rotation_90(grille):
2     n = len(grille)
3     p = len(grille[0])
4     resultat = []
5     for j in range(p):
6         ligne = []
7         for i in range(n - 1, -1, -1):
8             ligne.append(grille[i][j])
9         resultat.append(ligne)
10    return resultat
```



```
»> rotation_90([[1, 2], [3, 4]])
[[3, 1], [4, 2]]
```

3. Vérification : quatre rotations successives redonnent la grille initiale.

```
1 g = [[1, 2, 3], [4, 5, 6]]
2 r = g
3 for _ in range(4):
4     r = rotation_90(r)
5 assert r == g # True
```

Corrigé

- Les trois affichages sont :
 - ▶ Alice — valeur associée à la clé "nom".
 - ▶ 14.5 — valeur associée à la clé "moyenne".
 - ▶ 3 — le dictionnaire contient trois paires clé-valeur.
- Les clés sont "nom", "classe" et "moyenne". Les valeurs correspondantes sont "Alice", "1NSI" et 14.5.
- Pour ajouter une nouvelle paire clé-valeur :

```
1 eleve["email"] = "alice@lycee.fr"
```

Corrigé

- Les trois affichages sont :
 - ▶ 30 — la clé "temp" existait déjà ; sa valeur a été remplacée par 30.
 - ▶ True — la clé "humidite" a été ajoutée, elle est bien présente.
 - ▶ False — la clé "pluie" n'existe pas dans le dictionnaire.
- La syntaxe est identique : dico[cle] = valeur. Si la clé existe déjà, la valeur est remplacée (modification). Si la clé n'existe pas, une nouvelle paire clé-valeur est créée (ajout). Python ne distingue pas les deux cas syntaxiquement.

Corrigé

On construit et renvoie directement le dictionnaire :

```
1 def creer_joueur(nom, elo, rang):
2     return {"nom": nom, "elo": elo, "rang": rang}
```

```
»> creer_joueur("Alice", 2400, "diamant")
{'nom': 'Alice', 'elo': 2400, 'rang': 'diamant'}
```

Corrigé

- Le programme affiche :

```
nom : Paris
temp : 25
vent : 10
```

- La boucle `for cle in station` parcourt uniquement les **clés** du dictionnaire. Pour accéder aux valeurs, on utilise `station[cle]`.
- Avec `station.items()`, on obtient directement les paires (clé, valeur) :

```
1 for cle, valeur in station.items():
2     print(cle, ":", valeur)
```

L'affichage produit est identique.

Corrigé

On parcourt le tableau et, pour chaque élément, on teste s'il est déjà dans le dictionnaire. Si oui, on incrémente son compteur ; sinon, on l'initialise à 1.

```
1 def compter_occurrences(tab):
2     compteur = {}
3     for element in tab:
4         if element in compteur:
5             compteur[element] = compteur[element] + 1
6         else:
7             compteur[element] = 1
8     return compteur
```

Déroulement sur l'exemple `["a", "b", "a", "c", "b", "a"]` :

- ▶ "a" absent → compteur = {"a": 1}
- ▶ "b" absent → compteur = {"a": 1, "b": 1}
- ▶ "a" présent → compteur = {"a": 2, "b": 1}
- ▶ "c" absent → compteur = {"a": 2, "b": 1, "c": 1}
- ▶ "b" présent → compteur = {"a": 2, "b": 2, "c": 1}
- ▶ "a" présent → compteur = {"a": 3, "b": 2, "c": 1}

Corrigé

- Le programme affiche :

```
{'x': 1, 'y': 10, 'z': 3}
```

- Lorsqu'une clé est présente dans les deux dictionnaires (ici "y"), la valeur du second dictionnaire `d2` écrase celle du premier. En effet, la seconde boucle exécute `resultat["y"] = d2["y"]`, soit `resultat["y"] = 10`, qui remplace la valeur 2 précédemment copiée depuis `d1`.
- Non, les dictionnaires `a` et `b` ne sont pas modifiés. La fonction crée un nouveau dictionnaire `resultat` et n'écrit jamais dans `d1` ni `d2`. Après l'appel, `a == {"x": 1, "y": 2}` et `b == {"y": 10, "z": 3}`.

Corrigé

On utilise deux dictionnaires auxiliaires : `totaux` pour la somme des notes et effectifs pour le nombre d'élèves par classe. On calcule ensuite chaque moyenne.

```
1 def moyenne_par_classe(eleves):
2     totaux = {}
3     effectifs = {}
4     for e in eleves:
5         c = e["classe"]
6         if c not in totaux:
7             totaux[c] = 0
8             effectifs[c] = 0
```

```

9     totaux[c] = totaux[c] + e["note"]
10    effectifs[c] = effectifs[c] + 1
11    moyennes = {}
12    for c in totaux:
13        moyennes[c] = totaux[c] / effectifs[c]
14    return moyennes

```

Vérification :

```

>> moyenne_par_classe(elevés)
{'1A': 13.0, '1B': 15.0}

```

Pour la classe "1A", la somme vaut $12 + 14 = 26$ et l'effectif vaut 2, d'où une moyenne de $26/2 = 13,0$.

Corrigé

1. Le programme produit une erreur `KeyError: 'rang'`. La clé "rang" n'existe pas dans le dictionnaire joueur (qui ne contient que "nom" et "elo"). Accéder à une clé inexistante avec la notation `dico[cle]` provoque cette erreur.

2. Deux façons d'éviter l'erreur :

Méthode 1 — tester l'existence avec in :

```

1 if "rang" in joueur:
2     print(joueur["rang"])

```

Méthode 2 — utiliser get :

```

1 print(joueur.get("rang"))

```

La méthode `get` renvoie `None` si la clé n'existe pas, sans provoquer d'erreur.

3. Pour afficher "inconnu" par défaut :

```

1 print(joueur.get("rang", "inconnu"))

```

Le second argument de `get` est la valeur renvoyée si la clé est absente.

Corrigé

On parcourt les paires (clé, valeur) avec `d.items()` et on construit un nouveau dictionnaire en inversant les rôles :

```

1 def inverser_dico(d):
2     resultat = {}
3     for cle, valeur in d.items():
4         resultat[valeur] = cle
5     return resultat

```

Déroulement sur l'exemple {"a": 1, "b": 2, "c": 3} :

- ▶ `cle = "a", valeur = 1 → resultat[1] = "a"`
- ▶ `cle = "b", valeur = 2 → resultat[2] = "b"`
- ▶ `cle = "c", valeur = 3 → resultat[3] = "c"`

Résultat : {1: "a", 2: "b", 3: "c"}.

Remarque : cette méthode suppose que les valeurs sont toutes distinctes. Si deux clés avaient la même valeur, seule la dernière serait conservée dans le dictionnaire inversé.

Corrigé

On parcourt chaque match en déballant le tuple. Pour chaque équipe rencontrée pour la première fois, on initialise son compteur à 0. Puis on attribue les points selon le résultat.

```

1 def classement_tournoi(matches):
2     points = {}
3     for equipe1, score1, equipe2, score2 in matches:
4         if equipe1 not in points:
5             points[equipe1] = 0
6         if equipe2 not in points:
7             points[equipe2] = 0
8         if score1 > score2:
9             points[equipe1] = points[equipe1] + 3
10        elif score1 < score2:
11            points[equipe2] = points[equipe2] + 3
12        else:
13            points[equipe1] = points[equipe1] + 1
14            points[equipe2] = points[equipe2] + 1
15    return points

```

Déroulement sur l'exemple :

- ▶ ("A", 2, "B", 1) : A gagne ($2 > 1$) → A reçoit 3 points.
- ▶ ("A", 0, "C", 0) : match nul → A et C reçoivent chacun 1 point.
- ▶ ("B", 3, "C", 1) : B gagne ($3 > 1$) → B reçoit 3 points.

Résultat : {"A": 4, "B": 3, "C": 1}.

Corrigé

1. On regroupe les noms de stations par région dans un dictionnaire dont les valeurs sont des listes :

```

1 def stations_par_region(stations):
2     regions = {}
3     for s in stations:
4         r = s["region"]
5         if r not in regions:
6             regions[r] = []
7         regions[r].append(s["nom"])
8     return regions

```

Le point clé est l'initialisation : quand une région apparaît pour la première fois, on crée une liste vide avant d'y ajouter le nom de la station.

2. On calcule la température moyenne par région, puis on cherche le maximum :

```

1 def region_la_plus_chaude(stations):
2     totaux = {}
3     effectifs = {}
4     for s in stations:
5         r = s["region"]
6         if r not in totaux:
7             totaux[r] = 0
8             effectifs[r] = 0
9         totaux[r] = totaux[r] + s["temp"]
10        effectifs[r] = effectifs[r] + 1
11    meilleure = None
12    meilleure_moy = -1
13    for r in totaux:
14        moy = totaux[r] / effectifs[r]

```

```

15     if moy > meilleure_moy:
16         meilleure_moy = moy
17         meilleure = r
18     return meilleure

```

Sur l'exemple : la moyenne ARA vaut $(30 + 27)/2 = 28,5$ et la moyenne IDF vaut $28/1 = 28$. La région la plus chaude est donc "ARA".

Corrigé

1. Le programme affiche :

```

[1, 2, 3, 4]
True

```

2. L'affectation `b = a` ne crée pas une nouvelle liste : elle fait pointer `b` vers *le même objet* que `a`. On dit que `b` est un **alias** de `a`. Toute modification via `b` (ici `b.append(4)`) modifie l'unique liste partagée, donc `a` voit aussi le changement.
3. L'opérateur `is` teste si deux noms désignent **le même objet en mémoire** (même adresse), et non simplement si leurs valeurs sont égales. Ici `a is b` vaut `True` car `a` et `b` sont deux références vers la même liste.

Corrigé

1. Le programme affiche :

```

[1, 2, 3]
[1, 2, 3, 4]
False

```

2. Avec `b = a`, les deux noms désignent le même objet (alias). Avec `b = list(a)`, on crée une **nouvelle liste** contenant les mêmes éléments : `a` et `b` sont deux objets distincts. Modifier `b` ne modifie donc pas `a`.
3. Deux autres façons de créer une copie superficielle d'une liste :
 - ▶ `b = a[:]` (tranche complète) ;
 - ▶ `b = a.copy()` (méthode `copy`).

Corrigé

1. Le programme affiche :

```

[10, 20, 30]

```

2. Lorsqu'on passe une liste à une fonction, le paramètre `tab` reçoit une **référence** vers le même objet que `scores`. L'appel `tab.append(val)` modifie donc directement la liste d'origine, sans qu'un `return` soit nécessaire.
3. Un **effet de bord** est une modification d'un état extérieur à la fonction (ici la liste `scores`) produite par l'exécution de la fonction. La fonction ne se contente pas de calculer une valeur : elle modifie un objet mutable partagé. Les effets de bord rendent le code plus difficile à comprendre car le résultat ne dépend pas uniquement de la valeur de retour.

Corrigé

1. L'exécution provoque une erreur `TypeError: 'tuple' object does not support item assignment`. La ligne `t[0] = 10` échoue et le `print(t)` n'est jamais atteint.
2. Un tuple est **immuable** (non mutable) : une fois créé, on ne peut ni modifier, ni ajouter, ni supprimer ses éléments. Les opérations d'affectation par index (`t[i] = ...`), `append`, `remove`, etc., ne sont pas définies pour les tuples.
3. Utiliser un tuple garantit que les données ne seront pas modifiées par erreur (ni par un alias, ni par un effet de bord dans une fonction). Cela rend le code plus sûr et exprime clairement l'intention du programmeur : ces données sont fixes.

Corrigé

1. Le programme affiche :

```
[1, 0]
[1, 0]
[1, 0]
```

Ce n'est pas le résultat attendu : on voulait modifier uniquement la première ligne, mais les trois lignes sont identiques.

2. L'opérateur `*` appliqué à une liste ne crée pas trois sous-listes indépendantes : il duplique trois fois **la même référence** vers l'unique sous-liste `[0, 0]`. Les trois éléments de grille sont des alias du même objet. Modifier `grille[0][0]` modifie cet objet unique, ce qui se répercute sur les trois positions.
3. On construit la grille avec une liste en compréhension, qui crée une nouvelle sous-liste à chaque itération :

```
1 grille = [[0, 0] for _ in range(3)]
```

Ainsi, chaque ligne est un objet distinct et modifier `grille[0][0]` n'affecte que la première ligne.

Corrigé

1. Avec l'appel `supprimer_negatifs_v1([5, -2, -3, 8])` :
 - Itération sur l'index 0 : `val = 5`, positif, rien ne se passe.
 - Index 1 : `val = -2`, négatif, `tab.remove(-2)` supprime `-2`. La liste devient `[5, -3, 8]`.
 - Index 2 : l'itérateur passe à l'index 2, qui vaut maintenant 8. La valeur `-3` (à l'index 1) est sautée.

Le résultat est `[5, -3, 8]` : incorrect, car `-3` n'a pas été supprimé.

Le problème : on modifie une liste *pendant* qu'on itère dessus. La suppression décale les éléments, et l'itérateur saute l'élément suivant.

2. La version corrigée crée une **nouvelle liste** `resultat` et y ajoute uniquement les valeurs positives ou nulles. Elle n'itère pas sur la liste qu'elle modifie, ce qui évite le problème de décalage d'indices. De plus, le tableau d'origine `temperatures` n'est **pas modifié** : la fonction est sans effet de bord.

Corrigé

1. Le programme affiche :

```
200
True
```

2. Oui, les dictionnaires sont **mutables**. On peut ajouter, modifier ou supprimer des paires clé-valeur après la création. Ici, `copie["score"] = 200` modifie le dictionnaire en place. Comme `copie = equipe` crée un alias (même objet), la modification est visible via `equipe` aussi.

3. Pour obtenir une copie indépendante, on utilise `dict()` :

```
1 equipe = {"nom": "Phoenix", "score": 100}
2 copie = dict(equipe)
3 copie["score"] = 200
4 print(equipe["score"]) # 100
5 print(copie["score"]) # 200
```

Après cette modification, `equipe["score"]` vaut toujours 100 car `copie` est un objet distinct.

Corrigé

1. Le programme affiche :

```
99
[99, 2]
```

2. `list(grille)` crée une nouvelle liste, mais ses éléments (les sous-listes) restent les **mêmes objets** que dans `grille`. Ainsi, `copie[0]` et `grille[0]` pointent vers la même sous-liste. Quand on exécute `copie[0][0] = 99`, on modifie cette sous-liste partagée, donc `grille[0][0]` vaut aussi 99.
3. Une **copie superficielle** duplique la structure de premier niveau (la liste externe) mais pas les objets contenus. Les sous-listes ne sont pas copiées : elles restent partagées entre l'original et la copie. La ligne `copie[0] = [10, 20]` remplace la référence dans `copie` sans toucher `grille`, mais une modification à l'intérieur d'une sous-liste partagée affecte les deux.
4. Sous la précondition que les cellules sont des valeurs scalaires atomiques non mutables, sans conteneur imbriqué, la fonction suivante réalise une **copie des deux premiers niveaux** :

```
1 def copier_grille_deux_niveaux(grille):
2     """Copie la liste externe et chaque ligne.
3
4     Precondition : les cellules sont des valeurs scalaires atomiques non mutables,
5     sans conteneur imbriqué.
6     """
7     return [list(ligne) for ligne in grille]
```

La garantie est limitée aux modifications de la structure externe, aux remplacements de lignes et aux réaffectations de cellules dans la copie : la liste externe et chaque ligne sont distinctes de celles de la grille d'origine.

5. Hors contrat, pour `[[[1]]]`, la cellule est elle-même une liste mutable. La liste externe et la ligne sont copiées, mais la cellule-liste reste partagée. Ajouter une valeur dans cette cellule par la copie modifie donc aussi la grille d'origine. La fonction ne fournit pas de garantie générale sur des niveaux plus profonds.

Corrigé

1. Sous la précondition que les cellules sont des valeurs scalaires atomiques non mutables, sans conteneur imbriqué, cette fonction réalise une **copie des deux premiers niveaux** :

```
1 def copier_grille_deux_niveaux(grille):
2     """Copie la liste externe et chaque ligne.
3
4     Precondition : les cellules sont des valeurs scalaires atomiques non mutables,
5     sans conteneur imbriqué.
6     """
7     return [list(ligne) for ligne in grille]
```

La compréhension crée une nouvelle liste externe et `list(ligne)` crée une nouvelle liste pour chaque ligne. La garantie est limitée aux modifications de la structure externe, aux remplacements de lignes et aux réaffectations de cellules dans la copie : elles ne modifient pas la grille d'origine.

2. Le scénario numérique est :

```
1 g = [[1, 2], [3, 4]]
2 c = copier_grille_deux_niveaux(g)
3 c[0][0] = 99
4 print(g[0][0])
```

1

La liste externe et chaque ligne de `c` sont distinctes de celles de `g`. La réaffectation de la cellule de `c` ne modifie donc pas la ligne correspondante de `g`.

3. Le cas `[[[1]]]` est hors contrat : la cellule-liste reste partagée. Seuls la liste externe et le niveau des lignes sont copiés. Une mutation de cette liste interne par la copie affecte aussi l'original ; la fonction ne fournit aucune garantie générale sur des niveaux plus profonds.

Corrigé

1. Voici la fonction :

```
1 def historique_scores(equipes, resultats):
2     """Renvoie une nouvelle liste d'équipes avec les
3     scores ajoutés.
4
5     Precondition: equipes est une liste de dicos avec
6     cles "nom" et "scores".
7     Precondition: resultats est une liste de tuples
8     (nom, score).
9     Ne modifie pas la liste d'origine.
10    """
11    copie = []
12    for e in equipes:
13        copie.append({
14            "nom": e["nom"],
15            "scores": list(e["scores"])
16        })
17    for nom, score in resultats:
18        for e in copie:
19            if e["nom"] == nom:
20                e["scores"].append(score)
21    return copie
```

2. Test :

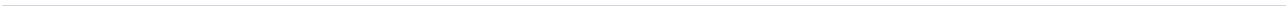
```
1 equipes = [{"nom": "A", "scores": [10, 20]},
2             {"nom": "B", "scores": [15]}]
3 nouvelles = historique_scores(
4     equipes, [("A", 30), ("B", 25)]
5 )
6 print(equipes[0]["scores"]) # [10, 20]
7 print(nouvelles[0]["scores"]) # [10, 20, 30]
```

La liste d'origine n'est pas modifiée.

3. Il faut copier **à la fois** les dictionnaires et les listes de scores car les deux sont des objets mutables :
- Si on ne copie pas les dictionnaires (par exemple `copie = list(equipes)`), modifier une clé dans `copie[0]` modifie aussi `equipes[0]` (même dictionnaire).

- Si on copie les dictionnaires mais pas les listes de scores (par exemple `copie.append(dict(e))`), la clé "scores" du nouveau dictionnaire pointe vers la **même liste** que dans l'original. Un `append` sur cette liste affecte les deux.

Il faut donc créer un nouveau dictionnaire **et** une nouvelle liste de scores avec `list(e["scores"])` pour garantir l'absence d'effet de bord.



3 Traitement de données en tables

3.1 Importer une table

DÉFINITION D1

Une **table** est une liste de dictionnaires ayant tous les mêmes clés. Chaque dictionnaire de la liste représente une **ligne** de la table ; chaque clé représente une **colonne**.

EXEMPLE Les adhérents d'un club sportif, sous forme de table :

```
1 adherents = [  
2     {"nom": "Ali", "age": 16, "categorie": "junior", "cotisation": 80},  
3     {"nom": "Sami", "age": 15, "categorie": "junior", "cotisation": 80},  
4     {"nom": "Nour", "age": 17, "categorie": "junior", "cotisation": 80},  
5     {"nom": "Yasmine", "age": 22, "categorie": "senior", "cotisation": 120},  
6     {"nom": "Karim", "age": 19, "categorie": "senior", "cotisation": 120},  
7 ]
```

DÉFINITION D2

Le format **CSV** (*comma-separated values*) représente une table dans un fichier texte : la première ligne contient les noms des colonnes (l'en-tête), chaque ligne suivante contient les valeurs séparées par une virgule (ou un autre séparateur, par exemple une tabulation pour un fichier « texte tabulé »).

EXEMPLE Le fichier `adherents.csv` correspondant à la table ci-dessus :

```
nom,age,categorie,cotisation  
Ali,16,junior,80  
Sami,15,junior,80  
Nour,17,junior,80  
Yasmine,22,senior,120  
Karim,19,senior,120
```

CODE DE RÉFÉRENCE — Importer un fichier CSV en table

```
1 import csv  
2  
3 def importer_csv(nom_fichier):  
4     """Importe un fichier CSV et renvoie une table (liste de dictionnaires).  
5  
6     Precondition : le fichier existe, la première ligne contient l'en-tête.  
7     """  
8     with open(nom_fichier, encoding="utf-8") as f:  
9         lecteur = csv.DictReader(f)
```

```

10     return list(lecteur)
11
12 table = importer_csv("adherents.csv")
13 print(table[0]) # {'nom': 'Ali', 'age': '16', 'categorie': 'junior', 'cotisation':
                  '80'}

```

Ali,16,junior,80 Sami,15,junior,80"

⚠ ERREUR FRÉQUENTE

Oublier que les valeurs importées d'un CSV sont **toujours des chaînes de caractères** (str), même si elles « ressemblent » à des nombres. `table[0]["age"]` vaut la chaîne "16", pas l'entier 16 : il faut convertir explicitement avec `int(...)` ou `float(...)` avant de faire des calculs ou des comparaisons numériques.

CODE DE RÉFÉRENCE — Convertir les types après import

```

1 def convertir_types(table):
2     """Convertit les colonnes numeriques age et cotisation en entiers."""
3     for ligne in table:
4         ligne["age"] = int(ligne["age"])
5         ligne["cotisation"] = int(ligne["cotisation"])
6     return table

```

» **Python À la machine.** Crée un fichier `adherents.csv` avec le contenu de l'exemple ci-dessus, importe-le avec `importer_csv`, puis convertis les colonnes `age` et `cotisation` en entiers avec `convertir_types`. Vérifie avec `type(table[0]["age"])` que la conversion a bien eu lieu.

3.2 Recherche dans une table

◆ **PROPRIÉTÉ Recherche par critère** Rechercher les lignes d'une table vérifiant un critère consiste à parcourir la table et à ne conserver que les lignes pour lesquelles une **expression booléenne** (le critère) est vraie. On peut combiner plusieurs conditions avec `and`, `or`, `not`.

CODE DE RÉFÉRENCE — Recherche par critère (compréhension de liste)

```

1 adherents = [
2     {"nom": "Ali", "age": 16, "categorie": "junior", "cotisation": 80},
3     {"nom": "Sami", "age": 15, "categorie": "junior", "cotisation": 80},
4     {"nom": "Nour", "age": 17, "categorie": "junior", "cotisation": 80},
5     {"nom": "Yasmine", "age": 22, "categorie": "senior", "cotisation": 120},
6     {"nom": "Karim", "age": 19, "categorie": "senior", "cotisation": 120},
7 ]
8
9 def rechercher(table, critere):
10     """Renvoie les lignes de 'table' pour lesquelles critere(ligne) est vrai."""
11     return [ligne for ligne in table if critere(ligne)]
12

```

```

13 juniors_16plus = rechercher(adherents, lambda ligne: ligne["categorie"] == "junior"
    and ligne["age"] >= 16)
14 print([ligne["nom"] for ligne in juniors_16plus]) # ['Ali', 'Nour']

```

◆ **PROPRIÉTÉ Recherche de doublons** Une table contient un **doublon** lorsque plusieurs lignes ont la même valeur pour une colonne censée être unique (par exemple un identifiant). On détecte les doublons en comptant les occurrences de chaque valeur de cette colonne.

CODE DE RÉFÉRENCE — Détecter les doublons d'une colonne

```

1 def doublons(table, colonne):
2     """Renvoie les valeurs de 'colonne' apparaissant plus d'une fois dans 'table'."""
3     valeurs = [ligne[colonne] for ligne in table]
4     return [v for v in set(valeurs) if valeurs.count(v) > 1]
5
6 adherents_avec_doublon = adherents + [{"nom": "Ali", "age": 16, "categorie": "junior", "cotisation": 80}]
7 print(doublons(adherents_avec_doublon, "nom")) # ['Ali']
8 print(doublons(adherents, "nom")) # []

```

◆ **PROPRIÉTÉ Test de cohérence** Un **test de cohérence** vérifie qu'une table respecte des contraintes attendues (types corrects, valeurs dans un intervalle plausible, absence de champ vide...). C'est une étape essentielle avant tout traitement automatisé de données réelles, souvent imparfaites.

CODE DE RÉFÉRENCE — Test de cohérence simple

```

1 def table_cohérente(table):
2     """Vérifie que chaque age est un entier positif raisonnable (0 à 120)."""
3     for ligne in table:
4         if not isinstance(ligne["age"], int) or not (0 <= ligne["age"] <= 120):
5             return False
6     return True
7
8 print(table_cohérente(adherents)) # True
9 print(table_cohérente(adherents + [{"nom": "X", "age": -5,
10     "categorie": "junior", "cotisation": 80}]))
    # False

```

⚠ ERREUR FRÉQUENTE

Rechercher des doublons en comparant chaque ligne **entière** (`ligne1 == ligne2`) alors que la consigne porte sur une seule colonne (comme nom). Deux lignes peuvent partager le même nom sans être identiques sur les autres colonnes (âge différent par exemple) : il faut comparer uniquement la colonne concernée.

» **Python À la machine.** Ajoute une ligne dupliquée (même nom qu'une ligne existante, mais age différent) à la table `adherents`. Utilise `doubletons` pour la détecter, puis écris une fonction `supprimer_doubletons(table, colonne)` qui ne garde que la première occurrence de chaque valeur de colonne.

3.3 Trier une table

♦ **PROPRIÉTÉ Tri suivant une colonne** Trier une table suivant une colonne consiste à réordonner ses lignes selon les valeurs de cette colonne (ordre croissant ou décroissant). En Python, la fonction `sorted` accepte un paramètre `key` : une fonction qui, pour chaque ligne, renvoie la valeur à utiliser pour le tri.

CODE DE RÉFÉRENCE — Trier une table avec `sorted`

```
1 adherents = [
2     {"nom": "Ali", "age": 16, "categorie": "junior", "cotisation": 80},
3     {"nom": "Sami", "age": 15, "categorie": "junior", "cotisation": 80},
4     {"nom": "Nour", "age": 17, "categorie": "junior", "cotisation": 80},
5     {"nom": "Yasmine", "age": 22, "categorie": "senior", "cotisation": 120},
6     {"nom": "Karim", "age": 19, "categorie": "senior", "cotisation": 120},
7 ]
8
9 tries_par_age = sorted(adherents, key=lambda ligne: ligne["age"])
10 print([ligne["nom"] for ligne in tries_par_age])
11 # ['Sami', 'Ali', 'Nour', 'Karim', 'Yasmine']
```

♦ **PROPRIÉTÉ Ordre décroissant** Pour trier en ordre **décroissant**, on ajoute le paramètre `reverse=True` à `sorted`.

EXEMPLE Trier les adhérents par cotisation décroissante :

⚠ ERREUR FRÉQUENTE

Écrire une fonction de tri « à la main » (par insertion ou sélection, vues au chapitre d'algorithmique) alors qu'une fonction intégrée existe et suffit largement. Le programme officiel autorise explicitement l'usage de `sorted` pour trier une table : implémenter son propre tri n'apporte rien ici et augmente le risque d'erreur.

DÉFINITION D3

`sorted(table, key=...)` renvoie une **nouvelle** liste triée, sans modifier `table`. La méthode `table.sort(key=...)`, elle, trie la liste **sur place** (elle modifie `table` et ne renvoie rien).

» **Python À la machine.** Trie la table `adherents` par ordre alphabétique du nom, puis par age décroissant. Vérifie, en affichant `adherents` avant et après un appel à `sorted` (pas `.sort()`), que la table d'origine n'est pas modifiée.

3.4 Fusionner deux tables

DÉFINITION D4

Le **domaine de valeurs** d'une colonne est l'ensemble des valeurs qu'elle peut prendre. Pour **fusionner** deux tables, on les combine en s'appuyant sur une colonne commune (une **clé**) dont le domaine de valeurs permet de faire correspondre les lignes de l'une avec celles de l'autre.

EXEMPLE On dispose d'une seconde table, les présences aux entraînements, partageant la colonne nom avec la table adherents. Les deux tables sont présentées dans le programme ci-dessous.

◆ **PROPRIÉTÉ Fusion par clé commune** Fusionner deux tables table1 et table2 suivant une colonne cle consiste à construire une nouvelle table où chaque ligne combine les colonnes des deux tables, pour les valeurs de cle présentes **dans les deux**. Une valeur de cle présente dans une seule des deux tables n'apparaît pas dans le résultat (elle est hors du domaine de valeurs commun).

Précondition : les colonnes hors clé des deux tables doivent être disjointes. Sinon, une colonne de la seconde table écraserait silencieusement la colonne de même nom dans la première table.

CODE DE RÉFÉRENCE — Fusionner deux tables par une colonne commune

```
1 def fusionner(table1, table2, cle):
2     """Fusionne table1 et table2 sur les valeurs communes de 'cle'.
3
4     Preconditions : 'cle' est presente dans les deux tables et les autres
5     colonnes sont disjointes.
6     """
7     colonnes1 = {nom for ligne in table1 for nom in ligne if nom != cle}
8     colonnes2 = {nom for ligne in table2 for nom in ligne if nom != cle}
9     if not colonnes1.isdisjoint(colonnes2):
10         raise ValueError("les colonnes hors cle doivent etre disjointes")
11     index2 = {ligne[cle]: ligne for ligne in table2}
12     fusion = []
13     for ligne1 in table1:
14         if ligne1[cle] in index2:
15             complement = {
16                 k: v for k, v in index2[ligne1[cle]].items() if k != cle
17             }
18             fusion.append(**ligne1, **complement)
19     return fusion
20
21
22 adherents = [
23     {"nom": "Ali", "age": 16},
24     {"nom": "Sami", "age": 15},
25     {"nom": "Yasmine", "age": 22},
26 ]
27 presences = [
28     {"nom": "Ali", "seances": 12},
29     {"nom": "Sami", "seances": 9},
30     {"nom": "Yasmine", "seances": 15},
31 ]
32 resultat = fusionner(adherents, presences, "nom")
33 print(resultat)
```

```
[{'nom': 'Ali', 'age': 16, 'seances': 12}, {'nom': 'Sami', 'age': 15, 'seances': 9},
 {'nom': 'Yasmine', 'age': 22, 'seances': 15}]
```

EXEMPLE Si presences ne contient pas d'entrée pour "Karim" et "Nour" (absents du domaine de valeurs commun), ces adhérents **n'apparaissent pas** dans la table fusionnée, même s'ils sont présents dans adherents :

⚠ ERREUR FRÉQUENTE

Croire que la fusion garde **toutes** les lignes des deux tables. Par défaut, seules les lignes dont la clé est présente **dans les deux** tables sont conservées (fusion sur le domaine de valeurs **commun**) : c'est le comportement attendu par le programme officiel, qui met l'accent sur cette notion de domaine de valeurs.

» **Python À la machine.** Ajoute une troisième table cotisations_payees (colonnes nom et montant_paye) avec seulement 3 des 5 adhérents. Fusionne-la avec adherents via fusionner, et vérifie que la table résultante ne contient que les adhérents présents dans les deux tables.

3.5 Importer une table

DÉFINITION D1

Une **table** est une liste de dictionnaires ayant tous les mêmes clés. Chaque dictionnaire de la liste représente une **ligne** de la table ; chaque clé représente une **colonne**.

EXEMPLE Les adhérents d'un club sportif, sous forme de table :

```
1 adherents = [
2     {"nom": "Ali", "age": 16, "categorie": "junior", "cotisation": 80},
3     {"nom": "Sami", "age": 15, "categorie": "junior", "cotisation": 80},
4     {"nom": "Nour", "age": 17, "categorie": "junior", "cotisation": 80},
5     {"nom": "Yasmine", "age": 22, "categorie": "senior", "cotisation": 120},
6     {"nom": "Karim", "age": 19, "categorie": "senior", "cotisation": 120},
7 ]
```

DÉFINITION D2

Le format **CSV** (*comma-separated values*) représente une table dans un fichier texte : la première ligne contient les noms des colonnes (l'en-tête), chaque ligne suivante contient les valeurs séparées par une virgule (ou un autre séparateur, par exemple une tabulation pour un fichier « texte tabulé »).

EXEMPLE Le fichier adherents.csv correspondant à la table ci-dessus :

```
nom,age,categorie,cotisation
Ali,16,junior,80
Sami,15,junior,80
Nour,17,junior,80
```



```
Yasmine,22,senior,120
Karim,19,senior,120
```

CODE DE RÉFÉRENCE — Importer un fichier CSV en table

```
1 import csv
2
3 def importer_csv(nom_fichier):
4     """Importe un fichier CSV et renvoie une table (liste de dictionnaires).
5
6     Precondition : le fichier existe, la première ligne contient l'en-tête.
7     """
8     with open(nom_fichier, encoding="utf-8") as f:
9         lecteur = csv.DictReader(f)
10        return list(lecteur)
11
12 table = importer_csv("adherents.csv")
13 print(table[0]) # {'nom': 'Ali', 'age': '16', 'categorie': 'junior', 'cotisation':
14                 '80'}
```

Ali,16,junior,80 Sami,15,junior,80"

⚠ ERREUR FRÉQUENTE

Oublier que les valeurs importées d'un CSV sont **toujours des chaînes de caractères** (str), même si elles « ressemblent » à des nombres. `table[0]["age"]` vaut la chaîne "16", pas l'entier 16 : il faut convertir explicitement avec `int(...)` ou `float(...)` avant de faire des calculs ou des comparaisons numériques.

CODE DE RÉFÉRENCE — Convertir les types après import

```
1 def convertir_types(table):
2     """Convertit les colonnes numeriques age et cotisation en entiers."""
3     for ligne in table:
4         ligne["age"] = int(ligne["age"])
5         ligne["cotisation"] = int(ligne["cotisation"])
6     return table
```

» **Python À la machine.** Crée un fichier `adherents.csv` avec le contenu de l'exemple ci-dessus, importe-le avec `importer_csv`, puis convertis les colonnes `age` et `cotisation` en entiers avec `convertir_types`. Vérifie avec `type(table[0]["age"])` que la conversion a bien eu lieu.

3.6 Recherche dans une table

◆ **PROPRIÉTÉ Recherche par critère** Rechercher les lignes d'une table vérifiant un critère consiste à parcourir la table et à ne conserver que les lignes pour lesquelles une **expression booléenne** (le critère) est vraie. On peut combiner plusieurs conditions avec `and`, `or`, `not`.

CODE DE RÉFÉRENCE — Recherche par critère (compréhension de liste)

```

1 adherents = [
2     {"nom": "Ali", "age": 16, "categorie": "junior", "cotisation": 80},
3     {"nom": "Sami", "age": 15, "categorie": "junior", "cotisation": 80},
4     {"nom": "Nour", "age": 17, "categorie": "junior", "cotisation": 80},
5     {"nom": "Yasmine", "age": 22, "categorie": "senior", "cotisation": 120},
6     {"nom": "Karim", "age": 19, "categorie": "senior", "cotisation": 120},
7 ]
8
9 def rechercher(table, critere):
10     """Renvoie les lignes de 'table' pour lesquelles critere(ligne) est vrai."""
11     return [ligne for ligne in table if critere(ligne)]
12
13 juniors_16plus = rechercher(adherents, lambda ligne: ligne["categorie"] == "junior"
14                             and ligne["age"] >= 16)
15 print([ligne["nom"] for ligne in juniors_16plus]) # ['Ali', 'Nour']

```

◆ **PROPRIÉTÉ Recherche de doublons** Une table contient un **doublon** lorsque plusieurs lignes ont la même valeur pour une colonne censée être unique (par exemple un identifiant). On détecte les doublons en comptant les occurrences de chaque valeur de cette colonne.

CODE DE RÉFÉRENCE — Détecter les doublons d'une colonne

```

1 def doublons(table, colonne):
2     """Renvoie les valeurs de 'colonne' apparaissant plus d'une fois dans 'table'."""
3
4     valeurs = [ligne[colonne] for ligne in table]
5     return [v for v in set(valeurs) if valeurs.count(v) > 1]
6
7 adherents_avec_doublon = adherents + [{"nom": "Ali", "age": 16, "categorie": "junior", "cotisation": 80}]
8 print(doublons(adherents_avec_doublon, "nom")) # ['Ali']
9 print(doublons(adherents, "nom")) # []

```

◆ **PROPRIÉTÉ Test de cohérence** Un **test de cohérence** vérifie qu'une table respecte des contraintes attendues (types corrects, valeurs dans un intervalle plausible, absence de champ vide...). C'est une étape essentielle avant tout traitement automatisé de données réelles, souvent imparfaites.

CODE DE RÉFÉRENCE — Test de cohérence simple

```

1 def table_cohérente(table):
2     """Vérifie que chaque age est un entier positif raisonnable (0 à 120)."""
3     for ligne in table:
4         if not isinstance(ligne["age"], int) or not (0 <= ligne["age"] <= 120):
5             return False
6     return True
7
8 print(table_cohérente(adherents)) # True
9 print(table_cohérente(adherents + [{"nom": "X", "age": -5,

```

```
10 | "categorie": "junior", "cotisation": 80}]))
    | # False
```

⚠ ERREUR FRÉQUENTE

Rechercher des doublons en comparant chaque ligne **entière** (ligne1 == ligne2) alors que la consigne porte sur une seule colonne (comme nom). Deux lignes peuvent partager le même nom sans être identiques sur les autres colonnes (âge différent par exemple) : il faut comparer uniquement la colonne concernée.

» **Python À la machine.** Ajoute une ligne dupliquée (même nom qu'une ligne existante, mais âge différent) à la table adherents. Utilise doublons pour la détecter, puis écris une fonction supprimer_doublons(table, colonne) qui ne garde que la première occurrence de chaque valeur de colonne.

3.7 Trier une table

◆ **PROPRIÉTÉ Tri suivant une colonne** Trier une table suivant une colonne consiste à réordonner ses lignes selon les valeurs de cette colonne (ordre croissant ou décroissant). En Python, la fonction sorted accepte un paramètre key : une fonction qui, pour chaque ligne, renvoie la valeur à utiliser pour le tri.

CODE DE RÉFÉRENCE — Trier une table avec sorted

```
1 | adherents = [
2 |     {"nom": "Ali", "age": 16, "categorie": "junior", "cotisation": 80},
3 |     {"nom": "Sami", "age": 15, "categorie": "junior", "cotisation": 80},
4 |     {"nom": "Nour", "age": 17, "categorie": "junior", "cotisation": 80},
5 |     {"nom": "Yasmine", "age": 22, "categorie": "senior", "cotisation": 120},
6 |     {"nom": "Karim", "age": 19, "categorie": "senior", "cotisation": 120},
7 | ]
8 |
9 | tries_par_age = sorted(adherents, key=lambda ligne: ligne["age"])
10 | print([ligne["nom"] for ligne in tries_par_age])
11 | # ['Sami', 'Ali', 'Nour', 'Karim', 'Yasmine']
```

◆ **PROPRIÉTÉ Ordre décroissant** Pour trier en ordre **décroissant**, on ajoute le paramètre reverse=True à sorted.

EXEMPLE Trier les adhérents par cotisation décroissante :

⚠ ERREUR FRÉQUENTE

Écrire une fonction de tri « à la main » (par insertion ou sélection, vues au chapitre d'algorithmique) alors qu'une fonction intégrée existe et suffit largement. Le programme officiel autorise explicitement l'usage de sorted pour trier une table : implémenter son propre tri n'apporte rien ici et augmente le risque d'erreur.

DÉFINITION D3

`sorted(table, key=...)` renvoie une **nouvelle** liste triée, sans modifier `table`. La méthode `table.sort(key=...)`, elle, trie la liste **sur place** (elle modifie `table` et ne renvoie rien).

» **Python À la machine.** Trie la table `adherents` par ordre alphabétique du nom, puis par âge décroissant. Vérifie, en affichant `adherents` avant et après un appel à `sorted` (pas `.sort()`), que la table d'origine n'est pas modifiée.

3.8 Fusionner deux tables

DÉFINITION D4

Le **domaine de valeurs** d'une colonne est l'ensemble des valeurs qu'elle peut prendre. Pour **fusionner** deux tables, on les combine en s'appuyant sur une colonne commune (une **clé**) dont le domaine de valeurs permet de faire correspondre les lignes de l'une avec celles de l'autre.

EXEMPLE On dispose d'une seconde table, les présences aux entraînements, partageant la colonne `nom` avec la table `adherents`. Les deux tables sont présentées dans le programme ci-dessous.

♦ **PROPRIÉTÉ Fusion par clé commune** Fusionner deux tables `table1` et `table2` suivant une colonne `cle` consiste à construire une nouvelle table où chaque ligne combine les colonnes des deux tables, pour les valeurs de `cle` présentes **dans les deux**. Une valeur de `cle` présente dans une seule des deux tables n'apparaît pas dans le résultat (elle est hors du domaine de valeurs commun).

Précondition : les colonnes hors clé des deux tables doivent être disjointes. Sinon, une colonne de la seconde table écraserait silencieusement la colonne de même nom dans la première table.

CODE DE RÉFÉRENCE — Fusionner deux tables par une colonne commune

```

1  def fusionner(table1, table2, cle):
2      """Fusionne table1 et table2 sur les valeurs communes de 'cle'.

```

```

21
22 adherents = [
23     {"nom": "Ali", "age": 16},
24     {"nom": "Sami", "age": 15},
25     {"nom": "Yasmine", "age": 22},
26 ]
27 presences = [
28     {"nom": "Ali", "seances": 12},
29     {"nom": "Sami", "seances": 9},
30     {"nom": "Yasmine", "seances": 15},
31 ]
32 resultat = fusionner(adherents, presences, "nom")
33 print(resultat)

[{'nom': 'Ali', 'age': 16, 'seances': 12}, {'nom': 'Sami', 'age': 15, 'seances': 9},
 {'nom': 'Yasmine', 'age': 22, 'seances': 15}]

```

EXEMPLE Si presences ne contient pas d'entrée pour "Karim" et "Nour" (absents du domaine de valeurs commun), ces adhérents **n'apparaissent pas** dans la table fusionnée, même s'ils sont présents dans adherents :

⚠ ERREUR FRÉQUENTE

Croire que la fusion garde **toutes** les lignes des deux tables. Par défaut, seules les lignes dont la clé est présente **dans les deux** tables sont conservées (fusion sur le domaine de valeurs **commun**) : c'est le comportement attendu par le programme officiel, qui met l'accent sur cette notion de domaine de valeurs.

» **Python À la machine.** Ajoute une troisième table cotisations_payees (colonnes nom et montant_paye) avec seulement 3 des 5 adhérents. Fusionne-la avec adherents via fusionner, et vérifie que la table résultante ne contient que les adhérents présents dans les deux tables.

Méthodes

- Méthode directe de travail sur le sujet.

Le Petit Prince,Saint-Exupery,7.5,12 1984,Orwell,9.9,5 Le Comte de Monte-Cristo,Dumas,12.0,3"

Exercice 1

Une librairie stocke son inventaire dans le fichier livres.csv :

```

titre,auteur,prix,stock
Le Petit Prince,Saint-Exupery,7.5,12
1984,Orwell,9.9,5
Le Comte de Monte-Cristo,Dumas,12.0,3

```

1. Écrire le code Python (avec le module csv) qui importe ce fichier dans une table livres.
2. Après import, quel est le type Python de livres[0]["prix"] ? Écrire le code qui convertit les colonnes prix (en float) et stock (en int) pour toutes les lignes.
3. En déduire la valeur totale du stock (somme de prix * stock pour chaque livre).

Exercice 2

On dispose de la table notes d'un contrôle continu :

```
1 notes = [
2     {"nom": "Amine", "matiere": "Maths", "note": 14},
3     {"nom": "Amine", "matiere": "NSI", "note": 16},
4     {"nom": "Lina", "matiere": "Maths", "note": 12},
5     {"nom": "Lina", "matiere": "NSI", "note": 18},
6     {"nom": "Omar", "matiere": "Maths", "note": 9},
7     {"nom": "Omar", "matiere": "NSI", "note": 11},
8 ]
```

1. Écrire une compréhension de liste renvoyant les lignes où la matière est "NSI" et la note est supérieure ou égale à 15.
2. Combien de lignes obtient-on ? Donner les noms correspondants.
3. Écrire une compréhension de liste renvoyant les noms des élèves ayant une note strictement inférieure à 10 (peu importe la matière).

Exercice 3

On dispose de la table commandes suivante :

```
1 commandes = [
2     {"id": "C001", "client": "Ali", "quantite": 3},
3     {"id": "C002", "client": "Sara", "quantite": 1},
4     {"id": "C001", "client": "Ali", "quantite": 3},
5     {"id": "C003", "client": "Nadia", "quantite": -2},
6 ]
```

1. Écrire une fonction doublons(table, colonne) qui renvoie les valeurs de colonne apparaissant plus d'une fois. L'appliquer à la colonne id : que trouve-t-on, et pourquoi est-ce a priori anormal pour un identifiant de commande ?
2. Écrire une fonction coherente(table) qui renvoie False si au moins une quantité n'est pas un entier strictement positif.
3. Identifier la ou les ligne(s) responsable(s) de l'incohérence détectée à la question 2.

Exercice 4

On reprend la table notes de l'exercice précédent.

1. Écrire l'instruction qui trie cette table par note **décroissante**.
2. Quelle est la première ligne de la table triée ? La dernière ?
3. La table notes d'origine est-elle modifiée par cette instruction ? Justifier en une phrase.

Exercice 5

Un lycée dispose de deux tables :

```
1 eleves = [
2     {"nom": "Amine", "classe": "1A"},
3     {"nom": "Lina", "classe": "1B"},
4     {"nom": "Omar", "classe": "1A"},
5     {"nom": "Sara", "classe": "1B"},
6 ]
7 absences = [
8     {"nom": "Amine", "nb_absences": 2},
```

```

9     {"nom": "Omar", "nb_absences": 0},
10    {"nom": "Sara", "nb_absences": 5},
11 ]

```

1. En utilisant la fonction `fusionner(table1, table2, cle)` du cours, fusionner `eleves` et `absences` sur la colonne `nom`. Combien de lignes contient le résultat ?
2. Quel élève de `eleves` n'apparaît **pas** dans la table fusionnée ? Pourquoi ?
3. Le CPE souhaite une table listant **tous** les élèves, y compris ceux sans donnée d'absence (avec `nb_absences` à 0 par défaut dans ce cas). Proposer, en français ou en pseudo-code, une modification de la fonction `fusionner` permettant d'obtenir ce résultat.

Le Petit Prince,Saint-Exupery,7.5,12 1984,Orwell,9.9,5 Le Comte de Monte-Cristo,Dumas,12.0,3"

Exercice 6

Une librairie stocke son inventaire dans le fichier `livres.csv` :

```

titre,auteur,prix,stock
Le Petit Prince,Saint-Exupery,7.5,12
1984,Orwell,9.9,5
Le Comte de Monte-Cristo,Dumas,12.0,3

```

1. Écrire le code Python (avec le module `csv`) qui importe ce fichier dans une table `livres`.
2. Après import, quel est le type Python de `livres[0]["prix"]` ? Écrire le code qui convertit les colonnes `prix` (en float) et `stock` (en int) pour toutes les lignes.
3. En déduire la valeur totale du stock (somme de `prix * stock` pour chaque livre).

Exercice 7

On dispose de la table `notes` d'un contrôle continu :

```

1 notes = [
2     {"nom": "Amine", "matiere": "Maths", "note": 14},
3     {"nom": "Amine", "matiere": "NSI", "note": 16},
4     {"nom": "Lina", "matiere": "Maths", "note": 12},
5     {"nom": "Lina", "matiere": "NSI", "note": 18},
6     {"nom": "Omar", "matiere": "Maths", "note": 9},
7     {"nom": "Omar", "matiere": "NSI", "note": 11},
8 ]

```

1. Écrire une compréhension de liste renvoyant les lignes où la matière est "NSI" et la note est supérieure ou égale à 15.
2. Combien de lignes obtient-on ? Donner les noms correspondants.
3. Écrire une compréhension de liste renvoyant les noms des élèves ayant une note strictement inférieure à 10 (peu importe la matière).

Exercice 8

On dispose de la table `commandes` suivante :

```

1 commandes = [
2     {"id": "C001", "client": "Ali", "quantite": 3},
3     {"id": "C002", "client": "Sara", "quantite": 1},
4     {"id": "C001", "client": "Ali", "quantite": 3},
5     {"id": "C003", "client": "Nadia", "quantite": -2},
6 ]

```

1. Écrire une fonction `doublons(table, colonne)` qui renvoie les valeurs de colonne apparaissant plus d'une fois. L'appliquer à la colonne `id` : que trouve-t-on, et pourquoi est-ce a priori anormal pour un identifiant de commande ?
2. Écrire une fonction `coherente(table)` qui renvoie `False` si au moins une quantité n'est pas un entier strictement positif.
3. Identifier la ou les ligne(s) responsable(s) de l'incohérence détectée à la question 2.

Exercice 9

On reprend la table `notes` de l'exercice précédent.

1. Écrire l'instruction qui trie cette table par note **décroissante**.
2. Quelle est la première ligne de la table triée ? La dernière ?
3. La table `notes` d'origine est-elle modifiée par cette instruction ? Justifier en une phrase.

Exercice 10

Un lycée dispose de deux tables :

```

1  eleves = [
2      {"nom": "Amine", "classe": "1A"},
3      {"nom": "Lina", "classe": "1B"},
4      {"nom": "Omar", "classe": "1A"},
5      {"nom": "Sara", "classe": "1B"},
6  ]
7  absences = [
8      {"nom": "Amine", "nb_absences": 2},
9      {"nom": "Omar", "nb_absences": 0},
10     {"nom": "Sara", "nb_absences": 5},
11 ]

```

1. En utilisant la fonction `fusionner(table1, table2, cle)` du cours, fusionner `eleves` et `absences` sur la colonne `nom`. Combien de lignes contient le résultat ?
2. Quel élève de `eleves` n'apparaît **pas** dans la table fusionnée ? Pourquoi ?
3. Le CPE souhaite une table listant **tous** les élèves, y compris ceux sans donnée d'absence (avec `nb_absences` à 0 par défaut dans ce cas). Proposer, en français ou en pseudo-code, une modification de la fonction `fusionner` permettant d'obtenir ce résultat.

Le Petit Prince,Saint-Exupery,7.5,12 1984,Orwell,9.9,5 Le Comte de Monte-Cristo,Dumas,12.0,3"

Exercice 11

Une librairie stocke son inventaire dans le fichier `livres.csv` :

```

titre,auteur,prix,stock
Le Petit Prince,Saint-Exupery,7.5,12
1984,Orwell,9.9,5
Le Comte de Monte-Cristo,Dumas,12.0,3

```

1. Écrire le code Python (avec le module `csv`) qui importe ce fichier dans une table `livres`.
2. Après `import`, quel est le type Python de `livres[0][ "prix" ]` ? Écrire le code qui convertit les colonnes `prix` (en `float`) et `stock` (en `int`) pour toutes les lignes.
3. En déduire la valeur totale du stock (somme de `prix * stock` pour chaque livre).

Exercice 12

On dispose de la table `notes` d'un contrôle continu :


```

1 notes = [
2     {"nom": "Amine", "matiere": "Maths", "note": 14},
3     {"nom": "Amine", "matiere": "NSI", "note": 16},
4     {"nom": "Lina", "matiere": "Maths", "note": 12},
5     {"nom": "Lina", "matiere": "NSI", "note": 18},
6     {"nom": "Omar", "matiere": "Maths", "note": 9},
7     {"nom": "Omar", "matiere": "NSI", "note": 11},
8 ]

```

1. Écrire une compréhension de liste renvoyant les lignes où la matière est "NSI" et la note est supérieure ou égale à 15.
2. Combien de lignes obtient-on ? Donner les noms correspondants.
3. Écrire une compréhension de liste renvoyant les noms des élèves ayant une note strictement inférieure à 10 (peu importe la matière).

Exercice 13 ♦♦

On dispose de la table commandes suivante :

```

1 commandes = [
2     {"id": "C001", "client": "Ali", "quantite": 3},
3     {"id": "C002", "client": "Sara", "quantite": 1},
4     {"id": "C001", "client": "Ali", "quantite": 3},
5     {"id": "C003", "client": "Nadia", "quantite": -2},
6 ]

```

1. Écrire une fonction doublons(table, colonne) qui renvoie les valeurs de colonne apparaissant plus d'une fois. L'appliquer à la colonne id : que trouve-t-on, et pourquoi est-ce a priori anormal pour un identifiant de commande ?
2. Écrire une fonction cohérente(table) qui renvoie False si au moins une quantité n'est pas un entier strictement positif.
3. Identifier la ou les ligne(s) responsable(s) de l'incohérence détectée à la question 2.

Exercice 14 ♦♦

On reprend la table notes de l'exercice précédent.

1. Écrire l'instruction qui trie cette table par note **décroissante**.
2. Quelle est la première ligne de la table triée ? La dernière ?
3. La table notes d'origine est-elle modifiée par cette instruction ? Justifier en une phrase.

Exercice 15 ♦♦♦

Un lycée dispose de deux tables :

```

1 eleves = [
2     {"nom": "Amine", "classe": "1A"},
3     {"nom": "Lina", "classe": "1B"},
4     {"nom": "Omar", "classe": "1A"},
5     {"nom": "Sara", "classe": "1B"},
6 ]
7 absences = [
8     {"nom": "Amine", "nb_absences": 2},
9     {"nom": "Omar", "nb_absences": 0},
10    {"nom": "Sara", "nb_absences": 5},
11 ]

```

1. En utilisant la fonction `fusionner(table1, table2, cle)` du cours, fusionner `eleves` et `absences` sur la colonne `nom`. Combien de lignes contient le résultat ?
2. Quel élève de `eleves` n'apparaît **pas** dans la table fusionnée ? Pourquoi ?
3. Le CPE souhaite une table listant **tous** les élèves, y compris ceux sans donnée d'absence (avec `nb_absences` à 0 par défaut dans ce cas). Proposer, en français ou en pseudo-code, une modification de la fonction `fusionner` permettant d'obtenir ce résultat.

Le Petit Prince,Saint-Exupery,7.5,12 1984,Orwell,9.9,5 Le Comte de Monte-Cristo,Dumas,12.0,3"

Exercice 16

Une librairie stocke son inventaire dans le fichier `livres.csv` :

```
titre,auteur,prix,stock
Le Petit Prince,Saint-Exupery,7.5,12
1984,Orwell,9.9,5
Le Comte de Monte-Cristo,Dumas,12.0,3
```

1. Écrire le code Python (avec le module `csv`) qui importe ce fichier dans une table `livres`.
2. Après import, quel est le type Python de `livres[0][ "prix" ]` ? Écrire le code qui convertit les colonnes `prix` (en float) et `stock` (en int) pour toutes les lignes.
3. En déduire la valeur totale du stock (somme de `prix * stock` pour chaque livre).

Exercice 17

On dispose de la table `notes` d'un contrôle continu :

```
1 notes = [
2     {"nom": "Amine", "matiere": "Maths", "note": 14},
3     {"nom": "Amine", "matiere": "NSI", "note": 16},
4     {"nom": "Lina", "matiere": "Maths", "note": 12},
5     {"nom": "Lina", "matiere": "NSI", "note": 18},
6     {"nom": "Omar", "matiere": "Maths", "note": 9},
7     {"nom": "Omar", "matiere": "NSI", "note": 11},
8 ]
```

1. Écrire une compréhension de liste renvoyant les lignes où la matière est "NSI" et la note est supérieure ou égale à 15.
2. Combien de lignes obtient-on ? Donner les noms correspondants.
3. Écrire une compréhension de liste renvoyant les noms des élèves ayant une note strictement inférieure à 10 (peu importe la matière).

Exercice 18

On dispose de la table `commandes` suivante :

```
1 commandes = [
2     {"id": "C001", "client": "Ali", "quantite": 3},
3     {"id": "C002", "client": "Sara", "quantite": 1},
4     {"id": "C001", "client": "Ali", "quantite": 3},
5     {"id": "C003", "client": "Nadia", "quantite": -2},
6 ]
```

1. Écrire une fonction `doublons(table, colonne)` qui renvoie les valeurs de colonne apparaissant plus d'une fois. L'appliquer à la colonne `id` : que trouve-t-on, et pourquoi est-ce a priori anormal pour un identifiant de commande ?

- Écrire une fonction cohérente(table) qui renvoie False si au moins une quantité n'est pas un entier strictement positif.
- Identifier la ou les ligne(s) responsable(s) de l'incohérence détectée à la question 2.

Exercice 19 ♦♦

On reprend la table notes de l'exercice précédent.

- Écrire l'instruction qui trie cette table par note **décroissante**.
- Quelle est la première ligne de la table triée ? La dernière ?
- La table notes d'origine est-elle modifiée par cette instruction ? Justifier en une phrase.

Exercice 20 ♦♦♦

Un lycée dispose de deux tables :

```

1  eleves = [
2      {"nom": "Amine", "classe": "1A"},
3      {"nom": "Lina", "classe": "1B"},
4      {"nom": "Omar", "classe": "1A"},
5      {"nom": "Sara", "classe": "1B"},
6  ]
7  absences = [
8      {"nom": "Amine", "nb_absences": 2},
9      {"nom": "Omar", "nb_absences": 0},
10     {"nom": "Sara", "nb_absences": 5},
11 ]

```

- En utilisant la fonction fusionner(table1, table2, cle) du cours, fusionner eleves et absences sur la colonne nom. Combien de lignes contient le résultat ?
- Quel élève de eleves n'apparaît **pas** dans la table fusionnée ? Pourquoi ?
- Le CPE souhaite une table listant **tous** les élèves, y compris ceux sans donnée d'absence (avec nb_absences à 0 par défaut dans ce cas). Proposer, en français ou en pseudo-code, une modification de la fonction fusionner permettant d'obtenir ce résultat.

Le Petit Prince,Saint-Exupery,7.5,12 1984,Orwell,9.9,5 Le Comte de Monte-Cristo,Dumas,12.0,3"

Exercice 21 ♦

Une librairie stocke son inventaire dans le fichier livres.csv :

```

titre,auteur,prix,stock
Le Petit Prince,Saint-Exupery,7.5,12
1984,Orwell,9.9,5
Le Comte de Monte-Cristo,Dumas,12.0,3

```

- Écrire le code Python (avec le module csv) qui importe ce fichier dans une table livres.
- Après import, quel est le type Python de livres[0]["prix"] ? Écrire le code qui convertit les colonnes prix (en float) et stock (en int) pour toutes les lignes.
- En déduire la valeur totale du stock (somme de prix * stock pour chaque livre).

Exercice 22 ♦

On dispose de la table notes d'un contrôle continu :

```

1 notes = [
2     {"nom": "Amine", "matiere": "Maths", "note": 14},
3     {"nom": "Amine", "matiere": "NSI", "note": 16},
4     {"nom": "Lina", "matiere": "Maths", "note": 12},
5     {"nom": "Lina", "matiere": "NSI", "note": 18},
6     {"nom": "Omar", "matiere": "Maths", "note": 9},
7     {"nom": "Omar", "matiere": "NSI", "note": 11},
8 ]

```

1. Écrire une compréhension de liste renvoyant les lignes où la matière est "NSI" et la note est supérieure ou égale à 15.
2. Combien de lignes obtient-on ? Donner les noms correspondants.
3. Écrire une compréhension de liste renvoyant les noms des élèves ayant une note strictement inférieure à 10 (peu importe la matière).

Exercice 23

On dispose de la table commandes suivante :

```

1 commandes = [
2     {"id": "C001", "client": "Ali", "quantite": 3},
3     {"id": "C002", "client": "Sara", "quantite": 1},
4     {"id": "C001", "client": "Ali", "quantite": 3},
5     {"id": "C003", "client": "Nadia", "quantite": -2},
6 ]

```

1. Écrire une fonction doublons(table, colonne) qui renvoie les valeurs de colonne apparaissant plus d'une fois. L'appliquer à la colonne id : que trouve-t-on, et pourquoi est-ce a priori anormal pour un identifiant de commande ?
2. Écrire une fonction coherente(table) qui renvoie False si au moins une quantite n'est pas un entier strictement positif.
3. Identifier la ou les ligne(s) responsable(s) de l'incohérence détectée à la question 2.

Exercice 24

On reprend la table notes de l'exercice précédent.

1. Écrire l'instruction qui trie cette table par note **décroissante**.
2. Quelle est la première ligne de la table triée ? La dernière ?
3. La table notes d'origine est-elle modifiée par cette instruction ? Justifier en une phrase.

QCM d'auto-évaluation — Traitement de données en tables

Pour chaque question, une seule réponse est exacte. Entourez la lettre correspondante.

C1 — Import de table

1. [Q1] Après import d'un fichier CSV avec csv.DictReader, une valeur numérique comme "16" est de type :
A. int
B. float

- C. str
- D. bool

C2 — Recherche, doublons, cohérence

2. [Q2] Pour rechercher les lignes d'une table vérifiant un critère, l'outil le plus adapté en Python est :
- A. une compréhension de liste avec condition.
 - B. une modification directe du fichier CSV.
 - C. la fonction `sorted`.
 - D. l'opérateur `+`.
3. [Q3] Pour détecter un doublon sur la colonne `id` d'une table, il faut :
- A. comparer chaque ligne entière avec toutes les autres.
 - B. compter les occurrences de chaque valeur de la colonne `id` uniquement.
 - C. trier la table par ordre alphabétique.
 - D. supprimer toutes les lignes identiques.

C3 — Tri

4. [Q4] `sorted(table, key=lambda ligne: ligne["age"])` :
- A. modifie table sur place et ne renvoie rien.
 - B. renvoie une nouvelle liste triée, sans modifier table.
 - C. provoque une erreur si table contient des dictionnaires.
 - D. trie uniquement les nombres, jamais les chaînes.

C4 — Fusion de tables

5. [Q5] Lors de la fusion de deux tables sur une colonne commune, une ligne de `table1` dont la clé n'existe pas dans `table2` :
- A. provoque une erreur du programme.
 - B. apparaît dans le résultat avec des valeurs vides pour les colonnes de `table2`.
 - C. n'apparaît pas dans le résultat, par défaut.
 - D. est dupliquée dans le résultat.
6. [Q6] Le « domaine de valeurs » d'une colonne désigne :
- A. le nombre de lignes de la table.
 - B. l'ensemble des valeurs que peut prendre cette colonne.
 - C. le type Python de la colonne uniquement.
 - D. le nom de la colonne dans le fichier CSV.

Corrigé

1. Avant conversion, `films[0]["annee"]` serait une chaîne de caractères (`str`), par exemple "2014".

```
1 for ligne in films:
2     ligne["annee"] = int(ligne["annee"])
3     ligne["note"] = float(ligne["note"])
```

- 2.

```
1 resultat = [f for f in films if f["genre"] == "SF" and f["note"] ≥ 8.5]
```

3. 2 films remplissent ce critère : **Interstellar** (note 8,6) et **Inception** (note 8,8).

Corrigé

1. `sorted(films, key=lambda ligne: ligne["annee"])`. Le premier film après tri est **Amelie** (2001, la plus ancienne année).
2. En fusionnant films et réalisateurs sur titre, on obtient 3 lignes (Interstellar, Inception, Amelie).
3. **Dune** n'apparaît pas dans le résultat : son titre n'est pas présent dans la table réalisateurs, il est donc hors du domaine de valeurs commun aux deux tables sur la colonne titre.

Évaluation A — Traitement de données en tables

Durée : 45 minutes. Ordinateur avec interpréteur Python autorisé.

Barème indicatif : Ex. 1 : 10 pts (import, recherche) ; Ex. 2 : 10 pts (tri, fusion)

Exercice 1 — Une table de films

(10 points)

```
1 films = [
2     {"titre": "Interstellar", "annee": 2014, "genre": "SF", "note": 8.6},
3     {"titre": "Inception", "annee": 2010, "genre": "SF", "note": 8.8},
4     {"titre": "Amelie", "annee": 2001, "genre": "Comedie", "note": 8.3},
5     {"titre": "Dune", "annee": 2021, "genre": "SF", "note": 8.0},
6 ]
```

1. (3 pts) Si cette table provenait d'un fichier CSV importé avec `csv.DictReader`, quel serait le type de `films[0]["annee"]` avant conversion ? Écrire le code de conversion nécessaire pour les colonnes `annee` (en int) et `note` (en float).
2. (4 pts) Écrire une compréhension de liste renvoyant les films de genre "SF" ayant une note supérieure ou égale à 8,5.
3. (3 pts) Combien de films remplissent ce critère ? Les nommer.

Exercice 2 — Tri et fusion

(10 points)

1. (3 pts) Écrire l'instruction qui trie la table films par année croissante. Donner le titre du premier film après ce tri.
2. (4 pts) On dispose de la table réalisateurs :

```
1 realiseurs = [
2     {"titre": "Interstellar", "realisateur": "Nolan"},
3     {"titre": "Inception", "realisateur": "Nolan"},
4     {"titre": "Amelie", "realisateur": "Jeunet"},
5 ]
```

En utilisant la fonction `fusionner(table1, table2, cle)` du cours, fusionner films et réalisateurs sur titre. Combien de lignes obtient-on ?

3. (3 pts) Quel film de la table films n'apparaît pas dans le résultat de la fusion ? Pourquoi ?

Corrigé

1. Avant conversion, `jeux[0]["note"]` serait une chaîne de caractères (str), par exemple "9.7".

```
1 for ligne in jeux:
2     ligne["annee"] = int(ligne["annee"])
3     ligne["note"] = float(ligne["note"])
```

2.

```
1 resultat = [j for j in jeux if j["genre"] == "Puzzle" and j["note"] ≥ 9.3]
```

3. 1 seul jeu remplit ce critère : **Portal 2** (note 9,5). Tetris, bien que du genre Puzzle, a une note de 9,0, inférieure au seuil de 9,3.

Corrigé

1. `sorted(jeux, key=lambda ligne: ligne["annee"])`. Le premier jeu après tri est **Tetris** (1984, la plus ancienne année).

2. En fusionnant jeux et studios sur titre, on obtient 3 lignes (Zelda, Portal 2, Celeste).

3. **Tetris** n'apparaît pas dans le résultat : son titre n'est pas présent dans la table studios, il est donc hors du domaine de valeurs commun aux deux tables sur la colonne titre.

Évaluation B — Traitement de données en tables

Durée : 45 minutes. Ordinateur avec interpréteur Python autorisé.

Barème indicatif : Ex. 1 : 10 pts (import, recherche) ; Ex. 2 : 10 pts (tri, fusion)

Exercice 1 — Une table de jeux vidéo

(10 points)

```
1 jeux = [
2     {"titre": "Zelda", "annee": 2017, "genre": "Aventure", "note": 9.7},
3     {"titre": "Portal 2", "annee": 2011, "genre": "Puzzle", "note": 9.5},
4     {"titre": "Celeste", "annee": 2018, "genre": "Plateforme", "note": 9.2},
5     {"titre": "Tetris", "annee": 1984, "genre": "Puzzle", "note": 9.0},
6 ]
```

- (3 pts) Si cette table provenait d'un fichier CSV importé avec `csv.DictReader`, quel serait le type de `jeux[0]["note"]` avant conversion ? Écrire le code de conversion nécessaire pour les colonnes `annee` (en int) et `note` (en float).
- (4 pts) Écrire une compréhension de liste renvoyant les jeux de genre "Puzzle" ayant une note supérieure ou égale à 9,3.
- (3 pts) Combien de jeux remplissent ce critère ? Les nommer.

Exercice 2 — Tri et fusion

(10 points)

- (3 pts) Écrire l'instruction qui trie la table jeux par année croissante. Donner le titre du premier jeu après ce tri.
- (4 pts) On dispose de la table studios :

```
1 studios = [
2     {"titre": "Zelda", "studio": "Nintendo"},
3     {"titre": "Portal 2", "studio": "Valve"},
4     {"titre": "Celeste", "studio": "Matt Makes Games"},
5 ]
```

En utilisant la fonction `fusionner(table1, table2, cle)` du cours, fusionner jeux et studios sur titre. Combien de lignes obtient-on ?

- (3 pts) Quel jeu de la table jeux n'apparaît pas dans le résultat de la fusion ? Pourquoi ?

Exercice 25

Un élève cherche les élèves inscrits deux fois (même nom) dans la table suivante, mais compare des **lignes entières** au lieu de la seule colonne nom :

```
1 eleves = [
2     {"nom": "Ali", "age": 16, "classe": "1A"},
3     {"nom": "Ali", "age": 17, "classe": "1B"},
4 ]
5 lignes_str = [str(e) for e in eleves]
6 doublons_naifs = [chaine for chaine in set(lignes_str) if lignes_str.count(chaine) > 1]
```

1. Que vaut doublons_naifs ? Le nom "Ali" est-il pourtant présent deux fois dans la table ?
2. Pourquoi la méthode de l'élève échoue-t-elle à détecter ce cas ?
3. Réécrire une détection de doublons qui compare uniquement la colonne nom, et vérifier qu'elle détecte bien "Ali".

Exercice 26

Un élève cherche les élèves inscrits deux fois (même nom) dans la table suivante, mais compare des **lignes entières** au lieu de la seule colonne nom :

```
1 eleves = [
2     {"nom": "Ali", "age": 16, "classe": "1A"},
3     {"nom": "Ali", "age": 17, "classe": "1B"},
4 ]
5 lignes_str = [str(e) for e in eleves]
6 doublons_naifs = [chaine for chaine in set(lignes_str) if lignes_str.count(chaine) > 1]
```

1. Que vaut doublons_naifs ? Le nom "Ali" est-il pourtant présent deux fois dans la table ?
2. Pourquoi la méthode de l'élève échoue-t-elle à détecter ce cas ?
3. Réécrire une détection de doublons qui compare uniquement la colonne nom, et vérifier qu'elle détecte bien "Ali".

Exercice 27

Un élève cherche les élèves inscrits deux fois (même nom) dans la table suivante, mais compare des **lignes entières** au lieu de la seule colonne nom :

```
1 eleves = [
2     {"nom": "Ali", "age": 16, "classe": "1A"},
3     {"nom": "Ali", "age": 17, "classe": "1B"},
4 ]
5 lignes_str = [str(e) for e in eleves]
6 doublons_naifs = [chaine for chaine in set(lignes_str) if lignes_str.count(chaine) > 1]
```

1. Que vaut doublons_naifs ? Le nom "Ali" est-il pourtant présent deux fois dans la table ?
2. Pourquoi la méthode de l'élève échoue-t-elle à détecter ce cas ?
3. Réécrire une détection de doublons qui compare uniquement la colonne nom, et vérifier qu'elle détecte bien "Ali".

Exercice 28

Un élève cherche les élèves inscrits deux fois (même nom) dans la table suivante, mais compare des **lignes entières** au lieu de la seule colonne nom :


```

1 eleves = [
2     {"nom": "Ali", "age": 16, "classe": "1A"},
3     {"nom": "Ali", "age": 17, "classe": "1B"},
4 ]
5 lignes_str = [str(e) for e in eleves]
6 doublons_naifs = [chaine for chaine in set(lignes_str) if lignes_str.count(chaine) > 1]

```

1. Que vaut `doublons_naifs` ? Le nom "Ali" est-il pourtant présent deux fois dans la table ?
2. Pourquoi la méthode de l'élève échoue-t-elle à détecter ce cas ?
3. Réécrire une détection de doublons qui compare uniquement la colonne nom, et vérifier qu'elle détecte bien "Ali".

Exercice 29 ◆

Un élève cherche les élèves inscrits deux fois (même nom) dans la table suivante, mais compare des **lignes entières** au lieu de la seule colonne nom :

```

1 eleves = [
2     {"nom": "Ali", "age": 16, "classe": "1A"},
3     {"nom": "Ali", "age": 17, "classe": "1B"},
4 ]
5 lignes_str = [str(e) for e in eleves]
6 doublons_naifs = [chaine for chaine in set(lignes_str) if lignes_str.count(chaine) > 1]

```

1. Que vaut `doublons_naifs` ? Le nom "Ali" est-il pourtant présent deux fois dans la table ?
2. Pourquoi la méthode de l'élève échoue-t-elle à détecter ce cas ?
3. Réécrire une détection de doublons qui compare uniquement la colonne nom, et vérifier qu'elle détecte bien "Ali".

Le Petit Prince,Saint-Exupery,7.5,12 1984,Orwell,9.9,5 Le Comte de Monte-Cristo,Dumas,12.0,3"

Corrigé

1.

```

1 import csv
2
3 with open("livres.csv", encoding="utf-8") as f:
4     livres = list(csv.DictReader(f))

```

2. `livres[0]["prix"]` est une chaîne de caractères (str), pas un flottant : "7.5", pas 7.5.

```

1 for ligne in livres:
2     ligne["prix"] = float(ligne["prix"])
3     ligne["stock"] = int(ligne["stock"])

```

3. $7,5 \times 12 + 9,9 \times 5 + 12,0 \times 3 = 90 + 49,5 + 36 = 175,5$ euros.

Corrigé

1.

```

1 resultat = [ligne for ligne in notes if ligne["matiere"] == "NSI" and ligne["note"] >= 15]

```

2. On obtient 2 lignes : celles d'Amine (NSI, 16) et de Lina (NSI, 18).

3.

```
1 sous_10 = [ligne["nom"] for ligne in notes if ligne["note"] < 10]
```

Résultat : ["Omar"] (seule sa note de Maths, 9, est inférieure à 10).

Corrigé

1.

```
1 def doublons(table, colonne):
2     valeurs = [ligne[colonne] for ligne in table]
3     return [v for v in set(valeurs) if valeurs.count(v) > 1]
```

doublons(commandes, "id") renvoie ["C001"] : l'identifiant "C001" apparaît deux fois. C'est anormal car un identifiant de commande est censé être **unique** : cela peut signaler une commande enregistrée deux fois par erreur.

2.

```
1 def coherente(table):
2     for ligne in table:
3         if not isinstance(ligne["quantite"], int) or ligne["quantite"] ≤ 0:
4             return False
5     return True
```

coherente(commandes) renvoie False.

3. La ligne responsable est {"id": "C003", "client": "Nadia", "quantite": -2} : sa quantité est négative, ce qui n'a pas de sens pour une commande.

Corrigé

1.

```
1 tri = sorted(notes, key=lambda ligne: ligne["note"], reverse=True)
```

2. La première ligne est celle de **Lina** en NSI (note 18) ; la dernière est celle d'**Omar** en Maths (note 9).

3. Non, notes n'est **pas** modifiée : sorted renvoie toujours une **nouvelle** liste triée, sans modifier la liste passée en argument (contrairement à la méthode .sort()).

Corrigé

1. La table fusionnée contient 3 lignes (Amine, Omar, Sara).

2. **Lina** n'apparaît pas dans le résultat : son nom n'est pas présent dans la table absences, donc il est hors du domaine de valeurs **commun** aux deux tables sur la colonne nom.

3. Il faut modifier la fonction pour que, lorsque le nom d'un élève de élèves n'est pas trouvé dans absences, on lui attribue des valeurs par défaut (nb_absences: 0) au lieu de l'exclure du résultat :

```
1 def fusionner_tout(table1, table2, cle, defaults):
2     colonnes1 = {
3         colonne for ligne in table1 for colonne in ligne if colonne ≠ cle
4     }
5     colonnes2 = {
6         colonne for ligne in table2 for colonne in ligne if colonne ≠ cle
7     }
8     colonnes2.update(colonne for colonne in defaults if colonne ≠ cle)
9     if not colonnes1.isdisjoint(colonnes2):
10         raise ValueError("les colonnes hors cle doivent etre disjointes")
11
12     index2 = {}
```

```

13     for ligne2 in table2:
14         index2.setdefault(ligne2[cle], []).append(ligne2)
15
16     fusion = []
17     for ligne1 in table1:
18         correspondances = index2.get(ligne1[cle], [defaults])
19         for ligne2 in correspondances:
20             complement = {k: v for k, v in ligne2.items() if k != cle}
21             fusion.append(**ligne1, **complement)
22     return fusion

```

Avec `fusionner_tout(eleves, absences, "nom", {"nb_absences": 0})`, on obtient bien les 4 élèves, Lina ayant `nb_absences` égal à 0. Si plusieurs lignes de la seconde table portent la même clé, chacune produit une ligne dans le résultat : aucune correspondance n'est écrasée dans l'index. Les colonnes hors clé doivent être disjointes : la fonction contrôle globalement les schémas et lève une erreur au lieu d'écraser silencieusement une valeur de la première table, même si les lignes concernées ne partagent pas la même clé ou si une valeur par défaut porte le même nom.

Corrigé

1. `doublons_naifs` vaut `[]` (liste vide) : aucun doublon n'est détecté. Pourtant, "Ali" apparaît bien deux fois dans la colonne `nom`.

2. Les deux lignes ont le même nom mais des valeurs **différentes** pour `age` et `classe` : converties en chaînes avec `str(...)`, ce sont donc deux chaînes **différentes**, qui n'apparaissent chacune qu'une seule fois. La comparaison ligne entière ne peut détecter que des lignes strictement identiques sur **toutes** les colonnes, pas un doublon sur une seule colonne.

3.

```

1 noms = [ligne["nom"] for ligne in eleves]
2 doublons_colonne = [n for n in set(noms) if noms.count(n) > 1]

```

`doublons_colonne` vaut `["Ali"]` : cette fois, le doublon est bien détecté, car on compare uniquement la colonne concernée.

Le Petit Prince,Saint-Exupery,7.5,12 1984,Orwell,9.9,5 Le Comte de Monte-Cristo,Dumas,12.0,3"

Corrigé

1.

```

1 import csv
2
3 with open("livres.csv", encoding="utf-8") as f:
4     livres = list(csv.DictReader(f))

```

2. `livres[0]["prix"]` est une chaîne de caractères (`str`), pas un flottant : `"7.5"`, pas `7.5`.

```

1 for ligne in livres:
2     ligne["prix"] = float(ligne["prix"])
3     ligne["stock"] = int(ligne["stock"])

```

3. $7,5 \times 12 + 9,9 \times 5 + 12,0 \times 3 = 90 + 49,5 + 36 = 175,5$ euros.

Corrigé

1.

```

1 resultat = [ligne for ligne in notes if ligne["matiere"] == "NSI" and ligne["note"] >= 15]

```

2. On obtient 2 lignes : celles d'**Amine** (NSI, 16) et de **Lina** (NSI, 18).
- 3.

```
1 sous_10 = [ligne["nom"] for ligne in notes if ligne["note"] < 10]
```

Résultat : ["Omar"] (seule sa note de Maths, 9, est inférieure à 10).

Corrigé

- 1.

```
1 def doublons(table, colonne):
2     valeurs = [ligne[colonne] for ligne in table]
3     return [v for v in set(valeurs) if valeurs.count(v) > 1]
```

doublons(commandes, "id") renvoie ["C001"] : l'identifiant "C001" apparaît deux fois. C'est anormal car un identifiant de commande est censé être **unique** : cela peut signaler une commande enregistrée deux fois par erreur.

- 2.

```
1 def coherente(table):
2     for ligne in table:
3         if not isinstance(ligne["quantite"], int) or ligne["quantite"] ≤ 0:
4             return False
5     return True
```

coherente(commandes) renvoie False.

3. La ligne responsable est {"id": "C003", "client": "Nadia", "quantite": -2} : sa quantité est négative, ce qui n'a pas de sens pour une commande.

Corrigé

- 1.

```
1 tri = sorted(notes, key=lambda ligne: ligne["note"], reverse=True)
```

2. La première ligne est celle de **Lina** en NSI (note 18) ; la dernière est celle d'**Omar** en Maths (note 9).

3. Non, notes n'est **pas** modifiée : sorted renvoie toujours une **nouvelle** liste triée, sans modifier la liste passée en argument (contrairement à la méthode .sort()).

Corrigé

1. La table fusionnée contient 3 lignes (Amine, Omar, Sara).

2. **Lina** n'apparaît pas dans le résultat : son nom n'est pas présent dans la table absences, donc il est hors du domaine de valeurs **commun** aux deux tables sur la colonne nom.

3. Il faut modifier la fonction pour que, lorsque le nom d'un élève de élèves n'est pas trouvé dans absences, on lui attribue des valeurs par défaut (nb_absences: 0) au lieu de l'exclure du résultat :

```
1 def fusionner_tout(table1, table2, cle, defaults):
2     colonnes1 = {
3         colonne for ligne in table1 for colonne in ligne if colonne ≠ cle
4     }
5     colonnes2 = {
6         colonne for ligne in table2 for colonne in ligne if colonne ≠ cle
7     }
8     colonnes2.update(colonne for colonne in defaults if colonne ≠ cle)
9     if not colonnes1.isdisjoint(colonnes2):
```

```

10     raise ValueError("les colonnes hors cle doivent etre disjointes")
11
12     index2 = {}
13     for ligne2 in table2:
14         index2.setdefault(ligne2[cle], []).append(ligne2)
15
16     fusion = []
17     for ligne1 in table1:
18         correspondances = index2.get(ligne1[cle], [defaults])
19         for ligne2 in correspondances:
20             complement = {k: v for k, v in ligne2.items() if k != cle}
21             fusion.append(**ligne1, **complement)
22     return fusion

```

Avec `fusionner_tout(eleves, absences, "nom", {"nb_absences": 0})`, on obtient bien les 4 élèves, Lina ayant `nb_absences` égal à 0. Si plusieurs lignes de la seconde table portent la même clé, chacune produit une ligne dans le résultat : aucune correspondance n'est écrasée dans l'index. Les colonnes hors clé doivent être disjointes : la fonction contrôle globalement les schémas et lève une erreur au lieu d'écraser silencieusement une valeur de la première table, même si les lignes concernées ne partagent pas la même clé ou si une valeur par défaut porte le même nom.

Corrigé

1. `doublons_naifs` vaut `[]` (liste vide) : aucun doublon n'est détecté. Pourtant, "Ali" apparaît bien deux fois dans la colonne `nom`.

2. Les deux lignes ont le même nom mais des valeurs **différentes** pour `age` et `classe` : converties en chaînes avec `str(...)`, ce sont donc deux chaînes **différentes**, qui n'apparaissent chacune qu'une seule fois. La comparaison ligne entière ne peut détecter que des lignes strictement identiques sur **toutes** les colonnes, pas un doublon sur une seule colonne.

3.

```

1 noms = [ligne["nom"] for ligne in eleves]
2 doublons_colonne = [n for n in set(noms) if noms.count(n) > 1]

```

`doublons_colonne` vaut `["Ali"]` : cette fois, le doublon est bien détecté, car on compare uniquement la colonne concernée.

Le Petit Prince,Saint-Exupery,7.5,12 1984,Orwell,9.9,5 Le Comte de Monte-Cristo,Dumas,12.0,3"

Corrigé

1.

```

1 import csv
2
3 with open("livres.csv", encoding="utf-8") as f:
4     livres = list(csv.DictReader(f))

```

2. `livres[0]["prix"]` est une chaîne de caractères (`str`), pas un flottant : "7.5", pas 7.5.

```

1 for ligne in livres:
2     ligne["prix"] = float(ligne["prix"])
3     ligne["stock"] = int(ligne["stock"])

```

3. $7,5 \times 12 + 9,9 \times 5 + 12,0 \times 3 = 90 + 49,5 + 36 = 175,5$ euros.

Corrigé

1.

```
1 resultat = [ligne for ligne in notes if ligne["matiere"] == "NSI" and ligne["note"] ≥ 15]
```

2. On obtient 2 lignes : celles d'**Amine** (NSI, 16) et de **Lina** (NSI, 18).
- 3.

```
1 sous_10 = [ligne["nom"] for ligne in notes if ligne["note"] < 10]
```

Résultat : ["Omar"] (seule sa note de Maths, 9, est inférieure à 10).

Corrigé

- 1.

```
1 def doublons(table, colonne):
2     valeurs = [ligne[colonne] for ligne in table]
3     return [v for v in set(valeurs) if valeurs.count(v) > 1]
```

doublons(commandes, "id") renvoie ["C001"] : l'identifiant "C001" apparaît deux fois. C'est anormal car un identifiant de commande est censé être **unique** : cela peut signaler une commande enregistrée deux fois par erreur.

- 2.

```
1 def coherente(table):
2     for ligne in table:
3         if not isinstance(ligne["quantite"], int) or ligne["quantite"] ≤ 0:
4             return False
5     return True
```

coherente(commandes) renvoie False.

3. La ligne responsable est {"id": "C003", "client": "Nadia", "quantite": -2} : sa quantité est négative, ce qui n'a pas de sens pour une commande.

Corrigé

- 1.

```
1 tri = sorted(notes, key=lambda ligne: ligne["note"], reverse=True)
```

2. La première ligne est celle de **Lina** en NSI (note 18) ; la dernière est celle d'**Omar** en Maths (note 9).
3. Non, notes n'est **pas** modifiée : sorted renvoie toujours une **nouvelle** liste triée, sans modifier la liste passée en argument (contrairement à la méthode .sort()).

Corrigé

1. La table fusionnée contient 3 lignes (Amine, Omar, Sara).

2. **Lina** n'apparaît pas dans le résultat : son nom n'est pas présent dans la table absences, donc il est hors du domaine de valeurs **commun** aux deux tables sur la colonne nom.

3. Il faut modifier la fonction pour que, lorsque le nom d'un élève de élèves n'est pas trouvé dans absences, on lui attribue des valeurs par défaut (nb_absences: 0) au lieu de l'exclure du résultat :

```
1 def fusionner_tout(table1, table2, cle, defaults):
2     colonnes1 = {
3         colonne for ligne in table1 for colonne in ligne if colonne ≠ cle
4     }
5     colonnes2 = {
```

```

6     colonne for ligne in table2 for colonne in ligne if colonne != cle
7 }
8 colonnes2.update(colonne for colonne in defaults if colonne != cle)
9 if not colonnes1.isdisjoint(colonnes2):
10     raise ValueError("les colonnes hors cle doivent etre disjointes")
11
12 index2 = {}
13 for ligne2 in table2:
14     index2.setdefault(ligne2[cle], []).append(ligne2)
15
16 fusion = []
17 for ligne1 in table1:
18     correspondances = index2.get(ligne1[cle], [defaults])
19     for ligne2 in correspondances:
20         complement = {k: v for k, v in ligne2.items() if k != cle}
21         fusion.append(**ligne1, **complement)
22 return fusion

```

Avec `fusionner_tout(elevés, absences, "nom", {"nb_absences": 0})`, on obtient bien les 4 élèves, Lina ayant `nb_absences` égal à 0. Si plusieurs lignes de la seconde table portent la même clé, chacune produit une ligne dans le résultat : aucune correspondance n'est écrasée dans l'index. Les colonnes hors clé doivent être disjointes : la fonction contrôle globalement les schémas et lève une erreur au lieu d'écraser silencieusement une valeur de la première table, même si les lignes concernées ne partagent pas la même clé ou si une valeur par défaut porte le même nom.

Corrigé

1. `doublons_naifs` vaut `[]` (liste vide) : aucun doublon n'est détecté. Pourtant, "Ali" apparaît bien deux fois dans la colonne `nom`.

2. Les deux lignes ont le même nom mais des valeurs **différentes** pour `age` et `classe` : converties en chaînes avec `str(...)`, ce sont donc deux chaînes **différentes**, qui n'apparaissent chacune qu'une seule fois. La comparaison ligne entière ne peut détecter que des lignes strictement identiques sur **toutes** les colonnes, pas un doublon sur une seule colonne.

3.

```

1 noms = [ligne["nom"] for ligne in élèves]
2 doublons_colonne = [n for n in set(noms) if noms.count(n) > 1]

```

`doublons_colonne` vaut `["Ali"]` : cette fois, le doublon est bien détecté, car on compare uniquement la colonne concernée.

Le Petit Prince,Saint-Exupéry,7.5,12 1984,Orwell,9.9,5 Le Comte de Monte-Cristo,Dumas,12.0,3"

Corrigé

1.

```

1 import csv
2
3 with open("livres.csv", encoding="utf-8") as f:
4     livres = list(csv.DictReader(f))

```

2. `livres[0]["prix"]` est une chaîne de caractères (`str`), pas un flottant : "7.5", pas 7.5.

```

1 for ligne in livres:
2     ligne["prix"] = float(ligne["prix"])
3     ligne["stock"] = int(ligne["stock"])

```

3. $7,5 \times 12 + 9,9 \times 5 + 12,0 \times 3 = 90 + 49,5 + 36 = 175,5$ euros.

Corrigé

1.

```
1 resultat = [ligne for ligne in notes if ligne["matiere"] == "NSI" and ligne["note"] ≥ 15]
```

2. On obtient 2 lignes : celles d'**Amine** (NSI, 16) et de **Lina** (NSI, 18).

3.

```
1 sous_10 = [ligne["nom"] for ligne in notes if ligne["note"] < 10]
```

Résultat : ["Omar"] (seule sa note de Maths, 9, est inférieure à 10).

Corrigé

1.

```
1 def doublons(table, colonne):
2     valeurs = [ligne[colonne] for ligne in table]
3     return [v for v in set(valeurs) if valeurs.count(v) > 1]
```

doublons(commandes, "id") renvoie ["C001"] : l'identifiant "C001" apparaît deux fois. C'est anormal car un identifiant de commande est censé être **unique** : cela peut signaler une commande enregistrée deux fois par erreur.

2.

```
1 def coherente(table):
2     for ligne in table:
3         if not isinstance(ligne["quantite"], int) or ligne["quantite"] ≤ 0:
4             return False
5     return True
```

coherente(commandes) renvoie False.

3. La ligne responsable est {"id": "C003", "client": "Nadia", "quantite": -2} : sa quantité est négative, ce qui n'a pas de sens pour une commande.

Corrigé

1.

```
1 tri = sorted(notes, key=lambda ligne: ligne["note"], reverse=True)
```

2. La première ligne est celle de **Lina** en NSI (note 18) ; la dernière est celle d'**Omar** en Maths (note 9).

3. Non, notes n'est **pas** modifiée : sorted renvoie toujours une **nouvelle** liste triée, sans modifier la liste passée en argument (contrairement à la méthode .sort()).

Corrigé

1. La table fusionnée contient 3 lignes (Amine, Omar, Sara).

2. **Lina** n'apparaît pas dans le résultat : son nom n'est pas présent dans la table absences, donc il est hors du domaine de valeurs **commun** aux deux tables sur la colonne nom.

3. Il faut modifier la fonction pour que, lorsque le nom d'un élève de élèves n'est pas trouvé dans absences, on lui attribue des valeurs par défaut (nb_absences: 0) au lieu de l'exclure du résultat :

```
1 def fusionner_tout(table1, table2, cle, defaults):
2     colonnes1 = {
```



```

3     colonne for ligne in table1 for colonne in ligne if colonne != cle
4 }
5 colonnes2 = {
6     colonne for ligne in table2 for colonne in ligne if colonne != cle
7 }
8 colonnes2.update(colonne for colonne in defaults if colonne != cle)
9 if not colonnes1.isdisjoint(colonnes2):
10     raise ValueError("les colonnes hors cle doivent etre disjointes")
11
12 index2 = {}
13 for ligne2 in table2:
14     index2.setdefault(ligne2[cle], []).append(ligne2)
15
16 fusion = []
17 for ligne1 in table1:
18     correspondances = index2.get(ligne1[cle], [defaults])
19     for ligne2 in correspondances:
20         complement = {k: v for k, v in ligne2.items() if k != cle}
21         fusion.append({**ligne1, **complement})
22 return fusion

```

Avec `fusionner_tout(elevés, absences, "nom", {"nb_absences": 0})`, on obtient bien les 4 élèves, Lina ayant `nb_absences` égal à 0. Si plusieurs lignes de la seconde table portent la même clé, chacune produit une ligne dans le résultat : aucune correspondance n'est écrasée dans l'index. Les colonnes hors clé doivent être disjointes : la fonction contrôle globalement les schémas et lève une erreur au lieu d'écraser silencieusement une valeur de la première table, même si les lignes concernées ne partagent pas la même clé ou si une valeur par défaut porte le même nom.

Corrigé

1. `doublons_naifs` vaut `[]` (liste vide) : aucun doublon n'est détecté. Pourtant, "Ali" apparaît bien deux fois dans la colonne `nom`.

2. Les deux lignes ont le même nom mais des valeurs **différentes** pour `age` et `classe` : converties en chaînes avec `str(...)`, ce sont donc deux chaînes **différentes**, qui n'apparaissent chacune qu'une seule fois. La comparaison ligne entière ne peut détecter que des lignes strictement identiques sur **toutes** les colonnes, pas un doublon sur une seule colonne.

3.

```

1 noms = [ligne["nom"] for ligne in élèves]
2 doublons_colonne = [n for n in set(noms) if noms.count(n) > 1]

```

`doublons_colonne` vaut `["Ali"]` : cette fois, le doublon est bien détecté, car on compare uniquement la colonne concernée.

Corrigé

1. `doublons_naifs` vaut `[]` (liste vide) : aucun doublon n'est détecté. Pourtant, "Ali" apparaît bien deux fois dans la colonne `nom`.

2. Les deux lignes ont le même nom mais des valeurs **différentes** pour `age` et `classe` : converties en chaînes avec `str(...)`, ce sont donc deux chaînes **différentes**, qui n'apparaissent chacune qu'une seule fois. La comparaison ligne entière ne peut détecter que des lignes strictement identiques sur **toutes** les colonnes, pas un doublon sur une seule colonne.

3.

```

1 noms = [ligne["nom"] for ligne in élèves]
2 doublons_colonne = [n for n in set(noms) if noms.count(n) > 1]

```

◆ Corrigés

doublons_colonne vaut ["Ali"] : cette fois, le doublon est bien détecté, car on compare uniquement la colonne concernée.

4 Langages et programmation

4.1 Constructions élémentaires

DÉFINITION D1

Les **constructions élémentaires** d'un langage de programmation impératif sont : la **séquence** (exécuter des instructions les unes après les autres), l'**affectation** (donner une valeur à une variable), la **conditionnelle** (if/else), les **boucles** (**bornées**, for, dont le nombre de répétitions est connu à l'avance ; **non bornées**, while, qui se répètent tant qu'une condition est vraie), et l'**appel de fonction**.

EXEMPLE La fonction `somme_chiffres` illustre ces cinq constructions :

CODE DE RÉFÉRENCE — Somme des chiffres d'un entier (toutes les constructions de base)

```
1 def somme_chiffres(n):
2     """Renvoie la somme des chiffres de l'entier positif n."""
3     total = 0                # affectation
4     while n > 0:              # boucle NON bornée (while)
5         chiffre = n % 10
6         total = total + chiffre # séquence d'instructions
7         n = n // 10
8     return total
9
10 for valeur in range(3):      # boucle bornée (for)
11     print(valeur, somme_chiffres(valeur * 100 + 34)) # appel de fonction
```

◆ **PROPRIÉTÉ Conditionnelle** L'instruction if ... elif ... else ... exécute un bloc d'instructions différent selon qu'une (ou plusieurs) condition(s) booléenne(s) est/sont vraie(s).

```
1 def est_pair(n):
2     if n % 2 == 0:
3         return True
4     else:
5         return False
```

⚠ ERREUR FRÉQUENTE

Confondre boucle **bornée** (for, nombre d'itérations connu à l'avance, par exemple `for i in range(10)`) et boucle **non bornée** (while, le nombre d'itérations dépend de l'évolution d'une condition et n'est pas toujours connu à l'avance). Utiliser une boucle while sans faire évoluer la condition qu'elle teste crée une **boucle infinie**.

» **Python À la machine.** Écris une fonction `compte_a_rebours(n)` qui affiche les entiers de n à 0 en utilisant une boucle `while` (boucle non bornée), puis une version `compte_a_rebours_for(n)` utilisant `for` (boucle bornée, avec `range` et un pas négatif). Compare les deux versions : laquelle te semble la plus naturelle ici ?

4.2 Diversité et unité des langages de programmation

♦ **PROPRIÉTÉ Unité** Tous les langages de programmation impératifs partagent le même noyau de **constructions élémentaires** (§0.1) : c'est ce qui permet de reconnaître un algorithme même écrit dans un langage inconnu. Ce qui varie, ce sont les **traits particuliers** : syntaxe, typage, gestion des blocs d'instructions.

EXEMPLE La même fonction, « renvoyer le plus grand de deux nombres », écrite en Python puis dans un langage de style C :

```
1 def maximum(a, b):
2     if a > b:
3         return a
4     else:
5         return b
```

```
int maximum(int a, int b) {
    if (a > b) {
        return a;
    } else {
        return b;
    }
}
```

♦ **PROPRIÉTÉ Traits communs et traits particuliers** **Traits communs** aux deux écritures ci-dessus : une fonction nommée `maximum` prenant deux paramètres, une conditionnelle `if/else`, un `return` dans chaque branche. **Traits particuliers** au langage de style C : les types sont déclarés explicitement (`int`), les blocs d'instructions sont délimités par des accolades `{}` et non par l'indentation, chaque instruction se termine par un point-virgule.

DÉFINITION D2

Un langage est dit à **typage explicite** si le programmeur doit indiquer le type de chaque variable (comme en C ou en Java) ; il est à **typage implicite** (ou dynamique) si l'interpréteur déduit le type automatiquement (comme en Python).

EXEMPLE En Python, l'indentation **délimite** les blocs (elle fait partie de la syntaxe) ; dans un langage de style C, ce sont les accolades qui délimitent les blocs, et l'indentation n'est qu'une convention de lisibilité (le programme fonctionnerait même mal indenté).

ERREUR FRÉQUENTE

Croire qu'un algorithme « appartient » à un langage précis. Un même algorithme (par exemple « chercher le maximum d'une liste ») peut s'écrire dans n'importe quel langage Turing-complet : c'est la **traduction** qui change, pas l'algorithme lui-même.

» **Python À la machine.** Recherche (dans la documentation d'un langage que tu ne connais pas, comme JavaScript ou Scratch) comment s'écrit une boucle for et une conditionnelle if. Identifie les traits communs avec Python et les traits particuliers à ce langage.

4.3 Spécifier une fonction

DÉFINITION D3

Spécifier une fonction, c'est décrire précisément ce qu'elle fait **avant** de l'écrire : son **prototype** (nom, paramètres, valeur renvoyée), ses **préconditions** (ce que les arguments doivent vérifier pour que la fonction ait un sens) et ses **postconditions** (ce que le résultat doit vérifier).

◆ **PROPRIÉTÉ Rôle des pré/postconditions** Une **précondition** est une hypothèse sur les **arguments**, que l'appelant doit garantir. Une **postcondition** est une garantie sur le **résultat**, que la fonction doit assurer si les préconditions sont respectées. En Python, on peut vérifier ces conditions avec l'instruction `assert`.

CODE DE RÉFÉRENCE — Spécification d'une fonction avec préconditions et postconditions

```
1 def aire_triangle(base, hauteur):
2     """Calcule l'aire d'un triangle à partir de sa base et sa hauteur.
3
4     Préconditions : base > 0 et hauteur > 0.
5     Postcondition : le resultat renvoie est strictement positif.
6     """
7     assert base > 0, "la base doit etre strictement positive"
8     assert hauteur > 0, "la hauteur doit etre strictement positive"
9     aire = base * hauteur / 2
10    assert aire > 0
11    return aire
12
13 print(aire_triangle(6, 4)) # 12.0
```

EXEMPLE Appeler `aire_triangle(-2, 4)` viole la précondition `base > 0` : l'instruction `assert` interrompt le programme avec une `AssertionError` et le message associé, plutôt que de renvoyer un résultat absurde (une aire négative).

◆ **PROPRIÉTÉ Le prototype** Le **prototype** d'une fonction résume sa signature : son nom, le nom et le rôle de chaque paramètre, et le type de la valeur renvoyée. En Python, on l'exprime généralement dans la première ligne de la **docstring** (chaîne de documentation entre triple guillemets), juste après la définition de la fonction.

⚠ ERREUR FRÉQUENTE

Écrire le code d'une fonction **avant** d'avoir réfléchi à ses préconditions. Sans préconditions explicites, une fonction peut recevoir des arguments absurdes (une base négative, une liste vide) et produire un résultat incorrect **sans qu'aucune erreur ne soit signalée** — le bug se manifeste alors bien plus tard, loin de sa cause réelle.

» **Python À la machine.** Écris une fonction `moyenne(notes)` qui calcule la moyenne d'une liste de notes. Réfléchis à sa précondition (la liste peut-elle être vide ?) et à sa postcondition (dans quel intervalle doit se situer le résultat, si les notes sont entre 0 et 20 ?). Ajoute les `assert` correspondants.

4.4 Mise au point de programmes

DÉFINITION D4

Un **jeu de tests** est un ensemble d'appels de fonction, chacun accompagné du résultat attendu, utilisé pour vérifier qu'un programme se comporte correctement. Mettre au point un programme, c'est faire évoluer son code jusqu'à ce qu'il réussisse tous les tests du jeu.

EXEMPLE Un élève écrit une fonction censée renvoyer le maximum d'une liste de nombres :

```
1 def maximum_bugue(liste):
2     maxi = 0
3     for x in liste:
4         if x > maxi:
5             maxi = x
6     return maxi
7
8 print(maximum_bugue([3, 7, 2]))
```

7

♦ **PROPRIÉTÉ** Un jeu de tests réussi ne prouve pas la correction. Le succès d'un jeu de tests montre seulement que le programme fonctionne **sur les cas testés**. Il ne garantit **pas** que le programme est correct pour tous les cas possibles : un jeu de tests incomplet peut laisser passer un bug.

⚠ **CONTRE-EXEMPLE** Le test précédent (liste de nombres positifs) réussit, mais `maximum_bugue` échoue sur une liste ne contenant que des nombres **négatifs** :

```
»> maximum_bugue([-5, -1, -8])
0
```

⚠ ERREUR FRÉQUENTE

Initialiser un accumulateur de maximum à 0 (EF). Pour une liste **non vide** (précondition : la liste est non vide), c'est correct **si et seulement si** elle contient une valeur positive ou nulle, c'est-à-dire si $\max(\text{liste}) \geq 0$. Ainsi, `[-5, 0, -8]` donne correctement 0, tandis que

`[-5, -1, -8]` donne à tort 0 au lieu de `-1`. Il faut initialiser l'accumulateur avec le **premier élément** de la liste, pas avec une constante arbitraire.

CODE DE RÉFÉRENCE — Version corrigée, avec jeu de tests plus complet

```

1 def maximum(liste):
2     """Renvoie le plus grand element de 'liste'.
3
4     Precondition : 'liste' est non vide.
5     """
6     assert len(liste) > 0
7     maxi = liste[0]
8     for x in liste[1:]:
9         if x > maxi:
10            maxi = x
11    return maxi
12
13 # Jeu de tests couvrant plusieurs cas
14 assert maximum([3, 7, 2]) == 7          # cas "normal", positifs
15 assert maximum([-5, -1, -8]) == -1      # cas limite : tous negatifs
16 assert maximum([42]) == 42             # cas limite : un seul element

```

◆ **PROPRIÉTÉ Concevoir un bon jeu de tests** Un jeu de tests efficace couvre : un cas « normal », un ou plusieurs cas **limites** (liste d'un seul élément, valeurs négatives, valeur nulle, chaîne vide...), et si pertinent un cas où une précondition est violée (pour vérifier qu'une erreur est bien signalée).

» **Python À la machine.** Écris une fonction `est_palindrome(mot)` qui teste si un mot se lit de la même façon à l'endroit et à l'envers. Écris un jeu de tests d'au moins 4 cas (un mot palindrome, un mot non palindrome, un mot d'une seule lettre, la chaîne vide) avant même d'écrire le code de la fonction, puis implémente-la jusqu'à ce que tous les tests passent.

4.5 Utiliser une bibliothèque

DÉFINITION D5

Une **bibliothèque** (ou *module*) est un ensemble de fonctions déjà écrites et testées, que l'on peut réutiliser avec l'instruction `import`. Utiliser une bibliothèque évite de « réinventer la roue » : c'est l'intérêt de la **modularisation**.

◆ **PROPRIÉTÉ Lire la documentation** Avant d'utiliser une fonction d'une bibliothèque, on consulte sa **documentation** : son prototype (paramètres, valeur renvoyée), une description de son comportement, parfois des exemples. En Python, la fonction `help(...)` affiche la docstring d'une fonction, directement dans l'interpréteur.

EXEMPLE Le module `math` fournit des fonctions mathématiques prêtes à l'emploi :

```

1 import math
2
3 print(math.gcd(48, 18))    # 6 : plus grand commun diviseur
4 print(math.isqrt(50))     # 7 : partie entiere de la racine carree
5 print(math.ceil(4.2))     # 5 : arrondi a l'entier superieur
6 print(math.floor(4.8))    # 4 : arrondi a l'entier inferieur

```

```

>>> help(math.isqrt)
Help on built-in function isqrt in module math:

isqrt(n, /)
    Return the integer part of the square root of the input.

```

⚠ ERREUR FRÉQUENTE

Utiliser une fonction d'une bibliothèque **sans en lire la documentation**, en devinant son comportement. Par exemple, `math.isqrt(48)` renvoie la partie **entière tronquée** de la racine carrée (6, car $6^2 = 36 \leq 48 < 49 = 7^2$), ce qui n'est **pas** toujours la même chose qu'un **arrondi** : `round(math.sqrt(48))` vaut 7 (la racine exacte $\approx 6,93$ s'arrondit à 7), une valeur différente. Deux fonctions qui se ressemblent peuvent avoir des comportements différents : il faut vérifier dans la documentation.

◆ **PROPRIÉTÉ Aucune connaissance exhaustive exigée** Le programme n'exige **aucune connaissance exhaustive** d'une bibliothèque particulière : savoir **chercher** et **lire** une documentation pour trouver la bonne fonction est plus important que de mémoriser toutes les fonctions disponibles.

» **Python À la machine.** Utilisez `help(str.split)` (ou consultez une documentation en ligne) pour comprendre ce que fait la méthode `split` des chaînes de caractères. Utilisez-la pour découper la chaîne "pomme,poire,banane" en une liste de trois mots, à partir de la virgule comme séparateur.

4.6 Constructions élémentaires

DÉFINITION D1

Les **constructions élémentaires** d'un langage de programmation impératif sont : la **séquence** (exécuter des instructions les unes après les autres), l'**affectation** (donner une valeur à une variable), la **conditionnelle** (if/else), les **boucles** (**bornées**, for, dont le nombre de répétitions est connu à l'avance ; **non bornées**, while, qui se répètent tant qu'une condition est vraie), et l'**appel de fonction**.

EXEMPLE La fonction `somme_chiffres` illustre ces cinq constructions :

CODE DE RÉFÉRENCE — Somme des chiffres d'un entier (toutes les constructions de base)

```

1 def somme_chiffres(n):
2     """Renvoie la somme des chiffres de l'entier positif n."""
3     total = 0                # affectation
4     while n > 0:             # boucle NON bornée (while)

```



```

5     chiffre = n % 10
6     total = total + chiffre # sequence d'instructions
7     n = n // 10
8     return total
9
10    for valeur in range(3):          # boucle bornee (for)
11        print(valeur, somme_chiffres(valeur * 100 + 34)) # appel de fonction

```

◆ **PROPRIÉTÉ Conditionnelle** L'instruction if ... elif ... else ... exécute un bloc d'instructions différent selon qu'une (ou plusieurs) condition(s) booléenne(s) est/sont vraie(s).

```

1 def est_paire(n):
2     if n % 2 == 0:
3         return True
4     else:
5         return False

```

⚠ ERREUR FRÉQUENTE

Confondre boucle **bornée** (for, nombre d'itérations connu à l'avance, par exemple for i in range(10)) et boucle **non bornée** (while, le nombre d'itérations dépend de l'évolution d'une condition et n'est pas toujours connu à l'avance). Utiliser une boucle while sans faire évoluer la condition qu'elle teste crée une **boucle infinie**.

» **Python À la machine.** Écris une fonction compte_a_rebours(n) qui affiche les entiers de n à 0 en utilisant une boucle while (boucle non bornée), puis une version compte_a_rebours_for(n) utilisant for (boucle bornée, avec range et un pas négatif). Compare les deux versions : laquelle te semble la plus naturelle ici ?

4.7 Diversité et unité des langages de programmation

◆ **PROPRIÉTÉ Unité** Tous les langages de programmation impératifs partagent le même noyau de **constructions élémentaires** (§0.1) : c'est ce qui permet de reconnaître un algorithme même écrit dans un langage inconnu. Ce qui varie, ce sont les **traits particuliers** : syntaxe, typage, gestion des blocs d'instructions.

EXEMPLE La même fonction, « renvoyer le plus grand de deux nombres », écrite en Python puis dans un langage de style C :

```

1 def maximum(a, b):
2     if a > b:
3         return a
4     else:
5         return b

```

```

int maximum(int a, int b) {
    if (a > b) {

```

```

        return a;
    } else {
        return b;
    }
}

```

◆ **PROPRIÉTÉ Traits communs et traits particuliers** **Traits communs** aux deux écritures ci-dessus : une fonction nommée maximum prenant deux paramètres, une conditionnelle if/else, un return dans chaque branche. **Traits particuliers** au langage de style C : les types sont déclarés explicitement (int), les blocs d'instructions sont délimités par des accolades {} et non par l'indentation, chaque instruction se termine par un point-virgule.

DÉFINITION D2

Un langage est dit à **typage explicite** si le programmeur doit indiquer le type de chaque variable (comme en C ou en Java) ; il est à **typage implicite** (ou dynamique) si l'interpréteur déduit le type automatiquement (comme en Python).

EXEMPLE En Python, l'indentation **délimite** les blocs (elle fait partie de la syntaxe) ; dans un langage de style C, ce sont les accolades qui délimitent les blocs, et l'indentation n'est qu'une convention de lisibilité (le programme fonctionnerait même mal indenté).

⚠ ERREUR FRÉQUENTE

Croire qu'un algorithme « appartient » à un langage précis. Un même algorithme (par exemple « chercher le maximum d'une liste ») peut s'écrire dans n'importe quel langage Turing-complet : c'est la **traduction** qui change, pas l'algorithme lui-même.

» **Python À la machine.** Recherche (dans la documentation d'un langage que tu ne connais pas, comme JavaScript ou Scratch) comment s'écrit une boucle for et une conditionnelle if. Identifie les traits communs avec Python et les traits particuliers à ce langage.

4.8 Constructions élémentaires

DÉFINITION D1

Les **constructions élémentaires** d'un langage de programmation impératif sont : la **séquence** (exécuter des instructions les unes après les autres), l'**affectation** (donner une valeur à une variable), la **conditionnelle** (if/else), les **boucles** (**bornées**, for, dont le nombre de répétitions est connu à l'avance ; **non bornées**, while, qui se répètent tant qu'une condition est vraie), et l'**appel de fonction**.

EXEMPLE La fonction somme_chiffres illustre ces cinq constructions :

CODE DE RÉFÉRENCE — Somme des chiffres d'un entier (toutes les constructions de base)

```

1 def somme_chiffres(n):

```

```

2 """Renvoie la somme des chiffres de l'entier positif n."""
3 total = 0 # affectation
4 while n > 0: # boucle NON bornee (while)
5     chiffre = n % 10
6     total = total + chiffre # sequence d'instructions
7     n = n // 10
8     return total
9
10 for valeur in range(3): # boucle bornee (for)
11     print(valeur, somme_chiffres(valeur * 100 + 34)) # appel de fonction

```

◆ **PROPRIÉTÉ Conditionnelle** L'instruction if ... elif ... else ... exécute un bloc d'instructions différent selon qu'une (ou plusieurs) condition(s) booléenne(s) est/sont vraie(s).

```

1 def est_paire(n):
2     if n % 2 == 0:
3         return True
4     else:
5         return False

```

⚠ ERREUR FRÉQUENTE

Confondre boucle **bornée** (for, nombre d'itérations connu à l'avance, par exemple for i in range(10)) et boucle **non bornée** (while, le nombre d'itérations dépend de l'évolution d'une condition et n'est pas toujours connu à l'avance). Utiliser une boucle while sans faire évoluer la condition qu'elle teste crée une **boucle infinie**.

» **Python À la machine.** Écris une fonction compte_a_rebours(n) qui affiche les entiers de n à 0 en utilisant une boucle while (boucle non bornée), puis une version compte_a_rebours_for(n) utilisant for (boucle bornée, avec range et un pas négatif). Compare les deux versions : laquelle te semble la plus naturelle ici ?

4.9 Diversité et unité des langages de programmation

◆ **PROPRIÉTÉ Unité** Tous les langages de programmation impératifs partagent le même noyau de **constructions élémentaires** (§0.1) : c'est ce qui permet de reconnaître un algorithme même écrit dans un langage inconnu. Ce qui varie, ce sont les **traits particuliers** : syntaxe, typage, gestion des blocs d'instructions.

EXEMPLE La même fonction, « renvoyer le plus grand de deux nombres », écrite en Python puis dans un langage de style C :

```

1 def maximum(a, b):
2     if a > b:
3         return a
4     else:
5         return b

```

```
int maximum(int a, int b) {
    if (a > b) {
        return a;
    } else {
        return b;
    }
}
```

◆ **PROPRIÉTÉ Traits communs et traits particuliers** **Traits communs** aux deux écritures ci-dessus : une fonction nommée maximum prenant deux paramètres, une conditionnelle if/else, un return dans chaque branche. **Traits particuliers** au langage de style C : les types sont déclarés explicitement (int), les blocs d'instructions sont délimités par des accolades {} et non par l'indentation, chaque instruction se termine par un point-virgule.

DÉFINITION D2

Un langage est dit à **typage explicite** si le programmeur doit indiquer le type de chaque variable (comme en C ou en Java) ; il est à **typage implicite** (ou dynamique) si l'interpréteur déduit le type automatiquement (comme en Python).

EXEMPLE En Python, l'indentation **délimite** les blocs (elle fait partie de la syntaxe) ; dans un langage de style C, ce sont les accolades qui délimitent les blocs, et l'indentation n'est qu'une convention de lisibilité (le programme fonctionnerait même mal indenté).

⚠ ERREUR FRÉQUENTE

Croire qu'un algorithme « appartient » à un langage précis. Un même algorithme (par exemple « chercher le maximum d'une liste ») peut s'écrire dans n'importe quel langage Turing-complet : c'est la **traduction** qui change, pas l'algorithme lui-même.

» **Python À la machine.** Recherche (dans la documentation d'un langage que tu ne connais pas, comme JavaScript ou Scratch) comment s'écrit une boucle for et une conditionnelle if. Identifie les traits communs avec Python et les traits particuliers à ce langage.

4.10 Spécifier une fonction

DÉFINITION D3

Spécifier une fonction, c'est décrire précisément ce qu'elle fait **avant** de l'écrire : son **prototype** (nom, paramètres, valeur renvoyée), ses **préconditions** (ce que les arguments doivent vérifier pour que la fonction ait un sens) et ses **postconditions** (ce que le résultat doit vérifier).

◆ **PROPRIÉTÉ Rôle des pré/postconditions** Une **précondition** est une hypothèse sur les **arguments**, que l'appelant doit garantir. Une **postcondition** est une garantie sur le **résultat**, que la fonction doit assurer si les préconditions sont respectées. En Python, on peut vérifier ces conditions avec l'instruction assert.

CODE DE RÉFÉRENCE — Spécification d'une fonction avec préconditions et postconditions

```

1 def aire_triangle(base, hauteur):
2     """Calcule l'aire d'un triangle à partir de sa base et sa hauteur.
3
4     Preconditions : base > 0 et hauteur > 0.
5     Postcondition : le resultat renvoyé est strictement positif.
6     """
7     assert base > 0, "la base doit être strictement positive"
8     assert hauteur > 0, "la hauteur doit être strictement positive"
9     aire = base * hauteur / 2
10    assert aire > 0
11    return aire
12
13 print(aire_triangle(6, 4)) # 12.0

```

EXEMPLE Appeler `aire_triangle(-2, 4)` viole la précondition `base > 0` : l'instruction `assert` interrompt le programme avec une `AssertionError` et le message associé, plutôt que de renvoyer un résultat absurde (une aire négative).

◆ **PROPRIÉTÉ Le prototype** Le **prototype** d'une fonction résume sa signature : son nom, le nom et le rôle de chaque paramètre, et le type de la valeur renvoyée. En Python, on l'exprime généralement dans la première ligne de la **docstring** (chaîne de documentation entre triple guillemets), juste après la définition de la fonction.

⚠ ERREUR FRÉQUENTE

Écrire le code d'une fonction **avant** d'avoir réfléchi à ses préconditions. Sans préconditions explicites, une fonction peut recevoir des arguments absurdes (une base négative, une liste vide) et produire un résultat incorrect **sans qu'aucune erreur ne soit signalée** — le bug se manifeste alors bien plus tard, loin de sa cause réelle.

» **Python À la machine.** Écris une fonction `moyenne(notes)` qui calcule la moyenne d'une liste de notes. Réfléchis à sa précondition (la liste peut-elle être vide ?) et à sa postcondition (dans quel intervalle doit se situer le résultat, si les notes sont entre 0 et 20 ?). Ajoute les `assert` correspondants.

4.11 Mise au point de programmes

DÉFINITION D4

Un **jeu de tests** est un ensemble d'appels de fonction, chacun accompagné du résultat attendu, utilisé pour vérifier qu'un programme se comporte correctement. Mettre au point un programme, c'est faire évoluer son code jusqu'à ce qu'il réussisse tous les tests du jeu.

EXEMPLE Un élève écrit une fonction censée renvoyer le maximum d'une liste de nombres :

```

1 def maximum_bugue(liste):
2     maxi = 0
3     for x in liste:

```

```

4         if x > maxi:
5             maxi = x
6         return maxi
7
8 print(maximum_bugue([3, 7, 2]))

```

7

◆ **PROPRIÉTÉ Un jeu de tests réussi ne prouve pas la correction** Le succès d'un jeu de tests montre seulement que le programme fonctionne **sur les cas testés**. Il ne garantit **pas** que le programme est correct pour tous les cas possibles : un jeu de tests incomplet peut laisser passer un bug.

⚠ **CONTRE-EXEMPLE** Le test précédent (liste de nombres positifs) réussit, mais `maximum_bugue` échoue sur une liste ne contenant que des nombres **négatifs** :

```

>>> maximum_bugue([-5, -1, -8])
0

```

⚠ ERREUR FRÉQUENTE

Initialiser un accumulateur de maximum à 0 (EF). Pour une liste **non vide** (précondition : la liste est non vide), c'est correct **si et seulement si** elle contient une valeur positive ou nulle, c'est-à-dire si $\max(\text{liste}) \geq 0$. Ainsi, `[-5, 0, -8]` donne correctement 0, tandis que `[-5, -1, -8]` donne à tort 0 au lieu de -1. Il faut initialiser l'accumulateur avec le **premier élément** de la liste, pas avec une constante arbitraire.

CODE DE RÉFÉRENCE — Version corrigée, avec jeu de tests plus complet

```

1 def maximum(liste):
2     """Renvoie le plus grand élément de 'liste'.

```

◆ **PROPRIÉTÉ Concevoir un bon jeu de tests** Un jeu de tests efficace couvre : un cas « normal », un ou plusieurs **cas limites** (liste d'un seul élément, valeurs négatives, valeur nulle, chaîne vide...), et si pertinent un cas où une précondition est violée (pour vérifier qu'une erreur est bien signalée).

» **Python À la machine.** Écris une fonction `est_palindrome(mot)` qui teste si un mot se lit de la même façon à l'endroit et à l'envers. Écris un jeu de tests d'au moins 4 cas (un mot palindrome, un mot non palindrome, un mot d'une seule lettre, la chaîne vide) avant même d'écrire le code de la fonction, puis implémente-la jusqu'à ce que tous les tests passent.

4.12 Utiliser une bibliothèque

DÉFINITION D5

Une **bibliothèque** (ou *module*) est un ensemble de fonctions déjà écrites et testées, que l'on peut réutiliser avec l'instruction `import`. Utiliser une bibliothèque évite de « réinventer la roue » : c'est l'intérêt de la **modularisation**.

♦ **PROPRIÉTÉ Lire la documentation** Avant d'utiliser une fonction d'une bibliothèque, on consulte sa **documentation** : son prototype (paramètres, valeur renvoyée), une description de son comportement, parfois des exemples. En Python, la fonction `help(...)` affiche la docstring d'une fonction, directement dans l'interpréteur.

EXEMPLE Le module `math` fournit des fonctions mathématiques prêtes à l'emploi :

```
1 import math
2
3 print(math.gcd(48, 18))    # 6 : plus grand commun diviseur
4 print(math.isqrt(50))     # 7 : partie entiere de la racine carree
5 print(math.ceil(4.2))     # 5 : arrondi a l'entier superieur
6 print(math.floor(4.8))    # 4 : arrondi a l'entier inferieur
```

```
>>> help(math.isqrt)
Help on built-in function isqrt in module math:

isqrt(n, /)
    Return the integer part of the square root of the input.
```

⚠ ERREUR FRÉQUENTE

Utiliser une fonction d'une bibliothèque **sans en lire la documentation**, en devinant son comportement. Par exemple, `math.isqrt(48)` renvoie la partie **entière tronquée** de la racine carrée (6, car $6^2 = 36 \leq 48 < 49 = 7^2$), ce qui n'est **pas** toujours la même chose qu'un **arrondi** : `round(math.sqrt(48))` vaut 7 (la racine exacte $\approx 6,93$ s'arrondit à 7), une valeur différente. Deux fonctions qui se ressemblent peuvent avoir des comportements différents : il faut vérifier dans la documentation.

♦ **PROPRIÉTÉ Aucune connaissance exhaustive exigée** Le programme n'exige **aucune connaissance exhaustive** d'une bibliothèque particulière : savoir **chercher** et **lire** une documentation pour trouver la bonne fonction est plus important que de mémoriser toutes les fonctions disponibles.

» **Python À la machine.** Utilisez `help(str.split)` (ou consultez une documentation en ligne) pour comprendre ce que fait la méthode `split` des chaînes de caractères. Utilisez-la pour découper la chaîne "pomme,poire,banane" en une liste de trois mots, à partir de la virgule comme séparateur.

Méthodes

► Méthode directe de travail sur le sujet.

Exercice 1 ♦

1. Écrire une fonction `somme_pairs(liste)` qui renvoie la somme des éléments **pairs** d'une liste d'entiers, en utilisant une boucle `for` et une conditionnelle.
2. Identifier, dans le code de cette fonction, une occurrence de chacune des constructions élémentaires suivantes : affectation, boucle bornée, conditionnelle.
3. Tester la fonction sur `[1, 2, 3, 4, 5, 6]` (résultat attendu : 12) et sur `[1, 3, 5]` (résultat attendu : 0).

Exercice 2 ♦

On donne le code suivant, écrit dans un langage de style C, qui calcule la factorielle d'un entier `n` :

```
int factorielle(int n) {
    int resultat = 1;
    for (int i = 1; i ≤ n; i = i + 1) {
        resultat = resultat * i;
    }
    return resultat;
}
```

1. Traduire ce code en Python (fonction `factorielle(n)`).
2. Identifier deux traits **communs** entre les deux écritures, et deux traits **particuliers** au langage de style C (absents en Python).
3. Vérifier que `factorielle(5)` vaut 120 et que `factorielle(0)` vaut 1.

Exercice 3 ♦♦

On veut écrire une fonction `perimetre_rectangle(largeur, longueur)` qui calcule le périmètre d'un rectangle.

1. Proposer une précondition sur `largeur` et `longueur`, et une postcondition sur le résultat.
2. Écrire la fonction, avec des `assert` correspondant à ces conditions.
3. Vérifier que `perimetre_rectangle(3, 5)` vaut 16, et que `perimetre_rectangle(-1, 5)` déclenche bien une erreur.

Exercice 4 ♦♦

Un élève teste sa fonction `est_premier_bugue` uniquement sur 7 (nombre premier) et 8 (non premier), et conclut qu'elle est correcte :

```
1 def est_premier_bugue(n):
2     for i in range(2, n):
3         if n % i == 0:
4             return False
5     return True
6
```



```

7 print(est_premier_bugue(7)) # True
8 print(est_premier_bugue(8)) # False

```

1. Que renvoie `est_premier_bugue(1)` ? Est-ce le résultat attendu (1 n'est pas un nombre premier) ?
2. Pourquoi le jeu de tests de l'élève (seulement 7 et 8) n'a-t-il pas permis de détecter ce bug ?
3. Corriger la fonction, et proposer un jeu de tests plus complet (au moins 4 cas, dont un cas limite) qui aurait détecté le bug.

Exercice 5

On souhaite calculer la moyenne et la médiane de la liste de notes [10, 12, 14, 20], sans les recalculer « à la main ».

1. À l'aide de `help(...)` ou d'une recherche dans la documentation, trouver le nom du module de la bibliothèque standard Python qui contient des fonctions statistiques de base, et le nom de la fonction qui calcule une moyenne.
2. Écrire le code qui importe ce module et calcule la moyenne, puis la médiane, de la liste de notes.
3. Une postcondition raisonnable pour une fonction de moyenne de notes sur 20 serait « le résultat est compris entre 0 et 20 ». Vérifier que le résultat obtenu respecte cette postcondition.

Exercice 6

1. Écrire une fonction `somme_pairs(liste)` qui renvoie la somme des éléments **pairs** d'une liste d'entiers, en utilisant une boucle `for` et une conditionnelle.
2. Identifier, dans le code de cette fonction, une occurrence de chacune des constructions élémentaires suivantes : affectation, boucle bornée, conditionnelle.
3. Tester la fonction sur [1, 2, 3, 4, 5, 6] (résultat attendu : 12) et sur [1, 3, 5] (résultat attendu : 0).

Exercice 7

On donne le code suivant, écrit dans un langage de style C, qui calcule la factorielle d'un entier `n` :

```

int factorielle(int n) {
    int resultat = 1;
    for (int i = 1; i ≤ n; i = i + 1) {
        resultat = resultat * i;
    }
    return resultat;
}

```

1. Traduire ce code en Python (fonction `factorielle(n)`).
2. Identifier deux traits **communs** entre les deux écritures, et deux traits **particuliers** au langage de style C (absents en Python).
3. Vérifier que `factorielle(5)` vaut 120 et que `factorielle(0)` vaut 1.

Exercice 8

On veut écrire une fonction `perimetre_rectangle(largeur, longueur)` qui calcule le périmètre d'un rectangle.

1. Proposer une précondition sur `largeur` et `longueur`, et une postcondition sur le résultat.
2. Écrire la fonction, avec des `assert` correspondant à ces conditions.
3. Vérifier que `perimetre_rectangle(3, 5)` vaut 16, et que `perimetre_rectangle(-1, 5)` déclenche bien une erreur.

Exercice 9

Un élève teste sa fonction `est_premier_bugue` uniquement sur 7 (nombre premier) et 8 (non premier), et conclut qu'elle est correcte :

```
1 def est_premier_bugue(n):
2     for i in range(2, n):
3         if n % i == 0:
4             return False
5     return True
6
7 print(est_premier_bugue(7)) # True
8 print(est_premier_bugue(8)) # False
```

1. Que renvoie `est_premier_bugue(1)` ? Est-ce le résultat attendu (1 n'est pas un nombre premier) ?
2. Pourquoi le jeu de tests de l'élève (seulement 7 et 8) n'a-t-il pas permis de détecter ce bug ?
3. Corriger la fonction, et proposer un jeu de tests plus complet (au moins 4 cas, dont un cas limite) qui aurait détecté le bug.

Exercice 10

On souhaite calculer la moyenne et la médiane de la liste de notes [10, 12, 14, 20], sans les recalculer « à la main ».

1. À l'aide de `help(...)` ou d'une recherche dans la documentation, trouver le nom du module de la bibliothèque standard Python qui contient des fonctions statistiques de base, et le nom de la fonction qui calcule une moyenne.
2. Écrire le code qui importe ce module et calcule la moyenne, puis la médiane, de la liste de notes.
3. Une postcondition raisonnable pour une fonction de moyenne de notes sur 20 serait « le résultat est compris entre 0 et 20 ». Vérifier que le résultat obtenu respecte cette postcondition.

Exercice 11

1. Écrire une fonction `somme_pairs(liste)` qui renvoie la somme des éléments **pairs** d'une liste d'entiers, en utilisant une boucle `for` et une conditionnelle.
2. Identifier, dans le code de cette fonction, une occurrence de chacune des constructions élémentaires suivantes : affectation, boucle bornée, conditionnelle.
3. Tester la fonction sur [1, 2, 3, 4, 5, 6] (résultat attendu : 12) et sur [1, 3, 5] (résultat attendu : 0).

Exercice 12

On donne le code suivant, écrit dans un langage de style C, qui calcule la factorielle d'un entier `n` :

```
int factorielle(int n) {
    int resultat = 1;
    for (int i = 1; i ≤ n; i = i + 1) {
        resultat = resultat * i;
    }
    return resultat;
}
```

1. Traduire ce code en Python (fonction `factorielle(n)`).
2. Identifier deux traits **communs** entre les deux écritures, et deux traits **particuliers** au langage de style C (absents en Python).
3. Vérifier que `factorielle(5)` vaut 120 et que `factorielle(0)` vaut 1.

Exercice 13 ♦♦

On veut écrire une fonction `perimetre_rectangle(largeur, longueur)` qui calcule le périmètre d'un rectangle.

1. Proposer une précondition sur `largeur` et `longueur`, et une postcondition sur le résultat.
2. Écrire la fonction, avec des `assert` correspondant à ces conditions.
3. Vérifier que `perimetre_rectangle(3, 5)` vaut 16, et que `perimetre_rectangle(-1, 5)` déclenche bien une erreur.

Exercice 14 ♦♦

Un élève teste sa fonction `est_premier_bugue` uniquement sur 7 (nombre premier) et 8 (non premier), et conclut qu'elle est correcte :

```
1 def est_premier_bugue(n):
2     for i in range(2, n):
3         if n % i == 0:
4             return False
5     return True
6
7 print(est_premier_bugue(7)) # True
8 print(est_premier_bugue(8)) # False
```

1. Que renvoie `est_premier_bugue(1)` ? Est-ce le résultat attendu (1 n'est pas un nombre premier) ?
2. Pourquoi le jeu de tests de l'élève (seulement 7 et 8) n'a-t-il pas permis de détecter ce bug ?
3. Corriger la fonction, et proposer un jeu de tests plus complet (au moins 4 cas, dont un cas limite) qui aurait détecté le bug.

Exercice 15 ♦♦♦

On souhaite calculer la moyenne et la médiane de la liste de notes `[10, 12, 14, 20]`, sans les recalculer « à la main ».

1. À l'aide de `help(...)` ou d'une recherche dans la documentation, trouver le nom du module de la bibliothèque standard Python qui contient des fonctions statistiques de base, et le nom de la fonction qui calcule une moyenne.
2. Écrire le code qui importe ce module et calcule la moyenne, puis la médiane, de la liste de notes.
3. Une postcondition raisonnable pour une fonction de moyenne de notes sur 20 serait « le résultat est compris entre 0 et 20 ». Vérifier que le résultat obtenu respecte cette postcondition.

Exercice 16 ♦

1. Écrire une fonction `somme_pairs(liste)` qui renvoie la somme des éléments **pairs** d'une liste d'entiers, en utilisant une boucle `for` et une conditionnelle.
2. Identifier, dans le code de cette fonction, une occurrence de chacune des constructions élémentaires suivantes : affectation, boucle bornée, conditionnelle.
3. Tester la fonction sur `[1, 2, 3, 4, 5, 6]` (résultat attendu : 12) et sur `[1, 3, 5]` (résultat attendu : 0).

Exercice 17 ♦

On donne le code suivant, écrit dans un langage de style C, qui calcule la factorielle d'un entier `n` :

```
int factorielle(int n) {
    int resultat = 1;
    for (int i = 1; i ≤ n; i = i + 1) {
```

```

        resultat = resultat * i;
    }
    return resultat;
}

```

1. Traduire ce code en Python (fonction `factorielle(n)`).
2. Identifier deux traits **communs** entre les deux écritures, et deux traits **particuliers** au langage de style C (absents en Python).
3. Vérifier que `factorielle(5)` vaut 120 et que `factorielle(0)` vaut 1.

Exercice 18 ♦♦

On veut écrire une fonction `perimetre_rectangle(largeur, longueur)` qui calcule le périmètre d'un rectangle.

1. Proposer une précondition sur largeur et longueur, et une postcondition sur le résultat.
2. Écrire la fonction, avec des `assert` correspondant à ces conditions.
3. Vérifier que `perimetre_rectangle(3, 5)` vaut 16, et que `perimetre_rectangle(-1, 5)` déclenche bien une erreur.

Exercice 19 ♦♦

Un élève teste sa fonction `est_premier_bugue` uniquement sur 7 (nombre premier) et 8 (non premier), et conclut qu'elle est correcte :

```

1 def est_premier_bugue(n):
2     for i in range(2, n):
3         if n % i == 0:
4             return False
5     return True
6
7 print(est_premier_bugue(7)) # True
8 print(est_premier_bugue(8)) # False

```

1. Que renvoie `est_premier_bugue(1)` ? Est-ce le résultat attendu (1 n'est pas un nombre premier) ?
2. Pourquoi le jeu de tests de l'élève (seulement 7 et 8) n'a-t-il pas permis de détecter ce bug ?
3. Corriger la fonction, et proposer un jeu de tests plus complet (au moins 4 cas, dont un cas limite) qui aurait détecté le bug.

Exercice 20 ♦♦♦

On souhaite calculer la moyenne et la médiane de la liste de notes `[10, 12, 14, 20]`, sans les recalculer « à la main ».

1. À l'aide de `help(...)` ou d'une recherche dans la documentation, trouver le nom du module de la bibliothèque standard Python qui contient des fonctions statistiques de base, et le nom de la fonction qui calcule une moyenne.
2. Écrire le code qui importe ce module et calcule la moyenne, puis la médiane, de la liste de notes.
3. Une postcondition raisonnable pour une fonction de moyenne de notes sur 20 serait « le résultat est compris entre 0 et 20 ». Vérifier que le résultat obtenu respecte cette postcondition.

Exercice 21 ♦

1. Écrire une fonction `somme_pairs(liste)` qui renvoie la somme des éléments **pairs** d'une liste d'entiers, en utilisant une boucle `for` et une conditionnelle.

2. Identifier, dans le code de cette fonction, une occurrence de chacune des constructions élémentaires suivantes : affectation, boucle bornée, conditionnelle.
3. Tester la fonction sur $[1, 2, 3, 4, 5, 6]$ (résultat attendu : 12) et sur $[1, 3, 5]$ (résultat attendu : 0).

Exercice 22

On donne le code suivant, écrit dans un langage de style C, qui calcule la factorielle d'un entier n :

```
int factorielle(int n) {
    int resultat = 1;
    for (int i = 1; i ≤ n; i = i + 1) {
        resultat = resultat * i;
    }
    return resultat;
}
```

1. Traduire ce code en Python (fonction `factorielle(n)`).
2. Identifier deux traits **communs** entre les deux écritures, et deux traits **particuliers** au langage de style C (absents en Python).
3. Vérifier que `factorielle(5)` vaut 120 et que `factorielle(0)` vaut 1.

Exercice 23

On veut écrire une fonction `perimetre_rectangle(largeur, longueur)` qui calcule le périmètre d'un rectangle.

1. Proposer une précondition sur `largeur` et `longueur`, et une postcondition sur le résultat.
2. Écrire la fonction, avec des `assert` correspondant à ces conditions.
3. Vérifier que `perimetre_rectangle(3, 5)` vaut 16, et que `perimetre_rectangle(-1, 5)` déclenche bien une erreur.

Exercice 24

Un élève teste sa fonction `est_premier_bugue` uniquement sur 7 (nombre premier) et 8 (non premier), et conclut qu'elle est correcte :

```
1 def est_premier_bugue(n):
2     for i in range(2, n):
3         if n % i == 0:
4             return False
5     return True
6
7 print(est_premier_bugue(7)) # True
8 print(est_premier_bugue(8)) # False
```

1. Que renvoie `est_premier_bugue(1)` ? Est-ce le résultat attendu (1 n'est pas un nombre premier) ?
2. Pourquoi le jeu de tests de l'élève (seulement 7 et 8) n'a-t-il pas permis de détecter ce bug ?
3. Corriger la fonction, et proposer un jeu de tests plus complet (au moins 4 cas, dont un cas limite) qui aurait détecté le bug.

Exercice 25

On souhaite calculer la moyenne et la médiane de la liste de notes $[10, 12, 14, 20]$, sans les recalculer « à la main ».

1. À l'aide de `help(...)` ou d'une recherche dans la documentation, trouver le nom du module de la bibliothèque standard Python qui contient des fonctions statistiques de base, et le nom de la fonction qui calcule une moyenne.
2. Écrire le code qui importe ce module et calcule la moyenne, puis la médiane, de la liste de notes.
3. Une postcondition raisonnable pour une fonction de moyenne de notes sur 20 serait « le résultat est compris entre 0 et 20 ». Vérifier que le résultat obtenu respecte cette postcondition.

Exercice 26

1. Écrire une fonction `somme_pairs(liste)` qui renvoie la somme des éléments **pairs** d'une liste d'entiers, en utilisant une boucle `for` et une conditionnelle.
2. Identifier, dans le code de cette fonction, une occurrence de chacune des constructions élémentaires suivantes : affectation, boucle bornée, conditionnelle.
3. Tester la fonction sur `[1, 2, 3, 4, 5, 6]` (résultat attendu : 12) et sur `[1, 3, 5]` (résultat attendu : 0).

Exercice 27

On donne le code suivant, écrit dans un langage de style C, qui calcule la factorielle d'un entier `n` :

```
int factorielle(int n) {
    int resultat = 1;
    for (int i = 1; i ≤ n; i = i + 1) {
        resultat = resultat * i;
    }
    return resultat;
}
```

1. Traduire ce code en Python (fonction `factorielle(n)`).
2. Identifier deux traits **communs** entre les deux écritures, et deux traits **particuliers** au langage de style C (absents en Python).
3. Vérifier que `factorielle(5)` vaut 120 et que `factorielle(0)` vaut 1.

Exercice 28

On veut écrire une fonction `perimetre_rectangle(largeur, longueur)` qui calcule le périmètre d'un rectangle.

1. Proposer une précondition sur largeur et longueur, et une postcondition sur le résultat.
2. Écrire la fonction, avec des `assert` correspondant à ces conditions.
3. Vérifier que `perimetre_rectangle(3, 5)` vaut 16, et que `perimetre_rectangle(-1, 5)` déclenche bien une erreur.

Exercice 29

Un élève teste sa fonction `est_premier_bugue` uniquement sur 7 (nombre premier) et 8 (non premier), et conclut qu'elle est correcte :

```
1 def est_premier_bugue(n):
2     for i in range(2, n):
3         if n % i == 0:
4             return False
5     return True
6
7 print(est_premier_bugue(7)) # True
8 print(est_premier_bugue(8)) # False
```

1. Que renvoie `est_premier_bugue(1)` ? Est-ce le résultat attendu (1 n'est pas un nombre premier) ?
2. Pourquoi le jeu de tests de l'élève (seulement 7 et 8) n'a-t-il pas permis de détecter ce bug ?
3. Corriger la fonction, et proposer un jeu de tests plus complet (au moins 4 cas, dont un cas limite) qui aurait détecté le bug.

Exercice 30

On souhaite calculer la moyenne et la médiane de la liste de notes [10, 12, 14, 20], sans les recalculer « à la main ».

1. À l'aide de `help(...)` ou d'une recherche dans la documentation, trouver le nom du module de la bibliothèque standard Python qui contient des fonctions statistiques de base, et le nom de la fonction qui calcule une moyenne.
2. Écrire le code qui importe ce module et calcule la moyenne, puis la médiane, de la liste de notes.
3. Une postcondition raisonnable pour une fonction de moyenne de notes sur 20 serait « le résultat est compris entre 0 et 20 ». Vérifier que le résultat obtenu respecte cette postcondition.

Exercice 31

1. Écrire une fonction `somme_pairs(liste)` qui renvoie la somme des éléments **pairs** d'une liste d'entiers, en utilisant une boucle `for` et une conditionnelle.
2. Identifier, dans le code de cette fonction, une occurrence de chacune des constructions élémentaires suivantes : affectation, boucle bornée, conditionnelle.
3. Tester la fonction sur [1, 2, 3, 4, 5, 6] (résultat attendu : 12) et sur [1, 3, 5] (résultat attendu : 0).

Exercice 32

On donne le code suivant, écrit dans un langage de style C, qui calcule la factorielle d'un entier `n` :

```
int factorielle(int n) {
    int resultat = 1;
    for (int i = 1; i ≤ n; i = i + 1) {
        resultat = resultat * i;
    }
    return resultat;
}
```

1. Traduire ce code en Python (fonction `factorielle(n)`).
2. Identifier deux traits **communs** entre les deux écritures, et deux traits **particuliers** au langage de style C (absents en Python).
3. Vérifier que `factorielle(5)` vaut 120 et que `factorielle(0)` vaut 1.

Exercice 33

On veut écrire une fonction `perimetre_rectangle(largeur, longueur)` qui calcule le périmètre d'un rectangle.

1. Proposer une précondition sur `largeur` et `longueur`, et une postcondition sur le résultat.
2. Écrire la fonction, avec des `assert` correspondant à ces conditions.
3. Vérifier que `perimetre_rectangle(3, 5)` vaut 16, et que `perimetre_rectangle(-1, 5)` déclenche bien une erreur.

Exercice 34

Un élève teste sa fonction `est_premier_bugue` uniquement sur 7 (nombre premier) et 8 (non premier), et conclut qu'elle est correcte :

```

1 def est_premier_bugue(n):
2     for i in range(2, n):
3         if n % i == 0:
4             return False
5     return True
6
7 print(est_premier_bugue(7)) # True
8 print(est_premier_bugue(8)) # False

```

1. Que renvoie `est_premier_bugue(1)` ? Est-ce le résultat attendu (1 n'est pas un nombre premier) ?
2. Pourquoi le jeu de tests de l'élève (seulement 7 et 8) n'a-t-il pas permis de détecter ce bug ?
3. Corriger la fonction, et proposer un jeu de tests plus complet (au moins 4 cas, dont un cas limite) qui aurait détecté le bug.

Exercice 35

On souhaite calculer la moyenne et la médiane de la liste de notes [10, 12, 14, 20], sans les recalculer « à la main ».

1. À l'aide de `help(...)` ou d'une recherche dans la documentation, trouver le nom du module de la bibliothèque standard Python qui contient des fonctions statistiques de base, et le nom de la fonction qui calcule une moyenne.
2. Écrire le code qui importe ce module et calcule la moyenne, puis la médiane, de la liste de notes.
3. Une postcondition raisonnable pour une fonction de moyenne de notes sur 20 serait « le résultat est compris entre 0 et 20 ». Vérifier que le résultat obtenu respecte cette postcondition.

Exercice 36

1. Écrire une fonction `somme_paires(liste)` qui renvoie la somme des éléments **pairs** d'une liste d'entiers, en utilisant une boucle `for` et une conditionnelle.
2. Identifier, dans le code de cette fonction, une occurrence de chacune des constructions élémentaires suivantes : affectation, boucle bornée, conditionnelle.
3. Tester la fonction sur [1, 2, 3, 4, 5, 6] (résultat attendu : 12) et sur [1, 3, 5] (résultat attendu : 0).

Exercice 37

On donne le code suivant, écrit dans un langage de style C, qui calcule la factorielle d'un entier `n` :

```

int factorielle(int n) {
    int resultat = 1;
    for (int i = 1; i ≤ n; i = i + 1) {
        resultat = resultat * i;
    }
    return resultat;
}

```

1. Traduire ce code en Python (fonction `factorielle(n)`).
2. Identifier deux traits **communs** entre les deux écritures, et deux traits **particuliers** au langage de style C (absents en Python).
3. Vérifier que `factorielle(5)` vaut 120 et que `factorielle(0)` vaut 1.

Exercice 38

On veut écrire une fonction `perimetre_rectangle(largeur, longueur)` qui calcule le périmètre d'un rectangle.

1. Proposer une précondition sur largeur et longueur, et une postcondition sur le résultat.
2. Écrire la fonction, avec des assert correspondant à ces conditions.
3. Vérifier que `perimetre_rectangle(3, 5)` vaut 16, et que `perimetre_rectangle(-1, 5)` déclenche bien une erreur.

Exercice 39 ♦♦

Un élève teste sa fonction `est_premier_bugue` uniquement sur 7 (nombre premier) et 8 (non premier), et conclut qu'elle est correcte :

```
1 def est_premier_bugue(n):
2     for i in range(2, n):
3         if n % i == 0:
4             return False
5     return True
6
7 print(est_premier_bugue(7)) # True
8 print(est_premier_bugue(8)) # False
```

1. Que renvoie `est_premier_bugue(1)` ? Est-ce le résultat attendu (1 n'est pas un nombre premier) ?
2. Pourquoi le jeu de tests de l'élève (seulement 7 et 8) n'a-t-il pas permis de détecter ce bug ?
3. Corriger la fonction, et proposer un jeu de tests plus complet (au moins 4 cas, dont un cas limite) qui aurait détecté le bug.

Exercice 40 ♦♦♦

On souhaite calculer la moyenne et la médiane de la liste de notes `[10, 12, 14, 20]`, sans les recalculer « à la main ».

1. À l'aide de `help(...)` ou d'une recherche dans la documentation, trouver le nom du module de la bibliothèque standard Python qui contient des fonctions statistiques de base, et le nom de la fonction qui calcule une moyenne.
2. Écrire le code qui importe ce module et calcule la moyenne, puis la médiane, de la liste de notes.
3. Une postcondition raisonnable pour une fonction de moyenne de notes sur 20 serait « le résultat est compris entre 0 et 20 ». Vérifier que le résultat obtenu respecte cette postcondition.

Exercice 41 ♦

1. Écrire une fonction `somme_pairs(liste)` qui renvoie la somme des éléments **pairs** d'une liste d'entiers, en utilisant une boucle `for` et une conditionnelle.
2. Identifier, dans le code de cette fonction, une occurrence de chacune des constructions élémentaires suivantes : affectation, boucle bornée, conditionnelle.
3. Tester la fonction sur `[1, 2, 3, 4, 5, 6]` (résultat attendu : 12) et sur `[1, 3, 5]` (résultat attendu : 0).

Exercice 42 ♦

On donne le code suivant, écrit dans un langage de style C, qui calcule la factorielle d'un entier `n` :

```
int factorielle(int n) {
    int resultat = 1;
    for (int i = 1; i ≤ n; i = i + 1) {
        resultat = resultat * i;
    }
    return resultat;
}
```

1. Traduire ce code en Python (fonction factorielle(n)).
2. Identifier deux traits **communs** entre les deux écritures, et deux traits **particuliers** au langage de style C (absents en Python).
3. Vérifier que factorielle(5) vaut 120 et que factorielle(0) vaut 1.

QCM d'auto-évaluation — Langages et programmation

Pour chaque question, une seule réponse est exacte. Entourez la lettre correspondante.

C1 — Constructions élémentaires

1. [Q1] Une boucle while est dite « non bornée » car :
 - A. elle ne peut jamais se terminer.
 - B. le nombre d'itérations n'est pas forcément connu à l'avance.
 - C. elle ne contient jamais de condition.
 - D. elle est plus rapide qu'une boucle for.

C2 — Diversité et unité des langages

2. [Q2] Ce qui varie principalement d'un langage de programmation impératif à un autre, c'est :
 - A. le noyau de constructions élémentaires (qui est différent à chaque fois).
 - B. la syntaxe et des traits particuliers comme le typage.
 - C. le fait qu'un même algorithme ne puisse s'écrire que dans un seul langage.
 - D. rien : tous les langages sont strictement identiques.

C3 — Spécification

3. [Q3] Une précondition d'une fonction porte sur :
 - A. le résultat renvoyé par la fonction.
 - B. les arguments passés à la fonction.
 - C. le nom de la fonction.
 - D. le nombre de lignes de code de la fonction.

C4 — Mise au point

4. [Q4] Un jeu de tests entièrement réussi :
 - A. prouve que le programme est correct dans tous les cas.
 - B. montre seulement que le programme fonctionne sur les cas testés.
 - C. garantit l'absence de tout bug futur.
 - D. n'a aucun intérêt si le programme compile.

C5 — Bibliothèques

5. [Q5] Avant d'utiliser une fonction inconnue d'une bibliothèque, il est recommandé de :
 - A. deviner son comportement à partir de son nom.
 - B. consulter sa documentation (docstring, help(...)).
 - C. recopier son code source avant de l'utiliser.

D. l'essayer uniquement sur des cas limites.

6. [Q6] Le programme officiel de NSI Première, concernant les bibliothèques, exige :

- A. une connaissance exhaustive d'au moins une bibliothèque.
- B. de savoir chercher et lire une documentation pour trouver la bonne fonction.
- C. de ne jamais utiliser de bibliothèque externe.
- D. de mémoriser toutes les fonctions du module math.

Corrigé

1. Précondition : valeurs et poids ont la même longueur, et la somme des poids est strictement positive (pour ne pas diviser par zéro).

2.

```
1 def moyenne_ponderee(valeurs, poids):
2     assert len(valeurs) == len(poids)
3     assert sum(poids) > 0
4     total = 0
5     for i in range(len(valeurs)):
6         total = total + valeurs[i] * poids[i]
7     return total / sum(poids)
```

$$3. \frac{12 \times 1 + 15 \times 2 + 8 \times 1}{1 + 2 + 1} = \frac{12 + 30 + 8}{4} = \frac{50}{4} = 12,5.$$

Corrigé

1. somme_positifs_bugue([3, -2, 5]) renvoie 3. Le résultat attendu est 8 (3 + 5, les deux éléments strictement positifs).

2. L'erreur est dans range(len(liste) - 1) : cette boucle parcourt les indices de 0 à len(liste) - 2, donc **exclut le dernier élément** de la liste (indice len(liste) - 1). Version corrigée :

```
1 def somme_positifs(liste):
2     total = 0
3     for i in range(len(liste)):
4         if liste[i] > 0:
5             total += liste[i]
6     return total
```

3. import math puis math.gcd(84, 30), qui renvoie 6.

Évaluation A — Langages et programmation

Durée : 45 minutes. Ordinateur avec interpréteur Python autorisé.

Barème indicatif : Ex. 1 : 10 pts (constructions, spécification) ; Ex. 2 : 10 pts (mise au point, bibliothèque)

Exercice 1 — Moyenne pondérée

(10 points)

1. (2 pts) Proposer une précondition pour une fonction moyenne_ponderee(valeurs, poids) calculant une moyenne pondérée (on pourra penser à la longueur des deux listes, et à la somme des poids).
2. (5 pts) Écrire cette fonction, avec les assert correspondants, en utilisant une boucle for bornée.
3. (3 pts) Calculer moyenne_ponderee([12, 15, 8], [1, 2, 1]) à la main, puis vérifier avec le code.

Exercice 2 — Mise au point et bibliothèque

(10 points)

Un élève écrit la fonction suivante, censée renvoyer la somme des éléments strictement positifs d'une liste :

```
1 def somme_positifs_bugue(liste):
2     total = 0
3     for i in range(len(liste) - 1):
4         if liste[i] > 0:
5             total += liste[i]
6     return total
```

1. (3 pts) Que renvoie `somme_positifs_bugue([3, -2, 5])` ? Quel est le résultat attendu ?
2. (4 pts) Identifier précisément l'erreur dans le code, et proposer une version corrigée.
3. (3 pts) En utilisant le module `math`, calculer le PGCD de 84 et 30 (indiquer le nom de la fonction utilisée).

Corrigé

1. Précondition : valeurs et poids ont la même longueur, et la somme des poids est strictement positive.

2.

```
1 def moyenne_ponderee(valeurs, poids):
2     assert len(valeurs) == len(poids)
3     assert sum(poids) > 0
4     total = 0
5     for i in range(len(valeurs)):
6         total = total + valeurs[i] * poids[i]
7     return total / sum(poids)
```

$$3. \frac{8 \times 1 + 12 \times 1 + 16 \times 2}{1 + 1 + 2} = \frac{8 + 12 + 32}{4} = \frac{52}{4} = 13.$$

Corrigé

1. `produit_negatifs_bugue([-2, -3, -4])` renvoie 0. Le résultat attendu est -24 ($(-2) \times (-3) \times (-4)$).

2. L'accumulateur `prod` est initialisé à 0 : dès la première multiplication (`prod *= x`), $0 \times x = 0$ quel que soit x , donc `prod` reste 0 pour toujours. Il faut initialiser un accumulateur de **produit** à 1 (l'élément neutre de la multiplication), pas à 0 :

```
1 def produit_negatifs(liste):
2     prod = 1
3     for x in liste:
4         if x < 0:
5             prod *= x
6     return prod
```

3. `import math` puis `math.gcd(60, 48)`, qui renvoie 12.

Évaluation B — Langages et programmation

Durée : 45 minutes. Ordinateur avec interpréteur Python autorisé.

Barème indicatif : Ex. 1 : 10 pts (constructions, spécification) ; Ex. 2 : 10 pts (mise au point, bibliothèque)

Exercice 1 — Moyenne pondérée

(10 points)

1. (2 pts) Proposer une précondition pour une fonction `moyenne_ponderee(valeurs, poids)` calculant une moyenne pondérée.
2. (5 pts) Écrire cette fonction, avec les `assert` correspondants, en utilisant une boucle `for` bornée.
3. (3 pts) Calculer `moyenne_ponderee([8, 12, 16], [1, 1, 2])` à la main, puis vérifier avec le code.

Exercice 2 — Mise au point et bibliothèque

(10 points)

Un élève écrit la fonction suivante, censée renvoyer le produit des éléments strictement négatifs d'une liste :

```
1 def produit_negatifs_bugue(liste):
2     prod = 0
3     for x in liste:
4         if x < 0:
5             prod *= x
6     return prod
```

1. (3 pts) Que renvoie `produit_negatifs_bugue([-2, -3, -4])` ? Quel est le résultat attendu ?
2. (4 pts) Identifier précisément l'erreur dans le code, et proposer une version corrigée.
3. (3 pts) En utilisant le module `math`, calculer le PGCD de 60 et 48 (indiquer le nom de la fonction utilisée).

Exercice 43

Précondition : dans toute cette fiche, l'argument est une **liste non vide**. Le cas `[]` est hors précondition.

Un élève écrit une fonction censée renvoyer le **minimum** d'une liste de nombres :

```
1 def minimum_bugue(liste):
2     mini = 0
3     for x in liste:
4         if x < mini:
5             mini = x
6     return mini
```

1. Que renvoie `minimum_bugue([5, 3, 8])` ? Est-ce le résultat attendu ?
2. Pourquoi cette fonction se trompe-t-elle sur cet exemple, alors que tous les éléments de la liste sont positifs ?
3. Corriger la fonction en initialisant l'accumulateur correctement, puis justifier le choix de la valeur initiale à partir de la précondition.

Exercice 44

Précondition : dans toute cette fiche, l'argument est une **liste non vide**. Le cas `[]` est hors précondition.

Un élève écrit une fonction censée renvoyer le **minimum** d'une liste de nombres :

```
1 def minimum_bugue(liste):
2     mini = 0
3     for x in liste:
4         if x < mini:
5             mini = x
6     return mini
```

1. Que renvoie `minimum_bugue([5, 3, 8])` ? Est-ce le résultat attendu ?

2. Pourquoi cette fonction se trompe-t-elle sur cet exemple, alors que tous les éléments de la liste sont positifs ?
3. Corriger la fonction en initialisant l'accumulateur correctement, puis justifier le choix de la valeur initiale à partir de la précondition.

Exercice 45

Précondition : dans toute cette fiche, l'argument est une **liste non vide**. Le cas `[]` est hors précondition.

Un élève écrit une fonction censée renvoyer le **minimum** d'une liste de nombres :

```
1 def minimum_bugue(liste):
2     mini = 0
3     for x in liste:
4         if x < mini:
5             mini = x
6     return mini
```

1. Que renvoie `minimum_bugue([5, 3, 8])` ? Est-ce le résultat attendu ?
2. Pourquoi cette fonction se trompe-t-elle sur cet exemple, alors que tous les éléments de la liste sont positifs ?
3. Corriger la fonction en initialisant l'accumulateur correctement, puis justifier le choix de la valeur initiale à partir de la précondition.

Exercice 46

Précondition : dans toute cette fiche, l'argument est une **liste non vide**. Le cas `[]` est hors précondition.

Un élève écrit une fonction censée renvoyer le **minimum** d'une liste de nombres :

```
1 def minimum_bugue(liste):
2     mini = 0
3     for x in liste:
4         if x < mini:
5             mini = x
6     return mini
```

1. Que renvoie `minimum_bugue([5, 3, 8])` ? Est-ce le résultat attendu ?
2. Pourquoi cette fonction se trompe-t-elle sur cet exemple, alors que tous les éléments de la liste sont positifs ?
3. Corriger la fonction en initialisant l'accumulateur correctement, puis justifier le choix de la valeur initiale à partir de la précondition.

Exercice 47

Précondition : dans toute cette fiche, l'argument est une **liste non vide**. Le cas `[]` est hors précondition.

Un élève écrit une fonction censée renvoyer le **minimum** d'une liste de nombres :

```
1 def minimum_bugue(liste):
2     mini = 0
3     for x in liste:
4         if x < mini:
5             mini = x
6     return mini
```

1. Que renvoie `minimum_bugue([5, 3, 8])` ? Est-ce le résultat attendu ?
2. Pourquoi cette fonction se trompe-t-elle sur cet exemple, alors que tous les éléments de la liste sont positifs ?
3. Corriger la fonction en initialisant l'accumulateur correctement, puis justifier le choix de la valeur initiale à partir de la précondition.

Exercice 48

Précondition : dans toute cette fiche, l'argument est une **liste non vide**. Le cas `[]` est hors précondition.

Un élève écrit une fonction censée renvoyer le **minimum** d'une liste de nombres :

```
1 def minimum_bugue(liste):
2     mini = 0
3     for x in liste:
4         if x < mini:
5             mini = x
6     return mini
```

1. Que renvoie `minimum_bugue([5, 3, 8])` ? Est-ce le résultat attendu ?
2. Pourquoi cette fonction se trompe-t-elle sur cet exemple, alors que tous les éléments de la liste sont positifs ?
3. Corriger la fonction en initialisant l'accumulateur correctement, puis justifier le choix de la valeur initiale à partir de la précondition.

Exercice 49

Précondition : dans toute cette fiche, l'argument est une **liste non vide**. Le cas `[]` est hors précondition.

Un élève écrit une fonction censée renvoyer le **minimum** d'une liste de nombres :

```
1 def minimum_bugue(liste):
2     mini = 0
3     for x in liste:
4         if x < mini:
5             mini = x
6     return mini
```

1. Que renvoie `minimum_bugue([5, 3, 8])` ? Est-ce le résultat attendu ?
2. Pourquoi cette fonction se trompe-t-elle sur cet exemple, alors que tous les éléments de la liste sont positifs ?
3. Corriger la fonction en initialisant l'accumulateur correctement, puis justifier le choix de la valeur initiale à partir de la précondition.

Exercice 50

Précondition : dans toute cette fiche, l'argument est une **liste non vide**. Le cas `[]` est hors précondition.

Un élève écrit une fonction censée renvoyer le **minimum** d'une liste de nombres :

```
1 def minimum_bugue(liste):
2     mini = 0
3     for x in liste:
4         if x < mini:
```

```
5     mini = x
6     return mini
```

1. Que renvoie `minimum_bugue([5, 3, 8])` ? Est-ce le résultat attendu ?
2. Pourquoi cette fonction se trompe-t-elle sur cet exemple, alors que tous les éléments de la liste sont positifs ?
3. Corriger la fonction en initialisant l'accumulateur correctement, puis justifier le choix de la valeur initiale à partir de la précondition.

Corrigé

1.

```
1 def somme_paires(liste):
2     total = 0
3     for x in liste:
4         if x % 2 == 0:
5             total = total + x
6     return total
```

2. Affectation : `total = 0`. **Boucle bornée** : `for x in liste:` (le nombre d'itérations est connu, c'est la longueur de la liste). **Conditionnelle** : `if x % 2 == 0:`.

3. `somme_paires([1, 2, 3, 4, 5, 6])` vaut $2 + 4 + 6 = 12$. `somme_paires([1, 3, 5])` vaut 0 (aucun élément pair).

Corrigé

1.

```
1 def factorielle(n):
2     resultat = 1
3     for i in range(1, n + 1):
4         resultat = resultat * i
5     return resultat
```

2. Traits communs : une fonction nommée `factorielle` prenant un paramètre `n` et renvoyant un résultat ; une boucle bornée qui parcourt les entiers de 1 à n en accumulant un produit. **Traits particuliers** au langage de style C : le type de chaque variable est déclaré explicitement (`int`) ; les blocs d'instructions sont délimités par des accolades `{}` (et non par l'indentation comme en Python) ; chaque instruction se termine par un point-virgule.

3. $5! = 1 \times 2 \times 3 \times 4 \times 5 = 120$. Par convention, $0! = 1$ (produit vide, la boucle `for i in range(1, 1)` ne s'exécute jamais et `resultat` reste égal à 1).

Corrigé

1. Précondition : `largeur > 0` et `longueur > 0` (un rectangle ne peut pas avoir de côté négatif ou nul). Postcondition : le résultat renvoyé est strictement positif.

2.

```
1 def perimetre_rectangle(largeur, longueur):
2     assert largeur > 0
3     assert longueur > 0
4     p = 2 * (largeur + longueur)
5     assert p > 0
6     return p
```


3. $\text{perimetre_rectangle}(3, 5) = 2 \times (3 + 5) = 16$. L'appel avec $\text{largeur} = -1$ viole la précondition $\text{largeur} > 0$: l'instruction `assert` déclenche une `AssertionError`.

Corrigé

1. `est_premier_bugue(1)` renvoie `True` : la boucle `for i in range(2, 1)` : ne s'exécute jamais (l'intervalle est vide), donc la fonction renvoie directement `True`. Ce n'est **pas** le résultat attendu : 1 n'est pas un nombre premier (par définition, un nombre premier a exactement deux diviseurs distincts, 1 et lui-même ; 1 n'en a qu'un seul).

2. Le jeu de tests (7 et 8) ne teste que des valeurs « normales » (≥ 2), aucune valeur limite comme $n = 1$ ou $n = 0$. Le succès du test sur 7 et 8 ne prouve pas que la fonction est correcte pour **tous** les cas, notamment les cas limites non testés.

3.

```
1 def est_premier(n):
2     if n < 2:
3         return False
4     for i in range(2, n):
5         if n % i == 0:
6             return False
7     return True
8
9 assert est_premier(1) is False # cas limite qui revele le bug
10 assert est_premier(2) is True # plus petit nombre premier
11 assert est_premier(7) is True # cas "normal", premier
12 assert est_premier(8) is False # cas "normal", non premier
```

Corrigé

1. Le module de la bibliothèque standard est `statistics`. La fonction qui calcule une moyenne est `statistics.mean`.

2.

```
1 import statistics
2
3 notes = [10, 12, 14, 20]
4 moyenne = statistics.mean(notes) # 14
5 mediane = statistics.median(notes) # 13.0
```

3. La moyenne obtenue est 14, qui est bien comprise entre 0 et 20 : la postcondition est respectée.

Corrigé

1. `minimum_bugue([5, 3, 8])` renvoie 0, alors que le minimum réel de la liste est 3.

2. L'accumulateur `mini` est initialisé à 0, qui est **inférieur à tous les éléments** de la liste `[5, 3, 8]` (tous positifs et ≥ 3). Aucun élément ne vérifie donc $x < \text{mini}$, et `mini` reste égal à 0 jusqu'à la fin : ce 0, qui n'appartient même pas à la liste, est renvoyé à tort.

3. **Précondition : la liste est non vide.** Un test explicite refuse donc `[]` en déclenchant une `ValueError`.

```
1 def minimum(liste):
2     """Renvoie le plus petit element.
3
4     Precondition : liste est non vide.
5     """
6     if len(liste) == 0:
7         raise ValueError("liste doit etre non vide")
8     mini = liste[0]
```

```

9     for x in liste[1:]:
10         if x < mini:
11             mini = x
12     return mini

```

En initialisant mini avec le **premier élément** de la liste (et non une constante arbitraire), la fonction renvoie bien 3 pour [5, 3, 8].

Corrigé

1.

```

1 def somme_paires(liste):
2     total = 0
3     for x in liste:
4         if x % 2 == 0:
5             total = total + x
6     return total

```

2. Affectation : total = 0. **Boucle bornée** : for x in liste: (le nombre d'itérations est connu, c'est la longueur de la liste). **Conditionnelle** : if x % 2 == 0:.

3. somme_paires([1, 2, 3, 4, 5, 6]) vaut 2 + 4 + 6 = 12. somme_paires([1, 3, 5]) vaut 0 (aucun élément pair).

Corrigé

1.

```

1 def factorielle(n):
2     resultat = 1
3     for i in range(1, n + 1):
4         resultat = resultat * i
5     return resultat

```

2. Traits communs : une fonction nommée factorielle prenant un paramètre n et renvoyant un résultat ; une boucle bornée qui parcourt les entiers de 1 à n en accumulant un produit. **Traits particuliers** au langage de style C : le type de chaque variable est déclaré explicitement (int) ; les blocs d'instructions sont délimités par des accolades {} (et non par l'indentation comme en Python) ; chaque instruction se termine par un point-virgule.

3. $5! = 1 \times 2 \times 3 \times 4 \times 5 = 120$. Par convention, $0! = 1$ (produit vide, la boucle for i in range(1, 1) ne s'exécute jamais et resultat reste égal à 1).

Corrigé

1. Précondition : largeur > 0 et longueur > 0 (un rectangle ne peut pas avoir de côté négatif ou nul). Postcondition : le résultat renvoyé est strictement positif.

2.

```

1 def perimetre_rectangle(largeur, longueur):
2     assert largeur > 0
3     assert longueur > 0
4     p = 2 * (largeur + longueur)
5     assert p > 0
6     return p

```

3. perimetre_rectangle(3,5) = 2 × (3 + 5) = 16. L'appel avec largeur=-1 viole la précondition largeur > 0 : l'instruction assert déclenche une AssertionError.

Corrigé

1. `est_premier_bugue(1)` renvoie `True` : la boucle `for i in range(2, 1)` : ne s'exécute jamais (l'intervalle est vide), donc la fonction renvoie directement `True`. Ce n'est **pas** le résultat attendu : 1 n'est pas un nombre premier (par définition, un nombre premier a exactement deux diviseurs distincts, 1 et lui-même ; 1 n'en a qu'un seul).

2. Le jeu de tests (7 et 8) ne teste que des valeurs « normales » (≥ 2), aucune valeur limite comme $n = 1$ ou $n = 0$. Le succès du test sur 7 et 8 ne prouve pas que la fonction est correcte pour **tous** les cas, notamment les cas limites non testés.

3.

```
1 def est_premier(n):
2     if n < 2:
3         return False
4     for i in range(2, n):
5         if n % i == 0:
6             return False
7     return True
8
9 assert est_premier(1) is False # cas limite qui revele le bug
10 assert est_premier(2) is True # plus petit nombre premier
11 assert est_premier(7) is True # cas "normal", premier
12 assert est_premier(8) is False # cas "normal", non premier
```

Corrigé

1. Le module de la bibliothèque standard est `statistics`. La fonction qui calcule une moyenne est `statistics.mean`.

2.

```
1 import statistics
2
3 notes = [10, 12, 14, 20]
4 moyenne = statistics.mean(notes) # 14
5 mediane = statistics.median(notes) # 13.0
```

3. La moyenne obtenue est 14, qui est bien comprise entre 0 et 20 : la postcondition est respectée.

Corrigé

1. `minimum_bugue([5, 3, 8])` renvoie 0, alors que le minimum réel de la liste est 3.

2. L'accumulateur `mini` est initialisé à 0, qui est **inférieur à tous les éléments** de la liste `[5, 3, 8]` (tous positifs et ≥ 3). Aucun élément ne vérifie donc `x < mini`, et `mini` reste égal à 0 jusqu'à la fin : ce 0, qui n'appartient même pas à la liste, est renvoyé à tort.

3. **Précondition : la liste est non vide.** Un test explicite refuse donc `[]` en déclenchant une `ValueError`.

```
1 def minimum(liste):
2     """Renvoie le plus petit element.
3
4     Precondition : liste est non vide.
5     """
6     if len(liste) == 0:
7         raise ValueError("liste doit etre non vide")
8     mini = liste[0]
9     for x in liste[1:]:
10         if x < mini:
11             mini = x
12     return mini
```

En initialisant `mini` avec le **premier élément** de la liste (et non une constante arbitraire), la fonction renvoie bien 3 pour `[5, 3, 8]`.

Corrigé

1.

```
1 def somme_paires(liste):
2     total = 0
3     for x in liste:
4         if x % 2 == 0:
5             total = total + x
6     return total
```

2. Affectation : `total = 0`. **Boucle bornée** : `for x in liste`: (le nombre d'itérations est connu, c'est la longueur de la liste). **Conditionnelle** : `if x % 2 == 0`..

3. `somme_paires([1, 2, 3, 4, 5, 6])` vaut $2 + 4 + 6 = 12$. `somme_paires([1, 3, 5])` vaut 0 (aucun élément pair).

Corrigé

1.

```
1 def factorielle(n):
2     resultat = 1
3     for i in range(1, n + 1):
4         resultat = resultat * i
5     return resultat
```

2. Traits communs : une fonction nommée `factorielle` prenant un paramètre `n` et renvoyant un résultat ; une boucle bornée qui parcourt les entiers de 1 à n en accumulant un produit. **Traits particuliers** au langage de style C : le type de chaque variable est déclaré explicitement (`int`) ; les blocs d'instructions sont délimités par des accolades `{}` (et non par l'indentation comme en Python) ; chaque instruction se termine par un point-virgule.

3. $5! = 1 \times 2 \times 3 \times 4 \times 5 = 120$. Par convention, $0! = 1$ (produit vide, la boucle `for i in range(1, 1)` ne s'exécute jamais et `resultat` reste égal à 1).

Corrigé

1. Précondition : `largeur > 0` et `longueur > 0` (un rectangle ne peut pas avoir de côté négatif ou nul). Postcondition : le résultat renvoyé est strictement positif.

2.

```
1 def perimetre_rectangle(largeur, longueur):
2     assert largeur > 0
3     assert longueur > 0
4     p = 2 * (largeur + longueur)
5     assert p > 0
6     return p
```

3. `perimetre_rectangle(3,5)` = $2 \times (3 + 5) = 16$. L'appel avec `largeur=-1` viole la précondition `largeur > 0` : l'instruction `assert` déclenche une `AssertionError`.

Corrigé

1. `est_premier_bugue(1)` renvoie `True` : la boucle `for i in range(2, 1)`: ne s'exécute jamais (l'intervalle est vide), donc la fonction renvoie directement `True`. Ce n'est **pas** le résultat attendu : 1 n'est pas

un nombre premier (par définition, un nombre premier a exactement deux diviseurs distincts, 1 et lui-même ; 1 n'en a qu'un seul).

2. Le jeu de tests (7 et 8) ne teste que des valeurs « normales » (≥ 2), aucune valeur limite comme $n = 1$ ou $n = 0$. Le succès du test sur 7 et 8 ne prouve pas que la fonction est correcte pour **tous** les cas, notamment les cas limites non testés.

3.

```
1 def est_premier(n):
2     if n < 2:
3         return False
4     for i in range(2, n):
5         if n % i == 0:
6             return False
7     return True
8
9 assert est_premier(1) is False # cas limite qui revele le bug
10 assert est_premier(2) is True # plus petit nombre premier
11 assert est_premier(7) is True # cas "normal", premier
12 assert est_premier(8) is False # cas "normal", non premier
```

Corrigé

1. Le module de la bibliothèque standard est statistics. La fonction qui calcule une moyenne est statistics.mean.

2.

```
1 import statistics
2
3 notes = [10, 12, 14, 20]
4 moyenne = statistics.mean(notes) # 14
5 mediane = statistics.median(notes) # 13.0
```

3. La moyenne obtenue est 14, qui est bien comprise entre 0 et 20 : la postcondition est respectée.

Corrigé

1. minimum_bug([5, 3, 8]) renvoie 0, alors que le minimum réel de la liste est 3.

2. L'accumulateur mini est initialisé à 0, qui est **inférieur à tous les éléments** de la liste [5, 3, 8] (tous positifs et ≥ 3). Aucun élément ne vérifie donc $x < \text{mini}$, et mini reste égal à 0 jusqu'à la fin : ce 0, qui n'appartient même pas à la liste, est renvoyé à tort.

3. **Précondition : la liste est non vide.** Un test explicite refuse donc [] en déclenchant une ValueError.

```
1 def minimum(liste):
2     """Renvoie le plus petit element.
3
4     Precondition : liste est non vide.
5     """
6     if len(liste) == 0:
7         raise ValueError("liste doit etre non vide")
8     mini = liste[0]
9     for x in liste[1:]:
10         if x < mini:
11             mini = x
12     return mini
```

En initialisant mini avec le **premier élément** de la liste (et non une constante arbitraire), la fonction renvoie bien 3 pour [5, 3, 8].

Corrigé

1.

```
1 def somme_paires(liste):
2     total = 0
3     for x in liste:
4         if x % 2 == 0:
5             total = total + x
6     return total
```

2. **Affectation** : $total = 0$. **Boucle bornée** : `for x in liste`: (le nombre d'itérations est connu, c'est la longueur de la liste). **Conditionnelle** : `if x % 2 == 0`..

3. `somme_paires([1, 2, 3, 4, 5, 6])` vaut $2 + 4 + 6 = 12$. `somme_paires([1, 3, 5])` vaut 0 (aucun élément pair).

Corrigé

1.

```
1 def factorielle(n):
2     resultat = 1
3     for i in range(1, n + 1):
4         resultat = resultat * i
5     return resultat
```

2. **Traits communs** : une fonction nommée `factorielle` prenant un paramètre n et renvoyant un résultat ; une boucle bornée qui parcourt les entiers de 1 à n en accumulant un produit. **Traits particuliers** au langage de style C : le type de chaque variable est déclaré explicitement (`int`) ; les blocs d'instructions sont délimités par des accolades `{}` (et non par l'indentation comme en Python) ; chaque instruction se termine par un point-virgule.

3. $5! = 1 \times 2 \times 3 \times 4 \times 5 = 120$. Par convention, $0! = 1$ (produit vide, la boucle `for i in range(1, 1)` ne s'exécute jamais et `resultat` reste égal à 1).

Corrigé

1. Précondition : $largeur > 0$ et $longueur > 0$ (un rectangle ne peut pas avoir de côté négatif ou nul). Postcondition : le résultat renvoyé est strictement positif.

2.

```
1 def perimetre_rectangle(largeur, longueur):
2     assert largeur > 0
3     assert longueur > 0
4     p = 2 * (largeur + longueur)
5     assert p > 0
6     return p
```

3. `perimetre_rectangle(3, 5) = 2 \times (3 + 5) = 16`. L'appel avec `largeur=-1` viole la précondition $largeur > 0$: l'instruction `assert` déclenche une `AssertionError`.

Corrigé

1. `est_premier_bugue(1)` renvoie `True` : la boucle `for i in range(2, 1)` : ne s'exécute jamais (l'intervalle est vide), donc la fonction renvoie directement `True`. Ce n'est **pas** le résultat attendu : 1 n'est pas un nombre premier (par définition, un nombre premier a exactement deux diviseurs distincts, 1 et lui-même ; 1 n'en a qu'un seul).

2. Le jeu de tests (7 et 8) ne teste que des valeurs « normales » (≥ 2), aucune valeur limite comme $n = 1$ ou $n = 0$. Le succès du test sur 7 et 8 ne prouve pas que la fonction est correcte pour **tous** les cas, notamment les cas limites non testés.

3.

```

1 def est_premier(n):
2     if n < 2:
3         return False
4     for i in range(2, n):
5         if n % i == 0:
6             return False
7     return True
8
9 assert est_premier(1) is False # cas limite qui revele le bug
10 assert est_premier(2) is True # plus petit nombre premier
11 assert est_premier(7) is True # cas "normal", premier
12 assert est_premier(8) is False # cas "normal", non premier

```

Corrigé

1. Le module de la bibliothèque standard est statistics. La fonction qui calcule une moyenne est statistics.mean.

2.

```

1 import statistics
2
3 notes = [10, 12, 14, 20]
4 moyenne = statistics.mean(notes) # 14
5 mediane = statistics.median(notes) # 13.0

```

3. La moyenne obtenue est 14, qui est bien comprise entre 0 et 20 : la postcondition est respectée.

Corrigé

1. minimum_bugue([5, 3, 8]) renvoie 0, alors que le minimum réel de la liste est 3.

2. L'accumulateur mini est initialisé à 0, qui est **inférieur à tous les éléments** de la liste [5, 3, 8] (tous positifs et ≥ 3). Aucun élément ne vérifie donc $x < \text{mini}$, et mini reste égal à 0 jusqu'à la fin : ce 0, qui n'appartient même pas à la liste, est renvoyé à tort.

3. **Précondition : la liste est non vide.** Un test explicite refuse donc [] en déclenchant une ValueError.

```

1 def minimum(liste):
2     """Renvoie le plus petit element.
3
4     Precondition : liste est non vide.
5     """
6     if len(liste) == 0:
7         raise ValueError("liste doit etre non vide")
8     mini = liste[0]
9     for x in liste[1:]:
10         if x < mini:
11             mini = x
12     return mini

```

En initialisant mini avec le **premier élément** de la liste (et non une constante arbitraire), la fonction renvoie bien 3 pour [5, 3, 8].

Corrigé

1.

```

1 def somme_paires(liste):

```

```

2     total = 0
3     for x in liste:
4         if x % 2 == 0:
5             total = total + x
6     return total

```

2. **Affectation** : `total = 0`. **Boucle bornée** : `for x in liste:` (le nombre d'itérations est connu, c'est la longueur de la liste). **Conditionnelle** : `if x % 2 == 0:`.

3. `somme_paires([1, 2, 3, 4, 5, 6])` vaut $2 + 4 + 6 = 12$. `somme_paires([1, 3, 5])` vaut 0 (aucun élément pair).

Corrigé

1.

```

1 def factorielle(n):
2     resultat = 1
3     for i in range(1, n + 1):
4         resultat = resultat * i
5     return resultat

```

2. **Traits communs** : une fonction nommée `factorielle` prenant un paramètre `n` et renvoyant un résultat ; une boucle bornée qui parcourt les entiers de 1 à n en accumulant un produit. **Traits particuliers** au langage de style C : le type de chaque variable est déclaré explicitement (`int`) ; les blocs d'instructions sont délimités par des accolades `{}` (et non par l'indentation comme en Python) ; chaque instruction se termine par un point-virgule.

3. $5! = 1 \times 2 \times 3 \times 4 \times 5 = 120$. Par convention, $0! = 1$ (produit vide, la boucle `for i in range(1, 1)` ne s'exécute jamais et `resultat` reste égal à 1).

Corrigé

1. Précondition : `largeur > 0` et `longueur > 0` (un rectangle ne peut pas avoir de côté négatif ou nul). Postcondition : le résultat renvoyé est strictement positif.

2.

```

1 def perimetre_rectangle(largeur, longueur):
2     assert largeur > 0
3     assert longueur > 0
4     p = 2 * (largeur + longueur)
5     assert p > 0
6     return p

```

3. `perimetre_rectangle(3,5) = 2 × (3 + 5) = 16`. L'appel avec `largeur=-1` viole la précondition `largeur > 0` : l'instruction `assert` déclenche une `AssertionError`.

Corrigé

1. `est_premier_bugue(1)` renvoie `True` : la boucle `for i in range(2, 1):` ne s'exécute jamais (l'intervalle est vide), donc la fonction renvoie directement `True`. Ce n'est **pas** le résultat attendu : 1 n'est pas un nombre premier (par définition, un nombre premier a exactement deux diviseurs distincts, 1 et lui-même ; 1 n'en a qu'un seul).

2. Le jeu de tests (7 et 8) ne teste que des valeurs « normales » (≥ 2), aucune valeur limite comme $n = 1$ ou $n = 0$. Le succès du test sur 7 et 8 ne prouve pas que la fonction est correcte pour **tous** les cas, notamment les cas limites non testés.

3.

```

1 def est_premier(n):

```



```

2     if n < 2:
3         return False
4     for i in range(2, n):
5         if n % i == 0:
6             return False
7     return True
8
9 assert est_premier(1) is False    # cas limite qui revele le bug
10 assert est_premier(2) is True    # plus petit nombre premier
11 assert est_premier(7) is True    # cas "normal", premier
12 assert est_premier(8) is False    # cas "normal", non premier

```

Corrigé

1. Le module de la bibliothèque standard est `statistics`. La fonction qui calcule une moyenne est `statistics.mean`.

2.

```

1 import statistics
2
3 notes = [10, 12, 14, 20]
4 moyenne = statistics.mean(notes)    # 14
5 mediane = statistics.median(notes)  # 13.0

```

3. La moyenne obtenue est 14, qui est bien comprise entre 0 et 20 : la postcondition est respectée.

Corrigé

1. `minimum_bugue([5, 3, 8])` renvoie 0, alors que le minimum réel de la liste est 3.

2. L'accumulateur `mini` est initialisé à 0, qui est **inférieur à tous les éléments** de la liste `[5, 3, 8]` (tous positifs et ≥ 3). Aucun élément ne vérifie donc $x < \text{mini}$, et `mini` reste égal à 0 jusqu'à la fin : ce 0, qui n'appartient même pas à la liste, est renvoyé à tort.

3. **Précondition : la liste est non vide.** Un test explicite refuse donc `[]` en déclenchant une `ValueError`.

```

1 def minimum(liste):
2     """Renvoie le plus petit element.
3
4     Precondition : liste est non vide.
5     """
6     if len(liste) == 0:
7         raise ValueError("liste doit etre non vide")
8     mini = liste[0]
9     for x in liste[1:]:
10         if x < mini:
11             mini = x
12     return mini

```

En initialisant `mini` avec le **premier élément** de la liste (et non une constante arbitraire), la fonction renvoie bien 3 pour `[5, 3, 8]`.

Corrigé

1.

```

1 def somme_paires(liste):
2     total = 0
3     for x in liste:
4         if x % 2 == 0:

```

```
5         total = total + x
6     return total
```

2. Affectation : `total = 0`. **Boucle bornée** : `for x in liste`: (le nombre d'itérations est connu, c'est la longueur de la liste). **Conditionnelle** : `if x % 2 == 0`:

3. `somme_paires([1, 2, 3, 4, 5, 6])` vaut $2 + 4 + 6 = 12$. `somme_paires([1, 3, 5])` vaut 0 (aucun élément pair).

Corrigé

1.

```
1 def factorielle(n):
2     resultat = 1
3     for i in range(1, n + 1):
4         resultat = resultat * i
5     return resultat
```

2. Traits communs : une fonction nommée `factorielle` prenant un paramètre `n` et renvoyant un résultat ; une boucle bornée qui parcourt les entiers de 1 à n en accumulant un produit. **Traits particuliers** au langage de style C : le type de chaque variable est déclaré explicitement (`int`) ; les blocs d'instructions sont délimités par des accolades `{}` (et non par l'indentation comme en Python) ; chaque instruction se termine par un point-virgule.

3. $5! = 1 \times 2 \times 3 \times 4 \times 5 = 120$. Par convention, $0! = 1$ (produit vide, la boucle `for i in range(1, 1)` ne s'exécute jamais et `resultat` reste égal à 1).

Corrigé

1. Précondition : `largeur > 0` et `longueur > 0` (un rectangle ne peut pas avoir de côté négatif ou nul). Postcondition : le résultat renvoyé est strictement positif.

2.

```
1 def perimetre_rectangle(largeur, longueur):
2     assert largeur > 0
3     assert longueur > 0
4     p = 2 * (largeur + longueur)
5     assert p > 0
6     return p
```

3. `perimetre_rectangle(3,5) = 2 × (3 + 5) = 16`. L'appel avec `largeur=-1` viole la précondition `largeur > 0` : l'instruction `assert` déclenche une `AssertionError`.

Corrigé

1. `est_premier_bugue(1)` renvoie `True` : la boucle `for i in range(2, 1)`: ne s'exécute jamais (l'intervalle est vide), donc la fonction renvoie directement `True`. Ce n'est **pas** le résultat attendu : 1 n'est pas un nombre premier (par définition, un nombre premier a exactement deux diviseurs distincts, 1 et lui-même ; 1 n'en a qu'un seul).

2. Le jeu de tests (7 et 8) ne teste que des valeurs « normales » (≥ 2), aucune valeur limite comme $n = 1$ ou $n = 0$. Le succès du test sur 7 et 8 ne prouve pas que la fonction est correcte pour **tous** les cas, notamment les cas limites non testés.

3.

```
1 def est_premier(n):
2     if n < 2:
3         return False
4     for i in range(2, n):
```

```

5     if n % i == 0:
6         return False
7     return True
8
9 assert est_premier(1) is False  # cas limite qui revele le bug
10 assert est_premier(2) is True  # plus petit nombre premier
11 assert est_premier(7) is True  # cas "normal", premier
12 assert est_premier(8) is False  # cas "normal", non premier

```

Corrigé

1. Le module de la bibliothèque standard est `statistics`. La fonction qui calcule une moyenne est `statistics.mean`.

2.

```

1 import statistics
2
3 notes = [10, 12, 14, 20]
4 moyenne = statistics.mean(notes)  # 14
5 mediane = statistics.median(notes)  # 13.0

```

3. La moyenne obtenue est 14, qui est bien comprise entre 0 et 20 : la postcondition est respectée.

Corrigé

1. `minimum_bugue([5, 3, 8])` renvoie 0, alors que le minimum réel de la liste est 3.

2. L'accumulateur `mini` est initialisé à 0, qui est **inférieur à tous les éléments** de la liste `[5, 3, 8]` (tous positifs et ≥ 3). Aucun élément ne vérifie donc $x < \text{mini}$, et `mini` reste égal à 0 jusqu'à la fin : ce 0, qui n'appartient même pas à la liste, est renvoyé à tort.

3. **Précondition : la liste est non vide.** Un test explicite refuse donc `[]` en déclenchant une `ValueError`.

```

1 def minimum(liste):
2     """Renvoie le plus petit element.
3
4     Precondition : liste est non vide.
5     """
6     if len(liste) == 0:
7         raise ValueError("liste doit etre non vide")
8     mini = liste[0]
9     for x in liste[1:]:
10         if x < mini:
11             mini = x
12     return mini

```

En initialisant `mini` avec le **premier élément** de la liste (et non une constante arbitraire), la fonction renvoie bien 3 pour `[5, 3, 8]`.

Corrigé

1.

```

1 def somme_paires(liste):
2     total = 0
3     for x in liste:
4         if x % 2 == 0:
5             total = total + x
6     return total

```

2. Affectation : $\text{total} = 0$. **Boucle bornée** : `for x in liste`: (le nombre d'itérations est connu, c'est la longueur de la liste). **Conditionnelle** : `if x % 2 == 0`.

3. `somme_pairs([1, 2, 3, 4, 5, 6])` vaut $2 + 4 + 6 = 12$. `somme_pairs([1, 3, 5])` vaut 0 (aucun élément pair).

Corrigé

1.

```
1 def factorielle(n):
2     resultat = 1
3     for i in range(1, n + 1):
4         resultat = resultat * i
5     return resultat
```

2. Traits communs : une fonction nommée `factorielle` prenant un paramètre `n` et renvoyant un résultat ; une boucle bornée qui parcourt les entiers de 1 à n en accumulant un produit. **Traits particuliers** au langage de style C : le type de chaque variable est déclaré explicitement (`int`) ; les blocs d'instructions sont délimités par des accolades `{}` (et non par l'indentation comme en Python) ; chaque instruction se termine par un point-virgule.

3. $5! = 1 \times 2 \times 3 \times 4 \times 5 = 120$. Par convention, $0! = 1$ (produit vide, la boucle `for i in range(1, 1)` ne s'exécute jamais et `resultat` reste égal à 1).

Corrigé

1. Précondition : `largeur > 0` et `longueur > 0` (un rectangle ne peut pas avoir de côté négatif ou nul). Postcondition : le résultat renvoyé est strictement positif.

2.

```
1 def perimetre_rectangle(largeur, longueur):
2     assert largeur > 0
3     assert longueur > 0
4     p = 2 * (largeur + longueur)
5     assert p > 0
6     return p
```

3. `perimetre_rectangle(3,5) = 2 \times (3 + 5) = 16`. L'appel avec `largeur=-1` viole la précondition `largeur > 0` : l'instruction `assert` déclenche une `AssertionError`.

Corrigé

1. `est_premier_bugue(1)` renvoie `True` : la boucle `for i in range(2, 1)`: ne s'exécute jamais (l'intervalle est vide), donc la fonction renvoie directement `True`. Ce n'est **pas** le résultat attendu : 1 n'est pas un nombre premier (par définition, un nombre premier a exactement deux diviseurs distincts, 1 et lui-même ; 1 n'en a qu'un seul).

2. Le jeu de tests (7 et 8) ne teste que des valeurs « normales » (≥ 2), aucune valeur limite comme $n = 1$ ou $n = 0$. Le succès du test sur 7 et 8 ne prouve pas que la fonction est correcte pour **tous** les cas, notamment les cas limites non testés.

3.

```
1 def est_premier(n):
2     if n < 2:
3         return False
4     for i in range(2, n):
5         if n % i == 0:
6             return False
7     return True
```

```

8
9 assert est_premier(1) is False # cas limite qui revele le bug
10 assert est_premier(2) is True # plus petit nombre premier
11 assert est_premier(7) is True # cas "normal", premier
12 assert est_premier(8) is False # cas "normal", non premier

```

Corrigé

1. Le module de la bibliothèque standard est statistics. La fonction qui calcule une moyenne est statistics.mean.

2.

```

1 import statistics
2
3 notes = [10, 12, 14, 20]
4 moyenne = statistics.mean(notes) # 14
5 mediane = statistics.median(notes) # 13.0

```

3. La moyenne obtenue est 14, qui est bien comprise entre 0 et 20 : la postcondition est respectée.

Corrigé

1. minimum_bugue([5, 3, 8]) renvoie 0, alors que le minimum réel de la liste est 3.

2. L'accumulateur mini est initialisé à 0, qui est **inférieur à tous les éléments** de la liste [5, 3, 8] (tous positifs et ≥ 3). Aucun élément ne vérifie donc $x < \text{mini}$, et mini reste égal à 0 jusqu'à la fin : ce 0, qui n'appartient même pas à la liste, est renvoyé à tort.

3. **Précondition : la liste est non vide.** Un test explicite refuse donc [] en déclenchant une ValueError.

```

1 def minimum(liste):
2     """Renvoie le plus petit element.
3
4     Precondition : liste est non vide.
5     """
6     if len(liste) == 0:
7         raise ValueError("liste doit etre non vide")
8     mini = liste[0]
9     for x in liste[1:]:
10         if x < mini:
11             mini = x
12     return mini

```

En initialisant mini avec le **premier élément** de la liste (et non une constante arbitraire), la fonction renvoie bien 3 pour [5, 3, 8].

Corrigé

1. minimum_bugue([5, 3, 8]) renvoie 0, alors que le minimum réel de la liste est 3.

2. L'accumulateur mini est initialisé à 0, qui est **inférieur à tous les éléments** de la liste [5, 3, 8] (tous positifs et ≥ 3). Aucun élément ne vérifie donc $x < \text{mini}$, et mini reste égal à 0 jusqu'à la fin : ce 0, qui n'appartient même pas à la liste, est renvoyé à tort.

3. **Précondition : la liste est non vide.** Un test explicite refuse donc [] en déclenchant une ValueError.

```

1 def minimum(liste):
2     """Renvoie le plus petit element.
3
4     Precondition : liste est non vide.
5     """

```

```
6     if len(liste) == 0:
7         raise ValueError("liste doit etre non vide")
8     mini = liste[0]
9     for x in liste[1:]:
10         if x < mini:
11             mini = x
12     return mini
```

En initialisant mini avec le **premier élément** de la liste (et non une constante arbitraire), la fonction renvoie bien 3 pour [5, 3, 8].

5 Algorithmique 1 : parcours et tris

5.1 Recherche dichotomique

DÉFINITION D1

La **recherche dichotomique** recherche une valeur dans un tableau **trié** en comparant systématiquement la valeur cherchée à l'élément **du milieu** de la zone de recherche restante, et en ne conservant que la moitié où la valeur peut encore se trouver.

CODE DE RÉFÉRENCE — Recherche dichotomique

```
1 def recherche_dichotomique(tableau_trie, valeur):
2     """Renvoie l'indice de 'valeur' dans 'tableau_trie' (trie), ou -1 si absente.
3     """
4     borne_inf, borne_sup = 0, len(tableau_trie) - 1
5     while borne_inf <= borne_sup:
6         milieu = (borne_inf + borne_sup) // 2
7         if tableau_trie[milieu] == valeur:
8             return milieu
9         elif tableau_trie[milieu] < valeur:
10            borne_inf = milieu + 1
11        else:
12            borne_sup = milieu - 1
13    return -1
14
15 t = [1, 3, 5, 7, 9, 11, 13, 15]
16 print(recherche_dichotomique(t, 11)) # 5
17 print(recherche_dichotomique(t, 4))  # -1
```

◆ **PROPRIÉTÉ Coût de la dichotomie** À chaque itération, la zone de recherche est **divisée par deux**. Le coût de la recherche dichotomique est donc **logarithmique** ($O(\log_2 n)$), bien plus rapide que le coût linéaire de la recherche séquentielle pour un grand tableau.

DÉMONSTRATION

Terminaison de la recherche dichotomique.

On considère le **variant de boucle** $V = \text{borne_sup} - \text{borne_inf}$.

V est **bien défini et positif ou nul** tant que la boucle continue, puisque la condition de la boucle while est précisément $\text{borne_inf} \leq \text{borne_sup}$.

V **décroît strictement** à chaque itération : si $\text{tableau_trie}[\text{milieu}] < \text{valeur}$, alors borne_inf devient $\text{milieu} + 1$, strictement supérieur à son ancienne valeur (puisque $\text{milieu} \geq \text{borne_inf}$), donc V diminue. Si $\text{tableau_trie}[\text{milieu}] > \text{valeur}$, alors borne_sup devient $\text{milieu} - 1$, strictement inférieur à son ancienne valeur (puisque $\text{milieu} \leq \text{borne_sup}$), donc V diminue également.

Conclusion. V est un entier positif ou nul qui décroît strictement à chaque tour de boucle : une suite d'entiers positifs strictement décroissante est nécessairement **finie**. La boucle while termine donc en un nombre fini d'itérations. ■

⚠ ERREUR FRÉQUENTE

Appliquer la dichotomie sur un tableau **non trié**. La dichotomie repose entièrement sur le tri : comparer à l'élément du milieu ne permet d'éliminer une moitié **que si** on est certain que tous les éléments de cette moitié sont, eux aussi, inférieurs (ou supérieurs) à la valeur cherchée — ce qui n'est garanti que si le tableau est trié.

» **Python À la machine.** Modifie `recherche_dichotomique` pour qu'elle compte le nombre d'itérations effectuées, et compare ce nombre à $\log_2(n)$ pour un tableau trié de 1000 éléments (utilise `math.log2`).

5.2 Algorithmes gloutons

DÉFINITION D2

Un **algorithme glouton** construit une solution étape par étape, en faisant à chaque étape le choix qui semble **le meilleur sur le moment** (le choix « localement optimal »), sans jamais revenir en arrière.

EXEMPLE Le **rendu de monnaie** : pour tenter d'obtenir un rendu avec peu de pièces, on choisit à chaque étape la plus grande pièce (ou billet) ne dépassant pas le montant restant. Les valeurs des pièces doivent être strictement positives. Si la stratégie gloutonne ne trouve pas de rendu exact, la fonction le signale au lieu de renvoyer un rendu partiel ; cela ne prouve pas qu'aucun autre rendu exact n'existe.

CODE DE RÉFÉRENCE — Rendu de monnaie glouton

```
1 def rendu_glouton(montant, pieces):
2     """Construit un rendu en choisissant d'abord les plus grandes pieces
3     disponibles.
4
5     Precondition : les valeurs des pieces sont strictement positives.
6     """
7     if not all(p > 0 for p in pieces):
8         raise ValueError("les pieces doivent etre strictement positives")
9     pieces_triees = sorted(pieces, reverse=True)
10    rendu = []
11    for p in pieces_triees:
12        while montant >= p:
13            rendu.append(p)
14            montant -= p
15    if montant != 0:
16        raise ValueError("l'algorithme glouton n'a pas trouve de rendu exact")
17    return rendu
18
19 print(rendu_glouton(78, [50, 20, 10, 5, 2, 1]))
```

◆ **PROPRIÉTÉ** Un **algorithme glouton n'est pas toujours optimal** Un algorithme glouton donne une solution **rapide à calculer**, mais qui n'est **pas garantie optimale** en général : le choix localement

optimal à chaque étape peut conduire à une solution globalement moins bonne qu'une autre approche.

⚠ **CONTRE-EXEMPLE** Avec le système de pièces {4, 3, 1} (non standard), pour rendre 6 : la méthode gloutonne choisit d'abord 4 (la plus grande pièce ≤ 6), puis doit compléter les 2 restants avec deux pièces de 1 : elle utilise 3 pièces (4 + 1 + 1). Or une solution avec 2 pièces existe : 3 + 3 = 6. L'algorithme glouton n'est donc, dans ce cas, **pas optimal**.

◆ **PROPRIÉTÉ** Quand l'approche gloutonne fonctionne-t-elle ? Pour le système de pièces **en euros** {1, 2, 5, 10, 20, 50, 100, 200} (centimes ou euros), la méthode gloutonne donne **toujours** le nombre minimal de pièces : ce système a une propriété mathématique particulière (chaque pièce est « assez grande » par rapport aux suivantes) qui garantit l'optimalité du choix glouton. Ce n'est pas le cas pour un système de pièces quelconque.

⚠ ERREUR FRÉQUENTE

Croire qu'un algorithme glouton donne toujours la meilleure solution possible. Un algorithme glouton est une **heuristique** (une méthode rapide et souvent efficace en pratique), pas une garantie d'optimalité : il faut vérifier, au cas par cas, si les propriétés du problème garantissent l'optimalité de l'approche gloutonne.

» **Python À la machine.** Teste `rendu_glouton` avec le système de pièces {1, 5, 6, 9} pour rendre 11. Combien de pièces la méthode gloutonne utilise-t-elle ? Trouve, en cherchant à la main, une solution utilisant moins de pièces.

5.3 L'algorithme des k plus proches voisins

DÉFINITION D3

L'algorithme des **k plus proches voisins** prédit la classe d'un nouvel élément en examinant les k éléments **déjà classés** les plus proches de lui (au sens d'une distance, par exemple la distance euclidienne), et en lui attribuant la classe **majoritaire** parmi ces k voisins. Le paramètre k doit être entier et vérifier $1 \leq k \leq n$, où n est le nombre d'éléments déjà classés.

◆ **PROPRIÉTÉ** Distance euclidienne entre deux points Pour deux points (x_1, y_1) et (x_2, y_2) du plan, la distance euclidienne est $d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$.

CODE DE RÉFÉRENCE — Algorithme des k plus proches voisins

```
1 def distance(p1, p2):
2     """Distance euclidienne entre deux points (x, y)."""
3     return ((p1[0] - p2[0]) ** 2 + (p1[1] - p2[1]) ** 2) ** 0.5
4
5
6 def k_plus_proches_voisins(points_classes, nouveau_point, k):
7     """Prédit la classe de 'nouveau_point' à partir de ses k plus proches voisins.
```

```

8
9     'points_classes' : liste de triplets (x, y, classe) deja etiquetes.
10    Precondition : k est un entier et 1 ≤ k ≤ len(points_classes).
11    """
12    if type(k) is not int or not 1 ≤ k ≤ len(points_classes):
13        raise ValueError(
14            "k doit etre un entier compris entre 1 et le nombre de points"
15        )
16    distances = [
17        (distance((x, y), nouveau_point), classe)
18        for (x, y, classe) in points_classes
19    ]
20    distances.sort(key=lambda d: d[0])
21    k_plus_proches = distances[:k]
22    classes = [c for (_, c) in k_plus_proches]
23    return max(set(classes), key=classes.count)
24
25
26 points = [
27     (1, 1, "A"), (2, 1, "A"), (1, 2, "A"),
28     (8, 8, "B"), (9, 8, "B"), (8, 9, "B"),
29 ]
30 print(k_plus_proches_voisins(points, (1.5, 1.5), 3))
31 print(k_plus_proches_voisins(points, (8.5, 8.5), 3))

```

◆ **PROPRIÉTÉ Choix de k** Le choix du nombre k de voisins influence le résultat : un k **trop petit** rend la prédiction sensible au bruit (un seul voisin atypique peut fausser le résultat) ; un k **trop grand** risque d'inclure des voisins trop éloignés, peu pertinents pour la classification. On choisit souvent k impair (pour éviter les égalités entre deux classes) et adapté à la taille du jeu de données.

⚠ ERREUR FRÉQUENTE

Oublier de **trier** les distances avant de sélectionner les k plus proches voisins. Sans tri (par exemple en prenant les k **premiers** points de la liste `points_classes`, dans l'ordre où ils sont donnés), on ne sélectionne pas du tout les voisins les plus proches, mais des points arbitraires.

» **Python À la machine.** Ajoute un point mal classé aux données de l'exemple, par exemple (1, 1, "B") au milieu du groupe "A", puis reclasse le point (1.5, 1.5) avec $k = 3$ puis $k = 5$. Le résultat change-t-il ? Explique en quoi cela illustre l'intérêt d'un k pas trop petit face au bruit dans les données.

5.4 Recherche dichotomique

DÉFINITION D1

La **recherche dichotomique** recherche une valeur dans un tableau **trié** en comparant systématiquement la valeur cherchée à l'élément **du milieu** de la zone de recherche restante, et en ne conservant que la moitié où la valeur peut encore se trouver.

CODE DE RÉFÉRENCE — Recherche dichotomique

```

1 def recherche_dichotomique(tableau_trie, valeur):
2     """Renvoie l'indice de 'valeur' dans 'tableau_trie' (trie), ou -1 si absente.
3     """
4     borne_inf, borne_sup = 0, len(tableau_trie) - 1
5     while borne_inf ≤ borne_sup:
6         milieu = (borne_inf + borne_sup) // 2
7         if tableau_trie[milieu] == valeur:
8             return milieu
9         elif tableau_trie[milieu] < valeur:
10            borne_inf = milieu + 1
11        else:
12            borne_sup = milieu - 1
13    return -1
14
15 t = [1, 3, 5, 7, 9, 11, 13, 15]
16 print(recherche_dichotomique(t, 11)) # 5
17 print(recherche_dichotomique(t, 4)) # -1

```

◆ **PROPRIÉTÉ Coût de la dichotomie** À chaque itération, la zone de recherche est **divisée par deux**. Le coût de la recherche dichotomique est donc **logarithmique** ($O(\log_2 n)$), bien plus rapide que le coût linéaire de la recherche séquentielle pour un grand tableau.

DÉMONSTRATION

Terminaison de la recherche dichotomique.

On considère le **variant de boucle** $V = \text{borne_sup} - \text{borne_inf}$.

V est **bien défini et positif ou nul** tant que la boucle continue, puisque la condition de la boucle while est précisément $\text{borne_inf} \leq \text{borne_sup}$.

V **décroît strictement** à chaque itération : si $\text{tableau_trie}[\text{milieu}] < \text{valeur}$, alors borne_inf devient $\text{milieu} + 1$, strictement supérieur à son ancienne valeur (puisque $\text{milieu} \geq \text{borne_inf}$), donc V diminue. Si $\text{tableau_trie}[\text{milieu}] > \text{valeur}$, alors borne_sup devient $\text{milieu} - 1$, strictement inférieur à son ancienne valeur (puisque $\text{milieu} \leq \text{borne_sup}$), donc V diminue également.

Conclusion. V est un entier positif ou nul qui décroît strictement à chaque tour de boucle : une suite d'entiers positifs strictement décroissante est nécessairement **finie**. La boucle while termine donc en un nombre fini d'itérations. ■

⚠ ERREUR FRÉQUENTE

Appliquer la dichotomie sur un tableau **non trié**. La dichotomie repose entièrement sur le tri : comparer à l'élément du milieu ne permet d'éliminer une moitié **que si** on est certain que tous les éléments de cette moitié sont, eux aussi, inférieurs (ou supérieurs) à la valeur cherchée — ce qui n'est garanti que si le tableau est trié.

» **Python À la machine.** Modifie `recherche_dichotomique` pour qu'elle compte le nombre d'itérations effectuées, et compare ce nombre à $\log_2(n)$ pour un tableau trié de 1000 éléments (utilise `math.log2`).

5.5 Algorithmes gloutons

DÉFINITION D2

Un **algorithme glouton** construit une solution étape par étape, en faisant à chaque étape le choix qui semble **le meilleur sur le moment** (le choix « localement optimal »), sans jamais revenir en arrière.

EXEMPLE Le **rendu de monnaie** : pour tenter d'obtenir un rendu avec peu de pièces, on choisit à chaque étape la plus grande pièce (ou billet) ne dépassant pas le montant restant. Les valeurs des pièces doivent être strictement positives. Si la stratégie gloutonne ne trouve pas de rendu exact, la fonction le signale au lieu de renvoyer un rendu partiel ; cela ne prouve pas qu'aucun autre rendu exact n'existe.

CODE DE RÉFÉRENCE — Rendu de monnaie glouton

```
1 def rendu_glouton(montant, pieces):
2     """Construit un rendu en choisissant d'abord les plus grandes pieces
3     disponibles.
4
5     Precondition : les valeurs des pieces sont strictement positives.
6     """
7     if not all(p > 0 for p in pieces):
8         raise ValueError("les pieces doivent etre strictement positives")
9     pieces_triees = sorted(pieces, reverse=True)
10    rendu = []
11    for p in pieces_triees:
12        while montant >= p:
13            rendu.append(p)
14            montant -= p
15    if montant != 0:
16        raise ValueError("l'algorithme glouton n'a pas trouve de rendu exact")
17    return rendu
18
19 print(rendu_glouton(78, [50, 20, 10, 5, 2, 1]))
```

◆ **PROPRIÉTÉ** Un algorithme glouton n'est pas toujours optimal Un algorithme glouton donne une solution **rapide à calculer**, mais qui n'est **pas garantie optimale** en général : le choix localement optimal à chaque étape peut conduire à une solution globalement moins bonne qu'une autre approche.

⚠ **CONTRE-EXEMPLE** Avec le système de pièces {4, 3, 1} (non standard), pour rendre 6 : la méthode gloutonne choisit d'abord 4 (la plus grande pièce ≤ 6), puis doit compléter les 2 restants avec deux pièces de 1 : elle utilise 3 pièces (4 + 1 + 1). Or une solution avec 2 pièces existe : 3 + 3 = 6. L'algorithme glouton n'est donc, dans ce cas, **pas optimal**.

◆ **PROPRIÉTÉ** Quand l'approche gloutonne fonctionne-t-elle ? Pour le système de pièces **en euros** {1, 2, 5, 10, 20, 50, 100, 200} (centimes ou euros), la méthode gloutonne donne **toujours** le nombre minimal de pièces : ce système a une propriété mathématique particulière (chaque pièce est « assez grande » par rapport aux suivantes) qui garantit l'optimalité du choix glouton. Ce n'est pas le cas pour un système de pièces quelconque.

ERREUR FRÉQUENTE

Croire qu'un algorithme glouton donne toujours la meilleure solution possible. Un algorithme glouton est une **heuristique** (une méthode rapide et souvent efficace en pratique), pas une garantie d'optimalité : il faut vérifier, au cas par cas, si les propriétés du problème garantissent l'optimalité de l'approche gloutonne.

» **Python À la machine.** Teste `rendu_glouton` avec le système de pièces {1, 5, 6, 9} pour rendre 11. Combien de pièces la méthode gloutonne utilise-t-elle ? Trouve, en cherchant à la main, une solution utilisant moins de pièces.

5.6 L'algorithme des k plus proches voisins

DÉFINITION D3

L'algorithme des **k plus proches voisins** prédit la classe d'un nouvel élément en examinant les **k éléments déjà classés** les plus proches de lui (au sens d'une distance, par exemple la distance euclidienne), et en lui attribuant la classe **majoritaire** parmi ces **k** voisins. Le paramètre **k** doit être entier et vérifier $1 \leq k \leq n$, où **n** est le nombre d'éléments déjà classés.

◆ **PROPRIÉTÉ Distance euclidienne entre deux points** Pour deux points (x_1, y_1) et (x_2, y_2) du plan, la distance euclidienne est $d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$.

CODE DE RÉFÉRENCE — Algorithme des k plus proches voisins

```

1 def distance(p1, p2):
2     """Distance euclidienne entre deux points (x, y)."""
3     return ((p1[0] - p2[0]) ** 2 + (p1[1] - p2[1]) ** 2) ** 0.5
4
5
6 def k_plus_proches_voisins(points_classes, nouveau_point, k):
7     """Prédit la classe de 'nouveau_point' à partir de ses k plus proches voisins.
8
9     'points_classes' : liste de triplets (x, y, classe) déjà étiquetées.
10    Precondition : k est un entier et 1 ≤ k ≤ len(points_classes).
11    """
12    if type(k) is not int or not 1 ≤ k ≤ len(points_classes):
13        raise ValueError(
14            "k doit être un entier compris entre 1 et le nombre de points"
15        )
16    distances = [
17        (distance((x, y), nouveau_point), classe)
18        for (x, y, classe) in points_classes
19    ]
20    distances.sort(key=lambda d: d[0])
21    k_plus_proches = distances[:k]
22    classes = [c for (_, c) in k_plus_proches]
23    return max(set(classes), key=classes.count)
24
25

```

```

26 points = [
27     (1, 1, "A"), (2, 1, "A"), (1, 2, "A"),
28     (8, 8, "B"), (9, 8, "B"), (8, 9, "B"),
29 ]
30 print(k_plus_proches_voisins(points, (1.5, 1.5), 3))
31 print(k_plus_proches_voisins(points, (8.5, 8.5), 3))

```

◆ **PROPRIÉTÉ Choix de k** Le choix du nombre k de voisins influence le résultat : un k **trop petit** rend la prédiction sensible au bruit (un seul voisin atypique peut fausser le résultat) ; un k **trop grand** risque d'inclure des voisins trop éloignés, peu pertinents pour la classification. On choisit souvent k impair (pour éviter les égalités entre deux classes) et adapté à la taille du jeu de données.

⚠ ERREUR FRÉQUENTE

Oublier de **trier** les distances avant de sélectionner les k plus proches voisins. Sans tri (par exemple en prenant les k **premiers** points de la liste `points_classes`, dans l'ordre où ils sont donnés), on ne sélectionne pas du tout les voisins les plus proches, mais des points arbitraires.

» **Python À la machine.** Ajoute un point mal classé aux données de l'exemple, par exemple (1, 1, "B") au milieu du groupe "A", puis reclasse le point (1.5, 1.5) avec $k = 3$ puis $k = 5$. Le résultat change-t-il ? Explique en quoi cela illustre l'intérêt d'un k pas trop petit face au bruit dans les données.

5.7 Parcours séquentiel d'un tableau

DÉFINITION D1

Un **parcours séquentiel** examine les éléments d'un tableau **un par un**, dans l'ordre, généralement à l'aide d'une boucle `for`. De nombreux algorithmes de base (recherche, extremum, moyenne) reposent sur ce principe.

◆ **PROPRIÉTÉ Recherche d'une occurrence** Rechercher une occurrence d'une valeur dans un tableau consiste à parcourir le tableau et à s'arrêter (en renvoyant l'indice trouvé) dès que la valeur est rencontrée. Si elle n'apparaît nulle part, on renvoie une valeur conventionnelle (par exemple -1).

CODE DE RÉFÉRENCE — Recherche d'une occurrence (coût linéaire)

```

1 def recherche_occurrence(tableau, valeur):
2     """Renvoie l'indice de la première occurrence de 'valeur', ou -1 si absente."""
3
4     for i in range(len(tableau)):
5         if tableau[i] == valeur:
6             return i
7     return -1
8
9 print(recherche_occurrence([5, 3, 8, 1], 8)) # 2

```

```
9 print(recherche_occurrence([5, 3, 8, 1], 99)) # -1
```

◆ **PROPRIÉTÉ Coût de la recherche** Dans le **pire cas** (valeur absente, ou en dernière position), la recherche d'occurrence examine **tous** les éléments du tableau : son coût est **linéaire** (proportionnel à la taille n du tableau, noté $O(n)$).

◆ **PROPRIÉTÉ Recherche d'un extremum** Rechercher le maximum (ou le minimum) d'un tableau non vide consiste à parcourir ses éléments en conservant, dans un accumulateur, la plus grande (ou plus petite) valeur rencontrée jusque-là. L'accumulateur doit être initialisé avec le **premier élément** du tableau, jamais avec une constante arbitraire.

CODE DE RÉFÉRENCE — Recherche du maximum et calcul d'une moyenne

```
1 def maximum(tableau):
2     """Renvoie le plus grand element de 'tableau' (non vide)."""
3     maxi = tableau[0]
4     for x in tableau[1:]:
5         if x > maxi:
6             maxi = x
7     return maxi
8
9 def moyenne(tableau):
10    """Renvoie la moyenne des elements de 'tableau' (non vide)."""
11    return sum(tableau) / len(tableau)
12
13 print(maximum([5, 3, 8, 1])) # 8
14 print(moyenne([5, 3, 8, 1])) # 4.25
```

⚠ ERREUR FRÉQUENTE

Initialiser l'accumulateur du maximum à 0 (EF). Cette erreur produit un résultat incorrect dès que **tous** les éléments du tableau sont négatifs : le maximum trouvé serait alors 0, une valeur absente du tableau. Il faut toujours initialiser l'accumulateur avec le **premier élément** du tableau.

» **Python À la machine.** Écris une fonction `recherche_toutes_occurrences(tableau, valeur)` qui renvoie la liste de **tous** les indices où valeur apparaît dans tableau (et non seulement le premier). Teste-la sur `[3, 7, 3, 3, 9]` avec `valeur=3` : le résultat attendu est `[0, 2, 3]`.

5.8 Le tri par insertion

DÉFINITION D2

Le **tri par insertion** construit progressivement une partie triée du tableau, en **insérant**, un par un, chaque élément restant à sa bonne place dans la partie déjà triée (comme on trie des cartes à jouer en main, une par une).

CODE DE RÉFÉRENCE — Tri par insertion

```

1 def tri_insertion(tableau):
2     """Trie 'tableau' par ordre croissant, par insertion (renvoie une copie triée).
3     """
4     tableau = tableau[:]
5     for i in range(1, len(tableau)):
6         valeur = tableau[i]
7         j = i - 1
8         while j ≥ 0 and tableau[j] > valeur:
9             tableau[j + 1] = tableau[j]
10            j -= 1
11            tableau[j + 1] = valeur
12     return tableau
13 print(tri_insertion([5, 3, 8, 1, 9, 2])) # [1, 2, 3, 5, 8, 9]

```

◆ **PROPRIÉTÉ Terminaison et coût** La boucle for externe s'exécute un nombre **fixe** de fois ($\max(n - 1, 0)$ fois pour un tableau de taille n) : elle termine nécessairement. Pour $n \leq 1$, aucune itération n'est nécessaire : ces tableaux sont déjà triés. Dans la boucle while interne, avant chaque tour exécuté, $j \geq 0$; le variant entier $j + 1$ est donc **strictement positif** et décroît de 1. Lorsque $j = -1$, la condition de la boucle devient fausse. Dans le **pire cas** (tableau trié en ordre décroissant), le coût est **quadratique** ($O(n^2)$) : chaque insertion peut nécessiter de décaler tous les éléments déjà triés.

DÉMONSTRATION

| **Invariant de boucle** : au début de chaque itération i de la boucle for (avant l'insertion de `tableau[i]`), le sous-tableau `tableau[0:i]` est **trié** (et contient les mêmes éléments que le sous-tableau original `tableau[0:i]` avant tout tri).

Initialisation. Lorsque $n \geq 2$, avant la première itération ($i = 1$), `tableau[0:1]` est un tableau à un seul élément : il est trivialement trié.

Conservation. Supposons `tableau[0:i]` trié au début de l'itération i . La boucle while interne décale vers la droite tous les éléments de `tableau[0:i]` strictement supérieurs à `valeur = tableau[i]`, puis place `valeur` à la position ainsi libérée. Le sous-tableau `tableau[0:i+1]` obtenu est donc trié : l'invariant est conservé pour l'itération suivante.

Terminaison. Si $n \leq 1$, la boucle ne s'exécute pas et le tableau entier est déjà trié. Si $n \geq 2$, la dernière itération a $i = n - 1$ et l'invariant donne : `tableau[0:n]` (le tableau entier) est trié. L'algorithme est donc **correct**. ■

⚠ ERREUR FRÉQUENTE

Croire qu'un invariant de boucle doit être vérifié **une seule fois** (par exemple seulement à la fin). Un invariant doit être vrai à **chaque** itération : c'est cette répétition, de l'initialisation jusqu'à la terminaison, qui constitue la preuve par récurrence de la correction de l'algorithme.

» **Python À la machine.** Ajoute des instructions `assert tableau[0:i] == sorted(tableau[0:i])` dans ta propre implémentation du tri par insertion, juste avant l'insertion de chaque nouvel élément, pour vérifier expérimentalement l'invariant sur plusieurs tableaux de ton choix.

5.9 Le tri par sélection

DÉFINITION D3

Le **tri par sélection** construit progressivement une partie triée en **choisissant**, à chaque étape, le plus petit élément **restant** et en le plaçant à la bonne position (au lieu d'insérer un élément dans la partie triée, comme pour le tri par insertion).

CODE DE RÉFÉRENCE — Tri par sélection

```

1 def tri_selection(tableau):
2     """Trie 'tableau' par ordre croissant, par selection (renvoie une copie triee).
3     """
4     tableau = tableau[:]
5     n = len(tableau)
6     for i in range(n - 1):
7         indice_min = i
8         for j in range(i + 1, n):
9             if tableau[j] < tableau[indice_min]:
10                 indice_min = j
11         tableau[i], tableau[indice_min] = tableau[indice_min], tableau[i]
12     return tableau
13 print(tri_selection([5, 3, 8, 1, 9, 2])) # [1, 2, 3, 5, 8, 9]
```

◆ **PROPRIÉTÉ Terminaison et coût** Pour un tableau de taille n , la boucle externe s'exécute $\max(n - 1, 0)$ itérations. Si $n \leq 1$, la boucle externe ne s'exécute pas et le tableau est déjà trié. Sinon, les deux boucles `for` parcourent des intervalles finis, déterminés à l'avance : l'algorithme termine nécessairement. Pour n variable, le coût est **quadratique** ($O(n^2)$) : contrairement au tri par insertion, la recherche du minimum restant parcourt toujours l'intégralité de la partie non triée, même si le tableau est déjà trié.

DÉMONSTRATION

■ **Invariant de boucle** : au début de chaque itération i de la boucle `for` externe, le sous-tableau `tableau[0:i]` est **trié**, et chacun de ses éléments est **inférieur ou égal** à tous les éléments de `tableau[i:n]`.

Initialisation. Avant la première itération ($i = 0$), `tableau[0:0]` est le tableau vide : l'invariant est trivialement vrai.

Conservation. Supposons l'invariant vrai au début de l'itération i . La boucle `for` interne détermine `indice_min`, l'indice du plus petit élément de `tableau[i:n]`. L'échange place cet élément minimal en position i . Comme `tableau[0:i]` était déjà trié et que ses éléments étaient tous inférieurs ou égaux à `tableau[i:n]` (donc au minimum choisi), `tableau[0:i+1]` est trié ; et ce minimum étant le plus petit du reste, tous les éléments de `tableau[0:i+1]` restent inférieurs ou égaux à `tableau[i+1:n]`. L'invariant est conservé.

Terminaison. Si $n \leq 1$, la boucle externe ne s'exécute pas et le tableau est déjà trié. Si $n \geq 2$, la dernière itération a $i = n - 2$. Après sa conservation, l'invariant donne que `tableau[0:n-1]` est trié et que son dernier élément est inférieur ou égal à l'unique élément de `tableau[n-1:n]` : le tableau entier est donc trié. ■

◆ **PROPRIÉTÉ Insertion contre sélection** Les deux tris ont le même coût dans le pire cas ($O(n^2)$), mais le tri par **insertion** est plus rapide en pratique sur un tableau **presque trié** (la boucle `while` interne s'arrête vite), alors que le tri par **sélection** effectue toujours le même nombre de comparaisons, quel que soit l'état initial du tableau.

⚠ ERREUR FRÉQUENTE

Confondre les deux tris : dans le tri par sélection, on **recherche le minimum** de toute la partie restante avant de l'échanger ; dans le tri par insertion, on **insère** l'élément courant à sa place en décalant les éléments plus grands. Ce ne sont pas les mêmes opérations, même si le résultat final est identique.

» **Python À la machine.** Compare expérimentalement le nombre de comparaisons effectuées par `tri_insertion` et `tri_selection` sur un tableau **déjà trié** de 20 éléments (ajoute un compteur global incrémenté à chaque comparaison dans les deux fonctions). Le résultat confirme-t-il la propriété du cours ?

① MÉTHODE M1 Deriver une fonction composée

Quand l'utiliser. Utiliser cette méthode lorsque la fonction à dériver fait intervenir une composition : $e^{u(x)}$, $\ln(u(x))$, $(u(x))^n$, $\sqrt{u(x)}$, ou plus généralement $v(u(x))$.

La méthode pas à pas.

1. Identifier la fonction extérieure v et la fonction intérieure u .
2. Calculer $u'(x)$.
3. Calculer $v'(t)$.
4. Appliquer la formule : $(v \circ u)'(x) = u'(x) \times v'(u(x))$.
5. Simplifier l'expression obtenue.

Exemple rédigé. Exemple. Dériver $f(x) = e^{x^2-3x}$.

Fonction intérieure : $u(x) = x^2 - 3x$, donc $u'(x) = 2x - 3$.

Fonction extérieure : $v(t) = e^t$, donc $v'(t) = e^t$.

$$f'(x) = u'(x) \times e^{u(x)} = (2x - 3) e^{x^2-3x}.$$

Les pièges.

- **Piège 1.** Oublier $u'(x)$: écrire $(e^u)' = e^u$ au lieu de $u' e^u$.
- **Piège 2.** Pour $(\ln u)'$, oublier le signe de u : vérifier que $u > 0$ sur l'intervalle d'étude.

Vérifier son résultat. Vérifier sur un cas simple : si $u(x) = 2x$, alors $(e^{2x})' = 2e^{2x}$ (et non e^{2x}).

S'entraîner. (renvois exercices M1)

Exercice 1 ♦

Calculer la dérivée de la fonction f définie sur $\mathbb{R}$ par :

$$f(x) = e^{3x^2+1}.$$

Exercice 2 ♦

Soit f la fonction définie par $f(x) = \ln(x^2 + 3x + 2)$.

1. Déterminer l'ensemble de définition de f .
2. Calculer $f'(x)$.

Exercice 3 ♦

Calculer la dérivée de $f(x) = (2x + 1)^5$ sur $\mathbb{R}$.

Exercice 4 ◆

Calculer la dérivée de $f(x) = \sqrt{x^2 + 4}$ sur $\mathbb{R}$.

Exercice 5 ◆◆

Soit f définie sur $\mathbb{R}^*$ par $f(x) = e^{1/x}$.

1. Calculer $f'(x)$.
2. Étudier le signe de $f'(x)$ sur $\mathbb{R}^*$ et en déduire les variations de f .

Exercice 6 ◆◆

Soit f définie sur $]0, +\infty[$ par $f(x) = \ln\left(\frac{1}{x}\right)$.

1. Simplifier l'expression de f .
2. Calculer $f'(x)$ de deux manières différentes.

Exercice 7 ◆◆

Soit f définie sur $\mathbb{R}$ par $f(x) = e^{x^2-2}$.

1. Calculer $f'(x)$.
2. Déterminer l'équation de la tangente à la courbe de f au point d'abscisse 0.
3. La courbe est-elle au-dessus ou au-dessous de cette tangente au voisinage de 0 ?

Exercice 8 ◆◆

Calculer la dérivée de $f(x) = (x^3 - 1)^4 e^{2x}$ sur $\mathbb{R}$.

Exercice 9 ◆◆◆

On considère la fonction f définie sur $[0, 2]$ par $f(x) = x\sqrt{4 - x^2}$.

1. Justifier que f est bien définie sur $[0, 2]$.
2. Calculer $f'(x)$ et déterminer les variations de f .
3. En déduire le maximum de f sur $[0, 2]$ et la valeur de x pour laquelle il est atteint.

Exercice 10 ◆

Étudier les variations de $f(x) = x^3 - 3x^2 + 2$ sur $\mathbb{R}$.

Exercice 11 ◆

Étudier les variations de $f(x) = e^x - x$ sur $\mathbb{R}$ et déterminer ses limites en $\pm\infty$.

Exercice 12 ◆

Étudier les variations de $f(x) = x \ln x$ sur $]0, +\infty[$, limites comprises.

Exercice 13 ◆

Étudier les variations de $f(x) = \frac{x^2 + 1}{x}$ sur $\mathbb{R}^*$.

Exercice 14 ◆◆

Soit $f(x) = x^2 e^{-x}$ définie sur $\mathbb{R}$.

1. Calculer $f'(x)$ et étudier son signe.
2. Déterminer les limites de f en $\pm\infty$.
3. Dresser le tableau de variations de f .

Exercice 15

Soit $f(x) = \frac{\ln x}{x}$ sur $]0, +\infty[$.

1. Calculer $f'(x)$ et étudier son signe.
2. Déterminer les limites de f en 0^+ et en $+\infty$.
3. Dresser le tableau de variations.

Exercice 16

Soit $f(x) = \frac{e^x}{x+1}$ définie sur $\mathbb{R} \setminus \{-1\}$.

1. Calculer $f'(x)$ et étudier son signe.
2. Déterminer les limites aux bornes du domaine de définition.
3. Dresser le tableau de variations complet.

Exercice 17

Soit $f(x) = x^3 - 6x^2 + 9x + 1$.

1. Dresser le tableau de variations de f .
2. Déterminer l'équation de la tangente à la courbe au point d'abscisse 2.
3. La courbe traverse-t-elle cette tangente en $x = 2$?

Exercice 18

On découpe aux quatre coins d'une plaque carrée de côté 20 cm des carrés de côté x cm, puis on replie pour former une boîte ouverte.

1. Exprimer le volume $V(x)$ de la boîte en fonction de x .
2. Pour quelle valeur de x le volume est-il maximal ? Quel est ce volume maximal ?

Exercice 19

Le coût de production de x unités est modélisé par $C(x) = \frac{x^3}{3} - 5x^2 + 30x$.

1. Calculer le coût marginal $C'(x)$. Son signe permet-il d'identifier un coût minimum ?
2. Calculer le coût moyen $\frac{C(x)}{x}$ et déterminer s'il admet un minimum.
3. Comparer les deux résultats.

Exercice 20

Démontrer que pour tout $x \in \mathbb{R}$, $e^x \geq 1 + x$.

Exercice 21

Démontrer que pour tout $x > 0$, $\ln x \leq x - 1$.

Exercice 22

Soit $f = e^x$. On rappelle que f est convexe sur $\mathbb{R}$.

1. Énoncer l'inégalité de Jensen pour f aux points x et y avec $t = \frac{1}{2}$.
2. Vérifier cette inégalité pour $x = 0$ et $y = 2$.

Exercice 23 ♦♦

Démontrer que pour tous réels x et y :

$$e^x + e^y \geq 2e^{\frac{x+y}{2}}.$$

Exercice 24 ♦♦

Démontrer que pour tous réels x et y :

$$x^2 + y^2 \geq \frac{(x+y)^2}{2}.$$

Exercice 25 ♦

Calculer la dérivée de la fonction f définie sur $\mathbb{R}$ par :

$$f(x) = e^{3x^2+1}.$$

Exercice 26 ♦

Soit f la fonction définie par $f(x) = \ln(x^2 + 3x + 2)$.

1. Déterminer l'ensemble de définition de f .
2. Calculer $f'(x)$.

Exercice 27 ♦

Calculer la dérivée de $f(x) = (2x + 1)^5$ sur $\mathbb{R}$.

Exercice 28 ♦

Calculer la dérivée de $f(x) = \sqrt{x^2 + 4}$ sur $\mathbb{R}$.

Exercice 29 ♦♦

Soit f définie sur $\mathbb{R}^*$ par $f(x) = e^{1/x}$.

1. Calculer $f'(x)$.
2. Étudier le signe de $f'(x)$ sur $\mathbb{R}^*$ et en déduire les variations de f .

Exercice 30 ♦♦

Soit f définie sur $]0, +\infty[$ par $f(x) = \ln\left(\frac{1}{x}\right)$.

1. Simplifier l'expression de f .
2. Calculer $f'(x)$ de deux manières différentes.

Exercice 31 ♦♦

Soit f définie sur $\mathbb{R}$ par $f(x) = e^{x^2-2}$.

1. Calculer $f'(x)$.
2. Déterminer l'équation de la tangente à la courbe de f au point d'abscisse 0.
3. La courbe est-elle au-dessus ou au-dessous de cette tangente au voisinage de 0 ?

Exercice 32 ♦♦

Calculer la dérivée de $f(x) = (x^3 - 1)^4 e^{2x}$ sur $\mathbb{R}$.

Exercice 33 ♦♦♦

On considère la fonction f définie sur $[0, 2]$ par $f(x) = x\sqrt{4 - x^2}$.

1. Justifier que f est bien définie sur $[0, 2]$.
2. Calculer $f'(x)$ et déterminer les variations de f .
3. En déduire le maximum de f sur $[0, 2]$ et la valeur de x pour laquelle il est atteint.

Exercice 34 ♦

Étudier les variations de $f(x) = x^3 - 3x^2 + 2$ sur $\mathbb{R}$.

Exercice 35 ♦

Étudier les variations de $f(x) = e^x - x$ sur $\mathbb{R}$ et déterminer ses limites en $\pm\infty$.

Exercice 36 ♦

Étudier les variations de $f(x) = x \ln x$ sur $]0, +\infty[$, limites comprises.

QCM d'auto-évaluation — Algorithmique 1 : parcours et tris

Pour chaque question, une seule réponse est exacte. Entourez la lettre correspondante.

C1 — Recherche d'occurrence

1. [Q1] Le coût, dans le pire cas, d'une recherche séquentielle d'une valeur dans un tableau de taille n est :
 - A. constant.
 - B. logarithmique.
 - C. linéaire, $O(n)$.
 - D. quadratique, $O(n^2)$.

C2 — Extremum

2. [Q2] Pour rechercher le maximum d'un tableau non vide, il faut initialiser l'accumulateur avec :
 - A. la valeur 0.
 - B. le premier élément du tableau.
 - C. la plus grande valeur possible.
 - D. le nombre d'éléments du tableau.

C3/C4 — Tri par insertion

3. [Q3] Le tri par insertion construit la partie triée du tableau en :
- A. recherchant le minimum de la partie restante à chaque étape.
 - B. insérant chaque nouvel élément à sa bonne place dans la partie déjà triée.
 - C. triant d'abord la seconde moitié du tableau.
 - D. échangeant des éléments au hasard.
4. [Q4] L'invariant de boucle du tri par insertion affirme qu'au début de l'itération i :
- A. le tableau entier est déjà trié.
 - B. le sous-tableau `tableau[0:i]` est trié.
 - C. le sous-tableau `tableau[i:n]` est trié.
 - D. aucune partie du tableau n'est triée.

C5/C6 — Tri par sélection

5. [Q5] Le tri par sélection construit la partie triée du tableau en :
- A. insérant chaque élément à sa bonne place.
 - B. recherchant le minimum de la partie restante et l'échangeant à sa place.
 - C. divisant le tableau en deux moitiés récursivement.
 - D. ne comparant jamais deux éléments.
6. [Q6] Contrairement au tri par insertion, le tri par sélection effectue, dans TOUS les cas (tableau déjà trié ou non) :
- A. un nombre de comparaisons variable selon l'ordre initial.
 - B. toujours le même nombre de comparaisons, quel que soit l'ordre initial.
 - C. zéro comparaison.
 - D. un nombre de comparaisons linéaire, $O(n)$.

Corrigé

1. La valeur 30 se trouve à l'indice 3.
2. Le maximum est 30.
3. $\frac{22 + 15 + 8 + 30 + 11}{5} = \frac{86}{5} = 17,2$.

Corrigé

1. Tableau initial : [9, 2, 6, 4].
 - Après $i = 1$ (insertion de 2) : [2, 9, 6, 4]
 - Après $i = 2$ (insertion de 6) : [2, 6, 9, 4]
 - Après $i = 3$ (insertion de 4) : [2, 4, 6, 9]
2. Après $i = 1$: [2, 9] trié. Après $i = 2$: [2, 6, 9] trié. Après $i = 3$: [2, 4, 6, 9] trié. L'invariant est vérifié à chaque étape.
3. Le coût dans le pire cas est **quadratique**, $O(n^2)$.

Évaluation A — Parcours et tris

Durée : 45 minutes. Ordinateur avec interpréteur Python autorisé.

Barème indicatif : Ex. 1 : 10 pts (recherche, extremum, moyenne) ; Ex. 2 : 10 pts (tri par insertion, invariant)

Exercice 1 — Parcours séquentiel

(10 points)

On donne le tableau [22, 15, 8, 30, 11].

- (3 pts) À quel indice se trouve la valeur 30 ? Utiliser recherche_occurrence du cours.
- (3 pts) Calculer le maximum de ce tableau avec maximum.
- (4 pts) Calculer la moyenne de ce tableau avec moyenne. Détailler le calcul à la main.

Exercice 2 — Tri par insertion

(10 points)

On applique le tri par insertion au tableau [9, 2, 6, 4].

- (5 pts) Dérouler l'algorithme : donner le contenu du tableau après chaque itération de la boucle externe.
- (3 pts) Vérifier, à chaque étape, l'invariant de boucle du cours (le préfixe traité est trié).
- (2 pts) Quel est le coût de cet algorithme dans le pire cas, en fonction de la taille n du tableau ?

Corrigé

- La valeur 6 se trouve à l'indice 2.
- Le maximum est 40.
- $\frac{17 + 40 + 6 + 25 + 12}{5} = \frac{100}{5} = 20$.

Corrigé

- Tableau initial : [7, 1, 4, 9].
 - $i = 0$: minimum de [7,1,4,9] à l'indice 1 (valeur 1). Après échange : [1, 7, 4, 9].
 - $i = 1$: minimum de [7,4,9] à l'indice 2 (valeur 4). Après échange : [1, 4, 7, 9].
 - $i = 2$: minimum de [7,9] à l'indice 2 (valeur 7, déjà en place). Après échange (avec elle-même) : [1, 4, 7, 9].
- Après $i = 0$: [1] trié, $1 \leq 4$ (min du reste). Après $i = 1$: [1,4] trié, $4 \leq 7$. Après $i = 2$: [1,4,7] trié, $7 \leq 9$. L'invariant est vérifié à chaque étape.
- Le coût dans le pire cas est **quadratique**, $O(n^2)$ (et c'est aussi le cas dans le **meilleur** cas pour ce tri, contrairement au tri par insertion).

Évaluation B — Parcours et tris

Durée : 45 minutes. Ordinateur avec interpréteur Python autorisé.

Barème indicatif : Ex. 1 : 10 pts (recherche, extremum, moyenne) ; Ex. 2 : 10 pts (tri par sélection, invariant)

Exercice 1 — Parcours séquentiel

(10 points)

On donne le tableau [17, 40, 6, 25, 12].

- (3 pts) À quel indice se trouve la valeur 6 ? Utiliser recherche_occurrence du cours.
- (3 pts) Calculer le maximum de ce tableau avec maximum.
- (4 pts) Calculer la moyenne de ce tableau avec moyenne. Détailler le calcul à la main.

Exercice 2 — Tri par sélection

(10 points)

On applique le tri par sélection au tableau [7, 1, 4, 9].

1. (5 pts) Dérouler l'algorithme : pour chaque itération de la boucle externe, donner l'indice du minimum trouvé et le contenu du tableau après l'échange.
2. (3 pts) Vérifier, à chaque étape, l'invariant de boucle du cours (préfixe trié et inférieur ou égal au reste).
3. (2 pts) Quel est le coût de cet algorithme dans le pire cas, en fonction de la taille n du tableau ?

Rappel – Derivation : derivees de reference et regles

Derivees usuelles : $(x^n)' = nx^{n-1}$, $(e^x)' = e^x$, $(\ln x)' = \frac{1}{x}$, $(\sqrt{x})' = \frac{1}{2\sqrt{x}}$.

Regles : $(u + v)' = u' + v'$, $(uv)' = u'v + uv'$, $\left(\frac{u}{v}\right)' = \frac{u'v - uv'}{v^2}$.

Exercice 37

Calculer les derivees des fonctions suivantes :

1. $f(x) = x^4 - 3x^2 + 5$
2. $g(x) = \frac{e^x}{x}$ sur $\mathbb{R}^*$
3. $h(x) = x \ln x$ sur $]0, +\infty[$

Rappel – Derivation locale : nombre derive et tangente

Nombre derive : $f'(a) = \lim_{h \rightarrow 0} \frac{f(a+h) - f(a)}{h}$.

Tangente : equation $y = f(a) + f'(a)(x - a)$.

Exercice 38

Soit $f(x) = x^2 + 3x - 1$.

1. Calculer $f'(1)$ par la formule du nombre derive.
2. Donner l'equation de la tangente a la courbe en $x = 1$.
3. En quel point la tangente est-elle horizontale ?

Exercice 39

Un élève écrit la recherche dichotomique suivante, avec une erreur dans la mise à jour de `borne_inf` :

```
1 def recherche_dichotomique_bugue(tableau_trie, valeur):
2     borne_inf, borne_sup = 0, len(tableau_trie) - 1
3     while borne_inf ≤ borne_sup:
4         milieu = (borne_inf + borne_sup) // 2
5         if tableau_trie[milieu] == valeur:
6             return milieu
7         elif tableau_trie[milieu] < valeur:
8             borne_inf = milieu # au lieu de milieu + 1
9         else:
10            borne_sup = milieu - 1
11    return -1
```

1. Sur le tableau $[1, 3, 5, 7, 9, 11, 13]$, que se passe-t-il si on cherche la valeur 4 (absente) avec cette fonction buggée ? (on pourra limiter le nombre d'itérations testées à 20 pour éviter une exécution trop longue)
2. Calculer le variant $V = \text{borne_sup} - \text{borne_inf}$ à chaque itération. Décroît-il toujours strictement avec cette version buggée ?

- Corriger le code pour que la terminaison soit à nouveau garantie.

Rappel – Derivation : dérivées de référence et règles

Dérivées usuelles : $(x^n)' = nx^{n-1}$, $(e^x)' = e^x$, $(\ln x)' = \frac{1}{x}$, $(\sqrt{x})' = \frac{1}{2\sqrt{x}}$.

Règles : $(u + v)' = u' + v'$, $(uv)' = u'v + uv'$, $\left(\frac{u}{v}\right)' = \frac{u'v - uv'}{v^2}$.

Exercice 40

Calculer les dérivées des fonctions suivantes :

- $f(x) = x^4 - 3x^2 + 5$
- $g(x) = \frac{e^x}{x}$ sur $\mathbb{R}^*$
- $h(x) = x \ln x$ sur $]0, +\infty[$

Rappel – Derivation locale : nombre dérivé et tangente

Nombre dérivé : $f'(a) = \lim_{h \rightarrow 0} \frac{f(a+h) - f(a)}{h}$.

Tangente : équation $y = f(a) + f'(a)(x - a)$.

Exercice 41

Soit $f(x) = x^2 + 3x - 1$.

- Calculer $f'(1)$ par la formule du nombre dérivé.
- Donner l'équation de la tangente à la courbe en $x = 1$.
- En quel point la tangente est-elle horizontale ?

Exercice 42

Un élève écrit la recherche dichotomique suivante, avec une erreur dans la mise à jour de `borne_inf` :

```
1 def recherche_dichotomique_bugue(tableau_trie, valeur):
2     borne_inf, borne_sup = 0, len(tableau_trie) - 1
3     while borne_inf <= borne_sup:
4         milieu = (borne_inf + borne_sup) // 2
5         if tableau_trie[milieu] == valeur:
6             return milieu
7         elif tableau_trie[milieu] < valeur:
8             borne_inf = milieu # au lieu de milieu + 1
9         else:
10            borne_sup = milieu - 1
11    return -1
```

- Sur le tableau $[1, 3, 5, 7, 9, 11, 13]$, que se passe-t-il si on cherche la valeur 4 (absente) avec cette fonction buggée ? (on pourra limiter le nombre d'itérations testées à 20 pour éviter une exécution trop longue)
- Calculer le variant $V = \text{borne_sup} - \text{borne_inf}$ à chaque itération. Décroît-il toujours strictement avec cette version buggée ?
- Corriger le code pour que la terminaison soit à nouveau garantie.

Exercice 43

Un élève écrit le tri par sélection suivant, mais place l'initialisation de `indice_min` à l'intérieur de la boucle for interne au lieu de l'externe :

```
1 def tri_selection_bugue(tableau):
2     tableau = tableau[:]
3     n = len(tableau)
4     for i in range(n - 1):
5         for j in range(i + 1, n):
6             indice_min = i # reinitialise a CHAQUE tour de la boucle interne
7             if tableau[j] < tableau[indice_min]:
8                 indice_min = j
9             tableau[i], tableau[indice_min] = tableau[indice_min], tableau[i]
10    return tableau
```

1. Tester cette fonction sur $[9, 1, 5, 3, 7]$. Le résultat est-il correctement trié ?
2. Expliquer précisément pourquoi replacer `indice_min = i` à l'intérieur de la boucle for j empêche l'algorithme de retenir le minimum **global** de la partie restante.
3. Corriger le code en plaçant l'initialisation au bon endroit.

Corrigé

On pose $u(x) = 3x^2 + 1$ et $v(t) = e^t$. Alors $u'(x) = 6x$ et $v'(t) = e^t$.

La formule $(v \circ u)' = u' \times (v' \circ u)$ donne :

$$f'(x) = 6x e^{3x^2+1}.$$

En particulier $f'(0) = 0$ et $f'(1) = 6e^4$.

Corrigé

Question 1. $x^2 + 3x + 2 = (x + 1)(x + 2) > 0$ si et seulement si $x \in]-\infty, -2[\cup]-1, +\infty[$.

Donc $D_f =]-\infty, -2[\cup]-1, +\infty[$.

Question 2. On pose $u(x) = x^2 + 3x + 2$. Alors $u'(x) = 2x + 3$.

$$f'(x) = \frac{u'(x)}{u(x)} = \frac{2x + 3}{x^2 + 3x + 2}.$$

Corrigé

On pose $u(x) = 2x + 1$. Alors $u'(x) = 2$.

La formule $(u^n)' = n u' u^{n-1}$ donne :

$$f'(x) = 5 \times 2 \times (2x + 1)^4 = 10(2x + 1)^4.$$

On vérifie : $f'(0) = 10$ et $f'(-\frac{1}{2}) = 0$.

Corrigé

On pose $u(x) = x^2 + 4$. Alors $u'(x) = 2x$ et $u(x) > 0$ sur $\mathbb{R}$.

La formule $(\sqrt{u})' = \frac{u'}{2\sqrt{u}}$ donne :

$$f'(x) = \frac{2x}{2\sqrt{x^2 + 4}} = \frac{x}{\sqrt{x^2 + 4}}.$$

On vérifie : $f'(0) = 0$ (la tangente est horizontale en 0).

Corrigé

Question 1. On pose $u(x) = \frac{1}{x}$. Alors $u'(x) = -\frac{1}{x^2}$.

$$f'(x) = u'(x) e^{u(x)} = -\frac{1}{x^2} e^{1/x}.$$

Question 2. Pour tout $x \in \mathbb{R}^*$, on a $x^2 > 0$ et $e^{1/x} > 0$, donc $f'(x) < 0$.

La fonction f est **strictement décroissante** sur $] -\infty, 0[$ et sur $] 0, +\infty[$.

Corrigé

Question 1. $f(x) = \ln\left(\frac{1}{x}\right) = -\ln(x)$ sur $] 0, +\infty[$.

Question 2.

► **Méthode 1** (simplification) : $f(x) = -\ln(x)$, donc $f'(x) = -\frac{1}{x}$.

► **Méthode 2** (composition) : on pose $u(x) = \frac{1}{x}$, $u'(x) = -\frac{1}{x^2}$.

$$f'(x) = \frac{u'(x)}{u(x)} = \frac{-1/x^2}{1/x} = -\frac{1}{x}.$$

Les deux méthodes donnent bien $f'(x) = -\frac{1}{x}$.

Corrigé

Question 1. $u(x) = x^2 - 2$, $u'(x) = 2x$. Donc $f'(x) = 2x e^{x^2-2}$.

Question 2. $f(0) = e^{-2}$ et $f'(0) = 0$. La tangente en 0 est :

$$y = e^{-2}.$$

Question 3. Étudions $g(x) = f(x) - e^{-2} = e^{x^2-2} - e^{-2}$.

Au voisinage de 0, $x^2 - 2 \geq -2$ donc $e^{x^2-2} \geq e^{-2}$, soit $g(x) \geq 0$.

La courbe est **au-dessus** de la tangente au voisinage de 0 (et en fait partout).

Corrigé

On écrit $f = u \times v$ avec $u = (x^3 - 1)^4$ et $v = e^{2x}$.

$u' = 4 \times 3x^2 \times (x^3 - 1)^3 = 12x^2(x^3 - 1)^3$ et $v' = 2e^{2x}$.

$$f'(x) = 12x^2(x^3 - 1)^3 e^{2x} + (x^3 - 1)^4 \times 2e^{2x}.$$

On factorise :

$$f'(x) = (x^3 - 1)^3 e^{2x} [12x^2 + 2(x^3 - 1)] = (x^3 - 1)^3 e^{2x} (2x^3 + 12x^2 - 2).$$

Corrigé

Question 1. Sur $[0, 2]$, on a $0 \leq x^2 \leq 4$, donc $4 - x^2 \geq 0$ et f est bien définie.

Question 2. $f = u \times v$ avec $u = x$ et $v = \sqrt{4 - x^2}$. $u' = 1$ et $v' = \frac{-2x}{2\sqrt{4 - x^2}} = \frac{-x}{\sqrt{4 - x^2}}$.

$$f'(x) = \sqrt{4-x^2} + x \times \frac{-x}{\sqrt{4-x^2}} = \frac{4-x^2-x^2}{\sqrt{4-x^2}} = \frac{4-2x^2}{\sqrt{4-x^2}}.$$

$$f'(x) = 0 \iff 4-2x^2 = 0 \iff x = \sqrt{2} \text{ (sur } [0, 2]).$$

f' est positive sur $[0, \sqrt{2}]$ et négative sur $[\sqrt{2}, 2]$: f est croissante puis décroissante.

Question 3. Le maximum est atteint en $x = \sqrt{2}$ et vaut :

$$f(\sqrt{2}) = \sqrt{2} \sqrt{4-2} = \sqrt{2} \times \sqrt{2} = \boxed{2}.$$

Corrigé

$$f'(x) = 3x^2 - 6x = 3x(x-2).$$

$$f'(x) \in]-\infty, 0[\cup]0, 2[\cup]2, +\infty[$$

$$f(0) = 2 \text{ (maximum local)}, f(2) = -2 \text{ (minimum local)}.$$

f est croissante sur $]-\infty, 0]$, décroissante sur $[0, 2]$, croissante sur $[2, +\infty[$.

Corrigé

$$f'(x) = e^x - 1.$$

$$f'(x) = 0 \iff x = 0 \text{ et } f'(x) > 0 \text{ pour } x > 0, f'(x) < 0 \text{ pour } x < 0.$$

f est décroissante sur $]-\infty, 0]$ puis croissante sur $[0, +\infty[$.

Le minimum est $f(0) = 1$.

Limites : $\lim_{x \rightarrow +\infty} f(x) = +\infty$ (croissance comparée) et $\lim_{x \rightarrow -\infty} f(x) = +\infty$ ($-x$ domine).

Corrigé

$$f'(x) = \ln x + x \times \frac{1}{x} = \ln x + 1.$$

$$f'(x) = 0 \iff x = e^{-1}. f' \text{ est négative sur }]0, e^{-1}] \text{ et positive sur } [e^{-1}, +\infty[.$$

Le minimum est $f(e^{-1}) = e^{-1} \times (-1) = -e^{-1}$.

$\lim_{x \rightarrow 0^+} x \ln x = 0$ (croissances comparées) et $\lim_{x \rightarrow +\infty} x \ln x = +\infty$.

Corrigé

$$f(x) = x + \frac{1}{x}, \text{ donc } f'(x) = 1 - \frac{1}{x^2} = \frac{x^2 - 1}{x^2}.$$

$$f'(x) = 0 \iff x^2 = 1 \iff x = \pm 1.$$

$$f' > 0 \text{ sur }]-\infty, -1[\cup]1, +\infty[\text{ et } f' < 0 \text{ sur }]-1, 0[\cup]0, 1[.$$

f admet un minimum local $f(1) = 2$ et un maximum local $f(-1) = -2$.

Corrigé

$$\textbf{Question 1. } f'(x) = 2xe^{-x} - x^2e^{-x} = x(2-x)e^{-x}.$$

Comme $e^{-x} > 0$, le signe de f' est celui de $x(2-x)$: $f' > 0$ sur $[0, 2]$, $f' < 0$ sur $]-\infty, 0[\cup]2, +\infty[$.

Question 2. $\lim_{x \rightarrow +\infty} x^2 e^{-x} = 0$ (croissances comparées) et $\lim_{x \rightarrow -\infty} x^2 e^{-x} = +\infty$.

Question 3. f admet un minimum local $f(0) = 0$ et un maximum local $f(2) = 4e^{-2}$.

Corrigé

Question 1. $f'(x) = \frac{\frac{1}{x} \cdot x - \ln x \cdot 1}{x^2} = \frac{1 - \ln x}{x^2}.$

$f'(x) = 0 \iff \ln x = 1 \iff x = e. f' > 0 \text{ sur }]0, e] \text{ et } f' < 0 \text{ sur } [e, +\infty[.$

Question 2. $\lim_{x \rightarrow 0^+} \frac{\ln x}{x} = -\infty$ et $\lim_{x \rightarrow +\infty} \frac{\ln x}{x} = 0$ (croissances comparées).

Question 3. f admet un maximum $f(e) = \frac{1}{e}.$

Corrigé

Question 1. $f'(x) = \frac{e^x(x+1) - e^x}{(x+1)^2} = \frac{xe^x}{(x+1)^2}.$

$f'(x) = 0 \iff x = 0. f' < 0 \text{ sur }]-\infty, -1[\cup]-1, 0[\text{ et } f' > 0 \text{ sur }]0, +\infty[.$

Question 2. $\lim_{x \rightarrow +\infty} f = +\infty, \lim_{x \rightarrow -\infty} f = 0$ (croissances comparées), $\lim_{x \rightarrow -1} f = +\infty.$

Question 3. Minimum local $f(0) = 1$. La droite $x = -1$ est asymptote verticale.

Corrigé

Question 1. $f'(x) = 3x^2 - 12x + 9 = 3(x-1)(x-3).$

$f' > 0 \text{ sur }]-\infty, 1[\cup]3, +\infty[, f' < 0 \text{ sur } [1, 3].$

Maximum local $f(1) = 5$, minimum local $f(3) = 1.$

Question 2. $f(2) = 3$ et $f'(2) = 3(1)(-1) = -3$. La tangente est $y = -3(x-2) + 3 = -3x + 9.$

Question 3. $f''(x) = 6x - 12$, donc $f''(2) = 0$: le point $x = 2$ est un **point d'inflexion**.

La courbe traverse sa tangente en $x = 2$ (changement de convexité).

Corrigé

Question 1. La base de la boîte est un carré de cote $(20 - 2x)$ et la hauteur est $x.$

$$V(x) = x(20 - 2x)^2 \text{ pour } x \in [0, 10].$$

Question 2. $V'(x) = (20 - 2x)^2 + x \times 2(20 - 2x)(-2) = (20 - 2x)[(20 - 2x) - 4x] = (20 - 2x)(20 - 6x).$

$V'(x) = 0 \iff x = 10 \text{ ou } x = \frac{10}{3}$ (dans $[0, 10]$).

Le maximum est atteint en $x = \frac{10}{3}$ cm et vaut :

$$V\left(\frac{10}{3}\right) = \frac{10}{3} \times \left(\frac{40}{3}\right)^2 = \frac{16000}{27} \approx 593 \text{ cm}^3.$$

Corrigé

Question 1. $C'(x) = x^2 - 10x + 30$. Le discriminant vaut $100 - 120 = -20 < 0.$

Le coût marginal est strictement positif pour tout x : il n'y a pas de coût minimum (le coût augmente toujours).

Question 2. $\frac{C(x)}{x} = \frac{x^2}{3} - 5x + 30$. Sa dérivée est $\frac{2x}{3} - 5$, nulle en $x = \frac{15}{2}.$

Le coût moyen admet un minimum en $x = 7,5$: $\frac{C(7,5)}{7,5} = \frac{56,25}{3} - 37,5 + 30 = -25,625.$

Question 3. Le coût marginal croît tandis que le coût moyen décroît puis croît : l'optimum économique correspond au minimum du coût moyen.

Corrigé

Posons $g(x) = e^x - 1 - x$. On a $g'(x) = e^x - 1$ et $g''(x) = e^x > 0$.

La fonction g est donc **convexe** sur $\mathbb{R}$. Son minimum est atteint en $x = 0$ (car $g'(0) = 0$), et $g(0) = 0$.

Par convexité, $g(x) \geq g(0) = 0$ pour tout x , soit $e^x \geq 1 + x$.

Corrigé

Posons $g(x) = \ln x - x + 1$ sur $]0, +\infty[$.

$g'(x) = \frac{1}{x} - 1$ et $g''(x) = -\frac{1}{x^2} < 0$: g est concave.

$g'(x) = 0 \Leftrightarrow x = 1$ et $g(1) = 0$. Par concavité, $g(x) \leq g(1) = 0$, soit $\ln x \leq x - 1$.

Corrigé

Question 1. Par convexité de e^x et l'inégalité de Jensen :

$$e^{\frac{x+y}{2}} \leq \frac{e^x + e^y}{2}.$$

Question 2. Pour $x = 0$ et $y = 2$:

$$e^1 \approx 2,718 \quad \text{et} \quad \frac{1 + e^2}{2} \approx \frac{1 + 7,389}{2} \approx 4,195.$$

On a bien $e < \frac{1 + e^2}{2}$.

Corrigé

La fonction e^t est convexe sur $\mathbb{R}$. Par l'inégalité de Jensen avec $t = \frac{1}{2}$:

$$e^{\frac{x+y}{2}} \leq \frac{e^x + e^y}{2}.$$

En multipliant par 2 :

$$2e^{\frac{x+y}{2}} \leq e^x + e^y.$$

Corrigé

La fonction $t \mapsto t^2$ est convexe sur $\mathbb{R}$. Par Jensen avec $t = \frac{1}{2}$:

$$\left(\frac{x+y}{2}\right)^2 \leq \frac{x^2 + y^2}{2}.$$

En multipliant par 2 : $\frac{(x+y)^2}{2} \leq x^2 + y^2$.

Vérification directe : $x^2 + y^2 - \frac{(x+y)^2}{2} = \frac{(x-y)^2}{2} \geq 0$.

Corrigé

1. La fonction ne termine **jamais** : elle entre dans une boucle infinie (après 20 itérations, aucune réponse n'est obtenue).

2. Dès que `tableau_trie[milieu] < valeur` et que la zone de recherche se réduit à deux éléments consécutifs (`borne_sup = borne_inf + 1`), on a `milieu = borne_inf` : l'instruction `borne_inf = milieu`

laisse alors borne_inf **inchangé**. Le variant $V = \text{borne_sup} - \text{borne_inf}$ cesse de décroître (il stagne à la même valeur indéfiniment) : la preuve de terminaison du cours ne s'applique plus, et effectivement, l'algorithme ne termine pas.

3. Il faut remettre borne_inf = milieu + 1 (comme dans le cours), pour garantir que borne_inf augmente strictement à chaque fois que cette branche est empruntée, ce qui restaure la décroissance stricte du variant V .

Corrigé

On pose $u(x) = 3x^2 + 1$ et $v(t) = e^t$. Alors $u'(x) = 6x$ et $v'(t) = e^t$.

La formule $(v \circ u)' = u' \times (v' \circ u)$ donne :

$$f'(x) = 6x e^{3x^2+1}.$$

En particulier $f'(0) = 0$ et $f'(1) = 6e^4$.

Corrigé

Question 1. $x^2 + 3x + 2 = (x+1)(x+2) > 0$ si et seulement si $x \in]-\infty, -2[\cup]-1, +\infty[$.

Donc $D_f =]-\infty, -2[\cup]-1, +\infty[$.

Question 2. On pose $u(x) = x^2 + 3x + 2$. Alors $u'(x) = 2x + 3$.

$$f'(x) = \frac{u'(x)}{u(x)} = \frac{2x+3}{x^2+3x+2}.$$

Corrigé

On pose $u(x) = 2x + 1$. Alors $u'(x) = 2$.

La formule $(u^n)' = n u' u^{n-1}$ donne :

$$f'(x) = 5 \times 2 \times (2x+1)^4 = 10(2x+1)^4.$$

On vérifie : $f'(0) = 10$ et $f'(-\frac{1}{2}) = 0$.

Corrigé

On pose $u(x) = x^2 + 4$. Alors $u'(x) = 2x$ et $u(x) > 0$ sur $\mathbb{R}$.

La formule $(\sqrt{u})' = \frac{u'}{2\sqrt{u}}$ donne :

$$f'(x) = \frac{2x}{2\sqrt{x^2+4}} = \frac{x}{\sqrt{x^2+4}}.$$

On vérifie : $f'(0) = 0$ (la tangente est horizontale en 0).

Corrigé

Question 1. On pose $u(x) = \frac{1}{x}$. Alors $u'(x) = -\frac{1}{x^2}$.

$$f'(x) = u'(x) e^{u(x)} = -\frac{1}{x^2} e^{1/x}.$$

Question 2. Pour tout $x \in \mathbb{R}^*$, on a $x^2 > 0$ et $e^{1/x} > 0$, donc $f'(x) < 0$.

La fonction f est **strictement décroissante** sur $] -\infty, 0[$ et sur $] 0, +\infty[$.

Corrigé

Question 1. $f(x) = \ln\left(\frac{1}{x}\right) = -\ln(x)$ sur $]0, +\infty[$.

Question 2.

► **Méthode 1** (simplification) : $f(x) = -\ln(x)$, donc $f'(x) = -\frac{1}{x}$.

► **Méthode 2** (composition) : on pose $u(x) = \frac{1}{x}$, $u'(x) = -\frac{1}{x^2}$.

$$f'(x) = \frac{u'(x)}{u(x)} = \frac{-1/x^2}{1/x} = -\frac{1}{x}.$$

Les deux méthodes donnent bien $f'(x) = -\frac{1}{x}$.

Corrigé

Question 1. $u(x) = x^2 - 2$, $u'(x) = 2x$. Donc $f'(x) = 2x e^{x^2-2}$.

Question 2. $f(0) = e^{-2}$ et $f'(0) = 0$. La tangente en 0 est :

$$y = e^{-2}.$$

Question 3. Étudions $g(x) = f(x) - e^{-2} = e^{x^2-2} - e^{-2}$.

Au voisinage de 0, $x^2 - 2 \geq -2$ donc $e^{x^2-2} \geq e^{-2}$, soit $g(x) \geq 0$.

La courbe est **au-dessus** de la tangente au voisinage de 0 (et en fait partout).

Corrigé

On écrit $f = u \times v$ avec $u = (x^3 - 1)^4$ et $v = e^{2x}$.

$u' = 4 \times 3x^2 \times (x^3 - 1)^3 = 12x^2(x^3 - 1)^3$ et $v' = 2e^{2x}$.

$$f'(x) = 12x^2(x^3 - 1)^3 e^{2x} + (x^3 - 1)^4 \times 2e^{2x}.$$

On factorise :

$$f'(x) = (x^3 - 1)^3 e^{2x} [12x^2 + 2(x^3 - 1)] = (x^3 - 1)^3 e^{2x} (2x^3 + 12x^2 - 2).$$

Corrigé

Question 1. Sur $[0, 2]$, on a $0 \leq x^2 \leq 4$, donc $4 - x^2 \geq 0$ et f est bien définie.

Question 2. $f = u \times v$ avec $u = x$ et $v = \sqrt{4 - x^2}$. $u' = 1$ et $v' = \frac{-2x}{2\sqrt{4 - x^2}} = \frac{-x}{\sqrt{4 - x^2}}$.

$$f'(x) = \sqrt{4 - x^2} + x \times \frac{-x}{\sqrt{4 - x^2}} = \frac{4 - x^2 - x^2}{\sqrt{4 - x^2}} = \frac{4 - 2x^2}{\sqrt{4 - x^2}}.$$

$f'(x) = 0 \iff 4 - 2x^2 = 0 \iff x = \sqrt{2}$ (sur $[0, 2]$).

f' est positive sur $[0, \sqrt{2}]$ et négative sur $[\sqrt{2}, 2]$: f est croissante puis décroissante.

Question 3. Le maximum est atteint en $x = \sqrt{2}$ et vaut :

$$f(\sqrt{2}) = \sqrt{2} \sqrt{4 - 2} = \sqrt{2} \times \sqrt{2} = \boxed{2}.$$

Corrigé

$f'(x) = 3x^2 - 6x = 3x(x - 2)$.

$$f'(x) \in]-\infty, 0[\cup]0, 2[\cup]2, +\infty[$$

$f(0) = 2$ (maximum local), $f(2) = -2$ (minimum local).

f est croissante sur $]-\infty, 0]$, décroissante sur $[0, 2]$, croissante sur $[2, +\infty[$.

Corrigé

$$f'(x) = e^x - 1.$$

$$f'(x) = 0 \iff x = 0 \text{ et } f'(x) > 0 \text{ pour } x > 0, f'(x) < 0 \text{ pour } x < 0.$$

f est décroissante sur $]-\infty, 0]$ puis croissante sur $[0, +\infty[$.

Le minimum est $f(0) = 1$.

Limites : $\lim_{x \rightarrow +\infty} f(x) = +\infty$ (croissance comparée) et $\lim_{x \rightarrow -\infty} f(x) = +\infty$ ($-x$ domine).

Corrigé

1. `tri_selection_bubble([9, 1, 5, 3, 7])` renvoie `[7, 1, 5, 3, 9]` : le tableau n'est **pas** correctement trié (la version correcte donnerait `[1, 3, 5, 7, 9]`).

2. En réinitialisant `indice_min = i` à **chaque** tour de la boucle interne, la comparaison `tableau[j] < tableau[indice_min]` ne compare jamais `tableau[j]` qu'à `tableau[i]` (la valeur d'origine), jamais au minimum **déjà trouvé** lors des tours précédents de la boucle interne. Seule la toute **dernière** comparaison effectuée (avec $j = n-1$) détermine la valeur finale de `indice_min` utilisée pour l'échange : le minimum global de la partie restante n'est donc, en général, pas retrouvé.

3. Il faut initialiser `indice_min = i` **avant** la boucle `for j` (donc à l'intérieur de la boucle `for i`, mais avant la boucle interne), pour que la comparaison se fasse contre le **meilleur candidat trouvé jusque-là**, et non contre `tableau[i]` à chaque tour.

6 Algorithmique 2 : dichotomie, algorithmes gloutons, k plus proches voisins

6.1 Recherche dichotomique

DÉFINITION D1

La **recherche dichotomique** recherche une valeur dans un tableau **trié** en comparant systématiquement la valeur cherchée à l'élément **du milieu** de la zone de recherche restante, et en ne conservant que la moitié où la valeur peut encore se trouver.

CODE DE RÉFÉRENCE — Recherche dichotomique

```
1 def recherche_dichotomique(tableau_trie, valeur):
2     """Renvoie l'indice de 'valeur' dans 'tableau_trie' (trie), ou -1 si absente.
3     """
4     borne_inf, borne_sup = 0, len(tableau_trie) - 1
5     while borne_inf ≤ borne_sup:
6         milieu = (borne_inf + borne_sup) // 2
7         if tableau_trie[milieu] == valeur:
8             return milieu
9         elif tableau_trie[milieu] < valeur:
10            borne_inf = milieu + 1
11        else:
12            borne_sup = milieu - 1
13    return -1
14
15 t = [1, 3, 5, 7, 9, 11, 13, 15]
16 print(recherche_dichotomique(t, 11)) # 5
17 print(recherche_dichotomique(t, 4)) # -1
```

◆ **PROPRIÉTÉ Coût de la dichotomie** À chaque itération, la zone de recherche est **divisée par deux**. Le coût de la recherche dichotomique est donc **logarithmique** ($O(\log_2 n)$), bien plus rapide que le coût linéaire de la recherche séquentielle pour un grand tableau.

DÉMONSTRATION

Terminaison de la recherche dichotomique.

On considère le **variant de boucle** $V = \text{borne_sup} - \text{borne_inf}$.

V est bien défini et positif ou nul tant que la boucle continue, puisque la condition de la boucle while est précisément $\text{borne_inf} \leq \text{borne_sup}$.

V décroît strictement à chaque itération : si $\text{tableau_trie}[\text{milieu}] < \text{valeur}$, alors borne_inf devient $\text{milieu} + 1$, strictement supérieur à son ancienne valeur (puisque $\text{milieu} \geq \text{borne_inf}$), donc V diminue. Si $\text{tableau_trie}[\text{milieu}] > \text{valeur}$, alors borne_sup devient $\text{milieu} - 1$, strictement inférieur à son ancienne valeur (puisque $\text{milieu} \leq \text{borne_sup}$), donc V diminue également.

Conclusion. V est un entier positif ou nul qui décroît strictement à chaque tour de boucle : une suite d'entiers positifs strictement décroissante est nécessairement **finie**. La boucle while termine donc en un nombre fini d'itérations. ■

ERREUR FRÉQUENTE

Appliquer la dichotomie sur un tableau **non trié**. La dichotomie repose entièrement sur le tri : comparer à l'élément du milieu ne permet d'éliminer une moitié **que si** on est certain que tous les éléments de cette moitié sont, eux aussi, inférieurs (ou supérieurs) à la valeur cherchée — ce qui n'est garanti que si le tableau est trié.

» Python À la machine. Modifie recherche_dichotomique pour qu'elle compte le nombre d'itérations effectuées, et compare ce nombre à $\log_2(n)$ pour un tableau trié de 1000 éléments (utilise `math.log2`).

6.2 Algorithmes gloutons

DÉFINITION D2

Un **algorithme glouton** construit une solution étape par étape, en faisant à chaque étape le choix qui semble **le meilleur sur le moment** (le choix « localement optimal »), sans jamais revenir en arrière.

EXEMPLE Le **rendu de monnaie** : pour tenter d'obtenir un rendu avec peu de pièces, on choisit à chaque étape la plus grande pièce (ou billet) ne dépassant pas le montant restant. Les valeurs des pièces doivent être strictement positives. Si la stratégie gloutonne ne trouve pas de rendu exact, la fonction le signale au lieu de renvoyer un rendu partiel ; cela ne prouve pas qu'aucun autre rendu exact n'existe.

CODE DE RÉFÉRENCE — Rendu de monnaie glouton

```
1 def rendu_glouton(montant, pieces):
2     """Construit un rendu en choisissant d'abord les plus grandes pieces
3     disponibles.
4
5     Precondition : les valeurs des pieces sont strictement positives.
6     """
7     if not all(p > 0 for p in pieces):
8         raise ValueError("les pieces doivent etre strictement positives")
9     pieces_triees = sorted(pieces, reverse=True)
10    rendu = []
11    for p in pieces_triees:
12        while montant >= p:
13            rendu.append(p)
14            montant -= p
```

```

14     if montant != 0:
15         raise ValueError("l'algorithme glouton n'a pas trouve de rendu exact")
16     return rendu
17
18
19 print(rendu_glouton(78, [50, 20, 10, 5, 2, 1]))

```

◆ **PROPRIÉTÉ Un algorithme glouton n'est pas toujours optimal** Un algorithme glouton donne une solution **rapide à calculer**, mais qui n'est **pas garantie optimale** en général : le choix localement optimal à chaque étape peut conduire à une solution globalement moins bonne qu'une autre approche.

⚠ **CONTRE-EXEMPLE** Avec le système de pièces {4, 3, 1} (non standard), pour rendre 6 : la méthode gloutonne choisit d'abord 4 (la plus grande pièce ≤ 6), puis doit compléter les 2 restants avec deux pièces de 1 : elle utilise 3 pièces (4 + 1 + 1). Or une solution avec 2 pièces existe : 3 + 3 = 6. L'algorithme glouton n'est donc, dans ce cas, **pas optimal**.

◆ **PROPRIÉTÉ Quand l'approche gloutonne fonctionne-t-elle ?** Pour le système de pièces en euros {1, 2, 5, 10, 20, 50, 100, 200} (centimes ou euros), la méthode gloutonne donne **toujours** le nombre minimal de pièces : ce système a une propriété mathématique particulière (chaque pièce est « assez grande » par rapport aux suivantes) qui garantit l'optimalité du choix glouton. Ce n'est pas le cas pour un système de pièces quelconque.

⚠ ERREUR FRÉQUENTE

Croire qu'un algorithme glouton donne toujours la meilleure solution possible. Un algorithme glouton est une **heuristique** (une méthode rapide et souvent efficace en pratique), pas une garantie d'optimalité : il faut vérifier, au cas par cas, si les propriétés du problème garantissent l'optimalité de l'approche gloutonne.

» **Python À la machine.** Teste `rendu_glouton` avec le système de pièces {1, 5, 6, 9} pour rendre 11. Combien de pièces la méthode gloutonne utilise-t-elle ? Trouve, en cherchant à la main, une solution utilisant moins de pièces.

6.3 L'algorithme des k plus proches voisins

DÉFINITION D3

L'algorithme des **k plus proches voisins** prédit la classe d'un nouvel élément en examinant les k éléments **déjà classés** les plus proches de lui (au sens d'une distance, par exemple la distance euclidienne), et en lui attribuant la classe **majoritaire** parmi ces k voisins. Le paramètre k doit être entier et vérifier $1 \leq k \leq n$, où n est le nombre d'éléments déjà classés.

◆ **PROPRIÉTÉ Distance euclidienne entre deux points** Pour deux points (x_1, y_1) et (x_2, y_2) du plan, la distance euclidienne est $d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$.

CODE DE RÉFÉRENCE — Algorithme des k plus proches voisins

```

1 def distance(p1, p2):
2     """Distance euclidienne entre deux points (x, y)."""
3     return ((p1[0] - p2[0]) ** 2 + (p1[1] - p2[1]) ** 2) ** 0.5
4
5
6 def k_plus_proches_voisins(points_classes, nouveau_point, k):
7     """Predit la classe de 'nouveau_point' a partir de ses k plus proches voisins.
8
9     'points_classes' : liste de triplets (x, y, classe) deja etiquetes.
10    Precondition : k est un entier et 1 ≤ k ≤ len(points_classes).
11    """
12    if type(k) is not int or not 1 ≤ k ≤ len(points_classes):
13        raise ValueError(
14            "k doit etre un entier compris entre 1 et le nombre de points"
15        )
16    distances = [
17        (distance((x, y), nouveau_point), classe)
18        for (x, y, classe) in points_classes
19    ]
20    distances.sort(key=lambda d: d[0])
21    k_plus_proches = distances[:k]
22    classes = [c for (_, c) in k_plus_proches]
23    return max(set(classes), key=classes.count)
24
25
26 points = [
27     (1, 1, "A"), (2, 1, "A"), (1, 2, "A"),
28     (8, 8, "B"), (9, 8, "B"), (8, 9, "B"),
29 ]
30 print(k_plus_proches_voisins(points, (1.5, 1.5), 3))
31 print(k_plus_proches_voisins(points, (8.5, 8.5), 3))

```

◆ **PROPRIÉTÉ Choix de k** Le choix du nombre k de voisins influence le résultat : un k **trop petit** rend la prédiction sensible au bruit (un seul voisin atypique peut fausser le résultat) ; un k **trop grand** risque d'inclure des voisins trop éloignés, peu pertinents pour la classification. On choisit souvent k impair (pour éviter les égalités entre deux classes) et adapté à la taille du jeu de données.

⚠ ERREUR FRÉQUENTE

Oublier de **trier** les distances avant de sélectionner les k plus proches voisins. Sans tri (par exemple en prenant les k **premiers** points de la liste `points_classes`, dans l'ordre où ils sont donnés), on ne sélectionne pas du tout les voisins les plus proches, mais des points arbitraires.

» **Python À la machine.** Ajoute un point mal classé aux données de l'exemple, par exemple (1, 1, "B") au milieu du groupe "A", puis reclasse le point (1.5, 1.5) avec $k = 3$ puis $k = 5$. Le résultat change-t-il ? Explique en quoi cela illustre l'intérêt d'un k pas trop petit face au bruit dans les données.

🔑 MÉTHODE M1 Deriver une fonction composée

Quand l'utiliser. Utiliser cette méthode lorsque la fonction à dériver fait intervenir une composition : $e^{u(x)}$, $\ln(u(x))$, $(u(x))^n$, $\sqrt{u(x)}$, ou plus généralement $v(u(x))$.

La méthode pas à pas.

1. Identifier la fonction extérieure v et la fonction intérieure u .
2. Calculer $u'(x)$.
3. Calculer $v'(t)$.
4. Appliquer la formule : $(v \circ u)'(x) = u'(x) \times v'(u(x))$.
5. Simplifier l'expression obtenue.

Exemple rédigé. Exemple. Dériver $f(x) = e^{x^2-3x}$.

Fonction intérieure : $u(x) = x^2 - 3x$, donc $u'(x) = 2x - 3$.

Fonction extérieure : $v(t) = e^t$, donc $v'(t) = e^t$.

$$f'(x) = u'(x) \times e^{u(x)} = (2x - 3) e^{x^2-3x}.$$

Les pièges.

- ▶ **Piège 1.** Oublier $u'(x)$: écrire $(e^u)' = e^u$ au lieu de $u' e^u$.
- ▶ **Piège 2.** Pour $(\ln u)'$, oublier le signe de u : vérifier que $u > 0$ sur l'intervalle d'étude.

Vérifier son résultat. Vérifier sur un cas simple : si $u(x) = 2x$, alors $(e^{2x})' = 2e^{2x}$ (et non e^{2x}).

S'entraîner. (renvois exercices M1)

Exercice 1 ♦

On applique recherche_dichotomique du cours au tableau trié $[2, 5, 8, 12, 16, 23, 38, 45, 56, 72]$ (indices 0 à 9), pour rechercher la valeur 23.

1. Pour chaque itération de la boucle while, donner les valeurs de borne_inf, borne_sup, milieu et tableau_trie[milieu].
2. Combien d'itérations sont nécessaires pour trouver la valeur 23 ? Quel indice est renvoyé ?
3. Calculer le variant de boucle $V = \text{borne_sup} - \text{borne_inf}$ au début de chaque itération. Vérifie qu'il décroît strictement.

Exercice 2 ♦♦

Un élève applique recherche_dichotomique du cours au tableau $[23, 5, 42, 8, 16, 3, 99]$, qui n'est pas trié, pour chercher la valeur 23.

1. Que renvoie recherche_dichotomique($[23, 5, 42, 8, 16, 3, 99]$, 23) ? La valeur 23 est-elle pourtant présente dans le tableau ?
2. Dérouler les deux premières itérations pour expliquer précisément pourquoi l'algorithme élimine à tort la portion du tableau contenant la valeur 23.
3. Quelle condition, énoncée dans le cours, ce tableau ne respecte-t-il pas ?

Exercice 3 ♦♦

1. Appliquer la méthode gloutonne du cours pour rendre 63 euros avec le système de pièces $\{50, 20, 10, 5, 2, 1\}$. Détailler le choix de chaque pièce.
2. Vérifier que la somme des pièces rendues vaut bien 63.

3. Ce système de pièces (euros) garantit-il que la méthode gloutonne donne toujours le nombre minimal de pièces ? Est-ce le cas pour **tout** système de pièces ? Justifier en citant le contre-exemple du cours.

Exercice 4

On dispose d'une base de fruits déjà classés, chacun décrit par sa masse (en g) et sa longueur (en cm) :

```
1 fruits = [
2     (150, 7, "Pomme"), (170, 7.5, "Pomme"), (140, 6.5, "Pomme"),
3     (120, 20, "Banane"), (130, 22, "Banane"), (110, 18, "Banane"),
4 ]
```

1. En utilisant `k_plus_proches_voisins` du cours, prédire la classe d'un nouveau fruit de masse 145 g et de longueur 8 cm, avec $k = 3$.
2. Cette prédiction te semble-t-elle raisonnable au vu des données (comparer masse et longueur du nouveau fruit à celles des deux groupes) ?
3. Que se passerait-il, à ton avis, si on utilisait $k = 6$ (tous les fruits de la base) au lieu de $k = 3$? La classe obtenue serait-elle nécessairement pertinente ?

Exercice 5

Le problème du **sac à dos** : on dispose d'objets, chacun avec un poids et une valeur, et d'un sac de capacité limitée. On veut maximiser la valeur totale des objets emportés sans dépasser la capacité. Une approche gloutonne classique choisit les objets par **rapport valeur/poids décroissant**.

```
1 def sac_a_dos_glouton(objets, capacite):
2     """objets : liste de (nom, poids, valeur)."""
3     objets_tries = sorted(objets, key=lambda o: o[2] / o[1], reverse=True)
4     charge = 0
5     valeur_totale = 0
6     choisis = []
7     for nom, poids, valeur in objets_tries:
8         if charge + poids <= capacite:
9             choisis.append(nom)
10            charge += poids
11            valeur_totale += valeur
12     return choisis, valeur_totale
```

On dispose des objets X (poids 6, valeur 30), Y (poids 5, valeur 20) et Z (poids 5, valeur 20), pour un sac de capacité 10.

1. Calculer le rapport valeur/poids de chaque objet, et en déduire l'ordre dans lequel l'algorithme les examine.
2. Exécuter `sac_a_dos_glouton` sur ces données. Quels objets sont choisis, et quelle est la valeur totale obtenue ?
3. Trouver, en cherchant à la main, une combinaison d'objets de poids total ≤ 10 offrant une valeur **strictement supérieure**. L'algorithme glouton est-il optimal dans ce cas ?

Exercice 6

Soit f définie sur $]0, +\infty[$ par $f(x) = \ln\left(\frac{1}{x}\right)$.

1. Simplifier l'expression de f .
2. Calculer $f'(x)$ de deux manières différentes.

Exercice 7 ♦♦

Soit f définie sur $\mathbb{R}$ par $f(x) = e^{x^2-2}$.

1. Calculer $f'(x)$.
2. Déterminer l'équation de la tangente à la courbe de f au point d'abscisse 0.
3. La courbe est-elle au-dessus ou au-dessous de cette tangente au voisinage de 0 ?

Exercice 8 ♦♦

Calculer la dérivée de $f(x) = (x^3 - 1)^4 e^{2x}$ sur $\mathbb{R}$.

Exercice 9 ♦♦♦

On considère la fonction f définie sur $[0, 2]$ par $f(x) = x\sqrt{4-x^2}$.

1. Justifier que f est bien définie sur $[0, 2]$.
2. Calculer $f'(x)$ et déterminer les variations de f .
3. En déduire le maximum de f sur $[0, 2]$ et la valeur de x pour laquelle il est atteint.

Exercice 10 ♦

Étudier les variations de $f(x) = x^3 - 3x^2 + 2$ sur $\mathbb{R}$.

Exercice 11 ♦

Étudier les variations de $f(x) = e^x - x$ sur $\mathbb{R}$ et déterminer ses limites en $\pm\infty$.

Exercice 12 ♦

Étudier les variations de $f(x) = x \ln x$ sur $]0, +\infty[$, limites comprises.

Exercice 13 ♦

Étudier les variations de $f(x) = \frac{x^2 + 1}{x}$ sur $\mathbb{R}^*$.

Exercice 14 ♦♦

Soit $f(x) = x^2 e^{-x}$ définie sur $\mathbb{R}$.

1. Calculer $f'(x)$ et étudier son signe.
2. Déterminer les limites de f en $\pm\infty$.
3. Dresser le tableau de variations de f .

Exercice 15 ♦♦

Soit $f(x) = \frac{\ln x}{x}$ sur $]0, +\infty[$.

1. Calculer $f'(x)$ et étudier son signe.
2. Déterminer les limites de f en 0^+ et en $+\infty$.
3. Dresser le tableau de variations.

Exercice 16 ♦♦

Soit $f(x) = \frac{e^x}{x+1}$ définie sur $\mathbb{R} \setminus \{-1\}$.

1. Calculer $f'(x)$ et étudier son signe.
2. Déterminer les limites aux bornes du domaine de définition.
3. Dresser le tableau de variations complet.

Exercice 17 ♦♦

Soit $f(x) = x^3 - 6x^2 + 9x + 1$.

1. Dresser le tableau de variations de f .
2. Déterminer l'équation de la tangente à la courbe au point d'abscisse 2.
3. La courbe traverse-t-elle cette tangente en $x = 2$?

Exercice 18 ♦♦♦

On découpe aux quatre coins d'une plaque carrée de côté 20 cm des carrés de côté x cm, puis on replie pour former une boîte ouverte.

1. Exprimer le volume $V(x)$ de la boîte en fonction de x .
2. Pour quelle valeur de x le volume est-il maximal ? Quel est ce volume maximal ?

Exercice 19 ♦♦♦

Le coût de production de x unités est modélisé par $C(x) = \frac{x^3}{3} - 5x^2 + 30x$.

1. Calculer le coût marginal $C'(x)$. Son signe permet-il d'identifier un coût minimum ?
2. Calculer le coût moyen $\frac{C(x)}{x}$ et déterminer s'il admet un minimum.
3. Comparer les deux résultats.

Exercice 20 ♦

Démontrer que pour tout $x \in \mathbb{R}$, $e^x \geq 1 + x$.

Exercice 21 ♦

Démontrer que pour tout $x > 0$, $\ln x \leq x - 1$.

Exercice 22 ♦

Soit $f = e^x$. On rappelle que f est convexe sur $\mathbb{R}$.

1. Énoncer l'inégalité de Jensen pour f aux points x et y avec $t = \frac{1}{2}$.
2. Vérifier cette inégalité pour $x = 0$ et $y = 2$.

Exercice 23 ♦♦

Démontrer que pour tous réels x et y :

$$e^x + e^y \geq 2e^{\frac{x+y}{2}}.$$

Exercice 24 ♦♦

Démontrer que pour tous réels x et y :

$$x^2 + y^2 \geq \frac{(x+y)^2}{2}.$$

QCM d'auto-évaluation — Dichotomie, gloutons, k plus proches voisins

Pour chaque question, une seule réponse est exacte. Entourez la lettre correspondante.

C1 — Dichotomie

1. [Q1] La recherche dichotomique nécessite que le tableau soit :
 - A. de taille paire.
 - B. trié.
 - C. composé uniquement d'entiers.
 - D. de taille inférieure à 100.
2. [Q2] Le variant de boucle $V = \text{borne_sup} - \text{borne_inf}$ de la recherche dichotomique sert à prouver :
 - A. que l'algorithme trouve toujours la valeur cherchée.
 - B. que la boucle termine en un nombre fini d'itérations.
 - C. que le tableau est trié.
 - D. que le coût est linéaire.

C2 — Algorithmes gloutons

3. [Q3] Un algorithme glouton fait, à chaque étape, le choix :
 - A. globalement optimal, garanti par calcul exhaustif.
 - B. localement optimal, sans revenir en arrière.
 - C. aléatoire.
 - D. identique au choix précédent.
4. [Q4] Avec le système de pièces $\{4, 3, 1\}$, rendre 6 de façon gloutonne donne :
 - A. la solution optimale à 2 pièces.
 - B. une solution à 3 pièces, non optimale.
 - C. une erreur, car 6 n'est pas atteignable.
 - D. une solution à 6 pièces.

C3 — k plus proches voisins

5. [Q5] L'algorithme des k plus proches voisins prédit la classe d'un nouvel élément à partir de :
 - A. la classe majoritaire de ses k plus proches voisins déjà classés.
 - B. la classe du point le plus éloigné.
 - C. une valeur aléatoire.
 - D. la moyenne de toutes les classes existantes.
6. [Q6] Choisir un k égal à la taille totale du jeu de données (tous les points) :
 - A. est toujours la meilleure stratégie.
 - B. fait perdre le sens de la méthode, basée sur la proximité locale.
 - C. accélère toujours le calcul.
 - D. est obligatoire pour que l'algorithme fonctionne.

Corrigé

1. Itération 1 : $\text{inf}=0$, $\text{sup}=7$, $\text{milieu}=3$ ($t[3]=21 < 27$). Itération 2 : $\text{inf}=4$, $\text{sup}=7$, $\text{milieu}=5$ ($t[5]=33 > 27$). Itération 3 : $\text{inf}=4$, $\text{sup}=4$, $\text{milieu}=4$ ($t[4]=27$, trouvé, indice 4).

2. V vaut successivement $7 - 0 = 7$, $7 - 4 = 3$, $4 - 4 = 0$: la suite 7, 3, 0 est strictement décroissante.
3. V est un entier positif ou nul qui décroît strictement : une telle suite est nécessairement finie (elle ne peut pas décroître indéfiniment en restant positive), donc la boucle termine forcément après un nombre fini d'itérations.

Corrigé

1. Pour 47 : 20 (reste 27), 20 (reste 7), 5 (reste 2), 2 (reste 0). Rendu : [20, 20, 5, 2].
2. `k_plus_proches_voisins(animaux, (5.5, 31), 3)` renvoie "Chat" : cet animal est proche en masse et en taille du groupe des chats.
3. Non, la méthode gloutonne n'est pas optimale pour **tout** système de pièces (seulement pour des systèmes ayant certaines propriétés, comme le système en euros) : avec {4, 3, 1} par exemple, rendre 6 de façon gloutonne utilise 3 pièces (4 + 1 + 1) alors que 3 + 3 n'en utilise que 2.

Évaluation A — Dichotomie, gloutons, k-NN

Durée : 45 minutes. Ordinateur avec interpréteur Python autorisé.

Barème indicatif : Ex. 1 : 10 pts (dichotomie, variant) ; Ex. 2 : 10 pts (glouton, k-NN)

Exercice 1 — Recherche dichotomique

(10 points)

On applique `recherche_dichotomique` du cours au tableau trié [3, 9, 14, 21, 27, 33, 40, 48] pour chercher la valeur 27.

1. (5 pts) Dérouler l'algorithme : borne_inf, borne_sup, milieu à chaque itération, jusqu'à trouver la valeur.
2. (3 pts) Calculer le variant $V = \text{borne_sup} - \text{borne_inf}$ à chaque itération et vérifier qu'il décroît strictement.
3. (2 pts) Pourquoi cette preuve de décroissance stricte suffit-elle à garantir la terminaison de l'algorithme ?

Exercice 2 — Glouton et k-NN

(10 points)

1. (4 pts) Utiliser `rendu_glouton` du cours pour rendre 47 euros avec le système {20, 10, 5, 2, 1}. Détailler le choix de chaque pièce.
2. (3 pts) On dispose de la base d'animaux (masse en kg, taille en cm, espèce) :

```
1 animaux = [
2     (5, 30, "Chat"), (6, 32, "Chat"), (4, 28, "Chat"),
3     (30, 80, "Chien"), (32, 85, "Chien"), (28, 78, "Chien"),
4 ]
```

Prédire l'espèce d'un animal de masse 5,5 kg et de taille 31 cm, avec $k = 3$.

3. (3 pts) La méthode gloutonne de rendu de monnaie donne-t-elle toujours le nombre minimal de pièces, quel que soit le système de pièces ? Justifier brièvement.

Corrigé

1. Itération 1 : inf=0, sup=7, milieu=3 ($t[3]=17 < 24$). Itération 2 : inf=4, sup=7, milieu=5 ($t[5]=31 > 24$). Itération 3 : inf=4, sup=4, milieu=4 ($t[4]=24$, trouvé, indice 4).
2. V vaut successivement $7 - 0 = 7$, $7 - 4 = 3$, $4 - 4 = 0$: la suite 7, 3, 0 est strictement décroissante.
3. V est un entier positif ou nul qui décroît strictement à chaque itération : une telle suite ne peut être infinie, donc la boucle se termine forcément.

Corrigé

1. Pour 38 : 20 (reste 18), 10 (reste 8), 5 (reste 3), 2 (reste 1), 1 (reste 0). Rendu : [20, 10, 5, 2, 1].
2. `k_plus_proches_voisins(fleurs, (5.1, 1.5), 3)` renvoie "Iris" : cette fleur est proche en longueur et largeur de pétale du groupe des iris.
3. Avec $k = 1$, la prédiction ne repose que sur le **seul** voisin le plus proche : si ce point unique est mal classé dans la base (une erreur de saisie, un cas atypique), la prédiction sera directement faussée, sans qu'aucun autre voisin ne vienne « corriger » cette erreur par un vote majoritaire.

Évaluation B — Dichotomie, gloutons, k-NN

Durée : 45 minutes. Ordinateur avec interpréteur Python autorisé.

Barème indicatif : Ex. 1 : 10 pts (dichotomie, variant); Ex. 2 : 10 pts (glouton, k-NN)

Exercice 1 — Recherche dichotomique

(10 points)

On applique `recherche_dichotomique` du cours au tableau trié [2, 6, 11, 17, 24, 31, 39, 50] pour chercher la valeur 24.

1. (5 pts) Dérouler l'algorithme : `borne_inf`, `borne_sup`, `milieu` à chaque itération, jusqu'à trouver la valeur.
2. (3 pts) Calculer le variant $V = \text{borne_sup} - \text{borne_inf}$ à chaque itération et vérifier qu'il décroît strictement.
3. (2 pts) Pourquoi cette preuve de décroissance stricte suffit-elle à garantir la terminaison de l'algorithme ?

Exercice 2 — Glouton et k-NN

(10 points)

1. (4 pts) Utiliser `rendu_glouton` du cours pour rendre 38 euros avec le système {20, 10, 5, 2, 1}. Détailler le choix de chaque pièce.
2. (3 pts) On dispose de la base de fleurs (longueur de pétale en cm, largeur de pétale en cm, espèce) :

```
1 fleurs = [
2     (5, 1.5, "Iris"), (5.2, 1.6, "Iris"), (4.8, 1.4, "Iris"),
3     (6.5, 2.2, "Rose"), (6.8, 2.3, "Rose"), (6.3, 2.1, "Rose"),
4 ]
```

Prédire l'espèce d'une fleur de longueur 5,1 cm et de largeur 1,5 cm, avec $k = 3$.

3. (3 pts) Un k trop petit (par exemple $k = 1$) présente quel risque, en présence de données bruitées (un point mal classé dans la base) ?

Rappel – Derivation : dérivées de référence et règles

Dérivées usuelles : $(x^n)' = nx^{n-1}$, $(e^x)' = e^x$, $(\ln x)' = \frac{1}{x}$, $(\sqrt{x})' = \frac{1}{2\sqrt{x}}$.

Règles : $(u + v)' = u' + v'$, $(uv)' = u'v + uv'$, $\left(\frac{u}{v}\right)' = \frac{u'v - uv'}{v^2}$.

Exercice 25 ♦

Calculer les dérivées des fonctions suivantes :

1. $f(x) = x^4 - 3x^2 + 5$

2. $g(x) = \frac{e^x}{x}$ sur $\mathbb{R}^*$
3. $h(x) = x \ln x$ sur $]0, +\infty[$

Rappel – Derivation locale : nombre derive et tangente

Nombre derive : $f'(a) = \lim_{h \rightarrow 0} \frac{f(a+h) - f(a)}{h}$.

Tangente : equation $y = f(a) + f'(a)(x - a)$.

Exercice 26 ◆

Soit $f(x) = x^2 + 3x - 1$.

1. Calculer $f'(1)$ par la formule du nombre derive.
2. Donner l'equation de la tangente a la courbe en $x = 1$.
3. En quel point la tangente est-elle horizontale ?

Exercice 27 ◆

Un élève écrit la recherche dichotomique suivante, avec une erreur dans la mise à jour de `borne_inf` :

```
1 def recherche_dichotomique_bugue(tableau_trie, valeur):
2     borne_inf, borne_sup = 0, len(tableau_trie) - 1
3     while borne_inf ≤ borne_sup:
4         milieu = (borne_inf + borne_sup) // 2
5         if tableau_trie[milieu] == valeur:
6             return milieu
7         elif tableau_trie[milieu] < valeur:
8             borne_inf = milieu # au lieu de milieu + 1
9         else:
10            borne_sup = milieu - 1
11    return -1
```

1. Sur le tableau $[1, 3, 5, 7, 9, 11, 13]$, que se passe-t-il si on cherche la valeur 4 (absente) avec cette fonction bugée ? (on pourra limiter le nombre d'itérations testées à 20 pour éviter une exécution trop longue)
2. Calculer le variant $V = \text{borne_sup} - \text{borne_inf}$ à chaque itération. Décroît-il toujours strictement avec cette version bugée ?
3. Corriger le code pour que la terminaison soit à nouveau garantie.

Corrigé

1.

borne_inf	borne_sup	milieu	tableau_trie[milieu]
0	9	4	16
5	9	7	45
5	6	5	23

2. 3 itérations sont nécessaires. L'indice renvoyé est 5.

3. V vaut successivement $9 - 0 = 9$, $9 - 5 = 4$, $6 - 5 = 1$: la suite 9,4,1 est bien strictement décroissante.

Corrigé

1. La fonction renvoie -1 (« non trouvée »). Pourtant, 23 est bien présent, à l'indice 0 du tableau.

2. Itération 1 : borne_inf=0, borne_sup=6, milieu=3, tableau_trie[3]=8. Comme $8 < 23$, l'algorithme suppose (à tort, car le tableau n'est pas trié) que 23 ne peut se trouver que dans la moitié **droite** : il fixe borne_inf=4, éliminant définitivement les indices 0 à 3 — alors que l'indice 0 contient justement la valeur cherchée ! Itération 2 : borne_inf=4, borne_sup=6, milieu=5, tableau_trie[5]=3. Comme $3 < 23$, on élimine encore, borne_inf=6. La suite ne peut plus trouver 23, qui a déjà été éliminé à tort.

3. Ce tableau ne respecte pas la condition d'être **trié**, condition indispensable pour que l'élimination d'une moitié de la zone de recherche soit valide.

Corrigé

1. On choisit la plus grande pièce possible à chaque étape : 50 (reste 13), 10 (reste 3, car $20 > 13$), 2 (reste 1), 1 (reste 0). Rendu : [50, 10, 2, 1].

2. $50 + 10 + 2 + 1 = 63$. ✓.

3. Oui, le système de pièces en euros garantit l'optimalité gloutonne (propriété du cours). Ce **n'est pas** le cas pour tout système : avec {4, 3, 1}, rendre 6 de façon gloutonne donne $4 + 1 + 1$ (3 pièces), alors que $3 + 3$ (2 pièces) est meilleur.

Corrigé

1. k_plus_proches_voisins(fruits, (145, 8), 3) renvoie "Pomme".

2. Oui, cette prédiction est raisonnable : le nouveau fruit (145 g, 8 cm) est proche en masse et en longueur du groupe des pommes (140 à 170 g, 6,5 à 7,5 cm), et très différent du groupe des bananes (masses proches mais longueurs bien supérieures, 18 à 22 cm).

3. Avec $k = 6$, on utiliserait **tous** les fruits de la base, dont exactement 3 pommes et 3 bananes : c'est une situation d'**égalité** entre les deux classes, qui ne reflète plus la proximité réelle du nouveau point (les bananes, pourtant très éloignées, compteraient autant que les pommes, très proches). Le résultat d'une égalité dépend alors de détails d'implémentation non garantis, ce qui n'est pas satisfaisant : un k trop grand (ici, égal à la taille totale de la base) fait perdre tout le sens de la méthode, qui repose justement sur la proximité **locale**.

Corrigé

1. Rapports valeur/poids : $X : 30/6 = 5$; $Y : 20/5 = 4$; $Z : 20/5 = 4$. L'algorithme examine donc X en premier (rapport le plus élevé), puis Y et Z.

2. L'algorithme choisit d'abord X (poids $6 \leq 10$, charge = 6). Il ne peut ensuite prendre ni Y ni Z (poids 5 chacun, charge $6 + 5 = 11 > 10$). Résultat : ["X"], valeur totale 30.

3. En prenant Y et Z ensemble (poids $5 + 5 = 10 \leq 10$), on obtient une valeur de $20 + 20 = 40 > 30$. L'algorithme glouton n'est donc **pas optimal** dans ce cas : privilégier le meilleur rapport valeur/poids objet par objet ne garantit pas la meilleure combinaison globale, exactement comme pour le rendu de monnaie avec un système de pièces non standard.

Corrigé

Question 1. $f(x) = \ln\left(\frac{1}{x}\right) = -\ln(x)$ sur $]0, +\infty[$.

Question 2.

► **Méthode 1** (simplification) : $f(x) = -\ln(x)$, donc $f'(x) = -\frac{1}{x}$.

► **Méthode 2** (composition) : on pose $u(x) = \frac{1}{x}$, $u'(x) = -\frac{1}{x^2}$.

$$f'(x) = \frac{u'(x)}{u(x)} = \frac{-1/x^2}{1/x} = -\frac{1}{x}.$$

Les deux méthodes donnent bien $f'(x) = -\frac{1}{x}$.

Corrigé

Question 1. $u(x) = x^2 - 2$, $u'(x) = 2x$. Donc $f'(x) = 2x e^{x^2-2}$.

Question 2. $f(0) = e^{-2}$ et $f'(0) = 0$. La tangente en 0 est :

$$y = e^{-2}.$$

Question 3. Étudions $g(x) = f(x) - e^{-2} = e^{x^2-2} - e^{-2}$.

Au voisinage de 0, $x^2 - 2 \geq -2$ donc $e^{x^2-2} \geq e^{-2}$, soit $g(x) \geq 0$.

La courbe est **au-dessus** de la tangente au voisinage de 0 (et en fait partout).

Corrigé

On écrit $f = u \times v$ avec $u = (x^3 - 1)^4$ et $v = e^{2x}$.

$$u' = 4 \times 3x^2 \times (x^3 - 1)^3 = 12x^2(x^3 - 1)^3 \text{ et } v' = 2e^{2x}.$$

$$f'(x) = 12x^2(x^3 - 1)^3 e^{2x} + (x^3 - 1)^4 \times 2e^{2x}.$$

On factorise :

$$f'(x) = (x^3 - 1)^3 e^{2x} [12x^2 + 2(x^3 - 1)] = (x^3 - 1)^3 e^{2x} (2x^3 + 12x^2 - 2).$$

Corrigé

Question 1. Sur $[0, 2]$, on a $0 \leq x^2 \leq 4$, donc $4 - x^2 \geq 0$ et f est bien définie.

Question 2. $f = u \times v$ avec $u = x$ et $v = \sqrt{4 - x^2}$. $u' = 1$ et $v' = \frac{-2x}{2\sqrt{4 - x^2}} = \frac{-x}{\sqrt{4 - x^2}}$.

$$f'(x) = \sqrt{4 - x^2} + x \times \frac{-x}{\sqrt{4 - x^2}} = \frac{4 - x^2 - x^2}{\sqrt{4 - x^2}} = \frac{4 - 2x^2}{\sqrt{4 - x^2}}.$$

$$f'(x) = 0 \Leftrightarrow 4 - 2x^2 = 0 \Leftrightarrow x = \sqrt{2} \text{ (sur } [0, 2]).$$

f' est positive sur $[0, \sqrt{2}]$ et négative sur $[\sqrt{2}, 2]$: f est croissante puis décroissante.

Question 3. Le maximum est atteint en $x = \sqrt{2}$ et vaut :

$$f(\sqrt{2}) = \sqrt{2} \sqrt{4 - 2} = \sqrt{2} \times \sqrt{2} = \boxed{2}.$$

Corrigé

$$f'(x) = 3x^2 - 6x = 3x(x - 2).$$

$$f'(x) \text{ sur }]-\infty, 0[\cup]0, 2[\cup]2, +\infty[$$

$f(0) = 2$ (maximum local), $f(2) = -2$ (minimum local).

f est croissante sur $]-\infty, 0[$, décroissante sur $[0, 2]$, croissante sur $[2, +\infty[$.

Corrigé

$$f'(x) = e^x - 1.$$

$$f'(x) = 0 \Leftrightarrow x = 0 \text{ et } f'(x) > 0 \text{ pour } x > 0, f'(x) < 0 \text{ pour } x < 0.$$

f est décroissante sur $]-\infty, 0]$ puis croissante sur $[0, +\infty[$.

Le minimum est $f(0) = 1$.

Limites : $\lim_{x \rightarrow +\infty} f(x) = +\infty$ (croissance comparée) et $\lim_{x \rightarrow -\infty} f(x) = +\infty$ ($-x$ domine).

Corrigé

$$f'(x) = \ln x + x \times \frac{1}{x} = \ln x + 1.$$

$$f'(x) = 0 \iff x = e^{-1}. f' \text{ est négative sur }]0, e^{-1}] \text{ et positive sur } [e^{-1}, +\infty[.$$

Le minimum est $f(e^{-1}) = e^{-1} \times (-1) = -e^{-1}$.

$\lim_{x \rightarrow 0^+} x \ln x = 0$ (croissances comparées) et $\lim_{x \rightarrow +\infty} x \ln x = +\infty$.

Corrigé

$$f(x) = x + \frac{1}{x}, \text{ donc } f'(x) = 1 - \frac{1}{x^2} = \frac{x^2 - 1}{x^2}.$$

$$f'(x) = 0 \iff x^2 = 1 \iff x = \pm 1.$$

$$f' > 0 \text{ sur }]-\infty, -1[\cup]1, +\infty[\text{ et } f' < 0 \text{ sur }]-1, 0[\cup]0, 1[.$$

f admet un minimum local $f(1) = 2$ et un maximum local $f(-1) = -2$.

Corrigé

Question 1. $f'(x) = 2xe^{-x} - x^2e^{-x} = x(2-x)e^{-x}$.

Comme $e^{-x} > 0$, le signe de f' est celui de $x(2-x)$: $f' > 0$ sur $[0, 2]$, $f' < 0$ sur $]-\infty, 0[\cup]2, +\infty[$.

Question 2. $\lim_{x \rightarrow +\infty} x^2 e^{-x} = 0$ (croissances comparées) et $\lim_{x \rightarrow -\infty} x^2 e^{-x} = +\infty$.

Question 3. f admet un minimum local $f(0) = 0$ et un maximum local $f(2) = 4e^{-2}$.

Corrigé

Question 1. $f'(x) = \frac{\frac{1}{x} \cdot x - \ln x \cdot 1}{x^2} = \frac{1 - \ln x}{x^2}$.

$$f'(x) = 0 \iff \ln x = 1 \iff x = e. f' > 0 \text{ sur }]0, e] \text{ et } f' < 0 \text{ sur } [e, +\infty[.$$

Question 2. $\lim_{x \rightarrow 0^+} \frac{\ln x}{x} = -\infty$ et $\lim_{x \rightarrow +\infty} \frac{\ln x}{x} = 0$ (croissances comparées).

Question 3. f admet un maximum $f(e) = \frac{1}{e}$.

Corrigé

Question 1. $f'(x) = \frac{e^x(x+1) - e^x}{(x+1)^2} = \frac{xe^x}{(x+1)^2}$.

$$f'(x) = 0 \iff x = 0. f' < 0 \text{ sur }]-\infty, -1[\cup]-1, 0[\text{ et } f' > 0 \text{ sur }]0, +\infty[.$$

Question 2. $\lim_{x \rightarrow +\infty} f = +\infty$, $\lim_{x \rightarrow -\infty} f = 0$ (croissances comparées), $\lim_{x \rightarrow -1^+} f = +\infty$.

Question 3. Minimum local $f(0) = 1$. La droite $x = -1$ est asymptote verticale.

Corrigé

Question 1. $f'(x) = 3x^2 - 12x + 9 = 3(x-1)(x-3)$.

$$f' > 0 \text{ sur }]-\infty, 1[\cup]3, +\infty[, f' < 0 \text{ sur } [1, 3].$$

Maximum local $f(1) = 5$, minimum local $f(3) = 1$.

Question 2. $f(2) = 3$ et $f'(2) = 3(1)(-1) = -3$. La tangente est $y = -3(x-2) + 3 = -3x + 9$.

Question 3. $f''(x) = 6x - 12$, donc $f''(2) = 0$: le point $x = 2$ est un **point d'inflexion**.

La courbe traverse sa tangente en $x = 2$ (changement de convexité).

Corrigé

Question 1. La base de la boîte est un carré de cote $(20 - 2x)$ et la hauteur est x .

$$V(x) = x(20 - 2x)^2 \quad \text{pour } x \in [0, 10].$$

Question 2. $V'(x) = (20 - 2x)^2 + x \times 2(20 - 2x)(-2) = (20 - 2x)[(20 - 2x) - 4x] = (20 - 2x)(20 - 6x)$.

$$V'(x) = 0 \iff x = 10 \text{ ou } x = \frac{10}{3} \text{ (dans } [0, 10]).$$

Le maximum est atteint en $x = \frac{10}{3}$ cm et vaut :

$$V\left(\frac{10}{3}\right) = \frac{10}{3} \times \left(\frac{40}{3}\right)^2 = \frac{16000}{27} \approx 593 \text{ cm}^3.$$

Corrigé

Question 1. $C'(x) = x^2 - 10x + 30$. Le discriminant vaut $100 - 120 = -20 < 0$.

Le coût marginal est strictement positif pour tout x : il n'y a pas de coût minimum (le coût augmente toujours).

Question 2. $\frac{C(x)}{x} = \frac{x^2}{3} - 5x + 30$. Sa dérivée est $\frac{2x}{3} - 5$, nulle en $x = \frac{15}{2}$.

Le coût moyen admet un minimum en $x = 7,5$: $\frac{C(7,5)}{7,5} = \frac{56,25}{3} - 37,5 + 30 = -25,625$.

Question 3. Le coût marginal croît tandis que le coût moyen décroît puis croît : l'optimum économique correspond au minimum du coût moyen.

Corrigé

Posons $g(x) = e^x - 1 - x$. On a $g'(x) = e^x - 1$ et $g''(x) = e^x > 0$.

La fonction g est donc **convexe** sur $\mathbb{R}$. Son minimum est atteint en $x = 0$ (car $g'(0) = 0$), et $g(0) = 0$.

Par convexité, $g(x) \geq g(0) = 0$ pour tout x , soit $e^x \geq 1 + x$.

Corrigé

Posons $g(x) = \ln x - x + 1$ sur $]0, +\infty[$.

$$g'(x) = \frac{1}{x} - 1 \text{ et } g''(x) = -\frac{1}{x^2} < 0 : g \text{ est concave.}$$

$$g'(x) = 0 \iff x = 1 \text{ et } g(1) = 0. \text{ Par concavité, } g(x) \leq g(1) = 0, \text{ soit } \ln x \leq x - 1.$$

Corrigé

Question 1. Par convexité de e^x et l'inégalité de Jensen :

$$e^{\frac{x+y}{2}} \leq \frac{e^x + e^y}{2}.$$

Question 2. Pour $x = 0$ et $y = 2$:

$$e^1 \approx 2,718 \quad \text{et} \quad \frac{1 + e^2}{2} \approx \frac{1 + 7,389}{2} \approx 4,195.$$

$$\text{On a bien } e < \frac{1 + e^2}{2}.$$

Corrigé

La fonction e^t est convexe sur $\mathbb{R}$. Par l'inégalité de Jensen avec $t = \frac{1}{2}$:

$$e^{\frac{x+y}{2}} \leq \frac{e^x + e^y}{2}.$$

En multipliant par 2 :

$$2e^{\frac{x+y}{2}} \leq e^x + e^y.$$

Corrigé

La fonction $t \mapsto t^2$ est convexe sur $\mathbb{R}$. Par Jensen avec $t = \frac{1}{2}$:

$$\left(\frac{x+y}{2}\right)^2 \leq \frac{x^2 + y^2}{2}.$$

En multipliant par 2 : $\frac{(x+y)^2}{2} \leq x^2 + y^2$.

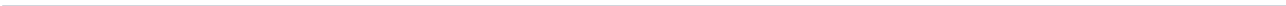
Verification directe : $x^2 + y^2 - \frac{(x+y)^2}{2} = \frac{(x-y)^2}{2} \geq 0$.

Corrigé

1. La fonction ne termine **jamais** : elle entre dans une boucle infinie (après 20 itérations, aucune réponse n'est obtenue).

2. Dès que `tableau_trie[milieu] < valeur` et que la zone de recherche se réduit à deux éléments consécutifs (`borne_sup = borne_inf + 1`), on a `milieu = borne_inf` : l'instruction `borne_inf = milieu` laisse alors `borne_inf` **inchangé**. Le variant $V = \text{borne_sup} - \text{borne_inf}$ cesse de décroître (il stagne à la même valeur indéfiniment) : la preuve de terminaison du cours ne s'applique plus, et effectivement, l'algorithme ne termine pas.

3. Il faut remettre `borne_inf = milieu + 1` (comme dans le cours), pour garantir que `borne_inf` augmente strictement à chaque fois que cette branche est empruntée, ce qui restaure la décroissance stricte du variant V .



7 Interactions entre l'homme et la machine sur le Web

7.1 Composants graphiques et événements

DÉFINITION D1

Un **composant graphique** d'une page Web est un élément avec lequel l'utilisateur peut interagir : bouton (`<button>`), champ de texte (`<input type="text">`), case à cocher (`<input type="checkbox">`), liste déroulante (`<select>`), lien (`<a>`)...

EXEMPLE Extrait de code HTML décrivant un bouton :

```
<button id="bouton_valider">Valider</button>
```

◆ **PROPRIÉTÉ Description vs comportement** Le code **HTML** décrit uniquement l'**apparence** et la **structure** d'un composant graphique (« il y a un bouton, avec ce texte »). Le **comportement** du composant (que se passe-t-il quand on clique dessus ?) est programmé séparément, en général en **JavaScript**.

DÉFINITION D2

Un **événement** est une action détectée par le navigateur (clic de souris, appui sur une touche, chargement de la page, soumission d'un formulaire...). Une **fonction associée à un événement** (un *gestionnaire d'événement*, ou *event handler*) est exécutée automatiquement lorsque cet événement se produit sur un composant donné.

EXEMPLE En JavaScript, on associe une fonction à l'événement « clic » d'un bouton :

```
document.getElementById("bouton_valider").addEventListener("click", function() {  
    alert("Formulaire valide !");  
});
```

◆ **PROPRIÉTÉ Simuler un gestionnaire d'événements** Le principe d'un gestionnaire d'événements peut se modéliser simplement : un **dictionnaire** associe à chaque identifiant de composant la fonction à exécuter lors du clic. « Déclencher un clic » revient alors à rechercher puis appeler la fonction associée.

CODE DE RÉFÉRENCE — Modélisation Python d'un gestionnaire d'événements

```

1 gestionnaires = {}
2
3 def ajouter_gestionnaire(id_composant, fonction):
4     """Associe 'fonction' a l'evenement clic du composant 'id_composant'."""
5     gestionnaires[id_composant] = fonction
6
7 def declencher_clic(id_composant):
8     """Simule un clic : execute la fonction associee, si elle existe."""
9     if id_composant in gestionnaires:
10         return gestionnaires[id_composant]()
11     return None
12
13 compteur = {"valeur": 0}
14 def incrementer():
15     compteur["valeur"] += 1
16     return compteur["valeur"]
17
18 ajouter_gestionnaire("bouton_plus", incrementer)
19 print(declencher_clic("bouton_plus")) # 1
20 print(declencher_clic("bouton_plus")) # 2

```

⚠ ERREUR FRÉQUENTE

Confondre la **description** HTML d'un composant (son apparence, son identifiant) avec son **comportement** programmé (ce qui se passe lors d'un événement). Le code HTML `<button id="bouton_plus">+</button>` ne fait **rien** par lui-même tant qu'aucune fonction n'est associée à l'un de ses événements.

» **Python À la machine.** Modifie le code Python ci-dessus pour ajouter un second gestionnaire, associé à "bouton_moins", qui décrémente compteur["valeur"]. Déclenche successivement "bouton_plus", "bouton_plus", "bouton_moins", et vérifie que compteur["valeur"] vaut 1.

7.2 Interaction client-serveur

DÉFINITION D3

Le **client** désigne le navigateur de l'utilisateur ; le **serveur** désigne la machine distante qui héberge le site Web. Une interaction Web typique suit un ordre précis : le client envoie une **requête**, le serveur la traite et renvoie une **réponse**, que le client affiche.

◆ **PROPRIÉTÉ Ce qui s'exécute où** Le code qui s'exécute côté serveur (souvent en Python, PHP, etc.) génère la page à partir de données (base de données, calculs). Ce code n'est normalement pas transmis dans la réponse HTTP destinée au navigateur. Ses sources peuvent toutefois être publiées ou divulguées par une autre voie ; cet accès éventuel ne signifie pas qu'elles figurent dans la réponse HTTP générée par leur exécution. Le code exécuté sur le **client** (en JavaScript, dans le navigateur) réagit aux interactions de l'utilisateur **sans nouvel échange avec le serveur** (par exemple, afficher un message d'alerte), sauf s'il déclenche explicitement une nouvelle requête.

CODE DE RÉFÉRENCE — Modélisation de l'ordre client-serveur

```

1  etapes = []
2
3  def navigateur_envoie_requete(url):
4      etapes.append(("client", "envoie requete vers " + url))
5
6  def serveur_recoit_et_traite(url):
7      etapes.append(("serveur", "traite la requete " + url))
8      return "page HTML generee"
9
10 def serveur_revoie_reponse(contenu):
11     etapes.append(("serveur", "renvoie la reponse"))
12
13 def navigateur_affiche(contenu):
14     etapes.append(("client", "affiche : " + contenu))
15
16 navigateur_envoie_requete("/accueil")
17 contenu = serveur_recoit_et_traite("/accueil")
18 serveur_revoie_reponse(contenu)
19 navigateur_affiche(contenu)
20
21 print([acteur for acteur, action in etapes])
22 # ['client', 'serveur', 'serveur', 'client']

```

DÉFINITION D4

Un **cookie** est une petite donnée que le serveur demande au navigateur de **mémoriser** côté client. À chaque requête ultérieure vers le même site, le navigateur **retransmet** automatiquement ce cookie au serveur (par exemple, pour reconnaître un utilisateur déjà connecté).

CODE DE RÉFÉRENCE — Modélisation d'un cookie mémorisé côté client

```

1  cookies_client = {}
2
3  def serveur_definit_cookie(nom, valeur):
4      cookies_client[nom] = valeur
5
6  def client_envoie_requete_avec_cookies(url):
7      """Le client retransmet automatiquement les cookies memorises."""
8      return {"url": url, "cookies": dict(cookies_client)}
9
10 serveur_definit_cookie("id_session", "abc123")
11 requete = client_envoie_requete_avec_cookies("/panier")
12 print(requete) # {'url': '/panier', 'cookies': {'id_session': 'abc123'}}

```

◆ **PROPRIÉTÉ Transmission chiffrée (HTTPS)** Sans chiffrement (protocole **HTTP**), les données échangées entre client et serveur circulent **en clair** : n'importe quel intermédiaire sur le réseau peut les lire. Le protocole **HTTPS** chiffre ces échanges, rendant leur contenu illisible pour un intermédiaire. On utilise HTTPS dès que des informations sensibles (mot de passe, numéro de carte bancaire) sont transmises.

ERREUR FRÉQUENTE

Croire qu'un site en HTTPS est à l'abri de **tout** risque. HTTPS protège la **transmission** des données (personne ne peut les lire en chemin), mais ne garantit rien sur ce que le serveur **fait** des données une fois reçues (stockage sécurisé ou non, revente à des tiers, etc.).

» **Python À la machine.** Reprends le code du gestionnaire de cookies. Ajoute une fonction `supprimer_cookie(nom)` qui retire un cookie de `cookies_client` (par exemple lors d'une déconnexion). Vérifie qu'après suppression, une nouvelle requête ne contient plus ce cookie.

7.3 Formulaires, requêtes GET et POST

DÉFINITION D5

Un **formulaire** Web (balise `<form>`) regroupe des composants de saisie (champs de texte, cases à cocher...) et un bouton d'envoi. Lorsque l'utilisateur **soumet** le formulaire, le navigateur envoie une requête au serveur, contenant les valeurs saisies sous forme de **paramètres**.

EXEMPLE Un formulaire de recherche simple :

```
<form action="/recherche" method="GET">
  <input type="text" name="q">
  <button type="submit">Rechercher</button>
</form>
```

◆ **PROPRIÉTÉ Requête GET** Avec la méthode GET, les paramètres du formulaire sont ajoutés **directement dans l'URL**, après un point d'interrogation, sous la forme `cle1=valeur1&cle2=valeur2`. Ils sont donc **visibles** (dans la barre d'adresse, dans l'historique du navigateur) et de **taille limitée**.

CODE DE RÉFÉRENCE — Construire une requête GET (module `urllib.parse`)

```
1 from urllib.parse import urlencode
2
3 parametres = {"q": "chaton", "page": 2}
4 url_get = "https://exemple.fr/recherche?" + urlencode(parametres)
5 print(url_get)
6 # https://exemple.fr/recherche?q=chaton&page=2
```

◆ **PROPRIÉTÉ Requête POST** Avec la méthode POST, les paramètres sont placés dans le **corps** de la requête, pas dans l'URL : ils ne sont **pas visibles** dans la barre d'adresse et ne sont pas limités en taille de la même façon.

EXEMPLE Le formulaire de connexion suivant utilise POST, pour ne pas exposer le mot de passe dans l'URL :

```
<form action="/connexion" method="POST">
```



```
<input type="text" name="identifiant">
<input type="password" name="mdp">
<button type="submit">Se connecter</button>
</form>
```

◆ **PROPRIÉTÉ Choisir entre GET et POST** On choisit GET pour des paramètres **non sensibles** et courts (recherche, filtres, pagination), en profitant du fait qu'une URL GET peut être partagée, mise en favori ou revisitée facilement. On choisit POST pour des données **confidentielles** (mots de passe) ou **volumineuses** (envoi d'un fichier), qui ne doivent pas apparaître dans l'URL ni dans l'historique du navigateur.

⚠ ERREUR FRÉQUENTE

Utiliser GET pour transmettre un mot de passe ou toute donnée confidentielle. Même sur une connexion chiffrée (HTTPS), l'URL complète (avec les paramètres GET) reste visible dans l'**historique du navigateur** et peut être enregistrée dans les journaux (*logs*) du serveur : ce n'est pas un canal adapté à des données sensibles.

» **Python À la machine.** Écris une fonction `construire_url(base, parametres)` qui utilise `urlencode` pour construire une URL de requête GET à partir d'un dictionnaire de paramètres quelconque. Teste-la avec `ville": "Tunis", "tri": "prix"` : `ville": "Tunis", "tri": "prix"`.

7.4 Composants graphiques et événements

DÉFINITION D1

Un **composant graphique** d'une page Web est un élément avec lequel l'utilisateur peut interagir : bouton (`<button>`), champ de texte (`<input type="text">`), case à cocher (`<input type="checkbox">`), liste déroulante (`<select>`), lien (`<a>`)...

EXEMPLE Extrait de code HTML décrivant un bouton :

```
<button id="bouton_valider">Valider</button>
```

◆ **PROPRIÉTÉ Description vs comportement** Le code **HTML** décrit uniquement l'**apparence** et la **structure** d'un composant graphique (« il y a un bouton, avec ce texte »). Le **comportement** du composant (que se passe-t-il quand on clique dessus ?) est programmé séparément, en général en **JavaScript**.

DÉFINITION D2

Un **événement** est une action détectée par le navigateur (clic de souris, appui sur une touche, chargement de la page, soumission d'un formulaire...). Une **fonction associée à un événement** (un *gestionnaire d'événement*, ou *event handler*) est exécutée automatiquement lorsque cet événement se produit sur un composant donné.

EXEMPLE En JavaScript, on associe une fonction à l'événement « clic » d'un bouton :

```
document.getElementById("bouton_valider").addEventListener("click", function() {
    alert("Formulaire valide !");
});
```

◆ **PROPRIÉTÉ Simuler un gestionnaire d'événements** Le principe d'un gestionnaire d'événements peut se modéliser simplement : un **dictionnaire** associe à chaque identifiant de composant la fonction à exécuter lors du clic. « Déclencher un clic » revient alors à rechercher puis appeler la fonction associée.

CODE DE RÉFÉRENCE — Modélisation Python d'un gestionnaire d'événements

```
1  gestionnaires = {}
2
3  def ajouter_gestionnaire(id_composant, fonction):
4      """Associe 'fonction' a l'evenement clic du composant 'id_composant'."""
5      gestionnaires[id_composant] = fonction
6
7  def declencher_clic(id_composant):
8      """Simule un clic : execute la fonction associee, si elle existe."""
9      if id_composant in gestionnaires:
10         return gestionnaires[id_composant]()
11     return None
12
13  compteur = {"valeur": 0}
14  def incrementer():
15      compteur["valeur"] += 1
16      return compteur["valeur"]
17
18  ajouter_gestionnaire("bouton_plus", incrementer)
19  print(declencher_clic("bouton_plus")) # 1
20  print(declencher_clic("bouton_plus")) # 2
```

⚠ ERREUR FRÉQUENTE

Confondre la **description** HTML d'un composant (son apparence, son identifiant) avec son **comportement** programmé (ce qui se passe lors d'un événement). Le code HTML `<button id="bouton_plus">+</button>` ne fait **rien** par lui-même tant qu'aucune fonction n'est associée à l'un de ses événements.

» **Python À la machine.** Modifie le code Python ci-dessus pour ajouter un second gestionnaire, associé à "bouton_moins", qui décrémente compteur["valeur"]. Déclenche successivement "bouton_plus", "bouton_plus", "bouton_moins", et vérifie que compteur["valeur"] vaut 1.

7.5 Interaction client-serveur

DÉFINITION D3

Le **client** désigne le navigateur de l'utilisateur ; le **serveur** désigne la machine distante qui héberge le site Web. Une interaction Web typique suit un ordre précis : le client envoie une **requête**, le serveur la traite et renvoie une **réponse**, que le client affiche.

◆ **PROPRIÉTÉ Ce qui s'exécute où** Le code qui s'exécute côté serveur (souvent en Python, PHP, etc.) génère la page à partir de données (base de données, calculs). Ce code n'est normalement pas transmis dans la réponse HTTP destinée au navigateur. Ses sources peuvent toutefois être publiées ou divulguées par une autre voie ; cet accès éventuel ne signifie pas qu'elles figurent dans la réponse HTTP générée par leur exécution. Le code exécuté sur le **client** (en JavaScript, dans le navigateur) réagit aux interactions de l'utilisateur **sans nouvel échange avec le serveur** (par exemple, afficher un message d'alerte), sauf s'il déclenche explicitement une nouvelle requête.

CODE DE RÉFÉRENCE — Modélisation de l'ordre client-serveur

```

1  etapes = []
2
3  def navigateur_envoie_requete(url):
4      etapes.append(("client", "envoie requete vers " + url))
5
6  def serveur_recoit_et_traite(url):
7      etapes.append(("serveur", "traite la requete " + url))
8      return "page HTML generee"
9
10 def serveur_renvoie_reponse(contenu):
11     etapes.append(("serveur", "renvoie la reponse"))
12
13 def navigateur_affiche(contenu):
14     etapes.append(("client", "affiche : " + contenu))
15
16 navigateur_envoie_requete("/accueil")
17 contenu = serveur_recoit_et_traite("/accueil")
18 serveur_renvoie_reponse(contenu)
19 navigateur_affiche(contenu)
20
21 print([acteur for acteur, action in etapes])
22 # ['client', 'serveur', 'serveur', 'client']

```

DÉFINITION D4

Un **cookie** est une petite donnée que le serveur demande au navigateur de **mémoriser** côté client. À chaque requête ultérieure vers le même site, le navigateur **retransmet** automatiquement ce cookie au serveur (par exemple, pour reconnaître un utilisateur déjà connecté).

CODE DE RÉFÉRENCE — Modélisation d'un cookie mémorisé côté client

```

1  cookies_client = {}

```

```

2
3 def serveur_definit_cookie(nom, valeur):
4     cookies_client[nom] = valeur
5
6 def client_envoie_requete_avec_cookies(url):
7     """Le client retransmet automatiquement les cookies memorises."""
8     return {"url": url, "cookies": dict(cookies_client)}
9
10 serveur_definit_cookie("id_session", "abc123")
11 requete = client_envoie_requete_avec_cookies("/panier")
12 print(requete) # {'url': '/panier', 'cookies': {'id_session': 'abc123'}}

```

◆ **PROPRIÉTÉ Transmission chiffrée (HTTPS)** Sans chiffrement (protocole **HTTP**), les données échangées entre client et serveur circulent **en clair** : n'importe quel intermédiaire sur le réseau peut les lire. Le protocole **HTTPS** chiffre ces échanges, rendant leur contenu illisible pour un intermédiaire. On utilise HTTPS dès que des informations sensibles (mot de passe, numéro de carte bancaire) sont transmises.

⚠ ERREUR FRÉQUENTE

Croire qu'un site en HTTPS est à l'abri de **tout** risque. HTTPS protège la **transmission** des données (personne ne peut les lire en chemin), mais ne garantit rien sur ce que le serveur **fait** des données une fois reçues (stockage sécurisé ou non, revente à des tiers, etc.).

» **Python À la machine.** Reprends le code du gestionnaire de cookies. Ajoute une fonction `supprimer_cookie(nom)` qui retire un cookie de `cookies_client` (par exemple lors d'une déconnexion). Vérifie qu'après suppression, une nouvelle requête ne contient plus ce cookie.

7.6 Formulaires, requêtes GET et POST

DÉFINITION D5

Un **formulaire** Web (balise `<form>`) regroupe des composants de saisie (champs de texte, cases à cocher...) et un bouton d'envoi. Lorsque l'utilisateur **soumet** le formulaire, le navigateur envoie une requête au serveur, contenant les valeurs saisies sous forme de **paramètres**.

EXEMPLE Un formulaire de recherche simple :

```

<form action="/recherche" method="GET">
  <input type="text" name="q">
  <button type="submit">Rechercher</button>
</form>

```

◆ **PROPRIÉTÉ Requête GET** Avec la méthode GET, les paramètres du formulaire sont ajoutés **directement dans l'URL**, après un point d'interrogation, sous la forme `cle1=valeur1&cle2=valeur2`. Ils sont donc **visibles** (dans la barre d'adresse, dans l'historique du navigateur) et de **taille limitée**.

CODE DE RÉFÉRENCE — Construire une requête GET (module urllib.parse)

```

1 from urllib.parse import urlencode
2
3 parametres = {"q": "chaton", "page": 2}
4 url_get = "https://exemple.fr/recherche?" + urlencode(parametres)
5 print(url_get)
6 # https://exemple.fr/recherche?q=chaton&page=2

```

◆ **PROPRIÉTÉ Requête POST** Avec la méthode POST, les paramètres sont placés dans le **corps** de la requête, pas dans l'URL : ils ne sont **pas visibles** dans la barre d'adresse et ne sont pas limités en taille de la même façon.

EXEMPLE Le formulaire de connexion suivant utilise POST, pour ne pas exposer le mot de passe dans l'URL :

```

<form action="/connexion" method="POST">
  <input type="text" name="identifiant">
  <input type="password" name="mdp">
  <button type="submit">Se connecter</button>
</form>

```

◆ **PROPRIÉTÉ Choisir entre GET et POST** On choisit GET pour des paramètres **non sensibles** et courts (recherche, filtres, pagination), en profitant du fait qu'une URL GET peut être partagée, mise en favori ou revisitée facilement. On choisit POST pour des données **confidentielles** (mots de passe) ou **volumineuses** (envoi d'un fichier), qui ne doivent pas apparaître dans l'URL ni dans l'historique du navigateur.

⚠ ERREUR FRÉQUENTE

Utiliser GET pour transmettre un mot de passe ou toute donnée confidentielle. Même sur une connexion chiffrée (HTTPS), l'URL complète (avec les paramètres GET) reste visible dans **l'historique du navigateur** et peut être enregistrée dans les journaux (*logs*) du serveur : ce n'est pas un canal adapté à des données sensibles.

» **Python À la machine.** Écris une fonction `construire_url(base, parametres)` qui utilise `urlencode` pour construire une URL de requête GET à partir d'un dictionnaire de paramètres quelconque. Teste-la avec `ville: "Tunis", "tri": "prix"` : `ville: "Tunis", "tri": "prix"`.

7.7 Composants graphiques et événements

DÉFINITION D1

Un **composant graphique** d'une page Web est un élément avec lequel l'utilisateur peut interagir : bouton (`<button>`), champ de texte (`<input type="text">`), case à cocher (`<input type="checkbox">`), liste déroulante (`<select>`), lien (`<a>`)...

EXEMPLE Extrait de code HTML décrivant un bouton :

```
<button id="bouton_valider">Valider</button>
```

◆ **PROPRIÉTÉ Description vs comportement** Le code **HTML** décrit uniquement l'**apparence** et la **structure** d'un composant graphique (« il y a un bouton, avec ce texte »). Le **comportement** du composant (que se passe-t-il quand on clique dessus ?) est programmé séparément, en général en **JavaScript**.

DÉFINITION D2

Un **événement** est une action détectée par le navigateur (clic de souris, appui sur une touche, chargement de la page, soumission d'un formulaire...). Une **fonction associée à un événement** (un *gestionnaire d'événement*, ou *event handler*) est exécutée automatiquement lorsque cet événement se produit sur un composant donné.

EXEMPLE En JavaScript, on associe une fonction à l'événement « clic » d'un bouton :

```
document.getElementById("bouton_valider").addEventListener("click", function() {
    alert("Formulaire valide !");
});
```

◆ **PROPRIÉTÉ Simuler un gestionnaire d'événements** Le principe d'un gestionnaire d'événements peut se modéliser simplement : un **dictionnaire** associe à chaque identifiant de composant la fonction à exécuter lors du clic. « Déclencher un clic » revient alors à rechercher puis appeler la fonction associée.

CODE DE RÉFÉRENCE — Modélisation Python d'un gestionnaire d'événements

```
1 gestionnaires = {}
2
3 def ajouter_gestionnaire(id_composant, fonction):
4     """Associe 'fonction' à l'événement clic du composant 'id_composant'."""
5     gestionnaires[id_composant] = fonction
6
7 def declencher_clic(id_composant):
8     """Simule un clic : exécute la fonction associée, si elle existe."""
9     if id_composant in gestionnaires:
10         return gestionnaires[id_composant]()
11     return None
12
13 compteur = {"valeur": 0}
14 def incrementer():
15     compteur["valeur"] += 1
16     return compteur["valeur"]
17
18 ajouter_gestionnaire("bouton_plus", incrementer)
19 print(declencher_clic("bouton_plus")) # 1
20 print(declencher_clic("bouton_plus")) # 2
```

ERREUR FRÉQUENTE

Confondre la **description** HTML d'un composant (son apparence, son identifiant) avec son **comportement** programmé (ce qui se passe lors d'un événement). Le code HTML `<button id="bouton_plus">+</button>` ne fait **rien** par lui-même tant qu'aucune fonction n'est associée à l'un de ses événements.

» **Python À la machine.** Modifie le code Python ci-dessus pour ajouter un second gestionnaire, associé à "bouton_moins", qui décrémente `compteur["valeur"]`. Déclenche successivement "bouton_plus", "bouton_plus", "bouton_moins", et vérifie que `compteur["valeur"]` vaut 1.

7.8 Interaction client-serveur

DÉFINITION D3

Le **client** désigne le navigateur de l'utilisateur ; le **serveur** désigne la machine distante qui héberge le site Web. Une interaction Web typique suit un ordre précis : le client envoie une **requête**, le serveur la traite et renvoie une **réponse**, que le client affiche.

◆ **PROPRIÉTÉ Ce qui s'exécute où** Le code qui s'exécute côté serveur (souvent en Python, PHP, etc.) génère la page à partir de données (base de données, calculs). Ce code n'est normalement pas transmis dans la réponse HTTP destinée au navigateur. Ses sources peuvent toutefois être publiées ou divulguées par une autre voie ; cet accès éventuel ne signifie pas qu'elles figurent dans la réponse HTTP générée par leur exécution. Le code exécuté sur le **client** (en JavaScript, dans le navigateur) réagit aux interactions de l'utilisateur **sans nouvel échange avec le serveur** (par exemple, afficher un message d'alerte), sauf s'il déclenche explicitement une nouvelle requête.

CODE DE RÉFÉRENCE — Modélisation de l'ordre client-serveur

```

1  etapes = []
2
3  def navigateur_envoie_requete(url):
4      etapes.append(("client", "envoie requete vers " + url))
5
6  def serveur_recoit_et_traite(url):
7      etapes.append(("serveur", "traite la requete " + url))
8      return "page HTML generee"
9
10 def serveur_renvoie_reponse(contenu):
11     etapes.append(("serveur", "renvoie la reponse"))
12
13 def navigateur_affiche(contenu):
14     etapes.append(("client", "affiche : " + contenu))
15
16 navigateur_envoie_requete("/accueil")
17 contenu = serveur_recoit_et_traite("/accueil")
18 serveur_renvoie_reponse(contenu)
19 navigateur_affiche(contenu)
20
21 print([acteur for acteur, action in etapes])

```

```
22 # ['client', 'serveur', 'serveur', 'client']
```

DÉFINITION D4

Un **cookie** est une petite donnée que le serveur demande au navigateur de **mémoriser** côté client. À chaque requête ultérieure vers le même site, le navigateur **retransmet** automatiquement ce cookie au serveur (par exemple, pour reconnaître un utilisateur déjà connecté).

CODE DE RÉFÉRENCE — Modélisation d'un cookie mémorisé côté client

```
1 cookies_client = {}
2
3 def serveur_definit_cookie(nom, valeur):
4     cookies_client[nom] = valeur
5
6 def client_envoie_requete_avec_cookies(url):
7     """Le client retransmet automatiquement les cookies memorises."""
8     return {"url": url, "cookies": dict(cookies_client)}
9
10 serveur_definit_cookie("id_session", "abc123")
11 requete = client_envoie_requete_avec_cookies("/panier")
12 print(requete) # {'url': '/panier', 'cookies': {'id_session': 'abc123'}}
```

◆ **PROPRIÉTÉ Transmission chiffrée (HTTPS)** Sans chiffrement (protocole **HTTP**), les données échangées entre client et serveur circulent **en clair** : n'importe quel intermédiaire sur le réseau peut les lire. Le protocole **HTTPS** chiffre ces échanges, rendant leur contenu illisible pour un intermédiaire. On utilise HTTPS dès que des informations sensibles (mot de passe, numéro de carte bancaire) sont transmises.

⚠ ERREUR FRÉQUENTE

Croire qu'un site en HTTPS est à l'abri de **tout** risque. HTTPS protège la **transmission** des données (personne ne peut les lire en chemin), mais ne garantit rien sur ce que le serveur **fait** des données une fois reçues (stockage sécurisé ou non, revente à des tiers, etc.).

» **Python À la machine.** Reprends le code du gestionnaire de cookies. Ajoute une fonction supprimer `_cookie(nom)` qui retire un cookie de `cookies_client` (par exemple lors d'une déconnexion). Vérifie qu'après suppression, une nouvelle requête ne contient plus ce cookie.

7.9 Formulaires, requêtes GET et POST

DÉFINITION D5

Un **formulaire** Web (balise `<form>`) regroupe des composants de saisie (champs de texte, cases à cocher...) et un bouton d'envoi. Lorsque l'utilisateur **soumet** le formulaire, le navigateur envoie une requête au serveur, contenant les valeurs saisies sous forme de **paramètres**.

EXEMPLE Un formulaire de recherche simple :

```
<form action="/recherche" method="GET">
  <input type="text" name="q">
  <button type="submit">Rechercher</button>
</form>
```

◆ **PROPRIÉTÉ Requête GET** Avec la méthode GET, les paramètres du formulaire sont ajoutés **directement dans l'URL**, après un point d'interrogation, sous la forme `cle1=valeur1&cle2=valeur2`. Ils sont donc **visibles** (dans la barre d'adresse, dans l'historique du navigateur) et de **taille limitée**.

CODE DE RÉFÉRENCE — Construire une requête GET (module `urllib.parse`)

```
1 from urllib.parse import urlencode
2
3 parametres = {"q": "chaton", "page": 2}
4 url_get = "https://exemple.fr/recherche?" + urlencode(parametres)
5 print(url_get)
6 # https://exemple.fr/recherche?q=chaton&page=2
```

◆ **PROPRIÉTÉ Requête POST** Avec la méthode POST, les paramètres sont placés dans le **corps** de la requête, pas dans l'URL : ils ne sont **pas visibles** dans la barre d'adresse et ne sont pas limités en taille de la même façon.

EXEMPLE Le formulaire de connexion suivant utilise POST, pour ne pas exposer le mot de passe dans l'URL :

```
<form action="/connexion" method="POST">
  <input type="text" name="identifiant">
  <input type="password" name="mdp">
  <button type="submit">Se connecter</button>
</form>
```

◆ **PROPRIÉTÉ Choisir entre GET et POST** On choisit GET pour des paramètres **non sensibles** et courts (recherche, filtres, pagination), en profitant du fait qu'une URL GET peut être partagée, mise en favori ou revisitée facilement. On choisit POST pour des données **confidentielles** (mots de passe) ou **volumineuses** (envoi d'un fichier), qui ne doivent pas apparaître dans l'URL ni dans l'historique du navigateur.

⚠ ERREUR FRÉQUENTE

Utiliser GET pour transmettre un mot de passe ou toute donnée confidentielle. Même sur une connexion chiffrée (HTTPS), l'URL complète (avec les paramètres GET) reste visible dans **l'historique du navigateur** et peut être enregistrée dans les journaux (*logs*) du serveur : ce n'est pas un canal adapté à des données sensibles.

» **Python À la machine.** Écris une fonction `construire_url(base, parametres)` qui utilise `urlencode` pour construire une URL de requête GET à partir d'un dictionnaire de paramètres quelconque. Teste-la avec `ville: "Tunis", "tri": "prix"` `ville: "Tunis", "tri": "prix"`.

7.10 Composants graphiques et événements

DÉFINITION D1

Un **composant graphique** d'une page Web est un élément avec lequel l'utilisateur peut interagir : bouton (`<button>`), champ de texte (`<input type="text">`), case à cocher (`<input type="checkbox">`), liste déroulante (`<select>`), lien (`<a>`)...

EXEMPLE Extrait de code HTML décrivant un bouton :

```
<button id="bouton_valider">Valider</button>
```

◆ **PROPRIÉTÉ Description vs comportement** Le code **HTML** décrit uniquement l'**apparence** et la **structure** d'un composant graphique (« il y a un bouton, avec ce texte »). Le **comportement** du composant (que se passe-t-il quand on clique dessus ?) est programmé séparément, en général en **JavaScript**.

DÉFINITION D2

Un **événement** est une action détectée par le navigateur (clic de souris, appui sur une touche, chargement de la page, soumission d'un formulaire...). Une **fonction associée à un événement** (un *gestionnaire d'événement*, ou *event handler*) est exécutée automatiquement lorsque cet événement se produit sur un composant donné.

EXEMPLE En JavaScript, on associe une fonction à l'événement « clic » d'un bouton :

```
document.getElementById("bouton_valider").addEventListener("click", function() {
    alert("Formulaire valide !");
});
```

◆ **PROPRIÉTÉ Simuler un gestionnaire d'événements** Le principe d'un gestionnaire d'événements peut se modéliser simplement : un **dictionnaire** associe à chaque identifiant de composant la fonction à exécuter lors du clic. « Déclencher un clic » revient alors à rechercher puis appeler la fonction associée.

CODE DE RÉFÉRENCE — Modélisation Python d'un gestionnaire d'événements

```
1 gestionnaires = {}
2
3 def ajouter_gestionnaire(id_composant, fonction):
4     """Associe 'fonction' à l'événement clic du composant 'id_composant'."""
5     gestionnaires[id_composant] = fonction
```

```

6
7 def declencher_clic(id_composant):
8     """Simule un clic : execute la fonction associee, si elle existe."""
9     if id_composant in gestionnaires:
10         return gestionnaires[id_composant]()
11     return None
12
13 compteur = {"valeur": 0}
14 def incrementer():
15     compteur["valeur"] += 1
16     return compteur["valeur"]
17
18 ajouter_gestionnaire("bouton_plus", incrementer)
19 print(declencher_clic("bouton_plus")) # 1
20 print(declencher_clic("bouton_plus")) # 2

```

⚠ ERREUR FRÉQUENTE

Confondre la **description** HTML d'un composant (son apparence, son identifiant) avec son **comportement** programmé (ce qui se passe lors d'un événement). Le code HTML `<button id="bouton_plus">+</button>` ne fait **rien** par lui-même tant qu'aucune fonction n'est associée à l'un de ses événements.

» **Python À la machine.** Modifie le code Python ci-dessus pour ajouter un second gestionnaire, associé à "bouton_moins", qui décrémente compteur["valeur"]. Déclenche successivement "bouton_plus", "bouton_plus", "bouton_moins", et vérifie que compteur["valeur"] vaut 1.

7.11 Interaction client-serveur

DÉFINITION D3

Le **client** désigne le navigateur de l'utilisateur ; le **serveur** désigne la machine distante qui héberge le site Web. Une interaction Web typique suit un ordre précis : le client envoie une **requête**, le serveur la traite et renvoie une **réponse**, que le client affiche.

◆ **PROPRIÉTÉ Ce qui s'exécute où** Le code qui s'exécute côté serveur (souvent en Python, PHP, etc.) génère la page à partir de données (base de données, calculs). Ce code n'est normalement pas transmis dans la réponse HTTP destinée au navigateur. Ses sources peuvent toutefois être publiées ou divulguées par une autre voie ; cet accès éventuel ne signifie pas qu'elles figurent dans la réponse HTTP générée par leur exécution. Le code exécuté sur le **client** (en JavaScript, dans le navigateur) réagit aux interactions de l'utilisateur **sans nouvel échange avec le serveur** (par exemple, afficher un message d'alerte), sauf s'il déclenche explicitement une nouvelle requête.

CODE DE RÉFÉRENCE — Modélisation de l'ordre client-serveur

```

1 etapes = []
2
3 def navigateur_envoie_requete(url):

```

```

4     etapes.append(("client", "envoi requete vers " + url))
5
6     def serveur_recoit_et_traite(url):
7         etapes.append(("serveur", "traite la requete " + url))
8         return "page HTML generee"
9
10    def serveur_revoie_reponse(contenu):
11        etapes.append(("serveur", "renvoie la reponse"))
12
13    def navigateur_affiche(contenu):
14        etapes.append(("client", "affiche : " + contenu))
15
16    navigateur_envoie_requete("/accueil")
17    contenu = serveur_recoit_et_traite("/accueil")
18    serveur_revoie_reponse(contenu)
19    navigateur_affiche(contenu)
20
21    print([acteur for acteur, action in etapes])
22    # ['client', 'serveur', 'serveur', 'client']

```

DÉFINITION D4

Un **cookie** est une petite donnée que le serveur demande au navigateur de **mémoriser** côté client. À chaque requête ultérieure vers le même site, le navigateur **retransmet** automatiquement ce cookie au serveur (par exemple, pour reconnaître un utilisateur déjà connecté).

CODE DE RÉFÉRENCE — Modélisation d'un cookie mémorisé côté client

```

1     cookies_client = {}
2
3     def serveur_definit_cookie(nom, valeur):
4         cookies_client[nom] = valeur
5
6     def client_envoie_requete_avec_cookies(url):
7         """Le client retransmet automatiquement les cookies memorises."""
8         return {"url": url, "cookies": dict(cookies_client)}
9
10    serveur_definit_cookie("id_session", "abc123")
11    requete = client_envoie_requete_avec_cookies("/panier")
12    print(requete) # {'url': '/panier', 'cookies': {'id_session': 'abc123'}}

```

◆ **PROPRIÉTÉ Transmission chiffrée (HTTPS)** Sans chiffrement (protocole **HTTP**), les données échangées entre client et serveur circulent **en clair** : n'importe quel intermédiaire sur le réseau peut les lire. Le protocole **HTTPS** chiffre ces échanges, rendant leur contenu illisible pour un intermédiaire. On utilise HTTPS dès que des informations sensibles (mot de passe, numéro de carte bancaire) sont transmises.

ERREUR FRÉQUENTE

Croire qu'un site en HTTPS est à l'abri de **tout** risque. HTTPS protège la **transmission** des données (personne ne peut les lire en chemin), mais ne garantit rien sur ce que le serveur **fait** des données une fois reçues (stockage sécurisé ou non, revente à des tiers, etc.).

» **Python À la machine.** Reprends le code du gestionnaire de cookies. Ajoute une fonction `supprimer_cookie(nom)` qui retire un cookie de `cookies_client` (par exemple lors d'une déconnexion). Vérifie qu'après suppression, une nouvelle requête ne contient plus ce cookie.

7.12 Formulaires, requêtes GET et POST

DÉFINITION D5

Un **formulaire** Web (balise `<form>`) regroupe des composants de saisie (champs de texte, cases à cocher...) et un bouton d'envoi. Lorsque l'utilisateur **soumet** le formulaire, le navigateur envoie une requête au serveur, contenant les valeurs saisies sous forme de **paramètres**.

EXEMPLE Un formulaire de recherche simple :

```
<form action="/recherche" method="GET">
  <input type="text" name="q">
  <button type="submit">Rechercher</button>
</form>
```

◆ **PROPRIÉTÉ Requête GET** Avec la méthode GET, les paramètres du formulaire sont ajoutés **directement dans l'URL**, après un point d'interrogation, sous la forme `cle1=valeur1&cle2=valeur2`. Ils sont donc **visibles** (dans la barre d'adresse, dans l'historique du navigateur) et de **taille limitée**.

CODE DE RÉFÉRENCE — Construire une requête GET (module `urllib.parse`)

```
1 from urllib.parse import urlencode
2
3 parametres = {"q": "chaton", "page": 2}
4 url_get = "https://exemple.fr/recherche?" + urlencode(parametres)
5 print(url_get)
6 # https://exemple.fr/recherche?q=chaton&page=2
```

◆ **PROPRIÉTÉ Requête POST** Avec la méthode POST, les paramètres sont placés dans le **corps** de la requête, pas dans l'URL : ils ne sont **pas visibles** dans la barre d'adresse et ne sont pas limités en taille de la même façon.

EXEMPLE Le formulaire de connexion suivant utilise POST, pour ne pas exposer le mot de passe dans l'URL :

```
<form action="/connexion" method="POST">
```

```
<input type="text" name="identifiant">
<input type="password" name="mdp">
<button type="submit">Se connecter</button>
</form>
```

◆ **PROPRIÉTÉ Choisir entre GET et POST** On choisit GET pour des paramètres **non sensibles** et courts (recherche, filtres, pagination), en profitant du fait qu'une URL GET peut être partagée, mise en favori ou revisitée facilement. On choisit POST pour des données **confidentielles** (mots de passe) ou **volumineuses** (envoi d'un fichier), qui ne doivent pas apparaître dans l'URL ni dans l'historique du navigateur.

⚠ ERREUR FRÉQUENTE

Utiliser GET pour transmettre un mot de passe ou toute donnée confidentielle. Même sur une connexion chiffrée (HTTPS), l'URL complète (avec les paramètres GET) reste visible dans l'**historique du navigateur** et peut être enregistrée dans les journaux (*logs*) du serveur : ce n'est pas un canal adapté à des données sensibles.

» **Python À la machine.** Écris une fonction `construire_url(base, parametres)` qui utilise `urlencode` pour construire une URL de requête GET à partir d'un dictionnaire de paramètres quelconque. Teste-la avec `ville": "Tunis", "tri": "prix"` et `ville": "Tunis", "tri": "prix"`.

Méthodes

► Méthode directe de travail sur le sujet.

Exercice 1 ◆

On donne l'extrait de code HTML suivant :

```
<form id="formulaire_avis">
  <input type="text" id="champ_pseudo" name="pseudo">
  <select id="liste_note" name="note">
    <option value="5">5 étoiles</option>
    <option value="1">1 étoile</option>
  </select>
  <button id="bouton_envoyer" type="submit">Envoyer</button>
</form>
```

1. Identifier les **trois** composants graphiques d'interaction présents dans ce formulaire (hors le formulaire lui-même).
2. Pour chacun, indiquer un événement plausible qu'une fonction JavaScript pourrait traiter (par exemple : saisie de texte, changement de sélection, clic).
3. Le code HTML seul suffit-il à faire réagir la page lorsqu'on clique sur le bouton ? Justifier.

Exercice 2 ◆

On reprend le modèle du cours qui enregistre un gestionnaire puis déclenche un clic. Un menu de navigation doit s'afficher ou se masquer à chaque clic sur le bouton "bouton_menu" (un menu « bascule », comme sur un site mobile).

1. Écrire une fonction `basculer_affichage()` qui inverse la valeur de `etat["visible"]` (initialement `False`) et renvoie la nouvelle valeur.
2. Associer cette fonction à `"bouton_menu"` avec `ajouter_gestionnaire`.
3. Simuler 3 clics successifs sur `"bouton_menu"`. Quelle est la séquence de valeurs renvoyées ?

Exercice 3 ♦♦

On reprend le modèle de cookies du cours. Un site de e-commerce fonctionne ainsi : un visiteur charge la page /accueil (aucun cookie n'existe encore), puis ajoute un article à son panier. Le serveur définit alors un cookie `"panier"` avec la valeur `"2 articles"`, puis le visiteur charge la page /paiement.

1. Que contient la requête envoyée pour /accueil, avant tout ajout au panier ?
2. Après l'ajout au panier, que contient la requête envoyée pour /paiement ?
3. Le serveur qui traite /paiement a-t-il besoin de « se souvenir » lui-même du panier du visiteur ? Expliquer, en une phrase, comment le cookie résout ce problème.

Exercice 4 ♦♦

Un site de paiement en ligne doit transmettre un numéro de carte bancaire et un code de sécurité au serveur.

1. Le site utilise HTTPS. Cela suffit-il à garantir que le numéro de carte ne sera visible par personne d'autre que le serveur destinataire ? Justifier en une phrase (on distinguera la protection pendant la **transmission** de ce qui se passe **avant** ou **après**).
2. Écrire le code qui construit l'URL qu'obtiendrait un attaquant regardant l'historique du navigateur, si (par erreur de conception) ce formulaire utilisait la méthode GET avec les paramètres `{"carte": "1234567812345678", "code": "123"}`. Le numéro de carte apparaît-il dans cette URL ?
3. Quelle méthode (GET ou POST) faut-il utiliser pour ce formulaire ? Justifier.

Exercice 5 ♦♦♦

Un site de voyage propose ce formulaire de recherche :

```
<form action="/recherche" method="GET">
  <input type="text" name="destination">
  <input type="number" name="voyageurs">
  <select name="tri">
    <option value="prix_croissant">Prix croissant</option>
  </select>
  <button type="submit">Rechercher</button>
</form>
```

Un visiteur saisit `"Djerba"` comme destination, 2 voyageurs, et laisse le tri sur `"prix_croissant"`.

1. En utilisant `urlencode`, construire l'URL complète de la requête GET qui serait envoyée (l'action est `"https://voyages.fr/recherche"`).
2. Ce visiteur peut-il mettre cette page de résultats en favori et la retrouver plus tard avec les mêmes critères de recherche ? Pourquoi cela fonctionne-t-il grâce à la méthode GET ?
3. Le concepteur du site hésite à passer ce formulaire en méthode POST, pour des raisons de sécurité. Est-ce justifié ici ? Argumenter en citant la nature des données transmises.

Exercice 6 ♦

On donne l'extrait de code HTML suivant :

```
<form id="formulaire_avis">
  <input type="text" id="champ_pseudo" name="pseudo">
  <select id="liste_note" name="note">
    <option value="5">5 étoiles</option>
```

```
<option value="1">1 étoile</option>
</select>
<button id="bouton_envoyer" type="submit">Envoyer</button>
</form>
```

1. Identifier les **trois** composants graphiques d'interaction présents dans ce formulaire (hors le formulaire lui-même).
2. Pour chacun, indiquer un événement plausible qu'une fonction JavaScript pourrait traiter (par exemple : saisie de texte, changement de sélection, clic).
3. Le code HTML seul suffit-il à faire réagir la page lorsqu'on clique sur le bouton ? Justifier.

Exercice 7

On reprend le modèle du cours qui enregistre un gestionnaire puis déclenche un clic. Un menu de navigation doit s'afficher ou se masquer à chaque clic sur le bouton "bouton_menu" (un menu « bascule », comme sur un site mobile).

1. Écrire une fonction basculer_affichage() qui inverse la valeur de etat["visible"] (initialement False) et renvoie la nouvelle valeur.
2. Associer cette fonction à "bouton_menu" avec ajouter_gestionnaire.
3. Simuler 3 clics successifs sur "bouton_menu". Quelle est la séquence de valeurs renvoyées ?

Exercice 8

On reprend le modèle de cookies du cours. Un site de e-commerce fonctionne ainsi : un visiteur charge la page /accueil (aucun cookie n'existe encore), puis ajoute un article à son panier. Le serveur définit alors un cookie "panier" avec la valeur "2 articles", puis le visiteur charge la page /paiement.

1. Que contient la requête envoyée pour /accueil, avant tout ajout au panier ?
2. Après l'ajout au panier, que contient la requête envoyée pour /paiement ?
3. Le serveur qui traite /paiement a-t-il besoin de « se souvenir » lui-même du panier du visiteur ? Expliquer, en une phrase, comment le cookie résout ce problème.

Exercice 9

Un site de paiement en ligne doit transmettre un numéro de carte bancaire et un code de sécurité au serveur.

1. Le site utilise HTTPS. Cela suffit-il à garantir que le numéro de carte ne sera visible par personne d'autre que le serveur destinataire ? Justifier en une phrase (on distinguera la protection pendant la **transmission** de ce qui se passe **avant** ou **après**).
2. Écrire le code qui construit l'URL qu'obtiendrait un attaquant regardant l'historique du navigateur, si (par erreur de conception) ce formulaire utilisait la méthode GET avec les paramètres {"carte": "1234567812345678", "code": "123"}. Le numéro de carte apparaît-il dans cette URL ?
3. Quelle méthode (GET ou POST) faut-il utiliser pour ce formulaire ? Justifier.

Exercice 10

Un site de voyage propose ce formulaire de recherche :

```
<form action="/recherche" method="GET">
  <input type="text" name="destination">
  <input type="number" name="voyageurs">
  <select name="tri">
    <option value="prix_croissant">Prix croissant</option>
  </select>
  <button type="submit">Rechercher</button>
</form>
```


Un visiteur saisit "Djerba" comme destination, 2 voyageurs, et laisse le tri sur "prix_croissant".

1. En utilisant urlencode, construire l'URL complète de la requête GET qui serait envoyée (l'action est "https://voyages.fr/recherche").
2. Ce visiteur peut-il mettre cette page de résultats en favori et la retrouver plus tard avec les mêmes critères de recherche ? Pourquoi cela fonctionne-t-il grâce à la méthode GET ?
3. Le concepteur du site hésite à passer ce formulaire en méthode POST, pour des raisons de sécurité. Est-ce justifié ici ? Argumenter en citant la nature des données transmises.

Exercice 11

On donne l'extrait de code HTML suivant :

```
<form id="formulaire_avis">
  <input type="text" id="champ_pseudo" name="pseudo">
  <select id="liste_note" name="note">
    <option value="5">5 étoiles</option>
    <option value="1">1 étoile</option>
  </select>
  <button id="bouton_envoyer" type="submit">Envoyer</button>
</form>
```

1. Identifier les **trois** composants graphiques d'interaction présents dans ce formulaire (hors le formulaire lui-même).
2. Pour chacun, indiquer un événement plausible qu'une fonction JavaScript pourrait traiter (par exemple : saisie de texte, changement de sélection, clic).
3. Le code HTML seul suffit-il à faire réagir la page lorsqu'on clique sur le bouton ? Justifier.

Exercice 12

On reprend le modèle du cours qui enregistre un gestionnaire puis déclenche un clic. Un menu de navigation doit s'afficher ou se masquer à chaque clic sur le bouton "bouton_menu" (un menu « bascule », comme sur un site mobile).

1. Écrire une fonction basculer_affichage() qui inverse la valeur de etat["visible"] (initialement False) et renvoie la nouvelle valeur.
2. Associer cette fonction à "bouton_menu" avec ajouter_gestionnaire.
3. Simuler 3 clics successifs sur "bouton_menu". Quelle est la séquence de valeurs renvoyées ?

Exercice 13

On reprend le modèle de cookies du cours. Un site de e-commerce fonctionne ainsi : un visiteur charge la page /accueil (aucun cookie n'existe encore), puis ajoute un article à son panier. Le serveur définit alors un cookie "panier" avec la valeur "2 articles", puis le visiteur charge la page /paiement.

1. Que contient la requête envoyée pour /accueil, avant tout ajout au panier ?
2. Après l'ajout au panier, que contient la requête envoyée pour /paiement ?
3. Le serveur qui traite /paiement a-t-il besoin de « se souvenir » lui-même du panier du visiteur ? Expliquer, en une phrase, comment le cookie résout ce problème.

Exercice 14

Un site de paiement en ligne doit transmettre un numéro de carte bancaire et un code de sécurité au serveur.

1. Le site utilise HTTPS. Cela suffit-il à garantir que le numéro de carte ne sera visible par personne d'autre que le serveur destinataire ? Justifier en une phrase (on distinguera la protection pendant la **transmission** de ce qui se passe **avant** ou **après**).

- Écrire le code qui construit l'URL qu'obtiendrait un attaquant regardant l'historique du navigateur, si (par erreur de conception) ce formulaire utilisait la méthode GET avec les paramètres {"carte": "1234567812345678", "code": "123"}. Le numéro de carte apparaît-il dans cette URL ?
- Quelle méthode (GET ou POST) faut-il utiliser pour ce formulaire ? Justifier.

Exercice 15

Un site de voyage propose ce formulaire de recherche :

```
<form action="/recherche" method="GET">
  <input type="text" name="destination">
  <input type="number" name="voyageurs">
  <select name="tri">
    <option value="prix_croissant">Prix croissant</option>
  </select>
  <button type="submit">Rechercher</button>
</form>
```

Un visiteur saisit "Djerba" comme destination, 2 voyageurs, et laisse le tri sur "prix_croissant".

- En utilisant urlencode, construire l'URL complète de la requête GET qui serait envoyée (l'action est "https://voyages.fr/recherche").
- Ce visiteur peut-il mettre cette page de résultats en favori et la retrouver plus tard avec les mêmes critères de recherche ? Pourquoi cela fonctionne-t-il grâce à la méthode GET ?
- Le concepteur du site hésite à passer ce formulaire en méthode POST, pour des raisons de sécurité. Est-ce justifié ici ? Argumenter en citant la nature des données transmises.

Exercice 16

On donne l'extrait de code HTML suivant :

```
<form id="formulaire_avis">
  <input type="text" id="champ_pseudo" name="pseudo">
  <select id="liste_note" name="note">
    <option value="5">5 étoiles</option>
    <option value="1">1 étoile</option>
  </select>
  <button id="bouton_envoyer" type="submit">Envoyer</button>
</form>
```

- Identifier les **trois** composants graphiques d'interaction présents dans ce formulaire (hors le formulaire lui-même).
- Pour chacun, indiquer un événement plausible qu'une fonction JavaScript pourrait traiter (par exemple : saisie de texte, changement de sélection, clic).
- Le code HTML seul suffit-il à faire réagir la page lorsqu'on clique sur le bouton ? Justifier.

Exercice 17

On reprend le modèle du cours qui enregistre un gestionnaire puis déclenche un clic. Un menu de navigation doit s'afficher ou se masquer à chaque clic sur le bouton "bouton_menu" (un menu « bascule », comme sur un site mobile).

- Écrire une fonction basculer_affichage() qui inverse la valeur de etat["visible"] (initialement False) et renvoie la nouvelle valeur.
- Associer cette fonction à "bouton_menu" avec ajouter_gestionnaire.
- Simuler 3 clics successifs sur "bouton_menu". Quelle est la séquence de valeurs renvoyées ?

Exercice 18 ♦♦

On reprend le modèle de cookies du cours. Un site de e-commerce fonctionne ainsi : un visiteur charge la page /accueil (aucun cookie n'existe encore), puis ajoute un article à son panier. Le serveur définit alors un cookie "panier" avec la valeur "2 articles", puis le visiteur charge la page /paiement.

1. Que contient la requête envoyée pour /accueil, avant tout ajout au panier ?
2. Après l'ajout au panier, que contient la requête envoyée pour /paiement ?
3. Le serveur qui traite /paiement a-t-il besoin de « se souvenir » lui-même du panier du visiteur ? Expliquer, en une phrase, comment le cookie résout ce problème.

Exercice 19 ♦♦

Un site de paiement en ligne doit transmettre un numéro de carte bancaire et un code de sécurité au serveur.

1. Le site utilise HTTPS. Cela suffit-il à garantir que le numéro de carte ne sera visible par personne d'autre que le serveur destinataire ? Justifier en une phrase (on distinguera la protection pendant la **transmission** de ce qui se passe **avant** ou **après**).
2. Écrire le code qui construit l'URL qu'obtiendrait un attaquant regardant l'historique du navigateur, si (par erreur de conception) ce formulaire utilisait la méthode GET avec les paramètres {"carte": "1234567812345678", "code": "123"}. Le numéro de carte apparaît-il dans cette URL ?
3. Quelle méthode (GET ou POST) faut-il utiliser pour ce formulaire ? Justifier.

Exercice 20 ♦♦♦

Un site de voyage propose ce formulaire de recherche :

```
<form action="/recherche" method="GET">
  <input type="text" name="destination">
  <input type="number" name="voyageurs">
  <select name="tri">
    <option value="prix_croissant">Prix croissant</option>
  </select>
  <button type="submit">Rechercher</button>
</form>
```

Un visiteur saisit "Djerba" comme destination, 2 voyageurs, et laisse le tri sur "prix_croissant".

1. En utilisant urlencode, construire l'URL complète de la requête GET qui serait envoyée (l'action est "https://voyages.fr/recherche").
2. Ce visiteur peut-il mettre cette page de résultats en favori et la retrouver plus tard avec les mêmes critères de recherche ? Pourquoi cela fonctionne-t-il grâce à la méthode GET ?
3. Le concepteur du site hésite à passer ce formulaire en méthode POST, pour des raisons de sécurité. Est-ce justifié ici ? Argumenter en citant la nature des données transmises.

Exercice 21 ♦

On donne l'extrait de code HTML suivant :

```
<form id="formulaire_avis">
  <input type="text" id="champ_pseudo" name="pseudo">
  <select id="liste_note" name="note">
    <option value="5">5 étoiles</option>
    <option value="1">1 étoile</option>
  </select>
  <button id="bouton_envoyer" type="submit">Envoyer</button>
</form>
```

1. Identifier les **trois** composants graphiques d'interaction présents dans ce formulaire (hors le formulaire lui-même).
2. Pour chacun, indiquer un événement plausible qu'une fonction JavaScript pourrait traiter (par exemple : saisie de texte, changement de sélection, clic).
3. Le code HTML seul suffit-il à faire réagir la page lorsqu'on clique sur le bouton ? Justifier.

Exercice 22

On reprend le modèle du cours qui enregistre un gestionnaire puis déclenche un clic. Un menu de navigation doit s'afficher ou se masquer à chaque clic sur le bouton "bouton_menu" (un menu « bascule », comme sur un site mobile).

1. Écrire une fonction `basculer_affichage()` qui inverse la valeur de `etat["visible"]` (initialement `False`) et renvoie la nouvelle valeur.
2. Associer cette fonction à "bouton_menu" avec `ajouter_gestionnaire`.
3. Simuler 3 clics successifs sur "bouton_menu". Quelle est la séquence de valeurs renvoyées ?

Exercice 23

On reprend le modèle de cookies du cours. Un site de e-commerce fonctionne ainsi : un visiteur charge la page /accueil (aucun cookie n'existe encore), puis ajoute un article à son panier. Le serveur définit alors un cookie "panier" avec la valeur "2 articles", puis le visiteur charge la page /paiement.

1. Que contient la requête envoyée pour /accueil, avant tout ajout au panier ?
2. Après l'ajout au panier, que contient la requête envoyée pour /paiement ?
3. Le serveur qui traite /paiement a-t-il besoin de « se souvenir » lui-même du panier du visiteur ? Expliquer, en une phrase, comment le cookie résout ce problème.

Exercice 24

Un site de paiement en ligne doit transmettre un numéro de carte bancaire et un code de sécurité au serveur.

1. Le site utilise HTTPS. Cela suffit-il à garantir que le numéro de carte ne sera visible par personne d'autre que le serveur destinataire ? Justifier en une phrase (on distinguera la protection pendant la **transmission** de ce qui se passe **avant** ou **après**).
2. Écrire le code qui construit l'URL qu'obtiendrait un attaquant regardant l'historique du navigateur, si (par erreur de conception) ce formulaire utilisait la méthode GET avec les paramètres `{"carte": "1234567812345678", "code": "123"}`. Le numéro de carte apparaît-il dans cette URL ?
3. Quelle méthode (GET ou POST) faut-il utiliser pour ce formulaire ? Justifier.

Exercice 25

Un site de voyage propose ce formulaire de recherche :

```
<form action="/recherche" method="GET">
  <input type="text" name="destination">
  <input type="number" name="voyageurs">
  <select name="tri">
    <option value="prix_croissant">Prix croissant</option>
  </select>
  <button type="submit">Rechercher</button>
</form>
```

Un visiteur saisit "Djerba" comme destination, 2 voyageurs, et laisse le tri sur "prix_croissant".

1. En utilisant `urlencode`, construire l'URL complète de la requête GET qui serait envoyée (l'action est "https://voyages.fr/recherche").

2. Ce visiteur peut-il mettre cette page de résultats en favori et la retrouver plus tard avec les mêmes critères de recherche ? Pourquoi cela fonctionne-t-il grâce à la méthode GET ?
3. Le concepteur du site hésite à passer ce formulaire en méthode POST, pour des raisons de sécurité. Est-ce justifié ici ? Argumenter en citant la nature des données transmises.

Exercice 26

On donne l'extrait de code HTML suivant :

```
<form id="formulaire_avis">
  <input type="text" id="champ_pseudo" name="pseudo">
  <select id="liste_note" name="note">
    <option value="5">5 étoiles</option>
    <option value="1">1 étoile</option>
  </select>
  <button id="bouton_envoyer" type="submit">Envoyer</button>
</form>
```

1. Identifier les **trois** composants graphiques d'interaction présents dans ce formulaire (hors le formulaire lui-même).
2. Pour chacun, indiquer un événement plausible qu'une fonction JavaScript pourrait traiter (par exemple : saisie de texte, changement de sélection, clic).
3. Le code HTML seul suffit-il à faire réagir la page lorsqu'on clique sur le bouton ? Justifier.

Exercice 27

On reprend le modèle du cours qui enregistre un gestionnaire puis déclenche un clic. Un menu de navigation doit s'afficher ou se masquer à chaque clic sur le bouton "bouton_menu" (un menu « bascule », comme sur un site mobile).

1. Écrire une fonction basculer_affichage() qui inverse la valeur de etat["visible"] (initialement False) et renvoie la nouvelle valeur.
2. Associer cette fonction à "bouton_menu" avec ajouter_gestionnaire.
3. Simuler 3 clics successifs sur "bouton_menu". Quelle est la séquence de valeurs renvoyées ?

Exercice 28

On reprend le modèle de cookies du cours. Un site de e-commerce fonctionne ainsi : un visiteur charge la page /accueil (aucun cookie n'existe encore), puis ajoute un article à son panier. Le serveur définit alors un cookie "panier" avec la valeur "2 articles", puis le visiteur charge la page /paiement.

1. Que contient la requête envoyée pour /accueil, avant tout ajout au panier ?
2. Après l'ajout au panier, que contient la requête envoyée pour /paiement ?
3. Le serveur qui traite /paiement a-t-il besoin de « se souvenir » lui-même du panier du visiteur ? Expliquer, en une phrase, comment le cookie résout ce problème.

Exercice 29

Un site de paiement en ligne doit transmettre un numéro de carte bancaire et un code de sécurité au serveur.

1. Le site utilise HTTPS. Cela suffit-il à garantir que le numéro de carte ne sera visible par personne d'autre que le serveur destinataire ? Justifier en une phrase (on distinguera la protection pendant la **transmission** de ce qui se passe **avant** ou **après**).
2. Écrire le code qui construit l'URL qu'obtiendrait un attaquant regardant l'historique du navigateur, si (par erreur de conception) ce formulaire utilisait la méthode GET avec les paramètres {"carte": "1234567812345678", "code": "123"}. Le numéro de carte apparaît-il dans cette URL ?

3. Quelle méthode (GET ou POST) faut-il utiliser pour ce formulaire ? Justifier.

Exercice 30

Un site de voyage propose ce formulaire de recherche :

```
<form action="/recherche" method="GET">
  <input type="text" name="destination">
  <input type="number" name="voyageurs">
  <select name="tri">
    <option value="prix_croissant">Prix croissant</option>
  </select>
  <button type="submit">Rechercher</button>
</form>
```

Un visiteur saisit "Djerba" comme destination, 2 voyageurs, et laisse le tri sur "prix_croissant".

1. En utilisant urlencode, construire l'URL complète de la requête GET qui serait envoyée (l'action est "https://voyages.fr/recherche").
2. Ce visiteur peut-il mettre cette page de résultats en favori et la retrouver plus tard avec les mêmes critères de recherche ? Pourquoi cela fonctionne-t-il grâce à la méthode GET ?
3. Le concepteur du site hésite à passer ce formulaire en méthode POST, pour des raisons de sécurité. Est-ce justifié ici ? Argumenter en citant la nature des données transmises.

Exercice 31

On donne l'extrait de code HTML suivant :

```
<form id="formulaire_avis">
  <input type="text" id="champ_pseudo" name="pseudo">
  <select id="liste_note" name="note">
    <option value="5">5 étoiles</option>
    <option value="1">1 étoile</option>
  </select>
  <button id="bouton_envoyer" type="submit">Envoyer</button>
</form>
```

1. Identifier les **trois** composants graphiques d'interaction présents dans ce formulaire (hors le formulaire lui-même).
2. Pour chacun, indiquer un événement plausible qu'une fonction JavaScript pourrait traiter (par exemple : saisie de texte, changement de sélection, clic).
3. Le code HTML seul suffit-il à faire réagir la page lorsqu'on clique sur le bouton ? Justifier.

Exercice 32

On reprend le modèle du cours qui enregistre un gestionnaire puis déclenche un clic. Un menu de navigation doit s'afficher ou se masquer à chaque clic sur le bouton "bouton_menu" (un menu « bascule », comme sur un site mobile).

1. Écrire une fonction basculer_affichage() qui inverse la valeur de etat["visible"] (initialement False) et renvoie la nouvelle valeur.
2. Associer cette fonction à "bouton_menu" avec ajouter_gestionnaire.
3. Simuler 3 clics successifs sur "bouton_menu". Quelle est la séquence de valeurs renvoyées ?

Exercice 33 ♦♦

On reprend le modèle de cookies du cours. Un site de e-commerce fonctionne ainsi : un visiteur charge la page /accueil (aucun cookie n'existe encore), puis ajoute un article à son panier. Le serveur définit alors un cookie "panier" avec la valeur "2 articles", puis le visiteur charge la page /paiement.

1. Que contient la requête envoyée pour /accueil, avant tout ajout au panier ?
2. Après l'ajout au panier, que contient la requête envoyée pour /paiement ?
3. Le serveur qui traite /paiement a-t-il besoin de « se souvenir » lui-même du panier du visiteur ? Expliquer, en une phrase, comment le cookie résout ce problème.

Exercice 34 ♦♦

Un site de paiement en ligne doit transmettre un numéro de carte bancaire et un code de sécurité au serveur.

1. Le site utilise HTTPS. Cela suffit-il à garantir que le numéro de carte ne sera visible par personne d'autre que le serveur destinataire ? Justifier en une phrase (on distinguera la protection pendant la **transmission** de ce qui se passe **avant** ou **après**).
2. Écrire le code qui construit l'URL qu'obtiendrait un attaquant regardant l'historique du navigateur, si (par erreur de conception) ce formulaire utilisait la méthode GET avec les paramètres {"carte": "1234567812345678", "code": "123"}. Le numéro de carte apparaît-il dans cette URL ?
3. Quelle méthode (GET ou POST) faut-il utiliser pour ce formulaire ? Justifier.

Exercice 35 ♦♦♦

Un site de voyage propose ce formulaire de recherche :

```
<form action="/recherche" method="GET">
  <input type="text" name="destination">
  <input type="number" name="voyageurs">
  <select name="tri">
    <option value="prix_croissant">Prix croissant</option>
  </select>
  <button type="submit">Rechercher</button>
</form>
```

Un visiteur saisit "Djerba" comme destination, 2 voyageurs, et laisse le tri sur "prix_croissant".

1. En utilisant urlencode, construire l'URL complète de la requête GET qui serait envoyée (l'action est "https://voyages.fr/recherche").
2. Ce visiteur peut-il mettre cette page de résultats en favori et la retrouver plus tard avec les mêmes critères de recherche ? Pourquoi cela fonctionne-t-il grâce à la méthode GET ?
3. Le concepteur du site hésite à passer ce formulaire en méthode POST, pour des raisons de sécurité. Est-ce justifié ici ? Argumenter en citant la nature des données transmises.

Exercice 36 ♦

On donne l'extrait de code HTML suivant :

```
<form id="formulaire_avis">
  <input type="text" id="champ_pseudo" name="pseudo">
  <select id="liste_note" name="note">
    <option value="5">5 étoiles</option>
    <option value="1">1 étoile</option>
  </select>
  <button id="bouton_envoyer" type="submit">Envoyer</button>
</form>
```

1. Identifier les **trois** composants graphiques d'interaction présents dans ce formulaire (hors le formulaire lui-même).
2. Pour chacun, indiquer un événement plausible qu'une fonction JavaScript pourrait traiter (par exemple : saisie de texte, changement de sélection, clic).
3. Le code HTML seul suffit-il à faire réagir la page lorsqu'on clique sur le bouton ? Justifier.

Exercice 37

On reprend le modèle du cours qui enregistre un gestionnaire puis déclenche un clic. Un menu de navigation doit s'afficher ou se masquer à chaque clic sur le bouton "bouton_menu" (un menu « bascule », comme sur un site mobile).

1. Écrire une fonction `basculer_affichage()` qui inverse la valeur de `etat["visible"]` (initialement `False`) et renvoie la nouvelle valeur.
2. Associer cette fonction à "bouton_menu" avec `ajouter_gestionnaire`.
3. Simuler 3 clics successifs sur "bouton_menu". Quelle est la séquence de valeurs renvoyées ?

Exercice 38

On reprend le modèle de cookies du cours. Un site de e-commerce fonctionne ainsi : un visiteur charge la page /accueil (aucun cookie n'existe encore), puis ajoute un article à son panier. Le serveur définit alors un cookie "panier" avec la valeur "2 articles", puis le visiteur charge la page /paiement.

1. Que contient la requête envoyée pour /accueil, avant tout ajout au panier ?
2. Après l'ajout au panier, que contient la requête envoyée pour /paiement ?
3. Le serveur qui traite /paiement a-t-il besoin de « se souvenir » lui-même du panier du visiteur ? Expliquer, en une phrase, comment le cookie résout ce problème.

Exercice 39

Un site de paiement en ligne doit transmettre un numéro de carte bancaire et un code de sécurité au serveur.

1. Le site utilise HTTPS. Cela suffit-il à garantir que le numéro de carte ne sera visible par personne d'autre que le serveur destinataire ? Justifier en une phrase (on distinguera la protection pendant la **transmission** de ce qui se passe **avant** ou **après**).
2. Écrire le code qui construit l'URL qu'obtiendrait un attaquant regardant l'historique du navigateur, si (par erreur de conception) ce formulaire utilisait la méthode GET avec les paramètres {"carte": "1234567812345678", "code": "123"}. Le numéro de carte apparaît-il dans cette URL ?
3. Quelle méthode (GET ou POST) faut-il utiliser pour ce formulaire ? Justifier.

Exercice 40

Un site de voyage propose ce formulaire de recherche :

```
<form action="/recherche" method="GET">
  <input type="text" name="destination">
  <input type="number" name="voyageurs">
  <select name="tri">
    <option value="prix_croissant">Prix croissant</option>
  </select>
  <button type="submit">Rechercher</button>
</form>
```

Un visiteur saisit "Djerba" comme destination, 2 voyageurs, et laisse le tri sur "prix_croissant".

1. En utilisant `urlencode`, construire l'URL complète de la requête GET qui serait envoyée (l'action est "https://voyages.fr/recherche").

2. Ce visiteur peut-il mettre cette page de résultats en favori et la retrouver plus tard avec les mêmes critères de recherche ? Pourquoi cela fonctionne-t-il grâce à la méthode GET ?
3. Le concepteur du site hésite à passer ce formulaire en méthode POST, pour des raisons de sécurité. Est-ce justifié ici ? Argumenter en citant la nature des données transmises.

Exercice 41

On donne l'extrait de code HTML suivant :

```
<form id="formulaire_avis">
  <input type="text" id="champ_pseudo" name="pseudo">
  <select id="liste_note" name="note">
    <option value="5">5 étoiles</option>
    <option value="1">1 étoile</option>
  </select>
  <button id="bouton_envoyer" type="submit">Envoyer</button>
</form>
```

1. Identifier les **trois** composants graphiques d'interaction présents dans ce formulaire (hors le formulaire lui-même).
2. Pour chacun, indiquer un événement plausible qu'une fonction JavaScript pourrait traiter (par exemple : saisie de texte, changement de sélection, clic).
3. Le code HTML seul suffit-il à faire réagir la page lorsqu'on clique sur le bouton ? Justifier.

Exercice 42

On reprend le modèle du cours qui enregistre un gestionnaire puis déclenche un clic. Un menu de navigation doit s'afficher ou se masquer à chaque clic sur le bouton "bouton_menu" (un menu « bascule », comme sur un site mobile).

1. Écrire une fonction basculer_affichage() qui inverse la valeur de etat["visible"] (initialement False) et renvoie la nouvelle valeur.
2. Associer cette fonction à "bouton_menu" avec ajouter_gestionnaire.
3. Simuler 3 clics successifs sur "bouton_menu". Quelle est la séquence de valeurs renvoyées ?

Exercice 43

On reprend le modèle de cookies du cours. Un site de e-commerce fonctionne ainsi : un visiteur charge la page /accueil (aucun cookie n'existe encore), puis ajoute un article à son panier. Le serveur définit alors un cookie "panier" avec la valeur "2 articles", puis le visiteur charge la page /paiement.

1. Que contient la requête envoyée pour /accueil, avant tout ajout au panier ?
2. Après l'ajout au panier, que contient la requête envoyée pour /paiement ?
3. Le serveur qui traite /paiement a-t-il besoin de « se souvenir » lui-même du panier du visiteur ? Expliquer, en une phrase, comment le cookie résout ce problème.

Exercice 44

Un site de paiement en ligne doit transmettre un numéro de carte bancaire et un code de sécurité au serveur.

1. Le site utilise HTTPS. Cela suffit-il à garantir que le numéro de carte ne sera visible par personne d'autre que le serveur destinataire ? Justifier en une phrase (on distinguera la protection pendant la **transmission** de ce qui se passe **avant** ou **après**).
2. Écrire le code qui construit l'URL qu'obtiendrait un attaquant regardant l'historique du navigateur, si (par erreur de conception) ce formulaire utilisait la méthode GET avec les paramètres {"carte": "1234567812345678", "code": "123"}. Le numéro de carte apparaît-il dans cette URL ?

3. Quelle méthode (GET ou POST) faut-il utiliser pour ce formulaire ? Justifier.

Exercice 45

Un site de voyage propose ce formulaire de recherche :

```
<form action="/recherche" method="GET">
  <input type="text" name="destination">
  <input type="number" name="voyageurs">
  <select name="tri">
    <option value="prix_croissant">Prix croissant</option>
  </select>
  <button type="submit">Rechercher</button>
</form>
```

Un visiteur saisit "Djerba" comme destination, 2 voyageurs, et laisse le tri sur "prix_croissant".

1. En utilisant urlencode, construire l'URL complète de la requête GET qui serait envoyée (l'action est "https://voyages.fr/recherche").
2. Ce visiteur peut-il mettre cette page de résultats en favori et la retrouver plus tard avec les mêmes critères de recherche ? Pourquoi cela fonctionne-t-il grâce à la méthode GET ?
3. Le concepteur du site hésite à passer ce formulaire en méthode POST, pour des raisons de sécurité. Est-ce justifié ici ? Argumenter en citant la nature des données transmises.

Exercice 46

On donne l'extrait de code HTML suivant :

```
<form id="formulaire_avis">
  <input type="text" id="champ_pseudo" name="pseudo">
  <select id="liste_note" name="note">
    <option value="5">5 étoiles</option>
    <option value="1">1 étoile</option>
  </select>
  <button id="bouton_envoyer" type="submit">Envoyer</button>
</form>
```

1. Identifier les **trois** composants graphiques d'interaction présents dans ce formulaire (hors le formulaire lui-même).
2. Pour chacun, indiquer un événement plausible qu'une fonction JavaScript pourrait traiter (par exemple : saisie de texte, changement de sélection, clic).
3. Le code HTML seul suffit-il à faire réagir la page lorsqu'on clique sur le bouton ? Justifier.

Exercice 47

On reprend le modèle du cours qui enregistre un gestionnaire puis déclenche un clic. Un menu de navigation doit s'afficher ou se masquer à chaque clic sur le bouton "bouton_menu" (un menu « bascule », comme sur un site mobile).

1. Écrire une fonction basculer_affichage() qui inverse la valeur de etat["visible"] (initialement False) et renvoie la nouvelle valeur.
2. Associer cette fonction à "bouton_menu" avec ajouter_gestionnaire.
3. Simuler 3 clics successifs sur "bouton_menu". Quelle est la séquence de valeurs renvoyées ?

Exercice 48 ♦♦

On reprend le modèle de cookies du cours. Un site de e-commerce fonctionne ainsi : un visiteur charge la page /accueil (aucun cookie n'existe encore), puis ajoute un article à son panier. Le serveur définit alors un cookie "panier" avec la valeur "2 articles", puis le visiteur charge la page /paiement.

1. Que contient la requête envoyée pour /accueil, avant tout ajout au panier ?
2. Après l'ajout au panier, que contient la requête envoyée pour /paiement ?
3. Le serveur qui traite /paiement a-t-il besoin de « se souvenir » lui-même du panier du visiteur ? Expliquer, en une phrase, comment le cookie résout ce problème.

Exercice 49 ♦♦

Un site de paiement en ligne doit transmettre un numéro de carte bancaire et un code de sécurité au serveur.

1. Le site utilise HTTPS. Cela suffit-il à garantir que le numéro de carte ne sera visible par personne d'autre que le serveur destinataire ? Justifier en une phrase (on distinguera la protection pendant la **transmission** de ce qui se passe **avant** ou **après**).
2. Écrire le code qui construit l'URL qu'obtiendrait un attaquant regardant l'historique du navigateur, si (par erreur de conception) ce formulaire utilisait la méthode GET avec les paramètres {"carte": "1234567812345678", "code": "123"}. Le numéro de carte apparaît-il dans cette URL ?
3. Quelle méthode (GET ou POST) faut-il utiliser pour ce formulaire ? Justifier.

Exercice 50 ♦♦♦

Un site de voyage propose ce formulaire de recherche :

```
<form action="/recherche" method="GET">
  <input type="text" name="destination">
  <input type="number" name="voyageurs">
  <select name="tri">
    <option value="prix_croissant">Prix croissant</option>
  </select>
  <button type="submit">Rechercher</button>
</form>
```

Un visiteur saisit "Djerba" comme destination, 2 voyageurs, et laisse le tri sur "prix_croissant".

1. En utilisant urlencode, construire l'URL complète de la requête GET qui serait envoyée (l'action est "https://voyages.fr/recherche").
2. Ce visiteur peut-il mettre cette page de résultats en favori et la retrouver plus tard avec les mêmes critères de recherche ? Pourquoi cela fonctionne-t-il grâce à la méthode GET ?
3. Le concepteur du site hésite à passer ce formulaire en méthode POST, pour des raisons de sécurité. Est-ce justifié ici ? Argumenter en citant la nature des données transmises.

QCM d'auto-évaluation — Interactions homme-machine sur le Web

Pour chaque question, une seule réponse est exacte. Entourez la lettre correspondante.

C1/C2 — Composants et événements

1. [Q1] Le comportement d'un bouton lorsqu'on clique dessus est généralement programmé :
 - A. directement dans le code HTML.
 - B. en JavaScript, via un gestionnaire d'événement.

- C. dans la feuille de style CSS.
- D. il n'est jamais possible de programmer un tel comportement.

C4 — Client-serveur

2. [Q2] Le code exécuté **sur le serveur** :
- A. est visible dans le code source de la page reçue par le navigateur.
 - B. n'est jamais visible par l'utilisateur.
 - C. s'exécute toujours après le code JavaScript du client.
 - D. ne peut jamais accéder à une base de données.

C6 — Chiffrement

3. [Q3] Le protocole HTTPS garantit que :
- A. les données sont illisibles pour un intermédiaire pendant la transmission.
 - B. le serveur stocke les données de façon sécurisée.
 - C. aucune donnée personnelle n'est jamais collectée.
 - D. le site est fiable et digne de confiance.

C8 — GET et POST

4. [Q4] Les paramètres d'une requête POST sont :
- A. visibles dans l'URL de la page.
 - B. placés dans le corps de la requête, invisibles dans l'URL.
 - C. toujours chiffrés automatiquement.
 - D. limités à un seul paramètre.

C9 — Choix du type de requête

5. [Q5] Pour transmettre un mot de passe lors d'une connexion, il est recommandé d'utiliser :
- A. GET, car c'est la méthode la plus simple.
 - B. POST, pour ne pas exposer le mot de passe dans l'URL.
 - C. indifféremment GET ou POST, cela n'a aucune importance.
 - D. aucune des deux, un mot de passe ne doit jamais être transmis.
6. [Q6] Une recherche de produits sur un site marchand (mots-clés, filtres) utilise généralement :
- A. GET, car les critères ne sont pas confidentiels et l'URL peut être partagée.
 - B. POST, systématiquement, quelle que soit la nature des données.
 - C. ni GET ni POST : uniquement des cookies.
 - D. un formulaire sans méthode définie.

Corrigé

1. Non. Le HTML décrit uniquement l'apparence des composants ; sans code JavaScript associant une fonction à l'événement clic du bouton, rien ne se produit automatiquement.

2.

```
1 panier = {"articles": 0}
2
3 def ajouter_article():
4     panier["articles"] += 1
```

```

5     return panier["articles"]
6
7     ajouter_gestionnaire("bouton_ajouter", ajouter_article)

```

3. La séquence est [1, 2, 3].

Corrigé

1.

```

1 from urllib.parse import urlencode
2
3 params = {"q": "ordinateur", "trie": "popularite"}
4 url = "https://boutique.fr/recherche?" + urlencode(params)
5 print(url)
6 # https://boutique.fr/recherche?q=ordinateur&trie=popularite

```

2. Oui, GET est adapté : les critères de recherche ne sont pas confidentiels, et l'URL résultante peut être partagée ou mise en favori pour retrouver la même recherche.

3. Il faut privilégier **POST** : un numéro de téléphone et une adresse postale sont des données à caractère personnel, qui ne doivent pas apparaître dans l'URL (historique du navigateur, journaux du serveur).

Évaluation A — Interactions homme-machine sur le Web

Durée : 45 minutes. Ordinateur avec interpréteur Python autorisé.

Barème indicatif : Ex. 1 : 10 pts (composants, gestionnaire d'événement); Ex. 2 : 10 pts (GET/POST)

Exercice 1 — Bouton « ajouter au panier »

(10 points)

On donne le code HTML suivant :

```

<button id="bouton_ajouter">Ajouter au panier</button>
<span id="compteur">0</span> article(s)

```

- (2 pts) Ce code HTML seul suffit-il à incrémenter le compteur lors d'un clic ? Justifier.
- (5 pts) En reprenant le modèle du cours (ajouter_gestionnaire, déclencher_clic), écrire une fonction ajouter_article() qui incrémente panier["articles"] (initialement 0) et renvoie la nouvelle valeur, puis l'associer à "bouton_ajouter".
- (3 pts) Simuler 3 clics successifs. Quelle est la séquence de valeurs renvoyées ?

Exercice 2 — Choisir GET ou POST

(10 points)

- (4 pts) Un formulaire de recherche de produits transmet les paramètres {"q": "ordinateur", "trie": "popularite"}. Construire, avec urlencode, l'URL de la requête GET correspondante (base : "https://boutique.fr/recherche").
- (3 pts) Cette recherche peut-elle raisonnablement utiliser GET ? Justifier en citant la nature des données.
- (3 pts) Un autre formulaire du même site transmet un numéro de téléphone et une adresse postale pour une livraison. Quelle méthode privilégier, et pourquoi ?

Corrigé

1. Non. Le HTML décrit uniquement l'apparence des composants ; sans code JavaScript associant une fonction à l'événement clic, rien ne se produit automatiquement.

2.

```

1 favoris = {"articles": 3}
2
3 def retirer_favori():
4     favoris["articles"] -= 1
5     return favoris["articles"]
6
7 ajouter_gestionnaire("bouton_retirer", retirer_favori)

```

3. La séquence est [2, 1].

Corrigé

1.

```

1 from urllib.parse import urlencode
2
3 params = {"ville": "Sousse", "type": "appartement"}
4 url = "https://immo.fr/recherche?" + urlencode(params)
5 print(url)
6 # https://immo.fr/recherche?ville=Sousse&type=appartement

```

2. Oui, GET est adapté : les critères de recherche (ville, type de bien) ne sont pas confidentiels, et l'URL résultante peut être partagée ou mise en favori.

3. Il faut privilégier **POST** : le nom, l'email et le message d'un visiteur sont des données à caractère personnel qui ne doivent pas apparaître dans l'URL.

Évaluation B — Interactions homme-machine sur le Web

Durée : 45 minutes. Ordinateur avec interpréteur Python autorisé.

Barème indicatif : Ex. 1 : 10 pts (composants, gestionnaire d'événement); Ex. 2 : 10 pts (GET/POST)

Exercice 1 — Bouton « retirer des favoris »

(10 points)

On donne le code HTML suivant, sur une page listant 3 favoris :

```

<button id="bouton_retirer">Retirer des favoris</button>
<span id="compteur">3</span> favori(s)

```

1. (2 pts) Ce code HTML seul suffit-il à décrémenter le compteur lors d'un clic ? Justifier.
2. (5 pts) En reprenant le modèle du cours, écrire une fonction `retirer_favori()` qui décrémente `favoris["articles"]` (initialisé à 3) et renvoie la nouvelle valeur, puis l'associer à "bouton_retirer".
3. (3 pts) Simuler 2 clics successifs. Quelle est la séquence de valeurs renvoyées ?

Exercice 2 — Choisir GET ou POST

(10 points)

1. (4 pts) Un formulaire de recherche immobilière transmet les paramètres `{"ville": "Sousse", "type": "appartement"}`. Construire, avec `urlencode`, l'URL de la requête GET correspondante (base : "https://immo.fr/recherche").
2. (3 pts) Cette recherche peut-elle raisonnablement utiliser GET ? Justifier en citant la nature des données.
3. (3 pts) Un autre formulaire du même site transmet le nom, l'email et le message d'un visiteur souhaitant être recontacté. Quelle méthode privilégier, et pourquoi ?

Exercice 51

Un élève conçoit un formulaire de réinitialisation de mot de passe avec la méthode GET :

```
<form action="/nouveau_mdp" method="GET">
  <input type="password" name="nouveau_mdp">
  <button type="submit">Valider</button>
</form>
```

1. En utilisant urlencode, construire l'URL qui serait générée si l'utilisateur saisit "MonSecret2026" comme nouveau mot de passe.
2. Le nouveau mot de passe apparaît-il dans cette URL ? Où cette URL risque-t-elle d'être enregistrée (historique, journaux serveur) ?
3. Réécrire l'attribut method du formulaire pour corriger ce problème.

Exercice 52

Un élève conçoit un formulaire de réinitialisation de mot de passe avec la méthode GET :

```
<form action="/nouveau_mdp" method="GET">
  <input type="password" name="nouveau_mdp">
  <button type="submit">Valider</button>
</form>
```

1. En utilisant urlencode, construire l'URL qui serait générée si l'utilisateur saisit "MonSecret2026" comme nouveau mot de passe.
2. Le nouveau mot de passe apparaît-il dans cette URL ? Où cette URL risque-t-elle d'être enregistrée (historique, journaux serveur) ?
3. Réécrire l'attribut method du formulaire pour corriger ce problème.

Exercice 53

Un élève conçoit un formulaire de réinitialisation de mot de passe avec la méthode GET :

```
<form action="/nouveau_mdp" method="GET">
  <input type="password" name="nouveau_mdp">
  <button type="submit">Valider</button>
</form>
```

1. En utilisant urlencode, construire l'URL qui serait générée si l'utilisateur saisit "MonSecret2026" comme nouveau mot de passe.
2. Le nouveau mot de passe apparaît-il dans cette URL ? Où cette URL risque-t-elle d'être enregistrée (historique, journaux serveur) ?
3. Réécrire l'attribut method du formulaire pour corriger ce problème.

Exercice 54

Un élève conçoit un formulaire de réinitialisation de mot de passe avec la méthode GET :

```
<form action="/nouveau_mdp" method="GET">
  <input type="password" name="nouveau_mdp">
  <button type="submit">Valider</button>
</form>
```

1. En utilisant urlencode, construire l'URL qui serait générée si l'utilisateur saisit "MonSecret2026" comme nouveau mot de passe.
2. Le nouveau mot de passe apparaît-il dans cette URL ? Où cette URL risque-t-elle d'être enregistrée (historique, journaux serveur) ?
3. Réécrire l'attribut method du formulaire pour corriger ce problème.

Exercice 55

Un élève conçoit un formulaire de réinitialisation de mot de passe avec la méthode GET :

```
<form action="/nouveau_mdp" method="GET">
  <input type="password" name="nouveau_mdp">
  <button type="submit">Valider</button>
</form>
```

1. En utilisant urlencode, construire l'URL qui serait générée si l'utilisateur saisit "MonSecret2026" comme nouveau mot de passe.
2. Le nouveau mot de passe apparaît-il dans cette URL ? Où cette URL risque-t-elle d'être enregistrée (historique, journaux serveur) ?
3. Réécrire l'attribut method du formulaire pour corriger ce problème.

Exercice 56

Un élève conçoit un formulaire de réinitialisation de mot de passe avec la méthode GET :

```
<form action="/nouveau_mdp" method="GET">
  <input type="password" name="nouveau_mdp">
  <button type="submit">Valider</button>
</form>
```

1. En utilisant urlencode, construire l'URL qui serait générée si l'utilisateur saisit "MonSecret2026" comme nouveau mot de passe.
2. Le nouveau mot de passe apparaît-il dans cette URL ? Où cette URL risque-t-elle d'être enregistrée (historique, journaux serveur) ?
3. Réécrire l'attribut method du formulaire pour corriger ce problème.

Exercice 57

Un élève conçoit un formulaire de réinitialisation de mot de passe avec la méthode GET :

```
<form action="/nouveau_mdp" method="GET">
  <input type="password" name="nouveau_mdp">
  <button type="submit">Valider</button>
</form>
```

1. En utilisant urlencode, construire l'URL qui serait générée si l'utilisateur saisit "MonSecret2026" comme nouveau mot de passe.
2. Le nouveau mot de passe apparaît-il dans cette URL ? Où cette URL risque-t-elle d'être enregistrée (historique, journaux serveur) ?
3. Réécrire l'attribut method du formulaire pour corriger ce problème.

Exercice 58

Un élève conçoit un formulaire de réinitialisation de mot de passe avec la méthode GET :


```
<form action="/nouveau_mdp" method="GET">
  <input type="password" name="nouveau_mdp">
  <button type="submit">Valider</button>
</form>
```

1. En utilisant urlencode, construire l'URL qui serait générée si l'utilisateur saisit "MonSecret2026" comme nouveau mot de passe.
2. Le nouveau mot de passe apparaît-il dans cette URL ? Où cette URL risque-t-elle d'être enregistrée (historique, journaux serveur) ?
3. Réécrire l'attribut method du formulaire pour corriger ce problème.

Exercice 59 ◆

Un élève conçoit un formulaire de réinitialisation de mot de passe avec la méthode GET :

```
<form action="/nouveau_mdp" method="GET">
  <input type="password" name="nouveau_mdp">
  <button type="submit">Valider</button>
</form>
```

1. En utilisant urlencode, construire l'URL qui serait générée si l'utilisateur saisit "MonSecret2026" comme nouveau mot de passe.
2. Le nouveau mot de passe apparaît-il dans cette URL ? Où cette URL risque-t-elle d'être enregistrée (historique, journaux serveur) ?
3. Réécrire l'attribut method du formulaire pour corriger ce problème.

Exercice 60 ◆

Un élève conçoit un formulaire de réinitialisation de mot de passe avec la méthode GET :

```
<form action="/nouveau_mdp" method="GET">
  <input type="password" name="nouveau_mdp">
  <button type="submit">Valider</button>
</form>
```

1. En utilisant urlencode, construire l'URL qui serait générée si l'utilisateur saisit "MonSecret2026" comme nouveau mot de passe.
2. Le nouveau mot de passe apparaît-il dans cette URL ? Où cette URL risque-t-elle d'être enregistrée (historique, journaux serveur) ?
3. Réécrire l'attribut method du formulaire pour corriger ce problème.

Corrigé

1. Trois composants : le champ de texte champ_pseudo, la liste déroulante liste_note, le bouton bouton_envoyer.

2. Champ de texte : événement de **saisie** (l'utilisateur tape du texte). Liste déroulante : événement de **changement** (l'utilisateur choisit une option). Bouton : événement de **clic** (ou de soumission du formulaire).

3. Non. Le code HTML décrit uniquement la **structure** et l'**apparence** des composants ; il ne contient **aucun** gestionnaire d'événement. Sans code JavaScript associant une fonction à l'événement clic du bouton, rien de particulier ne se produit à part la soumission par défaut du formulaire (rechargement de la page).

Corrigé

1.

```

1  etat = {"visible": False}
2
3  def basculer_affichage():
4      etat["visible"] = not etat["visible"]
5      return etat["visible"]

```

2.

```

1  ajouter_gestionnaire("bouton_menu", basculer_affichage)

```

3. La séquence est [True, False, True] : le premier clic affiche le menu (True), le deuxième le masque (False), le troisième le réaffiche (True).

Corrigé

1. La requête pour /accueil ne contient **aucun** cookie : {"url": "/accueil", "cookies": {}}.

2. La requête pour /paiement contient le cookie panier : {"url": "/paiement", "cookies": {"panier": "2 articles"}}.

3. Non : le serveur n'a pas besoin de conserver lui-même l'état du panier de chaque visiteur. C'est le **navigateur** qui mémorise le cookie et le **retransmet automatiquement** à chaque requête vers ce site : le serveur retrouve ainsi l'information sans avoir eu à la stocker de son côté.

Corrigé

1. Non, cela ne suffit pas. HTTPS protège uniquement les données **pendant leur transmission** sur le réseau (personne ne peut les intercepter en chemin). Cela ne dit rien sur la façon dont l'URL est enregistrée dans l'**historique du navigateur**, dans les **journaux du serveur**, ou sur la sécurité du **stockage** des données une fois reçues par le serveur.

2.

```

1  from urllib.parse import urlencode
2
3  donnees = {"carte": "1234567812345678", "code": "123"}
4  url_si_get = "https://banque.fr/paiement?" + urlencode(donnees)
5  print(url_si_get)

```

Oui, le numéro de carte complet apparaîtrait en clair dans l'URL, visible dans l'historique du navigateur et potentiellement dans les journaux du serveur.

3. Il faut utiliser **POST** : les données (numéro de carte, code de sécurité) sont confidentielles, et POST les place dans le corps de la requête plutôt que dans l'URL. Elles n'apparaissent donc pas dans l'historique des URL du navigateur. En revanche, un serveur web, un proxy ou l'application peuvent néanmoins journaliser le corps de la requête. Il faut aussi utiliser HTTPS, minimiser les données collectées et appliquer une politique de journalisation qui protège et limite ces enregistrements.

Corrigé

1.

```

1  from urllib.parse import urlencode
2
3  params = {"destination": "Djerba", "voyageurs": 2, "tri": "prix_croissant"}
4  url = "https://voyages.fr/recherche?" + urlencode(params)
5  print(url)
6  # https://voyages.fr/recherche?destination=Djerba&voyageurs=2&tri=prix_croissant

```

2. Oui : avec GET, les paramètres de recherche font partie intégrante de l'URL. Mettre cette URL en favori (ou la partager) permet de retrouver exactement la même recherche plus tard, car tous les critères sont encodés dans l'adresse elle-même.

3. Non, ce n'est pas justifié ici : les données transmises (destination, nombre de voyageurs, critère de tri) ne sont **pas confidentielles** — ce sont des critères de recherche publics, pas des mots de passe ni des informations sensibles. GET est adapté, et présente même un avantage (URL partageable/favorisable) que POST n'offre pas.

Corrigé

1.

```
1 from urllib.parse import urlencode
2
3 donnees = {"nouveau_mdp": "MonSecret2026"}
4 url = "https://site.fr/nouveau_mdp?" + urlencode(donnees)
5 print(url)
6 # https://site.fr/nouveau_mdp?nouveau_mdp=MonSecret2026
```

2. Oui, le mot de passe apparaît **en clair** dans l'URL. Cette URL risque d'être enregistrée dans l'**historique du navigateur** et dans les **journaux (logs) du serveur**, deux endroits où un mot de passe ne devrait jamais apparaître, même sur une connexion HTTPS (qui protège seulement la transmission, pas ces enregistrements).

3.

```
<form action="/nouveau_mdp" method="POST">
  <input type="password" name="nouveau_mdp">
  <button type="submit">Valider</button>
</form>
```

Avec `method="POST"`, le mot de passe est placé dans le corps de la requête, pas dans l'URL.

Corrigé

1. Trois composants : le champ de texte `champ_pseudo`, la liste déroulante `liste_note`, le bouton `bouton_envoyer`.

2. Champ de texte : événement de **saisie** (l'utilisateur tape du texte). Liste déroulante : événement de **changement** (l'utilisateur choisit une option). Bouton : événement de **clic** (ou de soumission du formulaire).

3. Non. Le code HTML décrit uniquement la **structure** et l'**apparence** des composants ; il ne contient **aucun** gestionnaire d'événement. Sans code JavaScript associant une fonction à l'événement clic du bouton, rien de particulier ne se produit à part la soumission par défaut du formulaire (rechargement de la page).

Corrigé

1.

```
1 etat = {"visible": False}
2
3 def basculer_affichage():
4     etat["visible"] = not etat["visible"]
5     return etat["visible"]
```

2.

```
1 ajouter_gestionnaire("bouton_menu", basculer_affichage)
```

3. La séquence est [True, False, True] : le premier clic affiche le menu (True), le deuxième le masque (False), le troisième le réaffiche (True).

Corrigé

1. La requête pour /accueil ne contient **aucun** cookie : {"url": "/accueil", "cookies": {}}.
2. La requête pour /paiement contient le cookie panier : {"url": "/paiement", "cookies": {"panier": "2 articles"}}.
3. Non : le serveur n'a pas besoin de conserver lui-même l'état du panier de chaque visiteur. C'est le **navigateur** qui mémorise le cookie et le **retransmet automatiquement** à chaque requête vers ce site : le serveur retrouve ainsi l'information sans avoir eu à la stocker de son côté.

Corrigé

1. Non, cela ne suffit pas. HTTPS protège uniquement les données **pendant leur transmission** sur le réseau (personne ne peut les intercepter en chemin). Cela ne dit rien sur la façon dont l'URL est enregistrée dans l'**historique du navigateur**, dans les **journaux du serveur**, ou sur la sécurité du **stockage** des données une fois reçues par le serveur.

2.

```
1 from urllib.parse import urlencode
2
3 donnees = {"carte": "1234567812345678", "code": "123"}
4 url_si_get = "https://banque.fr/paiement?" + urlencode(donnees)
5 print(url_si_get)
```

Oui, le numéro de carte complet apparaîtrait en clair dans l'URL, visible dans l'historique du navigateur et potentiellement dans les journaux du serveur.

3. Il faut utiliser **POST** : les données (numéro de carte, code de sécurité) sont confidentielles, et POST les place dans le corps de la requête plutôt que dans l'URL. Elles n'apparaissent donc pas dans l'historique des URL du navigateur. En revanche, un serveur web, un proxy ou l'application peuvent néanmoins journaliser le corps de la requête. Il faut aussi utiliser HTTPS, minimiser les données collectées et appliquer une politique de journalisation qui protège et limite ces enregistrements.

Corrigé

1.

```
1 from urllib.parse import urlencode
2
3 params = {"destination": "Djerba", "voyageurs": 2, "tri": "prix_croissant"}
4 url = "https://voyages.fr/recherche?" + urlencode(params)
5 print(url)
6 # https://voyages.fr/recherche?destination=Djerba&voyageurs=2&tri=prix_croissant
```

2. Oui : avec GET, les paramètres de recherche font partie intégrante de l'URL. Mettre cette URL en favori (ou la partager) permet de retrouver exactement la même recherche plus tard, car tous les critères sont encodés dans l'adresse elle-même.

3. Non, ce n'est pas justifié ici : les données transmises (destination, nombre de voyageurs, critère de tri) ne sont **pas confidentielles** — ce sont des critères de recherche publics, pas des mots de passe ni des informations sensibles. GET est adapté, et présente même un avantage (URL partageable/favorisable) que POST n'offre pas.

Corrigé

1.

```
1 from urllib.parse import urlencode
2
3 donnees = {"nouveau_mdp": "MonSecret2026"}
4 url = "https://site.fr/nouveau_mdp?" + urlencode(donnees)
5 print(url)
6 # https://site.fr/nouveau_mdp?nouveau_mdp=MonSecret2026
```

2. Oui, le mot de passe apparaît **en clair** dans l'URL. Cette URL risque d'être enregistrée dans l'**historique du navigateur** et dans les **journaux (logs) du serveur**, deux endroits où un mot de passe ne devrait jamais apparaître, même sur une connexion HTTPS (qui protège seulement la transmission, pas ces enregistrements).

3.

```
<form action="/nouveau_mdp" method="POST">
  <input type="password" name="nouveau_mdp">
  <button type="submit">Valider</button>
</form>
```

Avec method="POST", le mot de passe est placé dans le corps de la requête, pas dans l'URL.

Corrigé

1. Trois composants : le champ de texte champ_pseudo, la liste déroulante liste_note, le bouton bouton_envoyer.

2. Champ de texte : événement de **saisie** (l'utilisateur tape du texte). Liste déroulante : événement de **changement** (l'utilisateur choisit une option). Bouton : événement de **clic** (ou de soumission du formulaire).

3. Non. Le code HTML décrit uniquement la **structure** et l'**apparence** des composants ; il ne contient **aucun** gestionnaire d'événement. Sans code JavaScript associant une fonction à l'événement clic du bouton, rien de particulier ne se produit à part la soumission par défaut du formulaire (rechargement de la page).

Corrigé

1.

```
1 etat = {"visible": False}
2
3 def basculer_affichage():
4     etat["visible"] = not etat["visible"]
5     return etat["visible"]
```

2.

```
1 ajouter_gestionnaire("bouton_menu", basculer_affichage)
```

3. La séquence est [True, False, True] : le premier clic affiche le menu (True), le deuxième le masque (False), le troisième le réaffiche (True).

Corrigé

1. La requête pour /accueil ne contient **aucun** cookie : {"url": "/accueil", "cookies": {}}.

2. La requête pour /paiement contient le cookie panier : {"url": "/paiement", "cookies": {"panier": "2 articles"}}.

3. Non : le serveur n'a pas besoin de conserver lui-même l'état du panier de chaque visiteur. C'est le **navigateur** qui mémorise le cookie et le **retransmet automatiquement** à chaque requête vers ce site : le serveur retrouve ainsi l'information sans avoir eu à la stocker de son côté.

Corrigé

1. Non, cela ne suffit pas. HTTPS protège uniquement les données **pendant leur transmission** sur le réseau (personne ne peut les intercepter en chemin). Cela ne dit rien sur la façon dont l'URL est enregistrée dans l'**historique du navigateur**, dans les **journaux du serveur**, ou sur la sécurité du **stockage** des données une fois reçues par le serveur.

2.

```
1 from urllib.parse import urlencode
2
3 donnees = {"carte": "1234567812345678", "code": "123"}
4 url_si_get = "https://banque.fr/paiement?" + urlencode(donnees)
5 print(url_si_get)
```

Oui, le numéro de carte complet apparaîtrait en clair dans l'URL, visible dans l'historique du navigateur et potentiellement dans les journaux du serveur.

3. Il faut utiliser **POST** : les données (numéro de carte, code de sécurité) sont confidentielles, et POST les place dans le corps de la requête plutôt que dans l'URL. Elles n'apparaissent donc pas dans l'historique des URL du navigateur. En revanche, un serveur web, un proxy ou l'application peuvent néanmoins journaliser le corps de la requête. Il faut aussi utiliser HTTPS, minimiser les données collectées et appliquer une politique de journalisation qui protège et limite ces enregistrements.

Corrigé

1.

```
1 from urllib.parse import urlencode
2
3 params = {"destination": "Djerba", "voyageurs": 2, "tri": "prix_croissant"}
4 url = "https://voyages.fr/recherche?" + urlencode(params)
5 print(url)
6 # https://voyages.fr/recherche?destination=Djerba&voyageurs=2&tri=prix_croissant
```

2. Oui : avec GET, les paramètres de recherche font partie intégrante de l'URL. Mettre cette URL en favori (ou la partager) permet de retrouver exactement la même recherche plus tard, car tous les critères sont encodés dans l'adresse elle-même.

3. Non, ce n'est pas justifié ici : les données transmises (destination, nombre de voyageurs, critère de tri) ne sont **pas confidentielles** — ce sont des critères de recherche publics, pas des mots de passe ni des informations sensibles. GET est adapté, et présente même un avantage (URL partageable/favorisable) que POST n'offre pas.

Corrigé

1.

```
1 from urllib.parse import urlencode
2
3 donnees = {"nouveau_mdp": "MonSecret2026"}
4 url = "https://site.fr/nouveau_mdp?" + urlencode(donnees)
5 print(url)
6 # https://site.fr/nouveau_mdp?nouveau_mdp=MonSecret2026
```

2. Oui, le mot de passe apparaît **en clair** dans l'URL. Cette URL risque d'être enregistrée dans l'**historique du navigateur** et dans les **journaux (logs) du serveur**, deux endroits où un mot de passe ne devrait jamais apparaître, même sur une connexion HTTPS (qui protège seulement la transmission, pas ces enregistrements).

3.

```
<form action="/nouveau_mdp" method="POST">
  <input type="password" name="nouveau_mdp">
  <button type="submit">Valider</button>
</form>
```

Avec `method="POST"`, le mot de passe est placé dans le corps de la requête, pas dans l'URL.

Corrigé

1. Trois composants : le champ de texte `champ_pseudo`, la liste déroulante `liste_note`, le bouton `bouton_envoyer`.

2. Champ de texte : événement de **saisie** (l'utilisateur tape du texte). Liste déroulante : événement de **changement** (l'utilisateur choisit une option). Bouton : événement de **clik** (ou de soumission du formulaire).

3. Non. Le code HTML décrit uniquement la **structure** et l'**apparence** des composants ; il ne contient **aucun** gestionnaire d'événement. Sans code JavaScript associant une fonction à l'événement `clik` du bouton, rien de particulier ne se produit à part la soumission par défaut du formulaire (rechargement de la page).

Corrigé

1.

```
1 | etat = {"visible": False}
2 |
3 | def basculer_affichage():
4 |     etat["visible"] = not etat["visible"]
5 |     return etat["visible"]
```

2.

```
1 | ajouter_gestionnaire("bouton_menu", basculer_affichage)
```

3. La séquence est `[True, False, True]` : le premier clic affiche le menu (`True`), le deuxième le masque (`False`), le troisième le réaffiche (`True`).

Corrigé

1. La requête pour `/accueil` ne contient **aucun** cookie : `{"url": "/accueil", "cookies": {}}`.

2. La requête pour `/paiement` contient le cookie panier : `{"url": "/paiement", "cookies": {"panier": "2 articles"}}`.

3. Non : le serveur n'a pas besoin de conserver lui-même l'état du panier de chaque visiteur. C'est le **navigateur** qui mémorise le cookie et le **retransmet automatiquement** à chaque requête vers ce site : le serveur retrouve ainsi l'information sans avoir eu à la stocker de son côté.

Corrigé

1. Non, cela ne suffit pas. HTTPS protège uniquement les données **pendant leur transmission** sur le réseau (personne ne peut les intercepter en chemin). Cela ne dit rien sur la façon dont l'URL est enregistrée dans l'**historique du navigateur**, dans les **journaux du serveur**, ou sur la sécurité du **stockage** des données une fois reçues par le serveur.

2.

```
1 | from urllib.parse import urlencode
2 |
3 | donnees = {"carte": "1234567812345678", "code": "123"}
4 | url_si_get = "https://banque.fr/paiement?" + urlencode(donnees)
5 | print(url_si_get)
```

Oui, le numéro de carte complet apparaîtrait en clair dans l'URL, visible dans l'historique du navigateur et potentiellement dans les journaux du serveur.

3. Il faut utiliser **POST** : les données (numéro de carte, code de sécurité) sont confidentielles, et POST les place dans le corps de la requête plutôt que dans l'URL. Elles n'apparaissent donc pas dans l'historique des URL du navigateur. En revanche, un serveur web, un proxy ou l'application peuvent néanmoins journaliser le corps de la requête. Il faut aussi utiliser HTTPS, minimiser les données collectées et appliquer une politique de journalisation qui protège et limite ces enregistrements.

Corrigé

1.

```
1 from urllib.parse import urlencode
2
3 params = {"destination": "Djerba", "voyageurs": 2, "tri": "prix_croissant"}
4 url = "https://voyages.fr/recherche?" + urlencode(params)
5 print(url)
6 # https://voyages.fr/recherche?destination=Djerba&voyageurs=2&tri=prix_croissant
```

2. Oui : avec GET, les paramètres de recherche font partie intégrante de l'URL. Mettre cette URL en favori (ou la partager) permet de retrouver exactement la même recherche plus tard, car tous les critères sont encodés dans l'adresse elle-même.

3. Non, ce n'est pas justifié ici : les données transmises (destination, nombre de voyageurs, critère de tri) ne sont **pas confidentielles** — ce sont des critères de recherche publics, pas des mots de passe ni des informations sensibles. GET est adapté, et présente même un avantage (URL partageable/favorisable) que POST n'offre pas.

Corrigé

1.

```
1 from urllib.parse import urlencode
2
3 donnees = {"nouveau_mdp": "MonSecret2026"}
4 url = "https://site.fr/nouveau_mdp?" + urlencode(donnees)
5 print(url)
6 # https://site.fr/nouveau_mdp?nouveau_mdp=MonSecret2026
```

2. Oui, le mot de passe apparaît **en clair** dans l'URL. Cette URL risque d'être enregistrée dans l'**historique du navigateur** et dans les **journaux (logs) du serveur**, deux endroits où un mot de passe ne devrait jamais apparaître, même sur une connexion HTTPS (qui protège seulement la transmission, pas ces enregistrements).

3.

```
<form action="/nouveau_mdp" method="POST">
  <input type="password" name="nouveau_mdp">
  <button type="submit">Valider</button>
</form>
```

Avec `method="POST"`, le mot de passe est placé dans le corps de la requête, pas dans l'URL.

Corrigé

1. Trois composants : le champ de texte `champ_pseudo`, la liste déroulante `liste_note`, le bouton `bouton_envoyer`.

2. Champ de texte : événement de **saisie** (l'utilisateur tape du texte). Liste déroulante : événement de **changement** (l'utilisateur choisit une option). Bouton : événement de **clik** (ou de soumission du formulaire).

3. Non. Le code HTML décrit uniquement la **structure** et l'**apparence** des composants ; il ne contient **aucun** gestionnaire d'événement. Sans code JavaScript associant une fonction à l'événement clic du bouton, rien de particulier ne se produit à part la soumission par défaut du formulaire (rechargement de la page).

Corrigé

1.

```
1 etat = {"visible": False}
2
3 def basculer_affichage():
4     etat["visible"] = not etat["visible"]
5     return etat["visible"]
```

2.

```
1 ajouter_gestionnaire("bouton_menu", basculer_affichage)
```

3. La séquence est [True, False, True] : le premier clic affiche le menu (True), le deuxième le masque (False), le troisième le réaffiche (True).

Corrigé

1. La requête pour /accueil ne contient **aucun** cookie : {"url": "/accueil", "cookies": {}}.

2. La requête pour /paiement contient le cookie panier : {"url": "/paiement", "cookies": {"panier": "2 articles"}}.

3. Non : le serveur n'a pas besoin de conserver lui-même l'état du panier de chaque visiteur. C'est le **navigateur** qui mémorise le cookie et le **retransmet automatiquement** à chaque requête vers ce site : le serveur retrouve ainsi l'information sans avoir eu à la stocker de son côté.

Corrigé

1. Non, cela ne suffit pas. HTTPS protège uniquement les données **pendant leur transmission** sur le réseau (personne ne peut les intercepter en chemin). Cela ne dit rien sur la façon dont l'URL est enregistrée dans l'**historique du navigateur**, dans les **journaux du serveur**, ou sur la sécurité du **stockage** des données une fois reçues par le serveur.

2.

```
1 from urllib.parse import urlencode
2
3 donnees = {"carte": "1234567812345678", "code": "123"}
4 url_si_get = "https://banque.fr/paiement?" + urlencode(donnees)
5 print(url_si_get)
```

Oui, le numéro de carte complet apparaîtrait en clair dans l'URL, visible dans l'historique du navigateur et potentiellement dans les journaux du serveur.

3. Il faut utiliser **POST** : les données (numéro de carte, code de sécurité) sont confidentielles, et POST les place dans le corps de la requête plutôt que dans l'URL. Elles n'apparaissent donc pas dans l'historique des URL du navigateur. En revanche, un serveur web, un proxy ou l'application peuvent néanmoins journaliser le corps de la requête. Il faut aussi utiliser HTTPS, minimiser les données collectées et appliquer une politique de journalisation qui protège et limite ces enregistrements.

Corrigé

1.

```
1 from urllib.parse import urlencode
```

```

2
3 params = {"destination": "Djerba", "voyageurs": 2, "tri": "prix_croissant"}
4 url = "https://voyages.fr/recherche?" + urlencode(params)
5 print(url)
6 # https://voyages.fr/recherche?destination=Djerba&voyageurs=2&tri=prix_croissant

```

2. Oui : avec GET, les paramètres de recherche font partie intégrante de l'URL. Mettre cette URL en favori (ou la partager) permet de retrouver exactement la même recherche plus tard, car tous les critères sont encodés dans l'adresse elle-même.

3. Non, ce n'est pas justifié ici : les données transmises (destination, nombre de voyageurs, critère de tri) ne sont **pas confidentielles** — ce sont des critères de recherche publics, pas des mots de passe ni des informations sensibles. GET est adapté, et présente même un avantage (URL partageable/favorisable) que POST n'offre pas.

Corrigé

1.

```

1 from urllib.parse import urlencode
2
3 donnees = {"nouveau_mdp": "MonSecret2026"}
4 url = "https://site.fr/nouveau_mdp?" + urlencode(donnees)
5 print(url)
6 # https://site.fr/nouveau_mdp?nouveau_mdp=MonSecret2026

```

2. Oui, le mot de passe apparaît **en clair** dans l'URL. Cette URL risque d'être enregistrée dans l'**historique du navigateur** et dans les **journaux (logs) du serveur**, deux endroits où un mot de passe ne devrait jamais apparaître, même sur une connexion HTTPS (qui protège seulement la transmission, pas ces enregistrements).

3.

```

<form action="/nouveau_mdp" method="POST">
  <input type="password" name="nouveau_mdp">
  <button type="submit">Valider</button>
</form>

```

Avec `method="POST"`, le mot de passe est placé dans le corps de la requête, pas dans l'URL.

Corrigé

1. Trois composants : le champ de texte `champ_pseudo`, la liste déroulante `liste_note`, le bouton `bouton_envoyer`.

2. Champ de texte : événement de **saisie** (l'utilisateur tape du texte). Liste déroulante : événement de **changement** (l'utilisateur choisit une option). Bouton : événement de **clic** (ou de soumission du formulaire).

3. Non. Le code HTML décrit uniquement la **structure** et l'**apparence** des composants ; il ne contient **aucun** gestionnaire d'événement. Sans code JavaScript associant une fonction à l'événement clic du bouton, rien de particulier ne se produit à part la soumission par défaut du formulaire (rechargement de la page).

Corrigé

1.

```

1 etat = {"visible": False}
2
3 def basculer_affichage():
4     etat["visible"] = not etat["visible"]

```

```
5 return etat["visible"]
```

2.

```
1 ajouter_gestionnaire("bouton_menu", basculer_affichage)
```

3. La séquence est [True, False, True] : le premier clic affiche le menu (True), le deuxième le masque (False), le troisième le réaffiche (True).

Corrigé

1. La requête pour /accueil ne contient **aucun** cookie : {"url": "/accueil", "cookies": {}}.

2. La requête pour /paiement contient le cookie panier : {"url": "/paiement", "cookies": {"panier": "2 articles"}}.

3. Non : le serveur n'a pas besoin de conserver lui-même l'état du panier de chaque visiteur. C'est le **navigateur** qui mémorise le cookie et le **retransmet automatiquement** à chaque requête vers ce site : le serveur retrouve ainsi l'information sans avoir eu à la stocker de son côté.

Corrigé

1. Non, cela ne suffit pas. HTTPS protège uniquement les données **pendant leur transmission** sur le réseau (personne ne peut les intercepter en chemin). Cela ne dit rien sur la façon dont l'URL est enregistrée dans l'**historique du navigateur**, dans les **journaux du serveur**, ou sur la sécurité du **stockage** des données une fois reçues par le serveur.

2.

```
1 from urllib.parse import urlencode
2
3 donnees = {"carte": "1234567812345678", "code": "123"}
4 url_si_get = "https://banque.fr/paiement?" + urlencode(donnees)
5 print(url_si_get)
```

Oui, le numéro de carte complet apparaîtrait en clair dans l'URL, visible dans l'historique du navigateur et potentiellement dans les journaux du serveur.

3. Il faut utiliser **POST** : les données (numéro de carte, code de sécurité) sont confidentielles, et POST les place dans le corps de la requête plutôt que dans l'URL. Elles n'apparaissent donc pas dans l'historique des URL du navigateur. En revanche, un serveur web, un proxy ou l'application peuvent néanmoins journaliser le corps de la requête. Il faut aussi utiliser HTTPS, minimiser les données collectées et appliquer une politique de journalisation qui protège et limite ces enregistrements.

Corrigé

1.

```
1 from urllib.parse import urlencode
2
3 params = {"destination": "Djerba", "voyageurs": 2, "tri": "prix_croissant"}
4 url = "https://voyages.fr/recherche?" + urlencode(params)
5 print(url)
6 # https://voyages.fr/recherche?destination=Djerba&voyageurs=2&tri=prix_croissant
```

2. Oui : avec GET, les paramètres de recherche font partie intégrante de l'URL. Mettre cette URL en favori (ou la partager) permet de retrouver exactement la même recherche plus tard, car tous les critères sont encodés dans l'adresse elle-même.

3. Non, ce n'est pas justifié ici : les données transmises (destination, nombre de voyageurs, critère de tri) ne sont **pas confidentielles** — ce sont des critères de recherche publics, pas des mots de passe ni des informations sensibles. GET est adapté, et présente même un avantage (URL partageable/favorisable) que POST n'offre pas.

Corrigé

1.

```
1 from urllib.parse import urlencode
2
3 donnees = {"nouveau_mdp": "MonSecret2026"}
4 url = "https://site.fr/nouveau_mdp?" + urlencode(donnees)
5 print(url)
6 # https://site.fr/nouveau_mdp?nouveau_mdp=MonSecret2026
```

2. Oui, le mot de passe apparaît **en clair** dans l'URL. Cette URL risque d'être enregistrée dans l'**historique du navigateur** et dans les **journaux (logs) du serveur**, deux endroits où un mot de passe ne devrait jamais apparaître, même sur une connexion HTTPS (qui protège seulement la transmission, pas ces enregistrements).

3.

```
<form action="/nouveau_mdp" method="POST">
  <input type="password" name="nouveau_mdp">
  <button type="submit">Valider</button>
</form>
```

Avec method="POST", le mot de passe est placé dans le corps de la requête, pas dans l'URL.

Corrigé

1. Trois composants : le champ de texte champ_pseudo, la liste déroulante liste_note, le bouton bouton_envoyer.

2. Champ de texte : événement de **saisie** (l'utilisateur tape du texte). Liste déroulante : événement de **changement** (l'utilisateur choisit une option). Bouton : événement de **clic** (ou de soumission du formulaire).

3. Non. Le code HTML décrit uniquement la **structure** et l'**apparence** des composants ; il ne contient **aucun** gestionnaire d'événement. Sans code JavaScript associant une fonction à l'événement clic du bouton, rien de particulier ne se produit à part la soumission par défaut du formulaire (rechargement de la page).

Corrigé

1.

```
1 etat = {"visible": False}
2
3 def basculer_affichage():
4     etat["visible"] = not etat["visible"]
5     return etat["visible"]
```

2.

```
1 ajouter_gestionnaire("bouton_menu", basculer_affichage)
```

3. La séquence est [True, False, True] : le premier clic affiche le menu (True), le deuxième le masque (False), le troisième le réaffiche (True).

Corrigé

1. La requête pour /accueil ne contient **aucun** cookie : {"url": "/accueil", "cookies": {}}.

2. La requête pour /paiement contient le cookie panier : {"url": "/paiement", "cookies": {"panier": "2 articles"}}.

3. Non : le serveur n'a pas besoin de conserver lui-même l'état du panier de chaque visiteur. C'est le **navigateur** qui mémorise le cookie et le **retransmet automatiquement** à chaque requête vers ce site : le serveur retrouve ainsi l'information sans avoir eu à la stocker de son côté.

Corrigé

1. Non, cela ne suffit pas. HTTPS protège uniquement les données **pendant leur transmission** sur le réseau (personne ne peut les intercepter en chemin). Cela ne dit rien sur la façon dont l'URL est enregistrée dans l'**historique du navigateur**, dans les **journaux du serveur**, ou sur la sécurité du **stockage** des données une fois reçues par le serveur.

2.

```
1 from urllib.parse import urlencode
2
3 donnees = {"carte": "1234567812345678", "code": "123"}
4 url_si_get = "https://banque.fr/paiement?" + urlencode(donnees)
5 print(url_si_get)
```

Oui, le numéro de carte complet apparaîtrait en clair dans l'URL, visible dans l'historique du navigateur et potentiellement dans les journaux du serveur.

3. Il faut utiliser **POST** : les données (numéro de carte, code de sécurité) sont confidentielles, et POST les place dans le corps de la requête plutôt que dans l'URL. Elles n'apparaissent donc pas dans l'historique des URL du navigateur. En revanche, un serveur web, un proxy ou l'application peuvent néanmoins journaliser le corps de la requête. Il faut aussi utiliser HTTPS, minimiser les données collectées et appliquer une politique de journalisation qui protège et limite ces enregistrements.

Corrigé

1.

```
1 from urllib.parse import urlencode
2
3 params = {"destination": "Djerba", "voyageurs": 2, "tri": "prix_croissant"}
4 url = "https://voyages.fr/recherche?" + urlencode(params)
5 print(url)
6 # https://voyages.fr/recherche?destination=Djerba&voyageurs=2&tri=prix_croissant
```

2. Oui : avec GET, les paramètres de recherche font partie intégrante de l'URL. Mettre cette URL en favori (ou la partager) permet de retrouver exactement la même recherche plus tard, car tous les critères sont encodés dans l'adresse elle-même.

3. Non, ce n'est pas justifié ici : les données transmises (destination, nombre de voyageurs, critère de tri) ne sont **pas confidentielles** — ce sont des critères de recherche publics, pas des mots de passe ni des informations sensibles. GET est adapté, et présente même un avantage (URL partageable/favorisable) que POST n'offre pas.

Corrigé

1.

```
1 from urllib.parse import urlencode
2
3 donnees = {"nouveau_mdp": "MonSecret2026"}
4 url = "https://site.fr/nouveau_mdp?" + urlencode(donnees)
5 print(url)
6 # https://site.fr/nouveau_mdp?nouveau_mdp=MonSecret2026
```

2. Oui, le mot de passe apparaît **en clair** dans l'URL. Cette URL risque d'être enregistrée dans l'**historique du navigateur** et dans les **journaux (logs) du serveur**, deux endroits où un mot de passe

ne devrait jamais apparaître, même sur une connexion HTTPS (qui protège seulement la transmission, pas ces enregistrements).

3.

```
<form action="/nouveau_mdp" method="POST">
  <input type="password" name="nouveau_mdp">
  <button type="submit">Valider</button>
</form>
```

Avec `method="POST"`, le mot de passe est placé dans le corps de la requête, pas dans l'URL.

Corrigé

1. Trois composants : le champ de texte `champ_pseudo`, la liste déroulante `liste_note`, le bouton `bouton_envoyer`.

2. Champ de texte : événement de **saisie** (l'utilisateur tape du texte). Liste déroulante : événement de **changement** (l'utilisateur choisit une option). Bouton : événement de **clic** (ou de soumission du formulaire).

3. Non. Le code HTML décrit uniquement la **structure** et l'**apparence** des composants ; il ne contient **aucun** gestionnaire d'événement. Sans code JavaScript associant une fonction à l'événement clic du bouton, rien de particulier ne se produit à part la soumission par défaut du formulaire (rechargement de la page).

Corrigé

1.

```
1  etat = {"visible": False}
2
3  def basculer_affichage():
4      etat["visible"] = not etat["visible"]
5      return etat["visible"]
```

2.

```
1  ajouter_gestionnaire("bouton_menu", basculer_affichage)
```

3. La séquence est `[True, False, True]` : le premier clic affiche le menu (True), le deuxième le masque (False), le troisième le réaffiche (True).

Corrigé

1. La requête pour `/accueil` ne contient **aucun** cookie : `{"url": "/accueil", "cookies": {}}`.

2. La requête pour `/paiement` contient le cookie panier : `{"url": "/paiement", "cookies": {"panier": "2 articles"}}`.

3. Non : le serveur n'a pas besoin de conserver lui-même l'état du panier de chaque visiteur. C'est le **navigateur** qui mémorise le cookie et le **retransmet automatiquement** à chaque requête vers ce site : le serveur retrouve ainsi l'information sans avoir eu à la stocker de son côté.

Corrigé

1. Non, cela ne suffit pas. HTTPS protège uniquement les données **pendant leur transmission** sur le réseau (personne ne peut les intercepter en chemin). Cela ne dit rien sur la façon dont l'URL est enregistrée dans l'**historique du navigateur**, dans les **journaux du serveur**, ou sur la sécurité du **stockage** des données une fois reçues par le serveur.

2.

```

1 from urllib.parse import urlencode
2
3 donnees = {"carte": "1234567812345678", "code": "123"}
4 url_si_get = "https://banque.fr/paiement?" + urlencode(donnees)
5 print(url_si_get)

```

Oui, le numéro de carte complet apparaîtrait en clair dans l'URL, visible dans l'historique du navigateur et potentiellement dans les journaux du serveur.

3. Il faut utiliser **POST** : les données (numéro de carte, code de sécurité) sont confidentielles, et POST les place dans le corps de la requête plutôt que dans l'URL. Elles n'apparaissent donc pas dans l'historique des URL du navigateur. En revanche, un serveur web, un proxy ou l'application peuvent néanmoins journaliser le corps de la requête. Il faut aussi utiliser HTTPS, minimiser les données collectées et appliquer une politique de journalisation qui protège et limite ces enregistrements.

Corrigé

1.

```

1 from urllib.parse import urlencode
2
3 params = {"destination": "Djerba", "voyageurs": 2, "tri": "prix_croissant"}
4 url = "https://voyages.fr/recherche?" + urlencode(params)
5 print(url)
6 # https://voyages.fr/recherche?destination=Djerba&voyageurs=2&tri=prix_croissant

```

2. Oui : avec GET, les paramètres de recherche font partie intégrante de l'URL. Mettre cette URL en favori (ou la partager) permet de retrouver exactement la même recherche plus tard, car tous les critères sont encodés dans l'adresse elle-même.

3. Non, ce n'est pas justifié ici : les données transmises (destination, nombre de voyageurs, critère de tri) ne sont **pas confidentielles** — ce sont des critères de recherche publics, pas des mots de passe ni des informations sensibles. GET est adapté, et présente même un avantage (URL partageable/favorisable) que POST n'offre pas.

Corrigé

1.

```

1 from urllib.parse import urlencode
2
3 donnees = {"nouveau_mdp": "MonSecret2026"}
4 url = "https://site.fr/nouveau_mdp?" + urlencode(donnees)
5 print(url)
6 # https://site.fr/nouveau_mdp?nouveau_mdp=MonSecret2026

```

2. Oui, le mot de passe apparaît **en clair** dans l'URL. Cette URL risque d'être enregistrée dans l'**historique du navigateur** et dans les **journaux (logs) du serveur**, deux endroits où un mot de passe ne devrait jamais apparaître, même sur une connexion HTTPS (qui protège seulement la transmission, pas ces enregistrements).

3.

```

<form action="/nouveau_mdp" method="POST">
  <input type="password" name="nouveau_mdp">
  <button type="submit">Valider</button>
</form>

```

Avec `method="POST"`, le mot de passe est placé dans le corps de la requête, pas dans l'URL.

Corrigé

1. Trois composants : le champ de texte `champ_pseudo`, la liste déroulante `liste_note`, le bouton `bouton_envoyer`.

2. Champ de texte : événement de **saisie** (l'utilisateur tape du texte). Liste déroulante : événement de **changement** (l'utilisateur choisit une option). Bouton : événement de **clic** (ou de soumission du formulaire).

3. Non. Le code HTML décrit uniquement la **structure** et l'**apparence** des composants ; il ne contient **aucun** gestionnaire d'événement. Sans code JavaScript associant une fonction à l'événement clic du bouton, rien de particulier ne se produit à part la soumission par défaut du formulaire (rechargement de la page).

Corrigé

1.

```
1 etat = {"visible": False}
2
3 def basculer_affichage():
4     etat["visible"] = not etat["visible"]
5     return etat["visible"]
```

2.

```
1 ajouter_gestionnaire("bouton_menu", basculer_affichage)
```

3. La séquence est [True, False, True] : le premier clic affiche le menu (True), le deuxième le masque (False), le troisième le réaffiche (True).

Corrigé

1.

```
1 from urllib.parse import urlencode
2
3 donnees = {"nouveau_mdp": "MonSecret2026"}
4 url = "https://site.fr/nouveau_mdp?" + urlencode(donnees)
5 print(url)
6 # https://site.fr/nouveau_mdp?nouveau_mdp=MonSecret2026
```

2. Oui, le mot de passe apparaît **en clair** dans l'URL. Cette URL risque d'être enregistrée dans l'**historique du navigateur** et dans les **journaux (logs) du serveur**, deux endroits où un mot de passe ne devrait jamais apparaître, même sur une connexion HTTPS (qui protège seulement la transmission, pas ces enregistrements).

3.

```
<form action="/nouveau_mdp" method="POST">
  <input type="password" name="nouveau_mdp">
  <button type="submit">Valider</button>
</form>
```

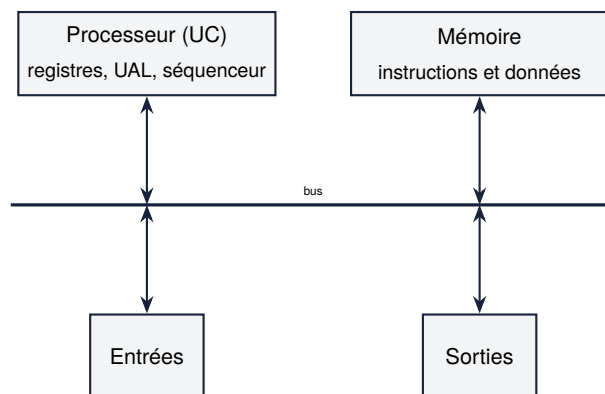
Avec `method="POST"`, le mot de passe est placé dans le corps de la requête, pas dans l'URL.

8 Architecture de von Neumann et systèmes d'exploitation

8.1 L'architecture de von Neumann

DÉFINITION D1

L'**architecture de von Neumann** décrit une machine composée de quatre constituants principaux, reliés par un **bus** : le **processeur** (ou UC, unité centrale), la **mémoire**, et les périphériques d'**entrée-sortie**.



◆ **PROPRIÉTÉ Rôle de chaque constituant** Le **processeur** exécute les instructions, une par une : il lit une instruction en mémoire, la decode, l'exécute, puis passe à la suivante. La **mémoire** stocke à la fois les **instructions** du programme et les **données** qu'il manipule (c'est le principe clé de von Neumann : un même espace mémoire pour le code et les données). Les **entrées-sorties** permettent à la machine de communiquer avec l'extérieur (clavier, écran, disque, réseau).

DÉFINITION D2

À l'intérieur du processeur, les **registres** sont de petites mémoires très rapides qui stockent temporairement les valeurs en cours de traitement. L'**UAL** (unité arithmétique et logique) effectue les calculs (addition, comparaison...). Le **séquenceur** pilote l'enchaînement des étapes d'exécution.

◆ **PROPRIÉTÉ Monoprocasseur et multiprocasseur** Une architecture **monoprocasseur** ne dispose que d'un seul processeur : les instructions s'exécutent **une à la fois**, séquentiellement. Une architecture **multiprocasseur** dispose de plusieurs processeurs (ou plusieurs *cœurs*), qui peuvent exécuter des instructions **en parallèle**.

⚠ ERREUR FRÉQUENTE

Croire que le processeur « sait » distinguer une instruction d'une donnée en mémoire. Dans l'architecture de von Neumann, instructions et données sont stockées dans le **même espace mémoire**, sous forme de suites de bits identiques dans leur nature : c'est uniquement le **contexte de lecture** (est-ce le séquenceur qui lit à cette adresse pour exécuter, ou l'UAL qui lit une donnée pour calculer ?) qui détermine comment ces bits sont interprétés.

» **Python À la machine.** Recherche le nombre de cœurs du processeur de l'ordinateur que tu utilises (par exemple avec la commande `nproc` sous Linux, ou dans les informations système). Ce processeur est-il monoprocesseur ou multiprocesseur au sens du cours ?

8.2 Dérouler l'exécution d'instructions machine

DÉFINITION D3

Une **instruction machine** est une opération élémentaire que le processeur sait exécuter directement : charger une valeur depuis la mémoire dans un registre, effectuer un calcul entre registres, écrire un registre en mémoire, etc. Un **programme machine** est une suite d'instructions, exécutées **dans l'ordre**, une par une.

EXEMPLE Un jeu d'instructions minimal, inspiré du langage machine :

- ▶ ("LOAD", registre, adresse) : charge la valeur de la mémoire à adresse dans registre.
- ▶ ("ADD", r1, r2) : ajoute le contenu de r2 à r1 (résultat dans r1).
- ▶ ("STORE", registre, adresse) : écrit le contenu de registre en mémoire à adresse.
- ▶ ("HALT",) : arrête l'exécution.

CODE DE RÉFÉRENCE — Simuler l'exécution d'un programme machine

```

1 def executer(programme, memoire):
2     """Execute un programme machine simplifie, instruction par instruction."""
3     registres = {}
4     for instruction in programme:
5         op = instruction[0]
6         if op == "LOAD":
7             _, reg, adresse = instruction
8             registres[reg] = memoire[adresse]
9         elif op == "ADD":
10            _, reg_dest, reg_src = instruction
11            registres[reg_dest] = registres[reg_dest] + registres[reg_src]
12        elif op == "STORE":
13            _, reg, adresse = instruction
14            memoire[adresse] = registres[reg]
15        elif op == "HALT":
16            break
17    return memoire, registres
18
19 memoire = [5, 3, 0] # memoire[0]=5, memoire[1]=3, memoire[2]=0
20 programme = [
21     ("LOAD", "R0", 0), # R0 <- memoire[0] = 5
22     ("LOAD", "R1", 1), # R1 <- memoire[1] = 3
23     ("ADD", "R0", "R1"), # R0 <- R0 + R1 = 8

```

```

24     ("STORE", "R0", 2), # memoire[2] <- R0 = 8
25     ("HALT",),
26 ]
27 memoire_finale, registres = executer(programme, memoire)
28 print(memoire_finale) # [5, 3, 8]
29 print(registres)      # {'R0': 8, 'R1': 3}

```

◆ **PROPRIÉTÉ Dérouler une exécution** Dérouler l'exécution d'un programme consiste à indiquer, après **chaque** instruction, l'état des registres et de la mémoire à cet instant précis — c'est ce qui permet de vérifier ou de prédire le résultat d'un programme sans avoir besoin de l'exécuter sur une vraie machine.

⚠ ERREUR FRÉQUENTE

Oublier que les instructions s'exécutent **dans l'ordre d'écriture** du programme, une à la fois. Un déroulé d'exécution qui « saute » une instruction ou en anticipe une plus loin (sans branchement explicite) est incorrect.

» **Python À la machine.** Ajoute une instruction ("SUB", r1, r2) (soustraction : $r1 \leftarrow r1 - r2$) au simulateur executer. Écris un programme qui calcule $\text{memoire}[0] - \text{memoire}[1]$ et le stocke dans $\text{memoire}[2]$, pour $\text{memoire} = [10, 4, 0]$.

8.3 Systèmes d'exploitation

DÉFINITION D4

Un **système d'exploitation** (OS, *operating system*) est le logiciel qui gère les ressources matérielles d'une machine (processeur, mémoire, disque, périphériques) et fournit une interface pour exécuter d'autres programmes.

◆ **PROPRIÉTÉ Fonctions d'un système d'exploitation** Un système d'exploitation assure notamment : la **gestion des processus** (démarrer, arrêter, partager le temps processeur entre plusieurs programmes), la **gestion de la mémoire** (allouer de l'espace à chaque programme), la **gestion des fichiers** (organiser le stockage sur disque), et la **gestion des droits** (contrôler qui a le droit de faire quoi).

⚠ ERREUR FRÉQUENTE

Confondre un système d'exploitation avec une simple « interface graphique ». L'interface graphique (bureau, fenêtres, icônes) n'est qu'une des façons de présenter le système d'exploitation à l'utilisateur : le système d'exploitation lui-même fonctionne indépendamment de la présence ou non d'une interface graphique (un serveur peut tourner sans aucun affichage graphique).

◆ **PROPRIÉTÉ Systèmes libres et propriétaires** Un système d'exploitation **libre** (comme Linux) met à disposition son code source, modifiable et redistribuable librement. Un système **propriétaire** (comme Windows ou macOS) ne diffuse pas son code source. Les élèves de NSI utilisent généralement un système libre en travaux pratiques.

8.4 La ligne de commande

DÉFINITION D5

La **ligne de commande** (ou *terminal*, ou *shell*) permet de piloter le système d'exploitation en tapant des commandes textuelles, plutôt qu'en cliquant sur des icônes.

◆ PROPRIÉTÉ Commandes de base sur fichiers et dossiers

ls	lister le contenu d'un dossier
cd chemin	se déplacer dans un dossier
mkdir nom	créer un dossier
touch fichier	créer un fichier vide
rm fichier	supprimer un fichier
cat fichier	afficher le contenu d'un fichier

CODE DE RÉFÉRENCE — Exécuter de vraies commandes shell depuis Python

```
1 import subprocess
2
3 subprocess.run(["mkdir", "dossier_test"])
4 subprocess.run(["touch", "dossier_test/notes.txt"])
5 resultat = subprocess.run(["ls", "dossier_test"], capture_output=True, text=True)
6 print(resultat.stdout) # notes.txt
```

8.5 Droits et permissions d'accès

DÉFINITION D6

Sous un système de type Unix (Linux, macOS), chaque fichier a trois catégories d'utilisateurs associées à des droits : le **propriétaire**, le **groupe**, et **les autres**. Pour chaque catégorie, trois droits peuvent être accordés ou refusés : **lecture** (r), **écriture** (w), **exécution** (x).

EXEMPLE La chaîne "rwxr-xr--" se lit par blocs de 3 caractères : rwx (propriétaire : lecture, écriture, exécution), r-x (groupe : lecture, exécution, pas d'écriture), r-- (autres : lecture seule).

CODE DE RÉFÉRENCE — Vérifier un droit d'accès à partir de sa représentation

```
1 def droit_accorde(permissions, categorie, action):
2     """Teste si 'action' est autorisee pour 'categorie' selon 'permissions'.
3
4     permissions : chaine de 9 caracteres, ex "rwxr-xr--".
```

```

5     categorie : "proprietaire", "groupe" ou "autres".
6     action : "lecture", "écriture" ou "exécution".
7     """
8     index = {"proprietaire": 0, "groupe": 3, "autres": 6}
9     decalage = {"lecture": 0, "écriture": 1, "exécution": 2}
10    position = index[categorie] + decalage[action]
11    return permissions[position] != "-"
12
13    perm = "rwxr-xr--"
14    print(droit_accorde(perm, "proprietaire", "écriture")) # True
15    print(droit_accorde(perm, "autres", "écriture"))      # False

```

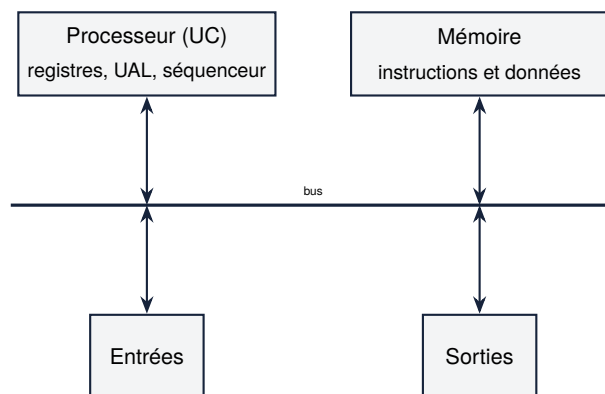
◆ **PROPRIÉTÉ Notation octale** Chaque bloc de 3 droits (lecture=4, écriture=2, exécution=1) se résume en un **chiffre octal** égal à la somme des droits accordés. rwx vaut $4 + 2 + 1 = 7$, r-x vaut $4 + 0 + 1 = 5$, r-- vaut 4 : on retrouve la notation chmod 754 bien connue des utilisateurs de Linux.

» **Python À la machine.** Dans un terminal Linux, exécute la commande `ls -l` dans un dossier de ton choix : observe la colonne de permissions (par exemple `-rw-r--r--`). Identifie le propriétaire, le groupe, et les droits accordés aux autres utilisateurs pour un des fichiers listés.

8.6 L'architecture de von Neumann

DÉFINITION D1

L'**architecture de von Neumann** décrit une machine composée de quatre constituants principaux, reliés par un **bus** : le **processeur** (ou UC, unité centrale), la **mémoire**, et les périphériques d'**entrée-sortie**.



◆ **PROPRIÉTÉ Rôle de chaque constituant** Le **processeur** exécute les instructions, une par une : il lit une instruction en mémoire, la decode, l'exécute, puis passe à la suivante. La **mémoire** stocke à la fois les **instructions** du programme et les **données** qu'il manipule (c'est le principe clé de von Neumann : un même espace mémoire pour le code et les données). Les **entrées-sorties** permettent à la machine de communiquer avec l'extérieur (clavier, écran, disque, réseau).

DÉFINITION D2

À l'intérieur du processeur, les **registres** sont de petites mémoires très rapides qui stockent temporairement les valeurs en cours de traitement. L'**UAL** (unité arithmétique et logique) effectue les calculs (addition, comparaison...). Le **séquenceur** pilote l'enchaînement des étapes d'exécution.

◆ **PROPRIÉTÉ Monoprocasseur et multiprocasseur** Une architecture **monoprocasseur** ne dispose que d'un seul processeur : les instructions s'exécutent **une à la fois**, séquentiellement. Une architecture **multiprocasseur** dispose de plusieurs processeurs (ou plusieurs *cœurs*), qui peuvent exécuter des instructions **en parallèle**.

⚠ ERREUR FRÉQUENTE

Croire que le processeur « sait » distinguer une instruction d'une donnée en mémoire. Dans l'architecture de von Neumann, instructions et données sont stockées dans le **même espace mémoire**, sous forme de suites de bits identiques dans leur nature : c'est uniquement le **contexte de lecture** (est-ce le séquenceur qui lit à cette adresse pour exécuter, ou l'UAL qui lit une donnée pour calculer ?) qui détermine comment ces bits sont interprétés.

» **Python À la machine.** Recherche le nombre de cœurs du processeur de l'ordinateur que tu utilises (par exemple avec la commande `nproc` sous Linux, ou dans les informations système). Ce processeur est-il monoprocasseur ou multiprocasseur au sens du cours ?

8.7 Dérouler l'exécution d'instructions machine

DÉFINITION D3

Une **instruction machine** est une opération élémentaire que le processeur sait exécuter directement : charger une valeur depuis la mémoire dans un registre, effectuer un calcul entre registres, écrire un registre en mémoire, etc. Un **programme machine** est une suite d'instructions, exécutées **dans l'ordre**, une par une.

EXEMPLE Un jeu d'instructions minimal, inspiré du langage machine :

- ▶ ("LOAD", registre, adresse) : charge la valeur de la mémoire à adresse dans registre.
- ▶ ("ADD", r1, r2) : ajoute le contenu de r2 à r1 (résultat dans r1).
- ▶ ("STORE", registre, adresse) : écrit le contenu de registre en mémoire à adresse.
- ▶ ("HALT",) : arrête l'exécution.

CODE DE RÉFÉRENCE — Simuler l'exécution d'un programme machine

```

1 def executer(programme, memoire):
2     """Execute un programme machine simplifie, instruction par instruction."""
3     registres = {}
4     for instruction in programme:
5         op = instruction[0]
6         if op == "LOAD":
7             _, reg, adresse = instruction

```

```

8     registres[reg] = memoire[adresse]
9     elif op == "ADD":
10        _, reg_dest, reg_src = instruction
11        registres[reg_dest] = registres[reg_dest] + registres[reg_src]
12    elif op == "STORE":
13        _, reg, adresse = instruction
14        memoire[adresse] = registres[reg]
15    elif op == "HALT":
16        break
17    return memoire, registres
18
19 memoire = [5, 3, 0] # memoire[0]=5, memoire[1]=3, memoire[2]=0
20 programme = [
21     ("LOAD", "R0", 0), # R0 <- memoire[0] = 5
22     ("LOAD", "R1", 1), # R1 <- memoire[1] = 3
23     ("ADD", "R0", "R1"), # R0 <- R0 + R1 = 8
24     ("STORE", "R0", 2), # memoire[2] <- R0 = 8
25     ("HALT",),
26 ]
27 memoire_finale, registres = executer(programme, memoire)
28 print(memoire_finale) # [5, 3, 8]
29 print(registres) # {'R0': 8, 'R1': 3}

```

◆ **PROPRIÉTÉ Dérouler une exécution** Dérouler l'exécution d'un programme consiste à indiquer, après **chaque** instruction, l'état des registres et de la mémoire à cet instant précis — c'est ce qui permet de vérifier ou de prédire le résultat d'un programme sans avoir besoin de l'exécuter sur une vraie machine.

⚠ ERREUR FRÉQUENTE

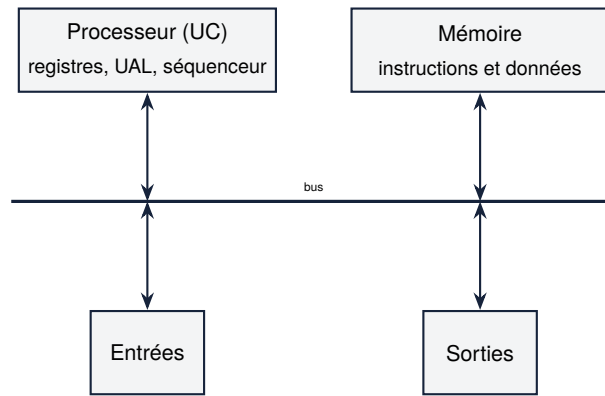
Oublier que les instructions s'exécutent **dans l'ordre d'écriture** du programme, une à la fois. Un déroulé d'exécution qui « saute » une instruction ou en anticipe une plus loin (sans branchement explicite) est incorrect.

» **Python À la machine.** Ajoute une instruction ("SUB", r1, r2) (soustraction : $r1 \leftarrow r1 - r2$) au simulateur executer. Écris un programme qui calcule $\text{memoire}[0] - \text{memoire}[1]$ et le stocke dans $\text{memoire}[2]$, pour $\text{memoire} = [10, 4, 0]$.

8.8 L'architecture de von Neumann

DÉFINITION D1

L'**architecture de von Neumann** décrit une machine composée de quatre constituants principaux, reliés par un **bus** : le **processeur** (ou UC, unité centrale), la **mémoire**, et les périphériques d'**entrée-sortie**.



◆ **PROPRIÉTÉ Rôle de chaque constituant** Le **processeur** exécute les instructions, une par une : il lit une instruction en mémoire, la decode, l'exécute, puis passe à la suivante. La **mémoire** stocke à la fois les **instructions** du programme et les **données** qu'il manipule (c'est le principe clé de von Neumann : un même espace mémoire pour le code et les données). Les **entrées-sorties** permettent à la machine de communiquer avec l'extérieur (clavier, écran, disque, réseau).

DÉFINITION D2

À l'intérieur du processeur, les **registres** sont de petites mémoires très rapides qui stockent temporairement les valeurs en cours de traitement. L'**UAL** (unité arithmétique et logique) effectue les calculs (addition, comparaison...). Le **séquenceur** pilote l'enchaînement des étapes d'exécution.

◆ **PROPRIÉTÉ Monoprocasseur et multiprocasseur** Une architecture **monoprocasseur** ne dispose que d'un seul processeur : les instructions s'exécutent **une à la fois**, séquentiellement. Une architecture **multiprocasseur** dispose de plusieurs processeurs (ou plusieurs *cœurs*), qui peuvent exécuter des instructions **en parallèle**.

⚠ ERREUR FRÉQUENTE

Croire que le processeur « sait » distinguer une instruction d'une donnée en mémoire. Dans l'architecture de von Neumann, instructions et données sont stockées dans le **même espace mémoire**, sous forme de suites de bits identiques dans leur nature : c'est uniquement le **contexte de lecture** (est-ce le séquenceur qui lit à cette adresse pour exécuter, ou l'UAL qui lit une donnée pour calculer ?) qui détermine comment ces bits sont interprétés.

» **Python À la machine.** Recherche le nombre de cœurs du processeur de l'ordinateur que tu utilises (par exemple avec la commande `nproc` sous Linux, ou dans les informations système). Ce processeur est-il monoprocasseur ou multiprocasseur au sens du cours ?

8.9 Dérouler l'exécution d'instructions machine

DÉFINITION D3

Une **instruction machine** est une opération élémentaire que le processeur sait exécuter directement : charger une valeur depuis la mémoire dans un registre, effectuer un calcul entre registres, écrire un registre en mémoire, etc. Un **programme machine** est une suite d'instructions, exécutées **dans l'ordre**, une par une.

EXEMPLE Un jeu d'instructions minimal, inspiré du langage machine :

- ▶ ("LOAD", registre, adresse) : charge la valeur de la mémoire à adresse dans registre.
- ▶ ("ADD", r1, r2) : ajoute le contenu de r2 à r1 (résultat dans r1).
- ▶ ("STORE", registre, adresse) : écrit le contenu de registre en mémoire à adresse.
- ▶ ("HALT",) : arrête l'exécution.

CODE DE RÉFÉRENCE — Simuler l'exécution d'un programme machine

```

1 def executer(programme, memoire):
2     """Execute un programme machine simplifie, instruction par instruction."""
3     registres = {}
4     for instruction in programme:
5         op = instruction[0]
6         if op == "LOAD":
7             _, reg, adresse = instruction
8             registres[reg] = memoire[adresse]
9         elif op == "ADD":
10            _, reg_dest, reg_src = instruction
11            registres[reg_dest] = registres[reg_dest] + registres[reg_src]
12        elif op == "STORE":
13            _, reg, adresse = instruction
14            memoire[adresse] = registres[reg]
15        elif op == "HALT":
16            break
17    return memoire, registres
18
19 memoire = [5, 3, 0] # memoire[0]=5, memoire[1]=3, memoire[2]=0
20 programme = [
21     ("LOAD", "R0", 0), # R0 <- memoire[0] = 5
22     ("LOAD", "R1", 1), # R1 <- memoire[1] = 3
23     ("ADD", "R0", "R1"), # R0 <- R0 + R1 = 8
24     ("STORE", "R0", 2), # memoire[2] <- R0 = 8
25     ("HALT",),
26 ]
27 memoire_finale, registres = executer(programme, memoire)
28 print(memoire_finale) # [5, 3, 8]
29 print(registres) # {'R0': 8, 'R1': 3}

```

◆ **PROPRIÉTÉ Dérouler une exécution** Dérouler l'exécution d'un programme consiste à indiquer, après **chaque** instruction, l'état des registres et de la mémoire à cet instant précis — c'est ce qui permet de vérifier ou de prédire le résultat d'un programme sans avoir besoin de l'exécuter sur une vraie machine.

⚠ ERREUR FRÉQUENTE

Oublier que les instructions s'exécutent **dans l'ordre d'écriture** du programme, une à la fois. Un déroulé d'exécution qui « saute » une instruction ou en anticipe une plus loin (sans branchement explicite) est incorrect.

» **Python À la machine.** Ajoute une instruction ("SUB", r1, r2) (soustraction : $r1 \leftarrow r1 - r2$) au simulateur exécuter. Écris un programme qui calcule `memoire[0] - memoire[1]` et le stocke dans `memoire[2]`, pour `memoire = [10, 4, 0]`.

8.10 Systèmes d'exploitation

DÉFINITION D4

Un **système d'exploitation** (OS, *operating system*) est le logiciel qui gère les ressources matérielles d'une machine (processeur, mémoire, disque, périphériques) et fournit une interface pour exécuter d'autres programmes.

♦ **PROPRIÉTÉ Fonctions d'un système d'exploitation** Un système d'exploitation assure notamment : la **gestion des processus** (démarrer, arrêter, partager le temps processeur entre plusieurs programmes), la **gestion de la mémoire** (allouer de l'espace à chaque programme), la **gestion des fichiers** (organiser le stockage sur disque), et la **gestion des droits** (contrôler qui a le droit de faire quoi).

⚠ ERREUR FRÉQUENTE

Confondre un système d'exploitation avec une simple « interface graphique ». L'interface graphique (bureau, fenêtres, icônes) n'est qu'une des façons de présenter le système d'exploitation à l'utilisateur : le système d'exploitation lui-même fonctionne indépendamment de la présence ou non d'une interface graphique (un serveur peut tourner sans aucun affichage graphique).

♦ **PROPRIÉTÉ Systèmes libres et propriétaires** Un système d'exploitation **libre** (comme Linux) met à disposition son code source, modifiable et redistribuable librement. Un système **propriétaire** (comme Windows ou macOS) ne diffuse pas son code source. Les élèves de NSI utilisent généralement un système libre en travaux pratiques.

8.11 La ligne de commande

DÉFINITION D5

La **ligne de commande** (ou *terminal*, ou *shell*) permet de piloter le système d'exploitation en tapant des commandes textuelles, plutôt qu'en cliquant sur des icônes.

◆ PROPRIÉTÉ Commandes de base sur fichiers et dossiers

ls	lister le contenu d'un dossier
cd chemin	se déplacer dans un dossier
mkdir nom	créer un dossier
touch fichier	créer un fichier vide
rm fichier	supprimer un fichier
cat fichier	afficher le contenu d'un fichier

CODE DE RÉFÉRENCE — Exécuter de vraies commandes shell depuis Python

```

1 import subprocess
2
3 subprocess.run(["mkdir", "dossier_test"])
4 subprocess.run(["touch", "dossier_test/notes.txt"])
5 resultat = subprocess.run(["ls", "dossier_test"], capture_output=True, text=True)
6 print(resultat.stdout) # notes.txt

```

8.12 Droits et permissions d'accès

DÉFINITION D6

Sous un système de type Unix (Linux, macOS), chaque fichier a trois catégories d'utilisateurs associées à des droits : le **propriétaire**, le **groupe**, et **les autres**. Pour chaque catégorie, trois droits peuvent être accordés ou refusés : **lecture** (r), **écriture** (w), **exécution** (x).

EXEMPLE La chaîne "rwxr-xr--" se lit par blocs de 3 caractères : rwx (propriétaire : lecture, écriture, exécution), r-x (groupe : lecture, exécution, pas d'écriture), r-- (autres : lecture seule).

CODE DE RÉFÉRENCE — Vérifier un droit d'accès à partir de sa représentation

```

1 def droit_accorde(permissions, categorie, action):
2     """Teste si 'action' est autorisée pour 'categorie' selon 'permissions'.
3
4     permissions : chaîne de 9 caractères, ex "rwxr-xr--".
5     categorie : "propriétaire", "groupe" ou "autres".
6     action : "lecture", "écriture" ou "exécution".
7     """
8     index = {"propriétaire": 0, "groupe": 3, "autres": 6}
9     decalage = {"lecture": 0, "écriture": 1, "exécution": 2}
10    position = index[categorie] + decalage[action]
11    return permissions[position] != "-"
12
13 perm = "rwxr-xr--"
14 print(droit_accorde(perm, "propriétaire", "écriture")) # True
15 print(droit_accorde(perm, "autres", "écriture")) # False

```

◆ **PROPRIÉTÉ Notation octale** Chaque bloc de 3 droits (lecture=4, écriture=2, exécution=1) se résume en un **chiffre octal** égal à la somme des droits accordés. rwx vaut $4 + 2 + 1 = 7$, r-x

vaut $4 + 0 + 1 = 5$, $r--$ vaut 4 : on retrouve la notation `chmod 754` bien connue des utilisateurs de Linux.

» **Python À la machine.** Dans un terminal Linux, exécute la commande `ls -l` dans un dossier de ton choix : observe la colonne de permissions (par exemple `-rw-r--r--`). Identifie le propriétaire, le groupe, et les droits accordés aux autres utilisateurs pour un des fichiers listés.

Méthodes

► Méthode directe de travail sur le sujet.

Exercice 1 ♦

1. Dans l'architecture de von Neumann, quel constituant exécute les instructions, et quel constituant les stocke (avec les données) ?
2. Un ordinateur a 8 cœurs. Peut-il exécuter plusieurs instructions de programmes différents **en même temps** ? Justifier en citant le vocabulaire du cours (mono/multiprocesseur).
3. Pourquoi le processeur ne peut-il pas, en lisant une suite de bits en mémoire, savoir « à l'avance » s'il s'agit d'une instruction ou d'une donnée ?

Exercice 2 ♦

On reprend le simulateur exécuter du cours.

1. Ajouter à exécuter le traitement de l'instruction ("`SUB`", `reg_dest`, `reg_src`), qui effectue `reg_dest <- reg_dest - reg_src`.
2. Dérouler « à la main » (sur papier) l'exécution du programme suivant, en indiquant l'état de registres après chaque instruction :

```
1 memoire = [10, 4, 0]
2 programme = [
3     ("LOAD", "R0", 0),
4     ("LOAD", "R1", 1),
5     ("SUB", "R0", "R1"),
6     ("STORE", "R0", 2),
7     ("HALT", ),
8 ]
```

3. Vérifier ta réponse en exécutant le programme avec ta fonction modifiée.

Exercice 3 ♦♦

On ouvre simultanément un navigateur Web, un traitement de texte et un lecteur de musique sur un ordinateur à un seul cœur.

1. Ces trois programmes semblent s'exécuter « en même temps ». Est-ce réellement le cas sur un processeur monoprocesseur ? Quelle fonction du système d'exploitation explique ce que l'utilisateur observe ?
2. Le lecteur de musique doit pouvoir écrire un fichier de log sur le disque, mais pas lire les fichiers personnels du traitement de texte. Quelle fonction du système d'exploitation est concernée ?
3. Linux est un système d'exploitation libre. Qu'est-ce que cela signifie concrètement, par opposition à un système propriétaire ?

Exercice 4 ♦♦

En t'aidant du tableau des commandes de base du cours (mkdir, touch, ls, rm), écrire (en Python, via subprocess.run, comme dans le cours) la séquence de commandes qui :

1. crée un dossier projet ;
2. crée dans ce dossier deux fichiers vides, main.py et readme.md ;
3. liste le contenu du dossier projet (vérifier que les deux fichiers y apparaissent) ;
4. supprime le fichier readme.md, puis liste à nouveau le contenu du dossier pour vérifier qu'il ne reste que main.py.

Exercice 5 ♦♦♦

Un fichier confidentiel a les permissions "rw-r-----".

1. En utilisant droit_accorde du cours, déterminer si le propriétaire peut exécuter ce fichier, si le groupe peut le lire, et si les autres utilisateurs peuvent le lire.
2. Convertir ces permissions en notation octale avec permissions_vers_octal.
3. Ce fichier est-il accessible en lecture par n'importe quel utilisateur du système ? Justifier, et proposer un scénario où de telles permissions seraient appropriées.

Exercice 6 ♦

1. Dans l'architecture de von Neumann, quel constituant exécute les instructions, et quel constituant les stocke (avec les données) ?
2. Un ordinateur a 8 cœurs. Peut-il exécuter plusieurs instructions de programmes différents **en même temps** ? Justifier en citant le vocabulaire du cours (mono/multiprocesseur).
3. Pourquoi le processeur ne peut-il pas, en lisant une suite de bits en mémoire, savoir « à l'avance » s'il s'agit d'une instruction ou d'une donnée ?

Exercice 7 ♦

On reprend le simulateur executer du cours.

1. Ajouter à executer le traitement de l'instruction ("SUB", reg_dest, reg_src), qui effectue $reg_dest \leftarrow reg_dest - reg_src$.
2. Dérouler « à la main » (sur papier) l'exécution du programme suivant, en indiquant l'état de registres après chaque instruction :

```

1 memoire = [10, 4, 0]
2 programme = [
3     ("LOAD", "R0", 0),
4     ("LOAD", "R1", 1),
5     ("SUB", "R0", "R1"),
6     ("STORE", "R0", 2),
7     ("HALT", ),
8 ]

```

3. Vérifier ta réponse en exécutant le programme avec ta fonction modifiée.

Exercice 8 ♦♦

On ouvre simultanément un navigateur Web, un traitement de texte et un lecteur de musique sur un ordinateur à un seul cœur.

1. Ces trois programmes semblent s'exécuter « en même temps ». Est-ce réellement le cas sur un processeur monoprocesseur ? Quelle fonction du système d'exploitation explique ce que l'utilisateur observe ?

2. Le lecteur de musique doit pouvoir écrire un fichier de log sur le disque, mais pas lire les fichiers personnels du traitement de texte. Quelle fonction du système d'exploitation est concernée ?
3. Linux est un système d'exploitation libre. Qu'est-ce que cela signifie concrètement, par opposition à un système propriétaire ?

Exercice 9 ♦♦

En t'aidant du tableau des commandes de base du cours (mkdir, touch, ls, rm), écrire (en Python, via subprocess.run, comme dans le cours) la séquence de commandes qui :

1. crée un dossier projet ;
2. crée dans ce dossier deux fichiers vides, main.py et readme.md ;
3. liste le contenu du dossier projet (vérifier que les deux fichiers y apparaissent) ;
4. supprime le fichier readme.md, puis liste à nouveau le contenu du dossier pour vérifier qu'il ne reste que main.py.

Exercice 10 ♦♦♦

Un fichier confidentiel a les permissions "rw-r-----".

1. En utilisant droit_accorde du cours, déterminer si le propriétaire peut exécuter ce fichier, si le groupe peut le lire, et si les autres utilisateurs peuvent le lire.
2. Convertir ces permissions en notation octale avec permissions_vers_octal.
3. Ce fichier est-il accessible en lecture par n'importe quel utilisateur du système ? Justifier, et proposer un scénario où de telles permissions seraient appropriées.

Exercice 11 ♦

1. Dans l'architecture de von Neumann, quel constituant exécute les instructions, et quel constituant les stocke (avec les données) ?
2. Un ordinateur a 8 cœurs. Peut-il exécuter plusieurs instructions de programmes différents **en même temps** ? Justifier en citant le vocabulaire du cours (mono/multiprocesseur).
3. Pourquoi le processeur ne peut-il pas, en lisant une suite de bits en mémoire, savoir « à l'avance » s'il s'agit d'une instruction ou d'une donnée ?

Exercice 12 ♦

On reprend le simulateur executer du cours.

1. Ajouter à executer le traitement de l'instruction ("SUB", reg_dest, reg_src), qui effectue $reg_dest \leftarrow reg_dest - reg_src$.
2. Dérouler « à la main » (sur papier) l'exécution du programme suivant, en indiquant l'état de registres après chaque instruction :

```
1 memoire = [10, 4, 0]
2 programme = [
3     ("LOAD", "R0", 0),
4     ("LOAD", "R1", 1),
5     ("SUB", "R0", "R1"),
6     ("STORE", "R0", 2),
7     ("HALT",),
8 ]
```

3. Vérifier ta réponse en exécutant le programme avec ta fonction modifiée.

Exercice 13 ♦♦

On ouvre simultanément un navigateur Web, un traitement de texte et un lecteur de musique sur un ordinateur à un seul cœur.

1. Ces trois programmes semblent s'exécuter « en même temps ». Est-ce réellement le cas sur un processeur monoprocesseur ? Quelle fonction du système d'exploitation explique ce que l'utilisateur observe ?
2. Le lecteur de musique doit pouvoir écrire un fichier de log sur le disque, mais pas lire les fichiers personnels du traitement de texte. Quelle fonction du système d'exploitation est concernée ?
3. Linux est un système d'exploitation libre. Qu'est-ce que cela signifie concrètement, par opposition à un système propriétaire ?

Exercice 14 ♦♦

En t'aidant du tableau des commandes de base du cours (mkdir, touch, ls, rm), écrire (en Python, via subprocess.run, comme dans le cours) la séquence de commandes qui :

1. crée un dossier projet ;
2. crée dans ce dossier deux fichiers vides, main.py et readme.md ;
3. liste le contenu du dossier projet (vérifier que les deux fichiers y apparaissent) ;
4. supprime le fichier readme.md, puis liste à nouveau le contenu du dossier pour vérifier qu'il ne reste que main.py.

Exercice 15 ♦♦♦

Un fichier confidentiel a les permissions "rw-r-----".

1. En utilisant droit_accorde du cours, déterminer si le propriétaire peut exécuter ce fichier, si le groupe peut le lire, et si les autres utilisateurs peuvent le lire.
2. Convertir ces permissions en notation octale avec permissions_vers_octal.
3. Ce fichier est-il accessible en lecture par n'importe quel utilisateur du système ? Justifier, et proposer un scénario où de telles permissions seraient appropriées.

Exercice 16 ♦

1. Dans l'architecture de von Neumann, quel constituant exécute les instructions, et quel constituant les stocke (avec les données) ?
2. Un ordinateur a 8 cœurs. Peut-il exécuter plusieurs instructions de programmes différents **en même temps** ? Justifier en citant le vocabulaire du cours (mono/multiprocesseur).
3. Pourquoi le processeur ne peut-il pas, en lisant une suite de bits en mémoire, savoir « à l'avance » s'il s'agit d'une instruction ou d'une donnée ?

Exercice 17 ♦

On reprend le simulateur executer du cours.

1. Ajouter à executer le traitement de l'instruction ("SUB", reg_dest, reg_src), qui effectue $reg_dest \leftarrow reg_dest - reg_src$.
2. Dérouler « à la main » (sur papier) l'exécution du programme suivant, en indiquant l'état de registres après chaque instruction :

```

1 memoire = [10, 4, 0]
2 programme = [
3     ("LOAD", "R0", 0),
4     ("LOAD", "R1", 1),
5     ("SUB", "R0", "R1"),
6     ("STORE", "R0", 2),
7     ("HALT", ),
8 ]

```

3. Vérifier ta réponse en exécutant le programme avec ta fonction modifiée.

Exercice 18 ♦♦

On ouvre simultanément un navigateur Web, un traitement de texte et un lecteur de musique sur un ordinateur à un seul cœur.

1. Ces trois programmes semblent s'exécuter « en même temps ». Est-ce réellement le cas sur un processeur monoprocesseur ? Quelle fonction du système d'exploitation explique ce que l'utilisateur observe ?
2. Le lecteur de musique doit pouvoir écrire un fichier de log sur le disque, mais pas lire les fichiers personnels du traitement de texte. Quelle fonction du système d'exploitation est concernée ?
3. Linux est un système d'exploitation libre. Qu'est-ce que cela signifie concrètement, par opposition à un système propriétaire ?

Exercice 19 ♦♦

En t'aidant du tableau des commandes de base du cours (mkdir, touch, ls, rm), écrire (en Python, via subprocess.run, comme dans le cours) la séquence de commandes qui :

1. crée un dossier projet ;
2. crée dans ce dossier deux fichiers vides, main.py et readme.md ;
3. liste le contenu du dossier projet (vérifier que les deux fichiers y apparaissent) ;
4. supprime le fichier readme.md, puis liste à nouveau le contenu du dossier pour vérifier qu'il ne reste que main.py.

Exercice 20 ♦♦♦

Un fichier confidentiel a les permissions "rw-r-----".

1. En utilisant droit_accorde du cours, déterminer si le propriétaire peut exécuter ce fichier, si le groupe peut le lire, et si les autres utilisateurs peuvent le lire.
2. Convertir ces permissions en notation octale avec permissions_vers_octal.
3. Ce fichier est-il accessible en lecture par n'importe quel utilisateur du système ? Justifier, et proposer un scénario où de telles permissions seraient appropriées.

Exercice 21 ♦

1. Dans l'architecture de von Neumann, quel constituant exécute les instructions, et quel constituant les stocke (avec les données) ?
2. Un ordinateur a 8 cœurs. Peut-il exécuter plusieurs instructions de programmes différents **en même temps** ? Justifier en citant le vocabulaire du cours (mono/multiprocesseur).
3. Pourquoi le processeur ne peut-il pas, en lisant une suite de bits en mémoire, savoir « à l'avance » s'il s'agit d'une instruction ou d'une donnée ?

Exercice 22 ♦

On reprend le simulateur executer du cours.

1. Ajouter à executer le traitement de l'instruction ("SUB", reg_dest, reg_src), qui effectue $reg_dest \leftarrow reg_dest - reg_src$.
2. Dérouler « à la main » (sur papier) l'exécution du programme suivant, en indiquant l'état de registres après chaque instruction :

```
1 memoire = [10, 4, 0]
2 programme = [
3     ("LOAD", "R0", 0),
4     ("LOAD", "R1", 1),
5     ("SUB", "R0", "R1"),
6     ("STORE", "R0", 2),
```



```

7   ("HALT",),
8   ]

```

3. Vérifier ta réponse en exécutant le programme avec ta fonction modifiée.

Exercice 23 ♦♦

On ouvre simultanément un navigateur Web, un traitement de texte et un lecteur de musique sur un ordinateur à un seul cœur.

1. Ces trois programmes semblent s'exécuter « en même temps ». Est-ce réellement le cas sur un processeur monoprocesseur ? Quelle fonction du système d'exploitation explique ce que l'utilisateur observe ?
2. Le lecteur de musique doit pouvoir écrire un fichier de log sur le disque, mais pas lire les fichiers personnels du traitement de texte. Quelle fonction du système d'exploitation est concernée ?
3. Linux est un système d'exploitation libre. Qu'est-ce que cela signifie concrètement, par opposition à un système propriétaire ?

Exercice 24 ♦♦

En t'aidant du tableau des commandes de base du cours (mkdir, touch, ls, rm), écrire (en Python, via subprocess.run, comme dans le cours) la séquence de commandes qui :

1. crée un dossier projet ;
2. crée dans ce dossier deux fichiers vides, main.py et readme.md ;
3. liste le contenu du dossier projet (vérifier que les deux fichiers y apparaissent) ;
4. supprime le fichier readme.md, puis liste à nouveau le contenu du dossier pour vérifier qu'il ne reste que main.py.

Exercice 25 ♦♦♦

Un fichier confidentiel a les permissions "rw-r-----".

1. En utilisant droit_accorde du cours, déterminer si le propriétaire peut exécuter ce fichier, si le groupe peut le lire, et si les autres utilisateurs peuvent le lire.
2. Convertir ces permissions en notation octale avec permissions_vers_octal.
3. Ce fichier est-il accessible en lecture par n'importe quel utilisateur du système ? Justifier, et proposer un scénario où de telles permissions seraient appropriées.

Exercice 26 ♦

1. Dans l'architecture de von Neumann, quel constituant exécute les instructions, et quel constituant les stocke (avec les données) ?
2. Un ordinateur a 8 cœurs. Peut-il exécuter plusieurs instructions de programmes différents **en même temps** ? Justifier en citant le vocabulaire du cours (mono/multiprocesseur).
3. Pourquoi le processeur ne peut-il pas, en lisant une suite de bits en mémoire, savoir « à l'avance » s'il s'agit d'une instruction ou d'une donnée ?

Exercice 27 ♦

On reprend le simulateur executer du cours.

1. Ajouter à executer le traitement de l'instruction ("SUB", reg_dest, reg_src), qui effectue $reg_dest \leftarrow reg_dest - reg_src$.
2. Dérouler « à la main » (sur papier) l'exécution du programme suivant, en indiquant l'état de registres après chaque instruction :

```

1 memoire = [10, 4, 0]
2 programme = [
3     ("LOAD", "R0", 0),
4     ("LOAD", "R1", 1),
5     ("SUB", "R0", "R1"),
6     ("STORE", "R0", 2),
7     ("HALT", ),
8 ]

```

3. Vérifier ta réponse en exécutant le programme avec ta fonction modifiée.

Exercice 28 ♦♦

On ouvre simultanément un navigateur Web, un traitement de texte et un lecteur de musique sur un ordinateur à un seul cœur.

1. Ces trois programmes semblent s'exécuter « en même temps ». Est-ce réellement le cas sur un processeur monoprocesseur ? Quelle fonction du système d'exploitation explique ce que l'utilisateur observe ?
2. Le lecteur de musique doit pouvoir écrire un fichier de log sur le disque, mais pas lire les fichiers personnels du traitement de texte. Quelle fonction du système d'exploitation est concernée ?
3. Linux est un système d'exploitation libre. Qu'est-ce que cela signifie concrètement, par opposition à un système propriétaire ?

Exercice 29 ♦♦

En t'aidant du tableau des commandes de base du cours (mkdir, touch, ls, rm), écrire (en Python, via subprocess.run, comme dans le cours) la séquence de commandes qui :

1. crée un dossier projet ;
2. crée dans ce dossier deux fichiers vides, main.py et readme.md ;
3. liste le contenu du dossier projet (vérifier que les deux fichiers y apparaissent) ;
4. supprime le fichier readme.md, puis liste à nouveau le contenu du dossier pour vérifier qu'il ne reste que main.py.

Exercice 30 ♦♦♦

Un fichier confidentiel a les permissions "rw-r-----".

1. En utilisant droit_accorde du cours, déterminer si le propriétaire peut exécuter ce fichier, si le groupe peut le lire, et si les autres utilisateurs peuvent le lire.
2. Convertir ces permissions en notation octale avec permissions_vers_octal.
3. Ce fichier est-il accessible en lecture par n'importe quel utilisateur du système ? Justifier, et proposer un scénario où de telles permissions seraient appropriées.

QCM d'auto-évaluation — Architecture de von Neumann et systèmes d'exploitation

Pour chaque question, une seule réponse est exacte. Entourez la lettre correspondante.

C1 — Architecture de von Neumann

1. [Q1] Dans l'architecture de von Neumann, instructions et données sont stockées :
A. dans deux mémoires physiquement séparées.

- B. dans le même espace mémoire.
- C. uniquement dans les registres du processeur.
- D. uniquement sur le disque dur.

C2 — Exécution d'instructions

2. [Q2] Les instructions d'un programme machine s'exécutent :
- A. dans un ordre aléatoire.
 - B. toutes en même temps.
 - C. une par une, dans l'ordre du programme (sauf branchement).
 - D. dans l'ordre inverse du programme.

C3 — Systèmes d'exploitation

3. [Q3] La gestion des processus par le système d'exploitation permet notamment de :
- A. donner l'illusion d'une exécution simultanée de plusieurs programmes sur un seul cœur.
 - B. empêcher tout programme de s'exécuter.
 - C. remplacer le processeur.
 - D. afficher uniquement des fenêtres graphiques.

C4 — Ligne de commande

4. [Q4] La commande qui liste le contenu d'un dossier est :
- A. cd
 - B. ls
 - C. mkdir
 - D. rm

C5 — Droits et permissions

5. [Q5] Dans la chaîne de permissions `rw-r-x---`, le **groupe** peut :
- A. lire et écrire, mais pas exécuter.
 - B. lire et exécuter, mais pas écrire.
 - C. tout faire (lecture, écriture, exécution).
 - D. rien faire du tout.
6. [Q6] La notation octale `644` correspond aux permissions :
- A. `rw-r--r--`
 - B. `rw-rwxrwx`
 - C. `r--r--r--`
 - D. `rw-rw-rw-`

Corrigé

1. Processeur, mémoire, entrées, sorties. La **mémoire** stocke à la fois les instructions et les données.
- 2.

```
1 def executer(programme, memoire):
2     registres = {}
3     for instruction in programme:
4         op = instruction[0]
```

```

5         if op == "LOAD":
6             _, reg, adresse = instruction
7             registres[reg] = memoire[adresse]
8         elif op == "MUL":
9             _, reg_dest, reg_src = instruction
10            registres[reg_dest] = registres[reg_dest] * registres[reg_src]
11        elif op == "STORE":
12            _, reg, adresse = instruction
13            memoire[adresse] = registres[reg]
14        elif op == "HALT":
15            break
16    return memoire, registres

```

3. Après LOAD R0,0 : R0=6. Après LOAD R1,1 : R1=7. Après MUL R0,R1 : R0=42. Après STORE R0,2 : memoire=[6, 7, 42].

Corrigé

1. Par exemple : gestion des processus (partage du temps processeur) et gestion des droits d'accès aux fichiers (autres réponses possibles : gestion de la mémoire, gestion des fichiers).
2. Le propriétaire **peut** exécuter (rwx). Le groupe **peut** exécuter (r-x). Les autres **ne peuvent pas** lire (---).
3. rwx=7, r-x=5, ---=0 : notation octale 750.

Évaluation A — Architecture et systèmes d'exploitation

Durée : 45 minutes. Ordinateur avec interpréteur Python autorisé.

Barème indicatif : Ex. 1 : 10 pts (von Neumann, instructions) ; Ex. 2 : 10 pts (OS, permissions)

Exercice 1 — Von Neumann et instructions machine

(10 points)

1. (3 pts) Nommer les quatre constituants principaux de l'architecture de von Neumann, et préciser lequel stocke à la fois instructions et données.
2. (4 pts) On ajoute au simulateur executer du cours l'instruction ("MUL", reg_dest, reg_src) (multiplication : reg_dest <- reg_dest * reg_src). Écrire le code de cette instruction.
3. (3 pts) Dérouler l'exécution de ce programme et donner l'état final de memoire :

```

1 memoire = [6, 7, 0]
2 programme = [
3     ("LOAD", "R0", 0),
4     ("LOAD", "R1", 1),
5     ("MUL", "R0", "R1"),
6     ("STORE", "R0", 2),
7     ("HALT",),
8 ]

```

Exercice 2 — Système d'exploitation et permissions

(10 points)

1. (3 pts) Citer deux fonctions assurées par un système d'exploitation.
2. (4 pts) Un fichier a les permissions "rwxr-x---". En utilisant droit_accorde du cours, déterminer si le propriétaire peut l'exécuter, si le groupe peut l'exécuter, et si les autres peuvent le lire.
3. (3 pts) Convertir ces permissions en notation octale avec permissions_vers_octal.

Corrigé

1. Processeur, mémoire, entrées, sorties. Le **processeur** exécute les instructions.
- 2.

```

1 def executer(programme, memoire):
2     registres = {}
3     for instruction in programme:
4         op = instruction[0]
5         if op == "LOAD":
6             _, reg, adresse = instruction
7             registres[reg] = memoire[adresse]
8         elif op == "SUB":
9             _, reg_dest, reg_src = instruction
10            registres[reg_dest] = registres[reg_dest] - registres[reg_src]
11        elif op == "STORE":
12            _, reg, adresse = instruction
13            memoire[adresse] = registres[reg]
14        elif op == "HALT":
15            break
16    return memoire, registres

```

3. Après LOAD R0,0 : R0=20. Après LOAD R1,1 : R1=8. Après SUB R0,R1 : R0=12. Après STORE R0,2 : memoire=[20, 8, 12].

Corrigé

1. Par exemple : gestion de la mémoire (allouer de l'espace à chaque programme) et gestion des fichiers (organiser le stockage sur disque).
2. Le propriétaire **peut** écrire (rw-). Le groupe **peut** écrire (rw-). Les autres **ne peuvent pas** écrire (r--).
3. rw--=6, rw--=6, r--=4 : notation octale 664.

Évaluation B — Architecture et systèmes d'exploitation

Durée : 45 minutes. Ordinateur avec interpréteur Python autorisé.

Barème indicatif : Ex. 1 : 10 pts (von Neumann, instructions) ; Ex. 2 : 10 pts (OS, permissions)

Exercice 1 — Von Neumann et instructions machine

(10 points)

1. (3 pts) Nommer les quatre constituants principaux de l'architecture de von Neumann, et préciser lequel exécute les instructions.
2. (4 pts) On ajoute au simulateur executer du cours l'instruction ("SUB", reg_dest, reg_src) (soustraction : reg_dest ← reg_dest - reg_src). Écrire le code de cette instruction.
3. (3 pts) Dérouler l'exécution de ce programme et donner l'état final de memoire :

```

1 memoire = [20, 8, 0]
2 programme = [
3     ("LOAD", "R0", 0),
4     ("LOAD", "R1", 1),
5     ("SUB", "R0", "R1"),
6     ("STORE", "R0", 2),
7     ("HALT", ),
8 ]

```

Exercice 2 — Système d'exploitation et permissions

(10 points)

1. (3 pts) Citer deux fonctions assurées par un système d'exploitation.
2. (4 pts) Un fichier a les permissions "rw-rw-r--". En utilisant droit_accorde du cours, déterminer si le propriétaire peut y écrire, si le groupe peut y écrire, et si les autres peuvent y écrire.
3. (3 pts) Convertir ces permissions en notation octale avec permissions_vers_octal.

Exercice 31

Un élève lit la permission "rwxrwxr--" et affirme que « le bloc du milieu (rwx) concerne les *autres* utilisateurs », alors qu'il concerne en réalité le *groupe*.

1. Rappeler l'ordre des trois catégories dans une chaîne de permissions.
2. En utilisant droit_accorde du cours, déterminer si le **groupe** peut écrire dans ce fichier, puis si les **autres** utilisateurs peuvent y écrire.
3. Ces deux réponses sont-elles identiques ? Qu'est-ce que cela montre sur l'erreur de l'élève ?

Exercice 32

Un élève lit la permission "rwxrwxr--" et affirme que « le bloc du milieu (rwx) concerne les *autres* utilisateurs », alors qu'il concerne en réalité le *groupe*.

1. Rappeler l'ordre des trois catégories dans une chaîne de permissions.
2. En utilisant droit_accorde du cours, déterminer si le **groupe** peut écrire dans ce fichier, puis si les **autres** utilisateurs peuvent y écrire.
3. Ces deux réponses sont-elles identiques ? Qu'est-ce que cela montre sur l'erreur de l'élève ?

Exercice 33

Un élève lit la permission "rwxrwxr--" et affirme que « le bloc du milieu (rwx) concerne les *autres* utilisateurs », alors qu'il concerne en réalité le *groupe*.

1. Rappeler l'ordre des trois catégories dans une chaîne de permissions.
2. En utilisant droit_accorde du cours, déterminer si le **groupe** peut écrire dans ce fichier, puis si les **autres** utilisateurs peuvent y écrire.
3. Ces deux réponses sont-elles identiques ? Qu'est-ce que cela montre sur l'erreur de l'élève ?

Exercice 34

Un élève lit la permission "rwxrwxr--" et affirme que « le bloc du milieu (rwx) concerne les *autres* utilisateurs », alors qu'il concerne en réalité le *groupe*.

1. Rappeler l'ordre des trois catégories dans une chaîne de permissions.
2. En utilisant droit_accorde du cours, déterminer si le **groupe** peut écrire dans ce fichier, puis si les **autres** utilisateurs peuvent y écrire.
3. Ces deux réponses sont-elles identiques ? Qu'est-ce que cela montre sur l'erreur de l'élève ?

Exercice 35

Un élève lit la permission "rwxrwxr--" et affirme que « le bloc du milieu (rwx) concerne les *autres* utilisateurs », alors qu'il concerne en réalité le *groupe*.

1. Rappeler l'ordre des trois catégories dans une chaîne de permissions.
2. En utilisant droit_accorde du cours, déterminer si le **groupe** peut écrire dans ce fichier, puis si les **autres** utilisateurs peuvent y écrire.
3. Ces deux réponses sont-elles identiques ? Qu'est-ce que cela montre sur l'erreur de l'élève ?

Exercice 36 ◆

Un élève lit la permission "rwxrwxr—" et affirme que « le bloc du milieu (rwx) concerne les *autres* utilisateurs », alors qu'il concerne en réalité le *groupe*.

1. Rappeler l'ordre des trois catégories dans une chaîne de permissions.
2. En utilisant `droit_accorde` du cours, déterminer si le **groupe** peut écrire dans ce fichier, puis si les **autres** utilisateurs peuvent y écrire.
3. Ces deux réponses sont-elles identiques ? Qu'est-ce que cela montre sur l'erreur de l'élève ?

Corrigé

1. L'ordre est toujours : **propriétaire** (caractères 1 à 3), **groupe** (caractères 4 à 6), **autres** (caractères 7 à 9).

2. Pour "rwxrwxr—" : le bloc **groupe** est rwx (caractères 4-6), donc le groupe **peut** écrire (`droit_accorde(perm, "groupe", "écriture")` vaut True). Le bloc **autres** est r-- (caractères 7-9), donc les autres **ne peuvent pas** écrire (False).

3. Non, ces deux réponses sont **différentes** (True contre False) : cela confirme que le bloc du milieu et le dernier bloc concernent des catégories distinctes, avec des droits potentiellement différents. L'élève avait tort de considérer le bloc du milieu comme celui des « autres » : c'est celui du **groupe**.

Corrigé

1. Le **processeur** (unité centrale) exécute les instructions. La **mémoire** stocke à la fois les instructions et les données.

2. Oui : avec 8 cœurs, l'ordinateur est **multiprocesseur** et peut exécuter plusieurs instructions en parallèle, un cœur par instruction (au sens large : plusieurs programmes ou plusieurs parties d'un même programme peuvent progresser simultanément).

3. Instructions et données sont stockées sous la **même forme** (des suites de bits) dans le même espace mémoire, sans marqueur qui les distinguerait intrinsèquement. C'est le **contexte de lecture** (le séquenceur lit-il cette adresse pour l'exécuter comme instruction, ou l'UAL la lit-elle comme donnée à calculer ?) qui détermine l'interprétation de ces bits, pas leur contenu lui-même.

Corrigé

1.

```

1 def executer(programme, memoire):
2     registres = {}
3     for instruction in programme:
4         op = instruction[0]
5         if op == "LOAD":
6             _, reg, adresse = instruction
7             registres[reg] = memoire[adresse]
8         elif op == "SUB":
9             _, reg_dest, reg_src = instruction
10            registres[reg_dest] = registres[reg_dest] - registres[reg_src]
11        elif op == "STORE":
12            _, reg, adresse = instruction
13            memoire[adresse] = registres[reg]
14        elif op == "HALT":
15            break
16    return memoire, registres

```

2. Déroulé :

- ▶ Après ("LOAD", "R0", 0) : registres = {"R0": 10}
- ▶ Après ("LOAD", "R1", 1) : registres = {"R0": 10, "R1": 4}

- ▶ Après ("SUB", "R0", "R1") : registres = {"R0": 6, "R1": 4}
 - ▶ Après ("STORE", "R0", 2) : memoire = [10, 4, 6]
3. L'exécution confirme : memoire_finale = [10, 4, 6].

Corrigé

1. Non, un processeur monoprocesseur n'exécute qu'une seule instruction à la fois. C'est la **gestion des processus** du système d'exploitation qui partage très rapidement le temps processeur entre les différents programmes (chacun reçoit un court intervalle de temps à tour de rôle), donnant l'**illusion** d'une exécution simultanée.

2. C'est la **gestion des droits** (permissions d'accès aux fichiers) qui est concernée : elle permet d'autoriser le lecteur de musique à écrire son propre fichier de log tout en lui refusant l'accès aux fichiers personnels d'un autre programme ou utilisateur.

3. Système libre signifie que le **code source** de Linux est mis à disposition de tous : n'importe qui peut le consulter, le modifier et le redistribuer librement. Un système propriétaire (comme Windows) ne diffuse pas son code source ; seul l'éditeur peut le modifier.

Corrigé

```
1 import subprocess
2
3 subprocess.run(["mkdir", "projet"])
4 subprocess.run(["touch", "projet/main.py"])
5 subprocess.run(["touch", "projet/readme.md"])
6
7 resultat = subprocess.run(["ls", "projet"], capture_output=True, text=True)
8 print(resultat.stdout) # main.py
9   readme.md
10
11 subprocess.run(["rm", "projet/readme.md"])
12 resultat2 = subprocess.run(["ls", "projet"], capture_output=True, text=True)
13 print(resultat2.stdout) # main.py
```

Après suppression, seul main.py reste dans le dossier projet.

Corrigé

1. Le propriétaire **ne peut pas** exécuter ce fichier (troisième caractère du premier bloc : -). Le groupe **peut** le lire (r----- → bloc groupe r--). Les autres utilisateurs **ne peuvent pas** le lire (dernier bloc : ---).

2. rw-r----- se décompose en rw- (propriétaire : 4 + 2 = 6), r-- (groupe : 4), --- (autres : 0) : notation octale 640.

3. Non, ce fichier n'est **pas** accessible en lecture par n'importe quel utilisateur : seuls le propriétaire et les membres du groupe peuvent le lire, les « autres » n'ont aucun droit. Un scénario approprié : un fichier de données personnelles partagé au sein d'une équipe (le groupe) mais qui ne doit pas être visible par les autres utilisateurs du système.

Corrigé

1. Le propriétaire **ne peut pas** exécuter ce fichier (troisième caractère du premier bloc : -). Le groupe **peut** le lire (r----- → bloc groupe r--). Les autres utilisateurs **ne peuvent pas** le lire (dernier bloc : ---).

2. rw-r----- se décompose en rw- (propriétaire : 4 + 2 = 6), r-- (groupe : 4), --- (autres : 0) : notation octale 640.

3. Non, ce fichier n'est **pas** accessible en lecture par n'importe quel utilisateur : seuls le propriétaire et les membres du groupe peuvent le lire, les « autres » n'ont aucun droit. Un scénario approprié : un

fichier de données personnelles partagé au sein d'une équipe (le groupe) mais qui ne doit pas être visible par les autres utilisateurs du système.

Corrigé

1. L'ordre est toujours : **propriétaire** (caractères 1 à 3), **groupe** (caractères 4 à 6), **autres** (caractères 7 à 9).
2. Pour "rwxrwxr--" : le bloc **groupe** est rwx (caractères 4-6), donc le groupe **peut** écrire (droit _accorde(perm, "groupe", "écriture") vaut True). Le bloc **autres** est r-- (caractères 7-9), donc les autres **ne peuvent pas** écrire (False).
3. Non, ces deux réponses sont **différentes** (True contre False) : cela confirme que le bloc du milieu et le dernier bloc concernent des catégories distinctes, avec des droits potentiellement différents. L'élève avait tort de considérer le bloc du milieu comme celui des « autres » : c'est celui du **groupe**.

Corrigé

1. Le **processeur** (unité centrale) exécute les instructions. La **mémoire** stocke à la fois les instructions et les données.
2. Oui : avec 8 cœurs, l'ordinateur est **multiprocesseur** et peut exécuter plusieurs instructions en parallèle, un cœur par instruction (au sens large : plusieurs programmes ou plusieurs parties d'un même programme peuvent progresser simultanément).
3. Instructions et données sont stockées sous la **même forme** (des suites de bits) dans le même espace mémoire, sans marqueur qui les distinguerait intrinsèquement. C'est le **contexte de lecture** (le séquenceur lit-il cette adresse pour l'exécuter comme instruction, ou l'UAL la lit-elle comme donnée à calculer ?) qui détermine l'interprétation de ces bits, pas leur contenu lui-même.

Corrigé

1.

```

1 def executer(programme, memoire):
2     registres = {}
3     for instruction in programme:
4         op = instruction[0]
5         if op == "LOAD":
6             _, reg, adresse = instruction
7             registres[reg] = memoire[adresse]
8         elif op == "SUB":
9             _, reg_dest, reg_src = instruction
10            registres[reg_dest] = registres[reg_dest] - registres[reg_src]
11        elif op == "STORE":
12            _, reg, adresse = instruction
13            memoire[adresse] = registres[reg]
14        elif op == "HALT":
15            break
16    return memoire, registres

```

2. Déroulé :

- Après ("LOAD", "R0", 0) : registres = {"R0": 10}
- Après ("LOAD", "R1", 1) : registres = {"R0": 10, "R1": 4}
- Après ("SUB", "R0", "R1") : registres = {"R0": 6, "R1": 4}
- Après ("STORE", "R0", 2) : memoire = [10, 4, 6]

3. L'exécution confirme : memoire_finale = [10, 4, 6].

Corrigé

1. Non, un processeur monoprocesseur n'exécute qu'une seule instruction à la fois. C'est la **gestion des processus** du système d'exploitation qui partage très rapidement le temps processeur entre les différents programmes (chacun reçoit un court intervalle de temps à tour de rôle), donnant l'**illusion** d'une exécution simultanée.

2. C'est la **gestion des droits** (permissions d'accès aux fichiers) qui est concernée : elle permet d'autoriser le lecteur de musique à écrire son propre fichier de log tout en lui refusant l'accès aux fichiers personnels d'un autre programme ou utilisateur.

3. Système libre signifie que le **code source** de Linux est mis à disposition de tous : n'importe qui peut le consulter, le modifier et le redistribuer librement. Un système propriétaire (comme Windows) ne diffuse pas son code source ; seul l'éditeur peut le modifier.

Corrigé

```
1 import subprocess
2
3 subprocess.run(["mkdir", "projet"])
4 subprocess.run(["touch", "projet/main.py"])
5 subprocess.run(["touch", "projet/readme.md"])
6
7 resultat = subprocess.run(["ls", "projet"], capture_output=True, text=True)
8 print(resultat.stdout) # main.py
9   readme.md
10
11 subprocess.run(["rm", "projet/readme.md"])
12 resultat2 = subprocess.run(["ls", "projet"], capture_output=True, text=True)
13 print(resultat2.stdout) # main.py
```

Après suppression, seul main.py reste dans le dossier projet.

Corrigé

1. Le propriétaire **ne peut pas** exécuter ce fichier (troisième caractère du premier bloc : -). Le groupe **peut** le lire (r----- → bloc groupe r--). Les autres utilisateurs **ne peuvent pas** le lire (dernier bloc : ---).

2. rw-r----- se décompose en rw- (propriétaire : 4 + 2 = 6), r-- (groupe : 4), --- (autres : 0) : notation octale 640.

3. Non, ce fichier n'est **pas** accessible en lecture par n'importe quel utilisateur : seuls le propriétaire et les membres du groupe peuvent le lire, les « autres » n'ont aucun droit. Un scénario approprié : un fichier de données personnelles partagé au sein d'une équipe (le groupe) mais qui ne doit pas être visible par les autres utilisateurs du système.

Corrigé

1. L'ordre est toujours : **propriétaire** (caractères 1 à 3), **groupe** (caractères 4 à 6), **autres** (caractères 7 à 9).

2. Pour "rwxrwxr--" : le bloc **groupe** est rwx (caractères 4-6), donc le groupe **peut** écrire (droit _accorde(perm, "groupe", "écriture") vaut True). Le bloc **autres** est r-- (caractères 7-9), donc les autres **ne peuvent pas** écrire (False).

3. Non, ces deux réponses sont **différentes** (True contre False) : cela confirme que le bloc du milieu et le dernier bloc concernent des catégories distinctes, avec des droits potentiellement différents. L'élève avait tort de considérer le bloc du milieu comme celui des « autres » : c'est celui du **groupe**.

Corrigé

1. Le **processeur** (unité centrale) exécute les instructions. La **mémoire** stocke à la fois les instructions et les données.

2. Oui : avec 8 cœurs, l'ordinateur est **multiprocesseur** et peut exécuter plusieurs instructions en parallèle, un cœur par instruction (au sens large : plusieurs programmes ou plusieurs parties d'un même programme peuvent progresser simultanément).

3. Instructions et données sont stockées sous la **même forme** (des suites de bits) dans le même espace mémoire, sans marqueur qui les distinguerait intrinsèquement. C'est le **contexte de lecture** (le séquenceur lit-il cette adresse pour l'exécuter comme instruction, ou l'UAL la lit-elle comme donnée à calculer ?) qui détermine l'interprétation de ces bits, pas leur contenu lui-même.

Corrigé

1.

```
1 def executer(programme, memoire):
2     registres = {}
3     for instruction in programme:
4         op = instruction[0]
5         if op == "LOAD":
6             _, reg, adresse = instruction
7             registres[reg] = memoire[adresse]
8         elif op == "SUB":
9             _, reg_dest, reg_src = instruction
10            registres[reg_dest] = registres[reg_dest] - registres[reg_src]
11        elif op == "STORE":
12            _, reg, adresse = instruction
13            memoire[adresse] = registres[reg]
14        elif op == "HALT":
15            break
16    return memoire, registres
```

2. Déroulé :

- ▶ Après ("LOAD", "R0", 0) : registres = {"R0": 10}
- ▶ Après ("LOAD", "R1", 1) : registres = {"R0": 10, "R1": 4}
- ▶ Après ("SUB", "R0", "R1") : registres = {"R0": 6, "R1": 4}
- ▶ Après ("STORE", "R0", 2) : memoire = [10, 4, 6]

3. L'exécution confirme : memoire_finale = [10, 4, 6].

Corrigé

1. Non, un processeur monoprocesseur n'exécute qu'une seule instruction à la fois. C'est la **gestion des processus** du système d'exploitation qui partage très rapidement le temps processeur entre les différents programmes (chacun reçoit un court intervalle de temps à tour de rôle), donnant l'**illusion** d'une exécution simultanée.

2. C'est la **gestion des droits** (permissions d'accès aux fichiers) qui est concernée : elle permet d'autoriser le lecteur de musique à écrire son propre fichier de log tout en lui refusant l'accès aux fichiers personnels d'un autre programme ou utilisateur.

3. Système libre signifie que le **code source** de Linux est mis à disposition de tous : n'importe qui peut le consulter, le modifier et le redistribuer librement. Un système propriétaire (comme Windows) ne diffuse pas son code source ; seul l'éditeur peut le modifier.

Corrigé

```
1 import subprocess
```

```

2
3 subprocess.run(["mkdir", "projet"])
4 subprocess.run(["touch", "projet/main.py"])
5 subprocess.run(["touch", "projet/readme.md"])
6
7 resultat = subprocess.run(["ls", "projet"], capture_output=True, text=True)
8 print(resultat.stdout) # main.py
9   readme.md
10
11 subprocess.run(["rm", "projet/readme.md"])
12 resultat2 = subprocess.run(["ls", "projet"], capture_output=True, text=True)
13 print(resultat2.stdout) # main.py

```

Après suppression, seul main.py reste dans le dossier projet.

Corrigé

1. Le propriétaire **ne peut pas** exécuter ce fichier (troisième caractère du premier bloc : -). Le groupe **peut** le lire (r----- → bloc groupe r--). Les autres utilisateurs **ne peuvent pas** le lire (dernier bloc : ---).
2. rw-r----- se décompose en rw- (propriétaire : 4 + 2 = 6), r-- (groupe : 4), --- (autres : 0) : notation octale 640.
3. Non, ce fichier n'est **pas** accessible en lecture par n'importe quel utilisateur : seuls le propriétaire et les membres du groupe peuvent le lire, les « autres » n'ont aucun droit. Un scénario approprié : un fichier de données personnelles partagé au sein d'une équipe (le groupe) mais qui ne doit pas être visible par les autres utilisateurs du système.

Corrigé

1. L'ordre est toujours : **propriétaire** (caractères 1 à 3), **groupe** (caractères 4 à 6), **autres** (caractères 7 à 9).
2. Pour "rwxrwxr--" : le bloc **groupe** est rwx (caractères 4-6), donc le groupe **peut** écrire (droit _accorde(perm, "groupe", "écriture") vaut True). Le bloc **autres** est r-- (caractères 7-9), donc les autres **ne peuvent pas** écrire (False).
3. Non, ces deux réponses sont **différentes** (True contre False) : cela confirme que le bloc du milieu et le dernier bloc concernent des catégories distinctes, avec des droits potentiellement différents. L'élève avait tort de considérer le bloc du milieu comme celui des « autres » : c'est celui du **groupe**.

Corrigé

1. Le **processeur** (unité centrale) exécute les instructions. La **mémoire** stocke à la fois les instructions et les données.
2. Oui : avec 8 cœurs, l'ordinateur est **multiprocesseur** et peut exécuter plusieurs instructions en parallèle, un cœur par instruction (au sens large : plusieurs programmes ou plusieurs parties d'un même programme peuvent progresser simultanément).
3. Instructions et données sont stockées sous la **même forme** (des suites de bits) dans le même espace mémoire, sans marqueur qui les distinguerait intrinsèquement. C'est le **contexte de lecture** (le séquenceur lit-il cette adresse pour l'exécuter comme instruction, ou l'UAL la lit-elle comme donnée à calculer ?) qui détermine l'interprétation de ces bits, pas leur contenu lui-même.

Corrigé

1.

```

1 def executer(programme, memoire):
2     registres = {}

```

```

3   for instruction in programme:
4       op = instruction[0]
5       if op == "LOAD":
6           _, reg, adresse = instruction
7           registres[reg] = memoire[adresse]
8       elif op == "SUB":
9           _, reg_dest, reg_src = instruction
10          registres[reg_dest] = registres[reg_dest] - registres[reg_src]
11      elif op == "STORE":
12          _, reg, adresse = instruction
13          memoire[adresse] = registres[reg]
14      elif op == "HALT":
15          break
16      return memoire, registres

```

2. Déroulé :

- ▶ Après ("LOAD", "R0", 0) : registres = {"R0": 10}
- ▶ Après ("LOAD", "R1", 1) : registres = {"R0": 10, "R1": 4}
- ▶ Après ("SUB", "R0", "R1") : registres = {"R0": 6, "R1": 4}
- ▶ Après ("STORE", "R0", 2) : memoire = [10, 4, 6]

3. L'exécution confirme : memoire_finale = [10, 4, 6].

Corrigé

1. Non, un processeur monoprocesseur n'exécute qu'une seule instruction à la fois. C'est la **gestion des processus** du système d'exploitation qui partage très rapidement le temps processeur entre les différents programmes (chacun reçoit un court intervalle de temps à tour de rôle), donnant l'**illusion** d'une exécution simultanée.

2. C'est la **gestion des droits** (permissions d'accès aux fichiers) qui est concernée : elle permet d'autoriser le lecteur de musique à écrire son propre fichier de log tout en lui refusant l'accès aux fichiers personnels d'un autre programme ou utilisateur.

3. Système libre signifie que le **code source** de Linux est mis à disposition de tous : n'importe qui peut le consulter, le modifier et le redistribuer librement. Un système propriétaire (comme Windows) ne diffuse pas son code source ; seul l'éditeur peut le modifier.

Corrigé

```

1  import subprocess
2
3  subprocess.run(["mkdir", "projet"])
4  subprocess.run(["touch", "projet/main.py"])
5  subprocess.run(["touch", "projet/readme.md"])
6
7  resultat = subprocess.run(["ls", "projet"], capture_output=True, text=True)
8  print(resultat.stdout) # main.py
9  readme.md
10
11 subprocess.run(["rm", "projet/readme.md"])
12 resultat2 = subprocess.run(["ls", "projet"], capture_output=True, text=True)
13 print(resultat2.stdout) # main.py

```

Après suppression, seul main.py reste dans le dossier projet.

Corrigé

1. Le propriétaire **ne peut pas** exécuter ce fichier (troisième caractère du premier bloc : -). Le groupe **peut** le lire (r----- → bloc groupe r--). Les autres utilisateurs **ne peuvent pas** le lire (dernier bloc : ---).

2. rw-r----- se décompose en rw- (propriétaire : $4 + 2 = 6$), r-- (groupe : 4), --- (autres : 0) : notation octale 640.

3. Non, ce fichier n'est **pas** accessible en lecture par n'importe quel utilisateur : seuls le propriétaire et les membres du groupe peuvent le lire, les « autres » n'ont aucun droit. Un scénario approprié : un fichier de données personnelles partagé au sein d'une équipe (le groupe) mais qui ne doit pas être visible par les autres utilisateurs du système.

Corrigé

1. L'ordre est toujours : **propriétaire** (caractères 1 à 3), **groupe** (caractères 4 à 6), **autres** (caractères 7 à 9).

2. Pour "rwxrwxr--" : le bloc **groupe** est rwx (caractères 4-6), donc le groupe **peut** écrire (droit _accorde(perm, "groupe", "écriture") vaut True). Le bloc **autres** est r-- (caractères 7-9), donc les autres **ne peuvent pas** écrire (False).

3. Non, ces deux réponses sont **différentes** (True contre False) : cela confirme que le bloc du milieu et le dernier bloc concernent des catégories distinctes, avec des droits potentiellement différents. L'élève avait tort de considérer le bloc du milieu comme celui des « autres » : c'est celui du **groupe**.

Corrigé

1. Le **processeur** (unité centrale) exécute les instructions. La **mémoire** stocke à la fois les instructions et les données.

2. Oui : avec 8 cœurs, l'ordinateur est **multiprocesseur** et peut exécuter plusieurs instructions en parallèle, un cœur par instruction (au sens large : plusieurs programmes ou plusieurs parties d'un même programme peuvent progresser simultanément).

3. Instructions et données sont stockées sous la **même forme** (des suites de bits) dans le même espace mémoire, sans marqueur qui les distinguerait intrinsèquement. C'est le **contexte de lecture** (le séquenceur lit-il cette adresse pour l'exécuter comme instruction, ou l'UAL la lit-elle comme donnée à calculer ?) qui détermine l'interprétation de ces bits, pas leur contenu lui-même.

Corrigé

1.

```

1 def executer(programme, memoire):
2     registres = {}
3     for instruction in programme:
4         op = instruction[0]
5         if op == "LOAD":
6             _, reg, adresse = instruction
7             registres[reg] = memoire[adresse]
8         elif op == "SUB":
9             _, reg_dest, reg_src = instruction
10            registres[reg_dest] = registres[reg_dest] - registres[reg_src]
11        elif op == "STORE":
12            _, reg, adresse = instruction
13            memoire[adresse] = registres[reg]
14        elif op == "HALT":
15            break
16    return memoire, registres

```

2. Déroulé :

- ▶ Après ("LOAD", "R0", 0) : registres = {"R0": 10}
- ▶ Après ("LOAD", "R1", 1) : registres = {"R0": 10, "R1": 4}
- ▶ Après ("SUB", "R0", "R1") : registres = {"R0": 6, "R1": 4}
- ▶ Après ("STORE", "R0", 2) : memoire = [10, 4, 6]

3. L'exécution confirme : memoire_finale = [10, 4, 6].

Corrigé

1. Non, un processeur monoprocesseur n'exécute qu'une seule instruction à la fois. C'est la **gestion des processus** du système d'exploitation qui partage très rapidement le temps processeur entre les différents programmes (chacun reçoit un court intervalle de temps à tour de rôle), donnant l'illusion d'une exécution simultanée.

2. C'est la **gestion des droits** (permissions d'accès aux fichiers) qui est concernée : elle permet d'autoriser le lecteur de musique à écrire son propre fichier de log tout en lui refusant l'accès aux fichiers personnels d'un autre programme ou utilisateur.

3. Système libre signifie que le **code source** de Linux est mis à disposition de tous : n'importe qui peut le consulter, le modifier et le redistribuer librement. Un système propriétaire (comme Windows) ne diffuse pas son code source ; seul l'éditeur peut le modifier.

Corrigé

```
1 import subprocess
2
3 subprocess.run(["mkdir", "projet"])
4 subprocess.run(["touch", "projet/main.py"])
5 subprocess.run(["touch", "projet/readme.md"])
6
7 resultat = subprocess.run(["ls", "projet"], capture_output=True, text=True)
8 print(resultat.stdout) # main.py
9   readme.md
10
11 subprocess.run(["rm", "projet/readme.md"])
12 resultat2 = subprocess.run(["ls", "projet"], capture_output=True, text=True)
13 print(resultat2.stdout) # main.py
```

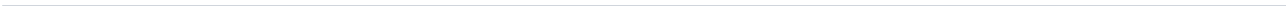
Après suppression, seul main.py reste dans le dossier projet.

Corrigé

1. Le propriétaire **ne peut pas** exécuter ce fichier (troisième caractère du premier bloc : -). Le groupe **peut** le lire (r----- → bloc groupe r--). Les autres utilisateurs **ne peuvent pas** le lire (dernier bloc : ---).

2. rw-r----- se décompose en rw- (propriétaire : 4 + 2 = 6), r-- (groupe : 4), --- (autres : 0) : notation octale 640.

3. Non, ce fichier n'est **pas** accessible en lecture par n'importe quel utilisateur : seuls le propriétaire et les membres du groupe peuvent le lire, les « autres » n'ont aucun droit. Un scénario approprié : un fichier de données personnelles partagé au sein d'une équipe (le groupe) mais qui ne doit pas être visible par les autres utilisateurs du système.



9 Réseaux : transmission de données

9.1 Découpage en paquets et encapsulation

DÉFINITION D1

Un **paquet** est un petit bloc de données, accompagné d'un **en-tête** contenant des informations nécessaires à son acheminement et à sa remise en ordre : numéro du paquet, nombre total de paquets, adresse d'origine, adresse de destination...

◆ **PROPRIÉTÉ Intérêt du découpage en paquets** Découper un message en paquets permet : de faire circuler plusieurs communications **simultanément** sur le même réseau (partage de la bande passante) ; de faire emprunter à des paquets d'un même message des **chemins différents** si nécessaire ; de ne **retransmettre que les paquets perdus** en cas de problème, plutôt que le message entier.

CODE DE RÉFÉRENCE — Découper et réassembler un message en paquets

```
1 def decouper_en_paquets(message, taille_max):
2     """Decoupe 'message' en paquets de taille au plus 'taille_max'."""
3     nb_paquets = (len(message) + taille_max - 1) // taille_max
4     paquets = []
5     for i in range(nb_paquets):
6         donnees = message[i * taille_max:(i + 1) * taille_max]
7         paquets.append({"numero": i, "total": nb_paquets, "donnees": donnees})
8     return paquets
9
10 def reassembler(paquets):
11     """Reassemble un message a partir de ses paquets, quel que soit leur ordre d'
12         arrivee."""
13     paquets_tries = sorted(paquets, key=lambda p: p["numero"])
14     return "".join(p["donnees"] for p in paquets_tries)
15
16 message = "BONJOURNSI"
17 paquets = decouper_en_paquets(message, 4)
18 print(paquets)
19 print(reassembler(paquets)) # BONJOURNSI
```

◆ **PROPRIÉTÉ Encapsulation** L'**encapsulation** consiste à ajouter successivement des en-têtes autour des données à mesure qu'elles descendent dans les couches réseau (données applicatives → en-tête de transport → en-tête réseau → en-tête de liaison), chaque couche n'interprétant que **son propre** en-tête, sans avoir besoin de connaître le contenu des couches supérieures.

EXEMPLE Peu importe le contenu du message (texte, image, vidéo), l'en-tête réseau qui indique l'adresse IP de destination a toujours le même format : c'est ce qui permet à un routeur d'acheminer n'importe quel type de données sans avoir à en comprendre le contenu.

CODE DE RÉFÉRENCE — Modéliser l'encapsulation par imbrication de dictionnaires

```

1 def encapsuler(donnees, en_tete_transport, en_tete_reseau):
2     """Encapsule des donnees dans deux en-tetes successifs (transport puis reseau).
3     """
4     paquet_transport = {"en_tete": en_tete_transport, "donnees": donnees}
5     paquet_reseau = {"en_tete": en_tete_reseau, "donnees": paquet_transport}
6     return paquet_reseau
7
8 paquet = encapsuler("Bonjour", {"port": 443}, {"ip_dest": "192.168.1.10"})
9 print(paquet)
10 # le routeur ne lit que paquet["en_tete"] (ip_dest), sans decoder paquet["donnees"]

```

⚠ ERREUR FRÉQUENTE

Croire que les paquets d'un même message arrivent toujours **dans l'ordre** d'envoi. Le numéro de paquet dans l'en-tête sert précisément à les **réordonner** à l'arrivée : sans lui, un réassemblage naïf (dans l'ordre de réception) produirait un message incorrect si des paquets arrivent dans le désordre.

» **Python À la machine.** Modifie reassembler pour qu'elle lève une erreur explicite (assert) si le nombre de paquets reçus ne correspond pas au total indiqué dans leurs en-têtes (paquet manquant).

9.2 Le protocole du bit alterné

DÉFINITION D2

Le protocole du **bit alterné** est un mécanisme simple de récupération de perte de paquets. L'émetteur associe à chaque paquet un **bit** (0 ou 1) qui **alterne** à chaque nouveau paquet. Le récepteur renvoie un **accusé de réception** (ACK) contenant ce même bit. Si l'émetteur ne reçoit **pas** l'ACK attendu (paquet ou ACK perdu), il **retransmet** le paquet avec le même bit, jusqu'à recevoir l'ACK correct.

◆ **PROPRIÉTÉ Pourquoi un simple bit suffit** Comme les paquets sont envoyés **un par un** (l'émetteur attend l'ACK avant d'envoyer le suivant), un seul bit alterné suffit à distinguer « un nouveau paquet » d'« une retransmission de l'ancien » : le récepteur sait qu'il a déjà reçu un paquet avec un bit donné, et ignore les doublons.

CODE DE RÉFÉRENCE — Simulation du protocole du bit alterné

```

1 def protocole_bit_alterne(messages, tentatives_perdues):
2     """Simule l'envoi de 'messages' avec le protocole du bit alterne.
3

```

```

4  'tentatives_perdues' : ensemble des numeros de tentatives (globales, a partir
    de 1)
5  qui echouent (paquet ou ACK perdu), pour simuler des pertes reseau.
6  Renvoie la liste des (bit, message) effectivement recus par le destinataire.
7  """
8  resultat = []
9  bit = 0
10  compteur_tentative = 0
11  for message in messages:
12      recu = False
13      while not recu:
14          compteur_tentative += 1
15          if compteur_tentative in tentatives_perdues:
16              continue # echec : on retransmet le meme message avec le meme bit
17          resultat.append((bit, message))
18          recu = True
19          bit = 1 - bit # on alterne le bit pour le message suivant
20  return resultat
21
22 # La 2e tentative globale echoue : le premier envoi de "B" est perdu, il est
    retransmis
23 envois = protocole_bit_alterne(["A", "B", "C"], tentatives_perdues={2})
24 print(envois) # [(0, 'A'), (1, 'B'), (0, 'C')]

```

◆ **PROPRIÉTÉ Robustesse du protocole** Quel que soit le nombre de pertes simulées, tant que le réseau finit par transmettre correctement (pas de perte **permanente**), tous les messages finissent par arriver : le protocole du bit alterné garantit la **livraison**, au prix éventuel de retransmissions et donc d'un délai supplémentaire.

9.3 Simuler un réseau

DÉFINITION D3

On peut modéliser un **réseau** local par un **graphe** : chaque machine est un sommet, chaque connexion directe (câble, Wi-Fi) est une arête. Deux machines peuvent communiquer si et seulement s'il existe un **chemin** entre elles dans ce graphe (éventuellement via des intermédiaires comme un commutateur ou un routeur).

CODE DE RÉFÉRENCE — Simuler un réseau et tester la communication entre deux machines

```

1  reseau = {
2      "PC1": ["Switch"],
3      "PC2": ["Switch"],
4      "Switch": ["PC1", "PC2", "Routeur"],
5      "Routeur": ["Switch", "Internet"],
6      "Internet": ["Routeur"],
7  }
8
9  def peuvent_communiquer(reseau, depart, arrivee, visites=None):
10     """Teste s'il existe un chemin entre 'depart' et 'arrivee' dans le reseau."""
11     if visites is None:
12         visites = set()

```

```

13     if depart == arrivee:
14         return True
15     visites.add(depart)
16     for voisin in reseau.get(depart, []):
17         if voisin not in visites:
18             if peuvent_communiquer(reseau, voisin, arrivee, visites):
19                 return True
20     return False
21
22 print(peuvent_communiquer(reseau, "PC1", "Internet")) # True

```

⚠ ERREUR FRÉQUENTE

Oublier de marquer les sommets **déjà visités** lors du parcours d'un réseau. Sans cette précaution, un réseau contenant un cycle (par exemple deux commutateurs reliés entre eux et à d'autres machines) provoquerait une **boucle infinie** lors de la recherche de chemin.

» **Python À la machine.** Ajoute au réseau simulé une machine "PC3" reliée uniquement à "PC1" (et non au Switch). Vérifie que PC3 peut communiquer avec PC1 mais pas directement avec Internet.

9.4 Capteurs, actionneurs et IHM programmée

DÉFINITION D4

Un **capteur** mesure une grandeur physique (température, luminosité, présence...) et la convertit en une donnée numérique exploitable par un programme. Un **actionneur** reçoit une commande d'un programme et agit sur le monde physique (allumer une lampe, ouvrir un volet, déclencher un moteur...).

EXEMPLE Un thermostat connecté : le **capteur** de température mesure la température ambiante ; le programme décide, selon cette mesure, d'activer ou non le chauffage ; l'**actionneur** (relais électrique) coupe ou active effectivement le chauffage.

◆ **PROPRIÉTÉ Cahier des charges d'une IHM** Réaliser une IHM (interface homme-machine) répondant à un **cahier des charges** consiste à traduire des règles énoncées en langage courant (« si la température descend sous 18 °C, le chauffage s'active ») en un programme précis, testé sur des cas concrets.

CODE DE RÉFÉRENCE — Thermostat programmé : capteur, décision, actionneur et IHM

```

1 COMMANDES = {"AUTO": None, "ON": True, "OFF": False}
2 etat = {"chauffage": False, "override_manuel": None}
3
4
5 def traiter_evenement(commande):
6     commande = commande.strip().upper()
7     if commande not in COMMANDES:
8         raise ValueError("commande attendue : AUTO, ON ou OFF")
9     etat["override_manuel"] = COMMANDES[commande]

```

```

10
11
12 def decider_chauffage(temperature, seuil=18):
13     if etat["override_manuel"] is not None:
14         return etat["override_manuel"]
15     return temperature < seuil
16
17
18 def actionner_chauffage(temperature, seuil=18):
19     etat["chauffage"] = decider_chauffage(temperature, seuil)
20     return etat["chauffage"]
21
22
23 def afficher_etat(temperature):
24     override = etat["override_manuel"]
25     if override is None:
26         mode = "AUTO"
27     elif override:
28         mode = "ON manuel"
29     else:
30         mode = "OFF manuel"
31     chauffage = "ON" if etat["chauffage"] else "OFF"
32     return f"temperature={temperature} C | mode={mode} | chauffage={chauffage}"
33
34
35 def interface_thermostat(commande, temperature, seuil=18):
36     traiter_evenement(commande)
37     actionner_chauffage(temperature, seuil)
38     return afficher_etat(temperature)
39
40
41 def lancer_ihm(temperature, seuil=18, lire=input, ecrire=print):
42     ecrire("Commandes : AUTO | ON | OFF")
43     commande = lire("Commande : ")
44     ecrire(interface_thermostat(commande, temperature, seuil))

```

EXEMPLE L'utilisateur lance `lancer_ihm(25)`, saisit une commande AUTO, ON ou OFF, puis lit dans l'affichage le mode et l'état effectif du chauffage. La commande saisie constitue l'événement traité par l'IHM.

⚠ ERREUR FRÉQUENTE

Programmer une IHM en oubliant de tester les **cas limites** du cahier des charges (température exactement égale au seuil, override activé puis désactivé). Un cahier des charges mal testé peut sembler fonctionner sur l'exemple de démonstration tout en étant incorrect dans des situations réelles fréquentes.

» **Python À la machine.** Modifie le thermostat pour ajouter une **hystérésis** : le chauffage s'active si la température descend sous 18 °C, mais ne se désactive que lorsqu'elle dépasse 20 °C (pour éviter des allumages/extinctions trop fréquents). Teste ta fonction sur une séquence de températures [15, 17, 19, 21, 19].

9.5 Découpage en paquets et encapsulation

DÉFINITION D1

Un **paquet** est un petit bloc de données, accompagné d'un **en-tête** contenant des informations nécessaires à son acheminement et à sa remise en ordre : numéro du paquet, nombre total de paquets, adresse d'origine, adresse de destination...

◆ **PROPRIÉTÉ Intérêt du découpage en paquets** Découper un message en paquets permet : de faire circuler plusieurs communications **simultanément** sur le même réseau (partage de la bande passante) ; de faire emprunter à des paquets d'un même message des **chemins différents** si nécessaire ; de ne **retransmettre que les paquets perdus** en cas de problème, plutôt que le message entier.

CODE DE RÉFÉRENCE — Découper et réassembler un message en paquets

```

1 def decouper_en_paquets(message, taille_max):
2     """Decoupe 'message' en paquets de taille au plus 'taille_max'."""
3     nb_paquets = (len(message) + taille_max - 1) // taille_max
4     paquets = []
5     for i in range(nb_paquets):
6         donnees = message[i * taille_max:(i + 1) * taille_max]
7         paquets.append({"numero": i, "total": nb_paquets, "donnees": donnees})
8     return paquets
9
10 def reassembler(paquets):
11     """Reassemble un message a partir de ses paquets, quel que soit leur ordre d'
12     arrivee."""
13     paquets_tries = sorted(paquets, key=lambda p: p["numero"])
14     return "".join(p["donnees"] for p in paquets_tries)
15
16 message = "BONJOURNSI"
17 paquets = decouper_en_paquets(message, 4)
18 print(paquets)
19 print(reassembler(paquets)) # BONJOURNSI

```

◆ **PROPRIÉTÉ Encapsulation** L'**encapsulation** consiste à ajouter successivement des en-têtes autour des données à mesure qu'elles descendent dans les couches réseau (données applicatives → en-tête de transport → en-tête réseau → en-tête de liaison), chaque couche n'interprétant que **son propre** en-tête, sans avoir besoin de connaître le contenu des couches supérieures.

EXEMPLE Peu importe le contenu du message (texte, image, vidéo), l'en-tête réseau qui indique l'adresse IP de destination a toujours le même format : c'est ce qui permet à un routeur d'acheminer n'importe quel type de données sans avoir à en comprendre le contenu.

CODE DE RÉFÉRENCE — Modéliser l'encapsulation par imbrication de dictionnaires

```

1 def encapsuler(donnees, en_tete_transport, en_tete_reseau):
2     """Encapsule des donnees dans deux en-tetes successifs (transport puis reseau).
3     """
4     paquet_transport = {"en_tete": en_tete_transport, "donnees": donnees}
5     paquet_reseau = {"en_tete": en_tete_reseau, "donnees": paquet_transport}
6     return paquet_reseau

```

```

6
7 paquet = encapsuler("Bonjour", {"port": 443}, {"ip_dest": "192.168.1.10"})
8 print(paquet)
9 # le routeur ne lit que paquet["en_tete"] (ip_dest), sans decoder paquet["donnees"]

```

⚠ ERREUR FRÉQUENTE

Croire que les paquets d'un même message arrivent toujours **dans l'ordre** d'envoi. Le numéro de paquet dans l'en-tête sert précisément à les **réordonner** à l'arrivée : sans lui, un réassemblage naïf (dans l'ordre de réception) produirait un message incorrect si des paquets arrivent dans le désordre.

» **Python À la machine.** Modifie `reassembler` pour qu'elle lève une erreur explicite (`assert`) si le nombre de paquets reçus ne correspond pas au total indiqué dans leurs en-têtes (paquet manquant).

9.6 Le protocole du bit alterné

DÉFINITION D2

Le protocole du **bit alterné** est un mécanisme simple de récupération de perte de paquets. L'émetteur associe à chaque paquet un **bit** (0 ou 1) qui **alterne** à chaque nouveau paquet. Le récepteur renvoie un **accusé de réception** (ACK) contenant ce même bit. Si l'émetteur ne reçoit **pas** l'ACK attendu (paquet ou ACK perdu), il **retransmet** le paquet avec le même bit, jusqu'à recevoir l'ACK correct.

◆ **PROPRIÉTÉ Pourquoi un simple bit suffit** Comme les paquets sont envoyés **un par un** (l'émetteur attend l'ACK avant d'envoyer le suivant), un seul bit alterné suffit à distinguer « un nouveau paquet » d' « une retransmission de l'ancien » : le récepteur sait qu'il a déjà reçu un paquet avec un bit donné, et ignore les doublons.

CODE DE RÉFÉRENCE — Simulation du protocole du bit alterné

```

1 def protocole_bit_alterne(messages, tentatives_perdues):
2     """Simule l'envoi de 'messages' avec le protocole du bit alterne.
3
4     'tentatives_perdues' : ensemble des numeros de tentatives (globales, a partir
5     de 1)
6     qui echouent (paquet ou ACK perdu), pour simuler des pertes reseau.
7     Renvoie la liste des (bit, message) effectivement recus par le destinataire.
8     """
9     resultat = []
10    bit = 0
11    compteur_tentative = 0
12    for message in messages:
13        recu = False
14        while not recu:
15            compteur_tentative += 1
16            if compteur_tentative in tentatives_perdues:

```

```

16         continue # echec : on retransmet le meme message avec le meme bit
17         resultat.append((bit, message))
18         recu = True
19         bit = 1 - bit # on alterne le bit pour le message suivant
20     return resultat
21
22 # La 2e tentative globale echoue : le premier envoi de "B" est perdu, il est
    retransmis
23 envois = protocole_bit_alterne(["A", "B", "C"], tentatives_perdues={2})
24 print(envois) # [(0, 'A'), (1, 'B'), (0, 'C')]

```

◆ **PROPRIÉTÉ Robustesse du protocole** Quel que soit le nombre de pertes simulées, tant que le réseau finit par transmettre correctement (pas de perte **permanente**), tous les messages finissent par arriver : le protocole du bit alterné garantit la **livraison**, au prix éventuel de retransmissions et donc d'un délai supplémentaire.

9.7 Simuler un réseau

DÉFINITION D3

On peut modéliser un **réseau** local par un **graphe** : chaque machine est un sommet, chaque connexion directe (câble, Wi-Fi) est une arête. Deux machines peuvent communiquer si et seulement s'il existe un **chemin** entre elles dans ce graphe (éventuellement via des intermédiaires comme un commutateur ou un routeur).

CODE DE RÉFÉRENCE — Simuler un réseau et tester la communication entre deux machines

```

1  reseau = {
2      "PC1": ["Switch"],
3      "PC2": ["Switch"],
4      "Switch": ["PC1", "PC2", "Routeur"],
5      "Routeur": ["Switch", "Internet"],
6      "Internet": ["Routeur"],
7  }
8
9  def peuvent_communiquer(reseau, depart, arrivee, visites=None):
10     """Teste s'il existe un chemin entre 'depart' et 'arrivee' dans le reseau."""
11     if visites is None:
12         visites = set()
13     if depart == arrivee:
14         return True
15     visites.add(depart)
16     for voisin in reseau.get(depart, []):
17         if voisin not in visites:
18             if peuvent_communiquer(reseau, voisin, arrivee, visites):
19                 return True
20     return False
21
22 print(peuvent_communiquer(reseau, "PC1", "Internet")) # True

```


ERREUR FRÉQUENTE

Oublier de marquer les sommets **déjà visités** lors du parcours d'un réseau. Sans cette précaution, un réseau contenant un cycle (par exemple deux commutateurs reliés entre eux et à d'autres machines) provoquerait une **boucle infinie** lors de la recherche de chemin.

» **Python À la machine.** Ajoute au réseau simulé une machine "PC3" reliée uniquement à "PC1" (et non au Switch). Vérifie que PC3 peut communiquer avec PC1 mais pas directement avec Internet.

9.8 Découpage en paquets et encapsulation

DÉFINITION D1

Un **paquet** est un petit bloc de données, accompagné d'un **en-tête** contenant des informations nécessaires à son acheminement et à sa remise en ordre : numéro du paquet, nombre total de paquets, adresse d'origine, adresse de destination...

◆ **PROPRIÉTÉ Intérêt du découpage en paquets** Découper un message en paquets permet : de faire circuler plusieurs communications **simultanément** sur le même réseau (partage de la bande passante) ; de faire emprunter à des paquets d'un même message des **chemins différents** si nécessaire ; de ne **retransmettre que les paquets perdus** en cas de problème, plutôt que le message entier.

CODE DE RÉFÉRENCE — Découper et réassembler un message en paquets

```
1 def decouper_en_paquets(message, taille_max):
2     """Decoupe 'message' en paquets de taille au plus 'taille_max'."""
3     nb_paquets = (len(message) + taille_max - 1) // taille_max
4     paquets = []
5     for i in range(nb_paquets):
6         donnees = message[i * taille_max:(i + 1) * taille_max]
7         paquets.append({"numero": i, "total": nb_paquets, "donnees": donnees})
8     return paquets
9
10 def reassembler(paquets):
11     """Reassemble un message a partir de ses paquets, quel que soit leur ordre d'
12     arrivée."""
13     paquets_tries = sorted(paquets, key=lambda p: p["numero"])
14     return "".join(p["donnees"] for p in paquets_tries)
15
16 message = "BONJOURNSI"
17 paquets = decouper_en_paquets(message, 4)
18 print(paquets)
19 print(reassembler(paquets)) # BONJOURNSI
```

◆ **PROPRIÉTÉ Encapsulation** L'**encapsulation** consiste à ajouter successivement des en-têtes autour des données à mesure qu'elles descendent dans les couches réseau (données applicatives

→ en-tête de transport → en-tête réseau → en-tête de liaison), chaque couche n'interprétant que **son propre** en-tête, sans avoir besoin de connaître le contenu des couches supérieures.

EXEMPLE Peu importe le contenu du message (texte, image, vidéo), l'en-tête réseau qui indique l'adresse IP de destination a toujours le même format : c'est ce qui permet à un routeur d'acheminer n'importe quel type de données sans avoir à en comprendre le contenu.

CODE DE RÉFÉRENCE — Modéliser l'encapsulation par imbrication de dictionnaires

```
1 def encapsuler(donnees, en_tete_transport, en_tete_reseau):
2     """Encapsule des donnees dans deux en-tetes successifs (transport puis reseau).
3     """
4     paquet_transport = {"en_tete": en_tete_transport, "donnees": donnees}
5     paquet_reseau = {"en_tete": en_tete_reseau, "donnees": paquet_transport}
6     return paquet_reseau
7
8 paquet = encapsuler("Bonjour", {"port": 443}, {"ip_dest": "192.168.1.10"})
9 print(paquet)
10 # le routeur ne lit que paquet["en_tete"] (ip_dest), sans decoder paquet["donnees"]
```

⚠ ERREUR FRÉQUENTE

Croire que les paquets d'un même message arrivent toujours **dans l'ordre** d'envoi. Le numéro de paquet dans l'en-tête sert précisément à les **réordonner** à l'arrivée : sans lui, un réassemblage naïf (dans l'ordre de réception) produirait un message incorrect si des paquets arrivent dans le désordre.

» **Python À la machine.** Modifie `reassembler` pour qu'elle lève une erreur explicite (`assert`) si le nombre de paquets reçus ne correspond pas au total indiqué dans leurs en-têtes (paquet manquant).

9.9 Le protocole du bit alterné

DÉFINITION D2

Le protocole du **bit alterné** est un mécanisme simple de récupération de perte de paquets. L'émetteur associe à chaque paquet un **bit** (0 ou 1) qui **alterne** à chaque nouveau paquet. Le récepteur renvoie un **accusé de réception** (ACK) contenant ce même bit. Si l'émetteur ne reçoit **pas** l'ACK attendu (paquet ou ACK perdu), il **retransmet** le paquet avec le même bit, jusqu'à recevoir l'ACK correct.

◆ **PROPRIÉTÉ Pourquoi un simple bit suffit** Comme les paquets sont envoyés **un par un** (l'émetteur attend l'ACK avant d'envoyer le suivant), un seul bit alterné suffit à distinguer « un nouveau paquet » d'« une retransmission de l'ancien » : le récepteur sait qu'il a déjà reçu un paquet avec un bit donné, et ignore les doublons.

CODE DE RÉFÉRENCE — Simulation du protocole du bit alterné

```

1 def protocole_bit_alterne(messages, tentatives_perdues):
2     """Simule l'envoi de 'messages' avec le protocole du bit alterné.
3
4     'tentatives_perdues' : ensemble des numeros de tentatives (globales, a partir
5     de 1)
6     qui echouent (paquet ou ACK perdu), pour simuler des pertes reseau.
7     Renvoie la liste des (bit, message) effectivement recus par le destinataire.
8     """
9     resultat = []
10    bit = 0
11    compteur_tentative = 0
12    for message in messages:
13        recu = False
14        while not recu:
15            compteur_tentative += 1
16            if compteur_tentative in tentatives_perdues:
17                continue # echec : on retransmet le meme message avec le meme bit
18            resultat.append((bit, message))
19            recu = True
20            bit = 1 - bit # on alterne le bit pour le message suivant
21    return resultat
22
23 # La 2e tentative globale echoue : le premier envoi de "B" est perdu, il est
24 # retransmis
25 envois = protocole_bit_alterne(["A", "B", "C"], tentatives_perdues={2})
26 print(envois) # [(0, 'A'), (1, 'B'), (0, 'C')]

```

◆ **PROPRIÉTÉ Robustesse du protocole** Quel que soit le nombre de pertes simulées, tant que le réseau finit par transmettre correctement (pas de perte **permanente**), tous les messages finissent par arriver : le protocole du bit alterné garantit la **livraison**, au prix éventuel de retransmissions et donc d'un délai supplémentaire.

9.10 Simuler un réseau

DÉFINITION D3

On peut modéliser un **réseau** local par un **graphe** : chaque machine est un sommet, chaque connexion directe (câble, Wi-Fi) est une arête. Deux machines peuvent communiquer si et seulement s'il existe un **chemin** entre elles dans ce graphe (éventuellement via des intermédiaires comme un commutateur ou un routeur).

CODE DE RÉFÉRENCE — Simuler un réseau et tester la communication entre deux machines

```

1 reseau = {
2     "PC1": ["Switch"],
3     "PC2": ["Switch"],
4     "Switch": ["PC1", "PC2", "Routeur"],
5     "Routeur": ["Switch", "Internet"],
6     "Internet": ["Routeur"],
7 }

```

```

8
9 def peuvent_communiquer(reseau, depart, arrivee, visites=None):
10     """Teste s'il existe un chemin entre 'depart' et 'arrivee' dans le reseau."""
11     if visites is None:
12         visites = set()
13     if depart == arrivee:
14         return True
15     visites.add(depart)
16     for voisin in reseau.get(depart, []):
17         if voisin not in visites:
18             if peuvent_communiquer(reseau, voisin, arrivee, visites):
19                 return True
20     return False
21
22 print(peuvent_communiquer(reseau, "PC1", "Internet")) # True

```

⚠ ERREUR FRÉQUENTE

Oublier de marquer les sommets **déjà visités** lors du parcours d'un réseau. Sans cette précaution, un réseau contenant un cycle (par exemple deux commutateurs reliés entre eux et à d'autres machines) provoquerait une **boucle infinie** lors de la recherche de chemin.

» **Python À la machine.** Ajoute au réseau simulé une machine "PC3" reliée uniquement à "PC1" (et non au Switch). Vérifie que PC3 peut communiquer avec PC1 mais pas directement avec Internet.

9.11 Capteurs, actionneurs et IHM programmée

DÉFINITION D4

Un **capteur** mesure une grandeur physique (température, luminosité, présence...) et la convertit en une donnée numérique exploitable par un programme. Un **actionneur** reçoit une commande d'un programme et agit sur le monde physique (allumer une lampe, ouvrir un volet, déclencher un moteur...).

EXEMPLE Un thermostat connecté : le **capteur** de température mesure la température ambiante ; le programme décide, selon cette mesure, d'activer ou non le chauffage ; l'**actionneur** (relais électrique) coupe ou active effectivement le chauffage.

♦ **PROPRIÉTÉ Cahier des charges d'une IHM** Réaliser une IHM (interface homme-machine) répondant à un **cahier des charges** consiste à traduire des règles énoncées en langage courant (« si la température descend sous 18 °C, le chauffage s'active ») en un programme précis, testé sur des cas concrets.

CODE DE RÉFÉRENCE — Thermostat programmé : capteur, décision, actionneur et IHM

```

1 COMMANDES = {"AUTO": None, "ON": True, "OFF": False}
2 etat = {"chauffage": False, "override_manuel": None}
3
4

```

```

5 def traiter_evenement(commande):
6     commande = commande.strip().upper()
7     if commande not in COMMANDES:
8         raise ValueError("commande attendue : AUTO, ON ou OFF")
9     etat["override_manuel"] = COMMANDES[commande]
10
11
12 def decider_chauffage(temperature, seuil=18):
13     if etat["override_manuel"] is not None:
14         return etat["override_manuel"]
15     return temperature < seuil
16
17
18 def actionner_chauffage(temperature, seuil=18):
19     etat["chauffage"] = decider_chauffage(temperature, seuil)
20     return etat["chauffage"]
21
22
23 def afficher_etat(temperature):
24     override = etat["override_manuel"]
25     if override is None:
26         mode = "AUTO"
27     elif override:
28         mode = "ON manuel"
29     else:
30         mode = "OFF manuel"
31     chauffage = "ON" if etat["chauffage"] else "OFF"
32     return f"temperature={temperature} C | mode={mode} | chauffage={chauffage}"
33
34
35 def interface_thermostat(commande, temperature, seuil=18):
36     traiter_evenement(commande)
37     actionner_chauffage(temperature, seuil)
38     return afficher_etat(temperature)
39
40
41 def lancer_ihm(temperature, seuil=18, lire=input, ecrire=print):
42     ecrire("Commandes : AUTO | ON | OFF")
43     commande = lire("Commande : ")
44     ecrire(interface_thermostat(commande, temperature, seuil))

```

EXEMPLE L'utilisateur lance `lancer_ihm(25)`, saisit une commande AUTO, ON ou OFF, puis lit dans l'affichage le mode et l'état effectif du chauffage. La commande saisie constitue l'événement traité par l'IHM.

⚠ ERREUR FRÉQUENTE

Programmer une IHM en oubliant de tester les **cas limites** du cahier des charges (température exactement égale au seuil, override activé puis désactivé). Un cahier des charges mal testé peut sembler fonctionner sur l'exemple de démonstration tout en étant incorrect dans des situations réelles fréquentes.

» **Python À la machine.** Modifie le thermostat pour ajouter une **hystérésis** : le chauffage s'active si la température descend sous 18 °C, mais ne se désactive que lorsqu'elle dépasse 20 °C (pour éviter des allumages/extinctions trop fréquents). Teste ta fonction sur une séquence de températures [15, 17, 19, 21, 19].

Méthodes

- Méthode directe de travail sur le sujet.

Exercice 1 ♦

On utilise la fonction de découpage du cours. Le message "PROTOCOLERESEAU" est découpé en paquets de taille maximale 5.

1. Combien de paquets sont créés ? Donner le contenu (donnees) de chacun.
2. On suppose que les paquets arrivent au destinataire dans l'ordre suivant : paquet 2, puis paquet 0, puis paquet 1. Le message est-il correctement reconstitué par reassembler malgré cet ordre d'arrivée ? Pourquoi ?
3. Que se passerait-il si l'on reconstituait le message en concaténant simplement les paquets **dans leur ordre d'arrivée**, sans utiliser leur numéro ?

Exercice 2 ♦

On envoie les messages ["X", "Y", "Z"] avec le protocole du bit alterné du cours. La première tentative globale (numéro 1) et la quatrième tentative globale (numéro 4) échouent (paquet ou ACK perdu) ; toutes les autres réussissent.

1. Dérouler, tentative par tentative, ce qui se passe : quel message est concerné par chaque tentative, réussit-elle, et quel est le bit utilisé ?
2. Donner la liste finale des messages effectivement reçus par le destinataire, avec leur bit.
3. Combien de tentatives ont été nécessaires au total pour transmettre les 3 messages ? Ce nombre est-il égal au nombre de messages ? Pourquoi ?

Exercice 3 ♦♦

Le réseau d'un établissement scolaire relie trois salles à deux commutateurs :

```

1 reseau = {
2     "Salle1": ["Switch1"],
3     "Salle2": ["Switch1"],
4     "Switch1": ["Salle1", "Salle2", "Switch2"],
5     "Switch2": ["Switch1", "Salle3"],
6     "Salle3": ["Switch2"],
7 }
```

1. En utilisant `peuvent_communiquer` du cours, vérifier que "Salle1" et "Salle3" peuvent communiquer, bien qu'elles ne soient reliées à aucun commutateur commun directement.
2. Ajouter une machine "SalleIsolee" qui n'est reliée à **aucune** autre machine du réseau (dictionnaire avec une liste vide). Vérifie qu'elle ne peut communiquer avec aucune autre salle.
3. Pourquoi la fonction marque-t-elle les sommets « visités » pendant sa recherche ? Que risquerait-il de se passer si ce réseau contenait un cycle (par exemple si Switch1 et Switch2 étaient reliés par **deux** câbles distincts, modélisés comme une arête supplémentaire) et que cette précaution était absente ?

Exercice 4 ♦♦

Un lampadaire de rue s'allume automatiquement lorsque la luminosité ambiante descend sous 300 lux.

1. Dans ce système, quel est le **capteur** ? Quel est l'**actionneur** ?
2. Écrire une fonction `decider_eclairage(luminosite, seuil=300)` qui renvoie True si le lampadaire doit s'allumer.

3. Tester cette fonction pour une luminosité de 150 lux (nuit), 500 lux (plein jour), et exactement 300 lux (le seuil). Le comportement au seuil exact te semble-t-il cohérent avec l'énoncé (« sous 300 lux ») ?

Exercice 5

Cahier des charges d'un système d'arrosage automatique : un capteur mesure l'humidité du sol (en %) ; la pompe se **déclenche** quand l'humidité descend sous 30 %, et ne s'**arrête** que lorsque l'humidité dépasse 60 % (pour éviter des démarrages/arrêts trop fréquents de la pompe — c'est le principe d'**hystérésis** déjà rencontré pour le thermostat du cours).

1. Écrire une fonction `irrigation_hysteresis(humidite, seuil_bas=30, seuil_haut=60)` qui met à jour et renvoie l'état `etat["pompe"]` (initialement `False`), en respectant ce cahier des charges.
2. Tester ta fonction sur la séquence de mesures `[20, 35, 50, 65, 40]` (une mesure par heure). Donner la séquence des états de la pompe.
3. À la mesure 35 (deuxième heure), la pompe est-elle active ? Pourquoi, alors que 35 % est supérieur au seuil bas de 30 % ?

Exercice 6

On utilise la fonction de découpage du cours. Le message "PROTOCOLERESEAU" est découpé en paquets de taille maximale 5.

1. Combien de paquets sont créés ? Donner le contenu (`donnees`) de chacun.
2. On suppose que les paquets arrivent au destinataire dans l'ordre suivant : paquet 2, puis paquet 0, puis paquet 1. Le message est-il correctement reconstitué par reassembler malgré cet ordre d'arrivée ? Pourquoi ?
3. Que se passerait-il si l'on reconstituait le message en concaténant simplement les paquets **dans leur ordre d'arrivée**, sans utiliser leur numéro ?

Exercice 7

On envoie les messages ["X", "Y", "Z"] avec le protocole du bit alterné du cours. La première tentative globale (numéro 1) et la quatrième tentative globale (numéro 4) échouent (paquet ou ACK perdu) ; toutes les autres réussissent.

1. Dérouler, tentative par tentative, ce qui se passe : quel message est concerné par chaque tentative, réussit-elle, et quel est le bit utilisé ?
2. Donner la liste finale des messages effectivement reçus par le destinataire, avec leur bit.
3. Combien de tentatives ont été nécessaires au total pour transmettre les 3 messages ? Ce nombre est-il égal au nombre de messages ? Pourquoi ?

Exercice 8

Le réseau d'un établissement scolaire relie trois salles à deux commutateurs :

```
1 reseau = {
2     "Salle1": ["Switch1"],
3     "Salle2": ["Switch1"],
4     "Switch1": ["Salle1", "Salle2", "Switch2"],
5     "Switch2": ["Switch1", "Salle3"],
6     "Salle3": ["Switch2"],
7 }
```

1. En utilisant `peuvent_communiquer` du cours, vérifier que "Salle1" et "Salle3" peuvent communiquer, bien qu'elles ne soient reliées à aucun commutateur commun directement.
2. Ajouter une machine "SalleIsolee" qui n'est reliée à **aucune** autre machine du réseau (dictionnaire avec une liste vide). Vérifie qu'elle ne peut communiquer avec aucune autre salle.

3. Pourquoi la fonction marque-t-elle les sommets « visités » pendant sa recherche ? Que risquerait-il de se passer si ce réseau contenait un cycle (par exemple si Switch1 et Switch2 étaient reliés par **deux** câbles distincts, modélisés comme une arête supplémentaire) et que cette précaution était absente ?

Exercice 9 ◆◆

Un lampadaire de rue s'allume automatiquement lorsque la luminosité ambiante descend sous 300 lux.

1. Dans ce système, quel est le **capteur** ? Quel est l'**actionneur** ?
2. Écrire une fonction `decider_eclairage(luminosite, seuil=300)` qui renvoie True si le lampadaire doit s'allumer.
3. Tester cette fonction pour une luminosité de 150 lux (nuit), 500 lux (plein jour), et exactement 300 lux (le seuil). Le comportement au seuil exact te semble-t-il cohérent avec l'énoncé (« sous 300 lux ») ?

Exercice 10 ◆◆◆

Cahier des charges d'un système d'arrosage automatique : un capteur mesure l'humidité du sol (en %) ; la pompe se **déclenche** quand l'humidité descend sous 30 %, et ne s'**arrête** que lorsque l'humidité dépasse 60 % (pour éviter des démarrages/arrêts trop fréquents de la pompe — c'est le principe d'**hystérésis** déjà rencontré pour le thermostat du cours).

1. Écrire une fonction `irrigation_hysteresis(humidite, seuil_bas=30, seuil_haut=60)` qui met à jour et renvoie l'état `etat["pompe"]` (initialement False), en respectant ce cahier des charges.
2. Tester ta fonction sur la séquence de mesures [20, 35, 50, 65, 40] (une mesure par heure). Donner la séquence des états de la pompe.
3. À la mesure 35 (deuxième heure), la pompe est-elle active ? Pourquoi, alors que 35 % est supérieur au seuil bas de 30 % ?

Exercice 11 ◆

On utilise la fonction de découpage du cours. Le message "PROTOCOLERESEAU" est découpé en paquets de taille maximale 5.

1. Combien de paquets sont créés ? Donner le contenu (donnees) de chacun.
2. On suppose que les paquets arrivent au destinataire dans l'ordre suivant : paquet 2, puis paquet 0, puis paquet 1. Le message est-il correctement reconstitué par reassembler malgré cet ordre d'arrivée ? Pourquoi ?
3. Que se passerait-il si l'on reconstituait le message en concaténant simplement les paquets **dans leur ordre d'arrivée**, sans utiliser leur numéro ?

Exercice 12 ◆

On envoie les messages ["X", "Y", "Z"] avec le protocole du bit alterné du cours. La première tentative globale (numéro 1) et la quatrième tentative globale (numéro 4) échouent (paquet ou ACK perdu) ; toutes les autres réussissent.

1. Dérouler, tentative par tentative, ce qui se passe : quel message est concerné par chaque tentative, réussit-elle, et quel est le bit utilisé ?
2. Donner la liste finale des messages effectivement reçus par le destinataire, avec leur bit.
3. Combien de tentatives ont été nécessaires au total pour transmettre les 3 messages ? Ce nombre est-il égal au nombre de messages ? Pourquoi ?

Exercice 13 ◆◆

Le réseau d'un établissement scolaire relie trois salles à deux commutateurs :


```

1 reseau = {
2     "Salle1": ["Switch1"],
3     "Salle2": ["Switch1"],
4     "Switch1": ["Salle1", "Salle2", "Switch2"],
5     "Switch2": ["Switch1", "Salle3"],
6     "Salle3": ["Switch2"],
7 }

```

1. En utilisant peuvent_communiquer du cours, vérifier que "Salle1" et "Salle3" peuvent communiquer, bien qu'elles ne soient reliées à aucun commutateur commun directement.
2. Ajouter une machine "SalleIsolee" qui n'est reliée à **aucune** autre machine du réseau (dictionnaire avec une liste vide). Vérifie qu'elle ne peut communiquer avec aucune autre salle.
3. Pourquoi la fonction marque-t-elle les sommets « visités » pendant sa recherche ? Que risquerait-il de se passer si ce réseau contenait un cycle (par exemple si Switch1 et Switch2 étaient reliés par **deux** câbles distincts, modélisés comme une arête supplémentaire) et que cette précaution était absente ?

Exercice 14 ♦♦

Un lampadaire de rue s'allume automatiquement lorsque la luminosité ambiante descend sous 300 lux.

1. Dans ce système, quel est le **capteur** ? Quel est l'**actionneur** ?
2. Écrire une fonction decider_eclairage(luminosite, seuil=300) qui renvoie True si le lampadaire doit s'allumer.
3. Tester cette fonction pour une luminosité de 150 lux (nuit), 500 lux (plein jour), et exactement 300 lux (le seuil). Le comportement au seuil exact te semble-t-il cohérent avec l'énoncé (« sous 300 lux ») ?

Exercice 15 ♦♦♦

Cahier des charges d'un système d'arrosage automatique : un capteur mesure l'humidité du sol (en %) ; la pompe se **déclenche** quand l'humidité descend sous 30 %, et ne s'**arrête** que lorsque l'humidité dépasse 60 % (pour éviter des démarrages/arrêts trop fréquents de la pompe — c'est le principe d'**hystérésis** déjà rencontré pour le thermostat du cours).

1. Écrire une fonction irrigation_hysteresis(humidite, seuil_bas=30, seuil_haut=60) qui met à jour et renvoie l'état etat["pompe"] (initialement False), en respectant ce cahier des charges.
2. Tester ta fonction sur la séquence de mesures [20, 35, 50, 65, 40] (une mesure par heure). Donner la séquence des états de la pompe.
3. À la mesure 35 (deuxième heure), la pompe est-elle active ? Pourquoi, alors que 35 % est supérieur au seuil bas de 30 % ?

Exercice 16 ♦

On utilise la fonction de découpage du cours. Le message "PROTOCOLERESEAU" est découpé en paquets de taille maximale 5.

1. Combien de paquets sont créés ? Donner le contenu (donnees) de chacun.
2. On suppose que les paquets arrivent au destinataire dans l'ordre suivant : paquet 2, puis paquet 0, puis paquet 1. Le message est-il correctement reconstitué par rassembler malgré cet ordre d'arrivée ? Pourquoi ?
3. Que se passerait-il si l'on reconstituait le message en concaténant simplement les paquets **dans leur ordre d'arrivée**, sans utiliser leur numéro ?

Exercice 17

On envoie les messages ["X", "Y", "Z"] avec le protocole du bit alterné du cours. La première tentative globale (numéro 1) et la quatrième tentative globale (numéro 4) échouent (paquet ou ACK perdu) ; toutes les autres réussissent.

1. Dérouler, tentative par tentative, ce qui se passe : quel message est concerné par chaque tentative, réussit-elle, et quel est le bit utilisé ?
2. Donner la liste finale des messages effectivement reçus par le destinataire, avec leur bit.
3. Combien de tentatives ont été nécessaires au total pour transmettre les 3 messages ? Ce nombre est-il égal au nombre de messages ? Pourquoi ?

Exercice 18

Le réseau d'un établissement scolaire relie trois salles à deux commutateurs :

```
1 reseau = {
2     "Salle1": ["Switch1"],
3     "Salle2": ["Switch1"],
4     "Switch1": ["Salle1", "Salle2", "Switch2"],
5     "Switch2": ["Switch1", "Salle3"],
6     "Salle3": ["Switch2"],
7 }
```

1. En utilisant `peuvent_communiquer` du cours, vérifier que "Salle1" et "Salle3" peuvent communiquer, bien qu'elles ne soient reliées à aucun commutateur commun directement.
2. Ajouter une machine "SalleIsolee" qui n'est reliée à **aucune** autre machine du réseau (dictionnaire avec une liste vide). Vérifie qu'elle ne peut communiquer avec aucune autre salle.
3. Pourquoi la fonction marque-t-elle les sommets « visités » pendant sa recherche ? Que risquerait-il de se passer si ce réseau contenait un cycle (par exemple si Switch1 et Switch2 étaient reliés par **deux** câbles distincts, modélisés comme une arête supplémentaire) et que cette précaution était absente ?

Exercice 19

Un lampadaire de rue s'allume automatiquement lorsque la luminosité ambiante descend sous 300 lux.

1. Dans ce système, quel est le **capteur** ? Quel est l'**actionneur** ?
2. Écrire une fonction `decider_eclairage(luminosite, seuil=300)` qui renvoie True si le lampadaire doit s'allumer.
3. Tester cette fonction pour une luminosité de 150 lux (nuit), 500 lux (plein jour), et exactement 300 lux (le seuil). Le comportement au seuil exact te semble-t-il cohérent avec l'énoncé (« sous 300 lux ») ?

Exercice 20

Cahier des charges d'un système d'arrosage automatique : un capteur mesure l'humidité du sol (en %) ; la pompe se **déclenche** quand l'humidité descend sous 30 %, et ne s'**arrête** que lorsque l'humidité dépasse 60 % (pour éviter des démarrages/arrêts trop fréquents de la pompe — c'est le principe d'**hystérésis** déjà rencontré pour le thermostat du cours).

1. Écrire une fonction `irrigation_hysteresis(humidite, seuil_bas=30, seuil_haut=60)` qui met à jour et renvoie l'état `etat["pompe"]` (initialement False), en respectant ce cahier des charges.
2. Tester ta fonction sur la séquence de mesures [20, 35, 50, 65, 40] (une mesure par heure). Donner la séquence des états de la pompe.
3. À la mesure 35 (deuxième heure), la pompe est-elle active ? Pourquoi, alors que 35 % est supérieur au seuil bas de 30 % ?

Exercice 21

On utilise la fonction de découpage du cours. Le message "PROTOCOLERESEAU" est découpé en paquets de taille maximale 5.

1. Combien de paquets sont créés ? Donner le contenu (donnees) de chacun.
2. On suppose que les paquets arrivent au destinataire dans l'ordre suivant : paquet 2, puis paquet 0, puis paquet 1. Le message est-il correctement reconstitué par reassembler malgré cet ordre d'arrivée ? Pourquoi ?
3. Que se passerait-il si l'on reconstituait le message en concaténant simplement les paquets **dans leur ordre d'arrivée**, sans utiliser leur numéro ?

Exercice 22

On envoie les messages ["X", "Y", "Z"] avec le protocole du bit alterné du cours. La première tentative globale (numéro 1) et la quatrième tentative globale (numéro 4) échouent (paquet ou ACK perdu) ; toutes les autres réussissent.

1. Dérouler, tentative par tentative, ce qui se passe : quel message est concerné par chaque tentative, réussit-elle, et quel est le bit utilisé ?
2. Donner la liste finale des messages effectivement reçus par le destinataire, avec leur bit.
3. Combien de tentatives ont été nécessaires au total pour transmettre les 3 messages ? Ce nombre est-il égal au nombre de messages ? Pourquoi ?

Exercice 23

Le réseau d'un établissement scolaire relie trois salles à deux commutateurs :

```
1 reseau = {
2   "Salle1": ["Switch1"],
3   "Salle2": ["Switch1"],
4   "Switch1": ["Salle1", "Salle2", "Switch2"],
5   "Switch2": ["Switch1", "Salle3"],
6   "Salle3": ["Switch2"],
7 }
```

1. En utilisant peuvent_communiquer du cours, vérifier que "Salle1" et "Salle3" peuvent communiquer, bien qu'elles ne soient reliées à aucun commutateur commun directement.
2. Ajouter une machine "SalleIsolee" qui n'est reliée à **aucune** autre machine du réseau (dictionnaire avec une liste vide). Vérifie qu'elle ne peut communiquer avec aucune autre salle.
3. Pourquoi la fonction marque-t-elle les sommets « visités » pendant sa recherche ? Que risquerait-il de se passer si ce réseau contenait un cycle (par exemple si Switch1 et Switch2 étaient reliés par **deux** câbles distincts, modélisés comme une arête supplémentaire) et que cette précaution était absente ?

Exercice 24

Un lampadaire de rue s'allume automatiquement lorsque la luminosité ambiante descend sous 300 lux.

1. Dans ce système, quel est le **capteur** ? Quel est l'**actionneur** ?
2. Écrire une fonction decider_eclairage(luminosite, seuil=300) qui renvoie True si le lampadaire doit s'allumer.
3. Tester cette fonction pour une luminosité de 150 lux (nuit), 500 lux (plein jour), et exactement 300 lux (le seuil). Le comportement au seuil exact te semble-t-il cohérent avec l'énoncé (« sous 300 lux ») ?

Exercice 25 ◆◆◆

Cahier des charges d'un système d'arrosage automatique : un capteur mesure l'humidité du sol (en %) ; la pompe se **déclenche** quand l'humidité descend sous 30 %, et ne s'**arrête** que lorsque l'humidité dépasse 60 % (pour éviter des démarrages/arrêts trop fréquents de la pompe — c'est le principe d'**hystérésis** déjà rencontré pour le thermostat du cours).

1. Écrire une fonction `irrigation_hysteresis(humidite, seuil_bas=30, seuil_haut=60)` qui met à jour et renvoie l'état `etat["pompe"]` (initialement `False`), en respectant ce cahier des charges.
2. Tester ta fonction sur la séquence de mesures `[20, 35, 50, 65, 40]` (une mesure par heure). Donner la séquence des états de la pompe.
3. À la mesure 35 (deuxième heure), la pompe est-elle active ? Pourquoi, alors que 35 % est supérieur au seuil bas de 30 % ?

Exercice 26 ◆

On utilise la fonction de découpage du cours. Le message "PROTOCOLERESEAU" est découpé en paquets de taille maximale 5.

1. Combien de paquets sont créés ? Donner le contenu (`donnees`) de chacun.
2. On suppose que les paquets arrivent au destinataire dans l'ordre suivant : paquet 2, puis paquet 0, puis paquet 1. Le message est-il correctement reconstitué par reassembler malgré cet ordre d'arrivée ? Pourquoi ?
3. Que se passerait-il si l'on reconstituait le message en concaténant simplement les paquets **dans leur ordre d'arrivée**, sans utiliser leur numéro ?

Exercice 27 ◆

On envoie les messages ["X", "Y", "Z"] avec le protocole du bit alterné du cours. La première tentative globale (numéro 1) et la quatrième tentative globale (numéro 4) échouent (paquet ou ACK perdu) ; toutes les autres réussissent.

1. Dérouler, tentative par tentative, ce qui se passe : quel message est concerné par chaque tentative, réussit-elle, et quel est le bit utilisé ?
2. Donner la liste finale des messages effectivement reçus par le destinataire, avec leur bit.
3. Combien de tentatives ont été nécessaires au total pour transmettre les 3 messages ? Ce nombre est-il égal au nombre de messages ? Pourquoi ?

Exercice 28 ◆◆

Le réseau d'un établissement scolaire relie trois salles à deux commutateurs :

```
1 reseau = {
2     "Salle1": ["Switch1"],
3     "Salle2": ["Switch1"],
4     "Switch1": ["Salle1", "Salle2", "Switch2"],
5     "Switch2": ["Switch1", "Salle3"],
6     "Salle3": ["Switch2"],
7 }
```

1. En utilisant `peuvent_communiquer` du cours, vérifier que "Salle1" et "Salle3" peuvent communiquer, bien qu'elles ne soient reliées à aucun commutateur commun directement.
2. Ajouter une machine "SalleIsolee" qui n'est reliée à **aucune** autre machine du réseau (dictionnaire avec une liste vide). Vérifie qu'elle ne peut communiquer avec aucune autre salle.
3. Pourquoi la fonction marque-t-elle les sommets « visités » pendant sa recherche ? Que risquerait-il de se passer si ce réseau contenait un cycle (par exemple si Switch1 et Switch2 étaient reliés par **deux** câbles distincts, modélisés comme une arête supplémentaire) et que cette précaution était absente ?

Exercice 29 ♦♦

Un lampadaire de rue s'allume automatiquement lorsque la luminosité ambiante descend sous 300 lux.

1. Dans ce système, quel est le **capteur** ? Quel est l'**actionneur** ?
2. Écrire une fonction `decider_eclairage(luminosite, seuil=300)` qui renvoie `True` si le lampadaire doit s'allumer.
3. Tester cette fonction pour une luminosité de 150 lux (nuit), 500 lux (plein jour), et exactement 300 lux (le seuil). Le comportement au seuil exact te semble-t-il cohérent avec l'énoncé (« sous 300 lux ») ?

Exercice 30 ♦♦♦

Cahier des charges d'un système d'arrosage automatique : un capteur mesure l'humidité du sol (en %) ; la pompe se **déclenche** quand l'humidité descend sous 30 %, et ne s'**arrête** que lorsque l'humidité dépasse 60 % (pour éviter des démarrages/arrêts trop fréquents de la pompe — c'est le principe d'**hystérésis** déjà rencontré pour le thermostat du cours).

1. Écrire une fonction `irrigation_hysteresis(humidite, seuil_bas=30, seuil_haut=60)` qui met à jour et renvoie l'état `etat["pompe"]` (initialement `False`), en respectant ce cahier des charges.
2. Tester ta fonction sur la séquence de mesures [20, 35, 50, 65, 40] (une mesure par heure). Donner la séquence des états de la pompe.
3. À la mesure 35 (deuxième heure), la pompe est-elle active ? Pourquoi, alors que 35 % est supérieur au seuil bas de 30 % ?

QCM d'auto-évaluation — Réseaux : transmission de données

Pour chaque question, une seule réponse est exacte. Entourez la lettre correspondante.

C1 — Paquets et encapsulation

1. [Q1] Dans l'en-tête d'un paquet, le numéro de paquet sert principalement à :
 - A. chiffrer les données transmises.
 - B. réordonner les paquets à l'arrivée, quel que soit leur ordre de réception.
 - C. accélérer la transmission.
 - D. indiquer la taille du disque du destinataire.

C2 — Protocole du bit alterné

2. [Q2] Si l'émetteur ne reçoit pas l'accusé de réception attendu, il :
 - A. abandonne définitivement l'envoi du message.
 - B. retransmet le paquet avec le même bit.
 - C. retransmet le paquet avec le bit opposé.
 - D. envoie directement le paquet suivant.

C3 — Simulation de réseau

3. [Q3] Pour éviter qu'une recherche de chemin dans un réseau contenant un cycle ne boucle indéfiniment, il faut :
 - A. interdire tout cycle dans le réseau.
 - B. marquer les sommets déjà visités.

- C. limiter le réseau à deux machines.
- D. ne jamais utiliser de fonction récursive.

C4 — Capteurs et actionneurs

4. [Q4] Un capteur de température et un relais de chauffage jouent respectivement les rôles de :
- A. actionneur puis capteur.
 - B. capteur puis actionneur.
 - C. deux capteurs.
 - D. deux actionneurs.

C5 — IHM programmée

5. [Q5] Programmer une IHM à partir d'un cahier des charges nécessite en particulier de :
- A. tester uniquement le cas de démonstration prévu.
 - B. tester aussi les cas limites (valeurs au seuil, changements d'état).
 - C. éviter tout test, le code étant supposé correct.
 - D. ignorer les exigences non précisées explicitement.
6. [Q6] Le principe d'hystérésis (deux seuils différents pour activer/désactiver un actionneur) sert à :
- A. rendre le système plus lent.
 - B. éviter des activations/désactivations trop fréquentes autour d'un seul seuil.
 - C. économiser de la mémoire.
 - D. remplacer le capteur par un actionneur.

Corrigé

1. 3 paquets sont créés : "DONN" (numéro 0), "EESN" (numéro 1), "SI" (numéro 2).
2. Tentative 1 : envoi de "P" avec bit 0 — échoue. Tentative 2 : retransmission de "P" avec bit 0 — réussit. Tentative 3 : envoi de "Q" avec bit 1 — réussit. Messages reçus : [(0, "P"), (1, "Q")].
3. L'émetteur **retransmet** systématiquement tant qu'il ne reçoit pas l'accusé de réception attendu : un message n'est jamais considéré comme envoyé tant que sa réception n'a pas été confirmée, ce qui garantit qu'il finit par arriver (en l'absence de perte permanente).

Corrigé

1. peuvent_communiquer(reseau, "A", "C") renvoie True : il existe un chemin A → B → C.
- 2.

```
1 def decider_arrosage(humidite, seuil=25):
2     return humidite < seuil
```

3. decider_arrosage(20) vaut True (sol sec, il faut arroser) ; decider_arrosage(30) vaut False (sol suffisamment humide). Le **capteur** est le capteur d'humidité du sol ; l'**actionneur** est la pompe d'arrosage.

Évaluation A — Réseaux

Durée : 45 minutes. Ordinateur avec interpréteur Python autorisé.

Barème indicatif : Ex. 1 : 10 pts (paquets, bit alterné) ; Ex. 2 : 10 pts (réseau, capteurs)

Exercice 1 — Paquets et bit alterné

(10 points)

1. (4 pts) Découper le message suivant en paquets de taille maximale 4 avec `decouper_en_paquets` du cours :

"DONNEESNSI"

Combien de paquets sont créés, et que contiennent-ils ?

2. (3 pts) On envoie les messages ["P", "Q"] avec le protocole du bit alterné ; la première tentative globale échoue. Dérouler l'envoi et donner la liste finale des messages reçus, avec leur bit.
3. (3 pts) Pourquoi le protocole du bit alterné garantit-il que tous les messages finissent par arriver, même en cas de pertes ponctuelles ?

Exercice 2 — Réseau et capteurs

(10 points)

1. (4 pts) On donne le réseau suivant :

`{"A": ["B"], "B": ["A", "C"], "C": ["B"]}`

En utilisant `peuvent_communiquer` du cours, vérifier que "A" et "C" peuvent communiquer.

2. (3 pts) Écrire une fonction `decider_arrosage(humidite, seuil=25)` qui renvoie `True` si l'humidité est inférieure au seuil (la pompe doit s'activer).
3. (3 pts) Tester cette fonction pour une humidité de 20 % et de 30 %. Identifier, dans ce système, le capteur et l'actionneur.

Corrigé

1. 3 paquets sont créés : "PAQUE" (numéro 0), "TSRES" (numéro 1), "EAU" (numéro 2).

2. Tentative 1 : envoi de "M" avec bit 0 — réussit. Tentative 2 : envoi de "N" avec bit 1 — échoue. Tentative 3 : retransmission de "N" avec bit 1 — réussit. Tentative 4 : envoi de "O" avec bit 0 — réussit. Messages reçus : [(0, "M"), (1, "N"), (0, "O")].

3. Les paquets sont envoyés **un par un** : l'émetteur attend toujours l'ACK avant d'envoyer le suivant. Un seul bit suffit donc à distinguer « un nouveau paquet » (bit différent du précédent) d'« une retransmission » (même bit que le paquet précédent, non encore confirmé).

Corrigé

1. `peuvent_communiquer(reseau, "X", "Z")` renvoie `True` : il existe un chemin $X \rightarrow Y \rightarrow Z$.

2.

```
1 def decider_ventilation(co2, seuil=800):
2     return co2 > seuil
```

3. `decider_ventilation(1000)` vaut `True` (air vicié, il faut ventiler) ; `decider_ventilation(500)` vaut `False` (air correct). Le **capteur** est le capteur de CO2 ; l'**actionneur** est le système de ventilation.

Évaluation B — Réseaux

Durée : 45 minutes. Ordinateur avec interpréteur Python autorisé.

Barème indicatif : Ex. 1 : 10 pts (paquets, bit alterné) ; Ex. 2 : 10 pts (réseau, capteurs)

Exercice 1 — Paquets et bit alterné

(10 points)

1. (4 pts) Découper le message suivant en paquets de taille maximale 5 avec `decouper_en_paquets` du cours :

"PAQUETSRESEAU"

Combien de paquets sont créés, et que contiennent-ils ?

2. (3 pts) On envoie les messages ["M", "N", "O"] avec le protocole du bit alterné ; la deuxième tentative globale échoue. Dérouler l'envoi et donner la liste finale des messages reçus, avec leur bit.
3. (3 pts) Pourquoi un seul bit (0 ou 1) suffit-il à distinguer un nouveau paquet d'une retransmission, dans ce protocole ?

Exercice 2 — Réseau et capteurs

(10 points)

1. (4 pts) On donne le réseau suivant :

```
{ "X": [ "Y" ], "Y": [ "X", "Z" ], "Z": [ "Y" ] }
```

En utilisant peuvent_communiquer du cours, vérifier que "X" et "Z" peuvent communiquer.

2. (3 pts) Écrire une fonction decider_ventilation(co2, seuil=800) qui renvoie True si le taux de CO2 (en ppm) dépasse le seuil (la ventilation doit s'activer).
3. (3 pts) Tester cette fonction pour un taux de 1000 ppm et de 500 ppm. Identifier, dans ce système, le capteur et l'actionneur.

Exercice 31 ◆

Le message "RESEauxNSI" est découpé en 4 paquets de taille maximale 3. Ils arrivent au destinataire dans l'ordre : paquet 3, puis 1, puis 0, puis 2. Un élève réassemble le message en concaténant simplement les paquets **dans leur ordre d'arrivée**, sans les trier :

```
1 ordre_arrivee = [paquets[3], paquets[1], paquets[0], paquets[2]]
2 message_reconstitue = "".join(p["donnees"] for p in ordre_arrivee)
```

1. Que contient message_reconstitue avec cette méthode ? Est-ce le message d'origine ?
2. Pourquoi cette méthode échoue-t-elle ici, alors qu'elle fonctionnerait si les paquets arrivaient dans l'ordre 0, 1, 2, 3 ?
3. Corriger le réassemblage pour qu'il fonctionne quel que soit l'ordre d'arrivée.

Exercice 32 ◆

Le message "RESEauxNSI" est découpé en 4 paquets de taille maximale 3. Ils arrivent au destinataire dans l'ordre : paquet 3, puis 1, puis 0, puis 2. Un élève réassemble le message en concaténant simplement les paquets **dans leur ordre d'arrivée**, sans les trier :

```
1 ordre_arrivee = [paquets[3], paquets[1], paquets[0], paquets[2]]
2 message_reconstitue = "".join(p["donnees"] for p in ordre_arrivee)
```

1. Que contient message_reconstitue avec cette méthode ? Est-ce le message d'origine ?
2. Pourquoi cette méthode échoue-t-elle ici, alors qu'elle fonctionnerait si les paquets arrivaient dans l'ordre 0, 1, 2, 3 ?
3. Corriger le réassemblage pour qu'il fonctionne quel que soit l'ordre d'arrivée.

Exercice 33 ◆

Le message "RESEauxNSI" est découpé en 4 paquets de taille maximale 3. Ils arrivent au destinataire dans l'ordre : paquet 3, puis 1, puis 0, puis 2. Un élève réassemble le message en concaténant simplement les paquets **dans leur ordre d'arrivée**, sans les trier :


```
1 ordre_arrivee = [paquets[3], paquets[1], paquets[0], paquets[2]]
2 message_reconstitue = "".join(p["donnees"] for p in ordre_arrivee)
```

1. Que contient `message_reconstitue` avec cette méthode ? Est-ce le message d'origine ?
2. Pourquoi cette méthode échoue-t-elle ici, alors qu'elle fonctionnerait si les paquets arrivaient dans l'ordre 0,1,2,3 ?
3. Corriger le réassemblage pour qu'il fonctionne quel que soit l'ordre d'arrivée.

Exercice 34 ◆

Le message "RESEauxNSI" est découpé en 4 paquets de taille maximale 3. Ils arrivent au destinataire dans l'ordre : paquet 3, puis 1, puis 0, puis 2. Un élève réassemble le message en concaténant simplement les paquets **dans leur ordre d'arrivée**, sans les trier :

```
1 ordre_arrivee = [paquets[3], paquets[1], paquets[0], paquets[2]]
2 message_reconstitue = "".join(p["donnees"] for p in ordre_arrivee)
```

1. Que contient `message_reconstitue` avec cette méthode ? Est-ce le message d'origine ?
2. Pourquoi cette méthode échoue-t-elle ici, alors qu'elle fonctionnerait si les paquets arrivaient dans l'ordre 0,1,2,3 ?
3. Corriger le réassemblage pour qu'il fonctionne quel que soit l'ordre d'arrivée.

Exercice 35 ◆

Le message "RESEauxNSI" est découpé en 4 paquets de taille maximale 3. Ils arrivent au destinataire dans l'ordre : paquet 3, puis 1, puis 0, puis 2. Un élève réassemble le message en concaténant simplement les paquets **dans leur ordre d'arrivée**, sans les trier :

```
1 ordre_arrivee = [paquets[3], paquets[1], paquets[0], paquets[2]]
2 message_reconstitue = "".join(p["donnees"] for p in ordre_arrivee)
```

1. Que contient `message_reconstitue` avec cette méthode ? Est-ce le message d'origine ?
2. Pourquoi cette méthode échoue-t-elle ici, alors qu'elle fonctionnerait si les paquets arrivaient dans l'ordre 0,1,2,3 ?
3. Corriger le réassemblage pour qu'il fonctionne quel que soit l'ordre d'arrivée.

Exercice 36 ◆

Le message "RESEauxNSI" est découpé en 4 paquets de taille maximale 3. Ils arrivent au destinataire dans l'ordre : paquet 3, puis 1, puis 0, puis 2. Un élève réassemble le message en concaténant simplement les paquets **dans leur ordre d'arrivée**, sans les trier :

```
1 ordre_arrivee = [paquets[3], paquets[1], paquets[0], paquets[2]]
2 message_reconstitue = "".join(p["donnees"] for p in ordre_arrivee)
```

1. Que contient `message_reconstitue` avec cette méthode ? Est-ce le message d'origine ?
2. Pourquoi cette méthode échoue-t-elle ici, alors qu'elle fonctionnerait si les paquets arrivaient dans l'ordre 0,1,2,3 ?
3. Corriger le réassemblage pour qu'il fonctionne quel que soit l'ordre d'arrivée.

Corrigé

1. 3 paquets sont créés : {"numero": 0, "donnees": "PROTO"}, {"numero": 1, "donnees": "COLER"}, {"numero": 2, "donnees": "ESEAU"}.

- Oui, le message est correctement reconstitué : "PROTOCOLERESEAU". reassembler **trie** les paquets selon leur champ numero avant de les concaténer, donc l'ordre d'arrivée n'a aucune importance.
- En concaténant simplement dans l'ordre d'arrivée (paquet 2, puis 0, puis 1), on obtiendrait "ESEAUPROTOCOLER" : un message **incorrect** et incompréhensible. C'est exactement pour éviter ce problème que chaque paquet porte un numéro dans son en-tête.

Corrigé

- Tentative 1 : envoi de "X" avec bit 0 — **échoue** (perdue). Tentative 2 : retransmission de "X" avec bit 0 — **réussit**. Tentative 3 : envoi de "Y" avec bit 1 — **réussit**. Tentative 4 : envoi de "Z" avec bit 0 — **échoue** (perdue). Tentative 5 : retransmission de "Z" avec bit 0 — **réussit**.
- Le destinataire reçoit finalement : [(0, "X"), (1, "Y"), (0, "Z")].
- 5 tentatives ont été nécessaires pour 3 messages : ce nombre est **supérieur** au nombre de messages, car 2 tentatives ont échoué et ont dû être retransmises (une pour "X", une pour "Z").

Corrigé

- peuvent_communiquer(reseau, "Salle1", "Salle3") renvoie True : il existe un chemin Salle1 → Switch1 → Switch2 → Salle3.
- Avec reseau["SalleIsolee"] = [], la fonction renvoie False : aucun chemin n'existe vers une machine qui n'a aucune connexion.
- Le marquage des sommets visités évite de **reparcourir indéfiniment** un même sommet. Sans cette précaution, un cycle (comme deux câbles entre Switch1 et Switch2) provoquerait des appels récursifs sans fin : la fonction repasserait continuellement entre les deux commutateurs, sans jamais terminer (**boucle infinie** / dépassement de la profondeur de récursion).

Corrigé

- Le **capteur** est le capteur de luminosité (photorésistance ou capteur optique). L'**actionneur** est l'ampoule du lampadaire (via son relais électrique).

2.

```
1 def decider_eclairage(luminosite, seuil=300):
2     return luminosite < seuil
```

- À 150 lux : True (le lampadaire s'allume, il fait sombre). À 500 lux : False (il fait jour). À exactement 300 lux : False (le test est <, strictement inférieur), ce qui est cohérent avec l'énoncé « sous 300 lux » : à 300 lux pile, on n'est pas encore *sous* le seuil.

Corrigé

1.

```
1 etat = {"pompe": False}
2
3 def irrigation_hysteresis(humidite, seuil_bas=30, seuil_haut=60):
4     if etat["pompe"]:
5         if humidite > seuil_haut:
6             etat["pompe"] = False
7     else:
8         if humidite < seuil_bas:
9             etat["pompe"] = True
10    return etat["pompe"]
```

- La séquence des états est [True, True, True, False, False].
- Oui, la pompe est active à la mesure 35. Elle a été **déclenchée** à la mesure précédente (20 % < 30 %), et le cahier des charges précise qu'elle ne s'**arrête** que lorsque l'humidité dépasse 60 % : à 35 %, elle s'arrête.

ce seuil haut n'est pas encore atteint, donc la pompe continue de fonctionner même si l'humidité n'est plus sous le seuil bas.

Corrigé

1. message_reconstitue vaut "IEAURESXNS" : ce n'est **pas** le message d'origine ("RESEauxXNSI").
2. Concaténer dans l'ordre d'arrivée ne fonctionne que si les paquets arrivent **déjà dans l'ordre d'envoi** (0, 1, 2, 3). Sur un vrai réseau, rien ne garantit cet ordre (les paquets peuvent emprunter des chemins différents, avec des délais différents) : il faut donc systématiquement se fier au **numéro** inscrit dans l'en-tête, jamais à l'ordre d'arrivée.
3. Il faut trier les paquets par leur champ numero avant de les concaténer :

```
1 paquets_tries = sorted(ordre_arrivee, key=lambda p: p["numero"])
2 message_correct = "".join(p["donnees"] for p in paquets_tries)
```

Corrigé

1. 3 paquets sont créés : {"numero": 0, "donnees": "PROTO"}, {"numero": 1, "donnees": "COLER"}, {"numero": 2, "donnees": "ESEAU"}.
2. Oui, le message est correctement reconstitué : "PROTOCOLERESEAU". reassembler **trie** les paquets selon leur champ numero avant de les concaténer, donc l'ordre d'arrivée n'a aucune importance.
3. En concaténant simplement dans l'ordre d'arrivée (paquet 2, puis 0, puis 1), on obtiendrait "ESEAPROTOCOLER" : un message **incorrect** et incompréhensible. C'est exactement pour éviter ce problème que chaque paquet porte un numéro dans son en-tête.

Corrigé

1. Tentative 1 : envoi de "X" avec bit 0 — **échoue** (perdue). Tentative 2 : retransmission de "X" avec bit 0 — **réussit**. Tentative 3 : envoi de "Y" avec bit 1 — **réussit**. Tentative 4 : envoi de "Z" avec bit 0 — **échoue** (perdue). Tentative 5 : retransmission de "Z" avec bit 0 — **réussit**.
2. Le destinataire reçoit finalement : [(0, "X"), (1, "Y"), (0, "Z")].
3. 5 tentatives ont été nécessaires pour 3 messages : ce nombre est **supérieur** au nombre de messages, car 2 tentatives ont échoué et ont dû être retransmises (une pour "X", une pour "Z").

Corrigé

1. peuvent_communiquer(reseau, "Salle1", "Salle3") renvoie True : il existe un chemin Salle1 → Switch1 → Switch2 → Salle3.
2. Avec reseau["SalleIsolee"] = [], la fonction renvoie False : aucun chemin n'existe vers une machine qui n'a aucune connexion.
3. Le marquage des sommets visités évite de **reparcourir indéfiniment** un même sommet. Sans cette précaution, un cycle (comme deux câbles entre Switch1 et Switch2) provoquerait des appels récursifs sans fin : la fonction repasserait continuellement entre les deux commutateurs, sans jamais terminer (**boucle infinie** / dépassement de la profondeur de récursion).

Corrigé

1. Le **capteur** est le capteur de luminosité (photorésistance ou capteur optique). L'**actionneur** est l'ampoule du lampadaire (via son relais électrique).
- 2.

```
1 def decider_eclairage(luminosite, seuil=300):
2     return luminosite < seuil
```

3. À 150 lux : True (le lampadaire s'allume, il fait sombre). À 500 lux : False (il fait jour). À exactement 300 lux : False (le test est $<$, strictement inférieur), ce qui est cohérent avec l'énoncé « sous 300 lux » : à 300 lux pile, on n'est pas encore *sous* le seuil.

Corrigé

1.

```
1  etat = {"pompe": False}
2
3  def irrigation_hysteresis(humidite, seuil_bas=30, seuil_haut=60):
4      if etat["pompe"]:
5          if humidite > seuil_haut:
6              etat["pompe"] = False
7      else:
8          if humidite < seuil_bas:
9              etat["pompe"] = True
10     return etat["pompe"]
```

2. La séquence des états est [True, True, True, False, False].

3. Oui, la pompe est active à la mesure 35. Elle a été **déclenchée** à la mesure précédente (20 % < 30 %), et le cahier des charges précise qu'elle ne s'**arrête** que lorsque l'humidité dépasse 60 % : à 35 %, ce seuil haut n'est pas encore atteint, donc la pompe continue de fonctionner même si l'humidité n'est plus sous le seuil bas.

Corrigé

1. message_reconstitue vaut "IEAURESXNS" : ce n'est **pas** le message d'origine ("RESEauxNSI").

2. Concaténer dans l'ordre d'arrivée ne fonctionne que si les paquets arrivent **déjà dans l'ordre d'envoi** (0, 1, 2, 3). Sur un vrai réseau, rien ne garantit cet ordre (les paquets peuvent emprunter des chemins différents, avec des délais différents) : il faut donc systématiquement se fier au **numéro** inscrit dans l'en-tête, jamais à l'ordre d'arrivée.

3. Il faut trier les paquets par leur champ numero avant de les concaténer :

```
1  paquets_tries = sorted(ordre_arrivee, key=lambda p: p["numero"])
2  message_correct = "".join(p["donnees"] for p in paquets_tries)
```

Corrigé

1. 3 paquets sont créés : {"numero": 0, "donnees": "PROTO"}, {"numero": 1, "donnees": "COLER"}, {"numero": 2, "donnees": "ESEAUX"}.

2. Oui, le message est correctement reconstitué : "PROTOCOLERESEAUX". reassembler **trie** les paquets selon leur champ numero avant de les concaténer, donc l'ordre d'arrivée n'a aucune importance.

3. En concaténant simplement dans l'ordre d'arrivée (paquet 2, puis 0, puis 1), on obtiendrait "ESEAUPROTOCOLER" : un message **incorrect** et incompréhensible. C'est exactement pour éviter ce problème que chaque paquet porte un numéro dans son en-tête.

Corrigé

1. Tentative 1 : envoi de "X" avec bit 0 — **échoue** (perdue). Tentative 2 : retransmission de "X" avec bit 0 — **réussit**. Tentative 3 : envoi de "Y" avec bit 1 — **réussit**. Tentative 4 : envoi de "Z" avec bit 0 — **échoue** (perdue). Tentative 5 : retransmission de "Z" avec bit 0 — **réussit**.

2. Le destinataire reçoit finalement : [(0, "X"), (1, "Y"), (0, "Z")].

3. 5 tentatives ont été nécessaires pour 3 messages : ce nombre est **supérieur** au nombre de messages, car 2 tentatives ont échoué et ont dû être retransmises (une pour "X", une pour "Z").

Corrigé

1. `peuvent_communiquer(reseau, "Salle1", "Salle3")` renvoie `True` : il existe un chemin `Salle1 → Switch1 → Switch2 → Salle3`.
2. Avec `reseau["SalleIsolee"] = []`, la fonction renvoie `False` : aucun chemin n'existe vers une machine qui n'a aucune connexion.
3. Le marquage des sommets visités évite de **reparcourir indéfiniment** un même sommet. Sans cette précaution, un cycle (comme deux câbles entre `Switch1` et `Switch2`) provoquerait des appels récursifs sans fin : la fonction repasserait continuellement entre les deux commutateurs, sans jamais terminer (**boucle infinie** / dépassement de la profondeur de récursion).

Corrigé

1. Le **capteur** est le capteur de luminosité (photorésistance ou capteur optique). L'**actionneur** est l'ampoule du lampadaire (via son relais électrique).

2.

```
1 def decider_eclairage(luminosite, seuil=300):
2     return luminosite < seuil
```

3. À 150 lux : `True` (le lampadaire s'allume, il fait sombre). À 500 lux : `False` (il fait jour). À exactement 300 lux : `False` (le test est `<`, strictement inférieur), ce qui est cohérent avec l'énoncé « sous 300 lux » : à 300 lux pile, on n'est pas encore *sous* le seuil.

Corrigé

1.

```
1 etat = {"pompe": False}
2
3 def irrigation_hysteresis(humidite, seuil_bas=30, seuil_haut=60):
4     if etat["pompe"]:
5         if humidite > seuil_haut:
6             etat["pompe"] = False
7     else:
8         if humidite < seuil_bas:
9             etat["pompe"] = True
10    return etat["pompe"]
```

2. La séquence des états est `[True, True, True, False, False]`.

3. Oui, la pompe est active à la mesure 35. Elle a été **déclenchée** à la mesure précédente (20 % < 30 %), et le cahier des charges précise qu'elle ne s'**arrête** que lorsque l'humidité dépasse 60 % : à 35 %, ce seuil haut n'est pas encore atteint, donc la pompe continue de fonctionner même si l'humidité n'est plus sous le seuil bas.

Corrigé

1. `message_reconstitue` vaut `"IEAURESXNS"` : ce n'est **pas** le message d'origine (`"RESEAUXNSI"`).
2. Concaténer dans l'ordre d'arrivée ne fonctionne que si les paquets arrivent **déjà dans l'ordre d'envoi** (0, 1, 2, 3). Sur un vrai réseau, rien ne garantit cet ordre (les paquets peuvent emprunter des chemins différents, avec des délais différents) : il faut donc systématiquement se fier au **numéro** inscrit dans l'en-tête, jamais à l'ordre d'arrivée.
3. Il faut trier les paquets par leur champ `numero` avant de les concaténer :

```
1 paquets_tries = sorted(ordre_arrivee, key=lambda p: p["numero"])
2 message_correct = "".join(p["donnees"] for p in paquets_tries)
```

Corrigé

- 3 paquets sont créés : {"numero": 0, "donnees": "PROTO"}, {"numero": 1, "donnees": "COLER"}, {"numero": 2, "donnees": "ESEAU"}.
- Oui, le message est correctement reconstitué : "PROTOCOLERESEAU". reassembler **trie** les paquets selon leur champ numero avant de les concaténer, donc l'ordre d'arrivée n'a aucune importance.
- En concaténant simplement dans l'ordre d'arrivée (paquet 2, puis 0, puis 1), on obtiendrait "ESEAUPROTOCOLER" : un message **incorrect** et incompréhensible. C'est exactement pour éviter ce problème que chaque paquet porte un numéro dans son en-tête.

Corrigé

- Tentative 1 : envoi de "X" avec bit 0 — **échoue** (perdue). Tentative 2 : retransmission de "X" avec bit 0 — **réussit**. Tentative 3 : envoi de "Y" avec bit 1 — **réussit**. Tentative 4 : envoi de "Z" avec bit 0 — **échoue** (perdue). Tentative 5 : retransmission de "Z" avec bit 0 — **réussit**.
- Le destinataire reçoit finalement : [(0, "X"), (1, "Y"), (0, "Z")].
- 5 tentatives ont été nécessaires pour 3 messages : ce nombre est **supérieur** au nombre de messages, car 2 tentatives ont échoué et ont dû être retransmises (une pour "X", une pour "Z").

Corrigé

- peuvent_communiquer(reseau, "Salle1", "Salle3") renvoie True : il existe un chemin Salle1 → Switch1 → Switch2 → Salle3.
- Avec reseau["SalleIsolee"] = [], la fonction renvoie False : aucun chemin n'existe vers une machine qui n'a aucune connexion.
- Le marquage des sommets visités évite de **reparcourir indéfiniment** un même sommet. Sans cette précaution, un cycle (comme deux câbles entre Switch1 et Switch2) provoquerait des appels récursifs sans fin : la fonction repasserait continuellement entre les deux commutateurs, sans jamais terminer (**boucle infinie** / dépassement de la profondeur de récursion).

Corrigé

- Le **capteur** est le capteur de luminosité (photorésistance ou capteur optique). L'**actionneur** est l'ampoule du lampadaire (via son relais électrique).

2.

```
1 def decider_eclairage(luminosite, seuil=300):
2     return luminosite < seuil
```

- À 150 lux : True (le lampadaire s'allume, il fait sombre). À 500 lux : False (il fait jour). À exactement 300 lux : False (le test est <, strictement inférieur), ce qui est cohérent avec l'énoncé « sous 300 lux » : à 300 lux pile, on n'est pas encore *sous* le seuil.

Corrigé

1.

```
1 etat = {"pompe": False}
2
3 def irrigation_hysteresis(humidite, seuil_bas=30, seuil_haut=60):
4     if etat["pompe"]:
5         if humidite > seuil_haut:
6             etat["pompe"] = False
7     else:
8         if humidite < seuil_bas:
9             etat["pompe"] = True
10    return etat["pompe"]
```

2. La séquence des états est [True, True, True, False, False].

3. Oui, la pompe est active à la mesure 35. Elle a été **déclenchée** à la mesure précédente (20 % < 30 %), et le cahier des charges précise qu'elle ne s'**arrête** que lorsque l'humidité dépasse 60 % : à 35 %, ce seuil haut n'est pas encore atteint, donc la pompe continue de fonctionner même si l'humidité n'est plus sous le seuil bas.

Corrigé

1. message_reconstitue vaut "IEAURESXNS" : ce n'est **pas** le message d'origine ("RESEauxNSI").

2. Concaténer dans l'ordre d'arrivée ne fonctionne que si les paquets arrivent **déjà dans l'ordre d'envoi** (0, 1, 2, 3). Sur un vrai réseau, rien ne garantit cet ordre (les paquets peuvent emprunter des chemins différents, avec des délais différents) : il faut donc systématiquement se fier au **numéro** inscrit dans l'en-tête, jamais à l'ordre d'arrivée.

3. Il faut trier les paquets par leur champ numero avant de les concaténer :

```
1 paquets_tries = sorted(ordre_arrivee, key=lambda p: p["numero"])
2 message_correct = "".join(p["donnees"] for p in paquets_tries)
```

Corrigé

1. 3 paquets sont créés : {"numero": 0, "donnees": "PROTO"}, {"numero": 1, "donnees": "COLER"}, {"numero": 2, "donnees": "ESEAU"}.

2. Oui, le message est correctement reconstitué : "PROTOCOLERESEAU". reassembler **trie** les paquets selon leur champ numero avant de les concaténer, donc l'ordre d'arrivée n'a aucune importance.

3. En concaténant simplement dans l'ordre d'arrivée (paquet 2, puis 0, puis 1), on obtiendrait "ESEAU PROTOCOLER" : un message **incorrect** et incompréhensible. C'est exactement pour éviter ce problème que chaque paquet porte un numéro dans son en-tête.

Corrigé

1. Tentative 1 : envoi de "X" avec bit 0 — **échoue** (perdue). Tentative 2 : retransmission de "X" avec bit 0 — **réussit**. Tentative 3 : envoi de "Y" avec bit 1 — **réussit**. Tentative 4 : envoi de "Z" avec bit 0 — **échoue** (perdue). Tentative 5 : retransmission de "Z" avec bit 0 — **réussit**.

2. Le destinataire reçoit finalement : [(0, "X"), (1, "Y"), (0, "Z")].

3. 5 tentatives ont été nécessaires pour 3 messages : ce nombre est **supérieur** au nombre de messages, car 2 tentatives ont échoué et ont dû être retransmises (une pour "X", une pour "Z").

Corrigé

1. peuvent_communiquer(reseau, "Salle1", "Salle3") renvoie True : il existe un chemin Salle1 → Switch1 → Switch2 → Salle3.

2. Avec reseau["SalleIsolee"] = [], la fonction renvoie False : aucun chemin n'existe vers une machine qui n'a aucune connexion.

3. Le marquage des sommets visités évite de **reparcourir indéfiniment** un même sommet. Sans cette précaution, un cycle (comme deux câbles entre Switch1 et Switch2) provoquerait des appels récursifs sans fin : la fonction repasserait continuellement entre les deux commutateurs, sans jamais terminer (**boucle infinie** / dépassement de la profondeur de récursion).

Corrigé

1. Le **capteur** est le capteur de luminosité (photorésistance ou capteur optique). L'**actionneur** est l'ampoule du lampadaire (via son relais électrique).

2.

```

1 def decider_eclairage(luminosite, seuil=300):
2     return luminosite < seuil

```

3. À 150 lux : True (le lampadaire s'allume, il fait sombre). À 500 lux : False (il fait jour). À exactement 300 lux : False (le test est <, strictement inférieur), ce qui est cohérent avec l'énoncé « sous 300 lux » : à 300 lux pile, on n'est pas encore *sous* le seuil.

Corrigé

1.

```

1 etat = {"pompe": False}
2
3 def irrigation_hysteresis(humidite, seuil_bas=30, seuil_haut=60):
4     if etat["pompe"]:
5         if humidite > seuil_haut:
6             etat["pompe"] = False
7     else:
8         if humidite < seuil_bas:
9             etat["pompe"] = True
10    return etat["pompe"]

```

2. La séquence des états est [True, True, True, False, False].

3. Oui, la pompe est active à la mesure 35. Elle a été **déclenchée** à la mesure précédente (20 % < 30 %), et le cahier des charges précise qu'elle ne s'**arrête** que lorsque l'humidité dépasse 60 % : à 35 %, ce seuil haut n'est pas encore atteint, donc la pompe continue de fonctionner même si l'humidité n'est plus sous le seuil bas.

Corrigé

1. message_reconstitue vaut "IEAURESXNS" : ce n'est **pas** le message d'origine ("RESEAUXNSI").

2. Concaténer dans l'ordre d'arrivée ne fonctionne que si les paquets arrivent **déjà dans l'ordre d'envoi** (0, 1, 2, 3). Sur un vrai réseau, rien ne garantit cet ordre (les paquets peuvent emprunter des chemins différents, avec des délais différents) : il faut donc systématiquement se fier au **numéro** inscrit dans l'en-tête, jamais à l'ordre d'arrivée.

3. Il faut trier les paquets par leur champ numero avant de les concaténer :

```

1 paquets_tries = sorted(ordre_arrivee, key=lambda p: p["numero"])
2 message_correct = "".join(p["donnees"] for p in paquets_tries)

```

Corrigé

1. message_reconstitue vaut "IEAURESXNS" : ce n'est **pas** le message d'origine ("RESEAUXNSI").

2. Concaténer dans l'ordre d'arrivée ne fonctionne que si les paquets arrivent **déjà dans l'ordre d'envoi** (0, 1, 2, 3). Sur un vrai réseau, rien ne garantit cet ordre (les paquets peuvent emprunter des chemins différents, avec des délais différents) : il faut donc systématiquement se fier au **numéro** inscrit dans l'en-tête, jamais à l'ordre d'arrivée.

3. Il faut trier les paquets par leur champ numero avant de les concaténer :

```

1 paquets_tries = sorted(ordre_arrivee, key=lambda p: p["numero"])
2 message_correct = "".join(p["donnees"] for p in paquets_tries)

```


10 Projet et méthodes

10.1 Repères d'histoire de l'informatique

DÉFINITION D1

L'histoire de l'informatique retrace comment les concepts d'**algorithme**, de **machine**, de **langage** et de **données** — les quatre notions fondamentales du programme — ont émergé et se sont articulés au fil du temps, à travers des personnes précises.

◆ PROPRIÉTÉ Quelques repères clés

- ▶ Les **algorithmes** sont présents dès l'Antiquité (par exemple l'algorithme d'Euclide).
- ▶ Les premières **machines à calculer** apparaissent progressivement au XVII^e siècle.
- ▶ En **1936**, Alan Turing introduit le concept de **machine universelle** : une machine capable, en principe, d'exécuter n'importe quel algorithme — c'est la naissance de l'idée moderne d'ordinateur.
- ▶ Les **premiers ordinateurs** sont construits en **1948**.
- ▶ Le réseau **Internet** se développe depuis **1969**.

CODE DE RÉFÉRENCE — Manipuler une chronologie avec des tableaux de p-uplets

```
1 evenements = [  
2     (1936, "Concept de machine universelle", "Alan Turing"),  
3     (1948, "Premier ordinateur a programme enregistre", "Equipe de Manchester"),  
4     (1969, "Debuts du reseau Internet (ARPANET)", "DARPA"),  
5     (1971, "Premier microprocesseur commercial", "Intel"),  
6     (1989, "Invention du World Wide Web", "Tim Berners-Lee"),  
7     (1991, "Premiere version publique de Python", "Guido van Rossum"),  
8 ]  
9  
10 def evenements_entre(evenements, debut, fin):  
11     """Renvoie les evenements dont l'annee est comprise entre debut et fin (inclus  
12     )."""  
13     return [e for e in evenements if debut ≤ e[0] ≤ fin]  
14  
15 for annee, fait, personne in evenements_entre(evenements, 1940, 1970):  
16     print(annee, "-", fait, "(" + personne + ")")
```

⚠ ERREUR FRÉQUENTE

Réduire l'histoire de l'informatique à une suite de « dates à retenir », sans les relier aux concepts. Le programme précise que ces repères doivent être **construits progressivement**, en lien avec les notions techniques déjà étudiées : par exemple, 1936 (Turing) prend tout son sens une fois qu'on a compris la notion d'algorithme et de machine universelle du chapitre sur l'architecture.

» **Python À la machine.** Ajoute à la liste evenements deux événements de ton choix (recherchés dans une source fiable), avec leur année et leur protagoniste. Vérifie que la liste reste triée chronologiquement avec sorted.

10.2 La démarche de projet

DÉFINITION D2

Un **cahier des charges** décrit, avant de commencer à programmer, ce que le projet doit accomplir : les fonctionnalités attendues, les contraintes, et les critères qui permettront de dire si le projet est réussi.

♦ **PROPRIÉTÉ Découper un projet en jalons** Un **jalon** (ou *point d'étape*) est une étape intermédiaire d'un projet, avec un objectif précis et vérifiable. Découper un projet en jalons permet de suivre sa progression, d'ajuster ses objectifs si nécessaire, et d'éviter de se retrouver bloqué en fin de projet.

Pour calculer un pourcentage d'avancement, on suppose qu'au moins un jalon a été défini : **Précondition : la liste des jalons est non vide.** Sans jalon, le nombre total utilisé comme dénominateur serait nul et le pourcentage ne serait pas défini.

CODE DE RÉFÉRENCE — Suivre l'avancement d'un projet par ses jalons

```

1 jalons = [
2     {"nom": "Cahier des charges", "termine": True},
3     {"nom": "Prototype minimal", "termine": True},
4     {"nom": "Fonctionnalites completes", "termine": False},
5     {"nom": "Tests et correction de bugs", "termine": False},
6     {"nom": "Presentation finale", "termine": False},
7 ]
8
9
10 def avancement(jalons):
11     """Renvoie le pourcentage de jalons terminés.
12
13     Precondition : la liste contient au moins un jalon.
14     """
15     if len(jalons) == 0:
16         raise ValueError("au moins un jalon est requis")
17     nb_terminees = sum(1 for j in jalons if j["termine"])
18     return nb_terminees / len(jalons) * 100
19
20
21 def prochain_jalon(jalons):
22     """Renvoie le nom du premier jalon non terminé, ou None si tout est fini."""
23     for j in jalons:
24         if not j["termine"]:
25             return j["nom"]
26     return None
27
28
29 print(avancement(jalons))

```

```
30 print(prochain_jalon(jalons))
```

```
40.0
```

```
Fonctionnalites completes
```

◆ **PROPRIÉTÉ Travailler en équipe** Le programme officiel préconise des groupes de 2 à 4 élèves. Un projet en équipe suppose de **répartir les tâches** (par exemple par fonctionnalité, ou par couche : données, algorithmes, interface), de **coordonner le travail** (éviter que deux personnes modifient la même partie en même temps), et de tenir des **points d'étape** réguliers avec le professeur.

⚠ ERREUR FRÉQUENTE

Vouloir réaliser un projet trop ambitieux, sans jalons intermédiaires. Le programme officiel insiste explicitement sur le fait que les projets doivent rester « d'une ambition raisonnable » pour permettre à l'équipe de les mener à terme : mieux vaut un projet modeste **complet** et testé, qu'un projet ambitieux inachevé.

» **Python À la machine.** Choisis un projet informatique simple de ton choix (par exemple un petit jeu, un gestionnaire de liste de tâches). Découpe-le en 4 jalons avec la structure jalons du cours, et calcule ton avancement au fur et à mesure que tu réalises chaque étape.

10.3 Déboguer méthodiquement

DÉFINITION D3

Débuguer un programme, c'est identifier puis corriger la cause d'un comportement incorrect (plantage, résultat faux). Une démarche **méthodique** vaut mieux qu'une correction « au hasard ».

EXEMPLE Le programme suivant plante lorsqu'on l'exécute :

```
1 def moyenne_ponderee_bugue(valeurs, poids):
2     total = 0
3     for i in range(len(valeurs)):
4         total += valeurs[i] * poids[i]
5     return total / sum(poids)
6
7 moyenne_ponderee_bugue([12, 15, 18], [1, 2])
```

```
Traceback (most recent call last):
  File "programme.py", line 7, in <module>
    moyenne_ponderee_bugue([12, 15, 18], [1, 2])
  File "programme.py", line 4, in moyenne_ponderee_bugue
    total += valeurs[i] * poids[i]
                ~~~~~^
IndexError: list index out of range
```

◆ **PROPRIÉTÉ Lire un message d'erreur (traceback)** Un **traceback** Python indique, de haut en bas, la **pile des appels** qui a mené à l'erreur, et se termine par le **type** de l'erreur (`IndexError`) et un **message** explicatif. La **dernière ligne de code** indiquée (ici, `total += valeurs[i] * poids[i]`) est l'endroit précis où l'erreur s'est produite — c'est le premier endroit à examiner.

◆ **PROPRIÉTÉ Démarche méthodique de débogage**

1. Lire **entièrement** le message d'erreur : type d'erreur, ligne concernée.
2. Identifier la **cause** : ici, `poids` a seulement 2 éléments, mais la boucle tente d'accéder à `poids[2]` (car `valeurs` en a 3).
3. Formuler une **hypothèse** : les deux listes doivent avoir la même longueur.
4. **Vérifier** l'hypothèse, par exemple avec un `print(len(valeurs), len(poids))` ou un contrôle explicite de la précondition.
5. **Corriger** : ajouter une précondition, ou corriger l'appel fautif.

CODE DE RÉFÉRENCE — Corriger avec une précondition explicite

```

1 def moyenne_ponderee(valeurs, poids):
2     """Renvoie la moyenne ponderee pour des poids valides."""
3     if len(valeurs) != len(poids):
4         raise ValueError("valeurs et poids doivent avoir la meme longueur")
5     if not all(poids_i >= 0 for poids_i in poids):
6         raise ValueError("les poids doivent etre non negatifs")
7     somme_poids = sum(poids)
8     if somme_poids <= 0:
9         raise ValueError("la somme des poids doit etre strictement positive")
10    total = 0
11    for i in range(len(valeurs)):
12        total += valeurs[i] * poids[i]
13    return total / somme_poids

```

◆ **PROPRIÉTÉ Interpréter les préconditions de la moyenne pondérée** Lorsque les listes ont la même longueur, que tous les poids sont positifs ou nuls et que leur somme est strictement positive, les poids normalisés forment une **combinaison convexe** des valeurs. La liste `valeurs` est alors nécessairement non vide et le résultat appartient à l'intervalle `[min(valeurs) ; max(valeurs)]`.

◆ **PROPRIÉTÉ Rôle du print, de assert et des exceptions en débogage** Insérer temporairement des `print(...)` pour afficher l'état de variables clés à des points précis du programme est une technique simple et efficace pour localiser un bug. Les `assert` documentent des hypothèses de développement, mais Python peut les supprimer en mode optimisé. Une précondition qui doit rester vérifiée à chaque exécution utilise donc un test explicite et lève ici une `ValueError`.

⚠ ERREUR FRÉQUENTE

Corriger un bug en modifiant du code **au hasard**, sans avoir d'abord compris sa cause exacte. Cette approche peut sembler « résoudre » le problème par chance sur un cas, tout en laissant le bug intact (ou en en créant un nouveau) dans d'autres cas.

» **Python À la machine.** Provoque volontairement une `TypeError` en appelant `moyenne_ponderee(["a", "b"], [1, 2])` (des chaînes au lieu de nombres). Lis le traceback obtenu, identifie la ligne et le type d'erreur, puis explique en une phrase la cause du problème.

10.4 Documenter son code et préparer une présentation orale

DÉFINITION D4

Documenter un programme, c'est expliquer, en langage naturel, ce qu'il fait : le rôle de chaque fonction (via une **docstring**), les choix de conception importants, et la manière de l'utiliser.

CODE DE RÉFÉRENCE — Une fonction bien documentée

```
1 def calcul_moyenne(valeurs):
2     """Calcule la moyenne arithmétique d'une liste de valeurs numeriques.
3
4     Precondition : valeurs est une liste non vide.
5     Postcondition : le resultat est compris entre min(valeurs) et max(valeurs).
6     """
7     return sum(valeurs) / len(valeurs)
8
9 def a_une_docstring(fonction):
10    """Teste si 'fonction' possede une docstring non vide."""
11    return fonction.__doc__ is not None and len(fonction.__doc__.strip()) > 0
12
13 print(a_une_docstring(calcul_moyenne)) # True
```

◆ **PROPRIÉTÉ Ce qu'une bonne documentation contient** Une docstring de fonction utile précise, dans l'ordre : **ce que fait** la fonction (une phrase claire), ses **préconditions** (ce que les arguments doivent vérifier), et éventuellement ses **postconditions** (ce que garantit le résultat) — exactement la spécification vue au chapitre sur les langages et la programmation.

◆ **PROPRIÉTÉ Préparer une présentation orale de projet** Le programme officiel souligne que cette spécialité contribue au développement des **compétences orales**, en particulier pour préparer l'épreuve orale de terminale (Grand Oral) — un travail qui commence dès la Première. Une présentation de projet efficace explique : le **problème** traité, les **choix** effectués (et pourquoi), les **difficultés** rencontrées et comment elles ont été surmontées, et une **démonstration** concrète du résultat.

ERREUR FRÉQUENTE

Préparer une présentation orale qui ne fait que **décrire le code ligne par ligne**. Un jury (ou un public) s'intéresse davantage au **raisonnement** : pourquoi ce problème est intéressant, pourquoi telle approche a été choisie plutôt qu'une autre, ce qui a été appris en le résolvant — pas à une lecture littérale du code source.

» **Python À la machine.** Choisis une fonction que tu as écrite dans un chapitre précédent de ce manuel (par exemple le tri par insertion ou la recherche dichotomique). Écris-lui une docstring complète

(rôle, précondition, postcondition), puis vérifie avec `a_une_docstring` qu'elle est bien documentée. Prépare ensuite, à l'oral, une explication de 2 minutes de cette fonction pour un camarade qui ne l'a jamais vue.

10.5 Repères d'histoire de l'informatique

DÉFINITION D1

L'histoire de l'informatique retrace comment les concepts d'**algorithme**, de **machine**, de **langage** et de **données** — les quatre notions fondamentales du programme — ont émergé et se sont articulés au fil du temps, à travers des personnes précises.

PROPRIÉTÉ Quelques repères clés

- ▶ Les **algorithmes** sont présents dès l'Antiquité (par exemple l'algorithme d'Euclide).
- ▶ Les premières **machines à calculer** apparaissent progressivement au XVII^e siècle.
- ▶ En **1936**, Alan Turing introduit le concept de **machine universelle** : une machine capable, en principe, d'exécuter n'importe quel algorithme — c'est la naissance de l'idée moderne d'ordinateur.
- ▶ Les **premiers ordinateurs** sont construits en **1948**.
- ▶ Le réseau **Internet** se développe depuis **1969**.

CODE DE RÉFÉRENCE — Manipuler une chronologie avec des tableaux de p-uplets

```

1  evenements = [
2      (1936, "Concept de machine universelle", "Alan Turing"),
3      (1948, "Premier ordinateur a programme enregistre", "Equipe de Manchester"),
4      (1969, "Debuts du reseau Internet (ARPANET)", "DARPA"),
5      (1971, "Premier microprocesseur commercial", "Intel"),
6      (1989, "Invention du World Wide Web", "Tim Berners-Lee"),
7      (1991, "Premiere version publique de Python", "Guido van Rossum"),
8  ]
9
10 def evenements_entre(evenements, debut, fin):
11     """Renvoie les evenements dont l'annee est comprise entre debut et fin (inclus)."""
12     return [e for e in evenements if debut ≤ e[0] ≤ fin]
13
14 for annee, fait, personne in evenements_entre(evenements, 1940, 1970):
15     print(annee, "-", fait, "(" + personne + ")")

```

ERREUR FRÉQUENTE

Réduire l'histoire de l'informatique à une suite de « dates à retenir », sans les relier aux concepts. Le programme précise que ces repères doivent être **construits progressivement**, en lien avec les notions techniques déjà étudiées : par exemple, 1936 (Turing) prend tout son sens une fois qu'on a compris la notion d'algorithme et de machine universelle du chapitre sur l'architecture.

» **Python À la machine.** Ajoute à la liste evenements deux événements de ton choix (recherchés dans une source fiable), avec leur année et leur protagoniste. Vérifie que la liste reste triée chronologiquement avec sorted.

10.6 La démarche de projet

DÉFINITION D2

Un **cahier des charges** décrit, avant de commencer à programmer, ce que le projet doit accomplir : les fonctionnalités attendues, les contraintes, et les critères qui permettront de dire si le projet est réussi.

◆ **PROPRIÉTÉ Découper un projet en jalons** Un **jalon** (ou *point d'étape*) est une étape intermédiaire d'un projet, avec un objectif précis et vérifiable. Découper un projet en jalons permet de suivre sa progression, d'ajuster ses objectifs si nécessaire, et d'éviter de se retrouver bloqué en fin de projet.

Pour calculer un pourcentage d'avancement, on suppose qu'au moins un jalon a été défini : **Pré-condition : la liste des jalons est non vide**. Sans jalon, le nombre total utilisé comme dénominateur serait nul et le pourcentage ne serait pas défini.

CODE DE RÉFÉRENCE — Suivre l'avancement d'un projet par ses jalons

```

1 jalons = [
2     {"nom": "Cahier des charges", "termine": True},
3     {"nom": "Prototype minimal", "termine": True},
4     {"nom": "Fonctionnalités complètes", "termine": False},
5     {"nom": "Tests et correction de bugs", "termine": False},
6     {"nom": "Présentation finale", "termine": False},
7 ]
8
9
10 def avancement(jalons):
11     """Renvoie le pourcentage de jalons terminés.
12
13     Précondition : la liste contient au moins un jalon.
14     """
15     if len(jalons) == 0:
16         raise ValueError("au moins un jalon est requis")
17     nb_terminees = sum(1 for j in jalons if j["termine"])
18     return nb_terminees / len(jalons) * 100
19
20
21 def prochain_jalon(jalons):
22     """Renvoie le nom du premier jalon non terminé, ou None si tout est fini."""
23     for j in jalons:
24         if not j["termine"]:
25             return j["nom"]
26     return None
27
28
29 print(avancement(jalons))

```

```
30 print(prochain_jalon(jalons))
```

```
40.0
```

```
Fonctionnalites completes
```

◆ **PROPRIÉTÉ Travailler en équipe** Le programme officiel préconise des groupes de 2 à 4 élèves. Un projet en équipe suppose de **répartir les tâches** (par exemple par fonctionnalité, ou par couche : données, algorithmes, interface), de **coordonner le travail** (éviter que deux personnes modifient la même partie en même temps), et de tenir des **points d'étape** réguliers avec le professeur.

⚠ ERREUR FRÉQUENTE

Vouloir réaliser un projet trop ambitieux, sans jalons intermédiaires. Le programme officiel insiste explicitement sur le fait que les projets doivent rester « d'une ambition raisonnable » pour permettre à l'équipe de les mener à terme : mieux vaut un projet modeste **complet** et testé, qu'un projet ambitieux inachevé.

» **Python À la machine.** Choisis un projet informatique simple de ton choix (par exemple un petit jeu, un gestionnaire de liste de tâches). Découpe-le en 4 jalons avec la structure jalons du cours, et calcule ton avancement au fur et à mesure que tu réalises chaque étape.

10.7 Déboguer méthodiquement

DÉFINITION D3

Débuguer un programme, c'est identifier puis corriger la cause d'un comportement incorrect (plantage, résultat faux). Une démarche **méthodique** vaut mieux qu'une correction « au hasard ».

EXEMPLE Le programme suivant plante lorsqu'on l'exécute :

```
1 def moyenne_ponderee_bugue(valeurs, poids):
2     total = 0
3     for i in range(len(valeurs)):
4         total += valeurs[i] * poids[i]
5     return total / sum(poids)
6
7 moyenne_ponderee_bugue([12, 15, 18], [1, 2])
```

```
Traceback (most recent call last):
  File "programme.py", line 7, in <module>
    moyenne_ponderee_bugue([12, 15, 18], [1, 2])
  File "programme.py", line 4, in moyenne_ponderee_bugue
    total += valeurs[i] * poids[i]
                ~~~~~^
IndexError: list index out of range
```


◆ **PROPRIÉTÉ Lire un message d'erreur (traceback)** Un **traceback** Python indique, de haut en bas, la **pile des appels** qui a mené à l'erreur, et se termine par le **type** de l'erreur (`IndexError`) et un **message** explicatif. La **dernière ligne de code** indiquée (ici, `total += valeurs[i] * poids[i]`) est l'endroit précis où l'erreur s'est produite — c'est le premier endroit à examiner.

◆ **PROPRIÉTÉ Démarche méthodique de débogage**

1. Lire **entièrement** le message d'erreur : type d'erreur, ligne concernée.
2. Identifier la **cause** : ici, `poids` a seulement 2 éléments, mais la boucle tente d'accéder à `poids[2]` (car `valeurs` en a 3).
3. Formuler une **hypothèse** : les deux listes doivent avoir la même longueur.
4. **Vérifier** l'hypothèse, par exemple avec un `print(len(valeurs), len(poids))` ou un contrôle explicite de la précondition.
5. **Corriger** : ajouter une précondition, ou corriger l'appel fautif.

CODE DE RÉFÉRENCE — Corriger avec une précondition explicite

```
1 def moyenne_ponderee(valeurs, poids):
2     """Renvoie la moyenne ponderee pour des poids valides."""
3     if len(valeurs) != len(poids):
4         raise ValueError("valeurs et poids doivent avoir la meme longueur")
5     if not all(poids_i >= 0 for poids_i in poids):
6         raise ValueError("les poids doivent etre non negatifs")
7     somme_poids = sum(poids)
8     if somme_poids <= 0:
9         raise ValueError("la somme des poids doit etre strictement positive")
10    total = 0
11    for i in range(len(valeurs)):
12        total += valeurs[i] * poids[i]
13    return total / somme_poids
```

◆ **PROPRIÉTÉ Interpréter les préconditions de la moyenne pondérée** Lorsque les listes ont la même longueur, que tous les poids sont positifs ou nuls et que leur somme est strictement positive, les poids normalisés forment une **combinaison convexe** des valeurs. La liste `valeurs` est alors nécessairement non vide et le résultat appartient à l'intervalle `[min(valeurs) ; max(valeurs)]`.

◆ **PROPRIÉTÉ Rôle du print, de assert et des exceptions en débogage** Insérer temporairement des `print(...)` pour afficher l'état de variables clés à des points précis du programme est une technique simple et efficace pour localiser un bug. Les `assert` documentent des hypothèses de développement, mais Python peut les supprimer en mode optimisé. Une précondition qui doit rester vérifiée à chaque exécution utilise donc un test explicite et lève ici une `ValueError`.

⚠ ERREUR FRÉQUENTE

Corriger un bug en modifiant du code **au hasard**, sans avoir d'abord compris sa cause exacte. Cette approche peut sembler « résoudre » le problème par chance sur un cas, tout en laissant le bug intact (ou en en créant un nouveau) dans d'autres cas.

» **Python À la machine.** Provoque volontairement une `TypeError` en appelant `moyenne_ponderee(["a", "b"], [1, 2])` (des chaînes au lieu de nombres). Lis le traceback obtenu, identifie la ligne et le type d'erreur, puis explique en une phrase la cause du problème.

10.8 Documenter son code et préparer une présentation orale

DÉFINITION D4

Documenter un programme, c'est expliquer, en langage naturel, ce qu'il fait : le rôle de chaque fonction (via une **docstring**), les choix de conception importants, et la manière de l'utiliser.

CODE DE RÉFÉRENCE — Une fonction bien documentée

```

1 def calcul_moyenne(valeurs):
2     """Calcule la moyenne arithmetique d'une liste de valeurs numeriques.
3
4     Precondition : valeurs est une liste non vide.
5     Postcondition : le resultat est compris entre min(valeurs) et max(valeurs).
6     """
7     return sum(valeurs) / len(valeurs)
8
9 def a_une_docstring(fonction):
10    """Teste si 'fonction' possede une docstring non vide."""
11    return fonction.__doc__ is not None and len(fonction.__doc__.strip()) > 0
12
13 print(a_une_docstring(calcul_moyenne)) # True

```

◆ **PROPRIÉTÉ Ce qu'une bonne documentation contient** Une docstring de fonction utile précise, dans l'ordre : **ce que fait** la fonction (une phrase claire), ses **préconditions** (ce que les arguments doivent vérifier), et éventuellement ses **postconditions** (ce que garantit le résultat) — exactement la spécification vue au chapitre sur les langages et la programmation.

◆ **PROPRIÉTÉ Préparer une présentation orale de projet** Le programme officiel souligne que cette spécialité contribue au développement des **compétences orales**, en particulier pour préparer l'épreuve orale de terminale (Grand Oral) — un travail qui commence dès la Première. Une présentation de projet efficace explique : le **problème** traité, les **choix** effectués (et pourquoi), les **difficultés** rencontrées et comment elles ont été surmontées, et une **démonstration** concrète du résultat.

⚠ ERREUR FRÉQUENTE

Préparer une présentation orale qui ne fait que **décrire le code ligne par ligne**. Un jury (ou un public) s'intéresse davantage au **raisonnement** : pourquoi ce problème est intéressant, pourquoi telle approche a été choisie plutôt qu'une autre, ce qui a été appris en le résolvant — pas à une lecture littérale du code source.

» **Python À la machine.** Choisis une fonction que tu as écrite dans un chapitre précédent de ce manuel (par exemple le tri par insertion ou la recherche dichotomique). Écris-lui une docstring complète

(rôle, précondition, postcondition), puis vérifie avec `a_une_docstring` qu'elle est bien documentée. Prépare ensuite, à l'oral, une explication de 2 minutes de cette fonction pour un camarade qui ne l'a jamais vue.

Méthodes

- Méthode directe de travail sur le sujet.

Exercice 1

On donne la chronologie suivante :

```
1 evenements = [
2     (1642, "Pascaline, machine a calculer mecanique", "Blaise Pascal"),
3     (1936, "Concept de machine universelle", "Alan Turing"),
4     (1971, "Premier microprocesseur commercial", "Intel"),
5     (1991, "Premiere version publique de Python", "Guido van Rossum"),
6 ]
```

1. Vérifier, avec `sorted`, que cette liste est bien dans l'ordre chronologique.
2. En utilisant `evenements_entre` du cours, lister les événements situés entre 1900 et 1980.
3. Situe brièvement, en une phrase chacun, ce qu'apporte historiquement le concept de « machine universelle » de Turing (1936) par rapport à la Pascaline (1642).

Exercice 2

Une équipe de 3 élèves développe un petit jeu vidéo, découpé en 6 jalons :

```
1 jalons = [
2     {"nom": "Specification du jeu", "termine": True},
3     {"nom": "Deplacement du personnage", "termine": True},
4     {"nom": "Gestion des collisions", "termine": True},
5     {"nom": "Score et niveaux", "termine": False},
6     {"nom": "Tests et equilibrage", "termine": False},
7     {"nom": "Presentation finale", "termine": False},
8 ]
```

1. En utilisant `avancement` du cours, calculer le pourcentage d'avancement du projet.
2. En utilisant `prochain_jalon` du cours, déterminer sur quel jalon l'équipe doit désormais se concentrer.
3. L'équipe est à mi-parcours de l'horaire prévu pour ce projet, et son avancement est justement de 50 %. Les 3 jalons restants (score/niveaux, tests, présentation) te semblent-ils du même volume de travail que les 3 jalons déjà réalisés ? Pourquoi ce simple pourcentage peut-il être trompeur ?

Exercice 3

Le programme suivant plante à l'exécution :

```
1 notes = {"Alice": 15, "Bob": 12, "Chloe": 18}
2
3 def afficher_note(notes, nom):
4     print(nom, ":", notes[nom])
5
6 afficher_note(notes, "David")
```

1. Quel type d'erreur ce programme provoque-t-il ? Quel est le message exact ?
2. En suivant la démarche méthodique du cours, identifier précisément la cause du problème (sans se contenter de « il y a une erreur »).
3. Proposer une correction de `afficher_note` qui affiche un message clair (par exemple "Nom inconnu") au lieu de planter, lorsque le nom n'existe pas dans notes.

Exercice 4

On reprend la fonction `tri_insertion` d'un chapitre précédent, sans aucune documentation :

```
1 def tri_insertion(tableau):
2     tableau = tableau[:]
3     for i in range(1, len(tableau)):
4         valeur = tableau[i]
5         j = i - 1
6         while j ≥ 0 and tableau[j] > valeur:
7             tableau[j + 1] = tableau[j]
8             j -= 1
9         tableau[j + 1] = valeur
10    return tableau
```

1. Vérifier, avec `a_une_docstring` du cours, que cette fonction n'est pas documentée.
2. Écrire une docstring complète pour cette fonction (rôle, précondition, postcondition), puis vérifier qu'elle est bien détectée comme documentée.
3. Si tu devais présenter cette fonction en 2 minutes à l'oral, quels seraient les 3 points les plus importants à expliquer (au-delà d'une lecture ligne par ligne du code) ?

Exercice 5

Une équipe développe un gestionnaire de tournoi. La veille de la présentation finale (dernier jalon du projet), un testeur signale que la fonction `rechercher_participant` ne trouve jamais le **dernier** inscrit de la liste :

```
1 def rechercher_participant_bugue(participants, pseudo):
2     for i in range(len(participants) - 1):
3         if participants[i] == pseudo:
4             return i
5     return -1
6
7 participants = ["Ali", "Sara", "Nora", "Yanis"]
8 print(rechercher_participant_bugue(participants, "Yanis"))
```

1. Que renvoie cet appel ? Explique précisément, en examinant la boucle `for`, pourquoi le dernier élément de `participants` n'est jamais examiné.
2. Corriger la fonction.
3. L'équipe est à la veille de la présentation, avec ce bug encore présent dans une fonctionnalité secondaire. En te référant à la démarche de projet du cours (jalons, ambition raisonnable), quelle serait une décision d'équipe raisonnable : corriger dans l'urgence juste avant de présenter, ou documenter le bug comme limite connue et le corriger après la présentation ? Justifie ta réponse en une ou deux phrases.

Exercice 6

On donne la chronologie suivante :

```

1 evenements = [
2     (1642, "Pascaline, machine a calculer mecanique", "Blaise Pascal"),
3     (1936, "Concept de machine universelle", "Alan Turing"),
4     (1971, "Premier microprocesseur commercial", "Intel"),
5     (1991, "Premiere version publique de Python", "Guido van Rossum"),
6 ]

```

1. Vérifier, avec `sorted`, que cette liste est bien dans l'ordre chronologique.
2. En utilisant `evenements_entre` du cours, lister les événements situés entre 1900 et 1980.
3. Situe brièvement, en une phrase chacun, ce qu'apporte historiquement le concept de « machine universelle » de Turing (1936) par rapport à la Pascaline (1642).

Exercice 7

Une équipe de 3 élèves développe un petit jeu vidéo, découpé en 6 jalons :

```

1 jalons = [
2     {"nom": "Specification du jeu", "termine": True},
3     {"nom": "Déplacement du personnage", "termine": True},
4     {"nom": "Gestion des collisions", "termine": True},
5     {"nom": "Score et niveaux", "termine": False},
6     {"nom": "Tests et équilibrage", "termine": False},
7     {"nom": "Présentation finale", "termine": False},
8 ]

```

1. En utilisant `avancement` du cours, calculer le pourcentage d'avancement du projet.
2. En utilisant `prochain_jalon` du cours, déterminer sur quel jalon l'équipe doit désormais se concentrer.
3. L'équipe est à mi-parcours de l'horaire prévu pour ce projet, et son avancement est justement de 50 %. Les 3 jalons restants (score/niveaux, tests, présentation) te semblent-ils du même volume de travail que les 3 jalons déjà réalisés ? Pourquoi ce simple pourcentage peut-il être trompeur ?

Exercice 8

Le programme suivant plante à l'exécution :

```

1 notes = {"Alice": 15, "Bob": 12, "Chloe": 18}
2
3 def afficher_note(notes, nom):
4     print(nom, ":", notes[nom])
5
6 afficher_note(notes, "David")

```

1. Quel type d'erreur ce programme provoque-t-il ? Quel est le message exact ?
2. En suivant la démarche méthodique du cours, identifier précisément la cause du problème (sans se contenter de « il y a une erreur »).
3. Proposer une correction de `afficher_note` qui affiche un message clair (par exemple "Nom inconnu") au lieu de planter, lorsque le nom n'existe pas dans `notes`.

Exercice 9

On reprend la fonction `tri_insertion` d'un chapitre précédent, sans aucune documentation :

```

1 def tri_insertion(tableau):

```

```

2     tableau = tableau[:]
3     for i in range(1, len(tableau)):
4         valeur = tableau[i]
5         j = i - 1
6         while j ≥ 0 and tableau[j] > valeur:
7             tableau[j + 1] = tableau[j]
8             j -= 1
9         tableau[j + 1] = valeur
10    return tableau

```

1. Vérifier, avec `a_une_docstring` du cours, que cette fonction n'est pas documentée.
2. Écrire une docstring complète pour cette fonction (rôle, précondition, postcondition), puis vérifier qu'elle est bien détectée comme documentée.
3. Si tu devais présenter cette fonction en 2 minutes à l'oral, quels seraient les 3 points les plus importants à expliquer (au-delà d'une lecture ligne par ligne du code) ?

Exercice 10

Une équipe développe un gestionnaire de tournoi. La veille de la présentation finale (dernier jalon du projet), un testeur signale que la fonction `rechercher_participant` ne trouve jamais le **dernier** inscrit de la liste :

```

1 def rechercher_participant_bugue(participants, pseudo):
2     for i in range(len(participants) - 1):
3         if participants[i] == pseudo:
4             return i
5     return -1
6
7 participants = ["Ali", "Sara", "Nora", "Yanis"]
8 print(rechercher_participant_bugue(participants, "Yanis"))

```

1. Que renvoie cet appel ? Explique précisément, en examinant la boucle `for`, pourquoi le dernier élément de `participants` n'est jamais examiné.
2. Corriger la fonction.
3. L'équipe est à la veille de la présentation, avec ce bug encore présent dans une fonctionnalité secondaire. En te référant à la démarche de projet du cours (jalons, ambition raisonnable), quelle serait une décision d'équipe raisonnable : corriger dans l'urgence juste avant de présenter, ou documenter le bug comme limite connue et le corriger après la présentation ? Justifie ta réponse en une ou deux phrases.

Exercice 11

On donne la chronologie suivante :

```

1 evenements = [
2     (1642, "Pascaline, machine a calculer mecanique", "Blaise Pascal"),
3     (1936, "Concept de machine universelle", "Alan Turing"),
4     (1971, "Premier microprocesseur commercial", "Intel"),
5     (1991, "Premiere version publique de Python", "Guido van Rossum"),
6 ]

```

1. Vérifier, avec `sorted`, que cette liste est bien dans l'ordre chronologique.
2. En utilisant `evenements_entre` du cours, lister les événements situés entre 1900 et 1980.
3. Situe brièvement, en une phrase chacun, ce qu'apporte historiquement le concept de « machine universelle » de Turing (1936) par rapport à la Pascaline (1642).

Exercice 12 ◆

Une équipe de 3 élèves développe un petit jeu vidéo, découpé en 6 jalons :

```
1 jalons = [
2     {"nom": "Specification du jeu", "termine": True},
3     {"nom": "Déplacement du personnage", "termine": True},
4     {"nom": "Gestion des collisions", "termine": True},
5     {"nom": "Score et niveaux", "termine": False},
6     {"nom": "Tests et équilibrage", "termine": False},
7     {"nom": "Présentation finale", "termine": False},
8 ]
```

1. En utilisant l'avancement du cours, calculer le pourcentage d'avancement du projet.
2. En utilisant prochain_jalon du cours, déterminer sur quel jalon l'équipe doit désormais se concentrer.
3. L'équipe est à mi-parcours de l'horaire prévu pour ce projet, et son avancement est justement de 50 %. Les 3 jalons restants (score/niveaux, tests, présentation) te semblent-ils du même volume de travail que les 3 jalons déjà réalisés ? Pourquoi ce simple pourcentage peut-il être trompeur ?

Exercice 13 ◆◆

Le programme suivant plante à l'exécution :

```
1 notes = {"Alice": 15, "Bob": 12, "Chloe": 18}
2
3 def afficher_note(notes, nom):
4     print(nom, ":", notes[nom])
5
6 afficher_note(notes, "David")
```

1. Quel type d'erreur ce programme provoque-t-il ? Quel est le message exact ?
2. En suivant la démarche méthodique du cours, identifier précisément la cause du problème (sans se contenter de « il y a une erreur »).
3. Proposer une correction de `afficher_note` qui affiche un message clair (par exemple "Nom inconnu") au lieu de planter, lorsque le nom n'existe pas dans `notes`.

Exercice 14 ◆◆

On reprend la fonction `tri_insertion` d'un chapitre précédent, sans aucune documentation :

```
1 def tri_insertion(tableau):
2     tableau = tableau[:]
3     for i in range(1, len(tableau)):
4         valeur = tableau[i]
5         j = i - 1
6         while j ≥ 0 and tableau[j] > valeur:
7             tableau[j + 1] = tableau[j]
8             j -= 1
9         tableau[j + 1] = valeur
10    return tableau
```

1. Vérifier, avec `a_une_docstring` du cours, que cette fonction n'est pas documentée.
2. Écrire une docstring complète pour cette fonction (rôle, précondition, postcondition), puis vérifier qu'elle est bien détectée comme documentée.

3. Si tu devais présenter cette fonction en 2 minutes à l'oral, quels seraient les 3 points les plus importants à expliquer (au-delà d'une lecture ligne par ligne du code) ?

Exercice 15 ◆◆◆

Une équipe développe un gestionnaire de tournoi. La veille de la présentation finale (dernier jalon du projet), un testeur signale que la fonction `rechercher_participant` ne trouve jamais le **dernier** inscrit de la liste :

```
1 def rechercher_participant_bugue(participants, pseudo):
2     for i in range(len(participants) - 1):
3         if participants[i] == pseudo:
4             return i
5     return -1
6
7 participants = ["Ali", "Sara", "Nora", "Yanis"]
8 print(rechercher_participant_bugue(participants, "Yanis"))
```

1. Que renvoie cet appel ? Explique précisément, en examinant la boucle `for`, pourquoi le dernier élément de `participants` n'est jamais examiné.
2. Corriger la fonction.
3. L'équipe est à la veille de la présentation, avec ce bug encore présent dans une fonctionnalité secondaire. En te référant à la démarche de projet du cours (jalons, ambition raisonnable), quelle serait une décision d'équipe raisonnable : corriger dans l'urgence juste avant de présenter, ou documenter le bug comme limite connue et le corriger après la présentation ? Justifie ta réponse en une ou deux phrases.

Exercice 16 ◆

On donne la chronologie suivante :

```
1 evenements = [
2     (1642, "Pascaline, machine a calculer mecanique", "Blaise Pascal"),
3     (1936, "Concept de machine universelle", "Alan Turing"),
4     (1971, "Premier microprocesseur commercial", "Intel"),
5     (1991, "Premiere version publique de Python", "Guido van Rossum"),
6 ]
```

1. Vérifier, avec `sorted`, que cette liste est bien dans l'ordre chronologique.
2. En utilisant `evenements_entre` du cours, lister les événements situés entre 1900 et 1980.
3. Situe brièvement, en une phrase chacun, ce qu'apporte historiquement le concept de « machine universelle » de Turing (1936) par rapport à la Pascaline (1642).

Exercice 17 ◆

Une équipe de 3 élèves développe un petit jeu vidéo, découpé en 6 jalons :

```
1 jalons = [
2     {"nom": "Specification du jeu", "termine": True},
3     {"nom": "Deplacement du personnage", "termine": True},
4     {"nom": "Gestion des collisions", "termine": True},
5     {"nom": "Score et niveaux", "termine": False},
6     {"nom": "Tests et equilibrage", "termine": False},
7     {"nom": "Presentation finale", "termine": False},
8 ]
```


1. En utilisant avancement du cours, calculer le pourcentage d'avancement du projet.
2. En utilisant prochain_jalon du cours, déterminer sur quel jalon l'équipe doit désormais se concentrer.
3. L'équipe est à mi-parcours de l'horaire prévu pour ce projet, et son avancement est justement de 50 %. Les 3 jalons restants (score/niveaux, tests, présentation) te semblent-ils du même volume de travail que les 3 jalons déjà réalisés ? Pourquoi ce simple pourcentage peut-il être trompeur ?

Exercice 18

Le programme suivant plante à l'exécution :

```
1 notes = {"Alice": 15, "Bob": 12, "Chloe": 18}
2
3 def afficher_note(notes, nom):
4     print(nom, ":", notes[nom])
5
6 afficher_note(notes, "David")
```

1. Quel type d'erreur ce programme provoque-t-il ? Quel est le message exact ?
2. En suivant la démarche méthodique du cours, identifier précisément la cause du problème (sans se contenter de « il y a une erreur »).
3. Proposer une correction de afficher_note qui affiche un message clair (par exemple "Nom inconnu") au lieu de planter, lorsque le nom n'existe pas dans notes.

Exercice 19

On reprend la fonction tri_insertion d'un chapitre précédent, sans aucune documentation :

```
1 def tri_insertion(tableau):
2     tableau = tableau[:]
3     for i in range(1, len(tableau)):
4         valeur = tableau[i]
5         j = i - 1
6         while j ≥ 0 and tableau[j] > valeur:
7             tableau[j + 1] = tableau[j]
8             j -= 1
9         tableau[j + 1] = valeur
10    return tableau
```

1. Vérifier, avec a_une_docstring du cours, que cette fonction n'est pas documentée.
2. Écrire une docstring complète pour cette fonction (rôle, précondition, postcondition), puis vérifier qu'elle est bien détectée comme documentée.
3. Si tu devais présenter cette fonction en 2 minutes à l'oral, quels seraient les 3 points les plus importants à expliquer (au-delà d'une lecture ligne par ligne du code) ?

Exercice 20

Une équipe développe un gestionnaire de tournoi. La veille de la présentation finale (dernier jalon du projet), un testeur signale que la fonction rechercher_participant ne trouve jamais le **dernier** inscrit de la liste :

```
1 def rechercher_participant_bugue(participants, pseudo):
2     for i in range(len(participants) - 1):
3         if participants[i] == pseudo:
4             return i
```

```

5     return -1
6
7     participants = ["Ali", "Sara", "Nora", "Yanis"]
8     print(rechercher_participant_bugue(participants, "Yanis"))

```

1. Que renvoie cet appel ? Explique précisément, en examinant la boucle for, pourquoi le dernier élément de participants n'est jamais examiné.
2. Corriger la fonction.
3. L'équipe est à la veille de la présentation, avec ce bug encore présent dans une fonctionnalité secondaire. En te référant à la démarche de projet du cours (jalons, ambition raisonnable), quelle serait une décision d'équipe raisonnable : corriger dans l'urgence juste avant de présenter, ou documenter le bug comme limite connue et le corriger après la présentation ? Justifie ta réponse en une ou deux phrases.

Exercice 21

On donne la chronologie suivante :

```

1 evenements = [
2     (1642, "Pascaline, machine a calculer mecanique", "Blaise Pascal"),
3     (1936, "Concept de machine universelle", "Alan Turing"),
4     (1971, "Premier microprocesseur commercial", "Intel"),
5     (1991, "Premiere version publique de Python", "Guido van Rossum"),
6 ]

```

1. Vérifier, avec sorted, que cette liste est bien dans l'ordre chronologique.
2. En utilisant evenements_entre du cours, lister les événements situés entre 1900 et 1980.
3. Situe brièvement, en une phrase chacun, ce qu'apporte historiquement le concept de « machine universelle » de Turing (1936) par rapport à la Pascaline (1642).

Exercice 22

Une équipe de 3 élèves développe un petit jeu vidéo, découpé en 6 jalons :

```

1 jalons = [
2     {"nom": "Specification du jeu", "termine": True},
3     {"nom": "Déplacement du personnage", "termine": True},
4     {"nom": "Gestion des collisions", "termine": True},
5     {"nom": "Score et niveaux", "termine": False},
6     {"nom": "Tests et équilibrage", "termine": False},
7     {"nom": "Présentation finale", "termine": False},
8 ]

```

1. En utilisant avancement du cours, calculer le pourcentage d'avancement du projet.
2. En utilisant prochain_jalon du cours, déterminer sur quel jalon l'équipe doit désormais se concentrer.
3. L'équipe est à mi-parcours de l'horaire prévu pour ce projet, et son avancement est justement de 50 %. Les 3 jalons restants (score/niveaux, tests, présentation) te semblent-ils du même volume de travail que les 3 jalons déjà réalisés ? Pourquoi ce simple pourcentage peut-il être trompeur ?

Exercice 23

Le programme suivant plante à l'exécution :

```

1 notes = {"Alice": 15, "Bob": 12, "Chloe": 18}
2
3 def afficher_note(notes, nom):
4     print(nom, ":", notes[nom])
5
6 afficher_note(notes, "David")

```

1. Quel type d'erreur ce programme provoque-t-il ? Quel est le message exact ?
2. En suivant la démarche méthodique du cours, identifier précisément la cause du problème (sans se contenter de « il y a une erreur »).
3. Proposer une correction de `afficher_note` qui affiche un message clair (par exemple "Nom inconnu") au lieu de planter, lorsque le nom n'existe pas dans `notes`.

Exercice 24 ♦♦

On reprend la fonction `tri_insertion` d'un chapitre précédent, sans aucune documentation :

```

1 def tri_insertion(tableau):
2     tableau = tableau[:]
3     for i in range(1, len(tableau)):
4         valeur = tableau[i]
5         j = i - 1
6         while j ≥ 0 and tableau[j] > valeur:
7             tableau[j + 1] = tableau[j]
8             j -= 1
9         tableau[j + 1] = valeur
10    return tableau

```

1. Vérifier, avec `a_une_docstring` du cours, que cette fonction n'est pas documentée.
2. Écrire une docstring complète pour cette fonction (rôle, précondition, postcondition), puis vérifier qu'elle est bien détectée comme documentée.
3. Si tu devais présenter cette fonction en 2 minutes à l'oral, quels seraient les 3 points les plus importants à expliquer (au-delà d'une lecture ligne par ligne du code) ?

QCM d'auto-évaluation — Projet et méthodes

Pour chaque question, une seule réponse est exacte. Entourez la lettre correspondante.

C1 — Histoire de l'informatique

1. [Q1] Le concept de machine universelle, capable en principe d'exécuter n'importe quel algorithme, est introduit par :
 - A. Blaise Pascal en 1642.
 - B. Alan Turing en 1936.
 - C. Guido van Rossum en 1991.
 - D. Tim Berners-Lee en 1989.

C2 — Démarche de projet

2. [Q2] Le programme officiel réserve à la conception de projets, en classe de Première :

- A. moins d'un dixième de l'horaire annuel.
 - B. au moins un quart de l'horaire annuel.
 - C. la totalité de l'horaire annuel.
 - D. aucune heure dédiée.
3. [Q3] Découper un projet en jalons permet notamment de :
- A. éviter tout travail en équipe.
 - B. suivre la progression et ajuster les objectifs si nécessaire.
 - C. garantir qu'aucun bug ne subsistera.
 - D. remplacer le cahier des charges.

C3 — Débogage

4. [Q4] Face à un message d'erreur (traceback) Python, la première chose à faire est de :
- A. corriger au hasard une ligne du programme.
 - B. lire entièrement le message pour identifier le type d'erreur et la ligne concernée.
 - C. ignorer le message et relancer le programme.
 - D. supprimer toute la fonction concernée.

C4 — Documentation et oral

5. [Q5] Une bonne docstring de fonction précise notamment :
- A. le nom de l'auteur du code uniquement.
 - B. le rôle de la fonction, ses préconditions et ses postconditions.
 - C. la date de la dernière modification uniquement.
 - D. rien : le code se suffit toujours à lui-même.
6. [Q6] Une présentation orale efficace d'un projet informatique privilégie :
- A. la lecture du code source ligne par ligne.
 - B. le raisonnement : problème traité, choix effectués, difficultés surmontées.
 - C. l'énumération de tous les noms de variables utilisées.
 - D. la récitation de la documentation officielle de Python.

Corrigé

1. Entre 1950 et 1990 : le premier microprocesseur commercial (1971) et l'invention du World Wide Web (1989).
2. $\frac{2}{4} \times 100 = 50,0\%$.
3. La démarche de projet est une compétence essentielle du programme (analyser un besoin, concevoir une solution, travailler en équipe, gérer le temps) qui ne peut s'acquérir qu'en la **pratiquant** réellement, pas seulement en l'étudiant en théorie : imposer un volume horaire minimal garantit que tous les élèves, quel que soit l'établissement, bénéficient effectivement de cette pratique.

Corrigé

1. Le programme lève une ZeroDivisionError, avec le message division by zero.
- 2.

```
1 def diviser(a, b):
2     assert b != 0, "le diviseur ne doit pas être nul"
3     return a / b
```

3. Une précondition explicite documente clairement, dans le code lui-même, l'hypothèse nécessaire au bon fonctionnement de la fonction, avec un **message personnalisé** compréhensible. Laisser l'erreur native se produire fonctionne aussi (le programme s'arrête bien), mais sans cette documentation, un autre développeur (ou soi-même plus tard) doit deviner pourquoi la division a été appelée avec un diviseur nul.

Évaluation A — Projet et méthodes

Durée : 45 minutes. Ordinateur avec interpréteur Python autorisé.

Barème indicatif : Ex. 1 : 10 pts (histoire, démarche de projet) ; Ex. 2 : 10 pts (débogage)

Exercice 1 — Histoire et gestion de projet

(10 points)

1. (4 pts) En utilisant `evenements_entre` du cours sur la chronologie [(1948, ...), (1971, ...), (1989, ...)], lister les événements situés entre 1950 et 1990.
2. (3 pts) Un projet est découpé en 4 jalons, dont 2 sont terminés. Calculer son avancement avec `avancement` du cours.
3. (3 pts) Pourquoi le programme officiel impose-t-il de réserver au moins un quart de l'horaire annuel à la démarche de projet, plutôt que de laisser cette part au libre choix du professeur ?

Exercice 2 — Débogage

(10 points)

```
1 def diviser_bugue(a, b):
2     return a / b
3
4 diviser_bugue(10, 0)
```

1. (3 pts) Quel type d'erreur ce programme provoque-t-il ? Quel est le message exact ?
2. (4 pts) Écrire une version corrigée `diviser(a, b)` qui utilise une précondition (`assert`) pour interdire explicitement une division par zéro, avec un message clair.
3. (3 pts) Pourquoi une précondition explicite est-elle préférable à laisser l'erreur native (`ZeroDivisionError`) se produire sans explication ?

Corrigé

1. Entre 1960 et 1991 : les débuts du réseau Internet (1969) et la première version publique de Python (1991).

2. $\frac{3}{5} \times 100 = 60,0\%$.

3. Avec seulement 2 à 4 élèves et un temps limité, un projet trop ambitieux risque de ne **jamais aboutir** : l'équipe s'épuise sur des fonctionnalités secondaires sans jamais disposer d'une version complète et testée. Un projet modeste mais terminé et fonctionnel est pédagogiquement plus utile (et plus valorisant) qu'un projet ambitieux resté à l'état d'ébauche.

Corrigé

1. Le programme lève une `IndexError`, avec le message `list index out of range`.
- 2.

```
1 def acceder_element(liste, indice):
2     assert 0 ≤ indice < len(liste), "indice hors des bornes de la liste"
3     return liste[indice]
```

3. Une précondition explicite documente, directement dans le code, l'hypothèse nécessaire (un indice valide) avec un message personnalisé et compréhensible. L'erreur native `IndexError` arrête aussi le programme, mais sans expliquer la **cause métier** du problème (ici, un indice incohérent avec la taille de la liste) aussi clairement qu'un message d'assertion rédigé pour ce cas précis.

Évaluation B — Projet et méthodes

Durée : 45 minutes. Ordinateur avec interpréteur Python autorisé.

Barème indicatif : Ex. 1 : 10 pts (histoire, démarche de projet) ; Ex. 2 : 10 pts (débogage)

Exercice 1 — Histoire et gestion de projet

(10 points)

1. (4 pts) En utilisant `evenements_entre` du cours sur la chronologie [(1936, ...), (1969, ...), (1991, ...)], lister les événements situés entre 1960 et 1991.
2. (3 pts) Un projet est découpé en 5 jalons, dont 3 sont terminés. Calculer son avancement avec `avancement` du cours.
3. (3 pts) Le programme officiel précise que les professeurs veillent à ce que les projets restent « d'une ambition raisonnable ». Pourquoi cette recommandation est-elle importante pour une équipe de 2 à 4 élèves ?

Exercice 2 — Débogage

(10 points)

```
1 def acceder_element_bugue(liste, indice):
2     return liste[indice]
3
4 acceder_element_bugue([1, 2, 3], 5)
```

1. (3 pts) Quel type d'erreur ce programme provoque-t-il ? Quel est le message exact ?
2. (4 pts) Écrire une version corrigée `acceder_element(liste, indice)` qui utilise une précondition (`assert`) pour vérifier que l'indice est valide, avec un message clair.
3. (3 pts) Pourquoi une précondition explicite est-elle préférable à laisser l'erreur native (`IndexError`) se produire sans explication ?

Exercice 25 ◆

Le programme suivant plante à l'exécution :

```
1 def message_bienvenue_bugue(nom, age):
2     return "Bonjour " + nom + ", tu as " + age + " ans."
3
4 message_bienvenue_bugue("Lina", 16)
```

1. Quel type d'erreur ce programme provoque-t-il ? Quel est le message exact ?
2. En appliquant la démarche méthodique du cours, identifier précisément quelle sous-expression du `return` pose problème, et pourquoi.
3. Corriger la fonction (au moins deux méthodes sont possibles : conversion explicite, ou f-string).

Exercice 26 ◆

Le programme suivant plante à l'exécution :

```
1 def message_bienvenue_bugue(nom, age):
```

```
2     return "Bonjour " + nom + ", tu as " + age + " ans."
3
4 message_bienvenue_bugue("Lina", 16)
```

1. Quel type d'erreur ce programme provoque-t-il ? Quel est le message exact ?
2. En appliquant la démarche méthodique du cours, identifier précisément quelle sous-expression du return pose problème, et pourquoi.
3. Corriger la fonction (au moins deux méthodes sont possibles : conversion explicite, ou f-string).

Exercice 27 ◆

Le programme suivant plante à l'exécution :

```
1 def message_bienvenue_bugue(nom, age):
2     return "Bonjour " + nom + ", tu as " + age + " ans."
3
4 message_bienvenue_bugue("Lina", 16)
```

1. Quel type d'erreur ce programme provoque-t-il ? Quel est le message exact ?
2. En appliquant la démarche méthodique du cours, identifier précisément quelle sous-expression du return pose problème, et pourquoi.
3. Corriger la fonction (au moins deux méthodes sont possibles : conversion explicite, ou f-string).

Exercice 28 ◆

Le programme suivant plante à l'exécution :

```
1 def message_bienvenue_bugue(nom, age):
2     return "Bonjour " + nom + ", tu as " + age + " ans."
3
4 message_bienvenue_bugue("Lina", 16)
```

1. Quel type d'erreur ce programme provoque-t-il ? Quel est le message exact ?
2. En appliquant la démarche méthodique du cours, identifier précisément quelle sous-expression du return pose problème, et pourquoi.
3. Corriger la fonction (au moins deux méthodes sont possibles : conversion explicite, ou f-string).

Exercice 29 ◆

Le programme suivant plante à l'exécution :

```
1 def message_bienvenue_bugue(nom, age):
2     return "Bonjour " + nom + ", tu as " + age + " ans."
3
4 message_bienvenue_bugue("Lina", 16)
```

1. Quel type d'erreur ce programme provoque-t-il ? Quel est le message exact ?
2. En appliquant la démarche méthodique du cours, identifier précisément quelle sous-expression du return pose problème, et pourquoi.
3. Corriger la fonction (au moins deux méthodes sont possibles : conversion explicite, ou f-string).

Corrigé

1. `evenements == sorted(evenements, key=lambda e: e[0])` vaut `True` : la liste est bien triée par année croissante.
2. Entre 1900 et 1980 : le concept de machine universelle de Turing (1936) et le premier microprocesseur commercial (1971).
3. La Pascaline (1642) est une machine **mécanique** conçue pour une tâche unique (l'addition) : elle ne peut exécuter qu'un seul type de calcul, câblé dans sa mécanique. Le concept de machine universelle de Turing (1936) est une avancée **théorique** majeure : il montre qu'une seule machine peut, en principe, exécuter **n'importe quel** algorithme, à condition de lui fournir le bon programme — c'est le principe fondateur de l'ordinateur moderne, programmable.

Corrigé

1. L'avancement est de 50,0 % (3 jalons terminés sur 6).
2. Le prochain jalon à traiter est "Score et niveaux".
3. Pas nécessairement : le **nombre** de jalons terminés ne dit rien sur leur **charge de travail respective**. « Tests et équilibrage » ou « Score et niveaux » peuvent s'avérer bien plus longs que « Spécification du jeu ». Un pourcentage calculé uniquement sur le **nombre** de jalons peut donc être trompeur : il vaudrait mieux pondérer chaque jalon par une estimation de sa durée, plutôt que de tous les compter à parts égales.

Corrigé

1. Le programme lève une `KeyError`, avec le message 'David'.
2. La ligne fautive est `notes[nom]` dans `afficher_note` : elle tente d'accéder à la clé "David" dans le dictionnaire `notes`, mais cette clé n'existe pas (seuls "Alice", "Bob" et "Chloe" y figurent). La cause est donc un appel de la fonction avec un nom absent du dictionnaire.
- 3.

```
1 def afficher_note_corrigee(notes, nom):
2     if nom in notes:
3         return f"{nom} : {notes[nom]}"
4     else:
5         return "Nom inconnu"
```

Corrigé

1. `a_une_docstring(tri_insertion)` vaut `False` : aucune docstring n'est présente.
- 2.

```
1 def tri_insertion_documente(tableau):
2     """Trie tableau par ordre croissant, par insertion.
3
4     Precondition : tableau est une liste d'elements comparables entre eux.
5     Postcondition : le resultat est une liste triee, de meme longueur.
6     """
7     tableau = tableau[:]
8     for i in range(1, len(tableau)):
9         valeur = tableau[i]
10        j = i - 1
11        while j >= 0 and tableau[j] > valeur:
12            tableau[j + 1] = tableau[j]
13            j -= 1
14        tableau[j + 1] = valeur
15    return tableau
```


`a_une_docstring(tri_insertion_documente)` vaut désormais `True`.

3. Trois points pertinents : (1) le **principe** de l'algorithme (construire une partie triée en insérant chaque élément à sa bonne place, comme trier des cartes en main) ; (2) sa **complexité** ($O(n^2)$ dans le pire cas, mais rapide sur un tableau presque trié) ; (3) l'**invariant de boucle** qui prouve sa correction (le préfixe traité reste toujours trié).

Corrigé

1. L'appel renvoie `-1` (« non trouvé »), alors que "Yanis" est bien présent. La boucle `range(len(participants) - 1)` parcourt les indices 0 à `len(participants) - 2` : avec 4 participants, cela correspond aux indices 0, 1, 2 — l'indice 3 (le dernier, "Yanis") n'est jamais testé.

2.

```
1 def rechercher_participant(participants, pseudo):
2     for i in range(len(participants)):
3         if participants[i] == pseudo:
4             return i
5     return -1
```

3. La démarche de projet du cours invite à rester « d'une ambition raisonnable » : corriger un bug clairement identifié et localisé (une seule ligne à changer) juste avant la présentation reste raisonnable, puisque le correctif est simple et déjà compris. En revanche, si la correction avait impliqué une refonte plus large, mieux aurait valu la documenter comme limite connue plutôt que de risquer d'introduire un nouveau bug dans la précipitation, juste avant de présenter le projet.

Corrigé

1. Le programme lève une `TypeError`, avec le message `can only concatenate str (not "int") to str`.

2. L'expression fautive est `"... tu as " + age + " ans."` : l'opérateur `+` entre chaînes de caractères **concatène** du texte, mais `age` est un entier (`int`), pas une chaîne. Python refuse de mélanger directement un `int` et un `str` avec `+`.

3. Deux corrections possibles :

```
1 def message_bienvenue(nom, age):
2     return "Bonjour " + nom + ", tu as " + str(age) + " ans."
3
4 def message_bienvenue_fstring(nom, age):
5     return f"Bonjour {nom}, tu as {age} ans."
```

La première convertit explicitement `age` en chaîne avec `str(...)` ; la seconde utilise une *f-string*, qui effectue cette conversion automatiquement.

Corrigé

1. `evenements == sorted(evenements, key=lambda e: e[0])` vaut `True` : la liste est bien triée par année croissante.

2. Entre 1900 et 1980 : le concept de machine universelle de Turing (1936) et le premier microprocesseur commercial (1971).

3. La Pascaline (1642) est une machine **mécanique** conçue pour une tâche unique (l'addition) : elle ne peut exécuter qu'un seul type de calcul, câblé dans sa mécanique. Le concept de machine universelle de Turing (1936) est une avancée **théorique** majeure : il montre qu'une seule machine peut, en principe, exécuter **n'importe quel** algorithme, à condition de lui fournir le bon programme — c'est le principe fondateur de l'ordinateur moderne, programmable.

Corrigé

1. L'avancement est de 50,0 % (3 jalons terminés sur 6).

2. Le prochain jalon à traiter est "Score et niveaux".

3. Pas nécessairement : le **nombre** de jalons terminés ne dit rien sur leur **charge de travail respective**. « Tests et équilibrage » ou « Score et niveaux » peuvent s'avérer bien plus longs que « Spécification du jeu ». Un pourcentage calculé uniquement sur le **nombre** de jalons peut donc être trompeur : il vaudrait mieux pondérer chaque jalon par une estimation de sa durée, plutôt que de tous les compter à parts égales.

Corrigé

1. Le programme lève une `KeyError`, avec le message 'David'.

2. La ligne fautive est `notes[nom]` dans `afficher_note` : elle tente d'accéder à la clé "David" dans le dictionnaire `notes`, mais cette clé n'existe pas (seuls "Alice", "Bob" et "Chloe" y figurent). La cause est donc un appel de la fonction avec un nom absent du dictionnaire.

3.

```
1 def afficher_note_corrigee(notes, nom):
2     if nom in notes:
3         return f"{nom} : {notes[nom]}"
4     else:
5         return "Nom inconnu"
```

Corrigé

1. `a_une_docstring(tri_insertion)` vaut `False` : aucune docstring n'est présente.

2.

```
1 def tri_insertion_documente(tableau):
2     """Trie tableau par ordre croissant, par insertion.
3
4     Precondition : tableau est une liste d'elements comparables entre eux.
5     Postcondition : le resultat est une liste triee, de meme longueur.
6     """
7     tableau = tableau[:]
8     for i in range(1, len(tableau)):
9         valeur = tableau[i]
10        j = i - 1
11        while j >= 0 and tableau[j] > valeur:
12            tableau[j + 1] = tableau[j]
13            j -= 1
14        tableau[j + 1] = valeur
15    return tableau
```

`a_une_docstring(tri_insertion_documente)` vaut désormais `True`.

3. Trois points pertinents : (1) le **principe** de l'algorithme (construire une partie triée en insérant chaque élément à sa bonne place, comme trier des cartes en main) ; (2) sa **complexité** ($O(n^2)$ dans le pire cas, mais rapide sur un tableau presque trié) ; (3) l'**invariant de boucle** qui prouve sa correction (le préfixe traité reste toujours trié).

Corrigé

1. L'appel renvoie `-1` (« non trouvé »), alors que "Yanis" est bien présent. La boucle `range(len(participants) - 1)` parcourt les indices `0` à `len(participants) - 2` : avec 4 participants, cela correspond aux indices `0`, `1`, `2` — l'indice `3` (le dernier, "Yanis") n'est jamais testé.

2.

```
1 def rechercher_participant(participants, pseudo):
```

```

2   for i in range(len(participants)):
3       if participants[i] == pseudo:
4           return i
5   return -1

```

3. La démarche de projet du cours invite à rester « d’une ambition raisonnable » : corriger un bug clairement identifié et localisé (une seule ligne à changer) juste avant la présentation reste raisonnable, puisque le correctif est simple et déjà compris. En revanche, si la correction avait impliqué une refonte plus large, mieux aurait valu la documenter comme limite connue plutôt que de risquer d’introduire un nouveau bug dans la précipitation, juste avant de présenter le projet.

Corrigé

1. Le programme lève une `TypeError`, avec le message `can only concatenate str (not "int") to str`.
2. L’expression fautive est `"... tu as " + age + " ans."` : l’opérateur `+` entre chaînes de caractères **concatène** du texte, mais `age` est un entier (`int`), pas une chaîne. Python refuse de mélanger directement un `int` et un `str` avec `+`.
3. Deux corrections possibles :

```

1 def message_bienvenue(nom, age):
2     return "Bonjour " + nom + ", tu as " + str(age) + " ans."
3
4 def message_bienvenue_fstring(nom, age):
5     return f"Bonjour {nom}, tu as {age} ans."

```

La première convertit explicitement `age` en chaîne avec `str(...)` ; la seconde utilise une *f-string*, qui effectue cette conversion automatiquement.

Corrigé

1. `evenements == sorted(evenements, key=lambda e: e[0])` vaut `True` : la liste est bien triée par année croissante.
2. Entre 1900 et 1980 : le concept de machine universelle de Turing (1936) et le premier microprocesseur commercial (1971).
3. La Pascaline (1642) est une machine **mécanique** conçue pour une tâche unique (l’addition) : elle ne peut exécuter qu’un seul type de calcul, câblé dans sa mécanique. Le concept de machine universelle de Turing (1936) est une avancée **théorique** majeure : il montre qu’une seule machine peut, en principe, exécuter **n’importe quel** algorithme, à condition de lui fournir le bon programme — c’est le principe fondateur de l’ordinateur moderne, programmable.

Corrigé

1. L’avancement est de 50,0 % (3 jalons terminés sur 6).
2. Le prochain jalon à traiter est "Score et niveaux".
3. Pas nécessairement : le **nombre** de jalons terminés ne dit rien sur leur **charge de travail respective**. « Tests et équilibrage » ou « Score et niveaux » peuvent s’avérer bien plus longs que « Spécification du jeu ». Un pourcentage calculé uniquement sur le **nombre** de jalons peut donc être trompeur : il vaudrait mieux pondérer chaque jalon par une estimation de sa durée, plutôt que de tous les compter à parts égales.

Corrigé

1. Le programme lève une `KeyError`, avec le message `'David'`.
2. La ligne fautive est `notes[nom]` dans `afficher_note` : elle tente d’accéder à la clé "David" dans le dictionnaire `notes`, mais cette clé n’existe pas (seuls "Alice", "Bob" et "Chloe" y figurent). La cause est donc un appel de la fonction avec un nom absent du dictionnaire.

3.

```

1 def afficher_note_corrigee(notes, nom):
2     if nom in notes:
3         return f"{nom} : {notes[nom]}"
4     else:
5         return "Nom inconnu"

```

Corrigé

1. `a_une_docstring(tri_insertion)` vaut `False` : aucune docstring n'est présente.

2.

```

1 def tri_insertion_documente(tableau):
2     """Trie tableau par ordre croissant, par insertion.
3
4     Precondition : tableau est une liste d'elements comparables entre eux.
5     Postcondition : le resultat est une liste triee, de meme longueur.
6     """
7     tableau = tableau[:]
8     for i in range(1, len(tableau)):
9         valeur = tableau[i]
10        j = i - 1
11        while j ≥ 0 and tableau[j] > valeur:
12            tableau[j + 1] = tableau[j]
13            j -= 1
14        tableau[j + 1] = valeur
15    return tableau

```

`a_une_docstring(tri_insertion_documente)` vaut désormais `True`.

3. Trois points pertinents : (1) le **principe** de l'algorithme (construire une partie triée en insérant chaque élément à sa bonne place, comme trier des cartes en main) ; (2) sa **complexité** ($O(n^2)$) dans le pire cas, mais rapide sur un tableau presque trié ; (3) l'**invariant de boucle** qui prouve sa correction (le préfixe traité reste toujours trié).

Corrigé

1. L'appel renvoie `-1` (« non trouvé »), alors que "Yanis" est bien présent. La boucle `range(len(participants) - 1)` parcourt les indices 0 à `len(participants) - 2` : avec 4 participants, cela correspond aux indices 0, 1, 2 — l'indice 3 (le dernier, "Yanis") n'est jamais testé.

2.

```

1 def rechercher_participant(participants, pseudo):
2     for i in range(len(participants)):
3         if participants[i] == pseudo:
4             return i
5     return -1

```

3. La démarche de projet du cours invite à rester « d'une ambition raisonnable » : corriger un bug clairement identifié et localisé (une seule ligne à changer) juste avant la présentation reste raisonnable, puisque le correctif est simple et déjà compris. En revanche, si la correction avait impliqué une refonte plus large, mieux aurait valu la documenter comme limite connue plutôt que de risquer d'introduire un nouveau bug dans la précipitation, juste avant de présenter le projet.

Corrigé

1. Le programme lève une `TypeError`, avec le message `can only concatenate str (not "int") to str`.

2. L'expression fautive est "... tu as " + age + " ans." : l'opérateur + entre chaînes de caractères **concatène** du texte, mais age est un entier (int), pas une chaîne. Python refuse de mélanger directement un int et un str avec +.

3. Deux corrections possibles :

```
1 def message_bienvenue(nom, age):
2     return "Bonjour " + nom + ", tu as " + str(age) + " ans."
3
4 def message_bienvenue_fstring(nom, age):
5     return f"Bonjour {nom}, tu as {age} ans."
```

La première convertit explicitement age en chaîne avec str(...) ; la seconde utilise une *f-string*, qui effectue cette conversion automatiquement.

Corrigé

1. `evenements == sorted(evenements, key=lambda e: e[0])` vaut True : la liste est bien triée par année croissante.

2. Entre 1900 et 1980 : le concept de machine universelle de Turing (1936) et le premier microprocesseur commercial (1971).

3. La Pascaline (1642) est une machine **mécanique** conçue pour une tâche unique (l'addition) : elle ne peut exécuter qu'un seul type de calcul, câblé dans sa mécanique. Le concept de machine universelle de Turing (1936) est une avancée **théorique** majeure : il montre qu'une seule machine peut, en principe, exécuter **n'importe quel** algorithme, à condition de lui fournir le bon programme — c'est le principe fondateur de l'ordinateur moderne, programmable.

Corrigé

1. L'avancement est de 50,0 % (3 jalons terminés sur 6).

2. Le prochain jalon à traiter est "Score et niveaux".

3. Pas nécessairement : le **nombre** de jalons terminés ne dit rien sur leur **charge de travail respective**. « Tests et équilibrage » ou « Score et niveaux » peuvent s'avérer bien plus longs que « Spécification du jeu ». Un pourcentage calculé uniquement sur le **nombre** de jalons peut donc être trompeur : il vaudrait mieux pondérer chaque jalon par une estimation de sa durée, plutôt que de tous les compter à parts égales.

Corrigé

1. Le programme lève une `KeyError`, avec le message 'David'.

2. La ligne fautive est `notes[nom]` dans `afficher_note` : elle tente d'accéder à la clé "David" dans le dictionnaire `notes`, mais cette clé n'existe pas (seuls "Alice", "Bob" et "Chloe" y figurent). La cause est donc un appel de la fonction avec un nom absent du dictionnaire.

3.

```
1 def afficher_note_corrige(notes, nom):
2     if nom in notes:
3         return f"{nom} : {notes[nom]}"
4     else:
5         return "Nom inconnu"
```

Corrigé

1. `a_une_docstring(tri_insertion)` vaut False : aucune docstring n'est présente.

2.

```

1 def tri_insertion_documente(tableau):
2     """Trie tableau par ordre croissant, par insertion.
3
4     Precondition : tableau est une liste d'elements comparables entre eux.
5     Postcondition : le resultat est une liste triee, de meme longueur.
6     """
7     tableau = tableau[:]
8     for i in range(1, len(tableau)):
9         valeur = tableau[i]
10        j = i - 1
11        while j ≥ 0 and tableau[j] > valeur:
12            tableau[j + 1] = tableau[j]
13            j -= 1
14        tableau[j + 1] = valeur
15    return tableau

```

`a_une_docstring(tri_insertion_documente)` vaut désormais `True`.

3. Trois points pertinents : (1) le **principe** de l'algorithme (construire une partie triée en insérant chaque élément à sa bonne place, comme trier des cartes en main) ; (2) sa **complexité** ($O(n^2)$) dans le pire cas, mais rapide sur un tableau presque trié) ; (3) l'**invariant de boucle** qui prouve sa correction (le préfixe traité reste toujours trié).

Corrigé

1. L'appel renvoie `-1` (« non trouvé »), alors que "Yanis" est bien présent. La boucle `range(len(participants) - 1)` parcourt les indices 0 à `len(participants) - 2` : avec 4 participants, cela correspond aux indices 0, 1, 2 — l'indice 3 (le dernier, "Yanis") n'est jamais testé.

2.

```

1 def rechercher_participant(participants, pseudo):
2     for i in range(len(participants)):
3         if participants[i] == pseudo:
4             return i
5     return -1

```

3. La démarche de projet du cours invite à rester « d'une ambition raisonnable » : corriger un bug clairement identifié et localisé (une seule ligne à changer) juste avant la présentation reste raisonnable, puisque le correctif est simple et déjà compris. En revanche, si la correction avait impliqué une refonte plus large, mieux aurait valu la documenter comme limite connue plutôt que de risquer d'introduire un nouveau bug dans la précipitation, juste avant de présenter le projet.

Corrigé

1. Le programme lève une `TypeError`, avec le message `can only concatenate str (not "int") to str`.

2. L'expression fautive est `"... tu as " + age + " ans."` : l'opérateur `+` entre chaînes de caractères **concatène** du texte, mais `age` est un entier (`int`), pas une chaîne. Python refuse de mélanger directement un `int` et un `str` avec `+`.

3. Deux corrections possibles :

```

1 def message_bienvenue(nom, age):
2     return "Bonjour " + nom + ", tu as " + str(age) + " ans."
3
4 def message_bienvenue_fstring(nom, age):
5     return f"Bonjour {nom}, tu as {age} ans."

```

La première convertit explicitement `age` en chaîne avec `str(...)` ; la seconde utilise une *f-string*, qui effectue cette conversion automatiquement.

Corrigé

1. Le programme lève une `TypeError`, avec le message `can only concatenate str (not "int") to str`.
2. L'expression fautive est `"... tu as " + age + " ans."` : l'opérateur `+` entre chaînes de caractères **concatène** du texte, mais `age` est un entier (`int`), pas une chaîne. Python refuse de mélanger directement un `int` et un `str` avec `+`.
3. Deux corrections possibles :

```
1 def message_bienvenue(nom, age):  
2     return "Bonjour " + nom + ", tu as " + str(age) + " ans."  
3  
4 def message_bienvenue_fstring(nom, age):  
5     return f"Bonjour {nom}, tu as {age} ans."
```

La première convertit explicitement `age` en chaîne avec `str(...)` ; la seconde utilise une *f-string*, qui effectue cette conversion automatiquement.

Apprendre, s'entraîner, se tester, progresser : la collection Nexus

Réussite accompagne chaque élève jusqu'à la maîtrise.

- 📖 Un cours structuré en strates, avec la rubrique « À la machine » : chaque notion se reproduit au clavier.
- » Du code Python vérifié par exécution : aucune sortie de programme imprimée sans avoir tourné.
- ⚙ Des méthodes pas à pas avec exemples résolus commentés et pièges à éviter.
- ♦♦ Trois parcours d'exercices gradués et chronométrés, avec coups de pouce.
- ↺ QCM diagnostiques, auto-évaluation par compétences et sujets aux formats officiels.
- 🩹 Des fiches de remédiation ciblées, reliées aux erreurs réellement diagnostiquées.